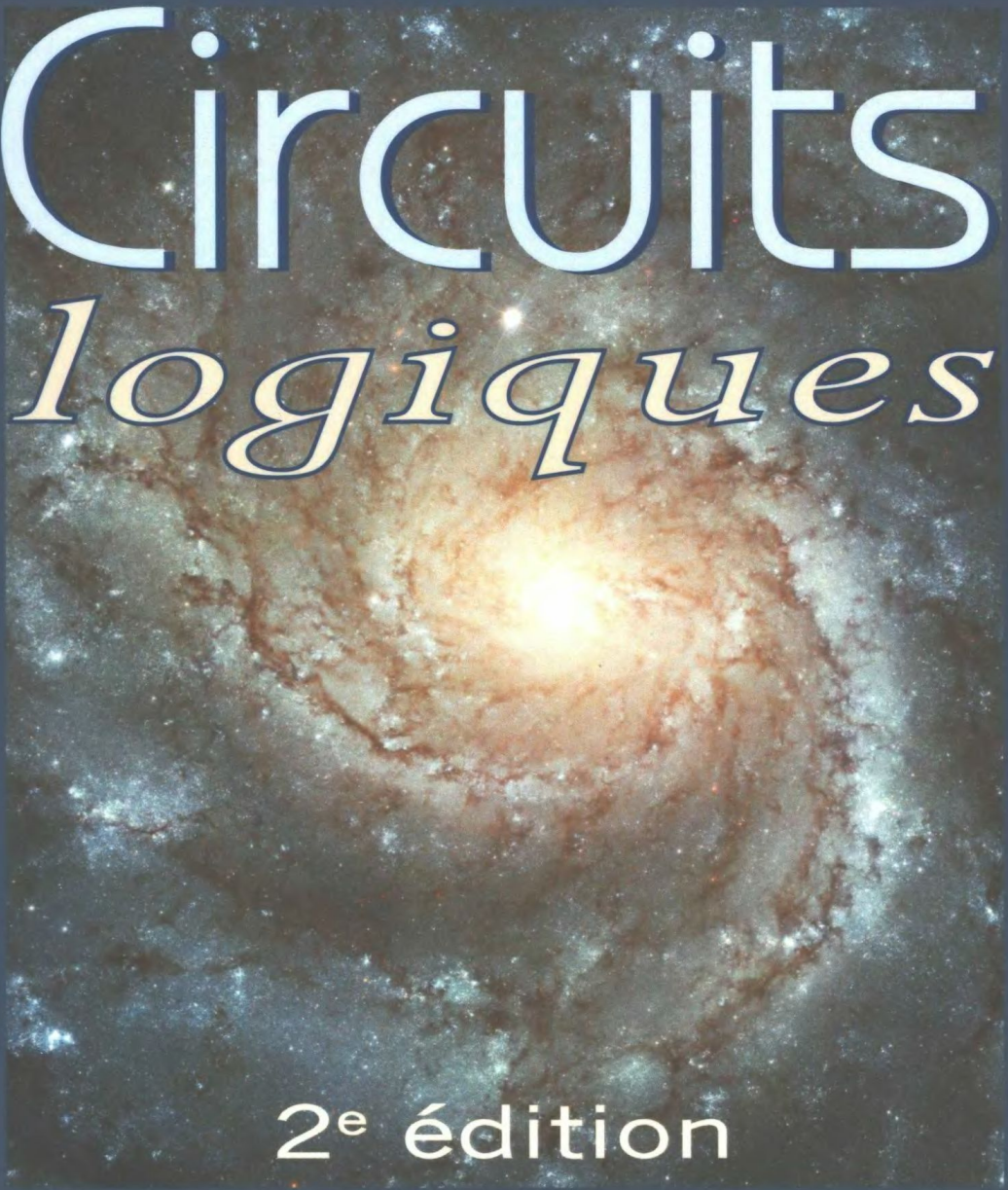
Xavier Maldague

Circuits logiques



2e édition

ID

Xavier Maldague

Circuits logiques

2e édition

Catalogage avant publication de Bibliothèque et Archives nationales du Québec et Bibliothèque et Archives Canada

Maldague, X. (Xavier), 1959-

Circuits logiques

2e édition.

Comprend des références bibliographiques et un index.

ISBN 978-2-923565-90-3

1. Circuits logiques. 2. Arithmétique interne des ordinateurs. 3. Ordinateurs - Architecture. 4. Électronique numérique. 5. Circuits logiques - Problèmes et exercices. I. Titre.

TK7868.L6M34 2015 621.39'5 C2015-941298-6

Copyright © 2015 Loze-Dion éditeur inc.
Courriel: commandes@lozedion.com
Web: www.lozedion.com

Tous droits réservés. On ne peut reproduire, enregistrer, ni diffuser aucune partie du présent ouvrage sous quelque forme ou par quelque procédé que ce soit, sans une autorisation écrite de l'éditeur.

Dépôt légal quatrième trimestre 2015
Bibliothèque et Archives nationales du Québec, 2015
Bibliothèque et Archives Canada, 2015

Imprimé au Canada

Société de développement des entreprises culturelles Québec

Gouvernement du Québec - Programme de crédit d'impôt pour l'édition de livres - Gestion SODEC.

Financé par le gouvernement du Canada

Funded by the Government of Canada

Canada

Avant-propos

Pourquoi un autre livre sur les circuits logiques ? Ce fut la question que je posai à M. Patrick Loze, le directeur de Loze-Dion éditeur, lorsqu'il me proposa d'écrire cet ouvrage. En effet, enseignant ce cours depuis plus de dix ans, ma bibliothèque était encombrée de livre intitulés « *Digital Circuits* » (ou « Circuits logiques »). Là était justement le point essentiel : plusieurs ouvrages anglophones sont disponibles sur le marché, mais on en trouve peu du côté francophone, d'où ce projet de livre issu à l'origine de mes notes de cours[1]. Le projet semblait simple au début, mais, plus le temps passa, plus il prit de l'ampleur pour finalement aboutir au livre actuel.

Ce livre abordant tous les aspects de l'arithmétique binaire de base aux circuits séquentiels complexes (avec un soupçon de microprogrammation), il est destiné à la clientèle universitaire de premier cycle de génie électrique, de génie informatique, de génie logiciel ou de génie physique qui a dans son curriculum un cours de base sur les circuits logiques : des circuits combinatoires aux circuits séquentiels. Il peut aussi servir à l'enseignement collégial, sans doute en excluant la partie séquentielle et en y ajoutant des notions d'électronique non couvertes ici. D'un format compact, il sera aisément couvert en un trimestre.

On débute au chapitre 1 par la présentation du système binaire et des portes logiques simples[2]. Avec le chapitre 2, on aborde les premiers circuits combinatoires, la notion de table de vérité, de même que la simplification par l'algèbre de Boole, qui permet de simplifier les circuits obtenus (si seulement ça ne devenait pas vite compliqué !). C'est ainsi qu'au chapitre 3, la notion de simplification graphique par tables de Karnaugh est présentée. Au terme de ce chapitre, on pourrait réaliser n'importe quel circuit logique combinatoire (jusqu'à cinq variables). Le chapitre 4 représente l'occasion d'étudier quelques familles de circuits combinatoires particulières qui permettent de faciliter la réalisation de circuits logiques à partir d'une table de vérité : les multiplexeurs, les démultiplexeurs, les mémoires. On y présente aussi brièvement les FPGA utiles pour les expériences pratiques (voir plus bas).

Avec le chapitre 5, on quitte le domaine des circuits exclusivement combinatoires pour aborder celui des circuits séquentiels synchrones. Ce chapitre concerne précisément la présentation des éléments de base des circuits séquentiels, soit les bascules de type R-S, D, J-K et T qui serviront par la suite. Au chapitre 6, les machines de Moore, donc les circuits séquentiels sans entrée, sont étudiées. On y présente les méthodes d'analyse et de design de ces circuits. Le chapitre 7 est certainement le plus intéressant de ce livre,

---

1 Rendons à César ce qui lui revient ! À l'origine, le livre *Digital Design from Zero to One* de J. D. Daniels, Wiley, 1996, a servi d'inspiration à ces notes de cours, qui se sont par la suite enrichies pendant la décennie de leur utilisation.

2 Aucun préalable n'est nécessaire à l'étude de ce livre sauf les notions mathématiques acquises au secondaire.

puisque son sujet concerne les machines de Mealy, donc les machines à nombre finis d'états. Ce sont des circuits avec entrées externes (et sorties) qui couvrent en quelque sorte tout ce qui a été vu dans les autres chapitres. En fin de chapitre, on aborde très brièvement la question de la microprogrammation qui est l'étape suivante vers la réalisation de circuits logiques plus complexes : la conception d'un séquenceur universel programmable. Une alternative serait aussi l'utilisation d'un micro-contrôleur. Nous en reparlerons au chapitre 7.

Des problèmes de fin de chapitres sont inclus, certains avec solutions. Comme leur nombre est raisonnable, il est conseillé aux étudiantes[3] de tous les essayer. Notons qu'un site Web existe où seront déposés les errata éventuels, de même qu'un éventail des solutions, des problèmes résolus, etc. Ce site est situé à l'adresse :

http://c-logiques.gel.ulaval.ca

Pour apprendre à nager, il ne suffit pas de lire un livre sur la natation, il faut aussi sauter dans la piscine. Il en va de même en circuits logiques. C'est ainsi qu'il est conseillé de compléter l'enseignement théorique de ce livre par un ensemble d'expériences pratiques de laboratoire (le site Web précité offre d'ailleurs une telle série d'expériences pratiques). Celles-ci peuvent être réalisées de plusieurs façons. On peut faire monter des circuits discrets sur des plaquettes de montage (voir la figure 1.38, page 37). On peut aussi simuler les circuits réalisés dans des logiciels de conception tel l'environnement Xilinx Foundation, le Xilinx WebPack, ou Logisim disponibles gratuitement sur le Web (voir la figure 2.42, page 77). Les circuits ainsi réalisés pourront être chargés dans la « circuiterie véritable » (circuits FPGA, CPLD ou simplement TTL) de sorte que « l'activité électrique » pourra être testée. Plusieurs produits commerciaux existent pour de telles fins. Mentionnons ainsi les produits des compagnies Digilent et XESS. Nous mentionnons ces produits qui ont été éprouvés par plusieurs générations d'étudiantes dans les cours de génie électrique et de génie informatique de mon département. Bien sûr, d'autres excellents produits existent sur le marché.

Une combinaison d'exposés théoriques magistraux et d'expériences pratiques est à notre avis la combinaison pédagogique idéale. Ce livre a été pensé en conséquence. C'est ainsi que chaque partie y est très détaillée. L'ajout de problèmes et de démonstrations (par exemple de simulations) réalisés en classe complétera judicieusement le contenu de ce volume. On favorise d'ailleurs cette approche *d'apprentissage par l'exemple* dans ce livre où les notions sont très souvent présentées à même les exemples résolus.

Plusieurs personnes ont contribué à faire de l'idée de départ l'ouvrage actuel. C'est ainsi que je tiens à remercier le personnel de secrétariat de mon institution qui a réalisé la version originale de mes notes de cours. En particulier, MM. Alain Laverdière et Richard Martineau pour la conception graphique initiale du document ; Mmes Nathalie Beaulieu et Ginette Ratté pour la mise en page initiale du document. Je remercie aussi MM. Nathaniel Zoso et Saeed Sojasi, étudiants du département, pour leur habileté avec VISIO. Plusieurs générations d'étudiantes et d'assistantes de cours ont servi d'inspiration

[3] Par souci de discrimination positive, la forme féminine est employée dans ce livre. On espère ainsi accomplir un geste qui pourra contribuer à intéresser les femmes à nos domaines, qui restent malheureusement trop tristement masculins. Le génie électrique et le génie informatique sont en effet parmi les disciplines (sinon les pires) souffrant d'une sous-représentation féminine chronique.

continuelle pour faire progresser mon enseignement. Le contact étudiant est un des aspects les plus enrichissants de la carrière professorale. Un grand merci aussi à toutes celles et à tous ceux qui ont souffert de mon manque de disponibilité pendant toute la gestation de ce livre : en particulier mon épouse, Marthe, et nos trois enfants.

Au terme de cette grande et magnifique aventure (j'en ai appris des choses !), il y a donc ce livre, un livre sans prétention, mais qui, je l'espère sincèrement, aidera les étudiantes à naviguer et à arriver à bon port sur la mer de la circuiterie logique : bonne traversée à toutes et à tous !

Sainte-Foy (Québec), mai 2015

À Marthe,

A Lucas, Marti et Flavie,

A mes parents

Il n'est pas besoin d'espérer pour entreprendre
Ni de réussir pour persévérer.
Marcel Pagnol, *Le temps des amours.*

Ah ! frappe-toi le cœur, c'est là qu'est le génie.
Alfred de Musset, *Premières poésies.*

Table des matières

AVANT-PROPOS iii

1 CODES BINAIRES ET PORTES SIMPLES 1

Cent cinquante ans d'histoire 1

1.1 Introduction 5

1.2 Du continu au discret 6

1.3 Pourquoi numériser ? 7

1.4 Avantages de deux états 9

1.5 Représentation binaire 10

1.6 Méthodes de conversion binaire en décimal 16

1.7 Calcul des fractions 17

1.8 Conversion de base décimale en base binaire 17

1.9 Conversion de fractions 19

1.10 Addition de nombres binaires non négatifs 20

1.11 Soustraction par la méthode du complément 2 22

1.12 Addition de fractions binaires 27

1.13 Codes et encodage 28

1.14 Portes logiques 31

1.15 Exemple de design : addition modulaire 35

1.16 Analyse et design des circuits logiques 38

1.17 Implantation matérielle des circuits logiques 40

Problèmes résolus 46

Exercices 48

2 TABLE DE VÉRITÉ ET ALGÈBRE DE BOOLE 51

George Boole 51

2.1 Introduction 52

2.2 Tables de vérité (tdv) 54

2.3 Primitives toutes faites et approche par mémorisation de la table de vérité 56

2.4 Première méthode de résolution d'une table de vérité : résolution canonique par la méthode de la Somme de Produits (SdP) 58

2.5 Algèbre booléen 64

2.6 Propriétés du OU-exclusif 69
2.7 Utilisation de portes NON-ET dans les designs SdP (Somme de Produits) 72
2.8 Impliquants 75
2.9 Recherche des 0 et design avec des portes NON-OU (méthode de synthèse par Produit de Sommes (PdS)) 75
2.10 Lois de De Morgan appliquées aux circuits PdS 78
2.11 Optimisation des designs numériques 80
Problèmes résolus 82
Exercices 85

3 MÉTHODE DES TABLES DE KARNAUGH POUR MINIMISER LES EXPRESSIONS BOOLÉENNES 87

Maurice Karnaugh à l'origine de la méthode de simplification graphique 87
3.1 Introduction 88
3.2 Diagrammes de Venn et tables de Karnaugh 89
3.3 Simplification graphique des tables de Karnaugh 96
3.4 Solution en rassemblant les 0 105
3.5 Les choix « sans importance » dans les tables de Karnaugh 107
3.6 Tables de Karnaugh pour plus de quatre entrées 109
3.7 Aléas 112
Problèmes résolus 116
Exercices 121

4 CIRCUITS PROGRAMMABLES POUR LES DESIGNS COMBINATOIRES 125

August De Morgan : des lois fondamentales pour la simplification des circuits logiques 125
4.1 Introduction 126
4.2 Synthèse avec des circuits à intégration moyenne 128
4.3 Synthèse avec multiplexeurs 129
4.4 Multiplexeurs reliés en cascades 135
4.5 Multiplexeurs et théorème d'expansion de Shannon 136
4.6 Démultiplexeur (ou décodeur) 142
4.7 Synthèse par démultiplexeurs : utilisation des décodeurs pour la synthèse des tables de vérité 149
4.8 Mise en cascade de démultiplexeurs 153
4.9 Circuits de mémoire 155
4.10 Circuits logiques programmables (PLD) 164
4.11 PAL : *Programmed Array Logic* 164
4.12 PLA : *Programmable Logic Array* 177

4.13 FPGA 178
4.14 Logiciel de programmation 180
Problèmes résolus 182
Exercices 187

5 BASCULES ET FLIP-FLOPS 191

Le premier microprocesseur : le 4004 191
5.1 Introduction 193
5.2 Retour de l'information et ses conséquences (*feed-back*) 194
5.3 Bascules R-S 196
5.4 État d'une bascule, état d'un circuit séquentiel 201
5.5 Bascule R-S améliorée : le *latch* D 202
5.6 Bascules à déclenchement sur front « maître-esclave » : flip-flop D 205
5.7 Formation de registres à partir de flip-flops D 209
5.8 Bascules à transition maître-esclave : flip-flop toggle (flip-flop T) 213
5.9 Flip-flop R-S maître-esclave et les contaminations aux « 1 » et aux « 0 » (« 1 *catching* » ou « 0 *catching* ») 216
5.10 Autre configuration pour le flip-flop D (section facultative) 217
5.11 Quelques paramètres importants des flip-flops synchrones 219
5.12 Flip-flop J-K 222
5.13 Résumé : flip-flops D, T, J-K 226
Problèmes résolus 229
Exercices 232

6 SÉQUENCEURS SYNCHRONES 235

Quel défi ! 235
6.1 Introduction 237
6.2 États et sortie 238
6.3 Circuits synchrones : considérations à propos de l'horloge 240
6.4 Étapes d'analyse et de design pour les circuits synchrones sans entrées 244
6.5 Analyse de circuits synchrones sans entrées 246
6.6 Design de séquenceurs synchrones sans entrées 250
6.7 Types de compteurs 261
6.8 Registres à décalage pour le design de séquenceurs synchrones 268
6.9 Unités cachées 281
6.10 Séquence arbitraire obtenue avec un compteur et une mémoire 284
Problèmes résolus 288
Exercices 297

7 MACHINE D'ÉTATS SYNCHRONE 301

L'avenir des circuits logiques passe par les « nanotubes de carbones » 301

7.1 Introduction 303

7.2 Machine de Mealy 304

7.3 Analyse et design 305

7.4 Analyse des circuits synchrones avec entrées externes par la méthode des variables conditionnelles 308

7.5 Design des circuits synchrones avec entrées externes par la méthode des variables conditionnelles 319

7.6 Synthèse avec MSI et LSI 337

7.7 Analyse de machines à nombre finis d'états avec compteurs synchrones 345

7.8 Conception de machines à nombre fini d'états avec compteurs synchrones 351

7.9 Microprogrammation 366

Problèmes résolus 372

Exercices 393

ANNEXE 401

LEXIQUE 407

BIBLIOGRAPHIE 429

RÉFÉRENCES 431

INDEX 433

1

Codes binaires et portes simples

CENT CINQUANTE ANS D'HISTOIRE

L'histoire des ordinateurs est fascinante. Même si ce domaine nous semble récent, son histoire est déjà longue et on retrouve même des musées sur le sujet, notamment à Boston[1] et à Mountain View (Californie)[2], où se trouve le plus complet du genre dans le monde (son site virtuel est aussi bien garni et mérite la visite tout comme le musée virtuel de l'IEEE[3]).

Il faut remonter à 1837, lorsque le visionnaire Charles Babbage conçoit un engin de calcul, la machine analytique. À l'époque, l'électronique était évidemment inexistante, même Ohm n'avait pas encore énoncé sa fameuse loi (1862). La conception de cette machine était donc entièrement mécanique ; un travail d'orfèvre puisque chaque engrenage devait être patiemment réalisé à la main. Charles Babbage y engloutit tous ses avoirs, et, même si le gouvernement britannique y investit aussi beaucoup d'argent, jamais la machine ne fut complétée ni fonctionnelle. L'objectif de cette réalisation était le calcul des phases de la Lune nécessaire à la détermination de la longitude par les navires en mer. Finalement une méthode par mesure de temps à l'aide de chronographes de précision a réglé la question avant que la machine ne fût réalisée! La réalisation de chronographes de précision est une saga en soi et est d'ailleurs considérée comme la réalisation technique majeure du XVIIIe siècle![4]

Par contre, on s'en souvient toujours aujourd'hui puisque cette machine comprenait tous les attributs d'un ordinateur moderne : entrées, sortie, mémoire, unité centrale de traitement et, surtout, possibilité de la programmer. D'ailleurs, Ada Byron Augusta Lovelace (1815-1852), mathématicienne, est considérée comme ayant écrit le premier « programme informatique ». En effet, dans une correspondance avec Babbage, elle

---

1. Museum of Science (Boston) : http://www.mos.org/, voir le volet virtuel à http://www.mos.org/exhibits/ComputingRevolution/
2. Museum of Computer History : http://www.computerhistory.org/
3. http://www.ieee-virtual-museum.org/
4. Le Musée de Greenwich (Royaume-Uni) y a consacré une exposition majeure en 2015 : Ships, Clocks & Stars: The Quest for Longitude!

décrivit un plan pour calculer les nombres de Bernouilli. En 1979, un langage développé par la Défense américaine fut nommé ADA en son honneur.

Première génération

Les guerres sont des événements terribles qui donnent parfois lieu à des percées technologiques d'importance. Que l'on songe par exemple aux premières fusées capables de mettre en orbite des satellites artificiels issues des V-2 allemands ou encore aux ordinateurs. En effet, pour ses canons, l'armée américaine était à la recherche de méthodes rapides pour le calcul de tables de tirs. L'élaboration de ces tables exigeait des calculs complexes étant donné le nombre de variables impliquées : poids des obus, distance, température, direction et vitesse du vent, humidité relative, etc. C'est ainsi que Howard Aiken de l'université Harvard fut financé pour la construction du premier ordinateur, le MARK I (1944). L'architecture du Mark I se caractérisait par une mémoire séparée pour les données et les programmes. Cette caractéristique lui valut l'appellation *d'architecture de Harvard*. La plupart des microcontrôleurs actuels utilisent cette architecture : pensons au cas du microcontrôleur d'un four à micro-ondes dont le programme est immuable alors que les données changent (temps de cuisson, puissance). Quand avez-vous fait la mise à jour du programme de votre micro-onde ?

Le Mark I était fabriqué avec des relais. Il fut suivi en 1946 par l'ENIAC (*Electronic Numerical Integrator And Computer*) qui est le premier ordinateur électronique, puisque construit avec 18 000 tubes à vide (il pesait 30 tonnes). Le nombre énorme de tubes à vide engendrait des pannes fréquentes et donc des périodes restreintes d'utilisation qui permirent néanmoins de procéder au calcul des tables de tirs. Il fut suivi en 1952 par l'EDVAC (*Electronic Discrete Variable Automatic Computer*), dont l'architecture est maintenant connue sous le nom *d'architecture de Von Neumann*, de son concepteur, John Von Neumann. À la différence de l'architecture de Harvard, celle de Von Neumann emploie une même mémoire pour les données et les programmes, ce qui fait que les programmes peuvent s'auto-modifier. La plupart des microprocesseurs sont construits avec l'*architecture de Von* Neumann (pensons par exemple au microprocesseur d'un PC ou d'une tablette qui exécute du code sans cesse différent, tantôt un traitement de texte, tantôt un logiciel de calcul, etc.). Le développement de l'EDVAC fut aussi l'occasion de mettre au point des langages informatiques plus évolués.

La première génération d'ordinateurs fait référence aux MARK I, ENIAC, EDVAC et autres machines similaires de cette époque, des machines fonctionnant au mieux avec des tubes à vide.

Deuxième génération

La deuxième génération fut permise grâce à la mise au point du transistor par John Bardeen et Walter H. Brattain de chez Bell Labs en 1947 (invention annoncée en 1948). Ce premier *transistor-contact* était constitué d'une barre de germanium sur laquelle des fils faisaient contact. Ce n'était pas seulement un interrupteur, mais aussi un

amplificateur. Par rapport aux tubes à vide, consommait très peu d'énergie et dissipait donc peu de chaleur tout en étant petit (pas plus grand que le bout d'un doigt). Peu après (1951), William Shockley, aussi de Bell Labs, transforma le fragile *transistor-contact* en un composant pratique et robuste en inventant le *transistor à jonction* réalisé comme un empilement de couches de germanium. Ce transistor de germanium fut un point décisif qui valut d'ailleurs le prix Nobel à ces inventeurs en 1956[5]. Par contre, il n'était pas très fiable ; un autre élément chimique était nécessaire. C'est ainsi que l'introduction du premier transistor au silicium[6] par Texas instruments en 1954 révolutionna le monde. À la fin des années cinquante, le petit transistor fiable et robuste avait remplacé le tube à vide fragile et chaud dans à peu près toutes les applications y compris les ordinateurs[7]. Le produit phare caractérisant cette époque est sans contredit la radio de poche à transistors. On annonçait même le nombre de transistors sur le boîtier, par exemple, une radio portable AM «Admiral à 7 transistors», avec écouteurs, à pile 9 Volt était le produit en vogue à la fin des années 60 auprès des jeunes!

Logique câblée

Les ordinateurs de cette génération sont caractérisés par une plus grande puissance de calcul et de la logique câblée réalisée à partir de composants discrets (transistors, résistances, condensateurs, etc.). La *logique câblée* est celle que nous étudierons dans ce livre. Elle s'oppose à la logique programmable. En logique câblée, le moindre changement exige un recâblage où à tout le moins des modifications au circuit de départ. En logique programmable, un changement est réalisé simplement par modification du programme de contrôle tout en gardant le même circuit de départ. La logique programmable est donc plus souple, par contre le « décorticage » du programme rend généralement l'exécution d'une tâche spécifique plus lente qu'avec la logique câblée.

Troisième génération et la Loi de Moore

Si un transistor c'est bien, deux c'est mieux ! La troisième génération d'ordinateurs est ainsi caractérisée par l'utilisation des premiers circuits électroniques construits dans les années soixante. En effet, le développement de la technologie des circuits électroniques permit l'incorporation de plusieurs fonctions logiques dans une même puce. C'est ainsi que les circuits SSI (*Small Scale Integration*) et MSI (*Medium Scale Integration*) furent employés pour la réalisation des ordinateurs de troisième génération. Cette époque vit aussi la proclamation de la fameuse loi de Moore. En 1961, Gordon E. Moore était alors ingénieur chez Fairchild, un fabricant bien connu de circuits intégrés. Cette année-là, Fairchild mettait sur le marché des circuits intégrés SSI comprenant $4 = 2^2$ transistors. Moore savait que les laboratoires de la compagnie planchaient sur la réalisation de circuits intégrés à $8 = 2^3$ transistors. Il savait aussi qu'en 1959, c'est seulement $1 = 2^0$

---

5 Corécipiendaires du prix Nobel de la physique pour leurs recherches sur les semi-conducteurs et leur découverte de l'effet *transistor* : William Shockley, John Bardeen, Walter H. Brattain.

6 Un de ceux-ci est visible au musée du Texas à Austin (la capitale du Texas).

7 « Principes de fonctionnement de l'Ordinateur électronique IBM, type 650 », Roger Maldague, *Bulletin de la Société belge de Photogrammétrie*, n° 51, p. 9-22, mars 1958.

transistor que la technologie savait encapsuler. C'est ainsi qu'il proclama sa fameuse loi qui stipule *que le nombre de transistors embarqués sur un circuit intégré double chaque année.* Aujourd'hui, cette loi est toujours vraie (avec un délai qui tourne plutôt autour des 18 mois). Étant donné l'effet exponentiel de la puissance de 2, le fait que cette loi n'ait jamais été prise en défaut est particulièrement extraordinaire. Les mauvaises langues disent d'ailleurs que c'est l'existence même de cette loi qui a fait que les grands fabricants électroniques mondiaux se sont toujours « arrangés » pour qu'elle reste valide (en construisant des usines plus performantes, etc.). Peu après (1968), Moore fonda Intel avec Robert Noyce.

Quatrième génération

La quatrième génération d'ordinateurs date des années 1970 lorsque les circuits LSI (*Large Scale Integration*) furent disponibles. Cette époque est d'ailleurs celle qui vit la commercialisation du premier microprocesseur, le 4004 de Intel (mise en marché : 1971, 2300 transistors, surface de silicium de 1.2 mm^2). Une discussion sur le 4004 se trouve d'ailleurs à la page 182[8].

Cinquième génération

De nos jours, nous en sommes à la cinquième génération d'ordinateurs avec l'utilisation des circuits VLSI (de l'anglais *Very Large Scale Integration*, voir plus bas dans ce chapitre). Suivant la Loi de Moore citée plus haut, l'intégration augmente toujours. Prenons par exemple le cas du microprocesseur équipant les ordinateurs personnels. Au début, ceux-ci étaient équipés d'un Intel 8088 (mise en marché : 1980, 29000 transistors, surface de silicium de 33 mm^2, fréquence maximum de 5 MHz). De nos jours, on parle du micro coprocesseur Xeon Phi, également appelé Knights Landing, qui est le plus puissant produit par Intel (2012). Il peut fournir plus de 3 Téra flops et contient près 5 milliards de transistors. Il est réalisé en technologie 22 nm et 61 de ces coprocesseurs peuvent être présents sur une seule matrice de silicium (cadencés à 1.238 GHz).

Quand la Loi de Moore sera-t-elle prise en défaut ? Le nombre d'atomes requis pour réaliser un transistor diminue sans cesse ; la limite physique est d'un atome par transistor. Les experts prédisent que, d'ici une dizaine d'années, la technologie actuelle de fabrication de circuits intégrés sera à bout de souffle. En effet, on note déjà que les croissances passées (nombre de transistors embarqués, fréquence de fonctionnement) s'estompent peu à peu au profit d'autres stratégies comme le «multi-cœur» [9]. Nous en reparlerons au chapitre 7.

---

[8] Voir aussi : http://www.antiquetech.com/

[9] Voir à ce sujet l'excellente discussion présentée dans le numéro d'avril 2015 de l'IEEE Spectrum à l'occasion du 50e anniversaire de la Loi de Moore.

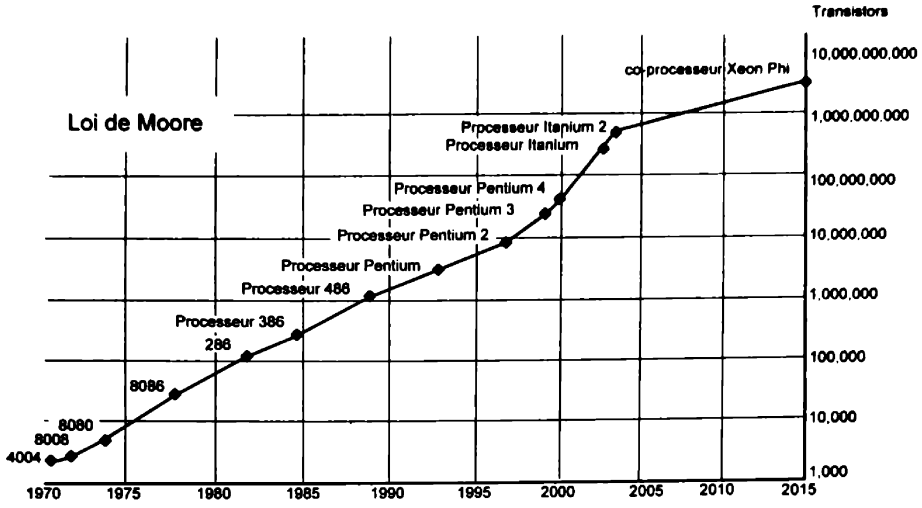


Figure 1.1 Loi de Moore illustrant l'évolution du nombre de transistors embarqués dans les circuits intégrés

(source : http://www.intel.com/intel/intelis/museum/archives/history_docs/index.htm?iid=intelmuseum+home_bfhistdocs&
Voir aussi : http://www.ieee-virtual-museum.org/collection/event.php?taid=&id=3457011&lid=1)

1.1 INTRODUCTION

Pour interagir sur le monde, il faut pouvoir se représenter celui-ci sous une forme propice. Par exemple, les mots que nous exprimons peuvent être transcrits sur papier grâce à l'écriture. Dans cet exemple, les mots sont représentés par des symboles (les lettres de l'alphabet), et toutes ces lettres associées ensemble suivant des règles précises (de grammaire) constituent la langue que nous parlons. Bien sûr, les règles dépendent de la langue (le français, l'allemand, le mandarin, etc.), et quoiqu'il n'y ait que 26 symboles (en français), on arrive vite à un ensemble complexe qui nécessite des années d'apprentissage pour en connaître toutes les finesses. Les notes de musique sont un autre ensemble de symboles bien connu qui représentent les sons.

Le domaine des circuits logiques fait bien sûr référence au domaine du « numérique », et, que l'on s'intéresse au microprocesseur de dernière génération ou à l'humble microcontrôleur du four à micro-ondes, le plus banal, ce vaste domaine représente le monde avec un système binaire très simple à deux états, le « 1 » et le « 0 ». En circuiterie numérique, tout (lettres de l'alphabet, notes de musique, points ou *pixels* dans une image[10]) sera représenté par ces deux symboles.

[10] Un point-image est souvent appelé *pixel* pour *picture element*.

Notre représentation des nombres est décimale et basée sur nos 10 doigts, si utiles pour compter. En circuiterie numérique les nombres seront représentés par les deux mêmes symboles (0, 1), et on parlera donc d'une arithmétique binaire, donc d'une arithmétique en base 2. Dans ce chapitre, nous nous attarderons d'ailleurs sur cette arithmétique un peu spéciale, un peu rébarbative de prime abord, mais si répandue dans les machines qui nous entourent.

Avant d'aborder l'étude de cette arithmétique binaire, il convient de s'interroger sur la façon de numériser.

1.2 DU CONTINU AU DISCRET

En termes mathématiques, le monde qui nous entoure est un monde continu, c'est-à-dire que les grandeurs physiques qui le décrivent, comme la température, la vitesse du vent, l'intensité lumineuse, peuvent avoir une infinité de valeurs. Prenons la température, qui est un exemple facile, puisqu'un simple thermomètre attaché à une fenêtre permet de la mesurer. La hauteur de la colonne de mercure dans le thermomètre pourra s'abaisser (ou s'élever) à n'importe quelle valeur (entre les graduations maximales et minimales de ce dernier), par exemple : +20 °C, mais aussi +20.123 °C ou +20.12345 °C. Bien sûr, ces dernières valeurs nécessitent une instrumentation spéciale et ne peuvent pas être lues directement sur les graduations.

Prenons maintenant un bloc d'alimentation de tension variable de 0 à 10 V, figure 1.2. Un bouton de réglage permet de modifier la tension de sortie dans la plage disponible. Cherchons maintenant à numériser ce signal de sortie (la tension aux bornes du bloc d'alimentation). Dans un premier temps, on pourrait simplement diviser la plage en deux : ce qui est supérieur à 5 V, serait associé à la valeur binaire 1, et ce qui serait inférieur à 5 V à la valeur logique 0.

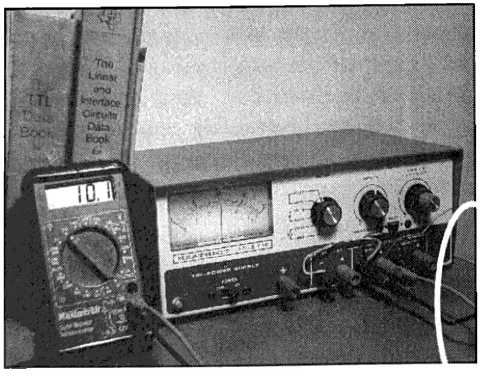


Figure 1.2 Photo d'un bloc d'alimentation dont une des sorties (encerclée) est réglée sur environ 10 Volt tel que confirmé par les indicateurs.

Le domaine continu contient un nombre infini de points car il s'agit du domaine *continu.*

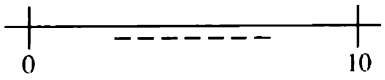


Figure 1.3

Ce domaine peut être numérisé en le divisant par 2, et en assignant à chaque tranche les symboles [0] et [1] :

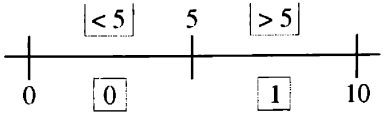


Figure 1.4

Bien sûr, cette première approximation est très grossière (car réalisée sur un bit seulement) ; on peut la poursuivre en redivisant chaque tranche. Voici d'ailleurs les résultats de cette conversion que l'on appelle *conversion analogique-numérique* respectivement sur deux et trois bits :

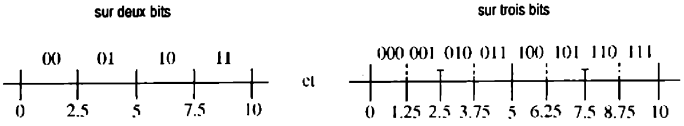


Figure 1.5

Dans le dernier cas, celui de la conversion sur trois bits (figure 1.5, à droite), on obtient huit nombres qui sont en fait l'équivalent binaire des chiffres décimaux allant de 0 à 7. Nous verrons ceci en détail un peu plus loin.

0	1	2	3	4	5	6	7
•	•	•	•	•	•	•	•
000	001	010	011	100	101	110	111

Figure 1.6

1.3 POURQUOI NUMÉRISER ?

Une question se pose alors : « Pourquoi numériser ? » En fait, il y a tellement d'avantages par rapport aux inconvénients. Observons d'ailleurs notre vie de tous les jours, on constate immédiatement que nous vivons à l'âge du numérique : transactions bancaires, enregistrements musicaux et vidéos numériques, commande numérique des moteurs des automobiles, etc. La liste est longue. Ce qui est important de noter – dans le cadre de ce livre –, c'est que cette « ère du numérique » correspond dans les faits à « l'ère du code binaire ». Les principaux avantages de cette « ère du numérique » sont les suivants :

1. Les données numériques sont moins sensibles au bruit par rapport aux données analogiques équivalentes et ceci aux étapes de l'acquisition, de la transmission et de la mémorisation de données. Soit *T* une grandeur qui est obtenue ou mesurée. En fait, *T* correspond à la grandeur physique combinée à un bruit qui peut être additif ou multiplicatif. Si on reprend l'exemple précédent de notre bloc d'alimentation de tension variable de 0 à 10 V, du « bruit » défini ici comme une fluctuation non désirée du signal, par exemple une variation de la tension alternative du secteur, peut affecter la valeur de sortie (la tension de sortie que l'on mesure). Comme on le constate sur la figure 1.7, lors de notre conversion analogique-numérique si l'ampleur du bruit fait que l'on reste dans une même « tranche », la valeur numérique correspondante restera inchangée à « 100 » pour le cas illustré. Différents mécanismes peuvent être introduits pour réduire l'influence néfaste du bruit, tel par exemple la mémorisation à l'avance d'une certaine durée de musique dans un lecteur CD ce qui lui permet de continuer à jouer la musique même en cas de choc qui déplace la tête de lecture sur le disque compact. Un autre exemple est le recours à des codes redondants lors de la transmission de données numériques. Ces codes permettent de détecter les erreurs de transmission (on transmet la valeur logique « un » au lieu de la valeur logique « zéro » ou inversement) et même éventuellement de corriger ces erreurs.

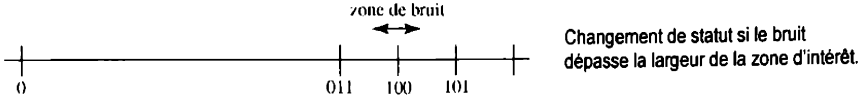


Figure 1.7

2. Une fois une grandeur physique obtenue sous forme numérique, une panoplie de traitements peuvent être employés pour mémoriser, transporter ou améliorer cette valeur. Ces opérations sont réalisées par des circuits numériques à semi-conducteur qui sont disponibles à très faibles coûts. Ainsi, les circuits intégrés contiennent des transistors qui sont en fait des interrupteurs logiques, ces interrupteurs commutent à des vitesses très rapides (plusieurs GHz comme dans le cas des microprocesseurs actuels). Peu d'énergie est nécessaire pour faire commuter (c'est-à-dire changer d'état) les transistors ce qui permet des applications portatives (alimentées en électricité par piles) et comme le coût par transistor est très faible (des µ$/transistor), des applications variées sont possibles. En fait, l'imagination en est la seule limite.

3. Une autre raison qui explique l'intérêt pour la circuiterie numérique est l'utilisation des théorèmes booléens qui permet une minimisation de l'usage du nombre de portes logiques (et donc de transistors) nécessaires pour l'implantation d'une fonction logique. L'Algèbre de Boole que nous étudierons quelque peu au chapitre 2 fait usage de théorèmes basés sur des relations vrai-faux. Par exemple : A–quand il pleut, B–je sors avec mon parapluie. Si A est vraie alors B le sera aussi. Dans le cas des circuits logiques la condition « vraie » est associée à l'état « 1 » logique et la condition « faux » à l'état « 0 ». L'algèbre booléen doit son nom au célèbre mathématicien Georges Boole qui vécut au XIXe siècle (lire le texte à ce propos au début du chapitre 2).

1.4 AVANTAGES DE DEUX ÉTATS

Une autre question se pose alors : « Pourquoi est-il si avantageux de recourir à un système mathématique limité à deux états (0,1) alors que notre système arithmétique traditionnel est, lui, décimal ? » La première raison vient justement de la disponibilité (massive, rapide, à faible coût) d'interrupteurs (les transistors dans les circuits intégrés) qui peuvent commuter entre deux états :

Un interrupteur dans un circuit intégré peut être : $\begin{cases} 1 - fermé \\ 0 - ouvert \end{cases}$.

Ces interrupteurs sont les « blocs Lego » de base qui permettent justement la réalisation des portes logiques, elles-mêmes à la base des circuits numériques arithmétiques. Ainsi les transistors permettent la réalisation des portes logiques et partant celle de circuits intégrés incorporant des fonctions logiques.

Nous avons déjà esquissé le lien entre le système décimal et le système binaire employé dans les circuits logiques, le lien entre les deux systèmes fait référence à l'encodage. L'encodage consiste à représenter quelque chose par un code.

Exemple : Prenons le cas d'un nombre plus petit que quatre. Nous devons donc encoder les chiffres {0, 1, 2, 3} ce qui donnera ceci (en faisant référence aux résultats de la section 1.2) :

On voit qu'ici, deux bits suffisent puisque, avec deux bits, on obtient quatre représentations différentes correspondant à nos quatre chiffres {0, 1, 2, 3}.

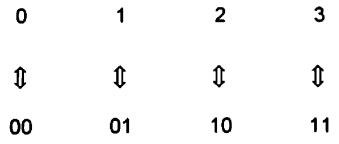


Figure 1.8

Introduisons un peu de terminologie. Nous avons vu le bit, qui fait référence à un seul chiffre binaire (donc ayant pour valeur « 0 » ou « 1 »). D'autres regroupements sont tellement communs en circuiterie logique qu'on leur a donné des noms particuliers. Ce sont le *nibble*, l'octet (*byte* en anglais) et le mot (*word* en anglais) qui désignent respectivement des blocs de 4, de 8 et de 16 bits. Finalement, la *string* (ou chaîne) est une suite de bits de longueur quelconque.

Terminologie : *nibble* : 4 bits

octet : 8 bits (*byte* en anglais)

mot : 16 bits (*word* en anglais ; 2 octets)

1 *string* : suite de bits (ou chaîne).

1.5 REPRÉSENTATION BINAIRE

Nous sommes prêts maintenant à introduire l'arithmétique binaire qui repose sur la base 2, tout comme notre arithmétique usuelle décimale, qui elle, repose sur la base 10. L'arithmétique binaire emploie deux symboles (0, 1), et, tout comme en décimal, la position des symboles est significative. Ainsi, avoir 123 $ dans ses poches ce n'est pas comme en avoir 321 $. Il est à noter que d'autres bases sont aussi très utiles en circuiterie logique, par exemple la base octale (8) et la base hexadécimale (16). Ces trois bases (binaire, octale, hexadécimale) sont liées entre elles par les puissances de 2 puisque $16 = 2^4$, $8 = 2^3$ et $2 = 2^1$. Les conversions d'une de ces bases en une autre seront donc très aisées, ce qui ne sera pas le cas avec la base décimale, qui n'est pas une puissance entière de 2. La base 10 requiert 10 symboles que nous connaissons bien : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. La base binaire en emploie deux (0 et 1), comme nous l'avons déjà vu. La base octale en emploie donc huit : 0, 1, 2, 3, 4, 5, 6, 7, et la base hexadécimale, seize : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, *A*, *B*, *C*, *D*, *E*, *F*. Comme on le voit ici, l'utilisation des lettres (*A*, *B*, *C*, *D*, *E*, *F*) permet d'obtenir les 16 symboles distincts requis de la base hexadécimale.

CONVERSION DE BASE QUELCONQUE EN UN NOMBRE DECIMAL

La relation suivante permet la conversion entre un nombre de base B quelconque en un nombre D exprimé en base 10 :

$$D = \sum_{i=0}^{i\,max} P_i B^i \leftarrow base \qquad (1.1)$$

où : i est la position courante dans le nombre,

où : i max est le nombre de positions –1 dans la base B,

où : P est le chiffre (aussi appelé le « poids ») à chaque position i du nombre à convertir.

Par habitude, on travaille toujours en base 10 et il n'est pas nécessaire de le spécifier. Cependant, lorsque l'on travaille dans une autre base, il est nécessaire de l'indiquer. Cela se fait en mettant la base de travail en indice juste après le nombre. En cas de confusion possible, on indique toujours la base de travail. Ainsi, 72_8 ou 72_o font référence au nombre 72 exprimé en base 8 (noter l'abréviation o pour octal) ; 72_{16} ou 72_{Hex} ou 72_H au nombre 72 en base hexadécimale (noter l'abréviation Hex ou H pour hexadécimal) et 72_{10} ou 72_d au nombre 72 en base décimale (noter l'abréviation d pour décimal). Évidemment, $72_{10} \neq 72_8 \neq 72_H$. Finalement, on emploie l'abréviation « b » (ou le chiffre 2) pour les nombres exprimés en binaire. Les quelques exemples suivants illustrent l'utilisation de la relation (1.1) précédente pour les conversions.

Exemple : Convertir le nombre 72_8 dans la base 10 :

$P_o = 2,\ P_1 = 7$

$D = 2\times8^0 + 7\times8^1 = 2 + 56 = 58_{10} = 58$

Exemple : Convertir le nombre 72_H dans la base 10 :

$P_0 = 2,\ P_1 = 7$

$D = 2\times16^0 + 7\times16^1 = 2+112 = 114_{10} = 114_d$

Comme on le constate 72_8 est différent de 72_H.

Exemple : Convertir le nombre 48_H dans la base 10 :

$P_0 = 8, P_1 = 4$

$D = 8\times16^0 + 4\times16^1 = 8+64 = 72_{10} = 72_d$

Exemple : Convertir le nombre $(1234)_5$ dans la base 10 :

$$D = 4\times5^0 + 3\times5^1 + 2\times5^2 + 1\times5^3$$
$$= 4+15+50+125 = 194_{10}$$

Exemple : Convertir le nombre $(1234)_{16}$ dans la base 10 :

$$D = 1\times16^3 + 2\times16^2 + 3 + 16^1 + 4\times16^0$$
$$= 4096+2\times256+3\times16+4$$
$$= 4660_{10}$$

REPRESENTATIONS DES NOMBRES DECIMAUX 0 A 16 DANS LES BASES BINAIRE, HEXADECIMALE ET OCTALE

La relation (1.1) permet de dresser le tableau suivant où sont exprimés les nombres 0 à 16 dans les quatre bases dont nous ferons usage (2, 8, 16 et 10).

Base décimale	Base binaire	Base hexadécimale		Base octale
0	0000	0		0
1	0001	1		1
2	0010	2		2
3	0011	3		3
4	0100	4		4
5	0101	5		5
6	0110	6		6
7	0111	7		7
8	1000	8		10
BCD 9	1001	9		11
10	1010	A		12
11	1011	B		13
12	1100	C	*Nouveaux*	14
13	1101	D	*symboles*	15
14	1110	E		16
15	1111	F		17
16	10000	10		20

Figure 1.9 Représentations des nombres décimaux 0 à 16 dans les bases binaire, hexadécimale et octale

De ce tableau, on remarque que quatre bits permettent de coder 16 états (car $16 = 2^4$), de 0 à 15_{10}. De même trois bits permettent de coder huit états (car $8 = 2^3$), de 0 à 7_{10}.

Une application amusante du code binaire «pur» est la montre «Samui Moon» de la compagnie allemande The One! En effet, l'affichage de l'heure est présenté sur deux rangées (heures avec 4 DEL lumineuses et minutes avec 5 DEL lumineuses), en binaire![11]

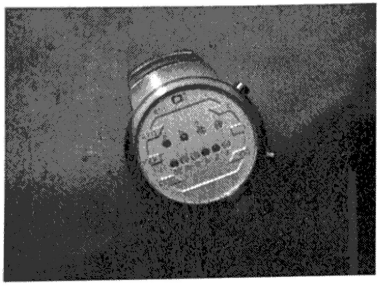


Figure 1.9.1 (RENUMÉROTER) Image courtoisie de Wikipedia (http://en.wikipedia.org/wiki/Binary_clock).

Code BCD

Même si quatre bits permettent le codage de 16 états, on restreint parfois le codage d'un nibble à dix états seulement, c'est ce que l'on appelle le codage BCD pour le sigle anglais « *Binary Coded Decimal* ». Le codage BCD encode les nombres décimaux sur quatre bits. Cela est pratique, car les conversions binaires à décimales et inversement sont alors directes comme illustré dans les exemples suivants :

[11] Voir : http://fr.wikipedia.org/wiki/Montre_%C3%A0_affichage_binaire

Exemple : En BCD : 99_{10} = [1001 1001]$_2$ (donc un codage sur 8 bits au total) ;
9 9

En binaire : 99_{10} = [110 0011]$_2$ (donc un codage sur 7 bits au total) ;

En BCD : 1024_{10} = [0001 0000 0010 0100]$_2$ (donc un codage sur 16 bits au total)
1 0 2 4

En binaire : 1024_{10} = [100 0000 0000]$_2$ (donc un codage sur 11 bits au total)

Le codage BCD est donc un code moins compact, mais plus simple. Il est notamment utilisé dans les calculatrices où l'interaction continuelle avec l'opératrice familière avec l'arithmétique décimale rend son usage pratique. Un autre usage commun concerne les affichages à 7 segments.

Code Gray

Le **code Gray** est un autre code très utile. La particularité du code Gray est que d'un code à l'autre chaque représentation ne change que d'un seul bit. Par ailleurs, on note aussi une symétrie par rapport au nombre médian des représentations. Le tableau 1.2 illustre le code Gray.

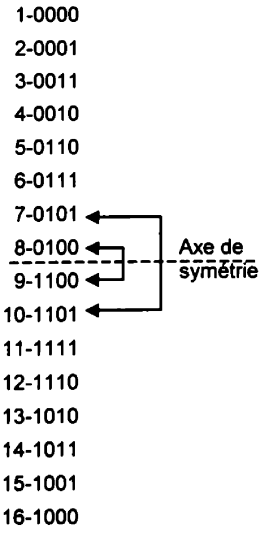


Figure 1.10

Une grande utilité du code Gray vient de ce que, lors de l'encodage d'entrées multiples changeant d'une façon incrémentale, un seul bit change à la fois dans la séquence de codes obtenus, ce qui offre une immunité accrue aux erreurs.

Cette disposition peut aussi être vue selon une arborescence horizontale, ce qui permet d'étendre éventuellement le concept à plus de bits :

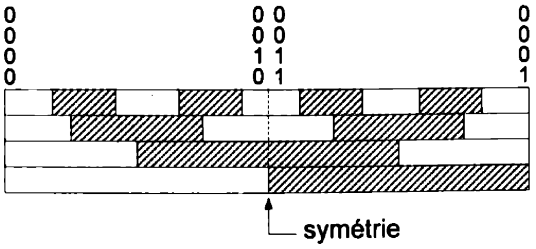


Figure 1.11

On note que l'état décimal que l'on a associé à chaque code Gray ne correspond bien sûr pas au poids binaire de ce code, il correspond plutôt au défilement de la séquence de quatre bits, de 0000 à 1111.

Le code Gray est très employé pour éviter les erreurs lors de la transmission de données qui se présentent de façon croissante ou décroissante puisqu'on peut immédiatement repérer les erreurs sachant que les codes ne peuvent changer que d'un bit à la fois. Ce n'est pas le cas du codage binaire pur où 4 bits changent simultanément lorsqu'on passe de 7_d (=0111_2) à 8_d (=1000_2) avec tous les risques d'erreurs lors d'une transmission sur une ligne de mauvaise qualité!

REPRESENTATION DE GRANDS NOMBRES EN BINAIRE

Comme on le constate à la figure 1.9, *Représentations des nombres décimaux 0 à 16*, la base binaire n'est pas très « forte » puisqu'il faut déjà quatre bits pour représenter le chiffre 8_d. Ainsi, en binaire on obtiendra rapidement des chiffres très longs (de plusieurs bits) pour représenter les grands nombres. Pour ceux-ci : on rajoute simplement des bits.

CONVERSIONS OCTALES ET HEXADECIMALES EN BINAIRE (ET INVERSEMENT)

Les conversions des bases octales et hexadécimales en binaire (et inversement) sont très aisées puisque les bases octales (2^3) et hexadécimales (2^4) sont des puissances de 2. La conversion binaire en octale s'effectuera donc en faisant des groupements de trois bits et la conversion binaire en hexadécimal en faisant des groupements de 4 bits. Les exemples suivants illustrent le processus :

Exemple : $(10\ 10\ 11\ 01\ 01)_2$ à convertir en hexadécimal et en octal.

octal : $\underbrace{001}_{1}\ \underbrace{010}_{2}\ \underbrace{110}_{6}\ \underbrace{101}_{5} \rightarrow 1265_8$ **groupe de 3**

hex : $\underbrace{0010}_{2}\ \underbrace{1011}_{B}\ \underbrace{0101}_{5} \rightarrow 2B5_H$ **groupe de 4**

Notez ici que l'on a « complété » le groupement le plus significatif (celui de l'extrême gauche) à l'aide de « zéros » de façon à obtenir un groupement complet de trois ou quatre bits.

Vérifions : $1265_8 = 1\times8^3 + 2\times8^2 + 6\times8^1 + 5\times8^0 = 512 + 2\times64 + 48 + 5 = 693_{10}$ et

$$2B5_H = 2\times16^2 + 11\times16^1 + 5\times16^0 = 2\times256 + 176 + 5 = 693_{10}$$

Les conversions hexadécimales (ou octales) en binaire se font simplement en écrivant la valeur en binaire de chacun des chiffres du nombre (selon les valeurs du tableau précédent des *Représentations des nombres décimaux 0 à 16*, figure 1.9). L'exemple suivant illustre le processus.

Exemple : Convertir 372_8 en binaire et en hexadécimal.

On effectue d'abord la conversion en binaire en écrivant la valeur de chaque chiffre octal. Une fois la valeur binaire du nombre obtenue, des groupements de quatre bits fournissent la valeur hexadécimale :

$$372_8 = ([011]\ [111]\ [010])_2 = (11111010)_2 = ([1111]\ [1010])_2 = FA_H$$

À noter que les groupements se font toujours de droite à gauche puisque les nombres que nous étudions ici font appel au codage *positionnel* (la position du code influençant sa valeur : 123 $ ≠ 321 $). C'est bien sûr un codage très employé.

Exemple : Convertir $2B5_H$ en binaire et octal.

On effectue d'abord la conversion en binaire en écrivant la valeur de chaque chiffre hexadécimal. Une fois la valeur binaire du nombre obtenue, des groupements de trois bits fournissent la valeur octale. Cela donne :

$$\underbrace{0010}_{2}\ \underbrace{1011}_{B}\ \underbrace{0101}_{5} \Rightarrow \underbrace{001}_{1}\ \underbrace{010}_{2}\ \underbrace{110}_{6}\ \underbrace{101}_{5}$$

La réponse s'obtient directement : 1265_8, et pour le binaire : $(10\ 1011\ 0101)_2$.

1.6 MÉTHODES DE CONVERSION BINAIRE EN DÉCIMAL

Il y a plusieurs façons de procéder pour réaliser de telles conversions, celles-ci sont passées en revue dans cette section. On obtient évidemment les mêmes résultats, peu importe la méthode choisie.

1. La première méthode emploie la relation suivante déjà vue à la section 1.5 dans le cas de l'arithmétique de base quelconque (relation 1.1, page 9), mais appliquée ici à la base 2 :

$$D = \sum_{i=0}^{N-1} B_i 2^i \text{ , où } B_i = 0 \text{ ou } 1, \text{ le } i^e \text{ bit et } N = \text{le nombre de bits.} \qquad (1.2)$$

Exemple : Que vaut $(1010)_2$ en base décimale ?

Ici, $N = 4$ puisque nous avons quatre bits, et on obtient directement avec la relation précédente :

$$= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$$

$$= 8 + 0 + 2 = 10_{10}$$

Exemple : Que vaut $(1\ 0101\ 1101)_2$ en base décimale ?

Ici, $N = 9$ puisque nous avons neuf bits, et on obtient directement avec la relation précédente :

$$= 1 \times 2^8 + 0 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 256 + 0 + 64 + 0 + 16 + 8 + 4 + 0 + 1 = 349_{10}$$

2. La deuxième méthode a recours à une base intermédiaire, généralement la base hexadécimale. Cette conversion intermédiaire s'obtient facilement en faisant des groupements de quatre bits. La conversion en décimale s'obtient finalement en utilisant la relation 1.1 vue précédemment. L'avantage d'une telle « astuce » est qu'elle requiert moins d'opérations puisque la base hexadécimale est plus « forte » que la base binaire.

Exemple : Que vaut $(1\ 0101\ 1101)_2$ en base décimale ?

En faisant des groupements de quatre bits, selon ce qui a été vu précédemment, on obtient immédiatement la valeur hexadécimale du nombre, soit $15D_H$. Par la suite, le nombre est exprimé dans ses puissances de 16 correspondantes :

$$\underbrace{0001}_{1}\ \underbrace{0101}_{5}\ \underbrace{1101}_{D_H} = 1\times16^2 + 5\times16^1 + 13\times16^0$$

$$= 256 + 80 + 13$$

$$= 349_{10}$$

On obtient évidemment le même résultat qu'à l'exemple précédent, mais on constate qu'il y a moins de risque d'erreur puisqu'il y a moins d'opérations à faire.

3. La troisième méthode consiste simplement à employer une calculatrice ou un logiciel de type Matlab qui exécute ces conversions directement. Bien évidemment, l'usage des deux méthodes précédentes permet de comprendre comment se font les calculs réalisés par la calculette ou le programme de conversion.

1.7 CALCUL DES FRACTIONS

Les fractions se calculent de la même façon que les nombres entiers étudiés précédemment. C'est en fait le même procédé, toutefois avec des exposants négatifs qui permettent de définir les fractions.

La liste suivante fournit la valeur des premières puissances de deux fractionnaires. Comme on le sait aussi $2^0 = 1$.

$$2^{-1} = 0.5$$
$$2^{-2} = 0.25$$
$$2^{-3} = 0.125$$
$$2^{-4} = 0.0625$$
$$2^{-5} = 0.03125$$

L'expression générale pour un nombre entier et fractionnaire est la même que celle vue précédemment (relation 1.1), sauf que cette fois on permet aux exposants de prendre des valeurs négatives, ce qui introduit les fractions :

$$D = \sum_{k=-N}^{+m} B_k 2^k \qquad (1.3)$$

Pour un nombre B en base binaire. Les B_k sont les bits du nombre B, et D sa valeur décimale correspondante. « k » correspond à la position de chaque bit dans le nombre :

$$B_m B_{m-1} \ldots B_2 B_1 B_0 . B_{-1} B_{-2} \ldots B_{-n}$$

Exemple : $(0.1)_2 = 0\times 2^0 + 1\times 2^{-1} = 0 + \frac{1}{2} = 0.5_{10}$

Exemple : $(0.1010)_2 = 0\times 2^0 + 1\times 2^{-1} + 0\times 2^{-2} + 1\times 2^{-3} = 0.5 + 0.125 = (0.625)_{10}$

1.8 CONVERSION DE BASE DÉCIMALE EN BASE BINAIRE

La conversion de la base décimale en base binaire est plus ardue que l'inverse puisque 10 n'est pas une puissance entière de 2. Il y a plusieurs façons pour réaliser de telles conversions, celles-ci sont passées en revue dans cette section. On obtient évidemment les mêmes résultats, peu importe la méthode choisie.

1. La méthode par *soustraction* successive de la plus grande puissance de 2. Ici, on décompose le nombre en une somme des plus grandes puissances de 2 qu'il contient. Les exemples suivants illustrent la procédure. Le tableau suivant reprend les premières puissances de 2 utiles pour ces calculs. Puisque les puissances de 2 ne sont pas « fortes », tel qu'il a été discuté précédemment, on remarque que la décomposition est fastidieuse pour les grands nombres nécessitant ainsi plusieurs étapes.

$2^0 = 1$	$2^1 = 2$	$2^2 = 4$
$2^3 = 8$	$2^4 = 16$	$2^5 = 32$
$2^6 = 64$	$2^7 = 128$	$2^8 = 256$

Figure 1.12

Exemple : Quel est l'équivalent binaire de $(33)_{10}$?

$$33_{10} = 32 + 1 = 2^5 + 2^0 = (10\ 0001)2$$

On note que seules les puissances de 2 présentes dans la décomposition fournissent un « 1 » dans le résultat, à une position qui correspond à leur puissance de 2 respective, ici aux positions 0 et 5.

Exemple : Quel est l'équivalent binaire de $(1997)_{10}$?

$$\begin{aligned} 1997 &= 1024+512+256+128+64+8+4+1 \\ &= 2^{10}+2^9+2^8+2^7+2^6+2^3+2^2+2^0 \end{aligned}$$

Les puissances de 2 en présence sont les suivantes :

1 1 1 1 1 0 0 1 1 0 1

10 9 8 7 6 5 4 3 2 1 0

Ce qui donne donc comme résultat : $(111\ 1100\ 1101)_2$

2. La méthode par *division* successive. Pour réduire le nombre d'étapes, on a recours ici à la base 16, et on divise le nombre à convertir successivement par la plus grande puissance de 16 possible. Comme on est à même de le constater, la liste des puissances de 16 nécessaires pour les calculs courants est très courte puisque déjà 16^4 égale 65 536. Une telle décomposition fournit l'équivalent hexadécimal du nombre. Sa valeur binaire est alors directement obtenue puisque la conversion hexadécimale en binaire est directe (section 1.5).

$16^1 = 16$ $16^2 = 256$

$16^3 = 4096$ $16^4 = 65636$

Exemple : Quel est l'équivalent binaire de $(33)_{10}$?

On voit qu'on peut diviser le nombre avec 16 seulement (la puissance suivante, 256, étant trop élevée) :

$$33/16 = 2$$

donc $33 = 2 \times 16 + 1 = 2 \times 16^1 + 1 \times 16^0 = 21_H$

et alors la conversion hexadécimale puis binaire s'obtient directement :

$$(21)_{hex} = 0010\ 0001 = (10\ 0001)_2$$

Exemple : Quel est l'équivalent binaire de $(1997)_{10}$?

Ici, on voit que l'on peut diviser le nombre par 256 et le reste obtenu par 16 :

1997 = 1997/256 = 7, reste 205 et 205/16 = 12, reste 13.

Le nombre correspondant hexadécimal, et donc binaire, est finalement :

$$= 7CD_H\ (\underbrace{0111}_{7}\ \underbrace{1100}_{12}\ \underbrace{1101}_{13})_2$$

Exemple : Quel est l'équivalent binaire $(321)_{10}$?

321/256 = 1, reste 65 et 65/16 = 4, reste 1.

$$321 = (141)_H = (0001\ 0100\ 0001)_2$$

Le résultat est alors : $(1\ 0100\ 0001)_2$

On pourrait diviser par 2 plutôt que par 16, mais les calculs seraient bien sûr plus long.

1.9 CONVERSION DE FRACTIONS

Nous avons vu qu'en binaire, les nombres entiers sont représentés par les puissances positives de 2 ; de leur côté, les puissances négatives permettent de représenter les fractions. En effet, $2^{-1} = ½ = 0.5$. De même, $2^{-2} = ¼ = 0.25$. En décomposant le nombre fractionnaire en ses puissances de 2 négatives correspondantes, on pourra représenter un nombre fractionnaire quelconque. Les méthodes vues précédemment s'appliquent donc. Cependant, le processus peut être sans fin ; il faut ainsi spécifier une *résolution*, par exemple sur 8, 16 ou 32 bits. Les exemples suivants illustrent le processus de décomposition en puissance de 2 par soustraction successive.

Exemple : Quel est l'équivalent binaire de $(0.4)_{10}$?

À l'aide du tableau (figure 1.13), on voit immédiatement que cette fraction se décompose comme suit :

$$\begin{aligned} 0.4 &= 0.25 + 0.125 + 0.015625 + \ldots \\ &= [0.390625] \\ &= 2^{-2} + 2^{-3} + 2^{-6} + \ldots \end{aligned}$$

Seules les puissances de 2 présentes dans la décomposition sont reprises dans le résultat :

0	1	1	0	0	1
-1	-2	-3	-4	-5	-6

La réponse est donc :

$$0.4_{10} = (0.011001)_2$$

On le constate, le résultat obtenu est une approximation (0.390625) de la valeur à convertir 0.4. On aurait pu poursuivre la décomposition sur plus de bits pour raffiner le calcul. Ici la conversion s'est faite avec une *résolution* de 6 bits.

$$2^{-1} = 0.5$$
$$2^{-2} = 0.25$$
$$2^{-3} = 0.125$$
$$2^{-4} = 0.0625$$
$$2^{-5} = 0.03125$$
$$2^{-6} = 0.015625$$

Figure 1.13 Table des fractions

TRONCATURE ET ARRONDISSEMENT

Si on stoppe simplement le calcul de la fraction après un nombre arbitraire de chiffres (comme dans l'exemple précédent), on parle alors de *troncature.*

Exemple : 3.14159 = π, tronqué ici à cinq chiffres après le point.

On peut aussi compléter, c'est-à-dire arrondir à la dernière position avant la conversion afin d'obtenir plus de précision, c'est ce qu'on appelle l'arrondissement.

Exemple : Reprenons la valeur de π exprimée sur quatre chiffres après le point décimal :

3.1416 est plus précis que 3.1415

Le premier est *arrondi*, alors que le deuxième est *tronqué.*

1.10 ADDITION DE NOMBRES BINAIRES NON NEGATIFS

L'addition est l'opération arithmétique la plus importante. Les mêmes principes que ceux de l'addition « décimale » sont employés en arithmétique binaire, mais en plus simple. Considérons d'abord l'addition simple de deux bits : il n'y a que quatre cas à considérer :

$$4 \text{ cas} \begin{cases} 0 + 0 = 0 & \text{somme} \\ 0 + 1 = 1 & \\ 1 + 0 = 1 & \\ 1 + 1 = 0 & \text{retenue}_1 \quad \text{somme}_0 \end{cases}$$

Le cas général de l'addition de deux nombres binaires de plusieurs bits, se raisonne alors comme le cas simple que nous venons de voir, mais en y ajoutant la retenue éventuelle issue de l'addition des deux bits précédents. Ainsi, le cas général revient à additionner trois bits (les deux bits considérés et la retenue). Examinons les huit cas possibles :

0	0	0	0	1	1	1	1}	Retenue
0	0	1	1	0	0	1	1}	+ nombre A
0	+1	+0	+1	+0	+1	+0	+1}	+ nombre B
00	01	01	10	01	10	10	11	résultat : retenue et somme

Figure 1.14

De là, l'addition de nombres binaires quelconque se généralise directement comme on l'explique dans les exemples suivants.

Exemple : Additionner $7_{10} + 22_{10}$ en binaire.

Convertissons d'abord les nombres décimaux en binaires. 7 s'obtient directement du tableau des *Représentations des nombres décimaux 0 à 16* (figure 1.9, page 11), et 22 est calculé par décomposition en puissance de deux (section 1.5) :

$$22 = (16 + 4 + 2) = (10110)_2$$

Le calcul est alors comme suit avec une retenue au 3e bit :

$$\begin{array}{r} 1 \\ 00111 \\ 10110 \\ \hline (11101)_2 = D_H = 16+13 = 29_{10} \end{array}$$

En reconvertissant en décimal, selon les méthodes étudiées précédemment, on retrouve le résultat attendu.

En terminant, notons que, puisque la terminologie anglo-saxonne est très commune, la retenue d'entrée est souvent appelée *carry in* ou C_{in}, et la retenue de sortie *carry out* ou C_{out}.

1.11 SOUSTRACTION PAR LA MÉTHODE DU COMPLÉMENT 2

Après l'addition, intéressons-nous à la soustraction de nombres binaires. En fait, si on sait additionner, on sait soustraire puisque la soustraction de deux nombres A et B peut s'exprimer comme suit :

$$A - B = A + (-B)$$

Pour soustraire deux nombres, il suffit donc d'additionner le premier (A) au complément du second ($-B$).

COMPLEMENT 1 (C1)

En arithmétique binaire, on distingue deux types de complément. Le « complément un » est obtenu par inversion de tous les bits du nombre à complémenter : les 1 sont remplacés par des 0 et inversement.

Exemple : Quel est le complément un de $(101)_2$?

Solution : En inversant tous les bits, on obtient : $(010)_2$.

L'utilité du *complément 1* est pour le calcul du *complément 2* (voir plus bas).

Dans le cas de la notation binaire signée, les nombres peuvent être soit positifs, soit négatifs. Le bit de gauche[12] correspond alors au bit de signe. Le signe moins est représenté par un « 1 » et le signe plus par un « 0 ». Ainsi, en notation binaire signée, le nombre de quatre bits $(1101)_2$ est négatif, mais le nombre de quatre bits $(0101)_2$ est positif. On note dans ce cas que le 0 le plus significatif est très important. Pour cette raison, lorsque l'on calcule le complément d'un nombre binaire, on spécifie d'abord le nombre de bits qui le représenteront. Comme on le verra un peu plus bas, pour un calcul donné, le nombre de bits choisis fixe la plage de valeurs possibles.

COMPLEMENT 2 (C2)

Le complément employé pour la soustraction en binaire est le « complément 2 », il s'obtient en additionnant 1 au *complément un* du nombre.

Méthode de calcul du complément 2 :

1. Choisir un nombre de bits pour représenter le nombre ;
2. Se rappeler que le bit à gauche est le bit de signe ;
3. Inverser tous les bits du chiffre puis additionner 1.

---

12 Notation : Le bit de l'extrême gauche est aussi appelé *bit le plus significatif*, car son poids binaire est le plus élevé (le sigle anglais couramment employé est MSB pour *Most Significant Bit*). De même, le bit de l'extrême droite est appelé *bit le moins significatif*, car son poids binaire est le moins élevé (le sigle anglais couramment employé est LSB pour *Least Significant Bit*). Ainsi, pour le nombre de deux bits $(10)_2$, 1 est le MSB, et 0 le LSB.

Important Si l'on veut calculer la valeur d'un nombre binaire signé négatif, il suffit de reprendre les étapes 2 et 3.

Exemple : Calculer le *complément 2* de 7 sur quatre bits.

+ 7 vaut (sur quatre bits) :	+ 7	0111
On inverse tous les bits et on ajoute 1 :		1000 + 1
Résultat final (on note que le MSB est à 1 puisque le nombre est négatif) :	– 7	1001

Si maintenant on veut déterminer la valeur de $(1001)_2$, on note que le bit de signe est 1 et qu'il s'agit d'un nombre signé et négatif pour lequel la valeur est obtenue comme suit :

	1001
On inverse tous les bits et on ajoute 1 :	0110 + 1
	0111

Le résultat final est –7 puisque l'on avait un nombre négatif au départ.

Exemple : Calculer le complément 2 du chiffre « 0 » sur quatre bits

+ 0 vaut (sur quatre bits) :	+ 0	0000
On inverse tous les bits et on ajoute 1 :		1111 + 1
On laisse tomber le cinquième 1, car ici on a choisi une représentation sur quatre bits.	– 0	~~1~~0000

Comme on le constate : – 0 = + 0 = 0, ce qui est bien logique.

Si on reprend les calculs que nous venons de faire pour tous les chiffres binaires sur quatre bits de $(0000)_2$ à $(1111)_2$, on obtient la représentation suivante, qui peut être mise sous la forme d'une « roue » :

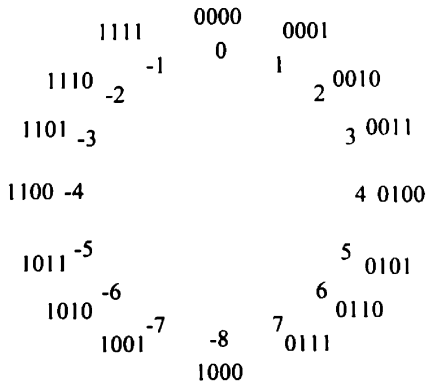


Figure 1.15

Comme on le constate avec cette « roue », le nombre de bits fixe en fait la plage des valeurs possibles, sur quatre bits de – 8 à + 7, c'est donc un paramètre essentiel à

stipuler lors d'un calcul afin d'éviter tout débordement (aussi appelé dépassement). On rediscutera de ce dernier point un peu plus loin, d'ailleurs.

Exemple : Quelle est la plage des valeurs signées sur cinq bits ?

En s'inspirant de la roue pour le cas des quatre bits, on obtient immédiatement que la plus grande valeur positive sera $(0\ 1111)_2$, et la valeur négative son *complément 1*,soit $(1\ 0000)_2$.

$(0\ 1111)_2$ vaut bien sûr + 15 en décimal (voir le tableau des *Représentations des nombres décimaux 0 à 16*, figure 1.9, page 11), et la valeur de $(1\ 0000)_2$ est obtenue en notant qu'il s'agit d'un nombre négatif (MSB à 1). En inversant tous les bits et en ajoutant 1, cela donne une valeur de – 16 en décimal (tel qu'il a été indiqué plus haut, on rapplique la notion de complément 2 pour déterminer la valeur).

SOUSTRACTION A L'AIDE DU COMPLEMENT 2 (C2)

Une fois à l'aise avec le concept de complément 2, on peut aborder la soustraction binaire en utilisant ce dernier et la relation annoncée au début de cette section : A – B = A + (–B). L'exemple suivant illustre la procédure.

Exemple : Calculer 7 – 3.

On choisit une représentation sur quatre bits. Les valeurs de 7 et de –3 sont obtenues de la « roue » précédente :

$7 - 3 = 7 + (-3) = (0111)_2 + (1101)_2$

Rappel sur le complément 2 pour –3 :

+ 3	⇔	0011
		1100
+		+ 1
– 3	⇔	1101

Finalement le résultat s'obtient par addition :

	0111	+ 7
+	1101	– 3
	10100	+ 4

Comme notre notation est sur quatre bits, on rejette donc le cinquième bit et ce résultat de $+4 = (0100)_2$ est bien sûr correct.

Exemple : Calculer 7 + 3 en employant une représentation en binaire signé sur 4 bits ? L'exercice paraît tout bête ; par contre, il révèle une attrape comme nous le verrons.

```
  0111 : + 7
  0011 : + 3
0 1010
```

On obtient un résultat de $(1010)_2$ avec une retenue de 0, ce qui correspond selon la « roue » vue plus haut à un nombre négatif (le MSB est à 1) valant : 1010 : –6. Ceci s'explique de façon très simple : en notation complément 2 sur 4 bits, (7 + 3) = + 10 n'existe pas. Ceci illustre l'importance du choix initial du nombre de bits requis pour effectuer une opération de calcul. En effet, si on avait choisi une représentation sur 5 bits, on aurait obtenu un résultat de $(01010)_2$, qui est un nombre positif (MSB = 0) qui correspond bien au résultat désiré de +10.

L'exemple précédent illustre la notion de dépassement (*overflow* en anglais). Un dépassement survient lorsque le résultat obtenu dépasse la plage des valeurs possibles dans la représentation choisie. Il est donc important de spécifier un nombre de bits suffisant qui permettra d'accommoder les valeurs extrêmes que l'on pourra obtenir dans les calculs envisagés. Cela s'applique aux nombres signés (attention donc à l'étape 1 du calcul des *compléments 2*), mais aussi bien sûr aux nombres non signés.

FRACTION NEGATIVE EN COMPLEMENT 2 (C2)

On peut calculer le complément 2 d'une fraction négative à l'aide d'une fraction positive. Par exemple, prenons la fraction $(-0.75)_{10}$. Cette fraction peut être réécrite comme ceci :

$$-0.75 = -1 + (1-0.75) = -1 + 0.25$$

Et tel que vu précédemment, -1 en complément 2 sur 2 bits donne $(11)_2$ et +0.25 correspond à $(0.01)_2$.

On obtient finalement que le complément 2 de $(-0.75)_{10}$ est $(11.01)_2$. Le bit le plus significatif est à 1, le nombre est donc bien négatif.

Notons qu'on aurait pu aussi procéder directement :

$$+ 0.75_{10} = 0.5_{10} + 0.25_{10} \text{ <-> } 0.1_2 + 0.01_2 = 0.11_2,$$

Selon la procédure d'obtention du complément 2, on inverse alors tous les bits et on additionne 1, ainsi nous obtenons (sur 2 bits):

$$\begin{array}{r} 11.00 \\ +\quad 1 \\ \hline (11.01)_2 \end{array}$$

Le résultat obtenu est bien sûr le même.

Exemple : Calculer le nombre $(-8.125)_{10}$ en complément 2.

$$(-8.125)_{10} = -8 - 0.125 = -8 -1 + (1 - 0.125) = -9 + 0.875$$

Et en binaire signé (C2) sur 5 bits cela donne $(10111)_2 + (0.1110)_2 = (10111.111)_2$.

En procédant directement sur 5 bits : $(+8.125)_{10}$ <-> $(01000.001)_2$, on inverse les bits et rajoute 1 :

$$\begin{array}{r} 10111.110 \\ +\quad 1 \\ \hline (10111.111)_2 \text{ <-> } (-8.125)_{10} \end{array}$$

DETECTION DE DEPASSEMENT

Il y a moyen de détecter les dépassements simplement en comparant les signes des nombres additionnés ensemble. Ainsi, si on additionne deux nombres de même signe, et que le signe de la somme obtenue est différent, c'est qu'il y a eu un dépassement. L'exemple précédent de « 7+3 » sur 4 bits est un exemple d'une telle situation. Bien sûr, il est très important de détecter les dépassements lors des opérations arithmétiques. Ainsi, bien des microprocesseurs et microcontrôleurs détectent cette condition de façon automatique grâce à un bit de statut appelé communément bit de dépassement ou bit d'*overflow* (abréviation OVR). En vérifiant ce bit de statut après chaque calcul, on peut

s'assurer de calculer sans dépassement, donc sans erreur (si OVR = 1, il y a eu dépassement et donc erreur). Un autre bit de statut relié aux opérations de système est le bit de « retenue » ou bit *carry out* (abréviation C_{out}) qui est simplement la retenue obtenue à la suite du calcul. L'exemple suivant illustre l'obtention de ces deux bits de statuts. De façon spécifique, si C_{out} est la retenue, SIG le bit de signe, alors OVR peut être défini comme ceci (dans le cas d'une addition) :

$$OVR = SIG \oplus C_{out}$$

où $\oplus$ correspond à l'opération de OU-exclusif, que nous verrons plus bas (section 1.14). En deux mots, le OU-exclusif « exclut » le cas où les deux arguments sont vrais. Dans le dernier exemple de 7 + 3, le résultat fournit les bits de status $C_{out} = 0$, SIG = 1, donc OVR est 1 et il y a dépassement.

Exemple : Calculer 6 + 1 en employant une représentation sur 4 bits.

On a immédiatement que +6 = $(0110)_2$ et que +1 = $(0001)_2$.

L'addition s'obtient alors comme suit :

$$\begin{array}{rr} & 0110 \\ + & 0001 \\ \hline & 0\,0111_2 \end{array} = +7$$

Ici, le bit de retenue C_{out} est 0 (pas de retenue), le bit de signe SIG = 0, et le bit de dépassement OVR vaut donc 0 également : il n'y a pas de dépassement.

Exemple : Calculer 6 + 2 en employant une représentation sur 4 bits.

On a immédiatement que +6 = $(0110)_2$ et que +2 = $(0010)_2$.

L'addition s'obtient alors comme suit :

$$\begin{array}{rr} & 0110 \\ + & 0010 \\ \hline & 0\,1000_2 \end{array} = -8$$

Ici le bit de retenue C_{out} est 0 (pas de retenue), le bit de signe SIG = 1, et le bit de dépassement OVR vaut donc 1. Il y a dépassement. Effectivement, – 8 ne correspond pas au résultat. Il faudrait employer ici une représentation sur 5 bits.

Exemple : Calculer – 1 + – 1 en employant une représentation sur 4 bits.

On a immédiatement que – 1 = $(1111)_2$.

L'addition s'obtient alors comme suit :

$$\begin{array}{rr} & 1111 \\ + & 1111 \\ \hline & 1\,1110_2 \end{array} = -2 \text{ (selon la roue précédente)}$$

Ici le bit de retenue C_{out} est 1 (retenue), le bit de signe SIG = 1, et le bit de dépassement OVR vaut donc 0, ce qui est juste, il n'y a pas de dépassement.

1.12 ADDITION ET SOUSTRACTION DE FRACTIONS BINAIRES

Le même procédé d'addition vu précédemment (avec ou sans complément 2) demeure valide. L'exemple suivant illustre le principe général.

Exemple : Déterminer l'équivalent binaire de 0.75_{10} et de 1.25_{10}. Puis faire la somme de ces deux nombres en binaire. Représentation binaire non signée.

À l'aide du tableau des puissances de 2 négatives, la décomposition s'obtient directement (pour les cas plus complexes, on reprendra la méthode des « soustractions successives » introduite précédemment) :

Équivalent binaire	Puissance de 2	Équivalent décimal
0.1	2^{-1}	0.5
0.01	2^{-2}	0.25
0.001	2^{-3}	0.125

Figure1.16

On décompose $(0.75 = 0.5 + 0.25)_{10}$, ce qui donne l'équivalent binaire (selon le tableau précédent) : $(0.11)_2$

On décompose $(1.25 = 1 + 0.25)_{10}$ ce qui donne l'équivalent binaire (selon le tableau et les méthodes vues précédemment pour les nombres entiers) : (1.01).

Donc :

$(0.75 + 1.25)_{10}$ $= 2_{10}$

$(0.5 + 0.25)$

$\boxed{0.11}$ et $\boxed{1.01}$

Et l'addition binaire :

$$\begin{array}{rr} & 1.01 \\ \text{et} & +\ \underline{0.11} \\ & 10.00 \end{array}$$

ce qui correspond à $+2_{10}$.

Le résultat est évidemment correct.

Nous allons maintenant constater que, puisque le point décimal est fixe ici (sa position étant définie selon les puissances de 2 positives et négatives), le cas des soustractions binaires se résout comme pour les nombres entiers, en utilisant les compléments 2. L'exemple suivant illustre l'approche à suivre :

Exemple : Calculer $(1.25 - 0.75)_{10}$.

$(1.25)_{10}$ correspond à $(1.01)_2$ selon ce que nous avons vu à l'exemple précédent. $(0.75)_{10}$ correspond à $(0.11)_2$ selon ce que nous avons vu à l'exemple précédent. En choisissant une représentation sur quatre bits $(00.11)_2$ pour éviter les débordements (ou dépassements, *overflow*), on obtient le complément 2, tel que vu précédemment, par inversion de tous les bits $(11.00)_2$ puis addition de 1 ce qui donne : $(11.01)_2$. Le calcul se termine par l'addition de ces deux nombres :

$$\begin{array}{lll} -0.75_{10} \text{ en C2} & \Rightarrow & 11.01 \\ +1.25 & \Rightarrow & +\ \underline{01.01} \\ & & \cancel{1}00.10 \end{array} = (0.5)_{10} \text{ qui est correct.}$$

Comme notre notation est sur 4 bits, on rejette donc le cinquième bit.

1.13 CODES ET ENCODAGE

Un code est une représentation d'une valeur sous une autre forme. Par exemple, le code morse employé par les télégraphes utilise deux symboles (barre et point) pour représenter les lettres de l'alphabet. Le code « SOS » (*Save Our Ship*, Sauvez notre bateau) bien connu se représente ainsi dans un appel d'urgence : « . . . - - - . . . » (en morse, S est représenté par trois points, et O est représenté par trois barres). Il est intéressant de préciser que le code morse a été employé de 1840 à 2000 année où celui-ci a été aboli dans les télécommunications officielles. Le code morse demeure cependant toujours employé par les radioamateurs qui le trouvent très pratique puisque son immunité aux perturbations électromagnétique est bonne.

Les codes à barres, dont le code UPC (*Universal Product Code*), sont très employés. Leur lecture optique combinée avec la facturation, la gestion des inventaires, etc. par ordinateur en a répandu l'usage sur à peu près tous les produits.

Les partitions de musique illustrent une autre forme de codage où les notes de musique correspondent un peu aux lettres qui représentent les mots que nous écrivons.

CODE ASCII

Un code très important dans le monde numérique est le code ASCII (*American Standard Code for Information Interchange* ou Code américain standard pour l'échange d'information). Ce code permet de représenter en binaire les lettres (de a à z, et de A à Z), les chiffres (de 0 à 9), les signes de ponctuation et des commandes (tels que saut de ligne) : voir les figures 1.17 et 1.18. Dans sa version de base, 7 bits (B_6 à B_0) sont employés pour représenter $2^7 = 128$ caractères (codes de 0 à 127). Dans sa version étendue, un huitième bit est rajouté pour le support des accents (codes de 128 à 255, bits : B_7 à B_0). Les tableaux suivants dressent la liste de ces représentations. Un aspect intéressant du code ASCII est que les chiffres y sont encodés en *BCD* sur 7 bits. Pour récupérer le nombre *BCD*, il suffit alors simplement de mettre à zéro tous les bits, sauf B_3-B_0

Exemples d'utilisation des codes ASCII :

1. « 9_{10} » en ASCII donne $011\ 1001_2$ (39_{hex}).

   Si on ne garde que les 4 bits les moins significatifs (B_3-B_0), on obtient 1001_2, soit 9 en BCD.

2. « 7_{10} » en ASCII donne $011\ 0111_2$ (37_{hex})

   Si on ne garde que les 4 bits les moins significatifs (B_3-B_0), on obtient 0111_2, soit 7 en BCD.

3. « 97_{10} » en ASCII est représenté dans la mémoire d'un ordinateur par deux octets, 39_{hex} et 37_{hex}. Selon les exemples précédents, on obtient alors immédiatement :
   $39_{hex} = 0011\ 1001_2 \rightarrow 9$ en *BCD*
   $37_{hex} = 0011\ 0111_2 \rightarrow 7$ en *BCD*

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	00	Nul	32	20	Space	64	40	@	96	60	`
1	01	Start of heading	33	21	!	65	41	A	97	61	a
2	02	Start of text	34	22	"	66	42	B	98	62	b
3	03	End of text	35	23	#	67	43	C	99	63	c
4	04	End of transmit	36	24	$	68	44	D	100	64	d
5	05	Enquiry	37	25	%	69	45	E	101	65	e
6	06	Acknowledge	38	26	&	70	46	F	102	66	f
7	07	Audible bell	39	27	'	71	47	G	103	67	g
8	08	Backspace	40	28	(	72	48	H	104	68	h
9	09	Horizontal tab	41	29	)	73	49	I	105	69	i
10	0A	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	47	2F	/	79	4F	O	111	6F	o
16	10	Data link escape	48	30	0	80	50	P	112	70	p
17	11	Device control 1	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	50	32	2	82	52	R	114	72	r
19	13	Device control 3	51	33	3	83	53	S	115	73	s
20	14	Device control 4	52	34	4	84	54	T	116	74	t
21	15	Neg. acknowledge	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	End trans. block	55	37	7	87	57	W	119	77	w
24	18	Cancel	56	38	8	88	58	X	120	78	x
25	19	End of medium	57	39	9	89	59	Y	121	79	y
26	1A	Substitution	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	60	3C	<	92	5C	\	124	7C	|
29	1D	Group separator	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	63	3F	?	95	5F	_	127	7F	⌂

Figure 1.17 Tableau des codes ASCII de base (exprimé sur 7 bits : B_6, B_5, B_4, B_3, B_2, B_1, B_0)[13].

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
128	80	Ç	160	A0	á	192	C0	└	224	E0	α
129	81	ü	161	A1	í	193	C1	┴	225	E1	ß
130	82	é	162	A2	ó	194	C2	┬	226	E2	Γ
131	83	â	163	A3	ú	195	C3	├	227	E3	π
132	84	ä	164	A4	ñ	196	C4	─	228	E4	Σ
133	85	à	165	A5	Ñ	197	C5	┼	229	E5	σ
134	86	å	166	A6	ª	198	C6	╞	230	E6	µ
135	87	ç	167	A7	º	199	C7	╟	231	E7	τ
136	88	ê	168	A8	¿	200	C8	╚	232	E8	Φ
137	89	è	169	A9	⌐	201	C9	╔	233	E9	Θ
138	8A	è	170	AA	¬	202	CA	╩	234	EA	Ω
139	8B	ï	171	AB	½	203	CB	╦	235	EB	δ
140	8C	î	172	AC	¼	204	CC	╠	236	EC	∞
141	8D	ì	173	AD	¡	205	CD	═	237	ED	ø
142	8E	Ä	174	AE	«	206	CE	╬	238	EE	ε
143	8F	Å	175	AF	»	207	CF	╧	239	EF	∩
144	90	É	176	B0	▒	208	D0	╨	240	F0	≡
145	91	æ	177	B1	▓	209	D1	╤	241	F1	±
146	92	Æ	178	B2	▓	210	D2	╥	242	F2	≥
147	93	ô	179	B3	│	211	D3	╙	243	F3	≤
148	94	ö	180	B4	┤	212	D4	╘	244	F4	⌠
149	95	ò	181	B5	╡	213	D5	╒	245	F5	⌡
150	96	û	182	B6	╢	214	D6	╓	246	F6	÷
151	97	ù	183	B7	╖	215	D7	╫	247	F7	≈
152	98	ÿ	184	B8	╕	216	D8	╪	248	F8	°
153	99	Ö	185	B9	╣	217	D9	┘	249	F9	∙
154	9A	Ü	186	BA	║	218	DA	┌	250	FA	·
155	9B	¢	187	BB	╗	219	DB	█	251	FB	√
156	9C	£	188	BC	╝	220	DC	▄	252	FC	ⁿ
157	9D	¥	189	BD	╜	221	DD	▌	253	FD	²
158	9E	₧	190	BE	╛	222	DE	▐	254	FE	■
159	9F	ƒ	191	BF	┐	223	DF	▀	255	FF	

Figure 1.18 Tableau des codes ASCII étendus (exprimé sur 8 bits : B_7, B_6, B_5, B_4, B_3, B_2, B_1, B_0).

[13] Source : http://www.cdrummond.qc.ca/cegep/informat/Professeurs/Alain/files/ascii.htm
Voir aussi : http://www.lookuptables.com/.

CARACTÉRISTIQUES D'UN BON CODE

La figure 1.19 illustre l'utilisation du codage. Étudions maintenant ses caractéristiques importantes.

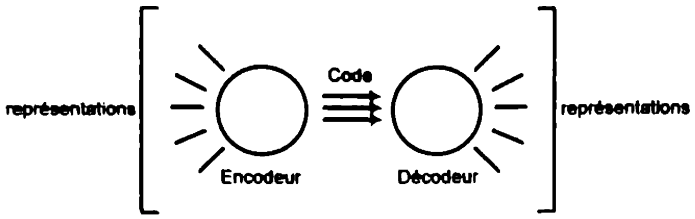


Figure 1.19

Il y a plusieurs caractéristiques qui déterminent la qualité d'un code. D'abord, s'il est *standard*, c'est-à-dire *adopté par le plus grand nombre*, donc universel. Qu'on se souvienne de la « guerre féroce » entre les standards Betamax ou simplement Beta (supporté par Sony) et VHS (supporté par JVC et RCA) pour les magnétoscopes (période 1976 à 2002). Même si le standard Beta lui était techniquement supérieur[14], c'est finalement le standard VHS qui l'a emporté à cause de sa plus grande popularité, reléguant de nos jour le Beta aux oubliettes. La même histoire s'est répétée au tournant des années 2000 lorsque les standards HD-DVD et le BlueRay se sont affrontés sur le marché des disques optiques pour emmagasiner des données et de la vidéo haute définition. Le BlueRay (supporté par Sony et Philips) a finalement mis le HD-DVD (lancé par Toshiba) K.O. en 2008!

Une autre caractéristique désirable est la *rapidité* pour encoder et décoder. Le code BCD est un bon exemple. Il est fort employé dans les calculatrices, car la conversion de BCD en décimal (et vice versa) est *instantanée*, ce qui n'est pas le cas de l'encodage binaire qui requiert quelques calculs. Puisqu'une calculatrice doit constamment interfacer avec l'utilisatrice, il est donc très approprié. Bien sûr, la rapidité de conversion dépend aussi du contexte. Songeons à la circulation des numéros de cartes de crédit sur Internet. Dans ce cas, la confidentialité prime et c'est pour cela que les clés de cryptage à 128 bits sont employées.

La *fiabilité* d'un code est aussi essentielle. Par fiabilité, on entend ici l'immunité aux erreurs lors de la manipulation des valeurs encodées. Certains codes incorporent des mécanismes de sécurité, ce qui les rend justement plus robustes. Nous avons déjà vu le code Gray pour lequel l'incrémentation d'une valeur à une autre donne lieu à un changement maximal d'un bit à la fois. Ce qui n'est pas le cas du codage binaire comme nous l'avions déjà fait remarquer précédemment.

La *compression*, c'est-à-dire le nombre de bits requis pour l'encodage, est aussi un aspect essentiel dans certaines applications. En ne considérant que cet aspect, le codage binaire d'un nombre décimal (inférieur à 100) est supérieur au codage BCD puisque 99_{10} s'exprime avec huit bits en BCD $(1001\ 1001)_2$, mais seulement avec sept en binaire

[14] Et en fait, le marché Beta existe toujours, on peut continuer à se procurer cassettes et lecteurs sur les sites de vente en ligne (ebay.com, etc.)!

$(110\ 0011)_2$. Le codage MP3 pour les enregistrements audio est un autre bon exemple de compression des données.

Finalement, la *sécurité* d'un code, donc sa capacité à conserver le caractère confidentiel des données est un autre aspect qui est à considérer, selon le contexte, telles les données bancaires ou fiscales par exemple. Des codes de cryptage existent pour ces applications.

1.14 PORTES LOGIQUES

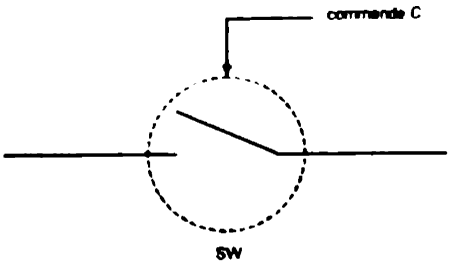
Les portes logiques sont en fait des circuits électroniques (constitués de transistors) dont l'état de sortie binaire (0 ou 1) dépend de la valeur des signaux à son (ses) entrée(s) (voir la figure 1.20). En pratique, les portes logiques sont directement implantées dans des circuits intégrés en silicium. La technologie moderne de fabrication permet d'implanter des millions de portes logiques qui peuvent commuter (c'est-à-dire changer d'état 0,1) des millions de fois par seconde tout en consommant très peu d'énergie. Les portes logiques sont, comme nous le verront au fil des chapitres de ce livre, à la base de tous les ordinateurs modernes (microprocesseurs, microcontrôleurs). Rappelons que ce livre n'est pas concerné par l'étude de l'électronique nous n'élaborerons donc pas l'étude des circuits électroniques sous-jacents. Pour les personnes intéressées, une excellente référence à ce sujet est l'ouvrage *The Art of Electronics* par Paul Horowitz et Winfield Hill, 3e édition, Cambridge, University Press, 2015.

Figure 1.20

De façon à illustrer le fonctionnement des portes logiques, on peut expliquer leur fonctionnement à l'aide d'interrupteurs, qui, comme on le sait, laissent passer ou non le courant électrique. Si un interrupteur est relié à une ampoule par un circuit électrique, tel celui d'une lampe de poche, l'ampoule sera allumée ou éteinte. On a donc ici deux états : 0 (éteint) et 1 (allumé). Dans le cas d'une lampe de poche, c'est l'utilisateur qui bascule l'interrupteur. Dans le cas d'une porte logique, c'est le ou les signaux d'entrée qui réalisent cette opération. Par ailleurs, la pile de la lampe de poche est la source d'énergie qui allumera l'ampoule. Dans une porte logique, la source d'énergie est l'alimentation électrique du circuit.

Une distinction très importante entre un interrupteur « normal » (celui du mur qui allume le plafonnier de la pièce) et un interrupteur numérique est la capacité de l'interrupteur numérique de commander un autre interrupteur numérique, ce qui permet un effet en cascade par l'interconnexion de plusieurs interrupteurs numériques les uns aux autres.

Donc, pour un interrupteur numérique, le schéma général est le suivant :



Si C > *seuil* ⇒ interrupteur SW fermé

Si C < *seuil* ⇒ interrupteur SW ouvert

où *seuil* est une tension qui dépend de la circuiterie employée (on reparlera de ceci plus loin).

Les interrupteurs numériques sont fabriqués à partir de transistors.

Figure 1.21

Une PORTE est donc un circuit contenant un ou plusieurs interrupteurs numériques et qui possède une fonction particulière. Plusieurs portes sont généralement assemblées entre elles afin de réaliser un système plus complexe tel que nous le verrons dans ce livre.

INVERSEUR : porte inverseuse ou porte NON

La porte inverseuse ou porte NON est la première que nous étudierons. Elle possède une seule entrée et une seule sortie. Sa fonction logique est simple, elle inverse le signal d'entrée *e*. Si le signal *e* est au 1 logique, le signal de sortie *s* sera au 0 logique, et inversement. Pour chaque porte étudiée, nous fournirons la table de vérité, le symbole logique ainsi que l'équation. Les symboles logiques serviront pour l'élaboration de circuits logiques complexes. Quant aux équations et aux tables de vérité, elles permettent de comprendre et d'analyser ces circuits.

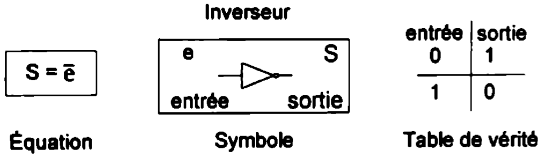


Figure 1.22

Un symbole très important est la « petite barre » au-dessus de la variable, petite barre qui dénote la négation. Dans certains ouvrages, l'apostrophe ou le point d'exclamation sont employés, ainsi $\overline{D} = !D = D'$.

TAMPON (*BUFFER*)

La porte « tampon » possède une table de vérité très simple : sa sortie est toujours égale à son entrée, $e = s$, et elle a une seule entrée et une seule sortie. Apparemment, cette porte ne sert donc à rien. D'un point de vue logique, effectivement ; par contre, d'un point de vue électrique, elle implante une isolation électrique et sert aussi à distribuer le signal logique à toutes les autres portes situées en aval (c'est ce qu'on appelle en anglais le *fan-out* ; cette expression imagée illustre très bien l'opération effectuée, c'est-à-dire la « ventilation extérieure maximale possible »). Une autre utilisation de la porte tampon est l'équilibrage des temps de propagation d'un circuit pour éviter les « courses » qui peuvent

survenir lorsqu'un signal arrive avant un autre. Cette dernière considération sera revue aux chapitres 3 et 5.

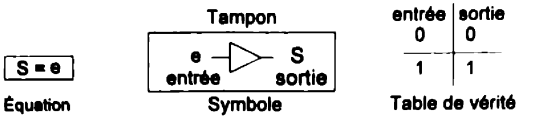


Figure 1.23

PORTE ET (*AND*) et NON-ET (*NAND*)

Les portes ET et NON-ET sont des portes à deux entrées (appelées ici *A* et *B*) et à une sortie *S*. Pour que la sortie d'une porte ET soit vraie (à 1), il faut que ses deux entrées soient vraies, donc que *A* ET *B* soient vraies. L'équation booléenne est en fait un produit booléen. Effectivement, $S = 1$ seulement si $A = 1$ ET $B = 1$ puisque $1 \cdot 1 = 1$. La porte NON-ET est simplement une porte ET suivie d'une porte inverseuse, comme l'examen de sa table de vérité le révèle. Dans l'équation de la porte NON-ET, on note la barre supérieure qui implante l'inversion. Cette notation est très importante et sera reprise dans les chapitres suivants.

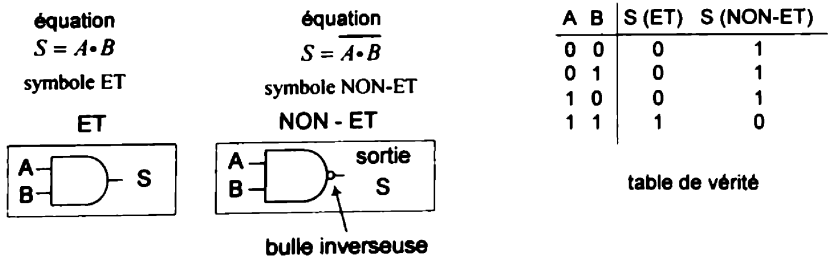
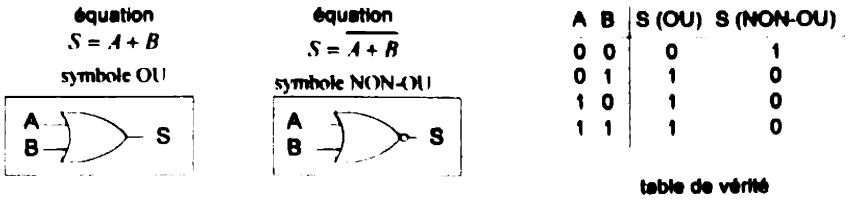


Figure 1.24

PORTE NON-OU (*NOR*)

Les portes OU et NON-OU sont des portes à deux entrées (appelées ici *A* et *B*) et à une sortie *S*. Pour que la sortie d'une porte OU soit à vraie (à 1), il faut que l'une des deux entrées soient vraies, *A* OU *B*. L'équation booléenne est en fait une somme booléenne (pas une somme arithmétique). Effectivement, $S = 1$ dès que $A = 1$ OU que $B = 1$ puisque $1 + 0 = 0 + 1 = 1 + 1 = 1$. *Important* : une *somme booléenne* fait l'association d'arguments qui sont VRAIS ou FAUX ; un résultat sera donc VRAI ou FAUX, mais jamais doublement VRAI (la somme booléenne de deux 1 donne 1, ainsi : $1 + 1 = 1$, le signe + étant ici une somme booléenne). La porte NON-OU est simplement une porte OU suivie d'une porte inverseuse, comme l'examen de sa table de vérité le révèle.



A	B	S (OU)	S (NON-OU)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

table de vérité

Figure 1.25

PORTES À ENTRÉES MULTIPLES

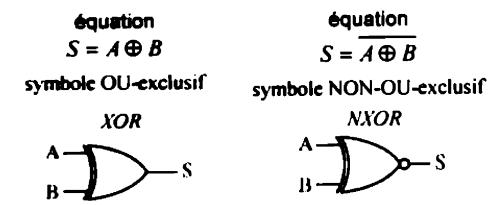
On peut étendre les concepts de portes ET, NON-ET, OU, NON-OU à des portes à plus d'une entrée, comme on l'illustre à la figure 1.26. Dans ces cas :

$S = A \cdot B \cdot C$ pour la porte ET triple, et $S = A + B + C$ pour la porte OU triple (addition booléenne et non pas arithmétique tel que discuté précédemment). A la figure suivante, on montre une porte ET triple et une porte OU quadruple.

Figure 1.26

PORTE OU-EXCLUSIF (*XOR*) et NON-OU-EXCLUSIF (*NXOR*)

Les portes OU-exclusif (*Exclusive OR*, appelées XOR) et NON-OU-exclusif (appelées NXOR) sont des portes à deux entrées (appelées ici A et B) et à une sortie S. On retrouve en fait la même table de vérité que pour la porte OU, à une seule différence près : le cas des deux entrées vraies ($A = B = 1$) est « exclu », et la sortie S est alors fausse ($S = 0$). Cette porte est plus complexe que les précédentes que nous venons d'étudier, mais peut être vue comme une combinaison de celles-ci, nous reviendrons là-dessus plus tard au chapitre 2. À noter, le symbole $\oplus$ dénotant le OU-exclusif dans les équations.



A	B	S(XOR)	S(NXOR)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

table de vérité

Figure 1.27

1.15 EXEMPLE DE DESIGN : ADDITION MODULAIRE

Ayant étudié les portes logiques simples, on peut les utiliser dans un premier exemple d'application, celui de l'addition modulaire. Notons que le projet que nous décrivons ici est propice à une réalisation pratique en laboratoire.

L'addition est un bon exemple d'une approche de design modulaire et donc réutilisable. L'idée consiste d'abord à réaliser un module simple pour l'addition de deux nombres d'un bit sans retenue. Nous appellerons ce module le *demi-additionneur*. Par la suite, l'addition de deux nombres d'un bit avec retenue s'obtient en employant deux demi-additionneurs, ce qui nous donnera le module d'*additionneur complet* d'un bit. Finalement, l'addition de nombres binaires quelconques (de n bits de long) s'obtient en juxtaposant simplement n additionneurs complets d'un bit côte à côte. L'avantage d'une telle approche modulaire est assez évident : si on veut additionner $n + 1$ bits, on ajoute simplement un module d'additionneur complet supplémentaire. Cette approche de design est à privilégier par rapport à l'approche du cas par cas qui consiste à repartir de zéro à chaque modification de spécification (comme ici lors du rajout d'un $(n + 1)^e$ bit supplémentaire).

L'addition de deux bits sans retenue est donc représentée par le module de demi-additionneur (appelé *Half-Adder* ou *HA* en anglais), dont la fonction apparaît ici :

A	B	Somme	Retenue
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

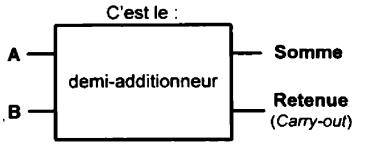


Figure 1.28

En observant cette table et aussi celle des portes ET et OU-exclusif étudiées précédemment, on note que la somme peut être réalisée avec une porte OU-exclusif (Somme = $A \oplus B$) et la retenue est alors obtenue avec une porte ET ($C_{out} = A \cdot B$) et, donc le circuit logique équivalent du *HA* est alors le suivant :

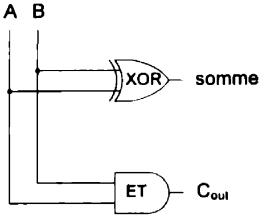


Figure 1.29 Schéma interne d'un demi-additionneur ou HA (*half-adder*) réalisé à partir de deux portes simples.

L'*additionneur complet sur un bit* (appelé *one bit Full adder* ou 1BFA), lui, implante la fonction suivante :

C_{in}	A	B	Somme	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

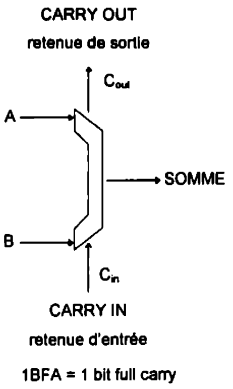


Figure 1.30

La réalisation du *1BFA* peut être vue sous l'angle de deux *HA* reliés un à l'autre à l'aide d'une porte OU. Nous verrons plus tard comment cet assemblage a été conçu. Pour l'instant, on constate seulement que le schéma fourni plus bas implante effectivement la fonction désirée.

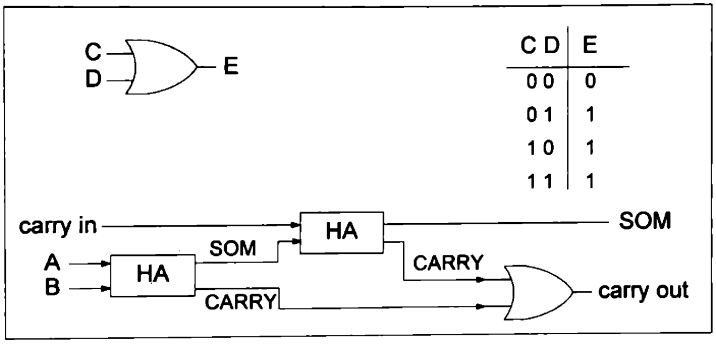


Figure 1.31 Schéma du 1BFA ou additionneur complet

Le montage se comprend bien à partir de la table du *IBFA* en considérant d'abord le cas avec C_{in} = 0 qui rend *transparent* le *HA* du haut et *inutile* la porte OU de sortie puisque le résultat du premier *HA* passe directement à la sortie *SOM* finale sans *CARRY*. Dans le cas où C_{in} = 1, le problème se décompose en deux étapes, comme dans le cas d'une addition manuelle de deux chiffres à la fois : somme partielle de (*A* + *B*) avec C_{in}.

Finalement, dans le cas de l'addition de nombres binaires quelconques (de *n* bits de long), on juxtapose simplement *n* *1BFA* puisqu'on a (en raisonnant sur 4 bits pour simplifier) : {*B*3, *B*2, *B*1, *B*0} + {*A*3, *A*2, *A*1, *A*0} = {Cout, *S*3, *S*2, *S*1, *S*0}. Ce qui est illustré sur le dessin suivant :

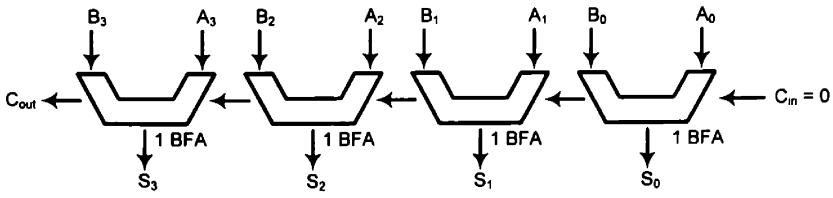


Figure 1.32

Telle qu'il a été indiqué plus haut, l'approche suivie ici est celle d'un bon design et est à privilégier, puisqu'elle est *modulaire* et qu'on a donc conçu un système à partir de sous-systèmes génériques. Cette approche requiert certainement plus de composantes qu'une approche directe, mais elle offre en contrepartie plusieurs avantages tels que ceux-ci :

- Conception plus simple puisque le problème de départ a été scindé en modules plus petits et donc plus aisés à concevoir ;
- Réparations éventuelles plus aisées, par exemple en interchangeant les modules pour déterminer l'origine de la panne ;
- Recyclage des modules dans d'autres projets.

À long terme, cette approche sera donc moins coûteuse.

Finalement, notons que la notation en complément 2 (C2) discutée plus haut permet d'employer les modules d'addition que nous venons d'étudier pour réaliser des soustractions. Cependant, il faut bien déterminer au préalable la longueur maximale des mots et procéder à l'obtention du nombre à soustraire en complément 2.

Exemple : Dans le cas de l'additionneur complet (*1BFA*), vérifier ce que donne l'addition de deux bits ($A = B = 1$) avec une retenue $C_{in} = 1$).

Solution : On reprend plus bas la figure du *1BFA* (figure 1.33). Le premier demi-additionneur (*HA*) fournit en sortie les valeurs suivantes : *SOM* = 0, et *Carry* = 1 (ce résultat est facilement obtenu à partir du circuit du demi-additionneur (discuté plus haut). Les entrées *A* et *B* du second demi-additionneur, sont respectivement ainsi : 1 et 0. Le second demi-additionneur nous renvoie alors à sa sortie : *SOM* = 1 et *Carry out* = 0. La porte OU du troisième étage réalise alors l'opération 1 (du premier demi-additionneur) +0 (du second demi-additionneur) ; la sortie du circuit complet est finalement obtenue : *SOM* = 1 et *Carry out* = 1, ce qui correspond bien à $(3)_{10} = (11)_2$. Tous les autres cas peuvent être vérifiés de la même manière.

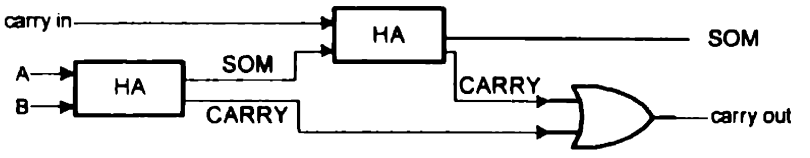


Figure 1.33

1.16 ANALYSE ET DESIGN DES CIRCUITS LOGIQUES

L'exemple précédent illustre un travail *d'analyse* de circuit. Lors d'une analyse, un circuit est disponible, et on cherche à déterminer son fonctionnement. La *conception* (communément appelée *design*) réalise l'opération inverse, à savoir, à partir d'une description (exprimée en mots, ou sous la forme d'une table de vérité), on détermine le circuit correspondant. La figure 1.34 illustre d'ailleurs la relation entre l'analyse et le design :

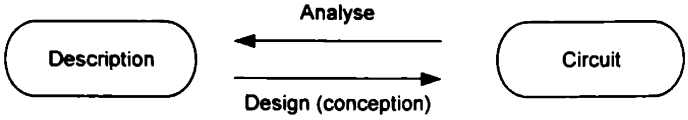


Figure 1.34

Un des objectifs de ce livre est d'ailleurs de présenter des méthodes systématiques d'analyse et de design. En effet, dans le cas du demi-additionneur étudié à la section précédente, la solution était très évidente et fut obtenue par simple inspection de la table de vérité des deux fonctions à implanter (celle de la *somme* et de la retenue C_{out}). Bien sûr, la solution n'est pas toujours aussi évidente. On verra justement dans les chapitres suivants des méthodes systématiques qui permettront « à coup sûr » d'analyser et de concevoir. Lors de la conception d'un circuit, on peut désirer optimiser, par exemple, la taille du design. Le recours à des méthodes rigoureuses nous garantira un résultat adéquat. Autrement, différents problèmes peuvent survenir.

Le circuit pourra ainsi être sensible aux aléas (appelé *glitches* en anglais). Les aléas sont des signaux transitoires qui peuvent être très néfastes dans les circuits numériques, car ils peuvent entraîner des pannes intermittentes difficiles à diagnostiquer (nous reviendrons d'ailleurs brièvement sur les aléas à la fin du chapitre 3).

La conception de petits circuits numériques peut se faire sur papier. Par contre, pour des circuits de plus grande importance, le travail devient vite fastidieux, et il vaut mieux alors recourir à des outils plus puissants tels les utilitaires schématiques (et leurs outils logiciels de support) ou les langages de description matérielle (*Hardware Description Language* ou HDL). Évidemment, un design important peut aussi être subdivisé en sous-blocs qui seront réalisés un par un sur papier. Il faut cependant noter que toute modification exigera un travail considérable.

Les utilitaires schématiques sont des logiciels où la conceptrice dessine son circuit à partir de fonctions prédéfinies (telles les portes logiques simples). Les fonctions sont d'abord invoquées, puis les connexions sont dessinées directement à l'écran.

Une description en langage HDL est en fait un fichier informatique qui décrit les entrées, les fonctions et les sorties du circuit numérique. Le VHDL (*VHSIC*[15] *Hardware Description Langage*), Verilog ou ABEL (*A Boolean Expression Language*) sont trois langages HDL communs. L'étude de ceux-ci dépasse le cadre de ce livre, par contre les méthodes vues ici sont utiles pour comprendre bien des aspects des langages HDL. Les langages HDL sont un exemple des méthodes de CAD (*Computer-Aided Design* ou design assisté par ordinateur) fort répandues en design électronique, numérique, mécanique, etc.

À titre d'exemple, il est intéressant de parcourir le fichier ABEL implantant le demi-additionneur ; celui-ci se trouve à la figure 1.35 (même sans connaître ABEL, on saisit ce que cela accomplit).

```
module demi_adi
Title 'demi_adi'
"Source ABEL implantant le circuit du demi-additionneur

Declarations

A PIN;
B PIN;
SOM PIN istype 'com';
COUT PIN istype 'com';

Equations

SOM = A $ B;   " operateur logique ou-exclusif
COUT = A & B; " operateur logique et

end demi_adi;
```

Figure 1.35 Source ABEL implantant le demi-additionneur
(*A* et *B* sont les entrées, *SOM* et *COUT* sont les sorties combinatoires).

En plus d'un temps de mise au point relativement rapide pour la réalisation de systèmes importants, une autre force de l'approche CAD est la possibilité qu'a la conceptrice de tester son design par simulation de celui-ci. Pour la simulation, il s'agit de fournir au logiciel une liste comprenant les entrées et les sorties (espérées). Le logiciel teste alors le circuit et signale par une erreur tout écart entre les sorties espérées et les sorties obtenues. La conceptrice doit alors revoir son circuit pour éliminer les erreurs. En plus des erreurs de logique, le simulateur signalera aussi d'autres problèmes tels les problèmes de synchronisation.

[15] Sigle anglais signifiant *Very High Speed Integrated Circuit* ou circuit intégré très rapide.

1.17 IMPLANTATION MATÉRIELLE DES CIRCUITS LOGIQUES

MONTAGES BREADBOARDS, WIRE-WRAPS *ET CIRCUITS IMPRIMES*

En terminant ce chapitre, il convient d'aborder très brièvement l'implantation matérielle des circuits logiques même si l'étude de «l'électronique » n'est pas le sujet de ce livre. Cette discussion sera utile pour le volet expérimental qui peut accompagner très judicieusement l'étude théorique présentée dans ces pages.

Les circuits logiques peuvent être implantés sous différentes formes. Traditionnellement, on a recours aux circuits intégrés de la famille TTL (de l'anglais : *Transistor Transistor Logic*). Cette famille, disponible en boîtiers DIP (*Dual in-line package*), a longtemps été populaire pour la fabrication des cartes électroniques (aussi appelées PCB, *Printed circuit board*, plaquette de circuit imprimé). De nos jours, les circuits en format DIP sont souvent remplacés par des circuits de format *surface mount* (montage de surface) moins coûteux à fixer sur les PCB. De plus, toutes les fonctions logiques sont généralement incorporées dans des modules à haute intégration de type VLSI (*Very Large Scale Integration*) sauf la « colle logique » (*glue logic*) qui sert souvent d'interface pour les modules principaux des PCB. La figure 1.36 montre des PCB réalisés avec des circuits en boîtiers DIP et *surface mount*.

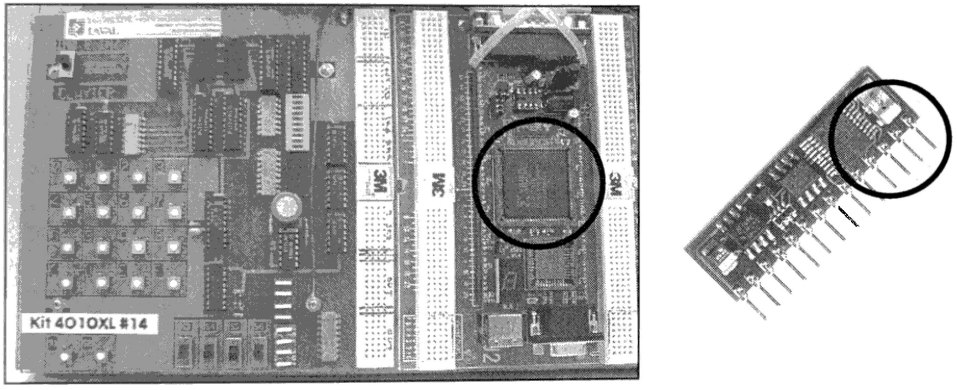


Figure 1.36 À gauche : circuit imprimé standard (plaquette qui sert pour les expériences de circuits logiques avec, encerclé, un FPGA Xilinx 4010 XL), et à droite : circuit imprimé standard avec montage de surface (il s'agit d'un *Basic stamp* de la compagnie Parallax avec, encerclé, le microcontrôleur PIC16C056)

Les boîtiers DIP ont un grand avantage à l'étape du prototypage d'un circuit puisque l'on peut réaliser celui-ci facilement, d'abord sur un *breadboard* (littéralement : planche à pain ou plaquette de montage) puis en *wire-wrap* (littéralement : bobinage de fil) avant de passer au PCB proprement dit.

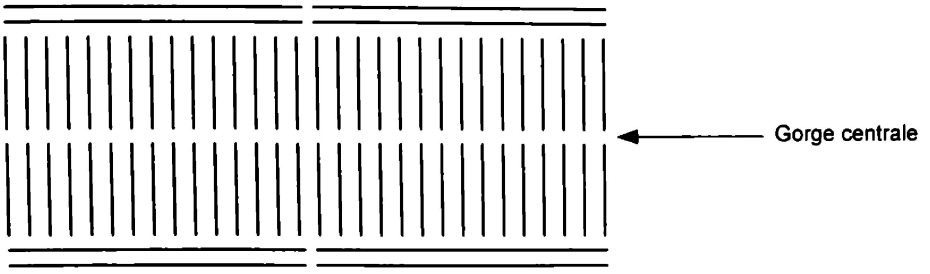


Figure 1.37 Schéma des connexions électriques d'un *breadboard*

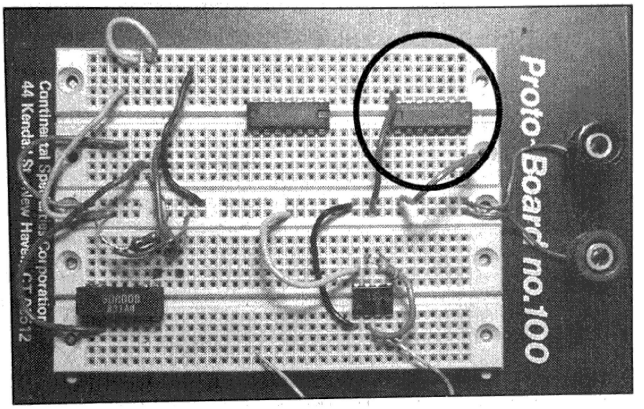


Figure 1.38 Photo de montages de circuits réalisés sur plaquette de montage (*breadboard*). Un 7400, encerclé, est visible en haut à droite avec son alimentation entre les broches 7 (la masse)et 14 (+ 5V).

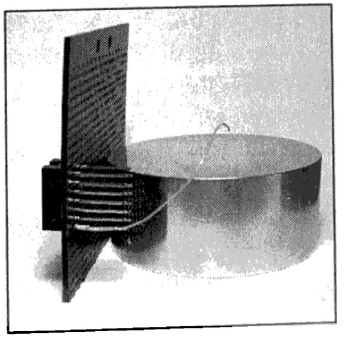


Figure 1.39 Principe du bobinage de fil (wire-wrap) : le fil montré est entouré sur une broche du réceptacle où l'on insérera le circuit intégré. Comme on le voit, ce réceptacle est monté sur une plaquette trouée pouvant aussi recevoir des résistances, des condensateurs, etc.

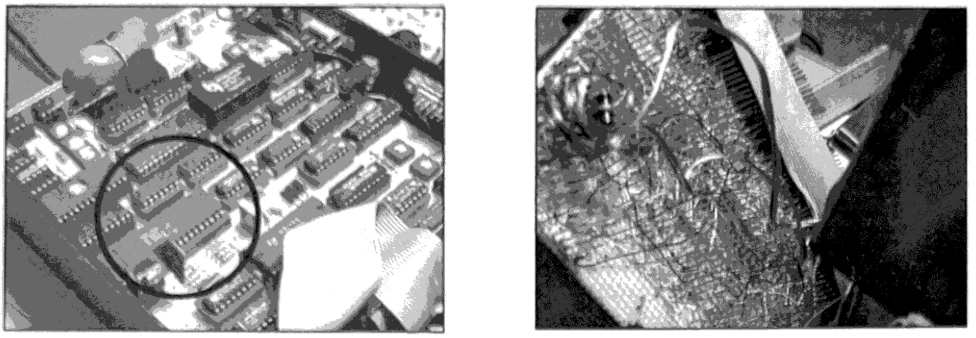


Figure 1.40 Photos d'un montage réalisé par bobinage de fil (*wire-wrap*) ; à gauche : vue du dessus du montage, et à droite : vue du dessous. Plusieurs circuits TTL 74 sont visibles à gauche (encerclés). Un microcontrôleur 8031 est aussi visible sur la gauche (encerclé).

Dans l'approche *breadboard*, les circuits intégrés (TTL), les autres composantes et les fils de connexion sont simplement enfichés dans les plots. Sur la photo de la figure 1.38, on voit que les circuits intégrés sont montés à cheval, de part et d'autre d'une gorge séparatrice (figure 1.37). De chaque côté (figure 1.38), on distingue cinq plots. Pour faciliter la réalisation des montages, ces plots sont reliés ensemble sous le *breadboard* tout en étant isolés électriquement des plots des autres rangées (figure 1.37). Ceci facilite bien sûr grandement les connexions. Dans l'approche *wire-wrap*, les composantes sont insérées dans une plaquette trouée, et les connexions se font par bobinage des fils de connexion sur leurs broches. La plaquette de montage (figure 1.39) est constituée d'un matériau isolant, comme de la bakélite par exemple.

Une fois la conception réalisée sur papier ou à l'aide d'un logiciel de CAD, l'étape suivante consiste à réaliser un prototype du circuit. Généralement, on vérifie d'abord le bon fonctionnement de celui-ci sur *breadboard* (ou en *wire-wrap*). Puisque les modifications se réalisent assez facilement à cette étape (surtout sur *breadboard*), les éventuelles altérations ne constituent pas un obstacle majeur. Par la suite, le circuit est monté sur PCB. Cette étape étant coûteuse (il faut réaliser le dessin de la plaquette et la plaquette elle-même), il est évidemment souhaitable de s'assurer au préalable que le circuit fonctionne correctement grâce au prototype sur *breadboard* (ou en *wire-wrap*). Une fois satisfait de la réalisation, on peut alors passer à l'étape de la production proprement dite.

Circuits logiques TTL

Selon leur complexité, c'est-à-dire le nombre de transistors nécessaires à leur implantation, on distingue les circuits intégrés *SSI*, *MSI* ou *LSI*, qui sont des sigles anglais signifiants respectivement *Small*, *Medium* ou *Large Scale Integration*. Les circuits *SSI* implantent jusqu'à environ 2-10 portes logiques, les circuits *MSI* jusqu'à 20-100 et les circuits *LSI* jusqu'à 200-1000 environ.

Parmi les circuits logiques *TTL SSI*, on retrouve les portes simples. Les circuits *TTL MSI* implantent des fonctions plus complexes tels les multiplexeurs (qui seront étudiés au chapitre 4). Enfin, les circuits *LSI* concernent des fonctions tels les *ALU* (*Arithmetic logic unit* ou unité d'arithmétique et de logique[16]), les multiplicateurs, etc. Traditionnellement, les circuits de la famille *TTL* fonctionnaient avec une alimentation électrique de 5 V (courant continu). Le 0 logique correspond alors à des tensions comprises entre 0 et 0.8 V ; le 1 logique, entre 2.0 et 5 Volt[17]. La plage des valeurs situées entre 0.8 et 2.0 V est une zone de tension qu'il faut éviter afin de ne créer aucune ambiguïté des niveaux logiques. Les circuits numériques actuels fonctionnent avec des alimentations plus basses (0.9 V, 1.2 V, 1.8 V, 2.4 V, 3.3 V) ce qui autorise des fonctionnements sur batteries convenant aux applications portables.

La famille *TTL* 74 à été employée par plusieurs générations de conceptrices. À l'origine, les circuits intégrés de cette famille étaient offerts en format DIP avec alimentation 5 V (broche 14 à +5 V et broche 7 à la masse). Les figures 1.41 et 1.42 illustrent d'ailleurs quelques détails de cette famille. Il est à noter que le chiffre employé fait référence à un numéro de fonction. Par exemple, un 7400 contient 4 portes ET à deux entrées, un 7404 contient 6 portes inverseuses. Dans ces deux cas, des boîtiers DIP de 14 broches sont employés.

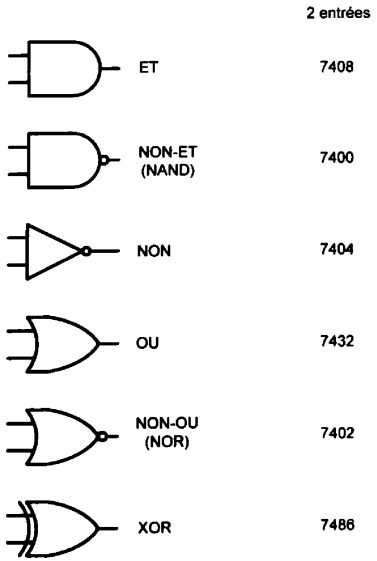


Figure 1.41 Portes simples et leur numéro de circuit intégré TTL.

[16] Un *ALU* est un circuit logique complexe qui réalise les opérations logiques (ET, OU, etc.) et arithmétiques (+, –, etc.).

[17] Pour la famille *Fast TTL Logic Series* de Philips. Le protocole de communication USB très répandu emploit une tension d'alimentation de 5 Volt.

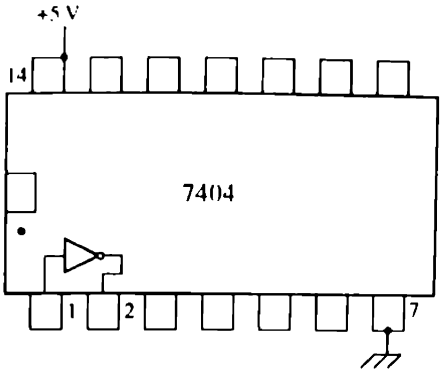


Figure 1.42 Connexion de l'alimentation électrique d'un circuit *TTL* 7404 (+ 5 V à la broche 14, masse à la broche 7). On montre aussi la localisation de la première porte inverseuse entre les broches 1 et 2.

Type de circuit[18]	Commentaires
PAL – *Programmable array logic* ou réseau logique programmable.	Implante un réseau ET (programmable) – OU (fixe) sera étudié au chapitre 4.
ROM – *Read only memory* ou mémoire en lecture seulement.	Implante un réseau ET (fixe) – OU (programmable) sera étudié au chapitre 4. Existe en différentes variantes : *PROM – Programmable ROM* (ROM programmable) *EROM – Erasable ROM* (ROM effaçable) *EEPROM - Electrically Erasable ROM* (ROM effaçable électriquement).
FPGA – *Field Programmable Gate Array* ou réseau de portes programmables localement.	On les configure sous forme de *CLB (Complex Logic Blocs* ou blocs de logiques complexes), d'éléments de mémoires (registres) et de zones d'interconnexions (pour relier les *CLB* entre eux). Les fonctions sont incorporées dans les *CLB*. La configuration du circuit est sauvegardée dans une mémoire externe et est chargée dans le *FPGA* à la mise sous tension. En cas de suspension de l'alimentation électrique, la configuration du *FPGA* est donc perdue.
CPLD – *Complex Programmable Logic Device* ou circuit logique programmable complexe.	Un CPLD est similaire à un FPGA puisqu'on peut y incorporer plusieurs fonctions logiques. Un point intéressant est que la configuration est sauvée dans de la mémoire non volatile, ainsi la configuration n'est pas perdue lors de la coupure de l'alimentation électrique. De plus, cette configuration peut être modifiée plusieurs fois. Spécifiquement, un CPLD est un arrangement de plusieurs PAL sur un circuit intégré.

Figure 1.43

Outre les applications de colle logique entre des blocs numériques d'importance et les applications pédagogiques, les circuits de la famille *TTL* ne sont plus vraiment employés. En effet, les conceptrices leur préfèrent des circuits intégrés programmables qui offrent une plus grande souplesse de configuration. Ces circuits de type *VLSI* contiennent des milliers de portes que la conceptrice (à l'aide de logiciels adaptés) peut associer entre elles pour réaliser les fonctions (complexes) désirées. Ces circuits sont offerts sous différentes formes, et certains sont configurables à la demande, ce qui est très pratique aux étapes

[18] Voir l'article : Brown, Stephen ; Rose Jonathan. « Architecture of FPGAs and CPLDs : A Tutorial », *IEEE Design and Test of Computers*, Vol. 13, nº 2, 1996, p. 42-57.

initiales de conception, pour la correction des erreurs éventuelles. Puisque des designs complets peuvent être incorporés dans un seul circuit, moins de place est nécessaire sur le circuit imprimé ce qui tend à réduire le coût global du système. Par ailleurs, les connexions internes et non externes accroissent évidemment la fiabilité générale de l'application. Le tableau suivant énumère quelques-uns des types de circuits programmables les plus communs. Ceux-ci seront discutés au chapitre 4. La figure 1.44 illustre le principe de l'utilisation de circuits programmables complexes pour incorporer des fonctions logiques.

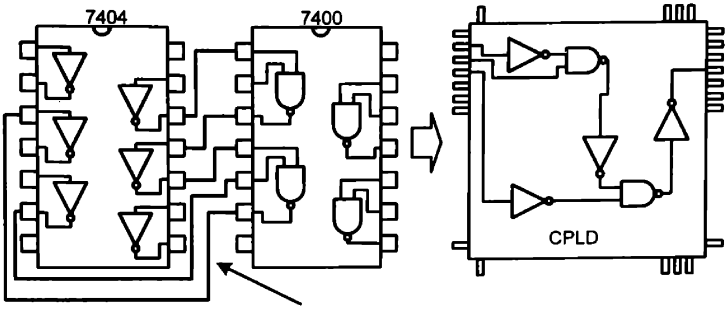


Figure 1.44 À gauche, fonction logique de quelques portes. À droite, on montre d'une façon conceptuelle comment cette fonction logique est incluse dans un seul circuit CPLD, ce qui évite les interconnexions externes pointées par la flèche[20].

Notons que la famille CMOS (de *National Semiconductor*) série 4000 implante à peu près les mêmes fonctions que la famille TTL série 7400, mais dans des circuits CMOS qui possèdent des caractéristiques particulières, soit une alimentation électrique variable (entre 3 V minimum et 15 V maximum) et surtout une consommation électrique très réduite. Par exemple, alors qu'un circuit de 4 portes NON-OU 74F02 (de type *Fast TTL Logic Series* de *Philips*) consomme 13 mA, un circuit CD4001BM ne consommera que 30 µA, soit environ 400 fois moins. Les applications nécessitant une consommation électrique réduite préféreront donc l'utilisation de circuits de type CMOS. Il est à noter que les familles 74 et 4000 ne font pas toujours bon ménage. Selon les types, il faudra employer des circuits d'interface spéciaux pour l'interconnexion. Notons cependant que la famille CD4000 précitée est compatible avec les circuits TTL de type *low power*.

À la section précédente, on a mentionné, parmi les méthodes CAD, les utilitaires schématiques. Parmi ceux-ci, les outils de la compagnie Xilinx sont très employés par exemple *Xilinx Foundation* (ou encore le *Xilinx Web Pack* disponible gratuitement sur le site Web www.xilinx.com). Dans ces outils, les fonctions de plusieurs circuits TTL se retrouvent sous l'appellation x74xxx où xxx correspond au numéro de fonction.

---

20 Source : http://www.interq.or.jp/japan/se-inoue/e_cpld1.htm (visité le 1er février 2005).

Un autre outil de CAD très convivial est le logiciel Logisim[21] qui est disponible gratuitement sous licence publique GNU et qui permet de simuler la plupart des montages de circuits logiques. Plusieurs écoles et universités emploient ce logiciel dans leur enseignement des circuits logiques.

Question coût, un circuit TTL pourra être obtenu pour quelque cent alors qu'un circuit FPGA nécessitera une sortie de fonds plus importante (jusqu'à plusieurs centaines de dollars pour les versions haut de gamme).

Dans ce chapitre, nous avons introduit le système de notation binaire, son arithmétique élémentaire, les portes logiques simples de même que des considérations d'implantation et de design. Les portes logiques simples que nous avons étudiées seront employées pour la réalisation de tous les circuits que nous verrons dans les autres chapitres.

PROBLÈMES RÉSOLUS

1.1. Effectuer les conversions suivantes :

a) $[1101011]_2$ en hexadécimal.

b) $[101111.0111]_2$ en octal.

c) $[FACE.BEBE]_{16}$ en binaire.

d) $[FACE.BEBE]_{16}$ en octal.

e) $[67.31]_8$ en décimal.

f) $[727]_{10}$ en binaire.

g) $[727]_{10}$ en hexadécimal.

Solutions :

Selon les concepts étudiés, on obtient directement :

a) En faisant des groupes de 4, on a : $\underbrace{0\,1\,1\,0}_{6}\underbrace{1\,0\,1\,1}_{B} = (6B)_{hex}$

b) En faisant des groupes de 3, on a : $\underbrace{1\,0\,1}_{5}\underbrace{1\,1\,1}_{7}.\underbrace{0\,1\,1}_{3}\underbrace{1\,0\,0}_{4} = (57.34)_8$

c) En faisant des groupes de 4, on a :

$$\underbrace{F}_{1111}\underbrace{A}_{1010}\underbrace{C}_{1100}\underbrace{E}_{1110}.\underbrace{B}_{1011}\underbrace{E}_{1110}\underbrace{B}_{1011}\underbrace{E}_{1110} = (\underbrace{1111}_{F}\underbrace{1010}_{A}\underbrace{1100}_{C}\underbrace{1110}_{E}.\underbrace{1011}_{B}\underbrace{1110}_{E}\underbrace{1011}_{B}\underbrace{1110}_{E})_2$$

d) On reforme des groupes de 3 à partir de la valeur binaire de l'exercice 1.1c) :

$$(\underbrace{1\,1\,1}_{7}\underbrace{1\,0\,1}_{5}\underbrace{0\,1\,1}_{3}\underbrace{0\,0\,1}_{1}\underbrace{1\,1\,0}_{6}.\underbrace{1\,0\,1}_{5}\underbrace{1\,1\,1}_{7}\underbrace{1\,0\,1}_{5}\underbrace{1\,1\,1}_{7})_2 = (175316.57537)_8$$

e) $(67.31)_8$ en décimal

[21] Logisim : http://www.cburch.com/logisim/

$= 6 \times 8^1 + 7 \times 8^0 + 3 \times 8^{-1} + 1 \times 8^{-2}$

$= 6 \times 8 + 7 + 3 \times 0.125 + 1 \times 0.015625$

$= (55.390625)_{10}$

f) et g) $(727)_{10}$ en binaire.

Employons la méthode de la division successive des plus grandes puissances de 16 :

727/256 = 2 reste 215

215/16 = 13 reste 7

donc $727 = 2 \times 256 + 13 \times 16 + 7$

et alors $727 = 2 \times 16^2 + D \times 16^1 + 7 \times 16^0$

Avec ce résultat, la réponse pour g) est obtenue :

$(727)_{10} = (2D7)_{16}$

En décomposant ce dernier résultat en blocs de 4 bits :

$(727)_{10} = (10\ 1101\ 0111)_b$, qui est la réponse pour f).

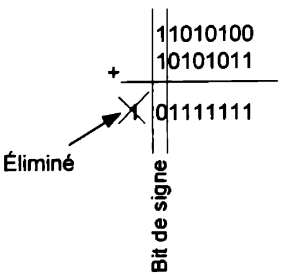
1.2. Pour chaque paire suivante de deux nombres (binaires signés de huit bits), spécifier si l'addition de ces deux nombres donnera ou non un débordement ; justifier la réponse ;

a) 11010100 et 10101011, débordement : OUI ou NON.

b) 10111001 et 11010110, débordement : OUI ou NON.

Solution : Avec huit bits, on note d'abord que les valeurs maximales s'étaleront de $-128_{10} = 1000\ 0000_b$ à $+127_{10} = 0111\ 1111_b$.

a)

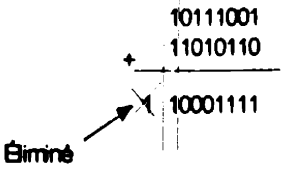


Deux nombres négatifs

On obtient un nombre positif : il y a donc DÉBORDEMENT (l'addition de deux nombres négatifs *doit* donner un nombre négatif.)

Par ailleurs, en se souvenant de la définition du bit OVR ($OVR = C_{out} \oplus SIG$), avec ici $C_{out} = 1$ et SIG = 0, $OVR = 1 \oplus 0 = 1$, il y a donc débordement. D'ailleurs, en décimal, l'opération consiste à additionner –44 à –85, ce qui donne –129, une valeur qui dépasse la capacité de codage des 8 bits signés que nous avons.

b)



Deux nombres négatifs

On obtient un nombre négatif : il n'y a donc pas de débordement *a priori*

Par ailleurs, en se souvenant de la définition du bit OVR ($OVR = C_{out} \oplus SIG$) avec ici $C_{out} = 1$ et $SIG = 1$, $OVR = 1 \oplus 1 = 0$, il n'y a donc pas de débordement. Par ailleurs, en vérifiant les valeurs binaires, on a que cette opération correspond en décimal à $-42 - 71 = -113$, valeur incluse dans la plage maximale de nos huit bits signés.

EXERCICES

1.1. Convertir le nombre binaire $(111\ 000)_2$ en un nombre décimal, octal et hexadécimal.

1.2. Convertir le nombre binaire $(1\ 101\ 101.11)_2$ en base octale et en base hexadécimale. Quelle est la valeur décimale ?

1.3. Convertir les nombres binaires : $(0.01)_2$ et $(101\ 101.0101)_2$ en base octale et en base hexadécimale. Quelle est la valeur décimale ?

1.4. Convertir le nombre décimal $(2005)_{10}$ en un nombre binaire, hexadécimal et octal.

1.5. Convertir le nombre décimal $(497)_{10}$ en un nombre BCD, octal, hexadécimal et binaire.

1.6. Convertir le nombre de base cinq $(2010)_5$ en un nombre binaire, hexadécimal, octal et décimal.

1.7. Convertir les nombres décimaux : $(250.72)_{10}$ et $(65\ 629.62)_{10}$ en base binaire, octale et hexadécimale.

a) Binaire. Se limiter au calcul des quatre premiers chiffres à droite du point.

b) Octale. Se limiter au calcul du premier chiffre à droite du point.

c) Hexadécimale. Se limiter au calcul du premier chiffre à droite du point.

1.8. Procéder à la soustraction de $32_{10} - 8_{10}$. Se servir de la méthode du complément 2 en binaire.

1.9. Additionner les paires de nombres suivants en binaire complément 2 sur cinq bits. Commenter brièvement le résultat obtenu.

a) -10_{10} et -8_{10}

b) -10_{10} et -2_{10}

1.10. Donner l'équivalent ASCII de :

a) Bonjour!

b) 2005_{BCD}

c) 2005_{10}

2

Table de vérité et algèbre de Boole

GEORGE BOOLE

George Boole[1], mathématicien et philosophe britannique (1815-1864). George Boole est l'inventeur de l'algèbre qui porte son nom, l'algèbre de Boole, si importante dans le « monde de l'électronique numérique », comme nous allons le voir dans ce chapitre. George Boole est issu d'une famille modeste. Son père, marchand à l'esprit très vivant, lui a donné ses premières leçons. Au cours de ses études primaires et secondaires, le jeune Boole était plutôt porté vers l'étude des classiques, et ce n'est qu'à la fin de ses études qu'il s'est vraiment intéressé aux mathématiques. Curieusement, sa vie nous apparaît aujourd'hui des plus linéaires, avec peu d'événements marquants outre sa nomination comme professeur de mathématiques à 34 ans au Queen College de Cork (Irlande), son mariage à 40 ans avec Mary Everest, qui d'ailleurs écrira plusieurs textes basés sur les travaux de son mari. Le caractère de G. Boole est décrit comme en étant un de génie modeste, et ses amis le tenaient en grande estime puisqu'il s'est toujours efforcé de faire triompher la vérité. Bien qu'il ait reçu une médaille de la Société Royale britannique pour son ouvrage *Philosophical Transactions* sur les mathématiques symboliques publié en 1844, et le titre honorifique de Docteur en loi de l'Université de Dublin (LL.D.), il n'a jamais obtenu les honneurs que ses découvertes auraient dû lui mériter.

G. Boole est connu pour ses publications et ouvrages mathématiques. Il fut aussi un des premiers mathématiciens à s'intéresser à la logique. Ses conceptions originales étaient liées à sa vision du raisonnement symbolique en mathématiques. Ses deux publications : *Mathematical Analysis of Logic* (1847) complétée en 1854 par une contribution plus importante : *An Investigation of the Laws of Thought, on which are founded the Mathematical Theories of Logic and Probabilities* définissent d'ailleurs sa conception de la logique qui reprend les mathématiques, mais avec seulement deux symboles 0 et 1. Par exemple, si x dénote « chèvre » et y dénote « cornu » alors la combinaison de ces deux symboles, (x et y), permet de dénoter toutes les choses cornues dans le monde, dont les chèvres. Ainsi, $(1 - y)$ représentera tout ce qui n'est pas cornu et $(1-x)\cdot(1-y)$ représentera

[1] http://en.wikipedia.org/wiki/George_Boole
http://www.bun.kyoto-u.ac.jp/phisci/Gallery/boole_note.html
http://www-gap.dcs.st-and.ac.uk/~history/PictDisplay/Boole.html

tout ce qui n'est ni chèvre, ni cornu. Comme on le voit, les opérations d'algèbre s'appliquent aux symboles. G. Boole étendra d'ailleurs cette conception aux propositions en général par l'inférence logique, ce qui permet d'obtenir une signification logique à partir d'un ensemble de propositions. Ainsi la phrase « Il pleut et George a pris son parapluie » pourra être déterminée comme vraie ou fausse selon la validité des propositions individuelles en appliquant les règles logiques avec a = « il pleut » et b = « George a pris son parapluie » puisqu'il suffit dès lors de dresser la table de vérité de la relation $a \cdot b$ pour en déterminer tous les états possibles.

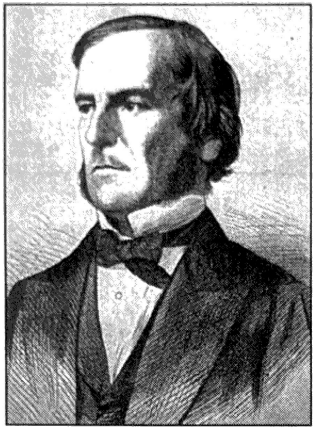


Figure 2.1 George Boole, un des « pères de la logique contemporaine ».
http://en.wikipedia.org/wiki/George_Boole

2.1 INTRODUCTION

Au chapitre 1, on a étudié le système binaire (deux états, 0 et 1). La question qui se pose maintenant est celle-ci : Comment, la relation entrée(s)/sortie(s) étant connue, peut-on réaliser le circuit logique correspondant ? La relation entrée(s)/sortie(s) peut être connue, par exemple avec une description écrite, une table de vérité (comme celles vues au chapitre 1). Deux situations peuvent alors se présenter :

1. Les sorties dépendent seulement des états présents aux entrées, et la sortie pourra changer *dès* que surviendra un changement d'entrée ; on parle alors de *circuits combinatoires*, qui seront étudiés aux chapitres 2, 3 et 4. Un exemple de circuit combinatoire est le circuit utilisé pour les afficheurs à sept segments (figure 2.2). L'entrée est la valeur du nombre BCD $\{d_0,d_1,d_2,d_3\}$ à afficher (de 0000 pour le chiffre « 0 » à 1001 pour le chiffre « 9 ») et la sortie, les valeurs associées à chacun des segments a-b-c-d-e-f-g. On aura ainsi respectivement, si on affiche 0 et 9 : 1-1-1-1-1-1-0 et 1-1-1-1-0-1-1 pour les valeurs a-b-c-d-e-f-g. Bien évidemment, dès qu'une des entrées $\{d_0,d_1,d_2,d_3\}$ change, l'affichage doit être immédiatement modifié en conséquence ; c'est donc un circuit sans mémoire (comme tous les circuits combinatoires d'ailleurs).

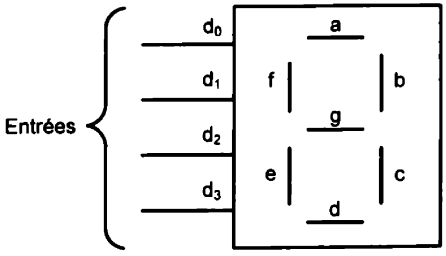


Figure 2.2 Exemple de circuit combinatoire : afficheur à sept segments.

2. Les sorties dépendent de l'historique des entrées, ce qui veut dire qu'à un instant t donné, la sortie du circuit dépend des entrées présentes, mais aussi de l'état dans lequel le circuit se trouve à ce moment ; on parle alors de *circuits séquentiels*. Un exemple d'un tel circuit est le contrôleur d'une machine distributrice. Dans ce cas, la connaissance de l'historique des entrées (les pièces insérées par la cliente et le choix de celle-ci) est primordiale et détermine la suite des opérations : attente de l'insertion d'autres pièces de monnaie, éjection du produit payé et sélectionné (on réalisera d'ailleurs le contrôleur pour une telle machine au chapitre 7). Ces circuits possèdent donc une mémoire de leur état actuel et sont souvent représentés par un diagramme d'états (*state diagram*). Ils seront étudiés à partir du chapitre 5.

Au chapitre 2, nous allons nous attarder à l'algèbre de Boole et aux tables de vérité qui seront nécessaires pour l'analyse et la synthèse des circuits combinatoires et séquentiels. Plus précisément, voici les deux idées maîtresses du chapitre 2 :

1. Une fois qu'une fonction quelconque [entrées–sorties] a été définie par une table de vérité, on peut directement en obtenir le circuit équivalent en identifiant les termes à 1 en sortie et en combinant ensuite ceux-ci ensemble. Cela nous donnera un circuit à deux niveaux (ou à deux couches). Le premier niveau sera représenté par une porte ET (ce niveau correspond aux termes à 1 en sortie dans la table de vérité). Le deuxième niveau est obtenu avec une porte OU réunissant les termes du premier niveau. On obtiendra alors un circuit de type *Somme de Produits* (SdP)[2].

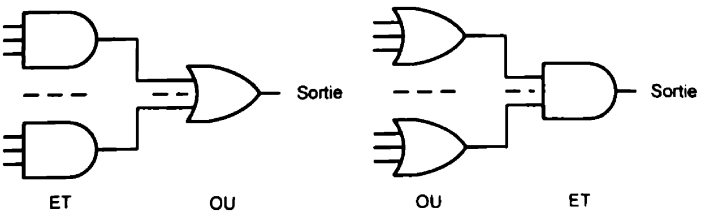


Figure 2.3 Illustration d'un circuit à deux niveaux réalisé selon deux topologies : *Somme de Produits* (SdP) à gauche et *Produit de Sommes* (PdS) à droite.

[2] Le sigle SdP indique la topologie du circuit, le OU faisant référence à la somme booléenne et les ET aux produits. On peut de même réaliser un circuit avec la topologie de type *Produit de sommes* (PdS) en s'intéressant aux termes à 0 dans la table de vérité. Dans ce cas, le premier niveau est représenté par une porte OU et correspond aux termes à 0 en sortie dans la table de vérité. Le deuxième niveau est obtenu avec une porte ET réunissant les termes du premier niveau. Les deux topologies sont illustrées à la figure 2.3. Elles seront étudiées en détail plus bas.

2. La démarche effectuée au point 1 revient en fait à écrire l'équation booléenne du circuit. Cependant, le circuit obtenu de cette démarche ne sera pas optimal pour ce qui est du nombre de portes requises. Différentes méthodes que nous étudierons permettront de réduire au minimum ce nombre. Une de celles-ci consiste à appliquer les théorèmes de l'algèbre de Boole pour simplifier l'équation obtenue et donc minimiser le nombre de portes nécessaire à sa réalisation.

2.2 TABLE DE VÉRITÉ (TDV)

Revenons maintenant au concept de table de vérité introduit brièvement au chapitre 1. Une table de vérité définit les relations entrée(s)/sortie(s) en faisant la liste de toutes les combinaisons possibles ; chaque combinaison représentant une ligne dans la table. Une table de vérité contient 2^n lignes où n est le nombre d'entrées. Ceci vient directement du fait que l'on emploie un système binaire à deux états. Ainsi, une entrée pourra avoir la valeur 0 ou 1 ($2^1 = 2$). Évidemment, on peut avoir plus d'une sortie. Ainsi, la petite table de vérité, pour une fonction à une entrée et une sortie, aura 2 lignes puisque $2^1 = 2$ et deux colonnes (une pour l'entrée et une pour la sortie). Les tables de vérité des tampons et des inverseurs (vues au chapitre 1) sont des exemples de telles tables. Si on a maintenant à dresser la table de vérité pour un circuit logique ayant n entrées et m sorties, on aura une table ayant 2^n lignes et $(n + m)$ colonnes. Il n'y a bien sûr aucune limite supérieure à la taille des tables de vérité, cela dépend uniquement des fonctions logiques à implanter.

Une notion importante concernant les tables de vérité est le « rang » de chaque ligne. Le rang est en fait le nombre binaire correspondant au numéro de la ligne en question. La première ligne aura le rang 0, la seconde, le rang 1, et ainsi de suite jusqu'à la dernière qui aura le rang $2^n - 1$.

Exemple : Dresser la table de vérité pour le segment « a » de l'afficheur à sept segments dessiné précédemment. L'entrée du circuit est un chiffre BCD sur quatre bits et les variables d'entrée sont : d_0, d_1, d_2, d_3 avec d_3, la variable la plus significative.

Solution : Il y a donc quatre entrées dans notre table de vérité, mais seulement 10 lignes (sur un maximum de $2^4 = 16$ puisqu'un chiffre BCD s'étend de 0 à 9. Le rang des lignes sera 0 à 9. La figure 2.4 nous donne les valeurs du segment « a » pour tous les chiffres :

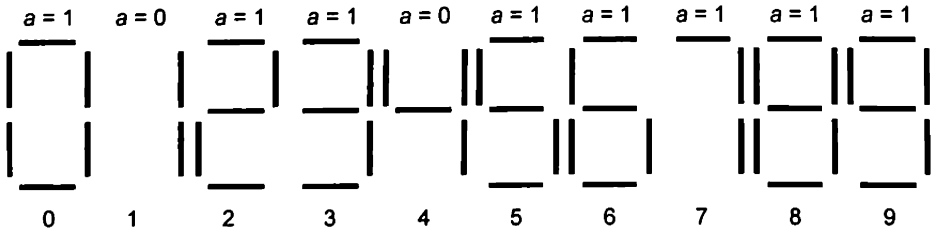


Figure 2.4

Ce qui nous permet de dresser immédiatement la table de vérité correspondante :

Rang	d_3	d_2	d_1	d_0	Sortie a
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	1

Table de vérité avec en entrées d_3, d_2, d_1, d_0

Rang	d_0	d_1	d_2	d_3	Sortie a
0	0	0	0	0	1
1	1	0	0	0	0
2	0	1	0	0	1
3	1	1	0	0	1
4	0	0	1	0	0
5	1	0	1	0	1
6	0	1	1	0	1
7	1	1	1	0	1
8	0	0	0	1	1
9	1	0	0	1	1

Table de vérité avec en entrées d_0, d_1, d_2, d_3

Figure 2.5

À noter que nous avons ici écrit la table avec la liste des variables d'entrée de la plus significative à la moins significative, donc de d_3 à d_0 et l'inverse de d_0 à d_3. Les deux formes sont acceptables : attention cependant de ne pas se tromper dans les valeurs binaires des entrées.

Important : la forme de gauche sera celle que nous emploierons toujours, sauf pour l'étude des registres à décalage au chapitre 6. Pour les deux tables, le rang de chaque ligne est aussi indiqué, il correspond à la valeur binaire des entrées correspondantes dans la table.

Exemple : Quelle est la taille de la table de vérité pour le circuit implantant une table de multiplication de deux chiffres de huit bits.

Solution : On a deux chiffres de huit bits, donc 16 entrées, ce qui implique une table de $2^{16} = 65356$ lignes. Quant au nombre de colonnes, il y en aura 16 pour les entrées (deux nombres de huit bits) et en sortie, on sait qu'un produit requiert deux fois plus de bits que le format des nombres que l'on multiplie, on aura donc 16 bits (8 + 8) en sortie. Au total (entrées + sorties), il y aura donc 32 colonnes (16 + 16) et 65356 lignes dans la table (ce qui exigerait 820 pages, en assumant 80 lignes par page).

Exemple : Considérons l'exemple de l'afficheur à sept segments pour le chiffre le plus significatif des multimètres numériques et dressons la table de vérité correspondante. Pour rappel, dans le cas du chiffre le plus significatif, seul la valeur « 0 » ou « 1 » est affichée tel qu'indiqué à la figure 2.6 :

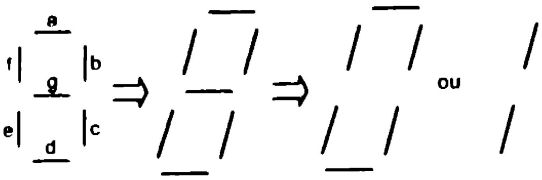


Figure 2.6

Solution : Si l'Entrée est 0, on affiche 0 ; si l'Entrée est 1, on affiche 1.

La table de vérité s'obtient alors directement :

Entrée	Sorties						
	a	*b*	*c*	*d*	*e*	*f*	*g*
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0

Figure 2.7

À noter que, dans la famille des circuits TTL, la fonction de décodeurs à sept segments existe toute faite. Par exemple, le circuit TTL 7448 prend comme entrée un chiffre binaire exprimé sur 4 bits $\{d_0,d_1,d_2,d_3\}$ et a comme sortie les valeurs assignées à chacun des sept segments (le 7448 est en fait un décodeur BCD-7 segments puisque les valeurs d'entrées supérieures à 9 ne sont pas considérées).

2.3 PRIMITIVES TOUTES FAITES ET APPROCHE PAR MÉMORISATION DE LA TABLE DE VÉRITÉ

Lorsqu'on doit réaliser un circuit logique, la première démarche consiste à vérifier si ce dont on a besoin n'existe pas déjà, il est en effet inutile de « réinventer la roue ». Une approche consiste donc à recourir à la famille 74xx qui comprend l'ensemble des fonctions logiques élémentaires. Il est à noter que la famille 74xx existe sous différentes formes : boîtiers TTL (avec une tension d'alimentation à 5 V), boîtiers CMOS (série de circuits 4000 à la plage de tension plus étendue, jusqu'à 20 V selon les versions), mais aussi sous forme de librairies de fonctions lorsque l'on choisit d'implanter son design dans un circuit VLSI, tel un circuit programmable de type FPGA ou CPLD (par exemple, le logiciel WebPack de Xilinx, disponible sur Internet[3] contient les fonctions les plus communes avec le préfixe x74). Il est donc utile de présenter très brièvement les fonctions 74xx les plus communes.

[3] www.xilinx.com

La figure 2.8 fournit les numéros des fonctions 74xx les plus usuelles.

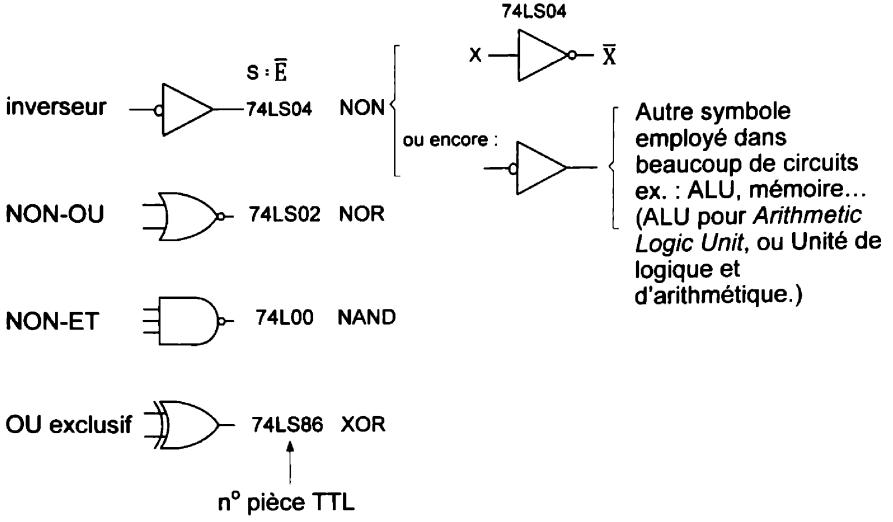


Figure 2.8 Numéros de fonctions 74xx les plus usuelles.

Comme on l'a vu au chapitre 1, les portes NON-ET et NON-OU incorporent un inverseur, elles sont donc des *blocs de construction* très utiles.

Notons aussi qu'une autre approche, de type « force brute », pour réaliser un circuit logique donné, consisterait à mémoriser la table de vérité correspondante dans un circuit de mémoire. C'est une approche possible, mais si coûteuse en matière de circuiterie nécessaire qu'elle n'est que rarement employée. Par exemple, si on reprend l'exemple de l'afficheur du « 1 » dans un multimètre, on pourrait loger cette fonction dans une mémoire de 14 bits (deux adresses et sept sorties, donc $7 \times 2 = 14$ bits). Si l'entrée est à 0 (adresse 0 de la mémoire), on a en sortie les sept bits correspondant au cas 0, et, si l'entrée est à 1 (adresse 1 de la mémoire), on a en sortie les sept bits correspondant au cas 1. On reparlera de cette approche au chapitre 4. La figure 2.9 illustre le circuit de mémoire qui pourrait être employé :

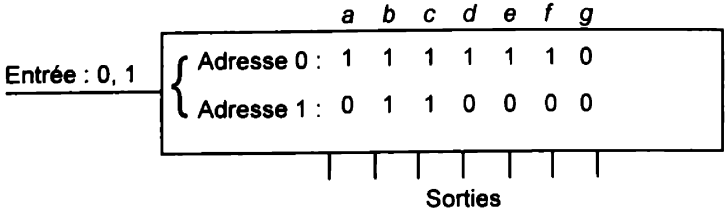


Figure 2.9 Circuit de mémoire de 14 bits incorporant la table de vérité de l'exemple précédent du multimètre.

2.4 Première méthode de résolution d'une table de vérité : résolution canonique par la méthode de la Somme de Produits (SdP)

La première méthode que nous allons étudier pour « résoudre » une table de vérité, c'est-à-dire pour en obtenir le circuit équivalent est la méthode Somme de Produits (SdP).

Il nous faut d'abord introduire le concept de « mintermes » que nous définissons immédiatement :

Un **minterme** est un *produit de toutes les variables présentes en entrée, produit qui donne un « 1 » dans la table de vérité pour les valeurs spécifiées des variables* (c'est-à-dire pour un rang donné).

Le **minterme (construction)** est un produit des variables présentes en entrée. Ces variables apparaissent non complémentées dans le produit si elles sont à « 1 » dans la table de vérité. Ces variables apparaissent complémentées si elles sont à « 0 » dans la table de vérité. L'abréviation pour un minterme est m suivi du rang de celui-ci. Ainsi m_0 sera le minterme correspondant au cas où toutes les variables d'entrée sont nulles (rang 0).

Puisque une table de vérité à une seule sortie et n variables d'entrée contient 2^n lignes, celle-ci aura aussi une possibilité 2^n de mintermes. On dit ici possibilité, puisqu'il n'est pas sûr que nous retrouvions en sortie un « 1 » sur chaque ligne, et que, dans la méthode de résolution canonique SdP, les seuls mintermes d'intérêt sont ceux correspondants aux « 1 » en sortie, ce sont les **mintermes actifs,** les mintermes inactifs étant les autres.

La méthode SdP (**Somme de Produits**) s'intéresse en effet aux « 1 » dans la table de vérité. Cette méthode donnera une sortie active (donc à « 1 ») pour toutes les combinaisons de variables qui donnent des « 1 » dans la table de vérité. Pour cela, on identifie d'abord les mintermes actifs. Une combinaison simple des mintermes actifs (par une porte OU) complète la méthode. En effet, une porte OU fournit un « 1 » en sortie dès qu'une de ses entrées est active.

La méthode SdP est donc la suivante :

> *Pour chaque sortie d'une table de vérité, établir les mintermes actifs, donc ceux qui sont à un dans la table de vérité (ce seront des produits représentés par des portes ET, voir plus bas) puis effectuer la somme (porte OU) de tous les mintermes actifs obtenus. On réalise ainsi une* ***Somme de Produits (SdP).***

Cette méthode est robuste (car un résultat est garanti), par contre elle n'est pas optimale puisque la solution obtenue peut requérir plus de portes logiques que ce qui est vraiment nécessaire.

La topologie obtenue sera celle d'un circuit SdP à deux couches, tel qu'il a été illustré à la section 2.1. Faisons un exemple pour clarifier tout ceci.

Exemple : Soit la table de vérité de la sortie $S(C,B,A$; C est le bit le plus significatif) = $m(3, 5, 7)$ où m dénote un minterme et le nombre le rang correspondant. Déterminer l'expression canonique pour la sortie S.

Solution : Dressons la table de vérité pour la sortie *S*. Celle-ci aura huit lignes, mais seules trois lignes (3, 5 et 7) sont vraiment significatives puisque contenant les trois **mintermes actifs** : *m*3, *m*5 et *m*7 (*S* = 0 pour les autres lignes). Les lignes pour lesquelles la sortie S est nulle peuvent être ignorées (elles ne contribuent pas à la sortie). On a donc la table suivante :

mintermes	Rang	C	B	A	Sortie S
$\bar{C}BA$	3	0	1	1	1
$C\bar{B}A$	5	1	0	1	1
CBA	7	1	1	1	1

alors,
$S = \bar{C}BA + C\bar{B}A + CBA$
est un circuit de type Somme de Produits (SdP)

Figure 2.10

Le minterme de rang 7, *m*7, vaut *CBA* puisque toutes les variables pour ce cas sont à 1. Strictement parlant, $m7 = C \cdot B \cdot A$ puisqu'il s'agit d'un produit des trois variables présentes en entrés, cependant on omet souvent le symbole de produit booléen « • » pour alléger l'écriture.

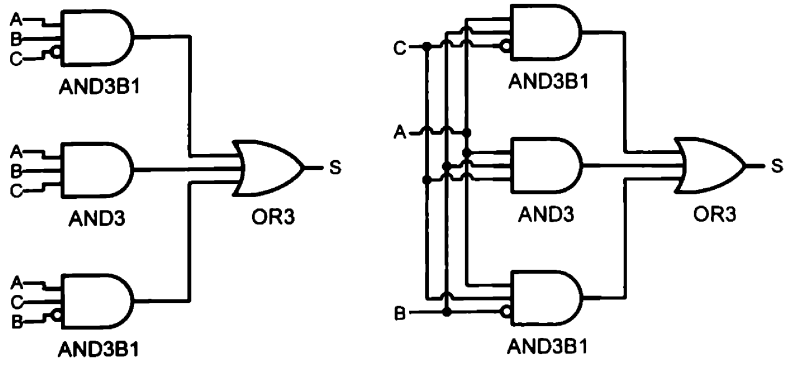
Le minterme de rang 5, *m*5, vaut $C\bar{B}A$ puisque seule la variable *B* est nulle pour ce cas (*C* = 1, *B* = 0, *A* = 1).

Le minterme de rang 3, *m*3, vaut $\bar{C}BA$ puisque seule la variable *C* est nulle pour ce cas (*C* = 0, *B* = 1, *A* = 1).

Selon la méthode canonique exposée précédemment, on peut alors écrire immédiatement :

$$S = \bar{C}BA + C\bar{B}A + CBA$$

Et le circuit correspondant en est évidemment un de type SdP avec trois portes ET (pour former les mintermes) et une porte OU pour la somme. Dans le schéma suivant, on a choisi d'employer les inverseurs à l'avant des portes ce qui est plus commode.



Circuit sans câblage des entrées (avec « étiquettes »)

Circuit avec câblage des entrées

Figure 2.11

Ces circuit ont été dessinés avec un logiciel de conception schématique (par exemple : *Xilinx Foundation ou Logisim*). On remarque à gauche qu'aux entrées des portes ET, on a employé des étiquettes avec le nom des variables plutôt que de réaliser un câblage reliant entre elles toutes les entrées pour *A*, pour *B*, pour *C* (comme montré au circuit de droite). Une étiquette (*label*), c'est le nom que l'on donne à un fil qui permet des connexions multiples : tous les fils ayant une étiquette identique se trouvent reliés ensemble sans que cela n'apparaisse directement sur le plan. Cette approche est à privilégier puisqu'elle rend la lecture d'un circuit plus facile tout en limitant le risque d'erreurs. Par comparaison, le même circuit est réalisé à droite avec un câblage explicite (et « spaghetti ») des entrées. Les deux circuits sont évidemment équivalents. On remarque aussi les deux niveaux ou deux couches du circuit, le premier niveau avec les portes ET et le second avec la porte OU. Cela est tout à fait typique des circuits à topologie Somme de Produits (SdP).

Il est à noter que la solution obtenue, appelée *SdP canonique* est correcte, mais que le nombre de portes employé n'est pas optimal. On verra plus tard des méthodes qui permettent de réduire le nombre de portes requises.

COUT D'UN CIRCUIT EN FONCTION DU NOMBRE DE PORTES DOUBLES ELEMENTAIRES

Afin de comparer entre elles les solutions que nous obtiendrons, on calculera souvent le nombre de portes requises en ramenant tous les éléments du circuit en portes élémentaires[4] équivalentes à deux entrées, sans tenir compte des inverseurs, sans tenir compte du type de portes (ET, OU, NON-ET, NON-OU) et en notant que, si on a une porte à n entrées, elle correspond en fait (en faisant abstraction du cumul éventuel des délais de propagation) à $(n - 1)$ portes élémentaires à deux entrées, comme on s'en convainc en examinant le dessin suivant :

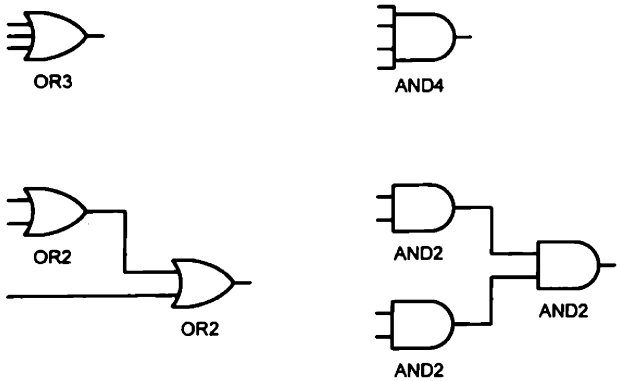


Figure 2.12 Décomposition de portes à 3 et 4 entrées en portes élémentaires à 2 entrées (OU triple à gauche, et ET quadruple à droite), abstraction faite du cumul éventuel des délais de propagation dans chaque porte.

Important : Les inverseurs ne sont jamais pris en compte dans le calcul du nombre de portes, car, en fait, il y en a toujours qui « traînent » ici et là dans un circuit. Dans un

[4] Portes élémentaires : porte ET, porte OU. La porte Ou-Exclusif n'est pas une porte élémentaire car elle est décomposable en portes ET et porte OU, voir exemple plus bas dans cette section.

circuit complexe, les variables (exemple : A) et leurs inverses (donc $\overline{A}$) étant disponibles, il n'est pas nécessaire de s'en préoccuper pour réaliser une portion d'un circuit.

Exemple : Calculons le « coût » pour un OU-exclusif à deux entrées (B, A ; B étant la variable la plus significative) :

Solution : On se rappelle la table de vérité du OU-exclusif étudiée au chapitre 1 :

Rang	B	A	S	Liste des mintermes
0	0	0	0	$\overline{B}\,\overline{A}$
1	0	1	1	$\overline{B}A$
2	1	0	1	$B\overline{A}$
3	1	1	0	BA

Figure 2.13

La Somme de Produits canonique regroupera donc les mintermes actifs 1 et 2 :

$$S = B \oplus A = \overline{B}A + B\overline{A}$$

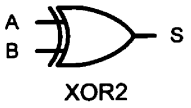


Figure 2.14

qui se décompose donc comme ceci :

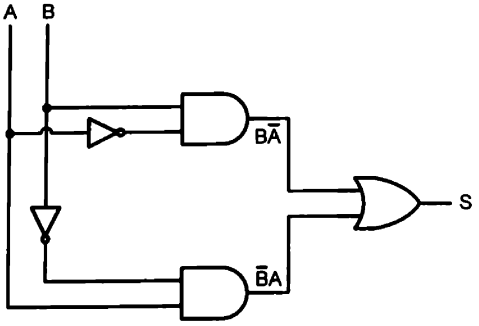


Figure 2.15 Décomposition d'une porte OU-exclusif à 2 entrées en portes élémentaires

En comptant le nombre de portes simples, on trouve immédiatement que le coût d'un OU-exclusif double est donc de 3 portes élémentaires à deux entrées (en excluant les inverseurs). La figure 2.15 nous montre aussi qu'il y a ici deux niveaux (couches).

Exemple : Calculer le « coût » du circuit obtenu à l'exemple précédent, $S(C,B,A) = m\,(3, 5, 7)$ termes de portes simples à 2 entrées.

Solution : Le circuit précédent contient 3 portes ET à 3 entrées et 1 porte OU à 3 entrées. On a donc 4 portes à 3 entrées (dans le calcul du coût, le type de portes importe peu, sauf si bien sûr on considère des OU-exclusifs tels qu'expliqués plus haut ; les inverseurs ne comptent pas non plus). On a aussi vu qu'une porte à n entrées correspond à $(n - 1)$ portes élémentaires à deux entrées, donc une porte à 3 entrées correspond à 2 portes

élémentaires à deux entrées. Comme le circuit comporte 4 de ces portes triples, on peut donc le ramener à 4 × 2 = 8 portes élémentaires à deux entrées. La réponse est donc 8 portes élémentaires doubles.

Exemple : On demande de réaliser un comparateur de deux nombres binaires A et B de 2 bits (a_1, a_0) et (b_1, b_0). On veut la solution de type SdP canonique ainsi que le schéma complet du circuit pour les sorties S_1 ($A>B$) et S_2 ($A = B$).

Solution : La table de vérité s'écrit immédiatement de la définition même du problème :

Rang	A		B		A > B	A = B
	a_1	a_0	b_1	b_0	S_1	S_2
0	0	0	0	0	0	1
1	0	0	0	1	0	0
2	0	0	1	0	0	0
3	0	0	1	1	0	0
4	0	1	0	0	1	0
5	0	1	0	1	0	1
6	0	1	1	0	0	0
7	0	1	1	1	0	0
8	1	0	0	0	1	0
9	1	0	0	1	1	0
10	1	0	1	0	0	1
11	1	0	1	1	0	0
12	1	1	0	0	1	0
13	1	1	0	1	1	0
14	1	1	1	0	1	0
15	1	1	1	1	0	1

Figure 2.16

On obtient alors les fonctions S_1 et S_2 comme les sommes des mintermes actifs (c'est-à-dire à 1 dans la table de vérité précédente) :

$S_1 = m(4, 8, 9, 12, 13, 14)$, $S_2 = m(0, 5, 10, 15)$ et en détail :

$$S_1 = \overline{a_1}a_0\overline{b_1}\,\overline{b_0} + a_1\overline{a_0}\,\overline{b_1}\,\overline{b_0} + a_1\overline{a_0}\,\overline{b_1}b_0 + a_1a_0\overline{b_1}\,\overline{b_0} + a_1a_0\overline{b_1}b_0 + a_1a_0b_1\overline{b_0}$$

$$S_2 = \overline{a_1}\,\overline{a_0}\,\overline{b_1}\,\overline{b_0} + \overline{a_1}a_0\overline{b_1}b_0 + a_1\overline{a_0}\,\overline{b_1}\,\overline{b_0} + a_1a_0b_1b_0$$

Ces équations nous permettent de dessiner directement le schéma correspondant pour le cas $S_1(A > B)$. Le schéma pour le cas $S_2(A = B)$ est laissé à faire en travail à la lectrice.

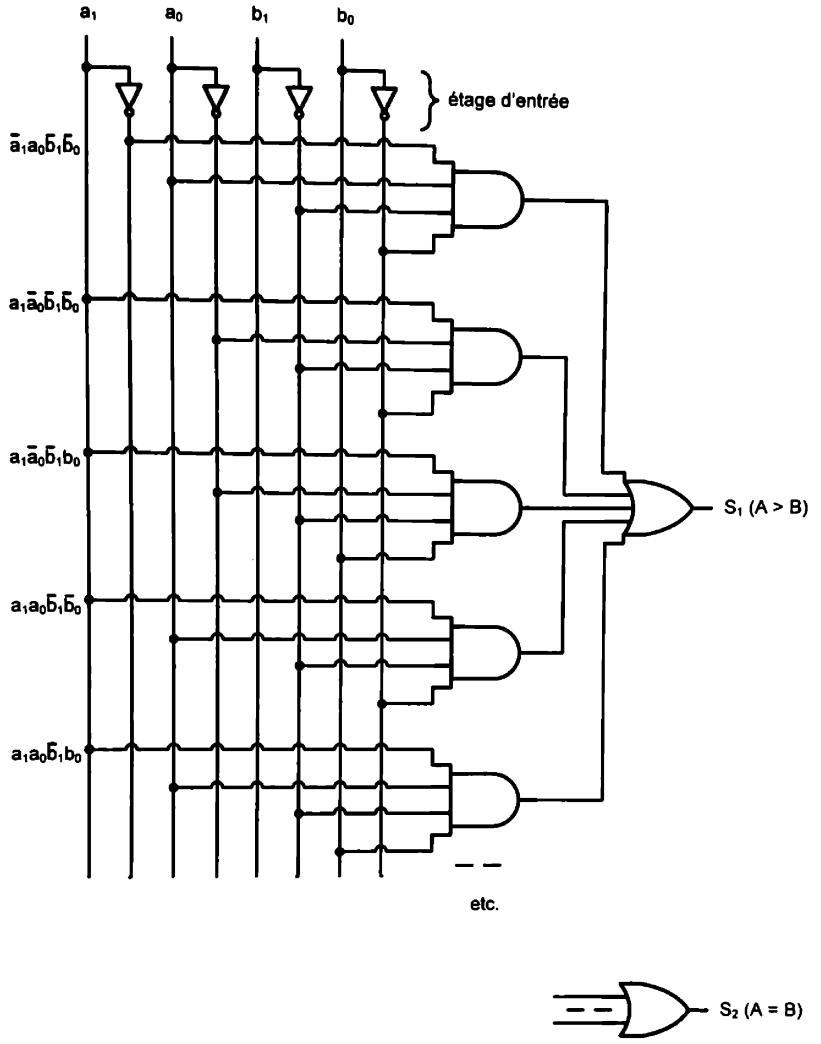


Figure 2.17

Tel qu'il a été noté, le schéma pour le cas $S_2(A = B)$ est laissé à faire en travail supplémentaire.

On voit qu'il y a 2 étages de portes requis pour obtenir la sortie, ce qui correspond bien à un circuit de type SdP (les inverseurs ne sont jamais considérés).

Calculons maintenant le coût du circuit, d'abord pour S_1 (en ce qui concerne les portes élémentaires à deux entrées). Nous avons ici :

6 portes ET à 4 entrées correspondant chacune à 3 portes doubles, donc 18 portes

1 porte OU à 6 entrées correspondant chacune à 5 portes doubles, donc 5 portes

sous-total : 23 portes doubles.

Calculons maintenant le coût du circuit, pour S_2 (en termes de portes élémentaires à deux entrées). Nous avons 4 mintermes de 4 variables chacun, la solution requiert donc :

4 portes ET à 4 entrées correspondant chacune à 3 portes doubles, donc 12 portes

1 porte OU 4 entrées correspondant chacune à 3 portes doubles, donc 3 portes

sous-total : 15 portes doubles.

Le grand total (S_1 et S_2) est donc de 23 + 15 = 38 portes élémentaires doubles pour le circuit complet (inverseurs exclus).

On se rappelle qu'une porte ET à 4 entrées correspond (abstraction faite du cumul des délais de propagation) à 3 portes ET à 2 entrées, et qu'une porte ET à 6 entrées correspond à 5 portes ET à 2 entrées selon le schéma suivant :

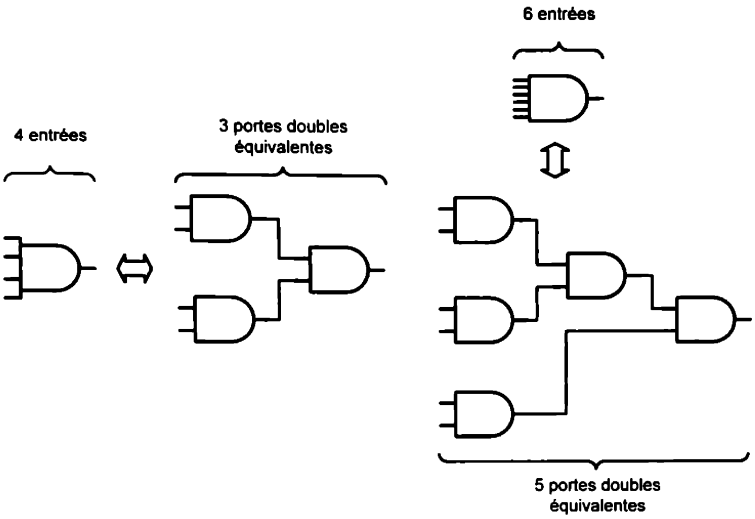


Figure 2.18

2.5 ALGÈBRE BOOLÉEN

Nous allons maintenant nous attarder à l'algèbre booléen qui, dans le contexte des circuits logiques, permet de « minimiser les expressions obtenues », c'est-à-dire de réduire le nombre de portes requises pour la réalisation d'un circuit donné. Dans cette section, on assumera que la lectrice a déjà une connaissance élémentaire de l'algèbre, ce qui permettra une présentation plus axée sur les résultats que sur les preuves et les démonstrations.

L'algèbre est un outil mathématique permettant de résoudre des équations afin de déterminer des inconnues qui sont représentées par des symboles aussi appelées variables.

Exemple : Toutes sont familières avec l'équation de la parabole :

$$y = ax^2 + bx + c$$

où y est la variable dépendante (de sortie), x est la variable indépendante (d'entrée) et (a,b,c) sont les paramètres spécifiques de la parabole considérée. De façon plus particulière, en algèbre booléen, on travaillera seulement sur deux valeurs :

0 (Faux) et 1 (Vrai), et leurs équivalents électriques, section 1.17.

Il est à noter qu'évidemment, une expression booléenne contiendra des variables booléennes (donc ayant deux valeurs possibles : 0 et 1). On ne considérera que les opérations suivantes : ET (•, produit booléen), OU (+, somme booléenne) et INVERSE (−).

Les postulats de Huntington (énoncés en 1904) nous seront très utiles ; en voici la liste pour le ET et pour le OU :

Fermeture :	$A + B \in \{0,1\}$	$A \bullet B \in \{0,1\}$	
Associativité :	$A + (B + C) = (A + B) + C$	$A \bullet (B \bullet C) = (A \bullet B) \bullet C$	
Commutativité	$A + B = B + A$	$A \bullet B = B \bullet A$	
Distributivité	$A \bullet (B + C) = A \bullet B + A \bullet C$	$A + (B \bullet C) = (A+B) \bullet (A+C)$ →	*Note* : N'est pas vrai avec des nombres réels. $2 + (3 \bullet 4) \neq (2 + 3) \bullet (2 + 4)$ $14 \neq 30$
Identités pour 0,1	$0 + A = A$ et $0 \bullet A = 0$	$1 \bullet A = A$ et $1 + A = 1$	
Complément :	$A + \overline{A} = 1$	$A \bullet \overline{A} = 0$	

Figure 2.19

On remarque qu'en changeant « + » par « • » on peut reproduire les 2 colonnes, c'est ce qu'on appelle la **dualité**.

Il est important de noter aussi que certains résultats « triviaux » nous seront très utiles, ainsi $\overline{\overline{X}} = X$, il en est de même des axiomes suivants :

Version OU (somme booléenne)	Version ET (produit booléen)
$X + \overline{X} = 1$	$X \bullet \overline{X} = 0$
$X + 1 = 1$	$X \bullet 0 = 0$
$X + 0 = X$	$X \bullet 1 = X$
$X + X = X$	$X \bullet X = X$

Figure 2.20

Tous ces théorèmes peuvent se prouver au moyen de règles simples.

DISTRIBUTIVITE EN ALGEBRE BOOLEEN

Il est important de noter qu'en algèbre booléen, la distributivité prend deux formes :

$$R \bullet (Q + P) = RQ + RP$$
$$R + (Q \bullet P) = (R + Q) \bullet (R + P)$$

La seconde forme n'est toutefois pas vraie pour les nombres réels, comme l'exemple suivant le démontre :

$$\left.\begin{array}{l} 2+(3\bullet 5)=2+15=17 \\ et\ (2+3)\bullet(2+5)=5\bullet 7=35 \end{array}\right\} \text{ pas valide pour les nombres réels,}$$

mais est vraie pour les nombres binaires, ainsi :

$$0+(1\bullet 0)=(0+1)\bullet(0+0)$$
$$0=0$$

Plus généralement, on peut facilement démontrer la validité de la seconde forme de la distributivité (celle de la somme sur le produit) en testant tous les cas avec une table de vérité, c'est ce qu'on appelle une *preuve par induction* :

La lectrice pourra faire la preuve par induction par elle-même :

P	Q	R	$R+(Q\cdot P)$	$(R+Q)\cdot(R+P)$
0	0	0	0	0
	⋮		⋮	⋮
1	1	1	1	1

Figure 2.21

THEOREMES BOOLEENS BASES SUR LA DISTRIBUTIVITE

Plusieurs théorèmes booléens basés sur la distributivité nous seront très utiles :

ABSORPTION

$\Rightarrow X+X\bullet Y=X$

Mettons le X en évidence, et on obtient : $X\bullet(1+Y)$. Par le théorème $(1+Y)=1$, on obtient le résultat $X\bullet(1)=X$.

ADSORPTION

$\Rightarrow X+\overline{X}\bullet Y=X+Y$

Se ramène au cas précédent en distribuant X, car, par distributivité, on a :

$$(X+\overline{X})\bullet(X+Y)=1\bullet(X+Y)=X+Y$$

L'adsorption est tellement utile que, si la lectrice veut retenir seulement un théorème, nous recommandons celui-ci $X+\overline{X}\bullet Y=X+Y$.

ADJACENCE

Sous ses deux formes :

$\Rightarrow X\bullet Y+\overline{X}\bullet Y=Y$

En mettant le Y en évidence, on obtient $Y\bullet(X+\overline{X})$, ce qui prouve le résultat.

$\Rightarrow (X+Y)\bullet(\overline{X}+Y)=Y$

En effectuant le calcul et en mettant le Y en évidence, on obtient $Y+(X\bullet\overline{X})$, ce qui prouve le résultat.

L'adjacence est un théorème particulièrement utile pour réduire les expressions algébriques booléennes.

LOIS DE DE MORGAN

Le mathématicien De Morgan nous a laissé deux lois qui sont fort employées en algèbre booléen. Ces lois permettent de changer d'opérateurs ($ET \Leftrightarrow OU$) et plus spécifiquement de convertir les opérations NON-ET et NON-OU, ce qui est très utile pour les simplifications des expressions logiques. Ces lois s'écrivent comme suit :

1. $\overline{A+B+C+\ldots} = \overline{A} \bullet \overline{B} \bullet \overline{C} \bullet \ldots$
2. $\overline{A \bullet B \bullet C \bullet \ldots} = \overline{A} + \overline{B} + \overline{C} + \ldots$

Figure 2.22

Un « truc » simple pour s'en souvenir consiste, lorsqu'on veut changer d'opérateurs, à inclure une négation que l'on « coupe » à chaque terme. L'exemple suivant expose le principe.

Exemple : Changer les opérateurs ($ET \Leftrightarrow OU$) dans l'expression booléenne suivante à l'aide des lois de De Morgan :

$$S = A + (C \cdot \overline{B})$$

Solution : Pour appliquer les lois de De Morgan, il faut introduire une négation. Cela se fait aisément en se rappelant que $\overline{\overline{X}} = X$, donc on réécrit l'expression de départ puis on « coupe les barres » en changeant les opérateurs suivant les lois de De Morgan :

$$S = \overline{\overline{\left[A+(C \bullet \overline{B})\right]}}$$

$$S = \overline{\left[\overline{A} \bullet \overline{(C \bullet \overline{B})}\right]}$$

$$S = \overline{\left[\overline{A} \bullet (\overline{C}+B)\right]}$$

Figure 2.23

Tous les opérateurs ont été changés dans cette dernière expression par rapport à l'expression de départ.

Exemple : Trouver les formes équivalentes des portes ET et OU.

Solution : Ceci revient à calculer les formes équivalentes pour $S1 = A \cdot B$ et $S2 = A + B$.

Note : Tel qu'on l'a déjà souligné, nous écrirons de façon indifférente : $A \cdot B$ ou AB (en omettant l'opérateur « • »).

On a donc :

$$S1 = A \cdot B \qquad\qquad S2 = A + B$$

$$S1 = \overline{\left[\overline{AB}\right]} \qquad \text{et} \qquad S2 = \overline{\left[\overline{A+B}\right]}$$

$$S1 = \overline{\left[\overline{A}+\overline{B}\right]} \qquad\qquad S2 = \overline{\left[\overline{A}\,\overline{B}\right]}$$

Ce qui nous donne les schémas équivalents suivants :

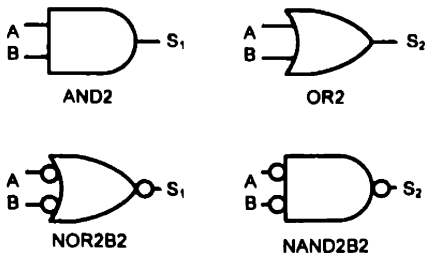


Figure 2.24 Schémas équivalents des portes ET et OU obtenus par application des lois de De Morgan

On remarque que, graphiquement, on peut simplement changer de type de portes (ET ⇔ OU) en mettant des inverseurs aux entrées et aux sorties en en changeant le type de porte. Cette constatation graphique nous sera utile pour la suite.

Exemple : Établir des formes équivalentes pour le OU-exclusif.

Solution : À partir des lois de De Morgan, on peut écrire les formes équivalentes suivantes (rappel : $X \cdot \overline{X} = 0$) :

$$A \oplus B = A\overline{B} + \overline{A}B = A\overline{A} + \left(\overline{A}B + A\overline{B}\right) + B\overline{B} = \left(A + B\right)\ \left(\overline{A} + \overline{B}\right) = (A + B)(\overline{AB})$$

En remplaçant les ET par des OU, on obtient plusieurs formes intéressantes :

La dernière $(A + B)(\overline{AB})$ se dessine comme suit :

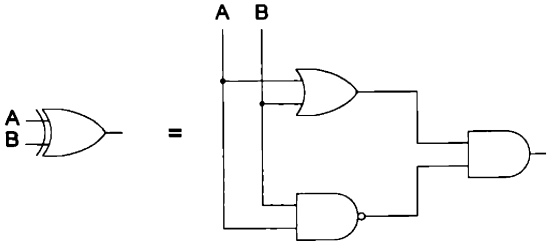


Figure 2.25

Et la forme plus traditionnelle $(A\overline{B} + \overline{A}B)$ conduit à d'autres schémas d'intérêt (en se rappelant les résultats de l'exemple précédent et aussi que $\overline{\overline{X}} = X$) :

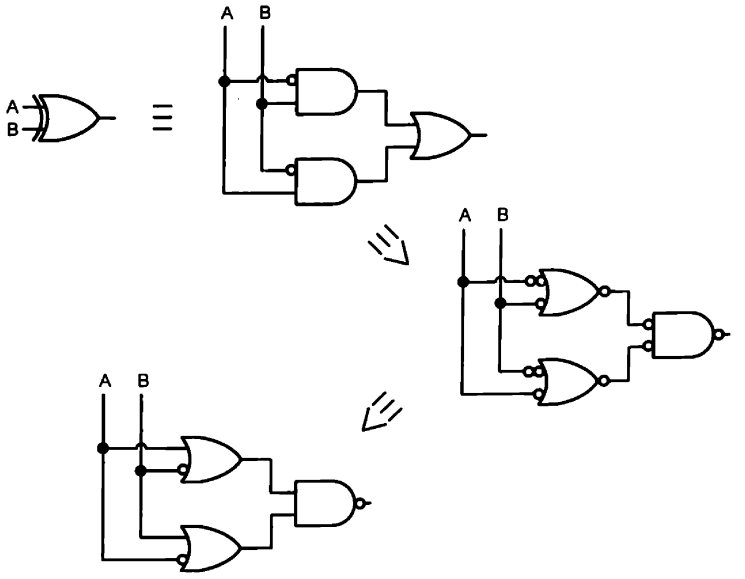


Figure 2.26

2.6 PROPRIÉTÉS DU OU-EXCLUSIF

Le OU-exclusif possède plusieurs propriétés intéressantes.

Rappelons d'abord que (selon ce que nous avons vu à l'exemple de la section 2.4):

$$A \oplus B = \bar{A}B + A\bar{B}.$$

Ainsi, le Ou-exclusif est commutatif :

$$A \oplus B = B \oplus A$$

et associatif :

$$A \oplus (B \oplus C) = (A \oplus B) \oplus C$$

Les deux identités suivantes sont aussi très utiles :

$$A \oplus A = 0$$

$$A \oplus \overline{A} = 1$$

elles se démontrent très aisément à l'aide de la table de vérité suivante :

A	$\overline{A}$	$A \oplus A$	$A \oplus \overline{A}$
0	1	0	1
1	0	0	1

Figure 2.27

La porte OU-exclusif est utile pour résoudre certains problèmes logiques. Par contre, comme elle est une combinaison de primitives, elle « coûte cher ». Comme on l'a vu

précédemment, elle équivaut à trois portes doubles élémentaires sans compter les délais de propagation à travers les deux couches du circuit équivalent.

Tout comme les portes ET et OU à entrées multiples (plus que deux), on peut aussi généraliser les portes OU-exclusif et en avoir des triples, des quadruples, etc.

Considérons la porte OU-exclusif à 3 entrées. On donne son schéma équivalent en termes de porte OU-Exclusif à deux entrées :

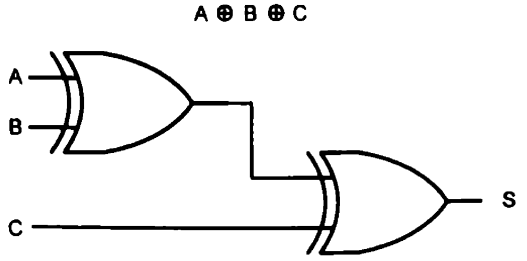


Figure 2.28

La table de vérité est la suivante :

Nombre de 1 en entrée	A	B	C	$(A \oplus B)$	$(A \oplus B) \oplus C$
0	0	0	0	0	0
1	0	0	1	0	1
1	0	1	0	1	1
2	0	1	1	1	0
1	1	0	0	1	1
2	1	0	1	1	0
2	1	1	0	0	0
3	1	1	1	0	1

Figure 2.29

De cette table de vérité, on remarque qu'on a « 1 » en sortie pour $(A \oplus B) \oplus C$ avec un nombre impair d'entrées à « 1 », donc avec 1 ou 3 entrées à « 1 ». Ceci est d'ailleurs un résultat général :

La sortie S d'une porte OU-exclusif à n entrées est à « 1 », seulement si un nombre impair d'entrées est à 1.

Cela se comprend aisément si on raisonne par groupes de deux entrées pour lesquels il faut avoir seulement une des deux entrées à « 1 » pour avoir une sortie à « 1 » (le cas « 1 ou 1 » étant bien sûr exclu).

Un autre résultat intéressant à propos de la porte OU-exclusif à 2 entrées, c'est qu'elle peut être vue comme un comparateur de la valeur des bits d'entrée. En effet,

XOR → sortie active, seulement si les entrées sont différentes ;

XNOR → sortie active, seulement si les entrées sont égales.

On peut aussi se servir des portes XOR pour modifier la valeur d'un signal, ce qui est utile dans certaines applications, par exemple pour inverser un bit (faire clignoter une DEL[5] sur une sortie numérique d'un microcontrôleur : 0, 1, 0, 1 à chaque itération).

Rappelons-nous d'abord l'axiome suivant :

$$A \oplus 1 = \overline{A}$$

Soit, maintenant, le signal A et un signal de commande CMD. Si CMD est à 1, le signal A sera inversé, comme on le constate avec la table de vérité suivante. Si CMD est à 0, la sortie est inchangée :

$$\text{Sortie} = A \oplus CMD$$

$$A \oplus CMD = \begin{cases} A \text{ si CMD} = 0 \\ \overline{A} \text{ si CMD} = 1 \end{cases}$$

A	CMD	Sortie
0	0	0
0	1	1
1	0	1
1	1	0

Figure 2.30

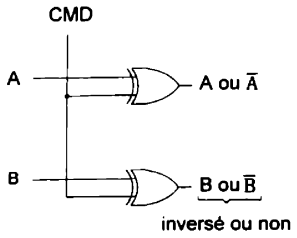


Figure 2.31
Illustration de l'inversion de deux bits A et B avec des portes OU-exclusif selon la valeur du signal logique CMD

Une autre application intéressante des portes OU-exclusif concerne la vérification de la parité d'une chaîne de bits. L'exemple suivant illustre cette utilisation.

Exemple : On demande de faire le design d'un circuit pour tester la parité impaire d'une entrée de 8 bits. On veut qu'il y ait un délai maximal de 3 portes.

Solution : La sortie devra donc être à 1 si un nombre impair de bits est présent parmi les 8 bits d'entrée. On peut ici employer judicieusement des portes OU-exclusif qui, selon la règle générale donnée plus haut, fourniront une sortie à « un » si un nombre impair de bits sont à « un » en entrée. Le schéma suivant avec 7 portes OU-exclusif fournit alors la solution (il peut être vu comme une porte OU-exclusif à huit entrées). Il est à noter qu'une autre approche possible avec la méthode des Sommes de Produits aurait été possible. Dans ce cas, toutefois, il aurait fallu dresser une table de vérité avec $2^8 = 128$ lignes. La solution aurait aussi exigé des portes ET à 8 entrées pour la réalisation des mintermes. L'utilisation des OU-exclusif se justifie donc ici pleinement. On respecte aussi la contrainte du délai maximal de 3 portes puisque nous obtenons un circuit à trois couches.

5 DEL = Diode électro-luminescente, LED (*Light emitting diode*) en anglais.

Le schéma retenu est le suivant :

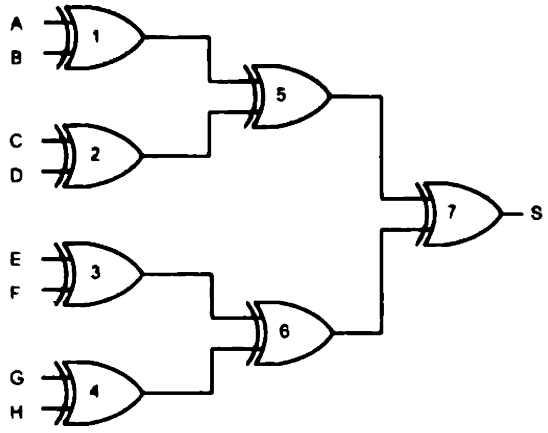


Figure 2.32

2.7 UTILISATION DE PORTES NON-ET DANS LES DESIGNS SDP (SOMME DE PRODUITS)

On a présenté l'approche ET-OU pour implanter les tables de vérité dans les circuits logiques, on a aussi étudié l'utilisation des portes OU-excusif pour certaines applications. On peut aussi employer des portes NON-ET, ce qui permet une réalisation complète d'un design donné avec un seul type de porte. Cela est possible grâce à l'utilisation des lois de De Morgan vues plus tôt. Dans l'étude suivante, nous emploierons la notation suivante :

m_i désignera le minterme actif de rang i (où i correspond au rang de tous les mintermes actifs dans la table de vérité considérée). Aussi :

$\Sigma \Leftrightarrow OU$ (somme booléenne)

$\Pi \Leftrightarrow ET$ (produit booléen)

alors une forme canonique de SdP s'exprime, avec De Morgan, comme ceci :

$$\text{SORTIE} = \Sigma m_i = \overline{\overline{\Sigma m_i}} = \overline{\overline{\Pi m_i}}$$

Ce résultat est très important. En effet, dans la forme de droite, on emploie une structure à deux couches avec des portes NON-ET. L'avantage manifeste est que cette forme ne requiert qu'un seul type de porte NON-ET au lieu de la forme ET-OU vue précédemment. Il est à noter que cette procédure ne simplifie pas l'expression puisqu'on conserve la forme canonique (si on en avait une au départ). L'exemple suivant illustre le principe.

Exemple : Soit la sortie S exprimée par la table de vérité suivante $S(A,B,C$; A : le bit le plus significatif) :

Minterme	A	B	C	S
	0	0	0	0
$\overline{A}\overline{B}C$	0	0	1	1
	0	1	0	0
	0	1	1	0
$A\overline{B}\overline{C}$	1	0	0	1
	1	0	1	0
$AB\overline{C}$	1	1	0	1
	1	1	1	0

Figure 2.33

On demande de trouver l'expression canonique de S sous la forme d'une SdP et son équivalent contenant uniquement des portes NON-ET.

Solution : La forme canonique de cette table de vérité est obtenue comme la somme des mintermes actifs (c'est-à-dire à 1 en sortie S) :

$$S = \overline{A}\overline{B}C + A\overline{B}\overline{C} + AB\overline{C}$$

Ce qui peut se réécrire :

$$S = \overline{\overline{\overline{A}\overline{B}C + A\overline{B}\overline{C} + AB\overline{C}}}$$

$$S = \overline{\overline{A}\overline{B}C} \cdot \overline{A\overline{B}\overline{C}} \cdot \overline{AB\overline{C}}$$

Notons au passage qu'on aurait pu facilement simplifier l'expression par simple mise en évidence :

$$S = \overline{A}\overline{B}C + A\overline{B}\overline{C} + AB\overline{C}$$

$$S = \overline{A}\overline{B}C + A\overline{C}\underbrace{(\overline{B} + B)}_{1}$$

$$S = \overline{A}\overline{B}C + A\overline{C}$$

mais ce n'était pas demandé dans le problème.

Revenant aux expressions précédentes, on a alors :

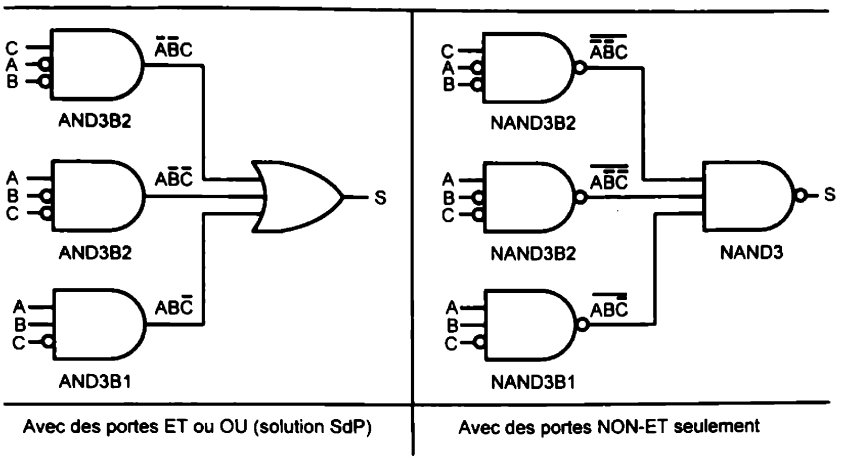


Figure 2.34

L'observation de ces deux circuits montre que ceux-ci ont une topologie identique.

En fait, on peut alors énoncer la règle suivante (qui vient directement de l'application des lois de De Morgan) :

On peut remplacer, sans affecter la sortie, tous les ET et tous les OU d'un design SdP à deux niveaux par des portes NON-ET.

La topologie qui emploie des portes NON-ET à deux entrées peut se réaliser uniquement avec des puces TTL, dont le numéro est 7400 (une puce 7400 contient 4 portes NON-ET à deux entrées). Le 7400 est le premier numéro de la série TTL, ce qui dénote bien l'intérêt que les NON-ET avaient en design à l'époque où les circuits à intégration SSI et MSI faisaient fureur. On peut en effet montrer que l'utilisation d'un seul type de puce électronique permet de réduire globalement le nombre de puces employées puisque l'utilisation de chacune sera maximale sur un circuit imprimé donné. En utilisant plusieurs types différents, la possibilité que certaines puces ne soient pas toutes employées à un endroit donné du circuit imprimé est plus grande. En plus, il n'est pas nécessaire de conserver un inventaire diversifié de puces électroniques.

Finalement, dans l'exemple précédent sur les formes équivalentes des portes ET et OU (figure 2.24), on avait noté que, graphiquement, on peut simplement changer de type de porte (ET $\Leftrightarrow$ OU) en mettant des inverseurs aux entrées et aux sorties en en changeant le type de porte. En appliquant ce résultat à la porte OU de sortie de la topologie SdP (circuit de gauche) et en ramenant les bulles inverseuses à l'étage ET (qui devient alors de type NON-ET), on retrouve le schéma en NON-ET seulement (circuit de droite), figure 2.34.

2.8 IMPLIQUANTS

La notion d'impliquant est employée dans certains ouvrages. Nous allons donc en parler très succinctement. Un impliquant est n'importe quel produit d'une expression SdP, donc n'importe quel minterme de cette SdP.

Un minterme qui ne peut être simplifié davantage, sauf en altérant la table de vérité correspondante, est appelé un impliquant principal (*prime implicant*).

Considérons par exemple la relation suivante :

$$S = C\overline{B} + \overline{B}A + CA$$

Puisqu'on ne peut simplifier ces termes davantage, ils sont tous des impliquants principaux : ils doivent être là pour fournir la bonne valeur de sortie.

De façon intéressante, la somme d'un ensemble complet d'impliquants principaux est l'expression SdP minimisée de la table de vérité correspondante. Au chapitre 3, nous étudierons des méthodes de simplification des SdP canoniques.

2.9 RECHERCHE DES 0 ET DESIGN AVEC DES PORTES NON-OU (MÉTHODE DE SYNTHÈSE PAR PRODUIT DE SOMMES (PDS))

Notre étude de la « mise en circuit » (aussi appelée la « résolution » de circuits) nous avait conduits à la topologie « Somme de Produits » (SdP) permettant de transcrire n'importe quelle table de vérité sous sa forme canonique. Dans cette étude, nous avions aussi introduit la notion de mintermes actifs associés aux 1 dans les tables de vérité. Peut-être certaines se sont-elles posées la question suivante : *Et les 0, ne sont-ils pas utiles aussi ?* La question est pertinente puisque dans un système binaire, si une valeur n'est pas à 1, alors elle est forcément à 0. En fait, il est clair que si on connaît tous les 0 d'une table donnée, on connaît du même coup tous les 1.

Dans cette section, on va donc maintenant s'attarder aux 0 présents dans les tables de vérité. Cela nous conduira à réaliser le design d'un circuit à l'aide de la méthode de *Produit de Sommes* (ou PdS, en anglais *Product of Sum*). Ces circuits PdS seront du type OU-ET. La topologie PdS est la version en logique négative de la topologie SdP. Cela nous sera utile pour la résolution des tables de vérité avec peu de 0, beaucoup de 1.

Il nous faut d'abord introduire le concept de Maxterme :

Un **Maxterme** est une somme qui donne 0 dans la table de vérité, pour les valeurs spécifiées des variables présentes en entrée. On dénotera le Maxterme par M majuscule par opposition au minterme représenté avec un m minuscule.

Le **Maxterme (construction)** est une somme des variables présentes en entrée. Ces variables apparaissent non complémentées dans le Maxterme si elles sont à 0 dans la table de vérité. Ces variables apparaissent complémentées dans le Maxterme si elles sont à 1 dans la table de vérité.

Se rappelant la définition d'un minterme vue précédemment, on peut comparer les deux définitions :

minterme : combinaison qui donne 1 (produit)

Maxterme : combinaison qui donne 0 (somme)

La relation suivante lie les Maxtermes aux mintermes à cause des lois de De Morgan comme nous le verrons dans un instant :

$$M_i = \overline{m_i} \tag{2.1}$$

où M_i = Maxterme, m_i = minterme et i est le rang dans la table de vérité considérée.

Cette relation nous sera fort utile car « intuitivement »[6] il est plus facile d'écrire la relation d'un minterme que celle d'un Maxterme!

Considérons le cas d'une table de vérité quelconque de deux variables A et B $S(B,A$; B : bit le plus significatif) :

Rang	B	A	S	Termes
0	0	0	x	m_0 et M_0
1	0	1	x	m_1 et M_2
2	1	0	x	m_2 et M_2
3	1	1	x	m_3 et M_3
	Entrées		Sortie quelconque	

Figure 2.35

Pour chaque ligne « i » d'une table de vérité, on a potentiellement d'actif soit un minterme, soit un Maxterme : M_i ou m_i. En effet, puisqu'une sortie ne peut être à « 1 » et à « 0 » à la fois, si le minterme est actif, alors le Maxterme correspondant sera inactif et inversement (cela explique aussi la relation précédente : $M_i = \overline{m_i}$). Ainsi, si la table de vérité précédente correspond à celle d'une porte ET, $S_{ET}(B,A) = m_3$. Si au contraire il s'agit de la table de vérité d'une porte OU, $S_{OU}(B,A) = m_0 + m_1 + m_2$, selon la définition d'une SdP vue précédemment.

Prenons la ligne $i = 2$ de la table précédente, correspondant à M_2 et à m_2 selon la valeur correspondante en sortie. Alors, on a :

$$m = \overline{A}B.$$

Et donc, à l'aide de la loi de De Morgan :

$$\overline{\overline{A}B} = \overline{\overline{\overline{A}} + \overline{B}} = \overline{A + \overline{B}}$$

Le terme $\overline{A}B$ donne un 1 dans la table de vérité pour la valeur spécifiée des variables ($B = 1$ et $A = 0$), c'est donc le minterme. Le terme $A + \overline{B}$ donne un 0 dans la table de vérité pour la valeur spécifiée des variables ($B = 1$ et $A = 0$), c'est donc le Maxterme correspondant. On a alors :

$$m_2 = \overline{A}B \text{ et } M_2 = A + \overline{B}$$

---

6 L'expérience empirique de l'enseignement de ces concepts sur plusieurs années confirme curieusement ce fait qui est sans doute explicable par la facilité avec laquelle notre esprit réalise les produits booléens : $X \cdot 1 = X$ et $X \cdot 0 = 0$.

et

$$m_2 = \overline{M}_2$$

Considérons maintenant la table de vérité suivante $S(A,B,C$; A : bit le plus significatif) :

A	B	C	S	
0	0	0	1	
0	0	1	0	$M_1 = A + B + \overline{C}$
0	1	0	1	
0	1	1	1	
1	0	0	1	
1	0	1	1	
1	1	0	0	$M_6 = \overline{A} + \overline{B} + C$
1	1	1	1	

Figure 2.36

On voit que, si on raisonne sur les 0, seuls deux Maxtermes retiendront notre attention : M_1 et M_6. La question qui se pose maintenant est la suivante : Comment combiner ces deux termes pour la trouver la fonction S ?

En fait, on veut un zéro en S lorsque n'importe lequel des deux Maxtermes est actif (c'est-à-dire nul) et justement une porte ET fournit un 0 si n'importe quel de ces arguments est nul selon l'axiome : $X \cdot 0 = 0$. Donc, la fonction S est obtenue en faisant simplement le produit des deux Maxtermes impliqués ici

$$S = M_1 \cdot M_6 = \left(A + B + \overline{C}\right) \cdot \left(\overline{A} + \overline{B} + C\right)$$

Toute combinaison de variables autre que celles correspondant à $M_1 \cdot M_6$ donnera une sortie S à 1 ce qui correspond bien à ce que l'on cherche. Essayons par exemple C = B = A = 1, on voit tout de suite que S = 1, essayons aussi $C = B = A = 0$. On voit tout de suite que S = 1. Par contre avec les combinaisons correspondant à M_1 et M_6, S = 0.

L'équation obtenue est de type Produit de Sommes (PdS) et le circuit correspondant se dessine comme suit :

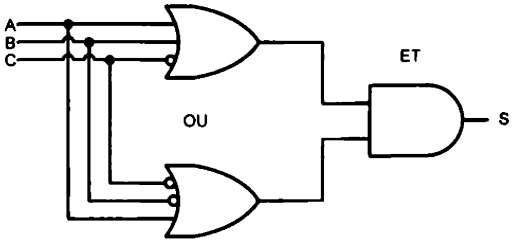


Figure 2.37

On peut résumer cette section avec les relations suivantes :

SdP		PdS	
$S = \sum m_i$	(2.2)	$S = \prod M_j$	(2.3)

où i est le rang de toutes les lignes ayant 1 en sortie S dans la table de vérité considérée ; où j est le rang de toutes les lignes ayant 0 en sortie S dans la table de vérité considérée.

2.10 Lois de De Morgan appliquée aux circuits PdS

À la section précédente, on a vu que :

$$S = \prod M_j$$

où M_j est le Maxterme actif qui synthétise le 0 présent à la ligne de rang j de la table de vérité de la fonction d'intérêt.

Les lois de De Morgan peuvent être appliquées aux Maxtermes afin de ne conserver qu'un seul type de porte, tout comme on l'avait fait pour les SdP, ce qui avait conduit à des solutions en portes NON-ET seulement. Le raisonnement vu précédemment est donc appliqué comme suit :

$$S = \prod M_j = \overline{\overline{\prod M_i}} = \overline{\overline{\sum M_j}} \leftarrow \text{Portes NON-OU}$$

où j est le rang de toutes les lignes ayant 0 en sortie S dans la table de vérité considérée.

Considérons la table de vérité suivante pour la fonction $S(A,B,C$; A : bit le plus significatif) :

A	B	C	S	
0	0	0	0	$M_0 = A+B+C$
0	0	1	1	
0	1	0	1	
0	1	1	1	
1	0	0	1	
1	0	1	1	
1	1	0	1	
1	1	1	0	$M_7 = \overline{A}+\overline{B}+\overline{C}$

Figure 2.38

Ici, il y a peu de 0, ce qui suggère un design de type PdS avec deux Maxtermes actifs, M_0 et M_7 :

$$S=(M_0 \cdot M_7)=(A+B+C)\cdot(\overline{A}+\overline{B}+\overline{C})$$

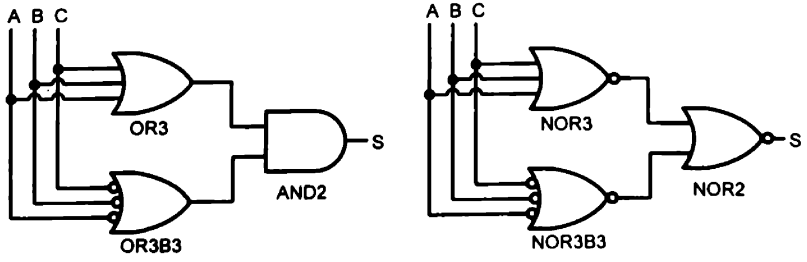
Cette expression s'appelle le Produit de Sommes canonique (PdS) puisqu'il s'agit d'une expression brute, non simplifiée. Au chapitre suivant, on présentera des méthodes de simplification efficaces.

Revenons à notre exemple ; si l'on veut établir le PdS canonique en portes NON-OU seulement, on procède alors comme suit :

$$S=\overline{\overline{M_0 \cdot M_7}}=\overline{\overline{M_0}+\overline{M_7}}=\overline{\overline{(A+B+C)}+\overline{(\overline{A}+\overline{B}+\overline{C})}}$$

Cette expression pourrait éventuellement être simplifiée.

Cette dernière expression donne les circuits finaux suivants (de topologie PdS et en Portes NON-OU seulement) :



Circuit de type PdS

Circuit avec uniquement des portes NON-OU

Figure 2.39

L'observation des deux circuits précédents montre que ceux-ci ont une topologie identique. En fait, on peut alors énoncer la règle suivante :

> *On peut remplacer, sans affecter la sortie, tous les OU et tous les ET d'un design PdS à 2 niveaux par des portes NON-OU.*

La topologie qui emploie des portes NON-OU à deux entrées peut se réaliser uniquement avec des puces TTL, dont le numéro est 7402 (une puce 7402 contient quatre portes NON-OU à deux entrées).

Finalement, dans notre exemple précédent sur les formes équivalentes des portes ET et OU (figure 2.24), on avait noté que, graphiquement, on peut simplement changer de type de porte (ET ⇔ OU) en mettant des inverseurs aux entrées et aux sorties en en changeant le type de porte. En appliquant ce résultat à la porte ET de sortie de la topologie PdS (circuit de gauche) et en ramenant les bulles inverseuses à l'étage OU (qui devient alors de type NON-OU), on retrouve le schéma en NON-OU seulement (circuit de droite).

2.11 OPTIMISATION DES DESIGNS NUMÉRIQUES

Résumons les techniques de résolution des tables de vérité vues jusqu'à présent. Il s'agit donc, à partir d'une table de vérité qui expose un problème logique, d'en synthétiser le circuit correspondant que l'on réalise avec les portes élémentaires ET, OU, NON et leur combinaison.

On a deux choix selon la table de vérité à résoudre :

- Si la sortie d'intérêt contient peu de 1, on emploie un design SdP :
  - $\rightarrow \Sigma m_i$
  - $\rightarrow \overline{\overline{\Pi m_i}}$ (avec portes NON - ET)
- Si la sortie d'intérêt contient peu de 0, on emploie un design PdS :
  - $\rightarrow \Pi M_j$
  - $\rightarrow \overline{\overline{\Sigma M_j}}$ (avec portes NON - OU)

- où m_i représente le minterme actif de rang i dans la table de vérité, et M_j représente le Maxterme actif de rang j de la table de vérité d'intérêt ;
- avec Σ représentant une somme booléenne implantée avec une porte OU et Π représentant un produit booléen implanté avec une porte ET ;
- où i est le rang de toutes les lignes ayant 1 pour la sortie considérée de la table de vérité d'intérêt ;
- où j est le rang de toutes les lignes ayant 0 pour la sortie considérée de la table de vérité d'intérêt.

On a aussi discuté de l'utilisation des portes OU-exclusif qui permettent des simplifications efficaces dans certains cas.

En appliquant directement ces techniques aux tables de vérité, ces techniques fournissent ce qu'on appelle les *solutions canoniques*, c'est-à-dire non simplifiées. Une simplification peut être obtenue à l'aide de l'algèbre de Boole dont on a discuté brièvement. Cette approche est cependant pénible et sujette aux erreurs à cause des manipulations algébriques qu'elle requiert. Au chapitre suivant, une technique graphique

(c'est-à-dire visuelle) par « tables de Karnaugh » permettra d'accéder à des simplifications rapidement et avec moins d'effort.

Ces techniques par « tables de Karnaugh » permettront de minimiser le nombre de portes employées. Cette question était cruciale dans les années soixante à quatre-vingt puisque la technologie des circuits intégrés numériques était limitée, rendant disponibles peu de portes dans les circuits intégrés. Depuis avec l'explosion du nombre de transistors embarqués sur une puce (chapitre 1), cette question est moins fondamentale, et c'est tant mieux ! En effet, il est important de noter qu'une solution avec un nombre minimal de portes ne correspond pas toujours à la meilleure solution. En effet, il y aura toujours un compromis entre le nombre de portes total, le nombre de variables dans les termes d'entrées et les délais de propagation.

D'autres critères sont aussi à considérer, tels :

- La modularité qui permet de « recycler un design » , comme on l'a vu au chapitre 1 avec le demi-additionneur qui peut servir à réaliser un additionneur complet à n bits.
- Le nombre de portes différentes si on doit limiter un inventaire de pièces. Cette contrainte n'est plus très actuelle avec la disponibilité de circuits FPGA ou CPLD génériques où les portes employées sont des librairies de fonction (comme dans le logiciel Xilinx WebPack) sauf dans le cas de la logique d'interconnexion aussi appelée *glue-logic* ou colle-logique qui sert à relier entre eux des modules d'importance de type FPGA et CPLD.
- La consommation électrique est aussi un facteur qui peut être crucial pour certaines applications embarquées ; pensons aux satellites, aux appareillages non connectés au réseau électrique (bouées marines, tours d'observation météorologique perchées sur les montagnes, appareils portables les plus divers, etc.). Même dans les applications connectées au réseau, il est important de limiter la consommation électrique pour des raisons de coût de l'énergie et de préservation de l'environnement.
- La tolérance au bruit électromagnétique est cruciale dans certaines applications sensibles, par exemple dans les papeteries où il y a commutation de moteurs électriques de grande puissance qui engendrent des perturbations sur les fils. Dans ces cas, des circuits avec redondance couplés à des mesures de protection passive (boîtiers mis à la masse) sont requis et nécessitent bien sûr plus de composantes.
- La synchronisation est un aspect important pour éviter les « courses » causées par des différences de temps de propagation dans les circuits combinatoires. De façon spécifique, les aléas (*glitches* en anglais) sont causés par des délais de propagation différents selon les parcours empruntés par les signaux logiques. Si certains signaux sont prêts avant d'autres cela peut causer des valeurs de sorties intermédiaires non désirées aux effets perturbateurs, voir dramatiques. Il y a là un réel danger ! Nous en reparlerons au chapitre 3 où une méthode sera introduite pour empêcher les aléas de se produire. Bien sûr, ces mesures nécessitent plus de composants.

PROBLÈMES RÉSOLUS

2.1. Donnez la table de vérité complète du circuit suivant $S(C,B,A$; C étant le bit le plus significatif) :

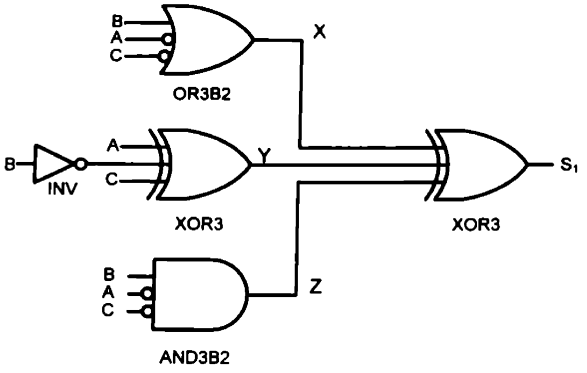


Figure 2.40

Solution : La table de vérité se dresse en testant un à un tous les cas possibles. Puisque trois variables sont impliquées, on aura $2^3 = 8$ lignes et donc huit cas à tester. Pour simplifier le processus, il est utile d'établir des résultats intermédiaires X, Y, Z. On inclut aussi les valeurs inverses $\overline{A},\overline{B},\overline{C}$. La table s'obtient donc :

Rang	C	B	A	$\overline{C}$	$\overline{B}$	$\overline{A}$	X	Y	Z	S1
0	0	0	0	1	1	1	1	1	0	0
1	0	0	1	1	1	0	1	0	0	1
2	0	1	0	1	0	1	1	0	1	0
3	0	1	1	1	0	0	1	1	0	0
4	1	0	0	0	1	1	1	0	0	1
5	1	0	1	0	1	0	0	1	0	1
6	1	1	0	0	0	1	1	1	0	0
7	1	1	1	0	0	0	1	0	0	1

Figure 2.41

On voit finalement que $S1(C,B,A) = m(1,4,5,7)$.

Vérification : il peut être intéressant de simuler le circuit si un tel programme est disponible, tel le Xilinx WebPACK ou Logisim. Le résultat d'une telle simulation pour le circuit précédent est montré à la figure suivante. À noter sur la figure 2.42, les symboles *AA*, *BB*, *CC*, *SS*1 correspondent respectivement aux signaux *A*, *B*, *C*, *S*1 du circuit. Lors de la simulation, on a modifié successivement toutes les entrées de façon à tester tous les cas possibles. La sortie correspondante obtenue chaque fois est aussi montrée. Par exemple, la ligne verticale à 4.3 ns montre la cas où $A = 0$, $B = 0$, $C = 1$ et $S1 = 1$.

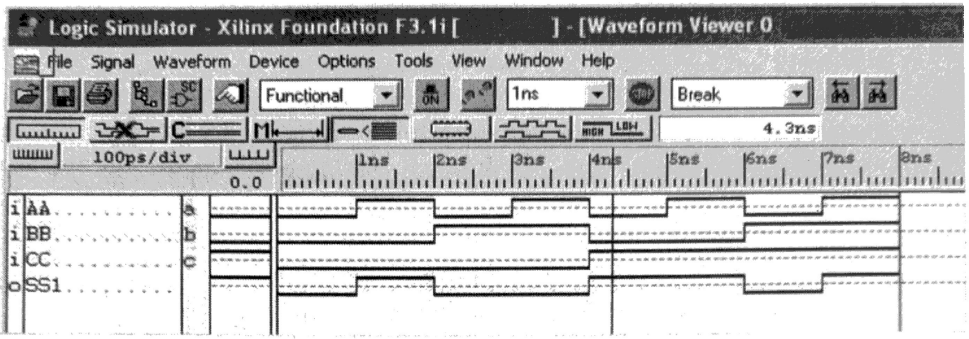


Figure 2.42

2.2. Soit la fonction suivante F(C,B,A) :

$$S(C,B,A) = (C\overline{B} \oplus \overline{C}BA) + C\overline{A}$$

avec (C = bit le plus significatif et A = bit le moins significatif).

a) Trouver une expression Somme de Produit canonique.

b) Trouver une expression canonique en Portes NON-ET seulement.

Solution : En développant le OU-exclusif et en effectuant les opérations de distributivité, on obtient successivement :

$$\begin{aligned} S &= (\overline{C\overline{B}} \cdot \overline{C}BA + C\overline{B} \cdot \overline{\overline{C}BA}) + C\overline{A} \\ &= (\overline{C} + B) \cdot \overline{C}BA + C\overline{B} \cdot (C + \overline{B} + \overline{A}) + C\overline{A} \\ &= \overline{C}\overline{C}BA + \overline{C}BBA + CC\overline{B} + C\overline{B}\,\overline{B} + C\overline{B}\,\overline{A} + C\overline{A} \\ &= \underbrace{\overline{C}BA + \overline{C}BA}_{\overline{C}BA} + \underbrace{C\overline{B} + C\overline{B}}_{C\overline{B}} + C\overline{B}\,\overline{A} + C\overline{A} \\ &= \overline{C}BA + C\overline{B} + C\overline{B}\,\overline{A} + C\overline{A} \end{aligned}$$

Pour avoir une solution canonique, il nous faut seulement des mintermes avec C, B et A. Se rappelant l'axiome $X + \overline{X} = 1$, on peut réécrire la dernière expression comme suit en introduisant les « variables manquantes » dans les termes à deux variables :

$$\begin{aligned} S &= \overline{C}BA + C\overline{B}(A + \overline{A}) + C\overline{B}\,\overline{A} + C\overline{A}(B + \overline{B}) \\ S &= \overline{C}BA + C\overline{B}A + C\overline{B}\,\overline{A} + C\overline{B}\,\overline{A} + C\overline{A}B + C\overline{A}\,\overline{B} \end{aligned}$$

Et en éliminant les termes répétés :

$$S = \overline{C}BA + C\overline{B}A + C\overline{B}\,\overline{A} + C\overline{A}\,\overline{B}$$

La dernière expression est une Somme de Produits canonique qui contient uniquement les mintermes actifs des trois variables présentes (C, B, A). On voit que $S(C, B, A$; C = $MSB) : m(3,4,5,6)$.

On obtient la solution en portes NON-ET seulement, en reprenant la dernière expression, en l'inversant deux fois et en appliquant les lois de De Morgan :

$$\overline{\bar{C}BA + C\bar{B}A + C\bar{B}\bar{A} + CB\bar{A}}$$

$$\overline{\bar{C}BA} \cdot \overline{C\bar{B}A} \cdot \overline{C\bar{B}\bar{A}} \cdot \overline{CB\bar{A}}$$

Cette dernière expression est une solution canonique en portes NON-ET seulement.

2.3. Simplifier l'expression suivante $F(Z,Y,X$; $Z = MSB)$ en appliquant les règles de l'algèbre de Boole. Calculer aussi les coûts en fonction du nombre de portes élémentaires doubles (sans compter les inverseurs).

$$F = \bar{Z}\bar{Y}X + \bar{Z}YX + \bar{Y}Z\bar{X} + Z\bar{Y}X + ZY\bar{X}$$

Solution : Par mise en évidence des variables communes, on obtient :

$$F = \bar{Z}X \cdot \underbrace{(\bar{Y} + Y)}_{1} + \bar{Y}Z \cdot \underbrace{(\bar{X} + X)}_{1} + ZY\bar{X}$$

$$= \bar{Z}X + \bar{Y}Z + ZY\bar{X}$$

$$= \bar{Z}X + Z \cdot \underbrace{(\bar{Y} + Y\bar{X})}_{adjonction}$$

Ici, on peut appliquer l'adjonction. Rappel : $C + \bar{C} \cdot B = C + B$, et alors :

$$= \bar{Z}X + Z \cdot (\bar{Y} + \bar{X})$$

$$= \underbrace{\bar{Z}X + Z\bar{X}}_{OU\text{-}exclusif} + Z\bar{Y}$$

Avec l'application de la définition du OU-exclusif, on obtient finalement :

$$F = Z \oplus X + Z\bar{Y}$$

Le coût en fonction des portes élémentaires doubles est de trois portes élémentaires doubles (il faut décomposer le OU-exclusif), plus une porte triple (équivalant à deux portes élémentaires doubles), donc un total de cinq portes élémentaires doubles.

L'expression de départ non simplifiée exigeait cinq portes triples et une porte quintuple donc un coût de $5 \times 2 + 1 \times 4 = 14$ portes élémentaires doubles. Cela valait donc la peine de se « casser la tête » pour simplifier l'expression.

2.4. Montrer que, pour le problème 2.3, $F = Z \oplus X + Z\bar{Y} = Z \oplus X + Z\bar{Y}X$.

Solution : Il s'agit d'obtenir tous les mintermes actifs par décomposition des deux expressions. On voit alors qu'elles sont équivalentes. On peut aussi dresser les deux tables de vérité complètes.

EXERCICES[7]

2.1. Donner les mintermes et les Maxtermes actifs pour la fonction $F(C,B,A\ ;\ C = MSB)$ définie par la table de vérité suivante :

C	B	A	F
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Figure 2.43

2.2. Transformer les SdP suivantes en PdS ; donner aussi les numéros des mintermes et des Maxtermes actifs (C = bit le plus significatif):

a) $A\overline{B}C + \overline{A}BC + A\overline{B}\,\overline{C} + ABC$

b) $ACD + \overline{A}\,\overline{B}\,\overline{C}\,\overline{D} + \overline{A}B\overline{C}D + A\overline{B}C\overline{D}$

2.3. Transformer les PdS suivants en SdP ; donner aussi les numéros des mintermes et des Maxtermes actifs (C = bit le plus significatif):

a) $(A+\overline{B}+C)\cdot(B+C)\cdot(A+\overline{B}+\overline{C})\cdot(A+B+C)$

b) $(A+C+D)\cdot(\overline{B}+\overline{C})\cdot(\overline{A}+B+\overline{C})\cdot(A+C+\overline{D})$

2.4. Soit la fonction suivante :

$$F(B,A\ ;\ B = MSB) = \sum\ m\,(0,3)$$

a) Donner la table de vérité correspondante.

b) Dessiner le circuit correspondant (SdP canonique) et calculer son coût selon le nombre de portes doubles élémentaires.

2.5. Que vaut $A \cdot B$ en matière de portes OU ?

2.6. Soit la fonction suivante :

$$S(C,B,A\ ;\ C = MSB) = \sum\ m(2,5,7)$$

[7] m = minterme, M = Maxterme.

Fournir les expressions algébriques suivantes pour S :

a) Somme de Produits canonique.

b) Produit de Sommes canonique.

c) Solution canonique en portes NON-ET, seulement.

d) Solution canonique en portes NON-OU, seulement.

2.7. Simplifier algébriquement l'expression suivante, $F(Z,Y,X$; $Z = MSB)$:

$$F(Z,X,Y) = \overline{X}Z + XY + X\overline{Y}Z$$

Dresser sa table de vérité.

2.8. Simplifier algébriquement l'expression suivante, $F(W,Z,Y,X$; $W = MSB)$:

$$F = (\overline{X} + \overline{Y}) \cdot (\overline{W} + \overline{X} + Y) \cdot (\overline{W} + X + Z)$$

Dresser sa table de vérité.

2.9. Simplifier algébriquement l'expression suivante, $F(W,Z,Y,X$; $W = MSB)$:

$$F = \overline{W}(Y\overline{Z} + XY) + Y(\overline{X}\,Z + WZ)$$

Dresser sa table de vérité.

2.10. Réaliser un circuit employant seulement des portes NON-OU, circuit pour lequel la sortie sera à 1 si un code BCD valide est placé en entrée, et pour lequel la sortie sera à 0 si un code BCD non valide est placé en entrée. Le circuit a donc une sortie S et quatre entrées. Employer les variables : $(D,C,B,A$; D $MSB)$.

3

Méthode des tables de Karnaugh pour minimiser les expressions booléennes

MAURICE KARNAUGH À L'ORIGINE DE LA MÉTHODE DE SIMPLIFICATION GRAPHIQUE

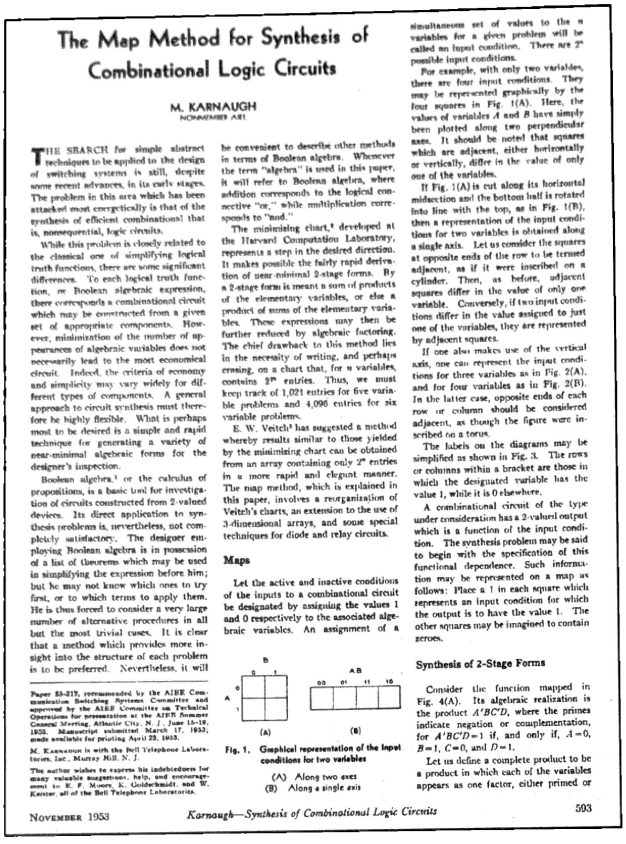
Il est intéressant de s'arrêter un instant au Dr Maurice Karnaugh[1], un ingénieur en télécommunications aux Laboratoires Bell. Les laboratoires de téléphone Bell, ou plus simplement les Laboratoires Bell, étaient à l'origine, la division de recherches et de développement de la Compagnie Bell des États-Unis. Ces laboratoires ont développé des technologies révolutionnaires allant des communications à distance aux commutateurs de téléphone en passant par le transistor. C'est aussi là que le système d'opération Unix de même que les langages de programmation C et C++ ont vu le jour. À leur apogée, les Laboratoires Bell avaient des sites un peu partout aux États-Unis. Aujourd'hui, ils sont concentrés à Murray Hill au New Jersey. Depuis 1996, ces laboratoires sont sous la juridiction de Lucent Technologies.

Le Dr Karnaugh a obtenu son doctorat en physique de l'Université de Yale en 1952. Par la suite, il a travaillé aux Laboratoires Bell jusqu'en 1966, puis s'est joint aux Laboratoires de recherche d'IBM jusqu'en 1993. Parallèlement à sa carrière de chercheur, il a aussi été professeur d'informatique à l'Institut Polytechnique de New York de 1980 à 1999. Il est actuellement retraité tout en étant toujours actif à *l'International Council for Computer Communication*, organisme qu'il a fondé en 1972.

La figure 3.1 reprend la première page du fameux article du Dr Karnaugh dont il sera question dans ce chapitre !

---

[1] Références :
http://en.wikipedia.org/wiki/Karnaugh_map
http://villemin.gerard.free.fr/Esprit/Karnaugh.htm
http://www.informationgenius.com/encyclopedia/g/ge/george_boole.html



The Map Method for Synthesis of Combinational Logic Circuits

M. KARNAUGH
NONMEMBER AIEE

THE SEARCH for simple abstract techniques to be applied to the design of switching systems is still, despite some recent advances, in its early stages. The problem in this area which has been attacked most energetically is that of the synthesis of efficient combinational that is, nonsequential, logic circuits.

While this problem is closely related to the classical one of simplifying logical truth functions, there are some significant differences. To each logical truth function, or Boolean algebraic expression, there corresponds a combinational circuit which may be constructed from a given set of appropriate components. However, minimization of the number of appearances of algebraic variables does not necessarily lead to the most economical circuit. Indeed, the criteria of economy and simplicity may vary widely for different types of components. A general approach to circuit synthesis must therefore be highly flexible. What is perhaps most to be desired is a simple and rapid technique for generating a variety of near-minimal algebraic forms for the designer's inspection.

Boolean algebra,[1] or the calculus of propositions, is a basic tool for investigation of circuits constructed from 2-valued devices. Its direct application to synthesis problems is, nevertheless, not completely satisfactory. The designer employing Boolean algebra is in possession of a list of theorems which may be used in simplifying the expression before him; but he may not know which ones to try first, or to which terms to apply them. He is thus forced to consider a very large number of alternative procedures in all but the most trivial cases. It is clear that a method which provides more insight into the structure of each problem is to be preferred. Nevertheless, it will be convenient to describe other methods in terms of Boolean algebra. Whenever the term "algebra" is used in this paper, it will refer to Boolean algebra, where addition corresponds to the logical connective "or," while multiplication corresponds to "and."

The minimizing chart,[2] developed at the Harvard Computation Laboratory, represents a step in the desired direction. It makes possible the fairly rapid derivation of near-minimal 2-stage forms. By a 2-stage form is meant a sum of products of the elementary variables, or else a product of sums of the elementary variables. These expressions may then be further reduced by algebraic factoring. The chief drawback to this method lies in the necessity of writing, and perhaps erasing, on a chart that, for n variables, contains 3^n entries. Thus, we must keep track of 1,021 entries for five variable problems and 4,096 entries for six variable problems.

E. W. Veitch[3] has suggested a method whereby results similar to those yielded by the minimizing chart can be obtained from an array containing only 2^n entries in a more rapid and elegant manner. The map method, which is explained in this paper, involves a reorganization of Veitch's charts, an extension to the use of 3-dimensional arrays, and some special techniques for diode and relay circuits.

Maps

Let the active and inactive conditions of the inputs to a combinational circuit be designated by assigning the values 1 and 0 respectively to the associated algebraic variables. An assignment of a simultaneous set of values to the n variables for a given problem will be called an input condition. There are 2^n possible input conditions.

For example, with only two variables, there are four input conditions. They may be represented graphically by the four squares in Fig. 1(A). Here, the values of variables A and B have simply been plotted along two perpendicular axes. It should be noted that squares which are adjacent, either horizontally or vertically, differ in the value of only one of the variables.

If Fig. 1(A) is cut along its horizontal midsection and the bottom half is rotated into line with the top, as in Fig. 1(B), then a representation of the input conditions for two variables is obtained along a single axis. Let us consider the squares at opposite ends of the row to be termed adjacent, as if it were inscribed on a cylinder. Then, as before, adjacent squares differ in the value of only one variable. Conversely, if two input conditions differ in the value assigned to just one of the variables, they are represented by adjacent squares.

If one also makes use of the vertical axis, one can represent the input conditions for three variables as in Fig. 2(A), and for four variables as in Fig. 2(B). In the latter case, opposite ends of each row or column should be considered adjacent, as though the figure were inscribed on a torus.

The labels on the diagrams may be simplified as shown in Fig. 3. The rows or columns within a bracket are those in which the designated variable has the value 1, while it is 0 elsewhere.

A combinational circuit of the type under consideration has a 2-valued output which is a function of the input condition. The synthesis problem may be said to begin with the specification of this functional dependence. Such information may be represented on a map as follows: Place a 1 in each square which represents an input condition for which the output is to have the value 1. The other squares may be imagined to contain zeroes.

Synthesis of 2-Stage Forms

Consider the function mapped in Fig. 4(A). Its algebraic realization is the product $A'BC'D$, where the primes indicate negation or complementation, for $A'BC'D=1$ if, and only if, $A=0$, $B=1$, $C=0$, and $D=1$.

Let us define a complete product to be a product in which each of the variables appears as one factor, either primed or

Fig. 1. Graphical representation of the input conditions for two variables
(A) Along two axes
(B) Along a single axis

Paper 53-217, recommended by the AIEE Communication Switching Systems Committee and approved by the AIEE Committee on Technical Operations for presentation at the AIEE Summer General Meeting, Atlantic City, N. J., June 15-19, 1953. Manuscript submitted March 17, 1953; made available for printing April 29, 1953.

M. Karnaugh is with the Bell Telephone Laboratories, Inc., Murray Hill, N. J.

The author wishes to express his indebtedness for many valuable suggestions, help, and encouragement to E. F. Moore, K. Goldschmidt, and W. Keister, all of the Bell Telephone Laboratories.

Figure 3.1 Page frontispice de l'article original de M. Karnaugh sur la minimisation des expressions logiques[2]

3.1 INTRODUCTION

Dans ce chapitre, nous allons nous attarder à la minimisation des expressions logiques par l'utilisation des tables de Karnaugh (aussi appelées *K-Map* dans les ouvrages américains). Les tables de Karnaugh abordent la minimisation des expressions logiques par une approche graphique, beaucoup plus facile d'emploi que la simplification algébrique des expressions logiques booléennes vues au chapitre précédent. Dans l'approche graphique par tables de Karnaugh, on voit directement les termes qui peuvent être associés ensemble et donc qui peuvent être simplifiés par mise en évidence comme ceci : $Y\overline{X}+YX=Y\cdot(\overline{X}+X)=Y\cdot 1=Y$, ce qui permet d'éliminer des variables (X, ici). Notons qu'évidemment les solutions trouvées sont identiques, que l'on prenne l'approche graphique ou l'approche algébrique.

[2] Karnaugh, Maurice. « The Map Method for Synthesis of Combinational Logic Circuits », *Trans. AIEE*, Pt I, n° 9, novembre 1953, p. 593-599.

Les méthodes que nous verrons dans ce chapitre permettent de minimiser, sur papier, des expressions ayant jusqu'à cinq variables, donc des tables de vérité à 32 lignes maximum ($2^5 = 32$).

Pour les expressions plus complexes, d'autres techniques existent. Ainsi, l'algorithme « Quine-McCluskey » effectue une réduction tabulaire qui permet de combiner les « mintermes » grâce au théorème de l'adjacence vu au chapitre 2 (c'est-à-dire en employant la relation : $PQ + P\overline{Q} = P$). Cet algorithme est employé pour des circuits ayant jusqu'à une vingtaine de variables. Au-delà, d'autres approches sont possibles, par exemple la méthode Espresso, l'algorithme EVE ainsi que des méthodes heuristiques particulières. Ces algorithmes sont employés dans différents langages de conception matérielle (exemple : ABEL) dont nous avions parlé au chapitre 1. En effet, il n'est pas vraiment concevable (au XXI^e siècle) de réaliser, sur papier, des minimisations de grande envergure. Les concepts que nous étudierons seront donc suffisants pour l'essentiel des cas auxquels seront confrontées les lectrices. De plus, la compréhension des concepts étudiés ici permettra une utilisation efficace des algorithmes précités pour les cas plus complexes.

Toutes ces techniques de minimisation ont été développées dans les années soixante et soixante-dix, à une époque où la réalisation des circuits numériques s'effectuait à partir de circuits intégrés SSI, MSI et LSI (chapitre 1). Il était donc essentiel de minimiser les expressions logiques pour limiter le nombre de circuits intégrés employés et donc la taille des circuits imprimés, la consommation électrique globale, etc. De nos jours, quoique la minimisation soit toujours importante, les circuits numériques sont généralement incorporés dans des circuits VLSI, par exemple des FPGA, des CPLD (chapitre 1). Comme ces circuits peuvent être « gigantesques » avec des densités allant au million de portes logiques embarquées, la minimisation de quelques portes logiques n'entraîne pas d'économie importante (sauf bien sûr si on doit passer à un modèle de circuit intégré supérieur). La conceptrice pourra dès lors considérer d'autres facteurs tels que la minimisation des aléas. À la fin de ce chapitre, nous aborderons d'ailleurs – brièvement – cette question.

Dans ce chapitre, nous allons donc détailler la méthode de minimisation par « cartes » aussi appelées tables de Karnaugh, une méthode élaborée en 1953. L'origine du présent chapitre remonte à la publication précitée de Maurice Karnaugh (figure 3.1). La première page de cette publication est reprise au début de ce chapitre.

3.2 Diagrammes de Venn et tables de Karnaugh

Afin d'introduire les tables de Karnaugh, nous pouvons nous référer aux diagrammes de Venn de la théorie des ensembles. En effet, ceux-ci illustrent très bien le principe employé. Considérons ainsi deux ensembles A et B contenus dans un univers de valeurs. Le cas le plus général est illustré à la figure 3.2 où l'on note la présence d'une intersection non nulle entre les ensembles A et B (représentée par la zone hachurée). L'ensemble des valeurs possibles est représenté par la frontière extérieure de cet univers alors que les frontières intérieures représentent les zones où une variable donnée sera vraie. Dans cette représentation, on distingue quatre zones (partie droite de la figure) :

- L'intersection entre A et B (notée $A \cdot B$ ou AB), ce qui signifie aussi que $A = 1$ (vraie) et $B = 1$ (vraie) ;
- La zone contenue dans A, mais hors B, notée $A \cdot \overline{B}$ (ou encore $A\overline{B}$ ou $A!B$), ce qui signifie aussi que $A = 1$ (vraie) et $B = 0$ (fausse) ;
- La zone contenue dans B, mais pas dans A, notée $\overline{A} \cdot B$ (ou encore $\overline{A}B$ ou $!AB$), ce qui signifie aussi que $A = 0$ (fausse) et B = 1 (vraie) ;
- Le reste, c'est-à-dire la zone hors A et B, notée $\overline{A} \cdot \overline{B}$ (ou encore $\overline{A}\,\overline{B}$ ou !A!B), ce qui signifie aussi que $A = 0$ (fausse) et $B = 0$ (fausse). Attention ici, $\overline{A} \cdot \overline{B} \neq \overline{A \cdot B}$. Pour s'en convaincre, il suffit de dresser les tables de vérité correspondantes ou d'appliquer la Loi de De Morgan vue au chapitre 2 : $\overline{A} \cdot \overline{B} \neq \overline{A \cdot B} = \overline{A} + \overline{B}$.

On notera ici qu'une barre (ou un point d'exclamation) représente la négation de la variable affectée.

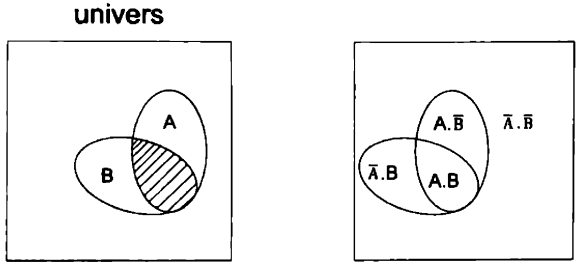


Figure 3.2 Illustration des tables de Karnaugh par un diagramme de Venn

En fait, au lieu de cercles, on peut redessiner ces quatre zones d'une façon plus structurée (et plus claire aussi) par des cases. La figure 3.3 illustre d'ailleurs le principe. On ne parle alors plus d'un diagramme de Venn, mais plutôt d'une table de Karnaugh.

	$A = 0$	$A = 1$
$B = 0$	$\overline{A}\,\overline{B}$	$A\overline{B}$
$B = 1$	$\overline{A}B$	AB

Figure 3.3 Représentation d'un diagramme de Venn sous forme de cases. Cette représentation est appelée table de Karnaugh.

On peut dès lors relier ces notions avec celles des tables de vérité introduites au chapitre 2. Ceci est illustré à la figure 3.4 dans les cas de deux variables A et B. En effet, nous avions vu que chaque ligne de la table de vérité correspond à une combinaison des

variables impliquées. Puisque nous avons ici deux variables, la table de toutes les combinaisons possibles est dressée avec $2^2 = 4$ lignes qui correspondent chacune aux quatre cases du diagramme de Venn de la figure 3.3. Chacune de ces combinaisons est appelée « minterme », comme nous l'avions vu à la section 2.4. Pris isolément, un « minterme » représente une combinaison de variables (un « terme ») qui ne peut être simplifiée, c'est donc un « terme minimum », d'où l'expression de minterme. Le tableau suivant rappelle les mintermes dans le cas d'une table de vérité à deux variables B et A :

B	A	minterme
0	0	$\overline{B}\,\overline{A}$
0	1	$\overline{B}A$
1	0	$B\overline{A}$
1	1	BA

Figure 3.4 Table de vérité pour deux variables B et A avec indication des quatre mintermes correspondants.

Exemple : Soit la table de vérité de deux variables A et B suivante. On demande de simplifier la sortie S.

rang	B	A	Sortie S
⓪	0	0	0
①	0	1	0
②	1	0	1
③	1	1	1

Figure 3.5

Avec une solution de type Somme de Produits (SdP) introduite au chapitre 2, on écrit directement :

$$S = BA + B\overline{A} = B \cdot (A + \overline{A}) = B$$

Ceci se réduit à sortie $S = B$ par simple mise en évidence. Représentons maintenant la sortie S sous la forme d'un diagramme de Venn avec deux ensembles A et B, et aussi sous la forme d'une table de Karnaugh (figure 3.6). La sortie S contient deux mintermes actifs, et seulement ceux-ci sont donc présents dans le diagramme de Venn et mis à 1 dans la table de Karnaugh correspondante. On note que l'ensemble B est entièrement hachuré. Ceci nous amène à entourer les deux « 1 » dans la table de Karnaugh, ce qui recouvre alors complètement la variable B, suggérant ainsi que $S = B$, qui est la solution trouvée par simplification algébrique.

Cet exemple nous donne un aperçu de ce que sera la simplification par table de Karnaugh, par combinaison de cases adjacentes. Les détails du mécanisme de simplification seront abordés plus loin.

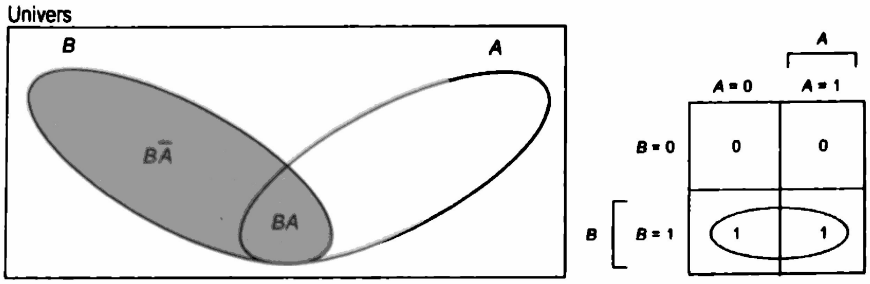


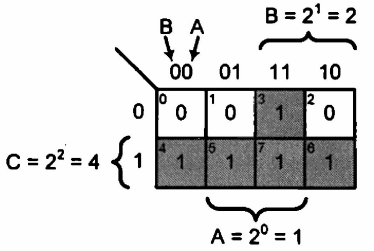
Figure 3.6 Solution graphique de l'exemple par diagramme de Venn à gauche, et table de Karnaugh à droite

Exemple : (Avec trois variables) Réalisons un distributeur de bonbons « charitable ». Le distributeur accepte trois types de pièces de monnaie, soit des 5 ¢, 10 ¢ et 25 ¢. Un bonbon est donné à la déposante dès qu'une somme minimale de 15 ¢ est fournie. Par contre, la monnaie n'est pas rendue, la différence constituant ainsi un don de charité pour la réfection de la toiture de l'église.

Solution : On a donc trois variables *C*, *B*, *A* correspondant donc aux trois types de pièce de monnaie (*C* pour les 25 ¢, *B* pour les 10 ¢ et *A* pour les 5 ¢). Notons que généralement la lettre ayant le plus haut rang dans l'ordre alphabétique correspond à la variable la plus significative. Donc ici *C* est la variable *MSB* et *A* la variable *LSB*. Si la sortie du distributeur est à « 1 », un bonbon est fourni. Dans le cas contraire, pas de bonbon.

On peut dès lors dresser la table de vérité. On donne aussi la table de Karnaugh à trois variables (nous reviendrons plus tard sur son format, tenons-le maintenant pour acquis). On note que les mintermes 3 à 7 sont à 1 ; c'est la solution SdP canonique.

25¢ C	10¢ B	5¢ A	Sortie
0 0	0	0	0
1 0	0	1	0
2 0	1	0	0
3- 0	1	1	1
4- 1	0	0	1
5- 1	0	1	1
6- 1	1	0	1
7- 1	1	1	1
MSB		LSB	



Rappel : MSB = *Most Significant Bit*
LSB = *Least Significant Bit*

Figure 3.7

Ici, on a les poids relatifs suivants :

$C \leftrightarrow 2^2 = 4$ – *MSB*

$B \leftrightarrow 2^1 = 2$

$A \leftrightarrow 2^0 = 1$ – *LSB*

On veut maintenant simplifier cette solution SdP :

$$S = m3 + m4 + m5 + m6 + m7$$

À cette fin, on peut grouper les 1 qui sont adjacents dans la table de Karnaugh de façon à obtenir les plus grands groupes possibles. De façon « intuitive » (pour le moment), on voit que la sortie S vaut alors $C + BA$. Vérifions maintenant ce résultat par simplification algébrique.

Simplifions :

$$\begin{aligned}
S &= \overline{C}BA + C\overline{A}\,\overline{B} + C\overline{B}A + CB\overline{A} + CBA \\
&= C\left[\overline{A}\,\overline{B} + \overline{B}A + B\overline{A} + BA\right] + \overline{C}BA \\
&= C\left[\overline{B}\left[\underbrace{\overline{A} + A}_{1}\right] + B\left[\underbrace{\overline{A} + A}_{1}\right]\right] + \overline{C}BA \\
&= C\left[\underbrace{\overline{B} + B}_{1}\right] + \overline{C}BA \qquad (\textit{Rappel}: X + \overline{X}Y = X + Y) \\
&= C + \overline{C}AB = C + AB \qquad \textit{Adsorption} \\
&= C + BA
\end{aligned}$$

Ce résultat pouvait « se voir » directement sur la table de Karnaugh précédente avec les cases ombragées tel qu'il a été indiqué plus tôt.

Il convient maintenant de commencer à formaliser les simplifications intuitives que nous venons de voir dans les exemples précédents.

On définit une table de Karnaugh comme étant un tableau carré ou rectangulaire de 2^N cases où chaque case représente un minterme de la fonction correspondante de N variables, et contient la valeur « 1 » s'il s'agit d'un minterme *actif* dans la fonction, et « 0 » si ce minterme n'est pas actif.

Disposition des variables

La figure 3.8 montre la disposition des variables que nous adopterons ici, et aussi tous les mintermes qui correspondent aux différentes cases, pour les cas de deux, de trois et de quatre variables. Bien évidemment, d'autres dispositions de variables sont aussi valables, par contre il est fort utile d'adopter une disposition « universelle » (tout comme le rouge est universellement employé dans les feux de circulation pour indiquer qu'il faut s'arrêter). Dans la figure, le rang de chaque minterme est aussi donné, par exemple le minterme *CBA* vaut 7 puisque C a un poids de $4 = 2^2$, B de $2 = 2^1$ et A de $1 = 2^0$. La somme de 4 + 2 + 1 donne bien 7. Quant à $C\overline{B}A$, il vaut 5 puisque C a un poids de $4 = 2^2$, ; B est absent du calcul, ce qui est signifié par la barre ; et A vaut $1 = 2^0$. La somme de 4+1 donne bien 5. Finalement, le minterme $D\overline{C}B\overline{A}$ vaut 10 puisque D a un poids de $8 = 2^3$; C et A sont absents du calcul, ce qui est signifié par les barres ; et B

vaut 2 = 2^1. La somme de 8 + 2 donne bien 10. Tous les autres mintermes se calculent de la même façon.

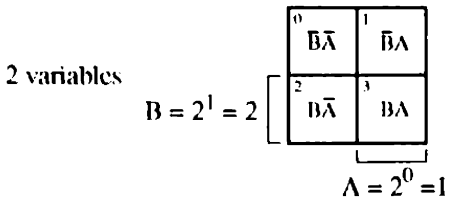


Figure 3.8 Cas de deux variables : *B*, *A* ; avec *B* la variable la plus significative.

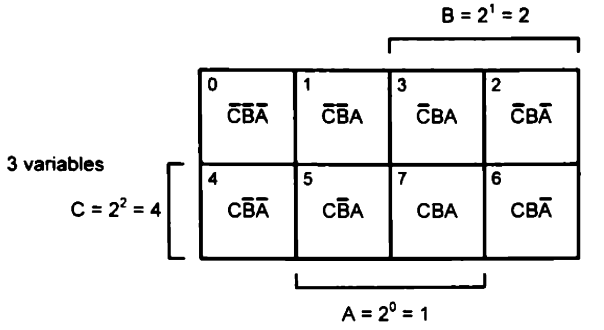


Figure 3.9 Cas de trois variables : *C*, *B*, *A* ; avec *C* la variable la plus significative.

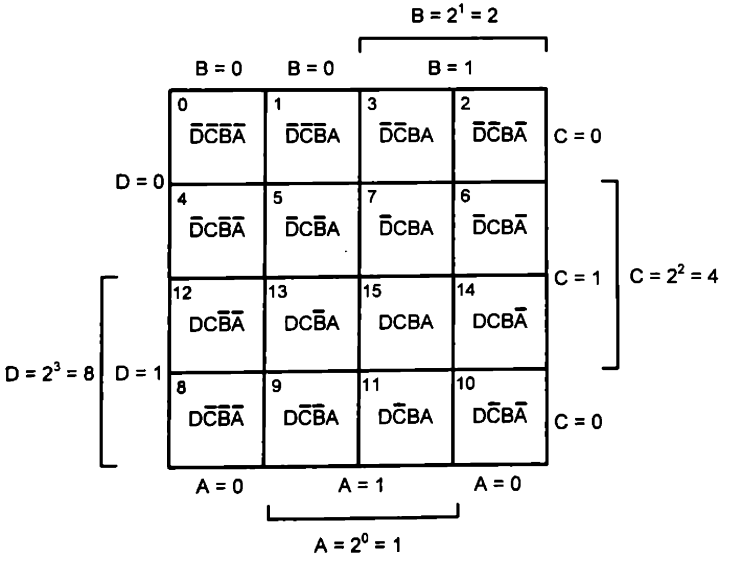


Figure 3.10 Cas de quatre variables : *D*, *C*, *B*, *A* ; avec *D* la variable la plus significative

Tel qu'il a été mentionné plus haut, il y a différentes façons de distribuer les variables dans les tables, par exemple en employant les diagrammes de Veitch (figure 3.11).

Néanmoins, l'usage des tables de Karnaugh étant le plus répandu, nous l'adopterons, d'autant plus qu'on peut facilement en étendre la notation à plus de quatre variables.

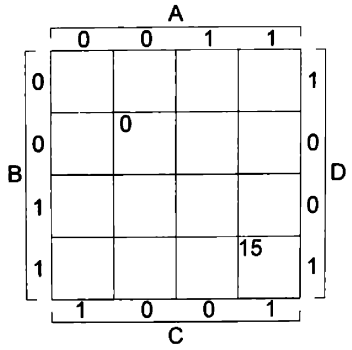


Figure 3.11 Diagramme de Veitch

Une autre considération importante est l'ordre des variables pour un minterme donné. Ici, on remarque que nous avons disposé les variables de poids binaire le plus élevé au moins élevé, donc on aura le minterme *DCBA* et non *ABCD* (avec *D* comme variable). Cette convention est arbitraire puisque *DCBA* = *ABCD* (selon la commutativité du produit booléen (section 2.5)), mais cependant utile pour les simplifications, la comparaison des solutions, etc.

En observant la table de Karnaugh à quatre variables, on remarque que, d'une case à l'autre, les mintermes ne diffèrent que par une seule variable, toutes les autres restant identiques. Ceci est vrai de haut en bas, mais aussi de gauche à droite et aussi pour les bordures. En effet, il faut imaginer que la table de Karnaugh se replie sur elle-même (*wrap around* en anglais) de sorte que la case 10 (pour la table de Karnaugh à quatre variables) voisine les cases 11 et 14, mais aussi avec les cases 2 et 8. Ainsi, chaque case a quatre voisins, et, pris deux à deux, ces mintermes voisins ne diffèrent entre eux que d'une seule variable. Par exemple, prenons les cases voisines 10 et 8 : les deux mintermes sont identiques sauf pour la variable B qui apparaît complémentée ($\overline{B}$) à la case 8. Nous verrons au paragraphe suivant que ceci est d'une importance fondamentale puisqu'on y trouve le principe de la simplification avec l'axiome $X + \overline{X} = 1$!

Exemple : Indiquer les quatre voisins de la case 0.

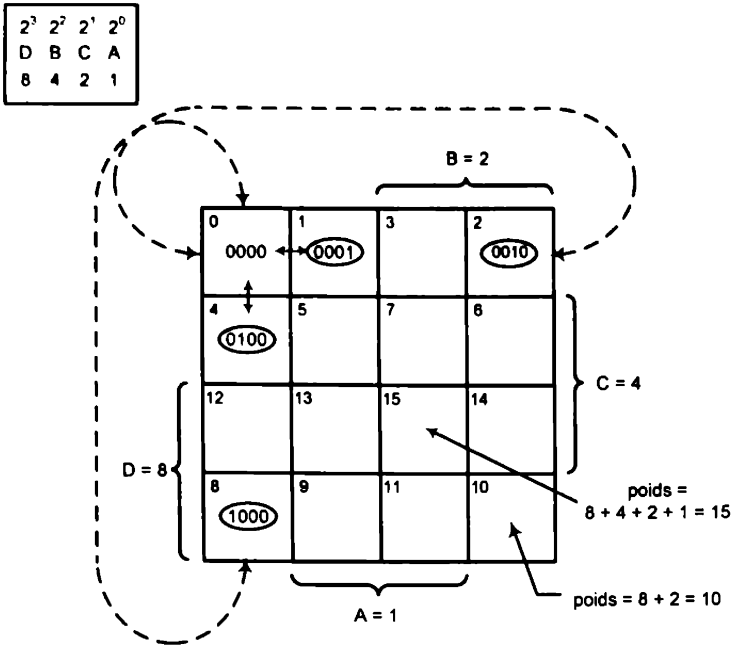


Figure 3.12

Solution : Les quatre voisins de la case 0 sont les cases 1, 2, 4, 8. À cette figure, on détaille aussi le poids binaire de quelques cases.

3.3 SIMPLIFICATION GRAPHIQUE DES TABLES DE KARNAUGH : SOLUTION EN RASSEMBLANT LES 1

CAS DE DEUX CASES VOISINES

L'idée de base de la simplification booléenne graphique est la suivante : La simplification d'une expression algébrique booléenne revient à éliminer le plus grand nombre de mintermes possible en regroupant ceux-ci de sorte qu'une expression plus compacte soit obtenue. Il s'agit donc de regrouper ce qui est possible, par mise en évidence des variables communes et application de l'axiome vu au chapitre 2: $X + \overline{X} = 1$.

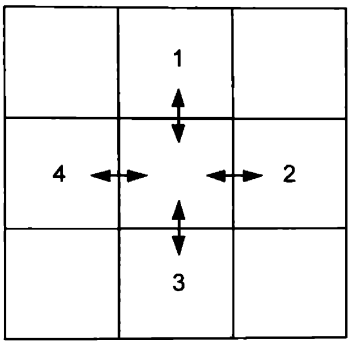
Exemple : Simplifions la fonction S composée des mintermes voisins $m3$ et $m7$ (figure 3.9, illustrant la table de Karnaugh à trois variables):

$$S = m3 + m7 = \overline{C}BA + CBA = BA \cdot (\underbrace{\overline{C} + C}_{1}) = BA$$

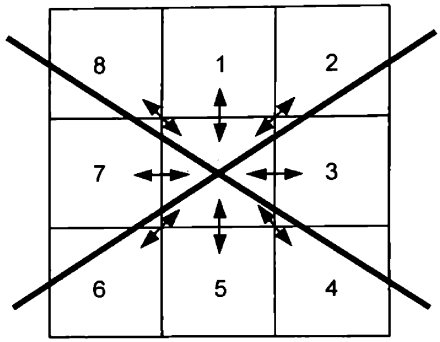
Solution : Comme on le voit, une simple mise en évidence de BA permet d'éliminer la variable C grâce à l'axiome vu au chapitre 2 ($X + \overline{X} = 1$). Ceci nous illustre le principe de simplification des tables de Karnaugh pour la simplification des expressions algébriques d'une façon graphique, et donc facile, par simple combinaison de cases voisines : chaque

fois qu'on regroupe deux cases voisines, on élimine une variable. Évidemment, il faut faire bien attention de regrouper les *cases voisines en croix (haut-bas, gauche-droite), et non en oblique.* Ceci est aussi connu sous l'appellation de *voisinage à quatre voisins* par opposition au voisinage à huit voisins, qui inclut justement les cas obliques (figure 3.13). Par exemple (figure 3.10 illustrant la table de Karnaugh à quatre variables), les cases 7 (minterme $\bar{D}CBA$) et 13 (minterme $DC\bar{B}A$) ne sont pas ici considérées voisines puisque les mintermes correspondants diffèrent par plus qu'une variable (deux variables différentes ici : D et B) empêchant la simplification par mise en évidence et application de l'axiome ($X + \bar{X} = 1$).

Voisinage à quatre voisins pour la case centrale **employé** en simplification graphique



Voisinage à huit voisins pour la case centrale **non employé** en simplification graphique



Note : Le cas inclut aussi le débordement sur les côtés (*wrap around* (figure 3.19))

Figure 3.13

CAS DE QUATRE CASES VOISINES

Nous venons de voir le cas de la simplification résultant du regroupement de deux cases voisines, étudions maintenant le cas du regroupement de quatre cases voisines. Soit donc la situation de la figure 3.14 où l'on cherche à regrouper les quatre mintermes de la zone hachurée qui correspondent à la sortie S. Qu'est-ce que ces quatre cases ont en commun ?

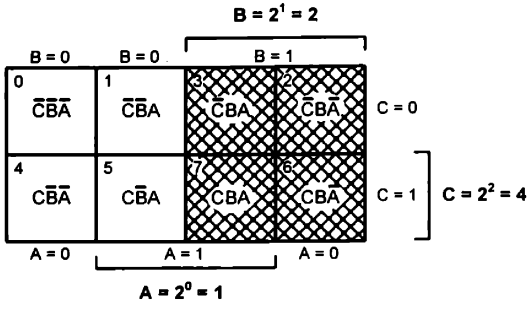


Figure 3.14

En fait, « l'œil averti » remarque que ces quatre cases recouvrent totalement la variable B.

En appliquant les théorèmes vus précédemment, on obtient le même résultat (la sortie S vaut B), selon :

$$S = m3 + m2 + m7 + m6$$

$$S = \overline{C}BA + \overline{C}B\overline{A} + CBA + CB\overline{A}$$

$$= B \cdot \left(\overline{C}A + \overline{C}\,\overline{A} + CA + C\overline{A}\right)$$

$$= B \cdot \left[\overline{A} \cdot \underbrace{\left(\overline{C} + C\right)}_{1} + A \cdot \underbrace{\left(C + \overline{C}\right)}_{1}\right] = B \cdot \left[\underbrace{\overline{A} + A}_{1}\right] = B$$

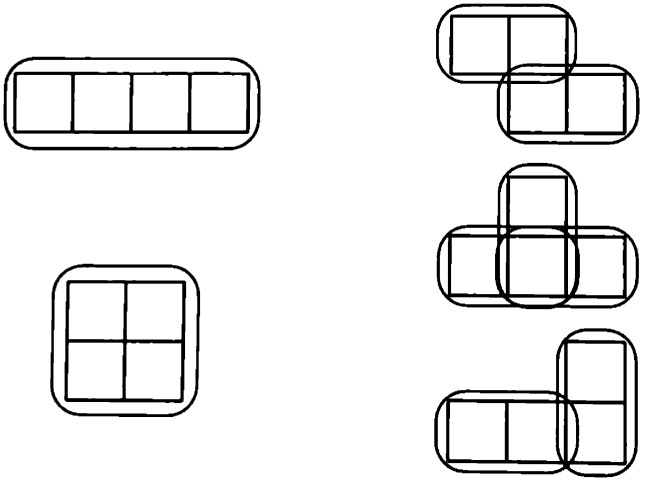
On pouvait aussi constater ce résultat directement sur la table de Karnaugh puisque les quatre cases impliquées recouvrent complètement la variable B. On peut ainsi généraliser en quelque sorte ce qu'on a déterminé plus haut d'une façon intuitive : *chaque fois que l'on regroupe 2^N cases, on élimine N variables parmi les mintermes des cases regroupées.* Ainsi, ici, le regroupement de quatre cases a permis d'éliminer deux variables ($4 = 2^2$) : C et A. *Les groupes de case doivent bien sûr être voisins au sens du voisinage à quatre voisins* vu à la section précédente (les cases à regrouper doivent être voisines en croix et non en oblique). On peut maintenant répondre à la question suivante :

Quelles cases peut-on regrouper ?

Réponse : Un groupe de cases en forme de *carré* ou de *rectangles* de $\mathbf{2^N}$ **cases**. Ce groupe permettra d'éliminer N variables, où N est un entier, c'est-à-dire que $N = 0, 1, 2, 3$, etc.

Bien sûr, on peut déterminer plusieurs groupes dans une table de Karnaugh donnée ; qui plus est, les groupes peuvent se chevaucher.

Voici d'ailleurs, illustré à la figure 3.15, des groupes de quatre cases et la façon de les réunir entre elles :



Groupe de quatre, donc élimination de deux variables

Groupes de deux, donc élimination d'une variable

Figure 3.15

Les simplifications indiquées (en encerclant les cases regroupées) permettent une réduction d'une ou de deux variables selon que l'on ait pu réaliser des groupes de deux ou de quatre cases.

Exemple : Soit la fonction $S(C,B,A$; $C = MSB)$ définie par la table de Karnaugh suivante :

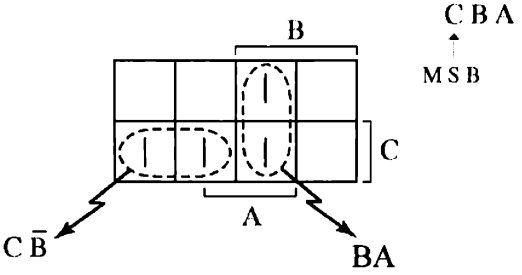


Figure 3.16

Solution : D'une façon détaillée, on voit que S correspond à quatre mintermes :

$$S = \underbrace{(C\,\bar{B}\,\bar{A} + C\,\bar{B}\,A)}_{\text{On peut éliminer } A} + \underbrace{(C\,B\,A + \bar{C}\,B\,A)}_{\text{On peut éliminer } C}$$

Dans chaque groupe, selon l'axiome $X + \bar{X} = 1$, on peut éliminer les variables indiquées, ce qui donne alors :

$$S = C\bar{B} + BA$$

Maintenant, par simplification graphique directe des quatre cases où les mintermes sont actifs (c'est-à-dire à 1), on réussit à faire deux groupes de deux cases (entourés en pointillés) ce qui nous donne directement les deux termes simplifiés : $C\bar{B}$ et BA (par élimination d'une variable). La fonction (et donc la sortie du circuit correspondant) vaut alors $C\bar{B} + BA$. On a retrouvé le résultat précédent.

GROUPEMENT DE CASES

Il est très important de pouvoir « lire » directement sur les tables de Karnaugh les termes simplifiés qui correspondent à des groupements de 2^N cases selon les dispositions vues précédemment. La lecture d'un groupement de cases revient à déterminer l'état (actif ou non) des variables qui recouvrent le groupement donné. Le terme simplifié final sera le produit de toutes les variables impliquées. On note que celles-ci seront inversées si la variable n'est pas couverte (c'est-à-dire à l'état 0). Par ailleurs, si, pour un groupement donné, une variable est couverte dans une case (ou des cases) et n'est pas couverte dans une autre case (ou d'autres cases), elle n'apparaîtra *pas* dans le terme simplifié final puisqu'elle aura été éliminée par mise en évidence et utilisation de l'axiome $X + \bar{X} = 1$. Finalement, on se rappelle que, dans un groupement de 2^N cases, N variables pourront ainsi être éliminées.

Exemple : Soit la fonction S représentée par la table de Karnaugh suivante. Simplifier graphiquement *S*.

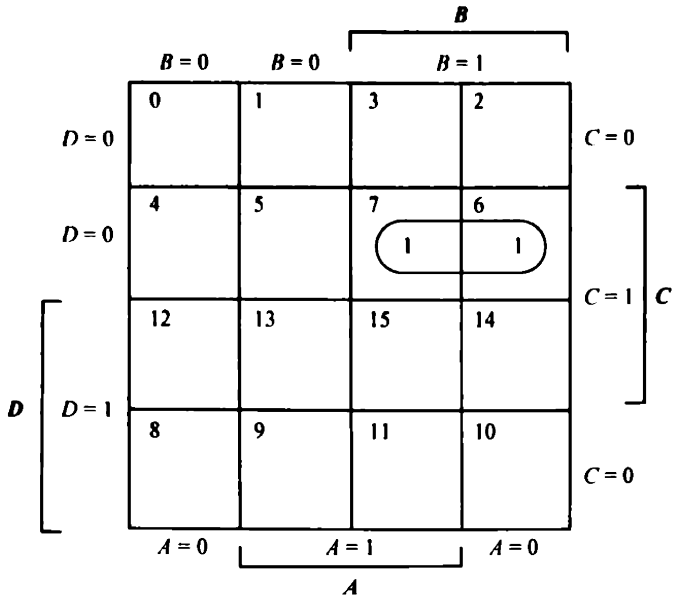


Figure 3.17

Solution : La fonction *S* comprend deux mintermes (*m*6 et *m*7). Comme ils sont voisins (au sens du voisinage à quatre voisins discuté précédemment), on peut les grouper tel qu'il a été indiqué sur la figure. Le minterme simplifié correspondant s'écrit alors directement. Le groupement est dans la zone où la variable *D* est inversée ($D=0$) ; $\overline{D}$ apparaîtra donc. Les variables *C* et *B* apparaîtront aussi puisque nous sommes dans la zone couverte par ces variables. Par contre, le groupement chevauche les zones où $A = 0$ et $A = 1$; celle-ci sera donc exclue. Le terme résultant sera donc $\overline{D}CB$. On peut d'ailleurs s'en convaincre facilement avec (se référer à la figure 3.10) :

$$S = m7 + m6$$

$$S = \overline{D}CBA + \overline{D}CB\overline{A}$$

$$= \overline{D}CB\left(A + \overline{A}\right)$$

$$= \overline{D}CB\left[\underbrace{\overline{A} + A}_{1}\right] = \overline{D}CB$$

Exemple : Soit la fonction S représentée par la table de Karnaugh suivante. Simplifier graphiquement S.

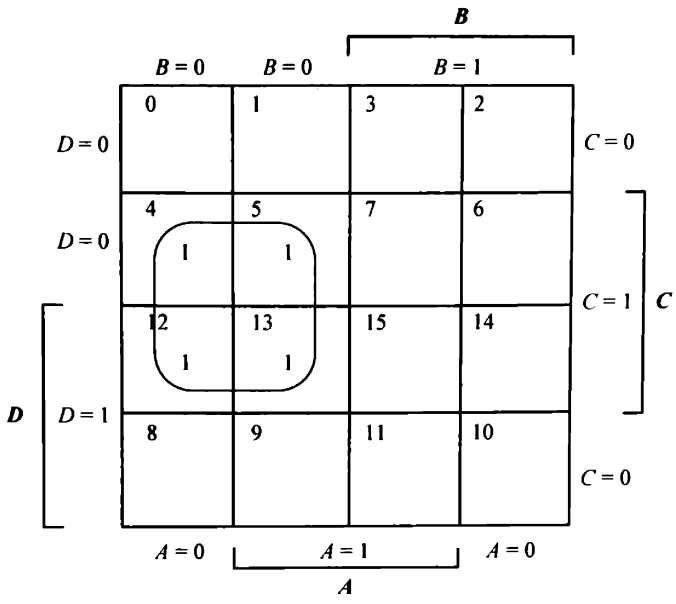


Figure 3.18

La fonction S comprend quatre mintermes ($m4$, $m5$, $m12$ et $m13$). Comme ils sont voisins (selon le voisinage à quatre voisins), on peut les grouper tel qu'il a été indiqué sur la figure. Le terme simplifié correspondant peut alors s'écrire directement. Le groupement est dans la zone où la variable C est active ($C = 1$) ; C apparaîtra donc. Le groupement est dans la zone où la variable B est inversée ($B = 0$), $\overline{B}$ apparaîtra donc. Le groupement chevauche les zones où $A = 0$ et $A = 1$; celle-ci sera donc exclue, de même pour D. Le terme résultant sera donc $C\overline{B}$. On a groupé $4 = 2^2$ cases ; deux variables ont donc été éliminées dans le terme simplifié final (D et A).

Cas des cotes

Tel qu'il a été mentionné précédemment, les tables de Karnaugh ont la propriété de débordement sur les côtés : les cases des coins pourront ainsi se grouper, tel qu'il a été illustré à la figure 3.19, où l'on voit que la présence de quatre mintermes actifs[3] dans les coins conduit à leur groupement puisque ces quatre termes sont voisins (au sens du voisinage à quatre voisins). Le groupement est dans la zone où la variable C est inversée ($C = 0$), $\overline{C}$ apparaîtra donc. Le groupement est dans la zone où la variable A est inversée ($A = 0$), $\overline{A}$ apparaîtra donc. Le groupement chevauche les zones où $B = 0$ et $B = 1$ elle sera donc exclue, de même pour D.

Le groupement de quatre cases de la figure 3.19 permet donc d'éliminer deux variables ($2^2 = 4$) et fournit une fonction simplifiée égale à $\overline{C}\,\overline{A}$.

[3] C'est-à-dire à un dans la table de vérité de la fonction correspondante.

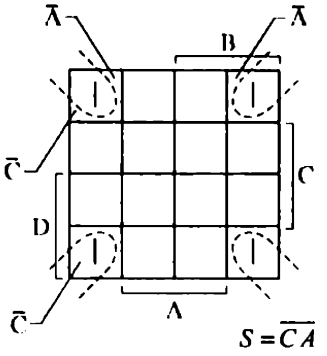


Figure 3.19

Les autres débordements possibles sont les côtés gauche et droit, et supérieur et inférieur, comme on le verra à l'exemple suivant.

Exemple : Soit la fonction S représentée par la table de Karnaugh suivante. Simplifier graphiquement S.

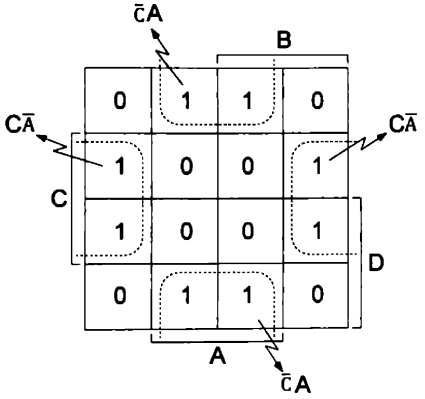


Figure 3.20

Solution : On note directement que $S = \bar{C}A + C\bar{A} = C \oplus A$ puisqu'on peut grouper les cases en deux groupements de quatre cases chacun. Pour $\bar{C}A$, le groupement se trouve dans les zones où la variable C est inversée et la variable A est active. Pour $C\bar{A}$, il se trouve dans les zones où la variable C est active et la variable A est inversée. Par contre, les variables D et B n'apparaissent pas dans les groupements puisqu'elles chevauchent ceux-ci.

TABLES DE KARNAUGH ET FONCTIONS OU-EXCLUSIF

Les portes OU-exclusif, quoique relativement coûteuses, en matière de quincaillerie (puisqu'elles sont issues de la combinaison de portes primitives), offrent des possibilités intéressantes de groupements dans les tables de Karnaugh. La figure 3.21 montre d'ailleurs les quatre cas possibles de dispositions que l'on peut obtenir avec trois variables *C*, *B*, *A*. À noter que ces mêmes patrons sont généralisables dans le cas de portes OU-exclusif à plus (ou moins) de variables, donc à deux, trois, quatre, et plus variables. A la figure 3.21, pour le cas $C \oplus A$, il est intéressant de noter que l'approche Somme de Produits directe conduit à une solution à 7 portes doubles simples alors que la solution en OU-exclusif correspond à 3 portes doubles simples seulement. Il y a donc un avantage indéniable à considérer les simplifications obtenues de l'usage du OU-exclusif.

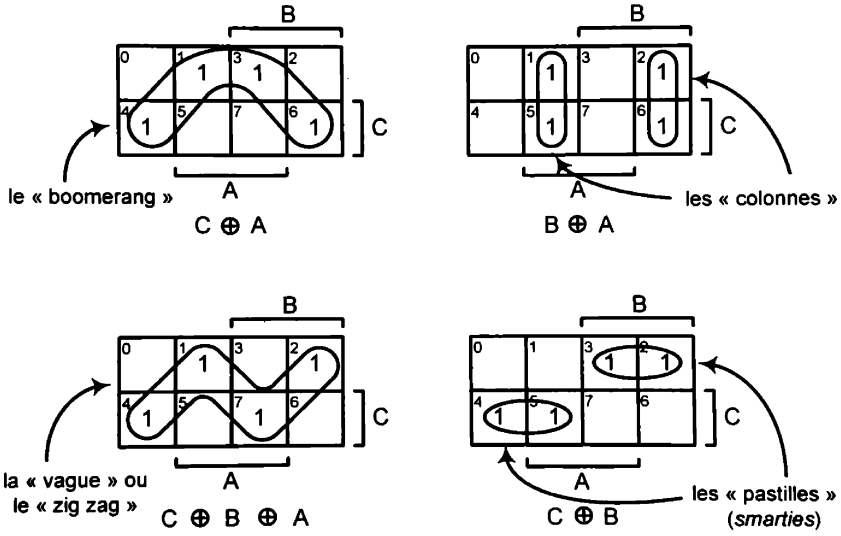


Figure 3.21

La table de vérité suivante reprend les quatre cas illustrés :

	C	*MSB* *B*	*A*	$C \oplus A$	$C \oplus B \oplus A$	$B \oplus A$	$C \oplus B$
⓪	0	0	0	0	0	0	0
①	0	0	1	1	1	1	0
②	0	1	0	0	1	1	1
③	0	1	1	1	0	0	1
④	1	0	0	1	1	0	1
⑤	1	0	1	0	0	1	1
⑥	1	1	0	1	0	1	0
⑦	1	1	1	0	1	0	0

Figure 3.22

Pour le cas $C \oplus B \oplus A$, on constate que le patron est en « zigzag ». C'est un point intéressant puisqu'une solution directe recourt à quatre mintermes ($m1$, $m2$, $m4$, $m7$). Il convient donc de se le rappeler lorsque la situation se présentera.

Exemple : Soit la fonction S suivante exprimée par la table $m(5,6)$. On demande de simplifier S.

Solution :

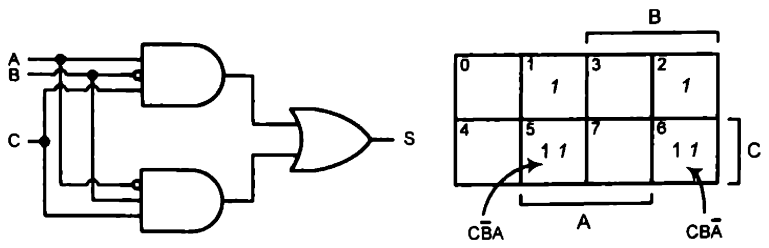


Figure 3.23

Pris isolément, ces deux mintermes ($m5$ et $m6$) ne peuvent pas être groupés puisqu'ils ne sont pas voisins ; la solution de type « SdP canonique » est donc $S = C\bar{B}A + CB\bar{A}$. Le circuit correspondant est donné à gauche avec trois portes. Pour ce qui est du nombre de portes doubles élémentaires, il équivaut à cinq portes doubles (une porte triple pouvant être remplacée par deux portes élémentaires tel qu'il a été vu au chapitre 2).

Une autre approche consiste à employer une porte OU-exclusif. On a vu plus haut que la combinaison $B \oplus A$ donnait un patron en « colonnes » dans la table de Karnaugh. La fonction S n'a toutefois que la moitié inférieure du patron en « colonnes » (illustré en italique dans la figure 3.23). Les deux mintermes de la fonction S représentent en fait la fonction $S = (B \oplus A) \cdot \bar{C}$ (soit seulement la partie du dessous des « colonnes », partie obtenue avec $\bar{C}$. Le schéma du circuit correspondant est donné plus bas. Ce circuit ne comprend que deux portes. Par contre, la porte OU-exclusif se décompose en trois portes doubles élémentaires (section 1.14). On obtient donc un total de quatre portes doubles élémentaires pour cette solution.

La solution avec porte OU-exclusif semble donc consommer moins de logique (si on considère la porte OU-exclusif comme une porte double élémentaire), par contre il faudrait aussi considérer les délais de propagation liés à ces deux solutions pour déterminer la meilleure approche. La porte ou-exclusif ayant deux étages, les délais associés à cette solution seront plus importants sans compter non plus les aléas qui seront abordés en fin de chapitre.

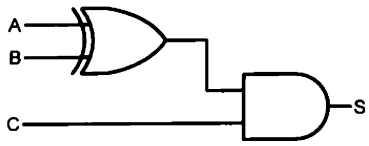


Figure 3.24

Notons néanmoins que l'utilisation de portes XOR est donc parfois avantageuse.

3.4 SOLUTION EN RASSEMBLANT LES 0

Jusqu'à présent, notre étude des tables de Karnaugh a porté sur des solutions rassemblant les mintermes actifs (donc à 1) selon l'approche « Somme de Produit » (SdP) introduite au chapitre 2 (mintermes symbolisés par m). L'approche « Produit de Sommes » (PdS) est bien sûr aussi possible, particulièrement si la table de Karnaugh considérée ne contient que quelques 0 et beaucoup de 1. On se rappelle qu'avec l'approche PdS, il s'agit de réaliser le produit booléen des Maxtermes actifs (Maxtermes symbolisés par *M*). On savait aussi (section 2.9) que $M_i = \overline{m_i}$. Si, dans une table de Karnaugh donnée peu de 0 sont présents, deux choix s'offrent à nous ; on peut :

1 réaliser un PdS, tel qu'il a été vu au chapitre 2 (c'est la solution en rassemblant les zéros) ;

2 inverser la table de Karnaugh (donc remplacer les 0 par des 1, et inversement) et réaliser une expression SdP (après l'inversion, c'est la solution en rassemblant les 1). Évidemment, la réponse finale devra être inversée pour correspondre à l'expression de départ selon l'axiome : $\overline{\overline{X}} = X$. On note que cette approche est souvent considérée comme *plus facile* car l'obtention directe des Maxtermes est parfois perçue comme ardue par certaines (section 2.9).

Les exemples suivants vont illustrer ces deux méthodes :

Exemple : Soit la fonction *S* représentée par la table de Karnaugh suivante. On demande de fournir l'expression simplifiée de *S* :

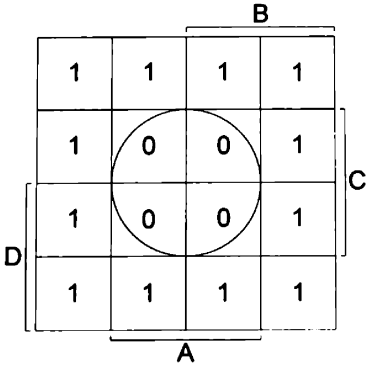


Figure 3.25

Solution :

- *Première possibilité* : Réalisons un Produit de Sommes. On voit que l'on peut entourer directement les quatre cases centrales (qui contiennent des 0). Ce groupement de 0 est un Maxterme des variables *C* et *A*, qui apparaîtront inversées (puisque c'est un groupement de 0 et que l'on est dans la zone où les variables sont actives). Les variables *D* et *B* sont exclues puisqu'elles chevauchent le groupement (c'est-à-dire que leurs valeurs directes et inverses sont impliquées). On se rappelle qu'un

Maxterme est une somme de variables qui donne zéro pour une combinaison donnée de celles-ci (section 2.9) ; on obtient alors directement que $S = \overline{C} + \overline{A}$.

- *Seconde possibilité* : On synthétise la fonction inversée $\overline{S}$ ce qui donne :

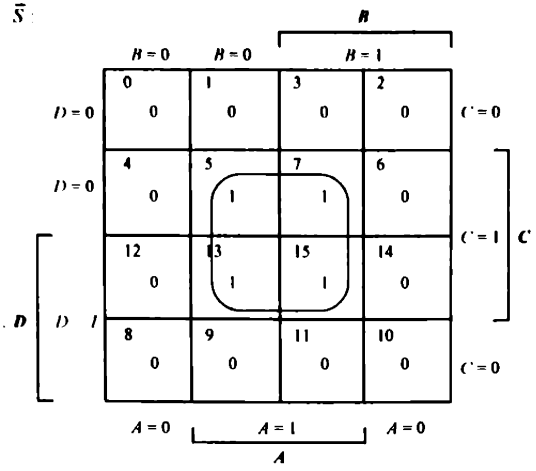


Figure 3.26

La solution en fonction de SdP (Somme de Produits) s'obtient directement :

$$\overline{S} = CA$$

$$\overline{\overline{S}} = \overline{CA} = \overline{C} + \overline{A}$$

donc $S = \overline{C} + \overline{A}$

On retrouve la même expression ; ce qui est normal.

Exemple : Soit la fonction S représentée par la table de Karnaugh suivante. On demande de fournir l'expression simplifiée de S :

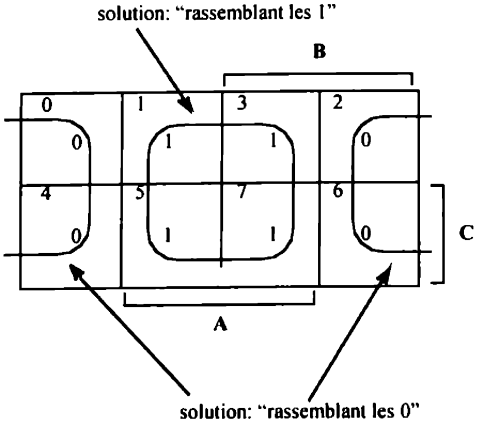


Figure 3.27

Solution :

- *Première possibilité (solution rassemblant les 0)* : Réalisons un PdS (Produit de Sommes). On voit que l'on peut entourer directement les quatre cases des extrémités (qui contiennent des 0). Ce groupement de zéro est un Maxterme de la variable A qui apparaîtra directement (puisque c'est un groupement de 0 et que l'on est dans la zone où la variable est inactive). Les variables C et B sont exclues puisqu'elles chevauchent le groupement (c'est-à-dire que leurs valeurs directes et inverses sont impliquées). On se rappelle qu'un Maxterme est une somme de variables ; on obtient alors directement que $S = A$.
- *Seconde possibilité (solution rassemblant les 1)* : On synthétise la fonction S directement (en rassemblant les 1), ce qui donne : $S = A$.

Les deux possibilités aboutissent bien sûr au même résultat.

Exemple : Soit la fonction S représentée par la table de Karnaugh suivante. On demande de fournir l'expression simplifiée de S :

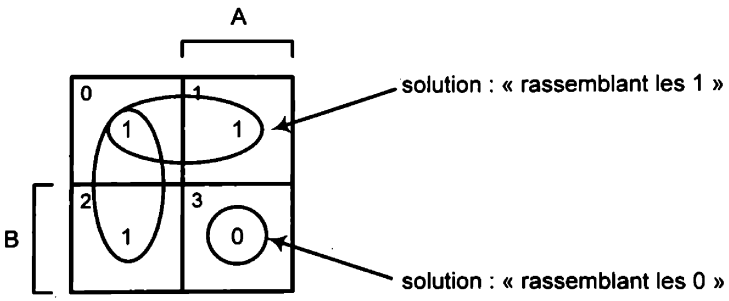


Figure 3.28

Solution :

- *Première possibilité (solution rassemblant les 0)* : Réalisons un Produit de Sommes en entourant l'unique 0 présent qui est un Maxterme des variables A et B, qui apparaîtront inversées dans celui-ci (puisque c'est un groupement de 0 et que l'on est dans la zone où les variables sont actives). On se rappelle qu'un maxterme est une somme de variables ; on obtient alors directement que $S = \overline{B} + \overline{A}$.
- *Seconde possibilité (solution rassemblant les 1)* : On synthétise la fonction S directement, ce qui donne la Somme de Produits (selon les deux groupements $\overline{B}$ et $\overline{A}$ indiqués sur la figure) : $S = \overline{B} + \overline{A}$.

Les deux possibilités aboutissent bien sûr au même résultat.

3.5 LES CHOIX « SANS IMPORTANCE » DANS LES TABLES DE KARNAUGH

Certaines tables de vérité (et donc de Karnaugh) possèdent des termes qui peuvent être soit « 1 », soit « 0 ». Ces termes seront indiqués par des « X » dans les tables de Karnaugh puisque, au départ, leur valeur n'est pas définie. On les appelle ces termes « les termes

sans importance » (« don't care » en anglais). Ils correspondent généralement à des cas qui ne peuvent pas survenir dans les tables de vérité. Par exemple si deux interrupteurs correspondants aux variables B et A ne peuvent jamais être enfoncés simultanément, alors le minterme $B \cdot A$ est un terme sans importance dans la table de vérité (B, A). Sa présence peut être carrément ignorée puisqu'il ne se manifestera jamais et qu'il ne participera donc pas à l'expression finale. La présence de ces termes est fort avantageuse et on peut en tirer profit lors des simplifications.

Considérons par exemple la table de Karnaugh $m(5, 6, 7, 9, 10, 11, 13, 14, 15)$, dans laquelle la ligne du haut (les mintermes $m0$ à $m3$) représente des termes « sans importance », et exerçons-nous à la simplifier. Trois solutions sont possibles :

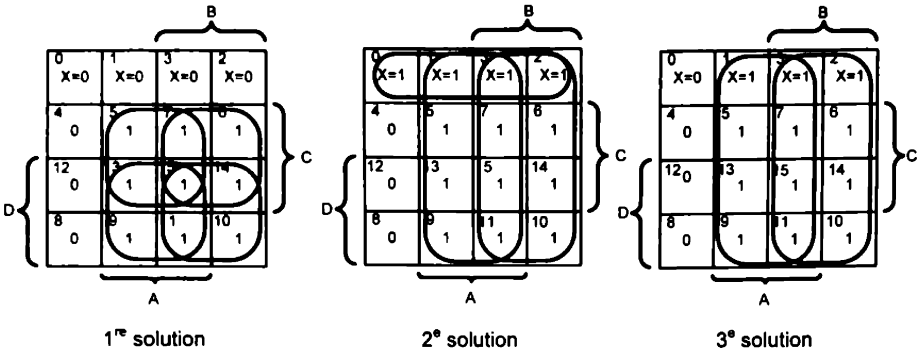


Figure 3.29

Solution :

- *Première possibilité* : On décide que les termes « sans importance » valent 0, ce qui nous donne la solution : $S = CA + DA + CB + DB$ (on rappelle que les groupements simplifiables ont un nombre de cases égal à une puissances de 2 ; ici, les plus grands groupements auront donc quatre cases).
- *Deuxième possibilité* : On décide que les termes « sans importance » valent 1, ce qui nous donne la solution : $S = B + A + \overline{D}\,\overline{C}$.
- *Troisième possibilité* : On décide que les termes « sans importance » valent 1 ou 0 de façon à maximiser les groupements réalisés, ce qui nous donne la solution : $S = B + A$.

La dernière approche est celle qui donne la simplification maximale ; c'est d'ailleurs celle qu'il faudra employer. Il est important de noter qu'une fois les choix établis, dans la solution finale, les « X » *auront* une valeur soit de 1, soit de 0. Dans l'exemple, selon la troisième solution, $m0 = 0$ et $m1 = m2 = m3 = 1$.

Donc, en terminant, on retiendra que les valeurs attribuées aux X le sont de façon à former les plus grands groupements possible.

Terminons cette section avec un exemple.

Exemple : Déterminer l'expression simplifiée pour la fonction S représentée par la table de Karnaugh suivante de quatre variables (D, C, B, A ; $D = MSB$) :

$$m(0, 1, 2, 3, 8, 10, 12, 14) + X(4, 5, 6, 7, 13, 15).$$

Note : On veut la solution de type PdS obtenue en rassemblant les 0.

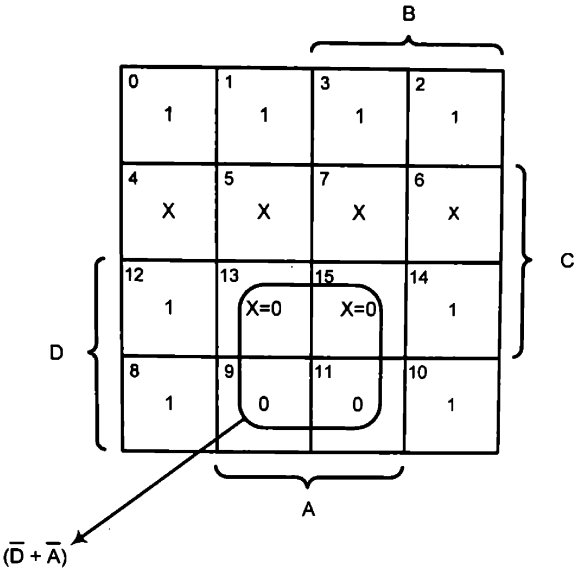


Figure 3.30

Solution : Puisqu'il faut réaliser les plus grands groupements possible (en fonction de la puissance de 2), on voit qu'il faut donc fixer les $X(13,15)$ à « 0 ». La solution de type Produit de Sommes s'obtient alors directement comme le produit d'un seul Maxterme groupant $4 = 2^2$ cases :

$$S = \overline{D} + \overline{A}$$

Au terme de la simplification, il est important de noter que les autres termes X se trouvent donc à avoir une valeur de 1, ce qui n'a de toute façon pas d'importance puisque ce sont précisément des « termes sans importance ». Notons qu'on n'aurait *pas* pu grouper les cases obligatoires 9, 11 avec les cases 13, 15, 5 et 7, et réaliser un groupement de six cases, car 6 n'est pas une puissance entière de 2 ($6 \approx 2^{2.585}$).

3.6 TABLES DE KARNAUGH POUR PLUS DE QUATRE ENTRÉES

Jusqu'à présent, on a vu les cas des tables de Karnaugh à 2, 3 ou 4 variables. Ces tables à respectivement 4, 8 ou 16 cases se représentent directement dans le plan (c'est-à-dire sur une feuille de papier). Le concept peut être étendu à plus de variables. Par contre, il s'agit alors de représentations dans l'espace plus difficiles à manipuler et à représenter. Pour ces représentations supérieures, il s'agit de généraliser les cas plus petits. Par exemple, on pourrait réaliser une table de Karnaugh à quatre variables (*D*, *C*, *B*, *A* ; *D* étant la variable la plus significative) à partir de deux tables de Karnaugh à trois variables tel qu'il a été illustré à la figure 3.31. La première table sert pour les cas où $D = 0$, et la seconde pour les cas où $D = 1$. On peut ensuite combiner les deux tables pour former la forme traditionnelle à 16 cases.

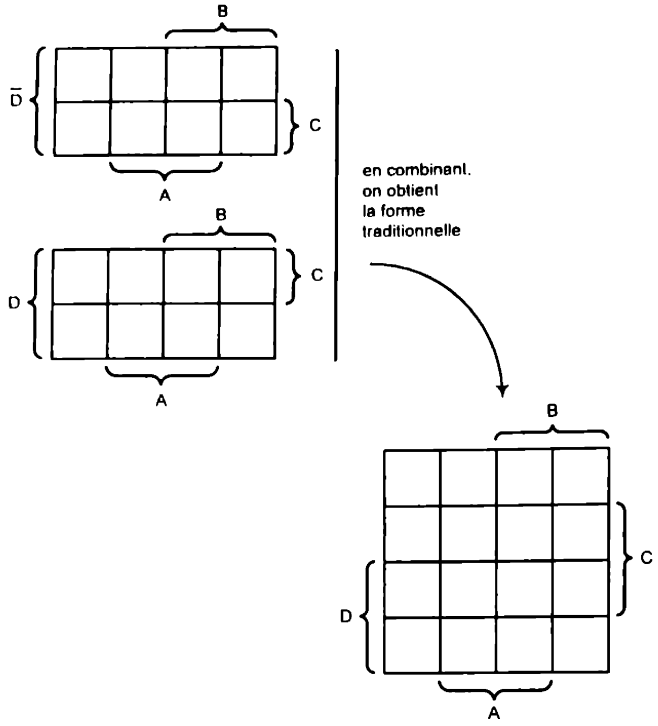


Figure 3.31

Pour le cas à cinq variables (*E*, *D*, *C*, *B*, *A* ; *E* étant la variable la plus significative), la table de Karnaugh possède $2^5 = 32$ cases. En reprenant l'idée précédente, on peut bâtir celle-ci à partir de deux tables de Karnaugh à quatre variables, l'une servant pour le cas $E = 1$ et l'autre pour le cas $E = 0$. Ceci est illustré à la figure 3.32, qui donne $32 = 16 + 16$ cases.

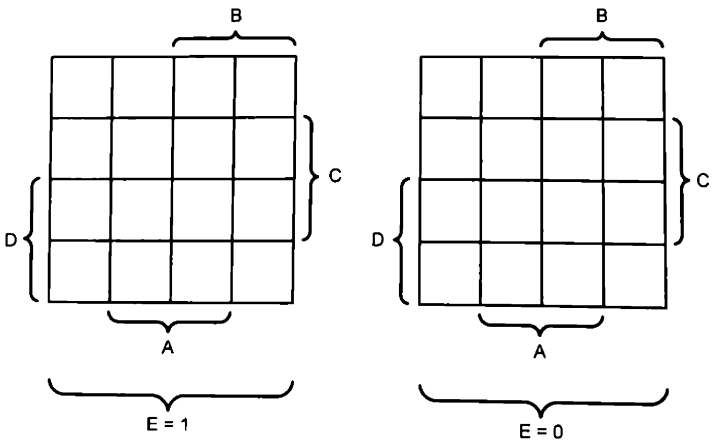


Figure 3.32

Notons qu'il est parfois plus aisé de représenter les deux tables, non pas côte à côte, mais en « pseudo trois dimensions », ce qui permet de mieux visualiser les termes communs et donc de faciliter les simplifications. Ceci est illustré à l'exemple suivant.

Exemple : Donner l'expression logique minimale de l'expression représentée par la table de Karnaugh à cinq variables suivante.

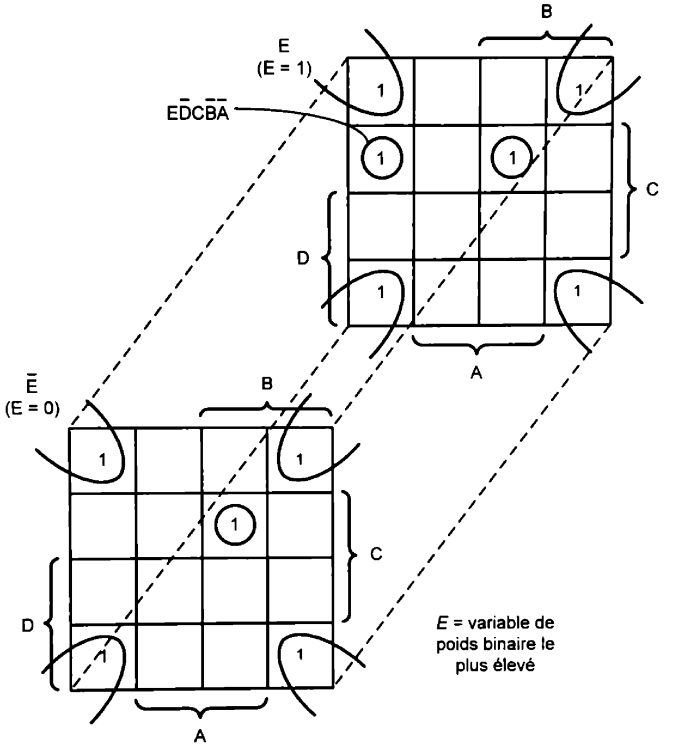


Figure 3.33

Solution :

On peut alors écrire :

Termes des côtés :

$$E\bar{C}\bar{A} + \bar{E}\bar{C}\bar{A} = \bar{C}\bar{A}\left(\underbrace{E + \bar{E}}_{1}\right) = \bar{C}\bar{A}$$

Terme central :

$$E\bar{D}CBA + \bar{E}\bar{D}CBA$$

$$\bar{D}CBA\left(\underbrace{E + \bar{E}}_{1}\right) = \bar{D}CBA$$

et la fonction simplifiée s'obtient comme ceci :

$$S = \bar{C}\bar{A} + \bar{D}CBA + E\bar{D}C\bar{B}\bar{A}\,.$$

La simplification se fait s'il y a superposition en E et en $\bar{E}$ (ce qui n'est pas le cas du terme isolé : $E\bar{D}C\bar{B}\bar{A}$). Les groupements et simplifications sont toutefois moins évidents. Dans notre exemple, on note que $\overline{CA}$ et $\bar{D}CBA$ auraient pu être écrits directement en observant ainsi la « transparence » de la table de Karnaugh à cinq variables (à cause de la superposition en E et en $\bar{E}$).

Au-delà de cinq variables, notre habileté à visualiser les groupements dans un espace multidimensionnel n'est plus très bonne, et il faut utiliser d'autres méthodes. L'algorithme « Quine-McCluskey » permet, tout comme la méthode par table de Karnaugh, d'obtenir une simplification minimale à deux niveaux (c'est-à-dire une solution de type Somme de Produits, ou Produit de Sommes) au prix toutefois d'une complexité qui croît exponentiellement avec le nombre n de variables. Ainsi, si $n = 25$, on peut démontrer qu'il y aura jusqu'à 30 milliards de termes à traiter. Cette approche perd donc rapidement de son intérêt. Notons en terminant que, si on est prêt à obtenir une solution « pas tout à fait optimale », il existe des algorithmes heuristiques qui fourniront une solution proche de la solution optimale, mais avec une complexité réaliste[4].

3.7 ALÉAS

Nous allons finir ce chapitre par une brève étude des aléas. Comme nous le verrons, un examen attentif des termes à regrouper dans les tables de Karnaugh permet d'en éviter certains. Mais tout d'abord, qu'est-ce qu'un aléa ? Un aléa est un malfonctionnement d'un circuit logique (combinatoire ou séquentiel) qui cause des erreurs transitoires survenant lorsqu'une entrée change. On en distingue trois types différents : les aléas statiques, les aléas dynamiques et les aléas essentiels.

Les aléas statiques

Ce sont des transitions simples et momentanées du signal de sortie causées par les délais inégaux entre plusieurs voies de transmission à travers un circuit combinatoire. On les étudiera plus en détails plus bas. La figure 3.34 illustre le type de transition temporelle qui est causée par un aléa statique lorsqu'un signal passe momentanément de 0 à 1 (ou de 1 à 0) :

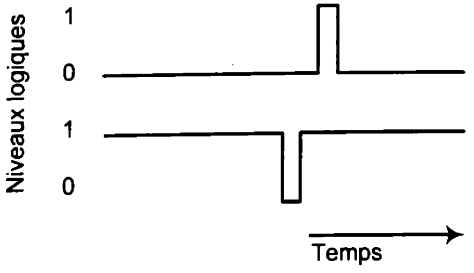


Figure 3.34

[4] http://134.193.15.25/vu/course/cs281/lectures/simplification/quine-McCluskey.html

Les aléas dynamiques

Ce sont des transitions multiples, mais momentanées du signal de sortie, transitions obtenues à la suite du changement du signal d'entrée. Leur forme la plus courante est le contact mécanique d'un interrupteur qui rebondit. Un crayon qu'on laisse tomber sur un bureau et qui rebondit illustre d'ailleurs très bien le type de comportement obtenu.

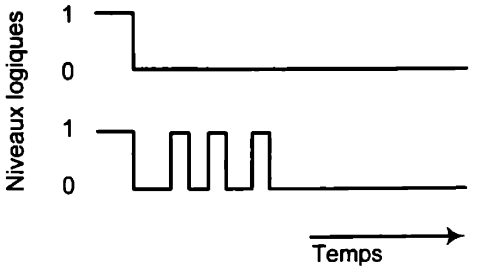


Figure 3.35

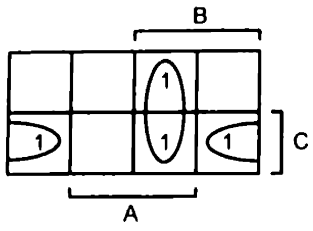
Les aléas essentiels

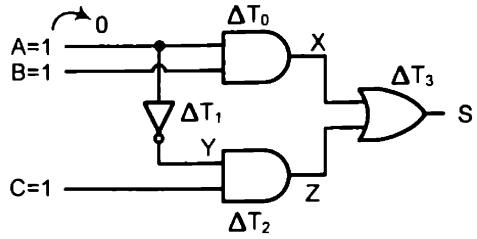
Ce sont des erreurs de fonctionnement du circuit logique, erreurs qui entraînent le circuit dans ses états non valides. Ce type d'aléa se retrouve dans les systèmes séquentiels synchrones. Ces aléas seront discutés aux chapitres 6 et 7 avec la notion d'état de retour.

ALEAS STATIQUES

Revenons maintenant aux aléas statiques. Ceux-ci peuvent être identifiés dans les tables de Karnaugh *puisqu'ils y apparaissent comme saut d'un groupement à l'autre lorsqu'on change une et seulement une variable* (dans le cas de groupements contigus). La transition résultante est causée par les différences de temps de propagation selon les voies de transmission. L'exemple suivant illustre ceci.

Exemple : Considérons la table de Karnaugh de la fonction suivante $S(C, B, A ; C = MSB)$ à trois variables. La simplification normale consiste à réaliser les deux groupements indiqués. On trace ensuite le circuit correspondant à l'expression trouvée.



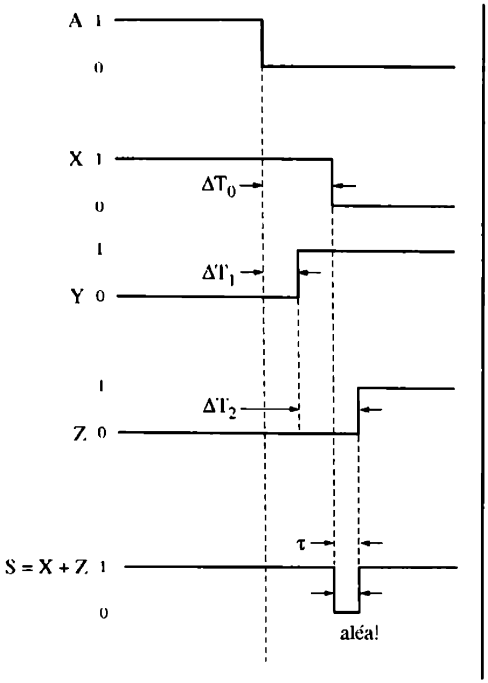


$$S = BA + C\overline{A}$$

Figure 3.36

Pour l'étude de ce circuit, on considère que les délais de propagation dans les portes logiques sont non-nuls et que le délai pour une porte simple est bien évidemment plus court que pour une porte double étant donné sa circuiterie moins élaborée.

Finalement, posons-nous la question suivante : Que se passe-t-il si, avec *B* et *C* à « 1 », *A* passe de « 1 » à « 0 ». On obtiendra en fait une transition dont l'origine peut s'expliquer en étudiant l'évolution temporelle des signaux impliqués. Ceci est illustré à la figure 3.37.



$$\tau = (\Delta T_1 + \Delta T_2 + \Delta T_3) - (\Delta T_0 + \Delta T_3)$$

Délai de propagation par la voie inférieure — *Délai de propagation par la voie supérieure*

Note 1 : Si $\Delta T_0 = \Delta T_1 + \Delta T_2$ **pas** d'aléas (cas improbable)

Note 2 : Dans la figure, par souci de clarté, ΔT_3 a été fixé à « 0 ».

Figure 3.37

Point important : Le dessin précédent est appelé « chronogramme » (*timing diagram* en anglais). Nous reverrons ce type de tracé à plusieurs reprises, surtout dans l'étude des circuits synchrones.

D'une façon plus pratique, ce phénomène peut aussi s'expliquer directement en observant la table de Karnaugh avec $C = B = 1$, et A qui passe de « 1 » à « 0 ». Normalement, la sortie restera à un puisque, comme on le voit sur la table, on passe du terme BA au terme $C\overline{A}$ et que ces deux termes sont entourés. Par contre, une transition pourra se produire (celle qui causera l'aléa) puisque ces deux termes contigus sont séparés. Au paragraphe suivant, on verra comment remédier à cette situation.

Notons finalement que ce phénomène s'appelle aussi course (ou *race* en anglais).

CORRECTION DES ALEAS

Dans les circuits pour lesquels un régime transitoire peut provoquer un fonctionnement « erratique », il faut éliminer les risques d'aléas. Cela se réalise heureusement assez facilement en suivant les deux règles simples suivantes :

Règle 1 : Amortir les interrupteurs mécaniques, ce qui permet de supprimer les aléas dynamiques (ceci sera vu au chapitre 5).

Règle 2 : Ajouter des termes redondants qui lient les groupements séparés par une seule transition dans les tables de Karnaugh.

Reprenons l'exemple précédent :

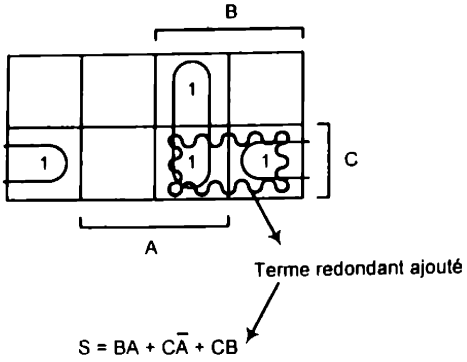


Figure 3.38

Revenons au cas précédent lorsque les variables C et B sont à « 1 » et que la variable A passe de « 1 » à « 0 ». On note que cette fois il n'y aura pas d'aléas puisque, même lors de la transition, le terme redondant CB reste à 1. L'ajout du terme redondant qui lie les groupements séparés par une seule transition (donc des groupements voisins) a éliminé le problème.

Aucun aléa. Aucune course.

PROBLÈMES RÉSOLUS

3.1. Déterminer l'expression simplifiée pour la fonction *S* représentée par la table de Karnaugh suivante de quatre variables : *m*(1, 4, 9, 11, 12, 14, 15) + X(2, 3, 6, 7, 8 10). *Note* : On veut la solution de type SdP obtenue en rassemblant les « 1 » (la *méthode habituelle* en quelque sorte).

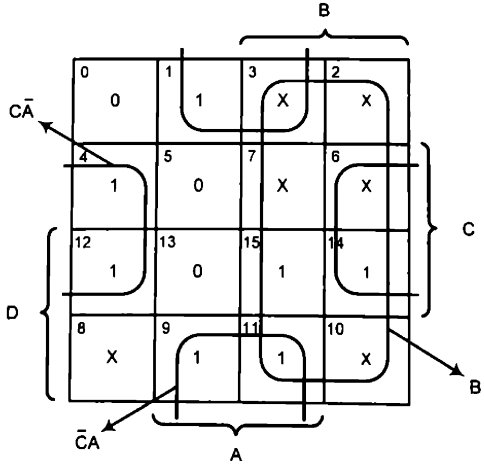


Figure 3.39

Solution : Les X prennent les valeurs indiquées (ils sont à 1 dans les groupements et à 0 autrement). Les valeurs des mintermes sont indiquées ce qui permet d'écrire la valeur de l'expression :

$$S = B + C\bar{A} + \bar{C}A$$
$$S = B + (C \oplus A)$$

On remarque ici l'utilisation du OU-exclusif. On voit aussi que les groupements peuvent se chevaucher. Par contre, on note qu'évidemment (!) un X ne peut pas être à « 1 » dans un groupement, et à « 0 » dans un autre.

3.2. Déterminer l'expression simplifiée pour la fonction S représentée par la table de Karnaugh suivante de cinq variables (R, P, Q, S, T ; R étant la plus significative) :

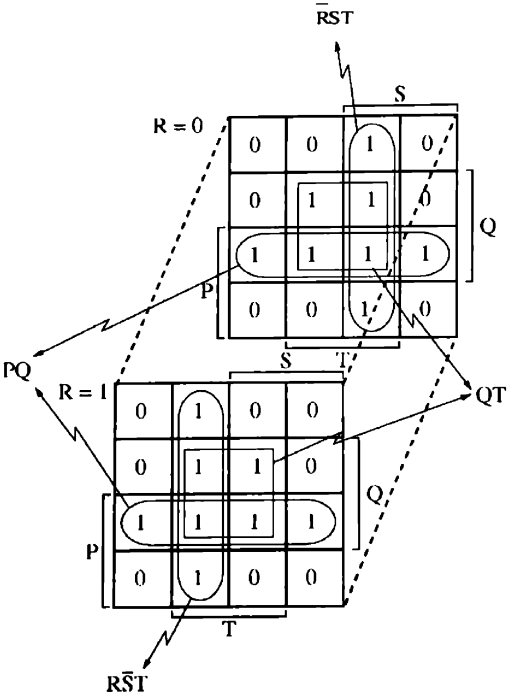


Figure 3.40

Les groupements de « 1 » pour une solution de type Somme de Produits sont indiqués sur la figure, ce qui nous permet d'écrire directement l'expression de S :

$$S = PQ + QT + R\bar{S}T + \bar{R}ST$$

3.3. Optimiser les fonctions suivantes (X : termes sans importance) :

a) Fournir l'expression minimisée en portes NON-ET seulement, pour
$S1$: $m(2, 3, 10, 14, 15) + X(5,7,11)$ (m : mintermes)

b) Fournir l'expression minimisée en portes NON-OU seulement, pour
$S2$: $M(0, 3, 4, 8, 9) + X(1, 5, 12)$ (M : Maxtermes)

Solution :

a) On note que la fonction *S*1 a quatre variables (à cause de *m*15, le minterme de plus haut rang), appelons-les (*D*, *C*, *B*, A ; *D* = *MSB*). Puisqu'on veut une solution en portes NON-ET seulement, on établit la solution de type SdP. On peut dès lors établir la table de Karnaugh directement :

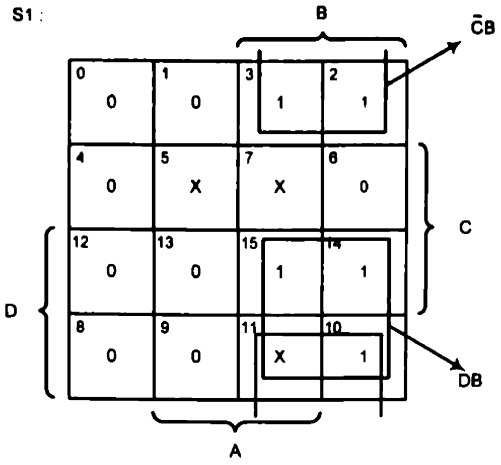


Figure 3.41

La fonction S1 s'écrit alors : $S1 = \bar{C}B + DB$

Pour la version en portes NON-ET seulement, il suffit d'appliquer la Loi de De Morgan à l'expression précédente, ce qui donne :

$$S1 = \overline{\overline{\bar{C}B + DB}} = \overline{\overline{\bar{C}B} \cdot \overline{DB}}$$

À noter que si on avait demandé la solution en portes NON-OU seulement, pour *S*1, on aurait rassemblé les « 0 » dans la table de Karnaugh précédente.

b) On note que la fonction *S*2 a quatre variables (à cause de *M*9, le Maxterme de plus haut rang, les termes *X* sont « sans importance »), appelons-les (*D*, *C*, *B*, *A* ; *D* = *MSB*). Puisqu'on veut une solution en portes NON-OU seulement, on établit la solution de type PdS. On peut dès lors établir la table de Karnaugh directement :

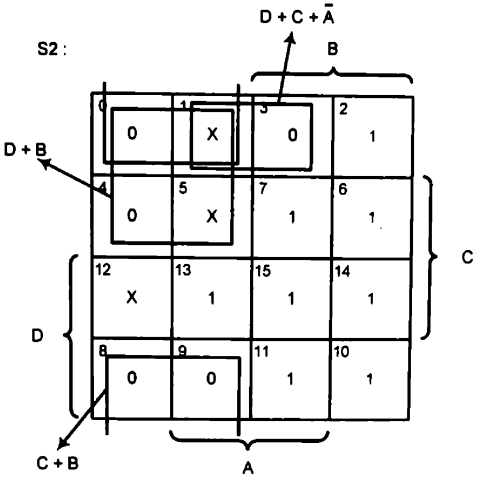


Figure 3.42

La fonction *S2* s'écrit alors : $S2 = (D+B)\cdot(C+B)\cdot(D+C+\bar{A})$

Pour la version en portes NON-OU seulement, il suffit d'appliquer la Loi de De Morgan à l'expression précédente, ce qui donne :

$$S2 = \overline{\overline{(D+B)\cdot(C+B)\cdot(D+C+\bar{A})}} = \overline{\overline{(D+B)} + \overline{(C+B)} + \overline{(D+C+\bar{A})}}$$

À noter que si on avait demandé la solution en portes NON-ET seulement, pour *S2*, on aurait rassemblé les « 1 » dans la table de Karnaugh précédente.

3.4. On veut réaliser un 1BFS (*1 bit full substractor* ou Soustracteur complet à 1 bit :)

a) Dresser la table de vérité avec le résultat de la soustraction et avec la retenue de sortie (le C_{out}, Carry out).

b) Établir et solutionner les tables de Karnaugh pour les deux sorties (S et C_{out}).

c) Dessiner le circuit logique résultant.

Solution :

a) On pose que le résultat pour $S = B-A$, B et A sont des nombres d'un bit (B = *MSB*). On se rappelle que, pour le 1BFA (*1 Bit Full Adder* (chapitre 1)), on avait en entrée les deux nombres à additionner de même que la retenue d'entrée (C_{in}). Pour le 1BFS, on aura donc aussi en entrée les deux nombres à additionner (B et A) de même que la retenue d'entrée (C_{in}). En sortie, nous aurons $S = B-A$ de même que C_{out}. Bien évidemment, la combinaison (C_{out}, S) correspondra à un nombre de 2 bits en complément deux (section 1.11). Cela donnera donc les huit cas suivants :

Rang	B	A	C_{in}	C_{out}	S	Remarque
0	0	0	0	0	0	$(0 - 0 - 0 = 0)_{10}$
1	0	0	1	1	1	$(0 - 0 - 1 = -1)_{10}$ 1 et $(11)_2 = -1_{10}$
2	0	1	0	1	1	$(0 - 1 - 0 = -1)_{10}$ et $(11)_2 = -1_{10}$
3	0	1	1	1	0	$(0 - 1 - 1 = -2)_{10}$ et $(10)_2 = -2_{10}$
4	1	0	0	0	1	$(1 - 0 - 0 = +1)_{10}$ et $(01)_2 = +1_{10}$
5	1	0	1	0	0	$(1 - 0 - 1 = 0)_{10}$
6	1	1	0	0	0	$(1 - 1 - 0 = 0)_{10}$
7	1	1	1	1	1	$(1 - 1 - 1 = -1)_{10}$ et $(11)_2 = -1_{10}$

Figure 3.43

b) Dressons maintenant les tables de Karnaugh.

Pour la sortie S :

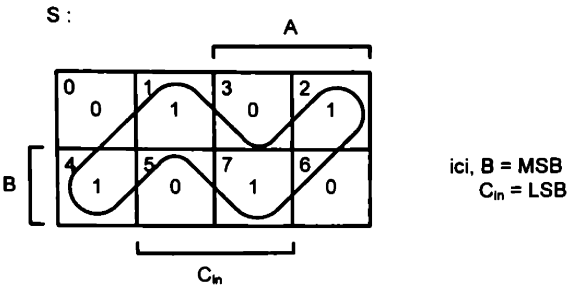
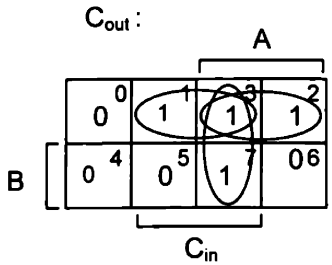


Figure 3.44

On se souvient du dessin du OU-exclusif qui donne le zigzag ou la vague :

$$S = m1 + m2 + m4 + m7 = A \oplus B \oplus C_{in}$$

Pour la sortie C_{out} :



$$C_{out} = \overline{B}C_{in} + \overline{B}A + C_{in}A$$

Figure 3.45

c) Des expressions obtenues en a), il est facile de déterminer les schémas (dessinés à l'aide de Xilinx Foundation) :

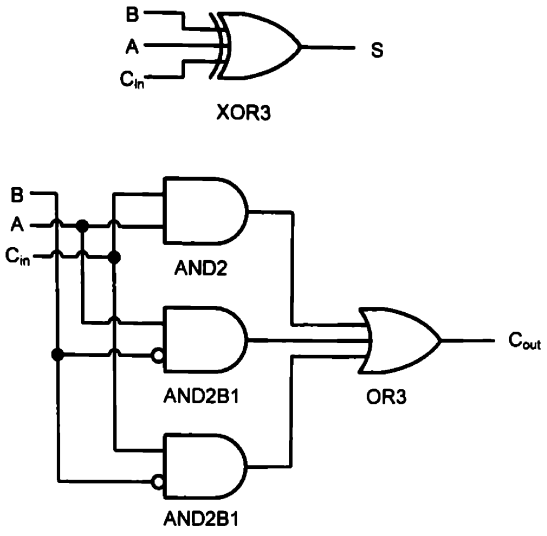


Figure 3.46

EXERCICES

3.1. Démontrer que, si tous les mintermes d'une table de Karnaugh sont à « 1 », l'expression simplifiée de cette table est « 1 ». Effectuer la démonstration sur une table à quatre variables avec : (*D,C,B,A* ; *D* = *MSB*).

3.2. Pour les fonctions suivantes :

$$F(W,Z,Y,X;W=MSB)=(\overline{X}+\overline{Y})\cdot(\overline{W}+\overline{X}+Y)\cdot(\overline{W}+X+Z)$$

$$G(E,D,C,B,A;E=MSB)=\sum m(4,5,6,11,12,13,18,20,21,23,28,29)+X\ (2,10,14,19,27,31)$$

a) Dresser la table de Karnaugh correspondante.

b) Trouver l'expression Somme de Produits minimale.

c) Trouver l'expression Produit de Sommes minimale.

d) Dessiner le schéma pour b) et c).

e) Calculer le coût du circuit en matière de nombre de portes doubles élémentaires pour b) et c).

3.3. Soit la fonction

$$F(Z,W,Y,X;Z=MSB)=W\cdot\overline{X}\cdot\overline{Y}+Y\cdot\overline{W}\cdot\overline{Z}+Y\cdot X\cdot Z+Y\cdot X\cdot W$$

Tracer le circuit implantant cette fonction en employant :

a) uniquement des portes NON-ET à trois entrées (solution minimale) ;

b) uniquement des portes NON-OU à trois entrées (solution minimale).

3.4. En employant la technique des tables de Karnaugh, trouver l'expression PdS minimale pour la fonction booléenne suivante (Z = *MSB*):

$$F(X,Y,Z) = (X+Y)\cdot(X+\bar{Y}+Z)$$

Calculer le coût du circuit en matière de nombre de portes doubles élémentaires.

3.5. Soit la fonction :

$$F(D,C,B,A;D=MSB) = \sum m(2,3,5,6,7,8,9,12,14)$$

a) En employant la technique des tables de Karnaugh, trouver l'expression SdP minimale et réaliser son implantation en NON-ET. Tracer le circuit et calculer son coût en matière de nombre de portes doubles élémentaires.

b) En employant la technique des tables de Karnaugh, trouver l'expression minimale en « rassemblant les 0 » et réaliser son implantation en NON-OU. Tracer le circuit et calculer son coût en matière de nombre de portes doubles élémentaires. Quelle est la solution la plus avantageuse ?

3.6. Soit la fonction suivante $F(D,C,B,A\ ;\ D = MSB)$:

$$F = \sum m(0,4,5,6,7,8,11,12,14)$$

Donner la table de Karnaugh correspondante.

Donner l'expression booléenne minimale, le schéma et le coût correspondant en matière de nombre de portes doubles élémentaires pour les deux cas suivants :

a) Solution de type seulement avec des portes NON-ET.

b) Solution de type avec des portes NON-OU. Quelle est la solution la plus avantageuse ?

3.7. Soit la fonction suivante :

$$S(C,B,A;C=MSB) = \sum m(3,5,7) + X\ (2,4)$$

a) Dresser la table de Karnaugh correspondante.

b) Trouver l'expression Somme de Produits minimale.

c) Trouver l'expression Produit de Sommes minimale.

d) Dessiner le schéma pour a) et b).

3.8. Soit la fonction suivante :

$$S(D,C,B,A;D=MSB) = \sum m(0,1,6,7,9,11) + X\ (2,4,8,10,15)$$

a) Dresser la table de Karnaugh correspondante.

b) Trouver l'expression Somme de produits minimale.

c) Trouver l'expression Produit de Sommes minimale.

d) Trouver l'expression simplifiée en portes NON-ET, seulement.

e) Trouver l'expression simplifiée en portes NON-OU, seulement.

3.9. Soit la fonction :

$$F(D,C,B,A;D=MSB)=\sum m\ (2,4,6,13,14)+X(3,7,9,11,12,15)$$

a) En employant la technique des tables de Karnaugh, trouver l'expression SdP minimale et réaliser son implantation en NON-ET. Tracer le circuit et calculer son coût en matière de nombre de portes doubles élémentaires.

b) En employant la technique des tables de Karnaugh, trouver l'expression PdS minimale et réaliser son implantation en NON-OU. Tracer le circuit et calculer son coût en matière de nombre de portes doubles élémentaires.

3.10. Soit les fonctions F et G suivantes (D, C, B, A ; $D = MSB$) :

$$F=\prod M(1,5,4,8)+X(3,6)$$

$$G=\prod M(2,4,6,13,14)+X(3,7,9,11,12,15)$$

Donner la table de Karnaugh correspondante.

Donner l'expression booléenne minimale, le schéma et le coût correspondants (en terme de nombre de portes doubles élémentaires) pour les quatre cas suivants :

a) Solution de type SdP.

b) Solution de type seulement avec des portes NON-ET.

c) Solution de type PdS.

d) Solution de type avec des portes NON-OU.

3.11. Tracer les patrons possibles obtenus de l'utilisation d'une porte OU-exclusif triple et quadruple dans une table de Karnaugh à quatre variables (D,C,B,A ; $D = MSB$).

4

Circuits programmables pour les designs combinatoires

AUGUSTUS DE MORGAN : DES LOIS FONDAMENTALES POUR LA SIMPLIFICATION DES CIRCUITS LOGIQUES

Augustus De Morgan[1], mathématicien britannique (27 juin 1806-18 mars 1871).

Le père d'Augustus De Morgan était colonel au service de la Compagnie des Indes occidentales. Né en Inde, De Morgan n'avait que sept mois lorsque, à cause des dangereuses rébellions sévissant là-bas, sa famille l'emmena vivre en Grande-Bretagne. Son enfance fut marquée par le décès de son père (alors qu'il n'avait que 10 ans), de fréquents déménagements au sud-ouest de l'Angleterre et une éducation banale. Alors que le jeune Augustus, à l'âge de 14 ans, dessinait des figures géométriques avec règle et compas, un ami de la famille découvrit les talents de celui-ci en mathématiques. Sa mère, très portée sur la religion, aurait voulu qu'il s'oriente vers la prêtrise, mais il montra peu de dispositions pour cela, plus intéressé par les mathématiques. Son attitude négative envers la religion le suivit toute sa vie. Plus tard, son épouse écrira un livre sur le spiritisme, les tables tournantes ; livre dont De Morgan signa la préface. L'infirmité dont il était affligé (il était aveugle d'un œil) l'empêcha de s'inscrire aux activités sportives des collèges qu'il fréquenta. Il se définissait d'ailleurs comme étant ni Anglais, ni Écossais, ni Irlandais, mais plutôt comme étant un « britton non attaché », terme consacré signifiant un finissant d'Oxford ou de Cambridge sans appartenance à l'un des prestigieux collèges de ces grandes écoles. Après Cambridge, il enseigna les mathématiques à la London University, et fut très apprécié de ses étudiants, dont il corrigeait les devoirs hebdomadaires lui-même. Sa fin fut tragique. Très engagé, il démissionna en 1866 lorsque la chaire de philosophie fut octroyée pour des motifs religieux et non académiques. Deux ans plus tard, son fils, avec qui il avait fondé la London Mathematical Society, mourut. Ce triste événement fut suivi du décès d'une de ses filles ; coup fatal puisqu'il décéda alors de prostration nerveuse. Contrairement à Boole (section 3.0), De Morgan eut une existence difficile, et son handicap y contribua. Il se plaignait d'ailleurs que celui-ci l'empêchait de réaliser des expériences. À propos de Londres, il se disait

[1] http://en.wikipedia.org/wiki/Augustus_de_Morgan

comme Socrate qui avait déclaré que *plus loin d'Athènes il se trouvait, plus heureux il se sentait.* D'ailleurs, De Morgan ne vota jamais, ne visita jamais les principaux monuments de Londres et ne participa à aucune réunion de la Société Royale.

Du point de vue des mathématiques, il fut un auteur prolifique puisque l'ensemble de ses écrits représente une « petite bibliothèque ». Ses travaux ont concerné l'algèbre, la trigonométrie et la logique. Contemporain de G. Boole, Augustus De Morgan a d'ailleurs publié son propre ouvrage sur la logique symbolique presque en même temps que lui. Son ouvrage le plus éclectique est certainement son *Budget of Paradoxes*, publié en 1849. Dans *Trigonometry and Double Algebra*, il reprend l'arithmétique et les opérations de base (+, −, etc.), puis les symboles ; il montre que des opérations peuvent être effectuées sur ceux-ci sans connaître leurs valeurs préalables, puis il démontre que l'association de symboles peut être représentée par une droite. Finalement, il étend le concept aux nombres imaginaires où ceux-ci sont représentés dans un quadrant tel qu'on le voit aujourd'hui, par exemple dans les cours de circuits électriques où les parties réelles et imaginaires forment deux axes d'un plan. En ce qui nous concerne en circuits logiques, De Morgan est connu pour sa fameuse règle qui permet de changer d'opérateur en employant une négation. Ce point fut d'ailleurs étudié au chapitre 2.

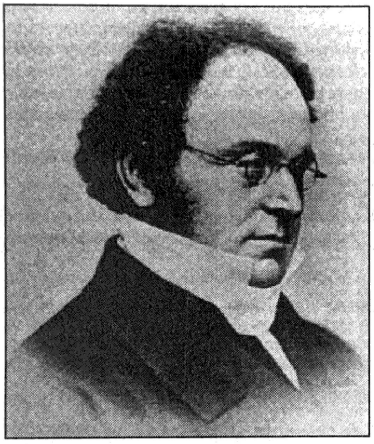


Figure 4.1 Augustus De Morgan, qui a beaucoup contribué aux progrès de la logique combinatoire actuelle (http://en.wikipedia.org/wiki/Augustus_de_Morgan)

4.1 INTRODUCTION

Jusqu'à présent, notre effort s'est porté sur la minimisation des designs et le recours aux portes simples (ET, OU, NON-ET, NON-OU, OU-Exclusif). Dans le présent chapitre, nous allons maintenant explorer d'autres circuits plus complexes et en particulier les familles suivantes :

- Les *multiplexeurs* (aussi communément appelés MUX) qui sont des circuits aiguilleurs à 2^N entrées qui permettent de sélectionner l'origine de la donnée qui sera

transmise à la sortie. Les multiplexeurs peuvent agir sur un bit ou sur plusieurs bits à la fois selon le modèle de MUX employé.

- Les *démultiplexeurs* ou décodeurs (aussi communément appelés DÉMUX) sont des circuits qui décodent la valeur d'un nombre binaire et activent la sortie correspondant à cette valeur.
- Les *mémoires ROM* (*Read Only Memory*, ou mémoire morte en français) sont des circuits qui mémorisent une donnée (sur un ou plusieurs bits) pour chaque adresse. Le contenu d'une mémoire ROM est inscrit en usine et est inaltérable par la suite. La variante PROM (*Programmble Read Only Memory*) est programmable par l'usagère et les variantes EPROM (*Erasable PROM*), EEPROM (*Electrically Erasable PROM*) sont effaçables et reprogrammables par l'usagère.
- Les *circuits PLD* (*Programmable Logic Device*, ou circuit logique programmable en Français) sont des circuits configurables par l'usagère. Selon leur type, ils peuvent notamment incorporer des configurations de type SdP (Somme de Produits) vues au chapitre 2. Selon le type, un tel circuit sera effaçable et reprogrammable par l'usagère ou programmable une seule fois (circuit de type OTP, *One Time Programmable*). Le sigle PLD réfère à un type très général de circuit. D'une façon particulière, on emploiera des circuits PAL (*Programmable Array Logic*, ou réseau logique programmable en français), FPGA (*Field Programmable Gate Array* ou Réseau de portes programmable par l'usagère en français) ou CPLD (*Complex Programmable Logic Device*, ou réseau logique programmable complexe en français).

On verra que les circuits MUX, DÉMUX, ROM et PLD peuvent implanter n'importe quelle table de vérité à n entrées ; ils sont donc très pratiques. Pour ce qui est des circuits de mémoire, une simple reprogrammation suffit pour adapter le design à une nouvelle spécification ; cela est donc pratique aussi. Toutefois, puisque ces circuits sont assez complexes, il est utile de simplifier au préalable la table de vérité initiale afin de réduire la taille du circuit qui sera requis. On pourra ainsi passer d'un « MUX 8 à 1 » à un « MUX 4 à 1 » couplé à quelques portes simples pour un design équivalent beaucoup plus petit. Nous nous attarderons d'ailleurs à la simplification des MUX plus tard dans ce chapitre. Les circuits PLD plus complexes requièrent des outils de configuration plus élaborés tel le Xilinx WebPack. Des langages de description du matériel (*Hardware Description Langage* en anglais), tels ABEL[2] (*Advanced Boolean Equation Language*), Verilog[3] ou VHDL[4] (VHSIC[5] *Hardware Description Langage)* peuvent aussi servir à configurer ceux-ci. Ces langages ne sont pas étudiés dans ce livre.

---

2 Une bonne introduction (en anglais) sur ABEL se trouve à l'adresse : http://www.seas.upenn.edu/ese/rca/software/abel/abel.primer.html

3 Une bonne introduction (en anglais) sur Verilog se trouve à l'adresse : http://www.eet.bme.hu/~poppe/vprimer.pdf

4 Une bonne introduction (en anglais) sur VHDL se trouve à l'adresse : http://tech-www.informatik.uni-hamburg.de/vhdl/doc/cookbook/VHDL-Cookbook.pdf

5 VHSIC = *Very High Speed Integrated Circuits* ou circuits intégrés à très grande vitesse.

4.2 SYNTHÈSE AVEC DES CIRCUITS À INTÉGRATION MOYENNE

Au chapitre 1 (section 1.0), on a brièvement relaté l'historique des circuits intégrés incluant les circuits à moyenne intégration, appelé en anglais circuit MSI (*Medium Scale Integration*), regroupant quelques fonctions simples telles que :

- Multiplexeurs (74151, 74153, ...) ;
- Décodeurs (74138, 74139, 74154, 74537, ...) ;
- Encodeurs de priorité (74148, ...) ;
- Générateurs de parité (74280, ...) ;
- Additionneurs (7483, ...) ;
- Multiplicateurs (74284, ...) ;
- Comparateurs (7485, 74521, 74524, ...) ;
- Compteurs, registres (74163, 74191, 74374, 74373, ...) ;
- ALU - *Arithmetic Logic Unit* ou unité de logique et d'arithmétique (famille TTL 74xxx : 74181, 74381, ...).

Dans cette liste, on a aussi inclus les numéros des composantes TTL correspondantes (74xxx). De nos jours, ces fonctions se retrouvent aussi sous la forme de librairie dans des programmes de conception tel le Xilinx WebPack ou Logisim, dont nous avons déjà parlé.

Un des avantages lié à l'utilisation de telles fonctions, est que moins de câblage est requis pour le montage final du circuit. De plus, la designer pourra travailler à un niveau de synthèse plus élevé. Par contre, il n'est pas sûr que l'usage de telles fonctions engendrera des « économies de silicium » puisque, malgré tout, ces fonctions sont plus gourmandes en logique que des portes simples. De nos jours, cependant, la minimisation de la quantité de logique employée n'est plus une contrainte absolue étant donné le niveau d'intégration atteint dans les circuits VLSI. C'est ainsi que les méthodes de synthèses par portes simples et basées sur des circuits (ou librairies optimisées correspondantes) de type SSI (*Small Scale Integration*, intégration à petite échelle) vues au chapitre précédent, sont des méthodes de dernier recours. Néanmoins, ces méthodes sont très utiles et très employées dans toutes les applications de *glue logic* (colle logique), c'est-à-dire les applications de connexion entre des grands systèmes logiques complexes. Des circuits intégrés qui incorporent seulement quelques portes sont d'ailleurs couramment employés et disponibles en divers formats, par exemple, chez Philips, la ligne de produits PicoGate qui incorpore respectivement une, deux ou trois portes logiques dans les familles 1G, 2G, 3G[6].

[6] http://www.standardproducts.philips.com/products/collateral/logic/pdf/guide-picogate.pdf

4.3 SYNTHÈSE AVEC MULTIPLEXEURS

STRUCTURE D'UN MULTIPLEXEUR

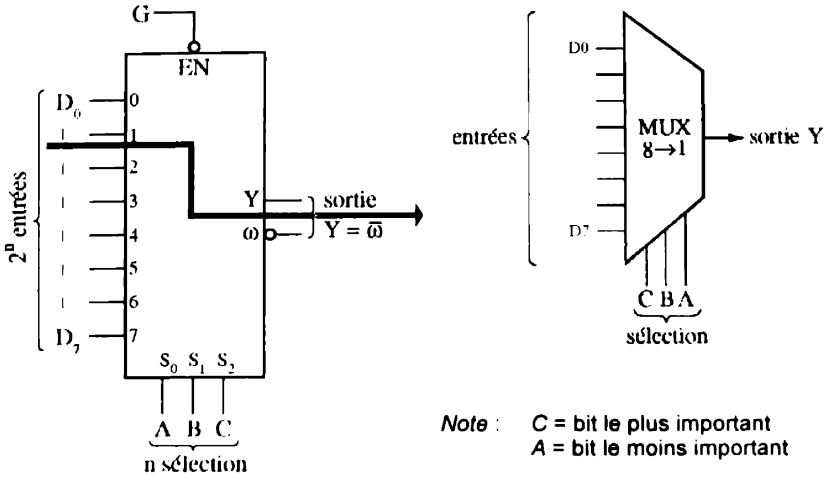
On peut définir un multiplexeur (communément appelé MUX) comme suit : un multiplexeur est un circuit combinatoire aiguilleur à 2^n entrées qui sélectionne l'origine de la donnée qui sera transmise à la sortie. Un multiplexeur possède donc 2^n entrées d'information, n entrées de sélection et une seule sortie. Un multiplexeur peut agir sur un bit ou sur plusieurs bits à la fois selon le modèle de MUX employé. Si c'est le cas, la sortie et les 2^n entrées d'information seront sur plusieurs bits. Le fonctionnement est le suivant : on retrouvera en sortie la valeur de l'entrée qui a été sélectionnée.

Étudions le multiplexeur 74151. Le multiplexeur 74151 est un MUX 8 à 1, puisqu'il possède huit entrées d'information et une sortie. Puisqu'il y a 8 entrées d'information, il y aura aussi trois bits de sélection ($2^3 = 8$). Pour le 74151, la sortie et les huit entrées d'information sont sur un bit (on dit parfois sur un bit de large). En plus, ce circuit possède un bit de commande d'activation (*Enable* ou EN en anglais) : l'entrée *G*. Le fonctionnement du bit *G* d'activation est le suivant :

Si $G = 1$ la sortie Y demeurera toujours à 0.

Si $G = 0$, on obtient le mode normal de fonctionnement de ce multiplexeur, selon la table de vérité donnée plus bas.

Sur le dessin de gauche, on a aussi indiqué la direction du transfert d'information entre l'entrée et la sortie pour le cas où celle-ci circule de D_1 à la sortie Y (à noter que le 74151 possède aussi une sortie inverse ω tel qu'il a été indiqué). À droite, on a dessiné un MUX 8 à 1 plus simple, sans bit d'activation ni de sortie inversée. À noter sur cette dernière figure, le schéma « trapézoïdal » souvent employé pour représenter les MUX.



Note : *C* = bit le plus important
A = bit le moins important

Figure 4.2

La table de vérité décrivant le fonctionnement du circuit 74151 est la suivante :

Bit de sélection			Bit d'activation *enable*	Sortie
S_2	S_1	S_0	G	Y
X	X	X	1	0
0	0	0	0	D_0
0	0	1	0	D_1
0	1	0	0	D_2
0	1	1	0	D_3
1	0	0	0	D_4
1	0	1	0	D_5
1	1	0	0	D_6
1	1	1	0	D_7

Figure 4.3

Ainsi, pour le 74151, avec $G = 0$, si les bits de sélection (S_2, S_1, S_0) valent « 001 », on aura en sortie $Y = D_1$, ce qui signifie que la valeur présente à l'entrée D_1 sera en fait passée ou « aiguillée » à la sortie. Dans ce cas, si l'entrée $D_1 = 1$, alors $Y = 1$, et si $D_1 = 0$, $Y = 0$.

Il est intéressant de calculer le coût du MUX 74151 en fonction du nombre de portes équivalentes à deux entrées tel qu'il a été discuté au chapitre précédent. À cette fin, voyons d'abord la structure interne de ce circuit dessiné sous la forme d'une SdP (Somme de Produits, section 2.4), ce qui rend sa compréhension assez claire. Ainsi, pour D_0, sa valeur sera passée à la sortie Y, seulement si G est à 0 et si les bits de sélection (S_2, S_1, S_0) sont tous nuls (minterme $\overline{S_2}\,\overline{S_1}\,\overline{S_0}$). Les autres cas se déduisent de la même façon, ainsi, l'équation pour la sortie Y de ce multiplexeur est :

$$Y = \underbrace{\overline{G} \cdot D0 \cdot \overline{S}_2 \cdot \overline{S}_1 \cdot \overline{S}_0}_{\text{Pour } D0} + \underbrace{\overline{G} \cdot D1 \cdot \overline{S}_2 \cdot \overline{S}_1 \cdot S_0}_{\text{Pour } D1} + \ldots$$

On remarque ainsi que les bits de sélection définissent en fait l'adresse de la ligne d'entrée qui sera employée (ou le minterme correspondant). Le schéma complet est le suivant :

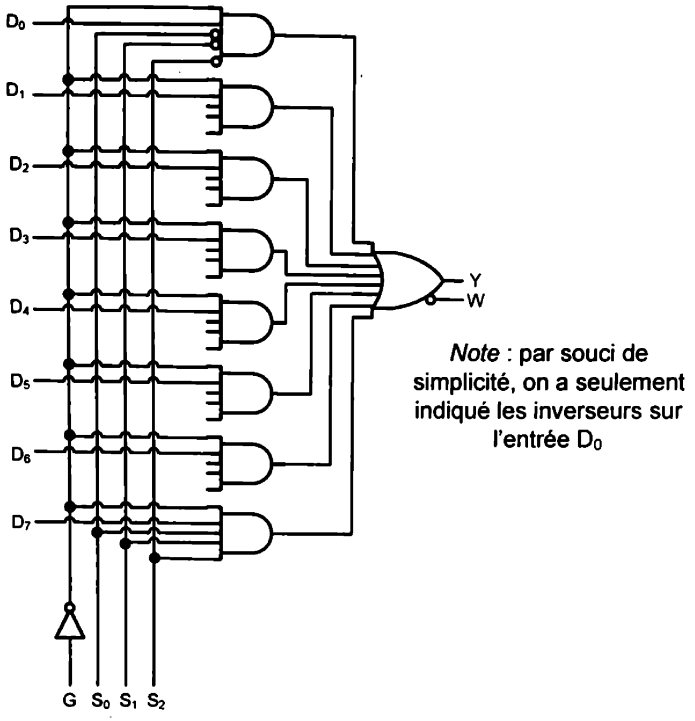


Figure 4.4

Le coût se calcule alors comme suit (sans tenir compte des inverseurs tel que spécifié précédemment à la section *Coût d'un circuit en fonction du nombre de portes doubles élémentaires*, page 55) :

- huit portes à cinq entrées équivalent à 8 × 4 = 32 portes doubles ;
- une porte à huit entrées équivaut à 1 × 7 = 7 portes doubles.

Le grand total porte donc le coût en fonction du nombre de portes double à 39 portes doubles, ce qui est assez encombrant.

L'étude du 74151 nous permet d'introduire maintenant l'expression générale pour la sortie d'un multiplexeur à n bit de sélection (avec bit d'activation *G* non inversé) :

$$Y = G\left(m_0\ D_0 + m_1\ D_1 + \ldots + m_{2^n-1}\ D_{2^n-1}\right)$$

m_i = valeur de la sélection (adresse ou minterme)

D_i = entrée

i = 0, 1, 2^n-1

G = bit d'activation

n = nombre de bits de sélection

Il est à noter que la famille TTL offre plusieurs modèles de multiplexeurs :

74150 – un multiplexeur simple (sur un bit) 16 à 1

74151 – un multiplexeur simple (sur un bit) 8 à 1

74153 – un multiplexeur double (sur 2 deux bits de large) 4 à 1

74157 – un multiplexeur quadruple (sur quatre bits de large) 2 à 1

La figure 4.5 montre le schéma et la structure du 74157[7] :

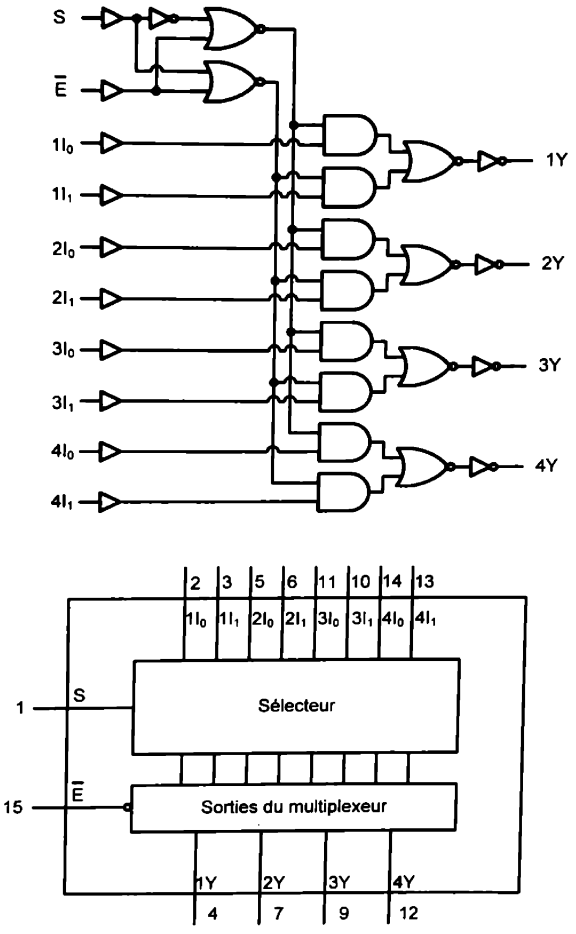


Figure 4.5

[7] http://www.semiconductors.philips.com/acrobat_download/datasheets/74HC_HCT157_CNV_2.pdf

APPLICATIONS DES MULTIPLEXEURS

Les applications des multiplexeurs sont multiples. Ils peuvent servir à sélectionner une source de données en sortie parmi plusieurs sources : c'est l'application « aiguillage typique ». Dans ce cas, on sélectionne le flux de données à transmettre sur un canal (de communication). La téléphonie par satellite est un bon exemple d'une telle sélection de flux de données. Si on appelle outre-mer, on remarque que, lors des temps morts d'une conversation, on « entend des voix », ceci confirme le multiplexage des conversations (numérisées). Comme le temps satellite coûte cher, les compagnies téléphoniques détectent les temps morts et les « comblent » avec d'autres conversations, ce qui permet globalement de faire passer plus de conversations que le nombre de canaux disponibles. Comme la suppression n'est pas complète, les interlocutrices perçoivent des voix indistinctes lors des temps morts.

Une autre application concerne la conversion de parallèle en série. Dans ce cas, un mot binaire exprimé sur plusieurs bits est présenté à l'entrée d'un multiplexeur de sorte que chaque bit du mot de 2^n bits correspond aux 2^n entrées de celui-ci. On réalise aisément que le mot sera converti en série si l'on sélectionne une à une en sortie chacune des entrées présentes.

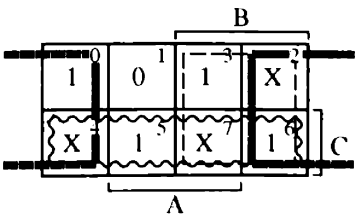
La troisième application des multiplexeurs est la génération de fonctions combinatoires ; c'est l'application qui nous intéresse ici. Dans ce cas, *la sortie du multiplexeur correspond à la sortie de la fonction à implanter, les mintermes de la fonction correspondent aux entrées du multiplexeur et les bits de sélection correspondent aux variables de celle-ci.* Illustrons ceci à l'aide d'un exemple.

Exemple : On veut implanter la table de vérité suivante avec et sans multiplexeur :

$S(C,B,A$; $C = MSB) = m(0, 3, 5, 6) + X(2, 4, 7)$

Solution : On obtient immédiatement la table de vérité et la table de Karnaugh de S :

rang	C	B	A	S
0	0	0	0	1
1	0	0	1	0
2	0	1	0	X
3	0	1	1	1
4	1	0	0	X
5	1	0	1	1
6	1	1	0	1
7	1	1	1	X



Sans recourir à un multiplexeur, la solution SdP (vue à la section 3.3) s'obtient comme précédemment avec la table de Karnaugh :

$$S = B + \bar{A} + C$$

Figure 4.6

Maintenant, employons un multiplexeur. Puisqu'on a trois variables (*C*, *B*, *A*), on peut employer un multiplexeur à trois bits de sélection, donc un multiplexeur 8 à 1 ($8 = 2^3$). Selon l'expression générale d'un multiplexeur vue précédemment, les huit entrées du multiplexeur correspondent directement à la sortie désirée pour chaque rang de la table de vérité, et les trois bits de sélection (bit d'adresse) correspondent aux trois variables impliquées. Dans le cas présent, les « choix sans importance » désignés par « *X* » peuvent être mis à 1 ou à 0. Dans le cas présent, on a choisi de les mettre à 1 (il faut bien sûr leur donner une valeur). Le schéma devient donc le suivant (à droite, on a dessiné la solution SdP avec les portes simples.) :

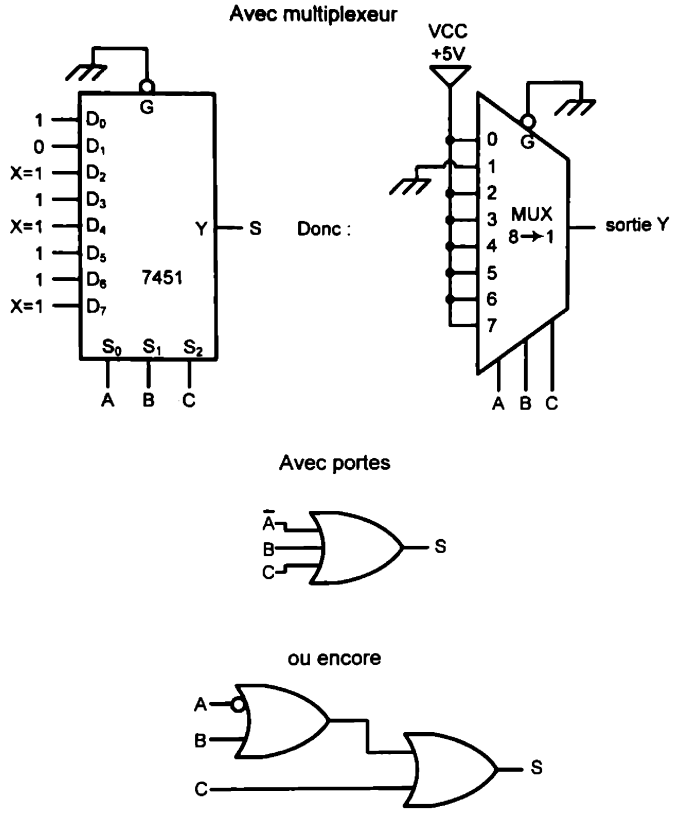


Figure 4.7

Évidemment, le bit d'activation G doit être mis à la masse, donc au 0 logique pour que le circuit fonctionne. Considérons maintenant le fonctionnement du circuit. Si par exemple les bits de sélection sont $C = 0$, $B = 0$, $A = 1$, alors la sortie sera à 0 ($= D_1$) tel qu'il a été indiqué dans la table de vérité précédente.

Calculons finalement le coût de la solution avec un multiplexeur 8 à 1. Ce coût est de 39 portes à deux entrées si on assume une implantation avec un 74151 (tel qu'il a été vu précédemment). Quant à elle, la solution SdP possède un coût matériel de deux portes à

deux entrées. Comme on est à même de le constater, la solution est donc facile à obtenir avec un multiplexeur, mais coûteuse en ce qui concerne le matériel nécessaire aussi. Plus loin dans ce chapitre, nous verrons comment on peut implanter une table de vérité dans des plus petits MUX par subdivision de la table de Karnaugh. Par exemple, ici, on aurait alors pu employer un MUX 4 à 1 au lieu d'un MUX 8 à 1.

4.4 MULTIPLEXEURS RELIÉS EN CASCADES

Il est parfois pratique de recourir à plusieurs multiplexeurs pour réaliser des « aiguillages plus complexes » que ce que permet un seul multiplexeur : c'est ce qu'on appelle une cascade de multiplexeurs. L'exemple suivant illustre cette situation.

Exemple : En employant des MUX 74157, on veut réaliser un multiplexeur 3 à 1 pour des mots de quatre bits de large.

Solution : Un circuit 74157 (ou son équivalent en librairie de fonctions dans des logiciels tels que le Xilinx WebPack) contient un multiplexeur 2 à 1 quadruple (donc sur quatre bits de large) avec un seul bit de sélection. Pour réaliser un multiplexeur 3 à 1 sur quatre bits de large, il faudra donc employer deux multiplexeurs 74157. On aura besoin de deux bits de sélection S_1 et S_0 permettant d'aiguiller les données provenant des trois entrées, que l'on désigne (*C*, *B*, *A*). Un premier multiplexeur (commandé par le bit de sélection S_0) implante le choix de *B* ou de *A*. La sortie de ce premier multiplexeur est reliée (ou « cascadée ») au second multiplexeur qui, avec le bit de sélection S_1, implante le choix de *C* ou (*B* ou *A*) issu du premier multiplexeur. Le schéma global est comme suit :

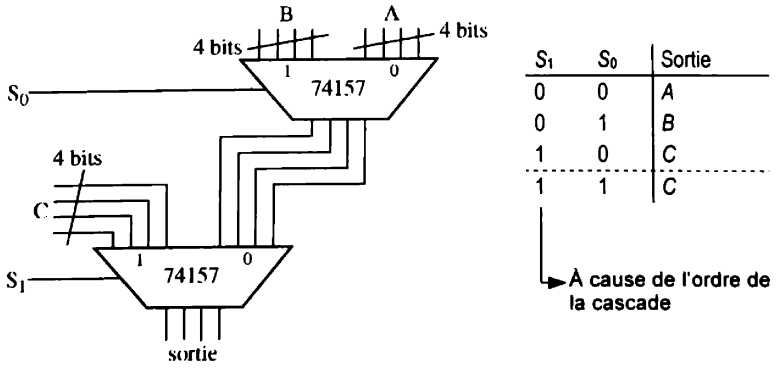


Figure 4.8

La table de droite indique les sélections possibles selon les valeurs de S_1 et de S_0. Comme on le voit, les cas (S_1, S_0) = (1, 0) ou (S_1, S_0) = (1, 1) entraînent la même sélection, c'est-à-dire « *C* » en sortie, puisque, dès que S_1 est activé, la sortie du premier multiplexeur n'est plus considérée.

4.5 MULTIPLEXEURS ET THÉORÈME D'EXPANSION DE SHANNON

Les sections précédentes montrent que l'utilisation des multiplexeurs pour l'implantation des tables de vérité en circuiterie logique est très intéressante. Par contre, sans simplification additionnelle, on se retrouve avec des solutions coûteuses en matière de matériel employé. Le théorème d'expansion de Shannon[8], dans sa version algébrique ou graphique, permet de réaliser de telles simplifications et donc de recourir à des multiplexeurs de plus petite taille. C'est ainsi que, grâce aux simplifications étudiées dans cette section, on pourra par exemple implanter une table de vérité dans un multiplexeur 4 à 1, alors que, sans simplification, un multiplexeur 8 à 1 aurait été nécessaire.

APPLICATION ALGEBRIQUE DU THEOREME D'EXPANSION DE SHANNON

Étudions d'abord la version algébrique du théorème de Shannon. Le théorème de Shannon stipule que n'importe quelle expression booléenne $F(X_1, X_2, \ldots X_N)$ de N variables peut-être réécrite (on dit aussi étendue) selon l'expression suivante :

$$F(X_1,\ldots X_N) = X_1 \cdot F(1, X_2,\ldots X_N) + \overline{X}_1 \cdot F(0, X_2,\ldots X_N)$$

Illustrons cette expression avec un exemple.

Exemple : Appliquer le théorème d'expansion de Shannon à la fonction F suivante :

$$F = [C + B + A]$$

Solution : Grâce au théorème d'expansion de Shannon, on peut réécrire la fonction F comme suit :

$$F = C \cdot \underbrace{[1 + B + A]}_{1} + \overline{C} \cdot [0 + B + A]$$

$$F = C \cdot 1 + \overline{C} \cdot [B + A]$$

Dès lors, cette « expansion » de F permet son implantation directe dans un multiplexeur 2 à 1, comme on le montre à la figure 4.9 :

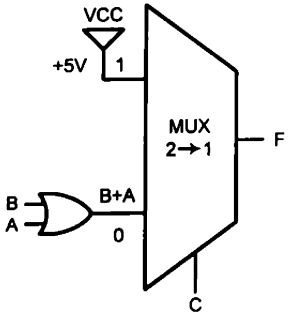


Figure 4.9

[8] La preuve de ce théorème est hors du propos de ce livre.

Pour se convaincre de ce résultat, procédons à sa vérification en dressant la table de vérité complète :

				Expression de départ			Expression avec MUX
$\overline{C}$	C	B	A	$C+B+A$	$B+A$	$\overline{C}\cdot[B+A]$	$C+\overline{C}\cdot[B+A]$
1	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	1
0	1	0	0	1	0	0	1
0	1	0	1	1	1	0	1
0	1	1	0	1	1	0	1
0	1	1	1	1	1	0	1

Les deux expressions fournissent le même résultat.

Figure 4.10

On aurait aussi pu vérifier l'égalité des deux expressions en appliquant le théorème de l'adjonction vu au chapitre 2. Se rappelant que $X + \overline{X}\cdot Y = X+Y$, on a en effet :

$$F = C\cdot 1 + \overline{C}\cdot[B+A]$$
$$F = C + \overline{C}\cdot[B+A]$$
$$F = C + [B+A] = C + B + A$$

Et on retrouve l'expression de départ.

Bien évidemment, le théorème d'expansion de Shannon, après sa première application, peut être appliqué de nouveau sur l'expression obtenue. C'est ainsi que ce théorème peut être appliqué une 2^e fois et même une N^e fois sur chacune des expressions obtenues à chaque étape intermédiaire (si l'expression d'origine contient N variables). Chaque application du théorème permet de mettre en évidence une variable de l'expression d'origine. On comprend immédiatement l'intérêt de l'application répétée de ce théorème pour l'utilisation des multiplexeurs. Une application permet le recours à un multiplexeur 2 à 1 ; deux applications permettent le recours à un multiplexeur 4 à 1, etc. Évidemment, la N^e application, qui implique le recours à un multiplexeur 2^N à 1, nous renvoie à la solution non simplifiée (donc à la liste complète des mintermes de la fonction) ; il s'agit ainsi de savoir s'arrêter au moment opportun. Reprenons l'expression du début de cette section et rappliquons ce théorème sur chacune des expressions issues de la première application. Cela permettra de mettre en évidence la variable X_2 :

$$\begin{aligned} F(X_1...X_N) &= X_1\cdot F(1,X_2...X_N) + \overline{X}_1\cdot F(0,X_2...X_N) \\ &= X_1X_2F(1,1,X_3...X_N) + X_1\overline{X}_2\cdot F(1,0,X_3...X_N) + \\ &\quad + \overline{X}_1X_2\cdot F(0,1,X_3...X_N) + \\ &\quad + \overline{X}_1\overline{X}_2\cdot F(0,0,X_3...X_N) \end{aligned}$$

L'expression résultante se prête bien à la réalisation par multiplexeur 4 à 1 :

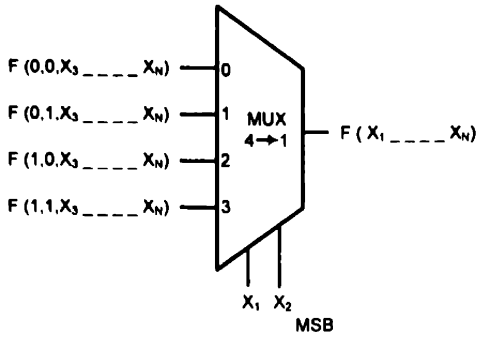


Figure 4.11

Exemple : Appliquons une seconde fois le théorème d'expansion de Shannon à la fonction F de l'exemple précédent.

On avait :

$$F = [C + B + A]$$

$$F = C \cdot \underbrace{[1 + B + A]}_{1} + \overline{C} \cdot [0 + B + A]$$

$$F = C \cdot 1 + \overline{C} \cdot [B + A] = C + \overline{C} \cdot [B + A]$$

Et la seconde application conduit à :

$$F = B \cdot \{C + \overline{C} \cdot [1 + A]\} + \overline{B} \cdot \{C + \overline{C} \cdot [0 + A]\}$$

$$F = CB + \overline{C}B \cdot \underbrace{[1 + A]}_{1} + C\overline{B} + \overline{C}\overline{B} \cdot \underbrace{[0 + A]}_{A}$$

$$F = CB + \overline{C}B + C\overline{B} + \overline{C}\overline{B}A$$

L'implantation dans un multiplexeur 4 à 1 est alors directe :

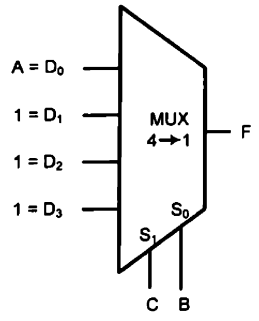


Figure 4.12

On peut vérifier que la sortie F sera toujours égale à 1 sauf si $C = B = A = 0$, ce qui correspond bien à la table de vérité de la fonction $F = C + B + A$. La réapplication du théorème de Shannon conduirait à l'utilisation d'un multiplexeur 8 à 1 au coût matériel

prohibitif. Par contre, aux entrées du multiplexeur, on ne retrouverait plus de variables, mais seulement des 1 et des 0.

APPLICATION GRAPHIQUE DU THEOREME D'EXPANSION DE SHANNON

Comme les deux exemples précédents l'ont montré, l'application algébrique directe du théorème d'expansion de Shannon est fastidieuse et sujette aux erreurs, surtout si on applique celui-ci plusieurs fois. Heureusement, le théorème de Shannon peut être mis en œuvre directement sur les tables de Karnaugh, ce qui rend son utilisation très commode. Bien évidemment, les deux approches, algébrique ou graphique, conduisent au même résultat.

La méthode pour l'implantation graphique d'une fonction F quelconque dans un multiplexeur à l'aide du théorème d'expansion de Shannon est la suivante :

1° Il s'agit tout d'abord de déterminer quelle(s) variable(s) doit(doivent) être mise(s) en évidence.

2° Ceci permet de subdiviser la table de Karnaugh de la fonction d'intérêt en blocs selon la(les) variable(s) qui reste(nt).

3° Pour chacun des blocs, on évalue alors l'expression minimale selon la méthode de Karnaugh.

4° Finalement, on dresse le schéma complet avec le multiplexeur. La(les) variable(s) mise(s) en évidence sert(servent) de bit de sélection. En entrée, on trouve les fonctions minimales pour chaque bloc. La sortie du multiplexeur correspond à F.

Exemple : Implanter la fonction F suivante (C, B, A ; C = *MSB*) dans un multiplexeur 4 à 1. Les variables de sélection du multiplexeur seront B et A. Le théorème d'expansion de Shannon sera appliqué graphiquement.

$$F = C\overline{B} + \overline{C}BA$$

Solution : Cette fonction correspond aux mintermes $m(3, 4, 5)$. Il s'agit donc de dessiner la table de Karnaugh en identifiant les blocs correspondant à la paire B et A des bits de sélection du multiplexeur :

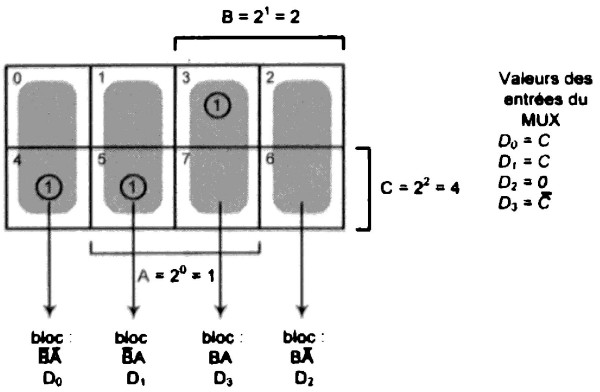


Figure 4.13

Pour chacun des blocs (D_0, D_1, D_2, D_3), on a aussi encerclé les simplifications selon la méthode de Karnaugh. En fait, il n'y a rien à simplifier ici puisque tous les 1 sont solitaires. De ce dessin, on obtient alors les valeurs des *Di* (indiquées à droite), ce qui permet finalement d'obtenir le schéma complet final. On a employé ici le logiciel Xilinx WebPack qui inclut la librairie *M4_1E* de multiplexeur 4 à 1 avec un bit d'activation (noter que le bit d'activation *E* est mis à « 1 » pour l'activation continue du MUX) :

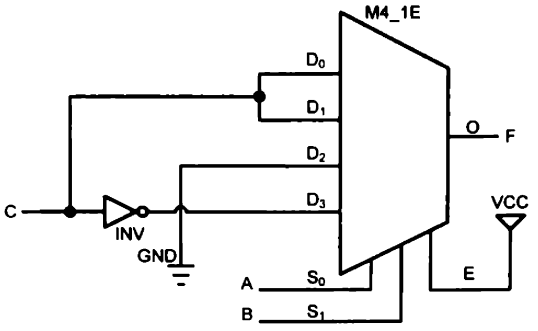


Figure 4.14

Ici, le choix des variables mises en évidence nous était imposé (*B* et *A*). Pour une implantation dans un multiplexeur 4 à 1, d'autres choix (*C* et *B*, *C* et *A*) auraient aussi été possibles. La question se pose, quelles variables choisir ? Comme variables mises en évidence il faut choisir celles qui conduisent aux simplifications maximales, dans les blocs de la table de Karnaugh. L'habitude permet de voir « à l'œil » quel est l'agencement idéal. L'exemple suivant, plus complexe, illustre d'ailleurs l'effet du choix des variables mises en évidence sur la solution.

Exemple : Soit la fonction suivante *F*(*D*, *C*, *B*, *A* ; *D* = *MSB*) :

$$F = M(0, 2, 5, 8, 9, 10, 11, 12, 13, 14) + X(1,3)$$

où *M* = Maxterme.

Fournir le circuit optimal à l'aide d'un multiplexeur (le plus petit possible parmi les formats suivants : 16 à 1, 8 à 1, 4 à 1) et de portes simples (le cas échéant).

Solution : Traçons d'abord la table de Karnaugh de cette fonction :

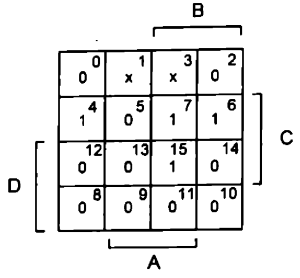


Figure 4.15

Les MUX 16 à 1 et 8 à 1 sont très coûteux en fonction de la circuiterie nécessaire : on avait d'ailleurs établi précédemment que le coût d'un MUX 8 à 1 était de 39 portes simples. Un MUX 4 à 1 apparaît alors directement comme le choix plus judicieux. Un MUX 4 à 1 possède deux bits de sélection, avec quatre entrées, cela nous fait six possibilités[9] de combinaisons de variables à considérer : (*D,C*), (*B,A*), (*D,A*), (*D,B*), (*C,B*), (*C,A*) qui subdivisent la table de Karnaugh selon les blocs suivants :

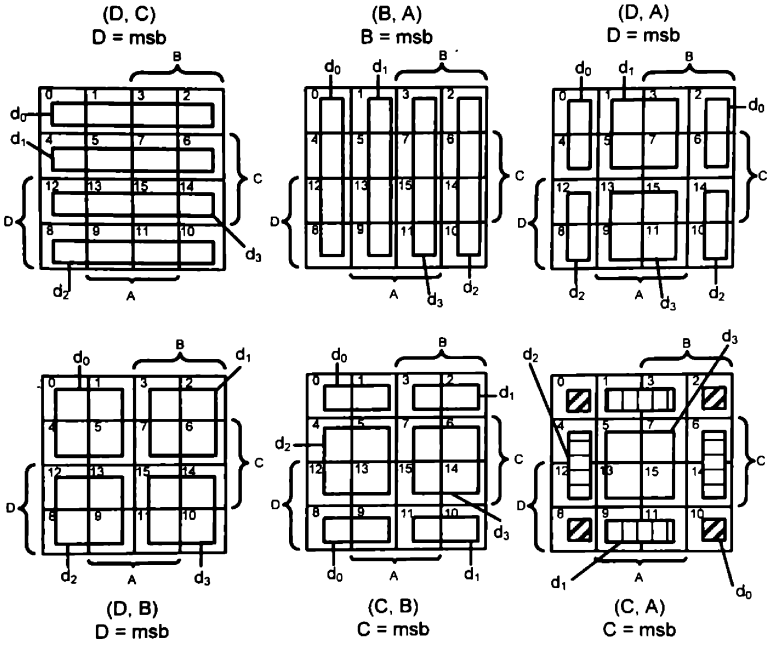


Figure 4.16

L'observation de ces modèles et de la table de Karnaugh de la fonction montre que la combinaison de variables (*C,A*) fournit des expressions simples pour les d_i, comme on le voit en superposant les subdivisions sur la table de Karnaugh :

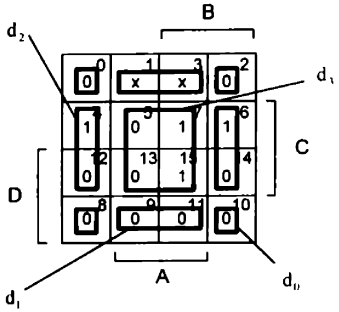


Figure 4.17

[9] Cela correspond à la combinaison $\binom{4}{2} = \frac{4!}{2!\,2!} = \frac{24}{4} = 6$.

On voit alors directement que :

$d_0 = 0$
$d_1 = 0$
$d_2 = \overline{D}$
$d_3 = B$

ce qui donne le circuit suivant (MUX avec bit d'activation) :

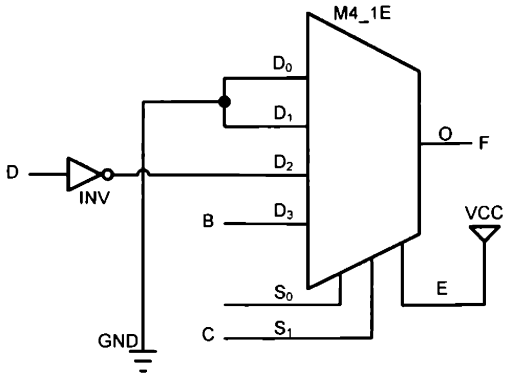


Figure 4.18

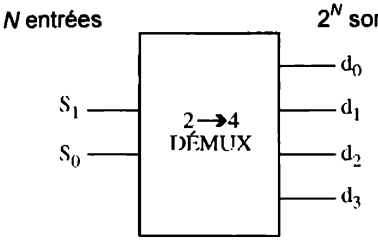
Le coût de ce circuit, en fonction du nombre de portes doubles élémentaires, suivant la structure interne vue à la section 4.3, est de (MUX avec bit d'activation E) : quatre portes quadruples (pour les bits E, S_0, S_1, d_i) et une porte OU quadruple (pour la sortie F), une porte quadruple étant équivalente à trois portes doubles ; on a donc finalement $5 \times 3 = 15$ portes simples. On constate donc que l'approche avec MUX 4 à 1 avec bits de sélection *judicieusement choisis* est un bon choix. En effet, les sources des d_i ne coûtent rien ici puisque les inverseurs sont exclus du décompte du coût. Par contre, si on avait choisi comme bits de sélection D et C, le circuit aurait été plus complexe. En effet, $d_1 = B + \overline{B}A = B + A$ (avec l'adjonction), donc un coût de une porte double élémentaire à ajouter (sans compter les autres d_i).

4.6 DÉMULTIPLEXEUR (OU DÉCODEUR)

DEMULTIPLEXEUR : DEFINITION, REALISATION

Les démultiplexeurs sont des circuits très utiles pour l'interface des microprocesseurs, des microcontrôleurs, des mémoires, des registres. Ils peuvent aussi servir pour l'implantation des fonctions booléennes (l'application qui nous intéresse particulièrement ici). Un démultiplexeur (DÉMUX en abrégé) est aussi appelé décodeur d'adresse, décodeur de minterme ou simplement décodeur. Le démultiplexeur est un circuit qui possède N entrées et 2^N sorties. Une seule sortie est activée à la fois, elle correspond au poids

binaire des N entrées présentes. Ce circuit « décode » donc le poids binaire d'une entrée. Le schéma d'un DÉMUX 2 à 4 est le suivant :



Principe du décodeur :
Sera activée la ligne d_i qui correspond au poids binaire des entrées présentes en S_1, S_0. Ainsi, pour une combinaison donnée des entrées, une seule sortie sera active.

Ici, N = 2, on a donc deux entrées et quatre sorties ($4 = 2^2$).

Figure 4.19

Une autre façon de décrire le fonctionnement d'un DÉMUX est de dire que les sorties correspondent aux mintermes des entrées. Par exemple, pour le démultiplexeur 2 à 4 de la figure 4.19 (donc deux entrées et quatre sorties), la sortie d3 correspond au minterme m3 et sera activée si S_1S_0 = 1 1 (minterme $m3$: S_1S_0). De même, d_1 correspond au minterme $m1$ et sera activée si S_1S_0 = 0 1 (minterme $m1$: $\overline{S_1}S_0$), etc. Puisqu'un décodeur fournit tous les mintermes des variables d'entrée, son utilisation pour l'implantation des tables de vérité est très commune (et sera traitée en détail un peu plus bas).

Voyons maintenant comment ces circuits peuvent être réalisés à l'aide de portes simples. Reprenons le cas du DÉMUX 2 à 4 de la figure 4.19 :

Réalisation d'un DÉMUX 2 à 4

(sans bit d'activation)

Fonctionnement du DÉMUX :

- si S_1S_0 = 11→ d_3 = 1
- si S_1S_0 = 10→ d_2 = 1
- si S_1S_0 = 01→ d_1 = 1
- si S_1S_0 = 00→ d_0 = 1

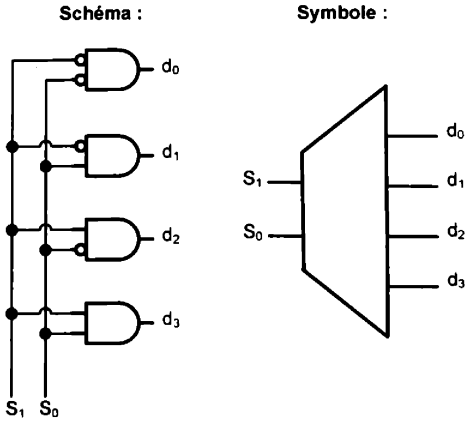


Figure 4.20

Du schéma précédent, on constate que le coût d'un démultiplexeur 2 à 4 sans bit d'activation est de quatre portes simples à deux entrées (tel qu'il a été spécifié précédemment, on exclut les inverseurs dans le calcul du coût total).

De façon pratique, les démultiplexeurs sont généralement « actifs bas », ce qui est réalisé avec des portes NON-ET en sortie. La logique négative (sorties « actives bas ») est pratique dans la famille TTL puisqu'une entrée non connectée est dès lors inactive (il faut vraiment

mettre un signal à « 0 » par exemple en le connectant à la masse pour que celui-ci soit activé). La figure 4.21 illustre d'ailleurs l'utilisation de la logique négative pour un démultiplexeur simple 1 à 2 :

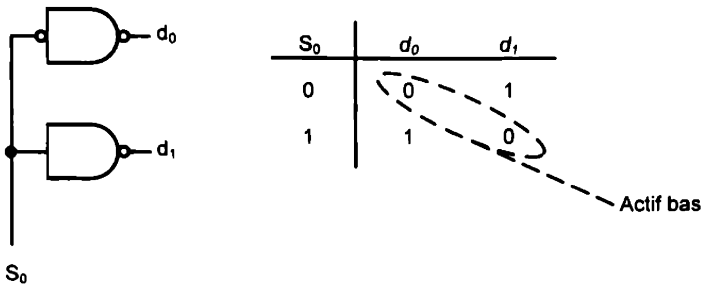


Figure 4.21

DEMULTIPLEXEUR AVEC BIT D'ACTIVATION

Certains démultiplexeurs possèdent un bit d'activation G (ou E pour *enable*, activation en français) qui est pratique pour bloquer sur demande leur fonctionnement. La figure 4.22 illustre le cas pour un démultiplexeur 1 à 2. Ce bit G commun peut servir aussi pour la transmission d'information. En effet, pour un S_i donné, la valeur de G est transmise à la sortie correspondant à S_i.

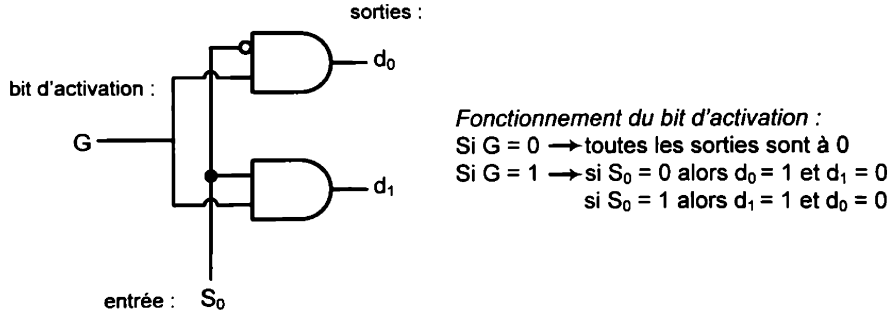


Figure 4.22

Ce type de DÉMUX avec bit d'activation peut servir dans une application intéressante, celle du « routeur » qui permet de faire transiter des signaux de différentes provenances vers diverses destinations à travers un lien de communication unique. Dans sa forme la plus simple, un routeur est obtenu en connectant en cascade une paire multiplexeur-démultiplexeur. Le schéma suivant illustre ceci pour un lien de communication à un bit. Cet arrangement transmet un des quatre signaux (A, B, C, D) vers n'importe quelle sortie d_A, d_B, d_C ou d_D à l'aide d'un seul lien :

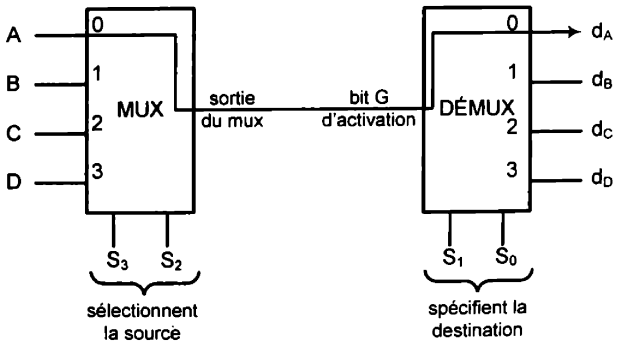


Figure 4.23

Ici, (S_3, S_2) spécifient la source, et (S_1, S_0) la destination. L'exemple illustre la transmission du signal de A vers d_A avec $S_3 = S_2 = S_1 = S_0 = 0$.

DEMULTIPLEXEURS COMMUNS

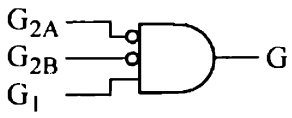
Il y a plusieurs circuits de démultiplexeurs communs en TTL (famille 74xxx), et qui sont d'ailleurs aussi disponibles sous forme de librairies dans des logiciels de conception tels que le Xilinx WebPack déjà discuté. Par exemple, on retrouve les circuits suivants :

Circuits communs		*Nombre d'entrées*		*Nombre de sorties*
74139	(2 DÉMUX)	2	→	4 (16 broches)
74138	(1 DÉMUX)	3	→	8 (16 broches)
74154	(1 DÉMUX)	4	→	16 (24 broches)

Figure 4.24

Le nombre de broches fait référence au nombre de points de connexions externes dans le cas des boîtiers TTL. Ces chips ont une « fonction d'activation » G qui permet de désactiver les sorties comme il a été vu précédemment. Il est à noter que cette fonction d'activation est parfois composée de plusieurs bits. Par exemple, dans le cas du 74138 (que nous étudierons dans un instant), on a : $G = G_1\overline{G}_{2A}\overline{G}_{2B}$. La fonction d'activation permet de mettre en cascade plusieurs démultiplexeurs.

Illustration de la fonction d'activation pour le démultiplexeur 74138 :



$$G = G_1\overline{G}_{2A}\overline{G}_{2B}$$

Figure 4.25

Il est à noter que, pour certains démultiplexeurs, le décodage n'est pas binaire, ainsi en est-il des circuits suivants :

7442 - BCD → décimal

7443 - XS-3[10]→ décimal

LE DEMULTIPLEXEUR 74138

Un démultiplexeur commun que nous emploierons est le 74138. Il s'agit d'un DÉMUX (décodeur) 3 à 8. Ce circuit a la particularité d'avoir trois bits d'activation *G1*, *G2A* et *G2B* tel qu'il a été indiqué précédemment. Plusieurs bits d'activation permettent de mettre plusieurs DÉMUX en cascade, comme nous le verrons plus loin. Une autre particularité du 74138 est qu'il fonctionne en logique négative ainsi qu'une sortie Y_i sera considérée active si nulle. Voici d'ailleurs son schéma et sa table de vérité :

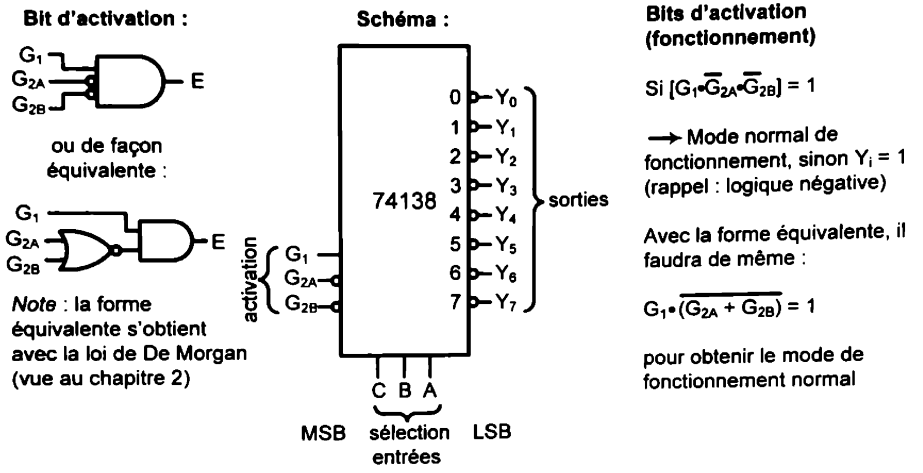


Figure 4.26 Schéma du 74138

[10] XS-3 ou Excess -3 est un code employé dans certains – anciens – ordinateurs. Il s'agit en fait du code BCD auquel on ajoute « 0011 ». Ainsi, 0_{10} devient $0011_{xs\text{-}3}$, 1_{10} devient $0100_{xs\text{-}3}$, 9_{10} devient $1100_{xs\text{-}3}$. L'avantage essentiel est que, pour les soustractions, un nombre XS-3 peut être « complémenté 9 » en inversant tous les bits, tout comme un nombre binaire. Lors des opérations arithmétiques (+, –), il faut cependant s'assurer de toujours conserver le « 3 » en conformité avec la définition du XS-3.
Voir : http://www.answers.com/topic/excess-3

ACTIVATION		Sélection			Sorties							
G_1	$(G_{2A} + G_{2B})$	C	B	A	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
X	1	X	X	X	1	1	1	1	1	1	1	1
0	X	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1	1	1
1	0	1	0	0	1	1	1	1	0	1	1	1
1	0	1	0	1	1	1	1	1	1	0	1	1
1	0	1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	1	1	1	1	1	1	1	0

Figure 4.27 Table de vérité du 74138

On peut aussi écrire l'équation de ce DÉMUX comme suit :

$$\overline{Y_i} = (sélection)_i \cdot (activation)$$

où Y_i = sortie de rang i ;

sélection = minterme i ;

activation = valeur de la combinaison des bits d'activation ;

i = 0, 1, 2^N–1 (avec N = nombre de bits de sélection).

Exemple : Pour la sortie 5, on a :

$$\overline{Y_5} = (G_1 \overline{G}_{2A} \overline{G}_{2B}) \cdot (C\overline{B}A).$$

Ainsi, Y_5 = 0 (c'est-à-dire la sortie 5 est active ; rappel : logique négative) seulement si on a activation (c'est-à-dire si $\left[G_1 \cdot \overline{G_{2A}} \cdot \overline{G_{2B}}\right] = 1$) et que la sélection vaut 5 (c'est-à-dire si (C, B, A) = (1, 0, 1)). Autrement dit, la sortie 5 est active si le minterme $m5$ est présent aux entrées. Ce résultat très important nous servira à réaliser la synthèse des tables de vérité par démultiplexeur.

Finalement, voici la structure interne du 74138[1] :

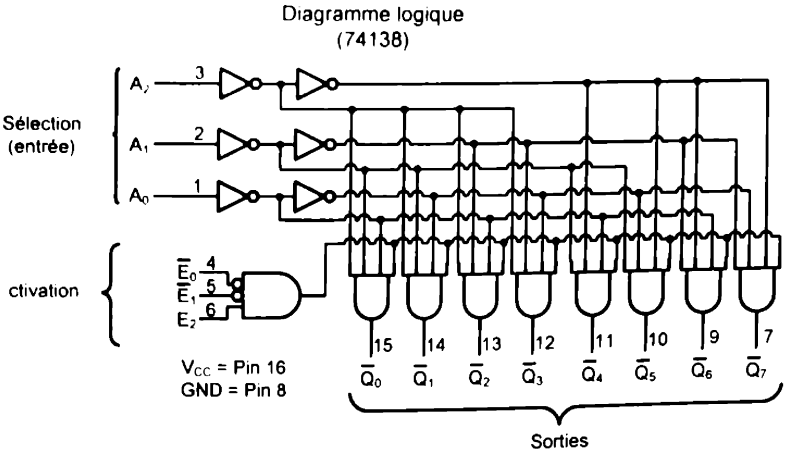


Table de fonctionnement

Entrées						Sorties							
$\overline{E_0}$	$\overline{E_1}$	E_2	A_0	A_1	A_2	$\overline{Q_0}$	$\overline{Q_1}$	$\overline{Q_2}$	$\overline{Q_3}$	$\overline{Q_4}$	$\overline{Q_5}$	$\overline{Q_6}$	$\overline{Q_7}$
H	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
X	X	L	X	X	X	H	H	H	H	H	H	H	H
L	L	H	L	L	L	L	H	H	H	H	H	H	H
L	L	H	H	L	L	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	L	H	H	H	H	H
L	L	H	H	H	L	H	H	H	L	H	H	H	H
L	L	H	L	L	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	L	H	H
L	L	H	L	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	L

Notes pour la table :

1. H = niveau logique HAUT
2. L = niveau logique BAS
3. X = niveau logique SANS IMPORTANCE

Figure 4.28

[1] http://www.semiconductors.philips.com/acrobat_download/datasheets/74HC_HCT138_CNV_2.pdf

4.7 SYNTHÈSE PAR DÉMULTIPLEXEURS : UTILISATION DES DÉCODEURS POUR LA SYNTHÈSE DES TABLES DE VÉRITÉ

On rappelle qu'un démultiplexeur est un circuit combinatoire à N entrées et 2^N sorties et dont seule une sortie est activée à la fois. Le rang de la sortie activée correspond à la valeur binaire des bits de sélection. Autrement dit, la sortie i est active si le minterme m_i est présent aux entrées.

Puisqu'un démultiplexeur peut fournir TOUS les mintermes correspondants aux variables impliquées, on pourra employer un tel circuit pour synthétiser une fonction booléenne quelconque. Ici, il est important de se rappeler la relation qui existe entre les mintermes m_i et les Maxtermes M_i (section 2.9) : $M_i = \overline{m_i}$. Ainsi, si un démultiplexeur fonctionne en logique négative (comme le 74138 étudié précédemment), les sorties Y_i correspondent en fait aux Maxtermes. Par contre, si un démultiplexeur fonctionne en logique directe ses sorties correspondent aux mintermes. La synthèse par démultiplexeurs pourra donc se réaliser soit selon une Somme de Produits (SdP) en considérant les « 1 » dans la table de Karnaugh (avec les mintermes) ou selon un Produit de Sommes (PdS) en considérant les « 0 » dans la table de Karnaugh (avec les Maxtermes). Ceci nous permettra d'opter soit pour une solution SdP ou PdS la plus simple selon le nombre de « 1 » ou de « 0 » présent dans la table de Karnaugh (section 2.10).

La synthèse par démultiplexeur se réalise donc comme suit :

a) Solution de type Somme de Produits (SdP) :

   1° activant adéquatement le décodeur (pour le 74138 ce sont les bits G_1, G_{2A}, G_{2B}),

   2° choisissant les **mintermes** actifs de la fonction,

   3° regroupant ceux-ci avec une porte OU.

b) Solution de type Produit de Sommes (PdS) :

   1° activant adéquatement le décodeur (pour le 74138 ce sont les bits G_1, G_{2A}, G_{2B}),

   2° choisissant les **Maxtermes** actifs de la fonction,

   3° regroupant ceux-ci avec une porte ET.

Les deux exemples suivants illustrent ces approches.

Exemple : On veut implanter la fonction $S(C,B,A) = m(2,3,7)$ à l'aide d'un décodeur 74138 (DÉMUX 3 à 8) et de quelques portes simples (C = bit le plus significatif, *MSB*). Employez une approche SdP.

Solution : On emploie le décodeur pour la synthèse des mintermes sommés. On active aussi celui-ci. La solution est directe et le schéma est donc le suivant :

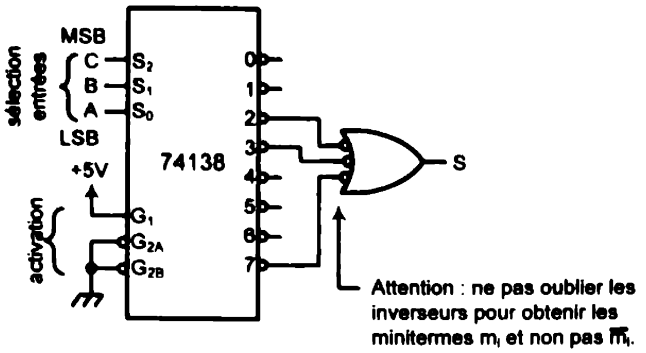


Figure 4.29

Dès que la sélection aux entrées correspondra à un des mintermes d'intérêt : $m2$, $m3$ ou $m7$ la sortie S sera activée (= 1).

Il est intéressant ici de se rappeler notre discussion sur les solutions en portes NON-ET seulement (section 2.7). On avait conclu de celle-ci que, grâce aux lois de De Morgan, $\overline{A} + \overline{B} = \overline{A \cdot B}$. Ceci permet de substituer directement une porte NON-ET à la place de la porte OU (et des inverseurs d'entrée) dans le circuit précédent, ce qui donne :

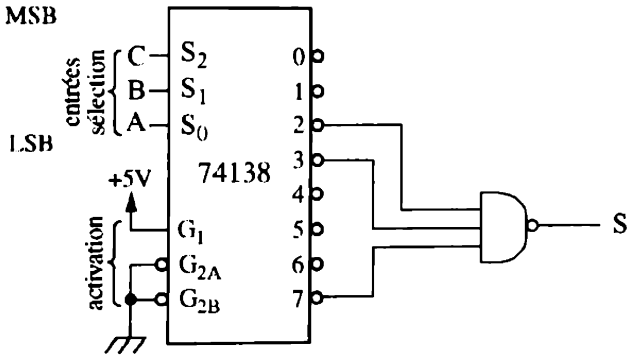


Figure 4.30

Exemple : On veut implanter la fonction $S(C,B,A) = m(2,3,7)$ à l'aide d'un décodeur 74138 (DÉMUX 3 à 8) et de quelques portes simples (C = bit le plus significatif, *MSB*). Employez une approche PdS (donc même exemple que précédemment mais une synthèse selon les Maxtermes).

Solution : On emploie le décodeur pour la synthèse des Maxtermes multipliés entre eux. On active aussi celui-ci. Selon la table de Karnaugh, les Maxtermes actifs sont ici : M_0, M_1, M_4, M_5, M_6.

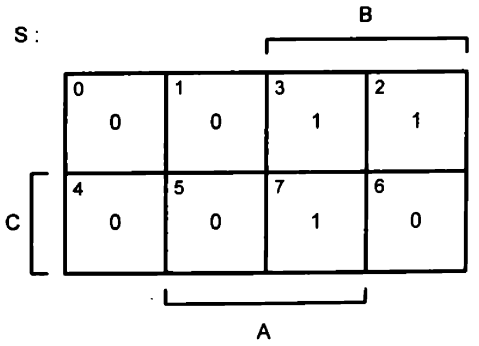


Figure 4.31

La solution est directe, et le schéma est donc le suivant :

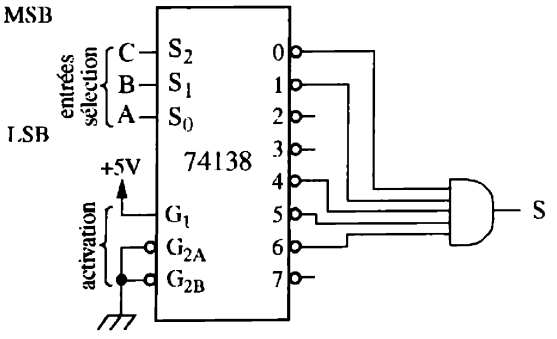


Figure 4.32

Il est intéressant de vérifier quelques cas pour « se convaincre » de la validité de cette solution ! Ainsi, si $C = B = A = 0$, on doit obtenir $S = 0$. Dans ce cas la sortie Y_0 est active (à zéro) et le produit final donne bien une sortie nulle puisque 0 (n'importe quoi) = 0 ! Par contre, si $C = B = A = 1$, on doit obtenir $S = 1$. Dans ce cas la sortie Y_7 est active (à zéro), toutes les autres sont à « 1 » et le produit final donne bien une sortie $S = 1$ puisque $1 \cdot 1 \cdot 1 \cdot 1 \cdot 1 = 1$.

On note qu'ici l'approche PdS sans simplification exige plus de logique que l'approche SdP de l'exemple précédent (à cause de la porte ET quintuple).

En simplifiant, on peut arriver cependant à une solution plus « économique ». La simplification graphique avec démultiplexeur se réalise d'une façon similaire à ce qui a été vu pour les multiplexeurs (Section 4.5.2). Ainsi, l'observation de la table de Karnaugh précédente révèle qu'on peut employer avantageusement un démultiplexeur 1 à 2 avec B comme bit de sélection. Cela permet de sélectionner directement le bloc de quatre zéros correspondants au Maxtermes M_0, M_1, M_4, M_5. Il reste alors seulement le Maxterme M_6 qui correspond à $\overline{C} + A$ (la variable B étant déjà considérée par le démultiplexeur, c'est pour cela qu'elle n'apparaît pas ici) :

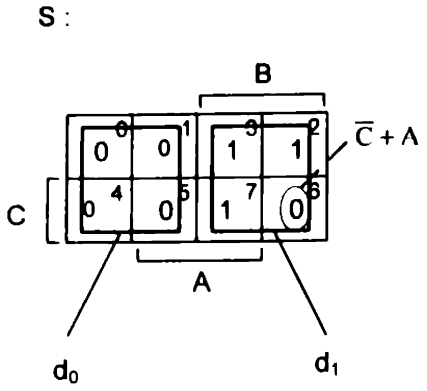


Figure 4.33

Le schéma est donc le suivant (on se rappelle que la solution PdS considère les « 0 ») :

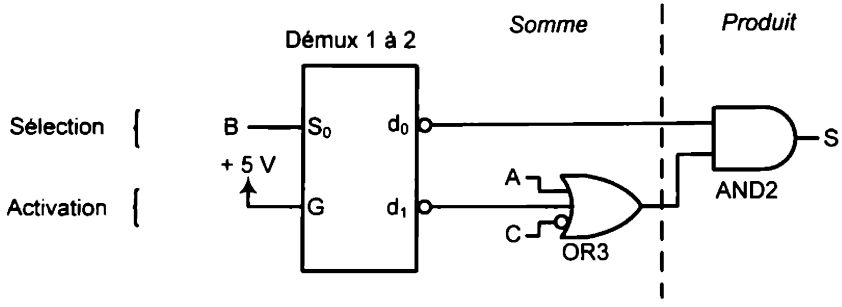


Figure 4.34

Dans la figure précédente, on a indiqué expressément les parties *Produit* et *Somme* séparées par un trait pointillé, ceci montre bien la structure P – d - S.

Il est intéressant de refaire cet exercice avec une solution SdP et un Démux. Cette fois, on se rappelle qu'une SdP s'intéresse aux « 1 » seulement. C'est ainsi que les cases 0, 1, 4, 5 toutes à 0 dans la table de Karnaugh précédente ne sont pas d'intérêt pour la solution SdP, ce qui explique la non-connexion de la sortie 0 du démux (si $B = 0$), figure suivante. Puisqu'on s'intéresse aux 1, il faut bien sûr inverser les sorties du démux (pour avoir un 1 en sortie du démux si $B = 1$). On obtient alors la solution de la figure suivante. Sur la figure, on a indiqué expressément les parties *Somme* et *Produit* séparées par un trait pointillé, ceci montre bien la structure S – d - P. On pourra vérifier quelques cas pour s'assurer que la solution fonctionne! Par exemple, tous les cas où $B = 0$ donneront 0 (puisque d1 = 1). Pour les cas où $B = 1$ (d1 = 0 alors), cela dépendra des valeurs de A et C : dès que $A = 1$ ou $C = 0$, on aura 1 en sortie, tel que prévu.

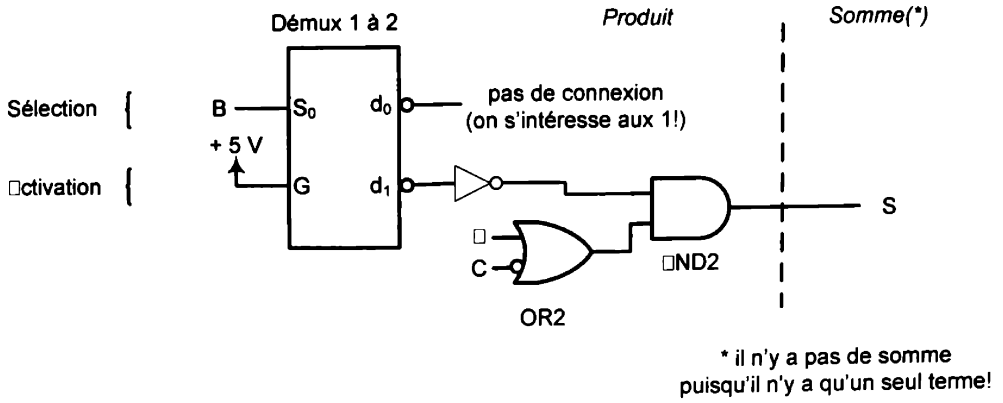


Figure 4.34.1

Ce circuit avec démultiplexeur est plus simple que les précédents (surtout en considérant le « coût d'un démultiplexeur 1 à 2 par rapport à celui d'un démultiplexeur 3 à 8). Vérifions le cas $C = 1$, $B = 1$, $A = 0$, la sortie S donne bien $S = 0$. Les autres cas se vérifient de façon similaire. La partie droite du circuit – la porte ET de sortie – correspond au « produit », le reste à gauche correspond à la « somme ». On a bien une solution de type PdS.

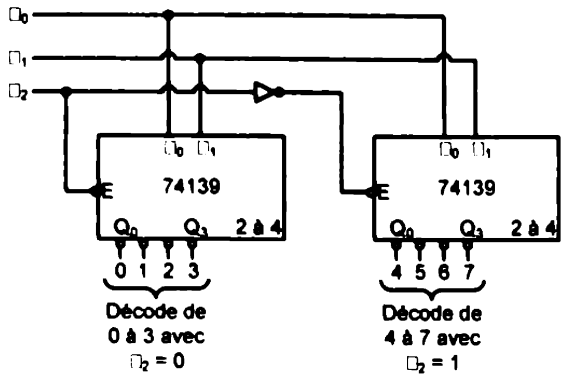
Note : on a employé ici un démultiplexeur avec un seul bit d'activation G.

4.8 MISE EN CASCADE DE DÉMULTIPLEXEURS

Les démultiplexeurs peuvent être mis en cascade, ce qui est pratique pour en réaliser de plus grands à partir de plus petits. L'exemple suivant illustre le principe.

Exemple : Réaliser un démultiplexeur 3 à 8 à partir de deux démultiplexeurs 74139 2 à 4.

Solution : Un démultiplexeur 3 à 8 possède trois entrées et huit sorties. Un démultiplexeur 2 à 4 possède deux entrées et quatre sorties. Puisque 8 sorties = 2 × 4 sorties, il faudra donc employer deux démultiplexeurs 74139 pour la réalisation demandée. Ainsi, pour la réalisation du démultiplexeur 3 à 8, deux des trois entrées seront communes aux deux démultiplexeurs mis en cascade, alors que la troisième servira à commander un des deux démultiplexeurs mis en cascade puisqu'il faut seulement qu'un des deux démultiplexeurs soit actif à la fois. Le schéma suivant reprend cette discussion. Les lignes A_0 et A_1 sont communes et la ligne A_2, passée à travers un inverseur et connectée au bit d'activation, active au choix un des deux démultiplexeurs. Le premier démultiplexeur décodera les mintermes 0, 1, 2, 3 ; et le second, les mintermes 4, 5, 6, 7 :



A_2	A_1	A_0	
0	0	0	Premier
0	0	1	74139 actif
0	1	0	
0	1	1	
1	0	0	
1	0	1	Second
1	1	0	74139 actif
1	1	1	

Figure 4.35

Exemple : Déterminer le nombre de démultiplexeurs requis pour réaliser un démultiplexeur 5 à 32 à partir de démultiplexeurs 74138 et 74139.

Solution : Le 74139 contient deux démultiplexeurs 2 à 4. Le 74138 contient un démultiplexeur 3 à 8. Il nous faut cinq entrées (A_0, A_1, A_2, A_3, A_4) et 32 sorties. La combinaison la plus simple est donc 32 sorties = 4 × 8 sorties. Ainsi, on emploiera quatre démultiplexeurs 74138. Le premier décodera les mintermes 0 à 7 ; le deuxième, les mintermes 8 à 15 ; le troisième, les mintermes 16 à 23 et le quatrième, les mintermes 24 à 31. Les trois lignes d'entrées les moins significatives (A_0, A_1, A_2) seront reliées en parallèle à ces quatre démultiplexeurs. Les deux autres lignes restantes (A_3, A_4) seront passées à travers un démultiplexeur 74139, ce qui permettra d'activer sélectivement un des quatre 74139. La disponibilité des bits d'activation permet la sélection du démultiplexeur 3 à 8 qui sera activé. *À noter* : les bits d'activation inemployés sont mis à 0 (masse) ou à 1 (VCC) pour se conformer à l'équation du 74138 : $G = G_1\overline{G}_{2A}\overline{G}_{2B}$. Quant au 74139, il est continuellement activé avec $\overline{G} = 0$.

Il faut donc quatre 74138 et un 74139. Le schéma complet est le suivant :

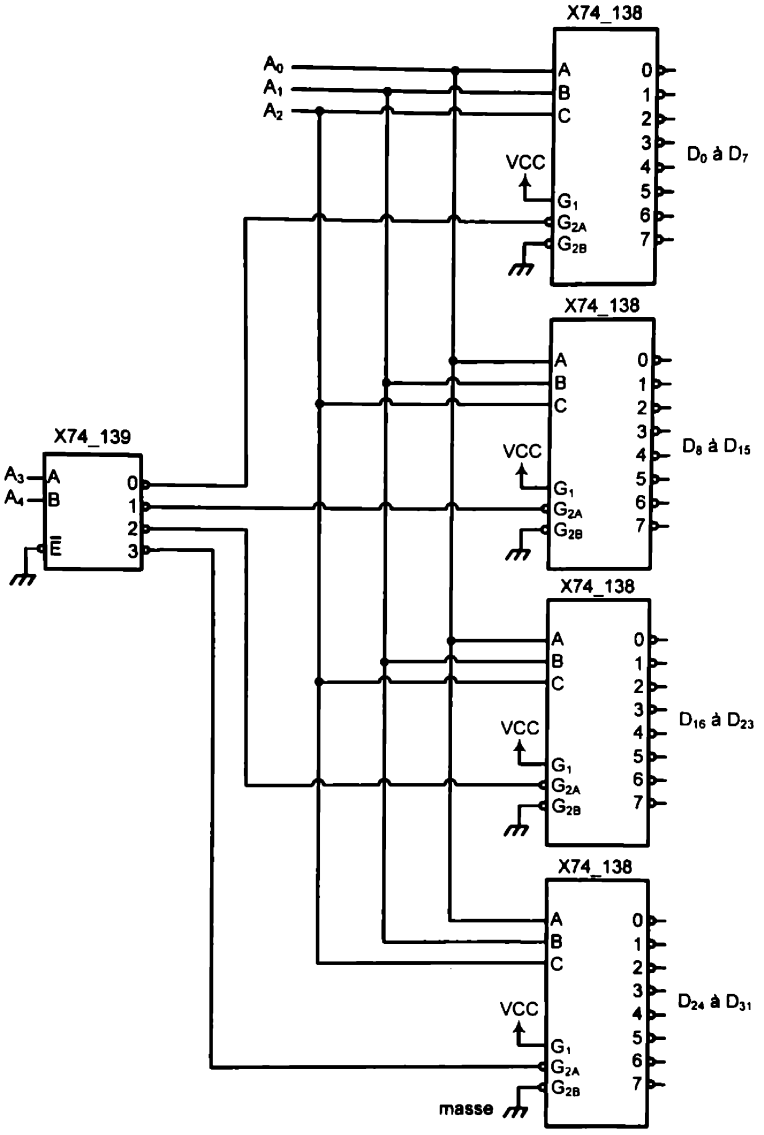


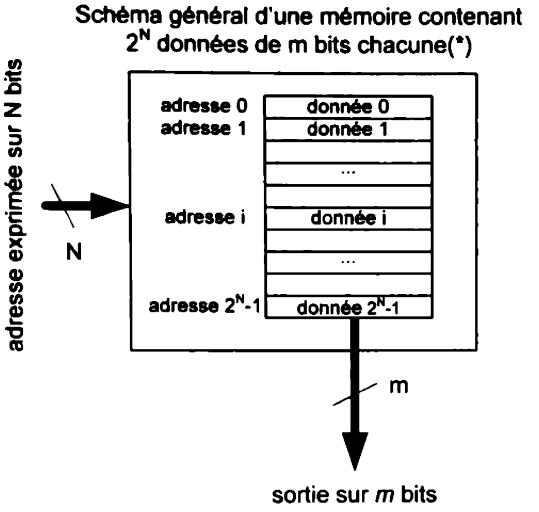
Figure 4.36

4.9 CIRCUITS DE MÉMOIRE

Une mémoire morte (ou ROM en anglais : *Read Only Memory*) est un circuit qui mémorise une donnée (sur un ou plusieurs bits) pour chacune de ses adresses. Si les adresses sont fournies sur N bits, la mémoire contiendra 2^N données (figure 4.37). Le contenu d'une mémoire morte est inscrit en usine et est inaltérable par la suite. La variante PROM est

programmable par l'usagère, et les variantes EPROM, EEPROM sont effaçables et reprogrammables par l'usagère :

- PROM : *Programmable Read Only Memory*, mémoire programmable et en lecture seulement, non effaçable.
- EPROM : *Erasable Programmable Read Only Memory*, mémoire effaçable (par exposition à la lumière ultraviolette) et programmable.
- EEPROM : *Electrically Erasable Programmable Read Only Memory*, mémoire effaçable électriquement et programmable. La mémoire FLASH très commune est une variante (par exemple celle employée dans les «clés de mémoire USB[12]») : les données y sont effacées par blocs et non par octets individuels comme dans les mémoire EEPROM).



(*) selon les variantes, le contenu est altérable

Figure 4.37

Il existe un autre type de mémoire très important appelé mémoire RAM (pour *Random Access Memory*) dont le contenu est accessible de façon aléatoire et qui peut être lu et réécrit à volonté. Différentes technologies existent pour la réalisation des mémoires RAM. Par exemple, dans les mémoires RAM dynamiques (DRAM), le bit à retenir est mémorisé sous la forme d'une charge électrique dans un condensateur minuscule. Comme cette charge s'efface avec le temps, on doit périodiquement lire toute la mémoire et réécrire son contenu. Au chapitre suivant nous étudierons une autre technologie basée sur le feed-back qui peut être employée pour la réalisation de circuits de mémorisation.

[12] «USB flash drive» en Anglais

Pour les mémoires EPROM, la lumière ultraviolette[13] très énergétique permet d'effacer le contenu de la mémoire en une quinzaine de minutes, typiquement. L'exposition se fait grâce à une fenêtre de quartz insérée au milieu du boîtier (voir figure 4.42, plus bas).

Inutile de dire que les circuits de mémoire sont essentiels en circuiterie numérique. En particulier, l'usage des mémoires ROM, EPROM ou EEPROM est très répandu ; ils servent entre autres :

- Dans les ordinateurs personnels pour loger le BIOS (*Basic Input Output System*, ou système d'entrée-sortie élémentaire). Il s'agit généralement de quelques kilo-octets de mémoire qui permettent le lancement de l'OS (*Operating System* en anglais, ou système d'opération).
- Dans les imprimantes pour la configuration des polices de caractère.
- Dans le cas des microcontrôleurs pour la mémorisation du programme de travail. Exemples d'utilisation : fours à micro-ondes, lecteurs de cassettes vidéo, CD et DVD, contrôleurs de moteur des véhicules, contrôleurs de clavier d'ordinateur, etc. Les microcontrôleurs sont des ordinateurs à un boîtier, autonomes et complets.

STRUCTURE D'UN CIRCUIT DE MEMOIRE

On peut voir la structure logique d'un ROM (ou de tout autre type de mémoire en fait) à partir de celle d'un démultiplexeur. En effet, prenons le cas d'une mémoire à 8 bits, son schéma pourrait être le suivant (à noter les bulles inverseuses qui « s'annulent » de part et d'autre ; le démultiplexeur employé ici n'a pas de bit d'activation) :

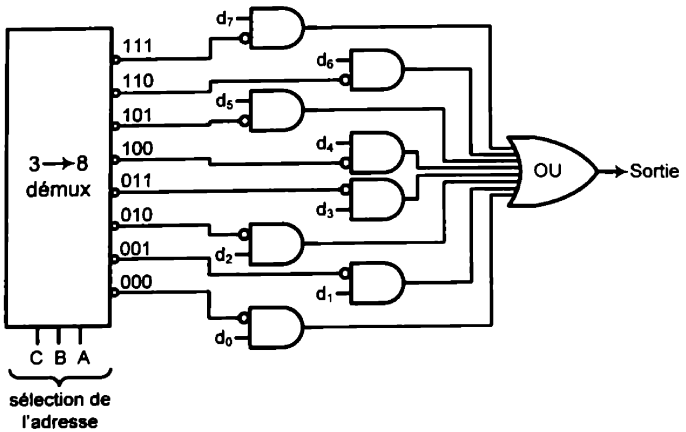


Figure 4.38

Avec : d_0, d_1, ..., d_7 , bits contenus dans la mémoire.

Ici, le démultiplexeur sert de décodeur d'adresse. Par exemple, si *CBA* = '011', c'est d_3 qui apparaîtra en 'Sortie' puisque toutes les autres sorties du DÉMUX sont à 0 et que donc

[13] Attention, cette lumière est très dangereuse pour les yeux : ne jamais la fixer !

toutes les portes ET sont à 0 (sauf celle de d_3). Finalement, la porte OU réalise la somme : $0 + \ldots 0 + 1 \bullet d_3 + 0 + \ldots + 0 = d_3$.

Évidemment, le schéma précédent n'est pas très pratique. Ainsi, pour la réalisation de grandes mémoires, plutôt que d'employer des portes ET pour les bits de données, des « interrupteurs internes » (sous la forme de fusibles), qui peuvent être mis à 1 ou à 0 selon le contenu de la mémoire, sont employés.

Plus formellement, donnons maintenant la définition d'une mémoire selon le schéma précédent :

> *Définition d'un circuit de mémoire à 2^N adresses (avec un bit par adresse)* : Circuit qui emmagasine des valeurs ordonnées (bit de contenu : $d_0, \ldots, d_{2^N-1}$) selon leurs adresses respectives (sélectionnées avec les N bits de sélection d'adresse). L'équation générale de ce circuit de mémoire est la suivante (avec bien sûr une seule adresse active à la fois) :
>
> $$\text{Sortie} = \text{adresse}_{2^N-1}\, d_{2^N-1} + \ldots + \text{adresse}_i\, d_i + \ldots + \text{adresse}_0\, d_0.$$

Bien sûr, au lieu de stocker un bit par adresse comme ici, on peut en stocker plus, alors les d_i de l'équation précédente feront référence à des octets, des mots, etc.

Pour la mémoire de huit bits de la figure 4.38, on a $N = 3$ (lignes d'adresse : C, B, A), ce qui donne :

$$\underset{\bullet}{Sortie} = \underset{\bullet}{d_7 CBA} + \underset{\bullet}{d_6 CB\overline{A}} + \ldots + \underset{\bullet}{d_0 \overline{CBA}}$$

d_i étant le contenu et $\bullet$ étant l'adresse (représentée par son minterme correspondant).

Selon l'équation précédente, on voit qu'une mémoire peut être vue comme un circuit SdP à contenu programmable.

D'une façon plus pratique, la *structure logique interne* d'une mémoire peut aussi être vue comme ceci en assumant un type de réalisation employant des « interrupteurs internes » (sous la forme de fusibles). À la figure 4.39, on représente le cas d'une mémoire à 4×2 bits, donc deux bits par adresse ($m = 2$) et quatre adresses ($N = 2$) :

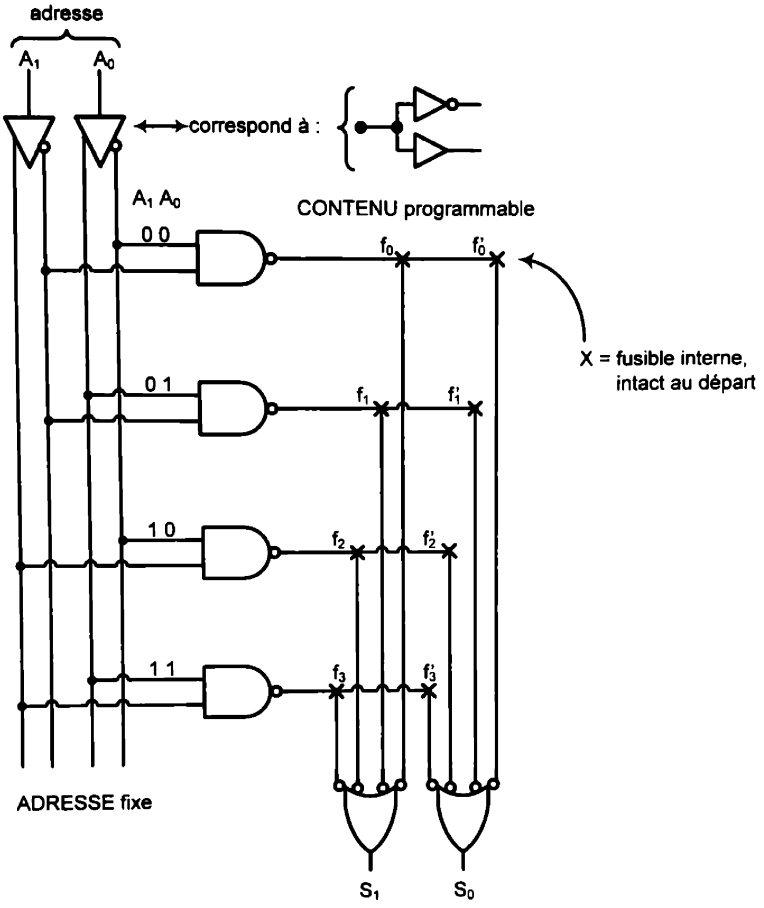


Figure 4.39

La partie gauche du circuit correspond à la structure d'un démultiplexeur (sans bit d'activation) tel qu'il a été vu précédemment. L'équation pour la sortie S_1 est :

$$S_1 = \overline{\overline{A_0}\,\overline{A_1} + f_0} + \overline{\overline{A_0}\,\overline{A_1} + f_1} + \overline{\overline{A_0}\,A_1 + f_2} + \overline{A_0 A_1 + f_3},$$

f_0, f_1, f_2, f_3 correspondant à l'état des fusibles respectifs. Un fusible sauté donne une valeur logique 1 (voir plus bas), par contre, à l'état intact, il a une valeur logique 0 (il y a continuité).

On peut obtenir une équation similaire pour S_0 avec les f_i. À noter dans ce schéma les tampons à sortie double (directe et inversée) pour les lignes d'adresse A_0 et A_1 qui permettent de dresser un schéma clair et lisible. Cette méthode de tracé est très commune ; nous la réemploierons lors de notre étude des PLD (*Programmable Logic Device* en anglais, ou circuit logique programmable) un peu plus loin.

La figure 4.40 fournit les explications concernant le fonctionnement des fusibles. Lorsque le circuit sort d'usine, en absence de configuration donc, tous les fusibles sont intacts, ce qui est représenté par des « *X* » dans les schémas (des mémoires et des PLD vus plus bas). Lors de la configuration du circuit, les fusibles sont « brûlés » sélectivement. S'il est brûlé (à gauche sur la figure), un fusible entraîne un 1 logique. En effet, la ligne de sortie « flottante » est forcée au 1 logique par une résistance *pull-up* interne (*R* sur la figure). Cette valeur « flottante » est causée par l'absence de contact provoquée par l'ouverture du fusible. S'il est intact, le contact est bon ; il y a continuité, et la résistance *pull-up* n'a pas d'effet (à droite sur la figure), le 0 (ou le 1 logique) issu de la porte ET correspondante se transmet directement (le 0 logique « forcera » la ligne de sortie à la valeur 0, étant donné la valeur élevée de la résistance R).

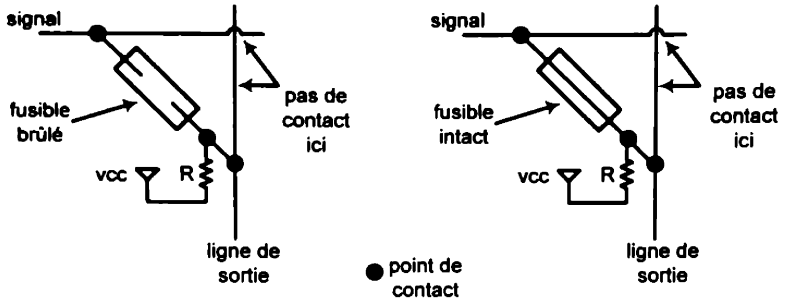


Figure 4.40

Dans le cas de la technologie OTP (*One Time Programming*, ou programmable une fois), les fusibles sont brûlés de façon permanente.

CIRCUIT DE MEMOIRE EPROM 2764

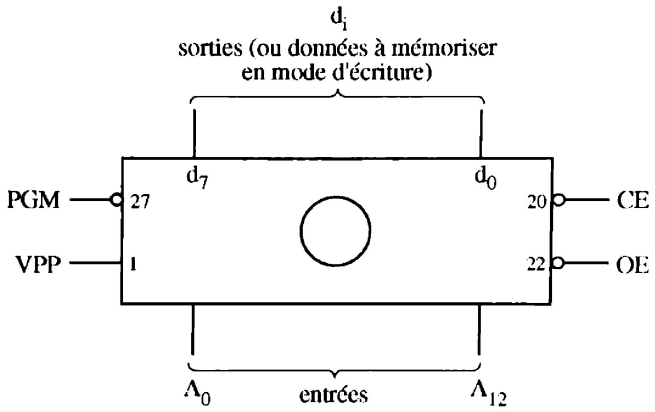
Étudions maintenant un véritable circuit de mémoire, et en particulier le circuit 2764, qui est de type EPROM et peut emmagasiner 64k bits (figures page suivante). Ce circuit est disponible dans un boîtier DIP (*Dual-In-line-Package*) de 28 pins. Les 64k bits (= 2^{16} = 65536 bits) sont configurés en 8192 (= 2^{13}) octets. N'importe quelle des 8192 adresses est accessible grâce aux 13 lignes d'adresse (A_0 à A_{12}). En cas de besoin, le 2764 peut être effacé par de la lumière ultraviolette ; le circuit complet retourne alors à l'état « 1111 1111 » (= FF_{hex}).

La figure 4.41 montre les connexions externes du 2764. On distingue :

- les 13 lignes d'adresse (A_0 à A_{12}) ;
- les 8 lignes de données (d_0 à d_7) ;
- les broches de configuration CE (*Chip Enable* en anglais, ou activation du circuit) et OE (*Output Enable* en anglais, ou activation de la sortie) pour l'activation du circuit et des sorties. Les sorties sont activées si CE et OE sont à 0. Autrement, les sorties du circuit « flottent » : elles ne sont ni à 1 ni à 0, mais dans un état de haute impédance (*Hi-Z* en anglais, *Z* est le symbole de l'impédance). OE et CE sont utiles pour sélectionner le circuit de mémoire lorsque ses sorties sont reliées à un bus de

données ou à d'autres circuits (seules les OE et CE du circuit d'intérêt sont mises à 0 ; ses sorties sont alors disponibles sur le bus) ;

- Les broches PGM et VPP qui servent à l'écriture du contenu de la mémoire. Pour écrire, il faut maintenir VPP à + 25 volts (ou + 12.5 volts selon le modèle) au lieu du VCC de + 5V comme d'habitude, et chaque fois qu'une « impulsion basse » (donc la succession des niveaux logiques « 1-0-1 ») est envoyée sur PGM (avec une durée du « 0 » de 50 ms), les valeurs présentes en d_i sont écrites (« brûlées ») à l'adresse présente en A_i. L'écriture complète d'un 2764 requiert trois à quatre minutes et se fait avec un programmeur d'EPROM : l'usagère écrit le contenu désiré dans un fichier, et le programmeur d'EPROM se charge de l'écriture dans la mémoire.



Pour le *mode lecture*, il faut : *OE* = *CE* = 0 et PGM = VPP = 1

Figure 4.41

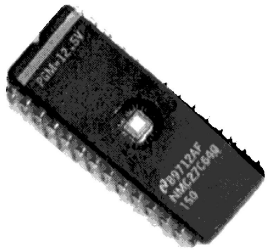
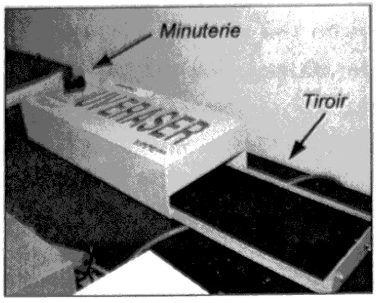


Figure 4.42 Photo d'une mémoire 2764.
On remarque la fenêtre de quartz qui sert pour effacer le contenu de la mémoire par lumière ultraviolette.
Le circuit électronique interne est bien visible avec ses fils de connexions.
Cette version se programme avec une tension de 12.5 volts tel qu'il a été indiqué.

Photo d'un programmeur d' PROM

Noter le réceptacle sans force d'insertion (en anglais : *zero insertion force socket*) : le circuit est posé, et les contacts se font en basculant le petit bras visible à gauche.

Photo d'une source lumineuse ultraviolette



Les composants à effacer sont déposés sur le tapis antistatique du tiroir, et le bouton de la minuterie est réglé. Pour raison de sécurité, il faut que le tiroir soit repoussé pour que l'appareil se mette en marche.

Figure 4.43

Exemple : Implanter les fonctions $F_j(D, C, B, A ; D = MSB)$ avec $j = 1, 2$ dans une mémoire ROM 16 × 2 ayant une commande OE (*output enable* en anglais) d'activation des sorties :

$$F_1 = m(0, 1, 3, 6, 8, 14, 15)$$

$$F_2 = M(0, 2, 4, 6, 8, 9, 12, 13, 15)$$

Le schéma de la mémoire est le suivant :

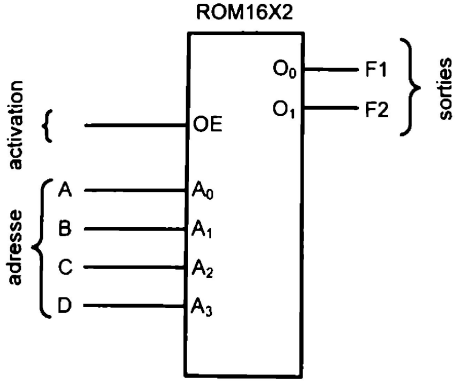


Figure 4.44

Solution : En fait, comme il s'agit d'une mémoire ROM, il suffit de fournir au manufacturier du circuit une liste avec le contenu pour chaque adresse. Celui-ci implantera le contenu lors de la fabrication du circuit. Il n'y a aucune simplification à faire puisque la mémoire accepte directement les quatre variables d'entrée ; il y a donc une adresse par minterme (ou Maxterme). Suivant l'énoncé du problème, on peut immédiatement dresser le tableau des contenus suivant :

	Adresses				Contenus	
Rang	*A3*	*A2*	*A1*	*A0*	$O0 = F_1$	$O1 = F_2$
0	0	0	0	0	1	0
1	0	0	0	1	1	1
2	0	0	1	0	0	0
3	0	0	1	1	1	1
4	0	1	0	0	0	0
5	0	1	0	1	0	1
6	0	1	1	0	1	0
7	0	1	1	1	0	1
8	1	0	0	0	1	0
9	1	0	0	1	0	0
10	1	0	1	0	0	1
11	1	0	1	1	0	1
12	1	1	0	0	0	0
13	1	1	0	1	0	0
14	1	1	1	0	1	1
15	1	1	1	1	1	0

Figure 4.45

La table de vérité du circuit est la suivante :

OE	Adresse	Sortie F_1	Sortie F_2
0	*i*	*Z*	*Z*
1	*i*	Contenu de l'adresse *i* pour F_1	Contenu de l'adresse i pour F_2

i = adresse quelconque de 0000_2 à 1111_2 (0_{10} à 15_{10}).

Z = état de haute impédance (en anglais *High Z*), donc sorties flottantes pour connexion à un bus de données, circuits multiples, etc.

Figure 4.46

4.10 CIRCUITS LOGIQUES PROGRAMMABLES (PLD)

Le terme de « circuit logique programmable », bien connu par son sigle PLD (de l'anglais : *Programmable Logic Device*), est un terme très générique qui regroupe une panoplie de circuits logiques qui sont configurables (ou programmables). Dans la vaste famille des PLD, on retrouve ainsi une catégorie très importante qui est basée sur la forme SdP (Somme de Produits), étudiée au chapitre 2 et qui a été présentée à la section précédente pour la structure interne des mémoires. Selon les types, le réseau ET, ou le réseau OU ou les réseaux ET et OU seront configurables. Par exemple, les mémoires que nous avons étudiées à la section précédente possèdent un réseau ET fixe (les adresses) et un réseau OU configurable (le contenu), tel qu'il a été indiqué à la figure 4.39, illustrant la mémoire 4 × 2 bits. À la section suivante, on étudiera les PAL (*Programmed Array Logic*) qui sont des PLD dont le réseau ET est configurable et le réseau OU fixe.

Quand on dit qu'un circuit est *programmable*, cela signifie que l'usagère peut le configurer avec une installation modeste (par exemple un programmeur d'EPROM, figure 4.43). Ceci est dit par opposition aux circuits qui sont configurés lors de leur fabrication en usine avec des masques lithographiques (par exemple les mémoires ROM). Ainsi, à strictement parler, une mémoire ROM n'est pas un PLD puisqu'elle n'est pas programmable. Une mémoire PROM ou EPROM l'est bien sûr.

Il y a aussi d'autres types de PLD, par exemple les FPGA (*Field Programmable Gate Array*, ou réseaux de portes configurables « dans le champ », donc faciles à configurer), les CPLD (*Complex PLD* ou circuit logique complexe programmable). Les FPGA seront abordés très brièvement à la section 4.13.

4.11 PAL : *PROGRAMMED ARRAY LOGIC*

Les PAL (de l'anglais, *Programmed Array Logic*, ou réseau logique programmable) sont des circuits programmables basés sur la forme SdP avec le réseau ET configurable et le réseau OU fixe. Il est à noter que le terme de PAL ™ est une marque de commerce de la compagnie Monolithic Memories, connue aussi sous le nom MMD[14]. En réponse à cela d'ailleurs, des concurrents ont mis sur le marché d'autres PLD baptisés PLA (*Programmed Logic Array*) dont nous discuterons plus bas.

Avant de présenter le schéma général des PAL, il faut d'abord introduire les conventions suivantes qui rendent les schémas plus clairs. Ainsi, une porte ET avec trois entrées (*C*, *B*, *A*) placées à l'horizontale selon ce que nous avons vu jusqu'à présent ($S = C \cdot B \cdot A$), peut être redessinée avec cette fois les trois entrées placées verticalement (à droite), ce qui permet une écriture plus compacte lorsque les entrées sont partagées par plusieurs portes :

[14] AMD (http://www.amd.com/us-en/) a acquis MMD en 1987.

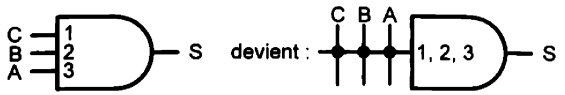


Figure 4.47

L'étape suivante consiste à introduire la notion de configuration par fusible tel qu'il a été discuté plus haut pour les circuits de mémoire. Ainsi, à chaque entrée (*C*, *B*, *A*) est associé un fusible. Si le fusible est intact, le signal est passé à l'entrée correspondante de la porte ET. On aura par exemple l'entrée 1 de la porte ET qui vaut *C* si le fusible de cette entrée est intact. Par contre, si le fusible associé à *C* est brûlé, alors l'entrée correspondante de la porte ET est fixée au 1 logique à cause de la résistance *pull-up* discutée précédemment (figure 4.40) ; le produit *S* sera donc $S = 1 \cdot B \cdot A = BA$ selon l'axiome $X \cdot 1 = X$. Tout comme pour les mémoires, on emploie le symbole « *X* » pour désigner la présence des fusibles. Notre porte ET triple devient ainsi :

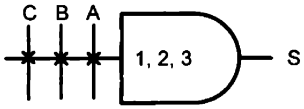


Figure 4.48

Toujours par souci d'alléger les schémas, les portes pour lesquelles aucun fusible n'est brûlé sont indiquées en marquant d'un « *X* » la porte elle-même, comme ceci :

Figure 4.49

À sa sortie d'usine, tous les fusibles d'un PLD sont intacts ; ces circuits sont alors « remplis de *X* ».

La même approche est suivie pour les portes OU, ce qui donne le schéma suivant. Ici, on a remplacé les fusibles par des points de contacts, donc $S = C + B + A$ sans possibilité de modification (il s'agit d'une porte OU fixe, telle qu'employée dans les PAL, comme nous le verrons dans un instant) :

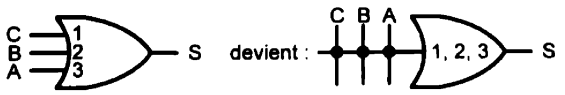


Figure 4.50

Ces notations nous permettent maintenant d'introduire la structure du PAL qui, tel qu'il a été mentionné plus haut, consiste en une SdP avec réseau ET programmable et réseau OU fixe. Chaque produit (chaque porte ET) réalise donc n'importe quel minterme des

variables présentes en entrée selon l'état des fusibles pour cette porte ET. Quant aux portes OU, chacune réalise la somme des mintermes qui lui sont connectés.

Le schéma pour un PAL à deux entrées, deux sorties et deux portes ET par porte OU est le suivant. À l'état neuf, tous les fusibles sont intacts du côté ET, ce qui explique les « X » dans ceux-ci. Par contre, du côté OU, la configuration a été spécifiée en usine et est inaltérable, d'où les « • » symbolisant les connexions fixes.

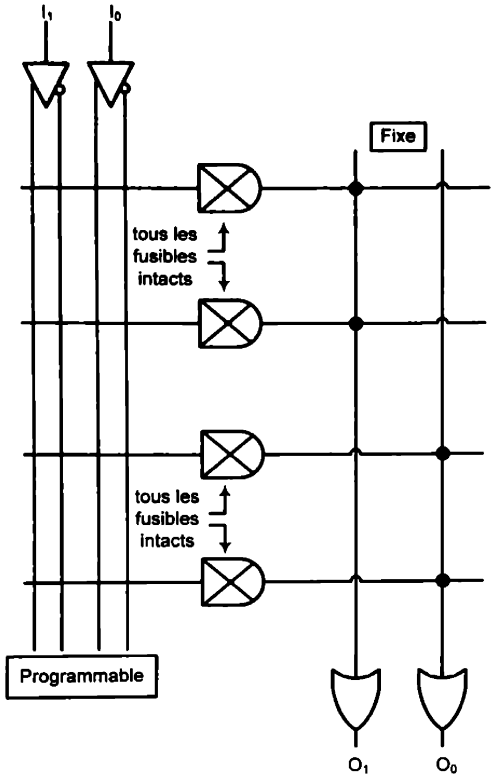


Figure 4.51

Si tous les fusibles sont intacts, il y a continuité de ceux-ci, et les équations pour les sorties du PAL sont :

$$O_0 = I_0\overline{I}_0 I_1\overline{I}_1 + I_0\overline{I}_0 I_1\overline{I}_1 = 0 \text{ et } O_1 = I_0\overline{I}_0 I_1\overline{I}_1 + I_0\overline{I}_0 I_1\overline{I}_1 = 0$$

Lorsque, pour une porte ET donnée, tous les fusibles sont intacts, cela signifie que la sortie de celle-ci sera nulle puisqu'on fait le produit des variables et de leur inverse, et que $I \cdot \overline{I} = 0$ selon l'axiome vu au chapitre 2. Par contre, si tous les fusibles sont brûlés, on a vu plus haut que le produit est alors au 1 logique[15]. Si seulement un des fusibles est

[15] Mais grâce aux résistances «pull-up» présentes, le signal pourra être forcé à zéro.

brûlé, on aura la valeur du produit des variables restantes à la sortie de la porte ET, tel qu'il a été discuté au début de la section.

Voyons maintenant comment une fonction booléenne peut être implantée dans un PAL.

Exemple : On demande de configurer la sortie O_1 du PAL de la figure 4.51 en OU-exclusif.

Solution : Cela revient à configurer la sortie O_1 avec la fonction :

$$O_1 = I_0 \oplus I_1 = I_0\overline{I}_1 + \overline{I}_0 I_1$$

Puisque la sortie d'un PAL implante une SdP, la configuration de celle-ci est directe. Il faudra donc brûler sélectivement quatre fusibles du côté O_1 seulement (la sortie O_0 n'est pas affectée et reste disponible pour une configuration ultérieure). Le schéma suivant présente le résultat :

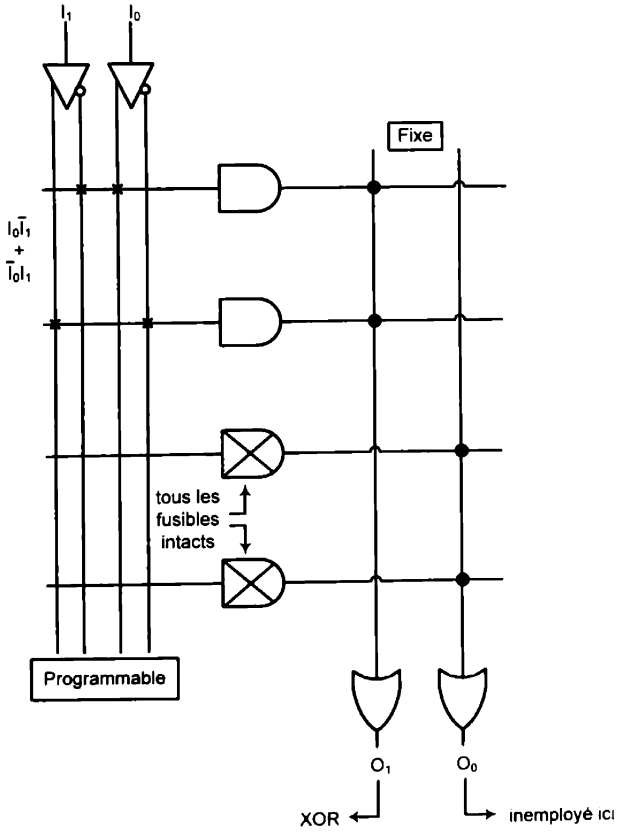
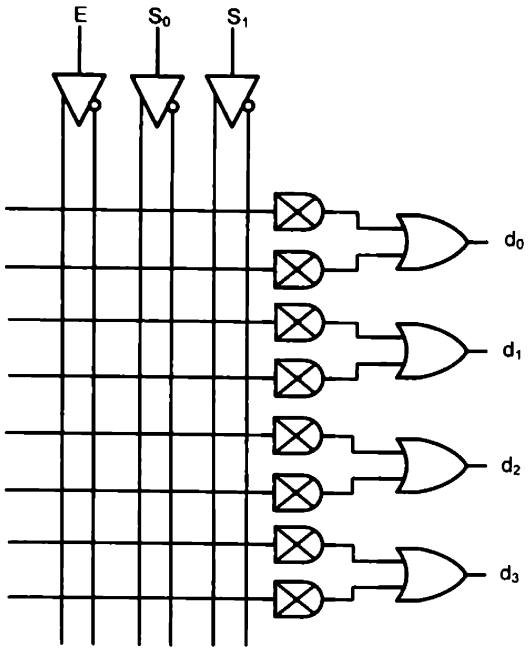


Figure 4.52

Exemple : Implanter le démultiplexeur 2 à 4 avec bit d'activation dans le PAL suivant :

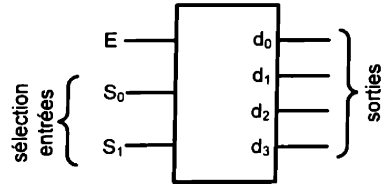


Il s'agit d'un PAL à quatre sorties, à trois entrées et à deux portes ET par porte OU

À noter : on a dessiné ici les portes OU « à l'horizontale » pour plus de clarté

Figure 4.53

Le schéma du démultiplexeur est fourni comme suit :



E = signal d'activation = Enable

Figure 4.54

Solution : Se rappelant la structure et le fonctionnement d'un démultiplexeur, on a immédiatement les équations suivantes pour les sorties :

$$d_0 = \overline{S}_0 \overline{S}_1 E$$
$$d_1 = S_0 \overline{S}_1 E$$
$$d_2 = \overline{S}_0 S_1 E$$
$$d_3 = S_0 S_1 E$$

Cela nous permet de dresser la table de vérité suivante en incluant le bit d'activation E :

E	S_1	S_0	d_0	d_1	d_2	d_3
L	X	X	L	L	L	L
H	0	0	H	L	L	L
H	0	1	L	H	L	L
H	1	0	L	L	H	L
H	1	1	L	L	L	H

Figure 4.55

À partir des équations, le schéma de l'implantation du DÉMUX dans le PAL s'obtient directement :

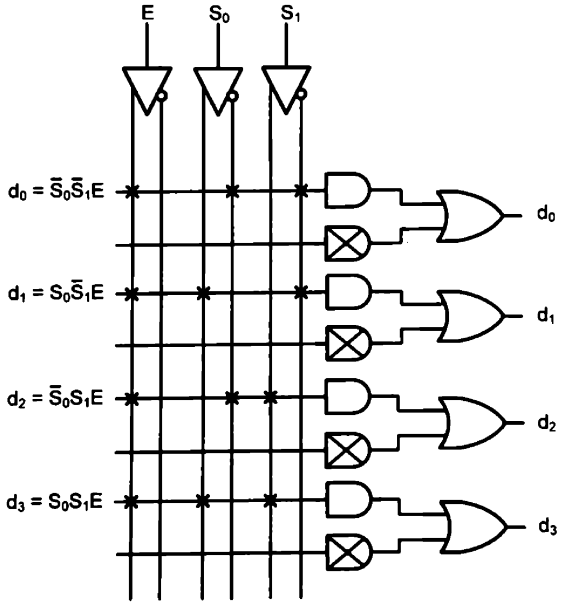


Figure 4.56

Puisque les équations pour les d_i ne contiennent chacune qu'un terme, seule une branche de chaque porte OU est utilisée ici (l'autre branche reste intacte et est inutilisable pour d'autres applications, car on affecterait alors la valeur des sorties).

Les PAL sont donc des circuits complexes de premier intérêt pour l'implantation des fonctions booléennes, comme on l'a vu avec les deux derniers exemples. D'un point de vue pratique, les PAL se programment généralement à l'aide de langages spécialisés tels ABEL, VHLD, Verilog (discutés précédemment). Pour une implantation réussie, il faut cependant être aussi conscient de leurs avantages et limitations.

AVANTAGES DES PAL

- Leur structure de type SdP (Somme de Produits) est facile à comprendre ; la configuration est donc évidente.
- Ce sont des circuits complexes incorporant plusieurs portes ET, plusieurs portes OU avec des entrées et des sorties multiples. Ils sont donc versatiles pour une foule d'applications. Ainsi, le PAL 10L8 (boîtier de 20 broches à 10 entrées et à 8 sorties) équivaut (voir la figure 4.57) à 16 portes ET à 20 entrées, plus 8 portes OU à 2 entrées.

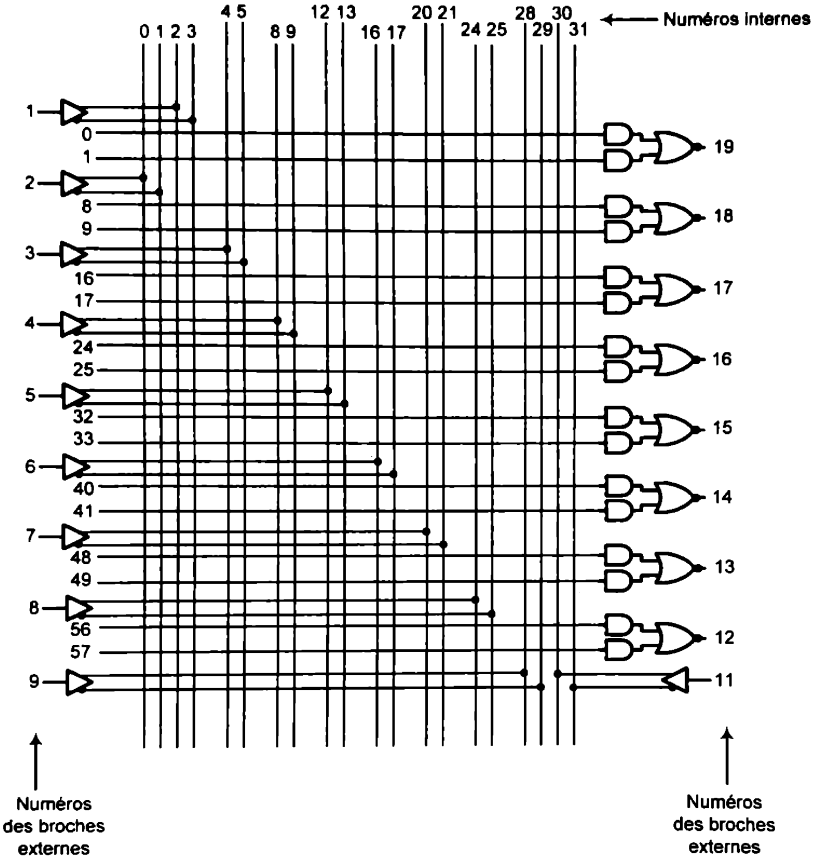


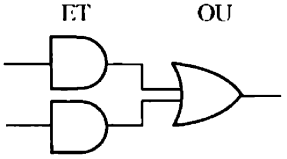
Figure 4.57 Schéma logique original d'un PAL 10 L8
(reproduit de *AMD - Monolithic Memories*, PAL Handbook, p. 3-29.)
Ces architectures avec réseaux ET-OU se retrouvent dans des circuits tels, entre autres, le UT22V10 Universal RAD PAL d'Aeroflex UTMC.

- Par rapport à d'autres types de PLD (comme les FPGA par exemple), ils coûtent peu cher ; ils sont donc fort employés.
- Ils peuvent être programmés en plusieurs étapes (comme on l'a vu pour l'exemple de la porte OU-exclusif), ce qui est avantageux lorsqu'un design se réalise en plusieurs fois.

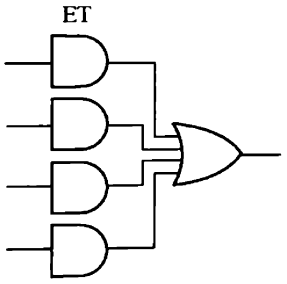
- Les délais de propagation sont plus petits que pour les circuits de mémoires puisqu'il n'y a pas de décodage d'adresse à faire.
- Leur réalisation requiert moins de place sur le silicium que pour les circuits ROM et TTL, car les cellules sont plus petites.

Limitations des PAL

- La limite fondamentale de tout PAL est le nombre de portes ET par porte OU (par exemple deux portes ET par porte OU pour le 10L8 de la figure 4.57). Ceci restreint la complexité des fonctions qu'on peut incorporer dans un PAL. Les deux figures suivantes illustrent ceci :



Ici, on peut effectuer la somme de deux termes (deux portes ET pour une porte OU : cas du 10L8).



Ici, on peut effectuer la somme de quatre termes (quatre portes ET pour une porte OU).

Figure 4.58

- Évidemment, les notions de table de Karnaugh étudiées précédemment peuvent servir à simplifier la table de vérité avant l'implantation dans le PAL. Cela est parfois essentiel autrement, « ça pourrait ne pas rentrer » s'il n'y a pas assez de produits par somme. Nous ferons un exemple un peu plus loin.
- Si le PAL utilisé est de type OTP (de l'anglais *One Time Programming*, ou programmable une fois seulement), il ne pourra être configuré qu'une seule fois, et toute erreur transformera le circuit en une résistance inutile (en une résistance uniquement puisque la seule « utilité » du circuit devient alors sa dissipation d'énergie en chaleur, si l'on peut dire !). Il est à noter cependant qu'il existe des versions de PAL reconfigurables après exposition à la lumière ultraviolette (comme pour les EPROM).

Exemple : On demande d'implanter la fonction $F(C,B,A$; $C = MSB) = m(3,4,7)$ dans le PAL suivant :

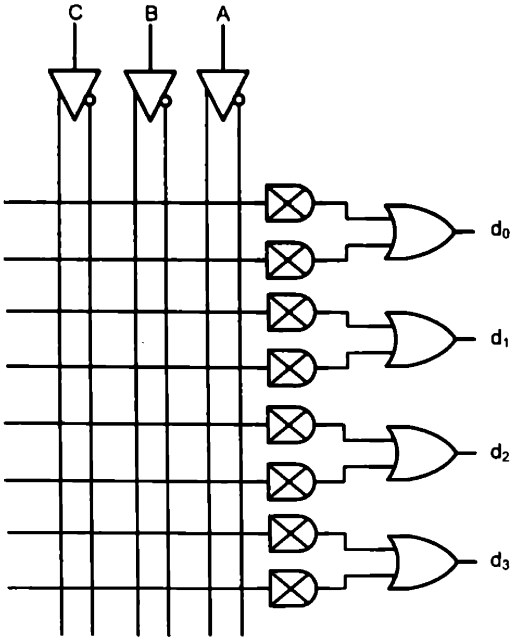


Figure 4.59

Solution : Puisqu'il y a trois mintermes ($m3$, $m4$, $m7$), la fonction ne peut pas être implantée directement dans le PAL fourni puisque les OU disponibles sont doubles, et non triples. La fonction doit donc être simplifiée au préalable. Examinons la table de Karnaugh de F :

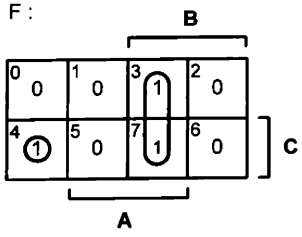


Figure 4.60

On voit que F peut être simplifiée avec deux termes seulement $F = BA + C\overline{BA}$, qui eux peuvent être incorporés dans le PAL :

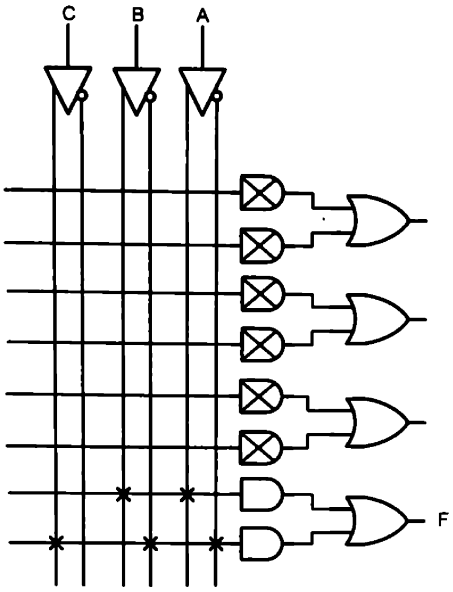


Figure 4.61

Le reste du PAL reste disponible pour d'autres applications configurées ultérieurement.

NOMENCLATURE DES PAL

Attardons-nous maintenant à la nomenclature des PAL. Leur numérotation suit généralement le format suivant :

EE T SS

EE étant le nombre d'entrées du réseau ET ; T, le type; et SS, le nombre de sorties du réseau OU.

Exemple : Pour le PAL 12H6, on a 12 entrées et 6 sorties. H signifie que ce PAL aura ses sorties non inversées.

Les PAL peuvent aussi être de différents types :

Type H

Les sorties sont directes (non inversées) ; on dit aussi : « active-haut ». Noter l'absence de « bulle inverseuse » en sortie, signifiant que la sortie est directe.

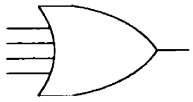


Figure 4.62 PAL de type H

Type L

Les sorties sont complémentées (inversées) ; on dit aussi : « active-bas ». La « bulle inverseuse » en sortie signifie que la sortie est inversée.

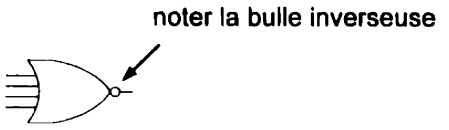


Figure 4.63 PAL de type L

Type C

Les sorties sont directes et complémentées. Cette fois, la sortie et son inverse sont simultanément disponibles.

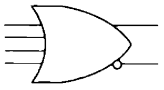


Figure 4.64 PAL de type C

Type X

Cette fois, on retrouve une porte OU-exclusif en sortie plutôt qu'une simple porte OU.

Ces PAL servent pour des applications plus complexes.

Figure 4.65 PAL de type X

Type R

Ces PAL contiennent des registres internes qui leur permettent de mémoriser leur état actuel. On étudiera les registres au chapitre 5.

Ces PAL servent pour des applications plus complexes.

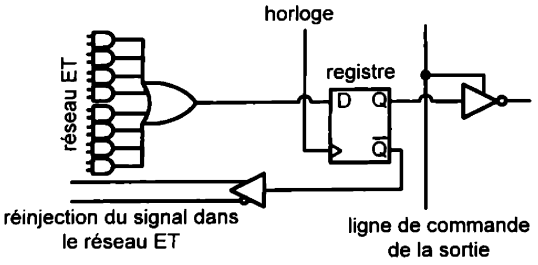


Figure 4.66 PAL de type R

Type A

Ces PAL peuvent effectuer des opérations arithmétiques élémentaires sur un bit. Ces PAL servent pour des applications plus complexes.

CASCADE DE PAL : PARTITION, RETOURNEMENT

La limitation essentielle des PAL est le nombre limité de produits ET pour un OU donné, sans compter non plus le nombre d'entrées et de sorties qui sont forcément limitées aussi. Parfois, même une simplification par tables de Karnaugh ne permet pas d'implanter les fonctions dans un PAL donné. Parfois, on n'a pas un nombre suffisant d'entrées ou de sorties. Que faire alors ? On peut bien sûr choisir un PAL plus complexe (avec plus de produits, plus d'entrées, plus de sorties). Une autre approche consiste à fractionner la solution logique et à la répartir dans plusieurs PAL, c'est ce qu'on appelle la partition. Il y a retournement (*folding* en anglais) quand une ou des sorties d'un PAL servent d'entrée(s) à celui-ci ou à d'autres PAL pour lesquels la logique a été partitionnée. L'exemple suivant illustre ce point.

Exemple : Implanter simultanément dans deux PAL à trois entrées et à quatre sorties les deux fonctions suivantes :

$$F = \overline{A}B + C\overline{D}$$

$$G = A\overline{B}\overline{C} + A\overline{B}D + CD$$

Les PAL à employer ont la structure suivante :

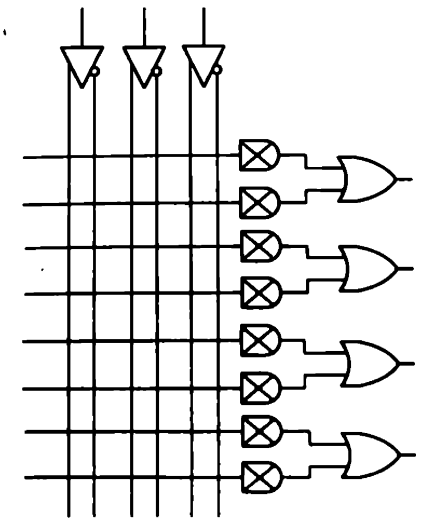


Figure 4.67

Solution : On constate qu'on a ici quatre entrées : A, B, C, D. Il ne sera donc pas possible d'implanter cette fonction dans un seul PAL (puisque chaque PAL n'a que trois entrées). Il faut donc recourir à la partition de la logique. Par ailleurs, en dressant les tables de Karnaugh, on vérifie aussi que les deux fonctions ne sont pas simplifiables outre mesure.

Puisque les PAL imposés n'ont que deux produits par somme, et qu'il nous en faut trois pour *G*, partition et retournement seront aussi nécessaires.

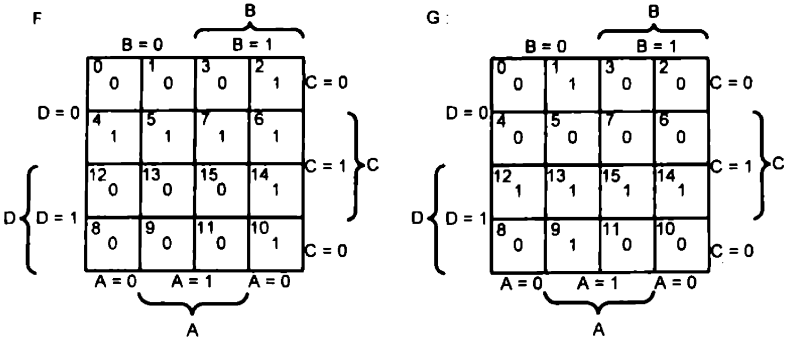


Figure 4.68

La première équation : $F = \overline{A}B + C\overline{D}$ devient, par application de la loi de De Morgan :

$$F = \overline{\overline{\overline{A}B + C\overline{D}}} = \overline{\overline{A}B} + \overline{(\overline{C} + D)}$$

La seconde équation devient : $G = A\overline{B}\overline{C} + A\overline{B}D + CD = A\overline{B}\cdot(\overline{C} + D) + CD$

Ainsi, le premier PAL aura comme entrées : *A*, *B*, $(\overline{C} + D)$, et comme sorties :

$A\overline{B}\cdot(\overline{C} + D)$, qui servira d'entrée au second PAL, et $F = \overline{A}B + C\overline{D}$.

Ainsi, le second PAL aura comme entrées : *C*, *D*, $A\overline{B}\cdot(\overline{C} + D)$, et comme sorties :

$G = A\overline{B}\cdot(\overline{C} + D) + CD$ et $(\overline{C} + D)$, qui servira d'entrée au premier PAL.

De cette discussion, on obtient le circuit final suivant :

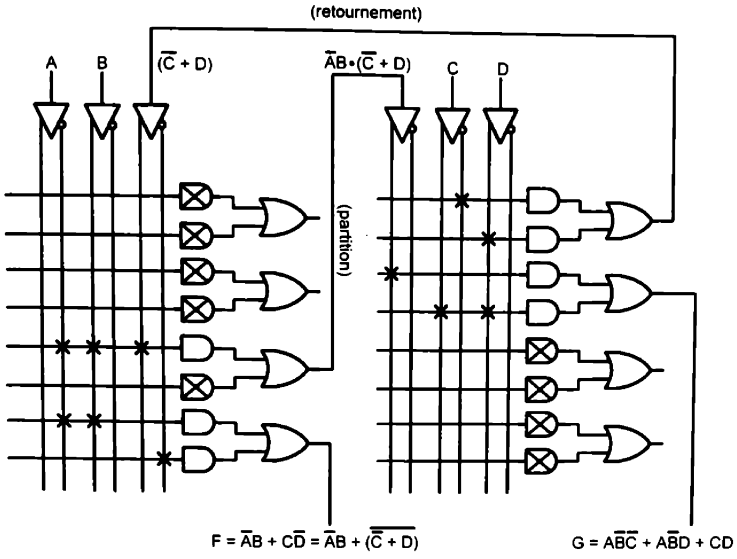


Figure 4.69

Puisque les sorties des PAL ne sont pas toutes employées, il y aurait moyen de rajouter d'autres fonctions pour de l'expansion future, le cas échéant.

4.12 PLA : *PROGRAMMABLE LOGIC ARRAY*

Les PAL sont basés sur la structure des SdP avec le réseau ET programmable (et réseau OU fixe). Les mémoires sont elles aussi basées sur la structure des SdP avec le réseau OU programmable (et le réseau ET fixe). Les PLA, eux, sont également basés sur la structure des SdP, mais cette fois avec les réseaux ET et OU programmables. Puisque les deux réseaux sont programmables, ces circuits sont généralement plus coûteux que les mémoires ou les PAL. Par ailleurs la programmation des deux réseaux rend le travail de la designer plus complexe que l'utilisation des mémoires ou des PAL. Ces circuits sont donc moins communs. La compagnie Philips/Signetics en fabrique et jumelle souvent ces PLA avec d'autres blocs ; des registres par exemple (un peu comme les PAL de type R). La figure 4.70 montre la structure d'un PLA simple avec deux sorties, deux entrées et une possibilité de trois produits par somme.

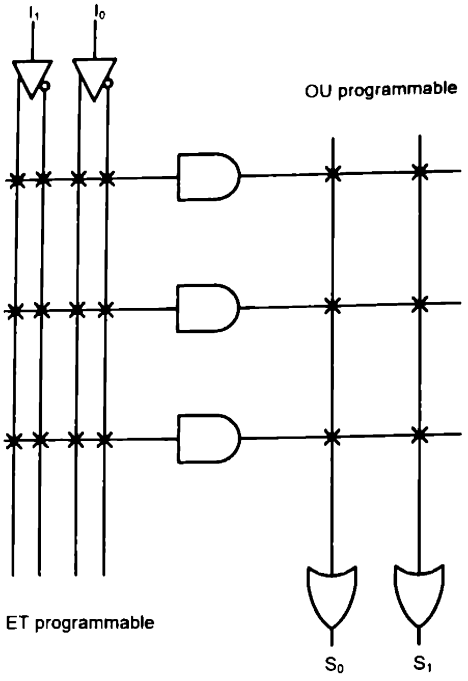


Figure 4.70

4.13 FPGA

Les *Field Programmable Gate Array*, ou réseaux de portes programmables dans le champ (c'est-à-dire par l'usagère), sont une approche plus évoluée que les simples PLD (mémoire, PAL et PLA inclus). En effet, d'un point de vue d'architecture interne, ils contiennent généralement les entités suivantes :

- Des blocs logiques (ou CLB = *Combinational Logic Block*) pouvant à loisir être configurés pour implanter : multiplexeurs, fonctions booléennes quelconques sur quelques bits (exemple : fonction de quatre bits, de huit entrées, de deux sorties) ;
- Des blocs de mémoire (ou BLOCK RAM) ayant différentes configurations de profondeur (nombre de données) et de largeur (taille des mots) ;
- Des lignes de communication (*data line*) appelées *interconnect* pour le transit des données à travers le circuit et entre les CLB. Certains FPGA contiennent même des bus (lignes de données sur plusieurs bits : 8, 16, 32, 64, etc.) ;
- Des circuits d'I/O (*input/output* ou entrées/sortie) pour l'interface avec le monde extérieur (c'est-à-dire avec l'extérieur du circuit intégré). Certaines entrées sont spécialisées, par exemple pour les circuits d'horloge (chapitre 5).

- La figure 4.71 illustre l'intérieur partiel d'un FPGA, d'un point de vue d'architecture interne.

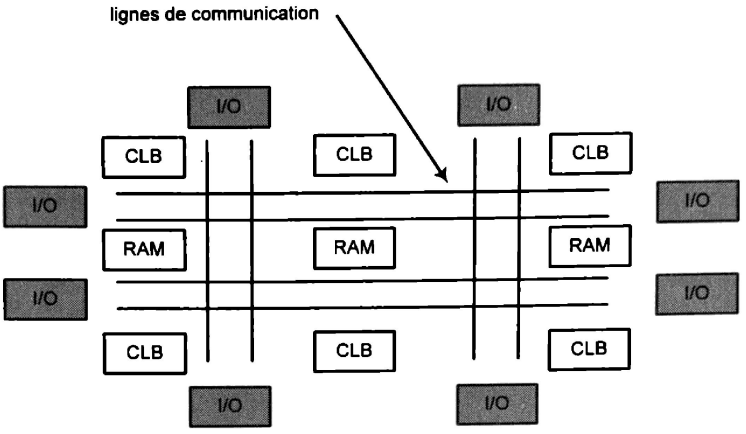


Figure 4.71

Les FPGA sont des circuits VLSI disponibles de plusieurs manufacturiers différents, par exemple Xilinx, Texas Instruments et d'autres. Une pléthore de modèles est aussi disponible dans une grande variété de boîtiers (DIP, PLCC, PWFP, etc.). Les premiers FPGA furent introduits dans les années quatre-vingt, par exemple le Xilinx XC2064 contenait 1200 portes (64 CLB, 58 I/O, 122 registres d'un bit). Aujourd'hui, la technologie VLSI permet d'incorporer plus d'un million de portes dans un circuit intégré, ce qui permet d'y incorporer des fonctions très complexes, même des processeurs complets (au moyen de librairies spécialisées) ; et c'est sûrement là leur avantage majeur : un design complexe peut y loger à l'aise.

On peut voir en effet un FPGA comme une page blanche sur laquelle on dispose tour à tour tous les blocs logiques dont on a besoin : processeurs, mémoires, circuits d'interface spécialisés (port série, etc.). Par la suite, on connecte ces blocs entre eux, puis l'implantation dans le circuit est réalisée. Les avantages sont multiples :

- Tout est inclus dans un seul circuit intégré, ce qui permet d'économiser de l'espace sur le circuit imprimé ;
- On réduit le nombre de connexions externes sur le circuit imprimé (les connexions sont internes). Il est reconnu que les connexions externes sont une des causes majeures des pannes électroniques ;
- Dans une certaine mesure, on réduit aussi la consommation électrique (puisqu'il n'y a qu'un seul circuit intégré).

Du côté des désavantages des FPGA :

- On peut mentionner leur coût (des centaines de dollars pour certains modèles), alors qu'une approche de design plus conventionnelle avec microcontrôleur associé à quelques portes logiques peut revenir à quelques dollars (pour une même application) ;

- La difficulté de faire circuler les signaux à l'intérieur du circuit (c'est ce qu'on appelle le routage des signaux), particulièrement lorsqu'on se rapproche de la capacité maximale du circuit (en fonction du nombre de portes logiques). Il existe bien sûr des logiciels spécialisés pour le routage, mais il faut parfois aussi mettre la main à la pâte.

4.14 LOGICIEL DE PROGRAMMATION

Alors que la réalisation de circuits avec de la logique discrète élémentaire (portes de base, circuits MSI tels des multiplexeurs ou des démultiplexeurs) peut se faire sur *breadboard* avec des fils (voir la figure 1.32, page 33), la configuration des circuits spécialisés tels les PLD implique le recours à des outils informatiques plus ou moins spécialisés selon les circuits à configurer (on dit aussi *circuits à programmer*).

Pour les circuits de mémoire (ROM, EPROM, etc.), la configuration est directe. À la limite, un simple logiciel d'édition tel le « Bloc note » dans Windows convient. Il suffit de dresser la liste des contenus pour chaque adresse consécutive. Un programmeur de circuits est aussi nécessaire pour la génération des signaux de programmation (on a précédemment exposé la procédure dans le cas des mémoires 2764). Cet appareil est connecté à un ordinateur, qui lit, au moyen d'un logiciel ad hoc, le fichier des contenus préalablement édité et le charge (*upload*) dans le circuit (figure 4.43).

Pour les PAL, des logiciels spécialisés sont disponibles tels ABEL (de Data I/O Corp), PALASM (gratuit de AMD, Vantis), CUPL (gratuit de Atmel[16]), etc. Ces logiciels contiennent des fonctions d'optimisation de la solution, car, souvent, il n'y a pas assez de mintermes disponibles dans le réseau pour la solution non simplifiée. L'optimisation à recours à des algorithmes spéciaux : algorithmes génétiques, Quine-McCluskey, Presto, Expresso, etc.

Ainsi :

- ABEL, qui est un langage de description matériel, permet de préparer la solution à un problème logique selon trois approches, ce qui est pratique, car on peut alors choisir l'approche qui convient le mieux au problème à résoudre. Les approches possibles sont :

  - Table de vérité ;

  - Équations ;

  - Diagrammes d'état (ils seront étudiés au chapitre 6).

- Pour le logiciel PALASM, il faut fournir la forme SdP qui aura préalablement été déterminée (à la main ou autrement).

- D'autres logiciels de configuration sont disponibles sur Internet, certains étant du domaine public.

16 Logiciel : http://www.atmel.com/products/PLD/Tutoriel : http://www.rexfisher.com/Downloads/CUPL%20Tutorial.htm

Pour les FPGA, il faut vraiment employer un logiciel spécialisé pour leur configuration. Quant à l'implantation physique dans le circuit intégré, on a d'habitude recours aux outils que le manufacturier du FPGA fournit.

Les langages de description de matériel (HDL : *Hardware Description Language*), tels VHDL, Verilog ou ABEL peuvent servir à configurer les PLD (y incluant les FPGA). L'utilisation de tels langages est recommandée car l'implantation survenant plus tard dans les étapes du travail devient indépendante du circuit intégré. Ainsi, il y aura moyen de « recycler » certaines portions de design d'une application à l'autre (selon l'approche de design modulaire discutée au chapitre 1), de créer ses propres librairies de fonctions, etc.

Certains logiciels de configuration (tel le Xilinx WebPack ou Logisim) incluent un mode graphique et de simulation. Le mode graphique est très pratique, car on « voit ce qui se passe ». Certains circuits de ce livre ont d'ailleurs été réalisés (et vérifiés) ainsi. Le mode graphique est utile pour de petites applications (la réalisation d'un exemple dans ce livre par exemple) ou encore pour connecter entre eux des modules logiques complexes. Il n'y a rien de tel en effet que visualiser les « fils » d'un bloc à l'autre.

Quant au mode de simulation, il est très pratique pour la vérification logique du circuit qu'on réalise. Selon les logiciels, l'usagère doit fournir des vecteurs de test, c'est-à-dire des combinaisons attendues d'entrées et de sorties qui seront validées. Si une erreur survient, c'est-à-dire qu'une combinaison d'entrées n'engendre pas les sorties attendues, celle-ci est signalée à la designer. Dans une autre approche de simulation, la designer fournit des entrées, et le logiciel renvoie les sorties correspondantes. La designer a la responsabilité de vérifier que les sorties correspondent aux entrées fournies.

Quoique très pratique, toute simulation a ses limites. En effet, il se peut que le circuit passe les tests de simulation, mais que son fonctionnement ne soit pas adéquat. C'est le cas si les spécifications de départ étaient erronées. Une autre faille typique est de ne pas considérer certaines combinaisons d'entrées qui « ne peuvent jamais se produire ». Quand elles surviennent (malgré tout), c'est le désastre. Donc, prudence. Il est à noter que des livres spécialisés sur le design existent[17] et qu'il est recommandé d'avoir de solides notions de design avant de se lancer dans la conception de circuits critiques.

Finalement, une dernière mise en garde. Il faut parfois se méfier des logiciels spécialisés. En effet, ils ne fournissent pas toujours la solution optimale puisqu'ils adoptent généralement une approche de « force brute ». Dans la section « Problèmes résolus », on présente ainsi un exemple où un peu de bon sens sera très utile ; un logiciel n'aurait sûrement pas pensé à la solution proposée. Dans un sens, cela est rassurant, et justifie, si l'on peut dire, l'écriture et la lecture de ce livre.

[17] Voir section «Bibliographie» à la fin du livre.

PROBLÈMES RÉSOLUS

4.1. Soit la fonction F (C, B, A ; C = MSB) = $m3 + m4 + m6$. On demande d'implanter celle-ci dans un multiplexeur 2 à 1 en employant la variable B pour la sélection.

Solution : Examinons la table de Karnaugh de F :

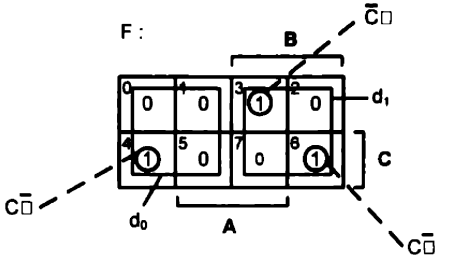


Figure 4.72

Sur celle-ci, on a indiqué les deux blocs sélectionnés par le MUX (la variable B commande la sélection de d_0 et de d_1) de même que pour ceux-ci les mintermes exprimés en fonction des variables restantes (C,A). On a ainsi :

$$d_0 = C\bar{A}$$

$$d_1 = \bar{C}A + C\bar{A} = C \oplus A$$

On est maintenant prêt à implanter le circuit (bit de sélection $S0$ = B) :

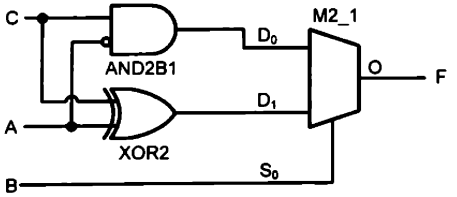


Figure 4.73

4.2. Soit la fonction suivante F(D, C, B, A ; D = MSB) :

$$F = m\,(1,2,3,4,10,12) + X(8,13,14,15)$$

Implanter F à l'aide de portes simples et d'un démultiplexeur 3 à 8 avec bit d'activation unique. Employer (D,C, A) comme bits de sélection du démultiplexeur (avec D = bit de sélection du démultiplexeur de poids fort). On veut une solution de type SdP.

Solution : Considérons d'abord une fonction G quelconque de quatre variables, G(D, C, B, A ; D = MSB), cette fonction peut s'exprimer comme ceci :

$$G = k_0 m_0 + k_1 m_1 + \ldots + k_j m_j + \ldots + k_{15} m_{15}$$

où les k_j sont à 1 ou 0 selon les mintermes m_i présents dans la table de vérité de la fonction, et $j = 0, 1, ..., 15$. Dans l'équation précédente, les mintermes sont définis avec les quatre variables, ainsi $m_0 = \overline{D}\,\overline{C}\,\overline{B}\,\overline{A}$. Maintenant, si on emploie trois variables (*D*, *C*, *A*) dans le problème pour subdiviser la table de Karnaugh en blocs, l'équation précédente s'écrit alors :

$$G = k_0\overline{D}\,\overline{C}\,\overline{B}\,\overline{A} + k_1\overline{D}\,\overline{C}\,\overline{B}\,A + k_2\overline{D}\,\overline{C}\,B\,\overline{A} + ... + k_{15}DCBA = \underbrace{\overline{D}\,\overline{C}\,\overline{A}}_{m_{3,0}}\underbrace{(k_0\overline{B} + k_2 B)}_{Bloc_0} + ...$$

$$G = m_{3,0}\text{bloc}_0 + m_{3,1}\text{bloc}_1 + ... + m_{3,7}\text{bloc}_7$$

Ici, la notation utilisée ($m_{3,i}$) fait référence aux mintermes des trois variables *D*, *C*, *A* employées comme bits de sélection (avec $i = 0, 1, ... 7$). Une fois la table de Karnaugh subdivisée en blocs, il suffira de trouver les valeurs de chaque bloc (les bloc_i dans l'équation précédente). On se rappelle qu'un démultiplexeur 8 à 1 possède trois bits de sélection et qu'il fournit en sortie les mintermes $m_{3,i}$ correspondant aux trois bits de sélection.

Considérons maintenant la table de Karnaugh de *F* ; on a :

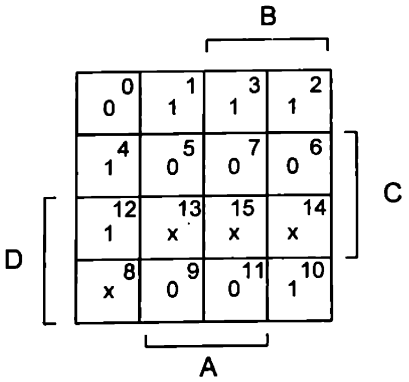
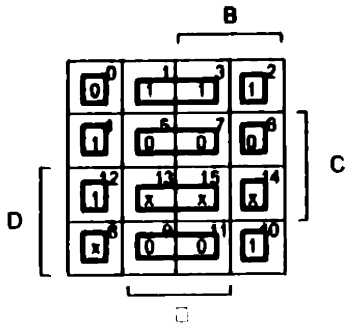


Figure 4.74

En superposant la table de Karnaugh précédente à la subdivision en blocs réalisée par le démultiplexeur (avec *D*, *C*, *A* comme bits de sélection), on obtient (le tableau fournit la distribution correspondante des bloc_i selon l'équation précédente) :



$bloc_0$	$bloc_1$	$bloc_0$
$bloc_2$	$bloc_3$	$bloc_2$
$bloc_6$	$bloc_7$	$bloc_6$
$bloc_4$	$bloc_5$	$bloc_4$

Figure 4.75

Cela nous permet immédiatement d'obtenir la valeur des $bloc_i$:

$bloc_0 = B$

$bloc_1 = 1$

$bloc_2 = \overline{B}$

$bloc_3 = 0$

$bloc_4 = 1$ (ou B)

$bloc_5 = 0$

$bloc_6 = 1$ (ou $\overline{B}$)

$bloc_7 = 0$ (ou 1)

Et donc :

$$F = m_{3,0} \cdot B + m_{3,1} \cdot 1 + m_{3,2} \cdot \overline{B} + m_{3,3} \cdot 0 + m_{3,4} \cdot 1 + m_{3,5} \cdot 0 + m_{3,6} \cdot 1 + m_{3,7} \cdot 0$$

$$F = m_{3,0} \cdot B + m_{3,1} \cdot 1 + m_{3,2} \cdot \overline{B} + m_{3,4} \cdot 1 + m_{3,6} \cdot 1$$

$$F = m_{3,0} \cdot B + m_{3,1} + m_{3,2} \cdot \overline{B} + m_{3,4} + m_{3,6}$$

Pour l'implantation, il faudra donc multiplier (produit booléen) les $m_{3,i}$ avec la valeur de chaque $bloc_i$ correspondant, tout en activant le démultiplexeur selon la dernière expression. Cela donne finalement le circuit suivant :

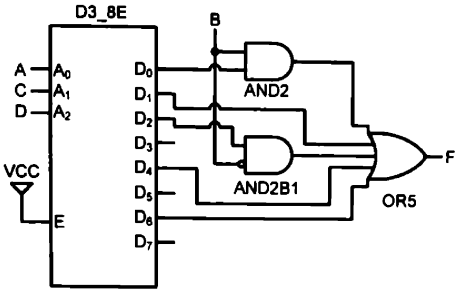


Figure 4.76

Pour la résolution de ce problème, une solution très détaillée a été employée. Il est évidemment possible d'aller à l'essentiel en omettant la « longue preuve algébrique ».

Note : Il est intéressant de voir comment on peut, avec trois bits de sélection, diviser une table de Karnaugh de 16 cases. En fait, il y aura les quatre configurations suivantes[18] : (*D*, *C*, *B*), (*D*, *C*, *A*), (*D*, *A*, *B*), (*C*, *B*, *A*). Évidemment, les subdivisions de la table de Karnaugh ont les mêmes formes que ce soit avec (*D*, *C*, *B*), (*B*, *C*, *D*) ou (*C*, *B*, *D*), etc. Seule la distribution des bloci changera ;

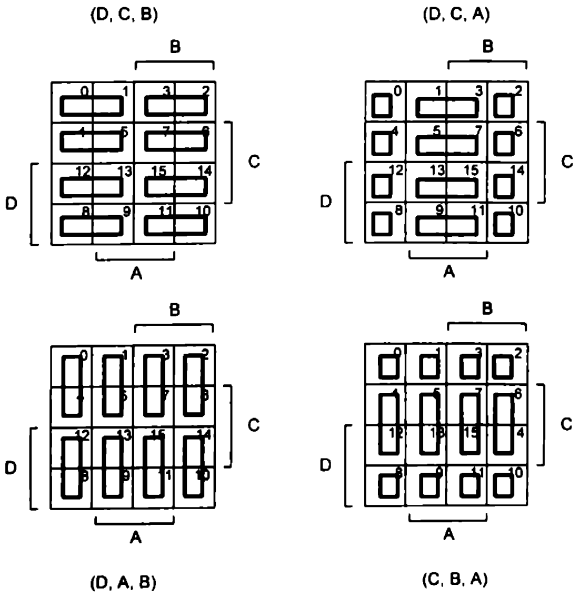


Figure 4.77

4.3. Implanter dans un PAL de votre choix la fonction qui indique si deux mots (*A* et *B*) de trois bits (B_2 B_1 B_0 et A_2 A_1 A_0) sont différents ; donc une fonction qui teste l'inégalité suivante :

---

[18] $C_4^3 = \frac{4!}{3!\,1!} = \frac{4 \cdot 3!}{3!\,1} =$ 4 combinaisons possibles de trois variables.

$$(B_2\ B_1 B_0) \neq (A_2\ A_1\ A_0)$$

Solution : Si on veut dresser la table de vérité exhaustive pour ce problème, celle-ci aura $2^{3+3} = 2^6 = 64$ lignes puisque nous aurons six entrées dans la table : $B_2\ B_1\ B_0\ A_2\ A_1\ A_0$. Cela serait pour le moins fastidieux. On peut grandement faciliter le travail en analysant les cas où il y a identité, c'est-à-dire lorsque $A = B$. Un mot de trois bits signifie que celui-ci pourra avoir $2^3 = 8$ possibilités (de 000 à 111). Ainsi, il y aura 8 cas sur les 64 cas possibles où $(A = B)$. Conséquemment, il y aura aussi $64 - 8 = 56$ cas où $(A \neq B)$. Il est bien sûr plus facile de synthétiser 8 termes plutôt que 56 termes. La solution consiste donc à implanter dans un PAL la fonction $(A = B)$ et à en inverser la sortie pour obtenir la fonction $(A \neq B)$ comme suit :

$$S_{(A \neq B)} = \overline{S}_{(A=B)}$$

Les huit cas à considérer sont :

minterme correspondant	B_2	B_1	B_0	A_2	A_1	A_0	$S_{(A=B)}$	$S_{(A \neq B)}$
$\overline{B}_2\overline{B}_1\overline{B}_0\overline{A}_2\overline{A}_1\overline{A}_0$	0	0	0	0	0	0	1	0
$\overline{B}_2\overline{B}_1B_0\overline{A}_2\overline{A}_1A_0$	0	0	1	0	0	1	1	0
$\overline{B}_2B_1\overline{B}_0\overline{A}_2A_1\overline{A}_0$	0	1	0	0	1	0	1	0
$\overline{B}_2B_1B_0\overline{A}_2A_1A_0$	0	1	1	0	1	1	1	0
$B_2\overline{B}_1\overline{B}_0A_2\overline{A}_1\overline{A}_0$	1	0	0	1	0	0	1	0
$B_2\overline{B}_1B_0A_2\overline{A}_1A_0$	1	0	1	1	0	1	1	0
$B_2B_1\overline{B}_0A_2A_1\overline{A}_0$	1	1	0	1	1	0	1	0
$B_2B_1B_0A_2A_1A_0$	1	1	1	1	1	1	1	0
Les 56 autres cas	Valeurs selon les cas						0	1

Figure 4.78

Ce qui nous permet de dessiner immédiatement le schéma du PAL requis ; celui-ci aura six entrées ($B_2\ B_1\ B_0\ A_2\ A_1\ A_0$), une sortie ($S_{(A \neq B)}$), et contiendra huit portes ET pour le OU unique avec sortie inversée :

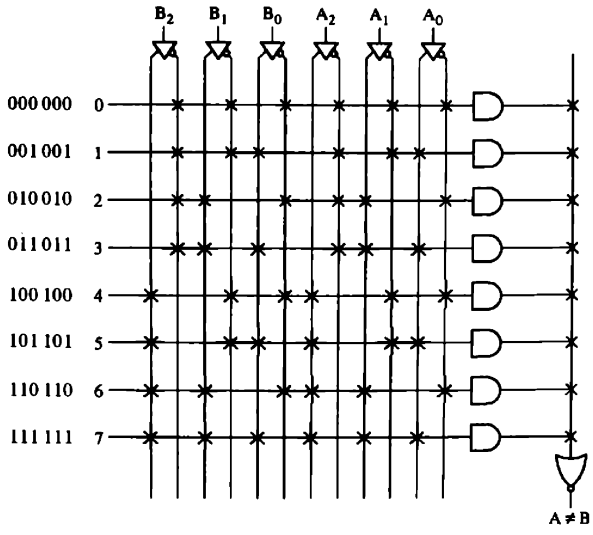


Figure 4.79

Finalement, il ne faut pas bien sûr oublier d'inverser la sortie du PAL pour obtenir ($A \neq B$). Un logiciel spécialisé (par exemple ABEL) n'aurait certainement pas « pensé » à cette « astuce » !

EXERCICES

4.1. On veut implanter la fonction :

$$F(D,C,B,A;D=MSB)=\sum m(1,3,4,11,12,15)+X(6,13)$$

dans un multiplexeur 4 à 1. Employer uniquement la méthode graphique.

a) Donner le schéma du circuit avec D et C comme bits de sélection du multiplexeur.

b) Donner le schéma du circuit avec D et B comme bits de sélection du multiplexeur.

c) Donner le schéma du circuit avec D et A comme bits de sélection du multiplexeur.

d) Donner le schéma du circuit avec C et B comme bits de sélection du multiplexeur.

e) Donner le schéma du circuit avec C et A comme bits de sélection du multiplexeur.

f) Donner le schéma du circuit avec B et A comme bits de sélection du multiplexeur.

Quelle est la solution la plus simple parmi toutes celles testées ?

4.2. Implanter la fonction :

$$F(D,C,B,A\ ;D=MSB)=\sum m(2,4,6,13,14)+X(3,7,9,11,12,15)$$

dans un MUX. Déterminer graphiquement la combinaison la plus avantageuse en matière de coût (nombre de portes doubles élémentaires) parmi les MUX suivants : 8 à 1, 4 à 1, 2 à 1. Tracer le schéma du circuit retenu et calculer son coût en matière de nombre de portes doubles élémentaires.

4.3. Soit la fonction suivante $F(D, C, B, A\ ;D = MSB)$:

$$F=\sum m(3,4,6,7,8,10,15)+X(14)$$

a) Donner la solution PdS (expression booléenne minimale, schéma, coût en matière de nombre de portes doubles élémentaires).

b) Donner la solution en employant la cascade de multiplexeurs 2 à 1 suivante (schéma et coût en matière de nombre de portes doubles élémentaires). Les bits de sélection S sont imposés comme il a été indiqué. On peut rajouter quelques portes simples si nécessaire.

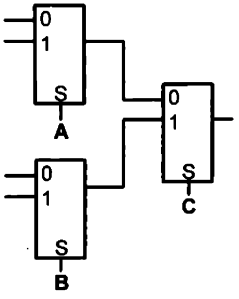


Figure 4.80

4.4 Soit la fonction suivante $F(D, C, B, A\ ;\ D = MSB)$:

$$F=\bar{C}B\bar{A}+D\bar{C}\ \bar{B}+CBA+DC\bar{A}+\ \bar{D}\ \bar{C}\ \bar{B}\ \bar{A}$$

On veut l'implanter dans le type de PAL dessiné plus bas.

a) Donner la table de Karnaugh correspondante

b) Donner la solution en employant ce type de PAL.

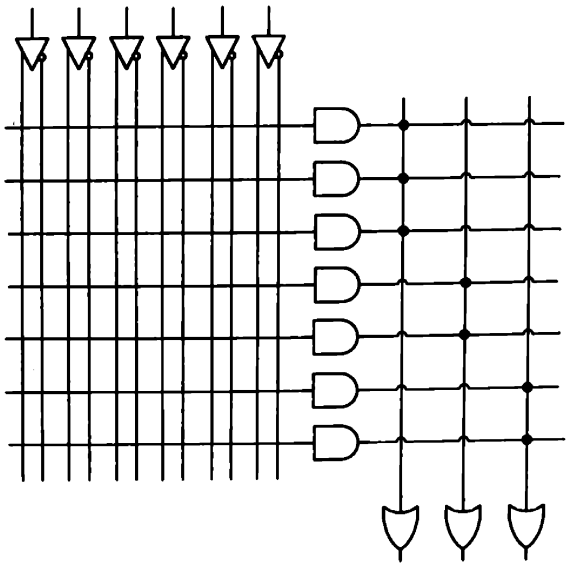


Figure 4.81

4.5 Implanter la fonction :

$$F(D,C,B,A;D=\text{MSB}) = \bar{D}B + DC\bar{A} + \bar{D}\bar{C}B$$

dans un nombre minimal de PAL. L'architecture du type de PAL à employer est la suivante. Tracer le schéma du circuit à l'aide de ce dessin.

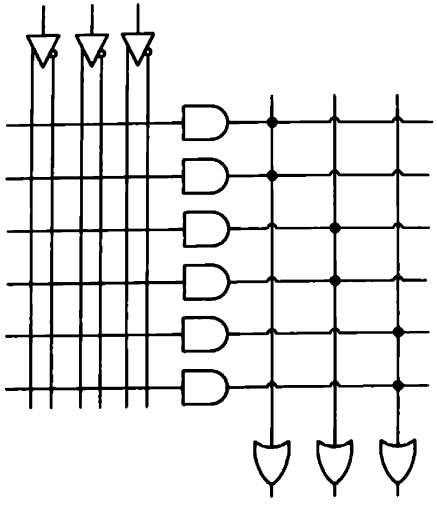


Figure 4.82

4.6. En optimisant le design, implanter la fonction d'inégalité pour deux mots de deux bits (A_1, A_0 et B_1, B_0) dans un (des) PAL dont il faudra établir la structure :

$$F[(A_1, A_0) \neq (B_1, B_0)]$$

F = 1 si les mots sont différents ; et 0, autrement.

4.7. Soit la fonction suivante $F(D, C, B, A$; $D = MSB)$:

$$F = \bar{C}B\bar{A} + D\bar{C}\,\bar{B} + CBA + DC\bar{A} + \bar{D}\,\bar{C}\,\bar{B}\,\bar{A}$$

a) Donner la table de Karnaugh correspondante.

b) Donner la solution en employant le démultiplexeur (= décodeur) 2 à 4 suivant (schéma et coût en matière de nombre de portes doubles élémentaires). Les bits de sélection $S1$ et $S0$ sont imposés comme il a été indiqué. On peut rajouter des portes simples si nécessaire.

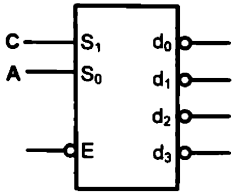


Figure 4.83

4.8. Soit la fonction suivante $F(D, C, B, A$; $D = MSB)$:

$$F = \sum m(0,2,5,11,14) + X(1,4)$$

Implanter cette fonction dans un décodeur 2 à 4 avec bit d'activation (*enable*) simple (c'est-à-dire que l'entrée du bit d'activation est unique et sans bulle d'inversion). Les bits de sélection imposés sont D (+ significatif) et A (− significatif).

a) Donner la table de Karnaugh correspondante.

b) Donner le schéma complet correspondant.

c) Calculer le coût total du circuit en matière de nombre de portes doubles élémentaires (incluant le décodeur).

4.9. Implanter simultanément les expressions suivantes dans un démultiplexeur 74138 en ajoutant uniquement quelques portes simples, $F_i(Z, Y, X$; $Z = MSB)$:

$$F_1 = \overline{XYZ} + XYZ \quad F_2 = \overline{X}\overline{Y}Z + XY\overline{Z}$$
$$F_3 = \overline{X}Y\overline{Z} + X\overline{Y}Z \quad F_4 = X\overline{YZ} + \overline{X}YZ$$

4.10. Implanter la fonction $F(D,C,B,A$; D = $MSB)$ dans une mémoire de 16 bits dont il faudra définir la structure :

a) avec : $$F = \sum m(0,1,3,6,7,8,9) + X(13,14)$$

b) avec : $$F = \prod M(0,1,3,6,7,8,9) + X(13,14)$$

5
Bascules et flip-flops

LE PREMIER MICROPROCESSEUR : LE 4004

Le premier microprocesseur, dont le design fut réalisé par Federico Faggin[1], le 4004 de Intel, fut mis en marché en 1971. Il contenait 2300 transistors, traitait quatre bits de données à la fois, fonctionnait à une vitesse d'horloge de 735 kHz et coûtait un peu moins de 100 $. Par comparaison, le Xeon Phi, un «lointain» successeur du 4004, mis en marché en 2012, contient près 5 milliards de transistors (plus de 2 millions de plus que le 4004), fonctionne à une vitesse d'horloge de 1.238 GHz et coûte autour de 2000 $ (selon les versions). Le super-ordinateur chinois TIANHE-2 emploie ces processeurs et réalise 33.86 peta FLOPS ce qui le place au premier rang mondial (juin 2013). Quel progrès[2] !

Il est intéressant de savoir que le microprocesseur 4004 a fourni la puissance de calcul à la sonde spatiale Pioneer 10, lancée le 2 mars 1972. La sonde Pioneer 10 a été la première sonde spatiale à atteindre la ceinture d'astéroïdes au-delà de l'orbite de la planète Mars (elle pèse 259 kg et avait une puissance embarquée de 155 W). Elle a aussi été la première à renvoyer des images rapprochées de Jupiter, a permis de déterminer la position de ses champs magnétiques et a révélé que cette planète était essentiellement liquide. Jusqu'en mars 1997 Pionner 10 a continué l'exploration des régions lointaines du système solaire en étudiant le vent solaire issu du Soleil et les rayons cosmiques émanant de la Voie lactée. Par la suite, la NASA a réussi à maintenir le contact radio jusqu'au 7 février 2003, journée où les derniers signaux, très faibles, ont été reçus. Par la suite, le contact n'a jamais été rétabli. Pioneer 10 poursuit maintenant sa trajectoire silencieuse vers l'étoile rouge Aldébaran (l'œil de la constellation du Taureau), qu'elle atteindra dans deux millions d'années. Avec sa jumelle Pioneer 11, Pioneer 10 est l'objet, construit par l'Homme, le plus éloigné de la Terre (à environ 100 UA[3]). En plus du 4004, Pioneer 10 contient aussi une plaque commémorative retraçant ses origines. De façon intéressante, le 4004 est aujourd'hui considéré comme l'un des produits les plus révolutionnaires dans l'histoire de l'humanité.

---

1 Voir http://www.intel4004.com/

2 Source : www.tomax7.com/aplus/APlusCD/CPU%20Transistors.doc
http://www.intel.com/intel/intelis/museum/archives/history_docs/index.htm?iid=intelmuseum+home_bfhistdocs&

3 une Unité Astronomique ou UA correspond à la distance moyenne Terre-Soleil, soit environ 150 millions de km

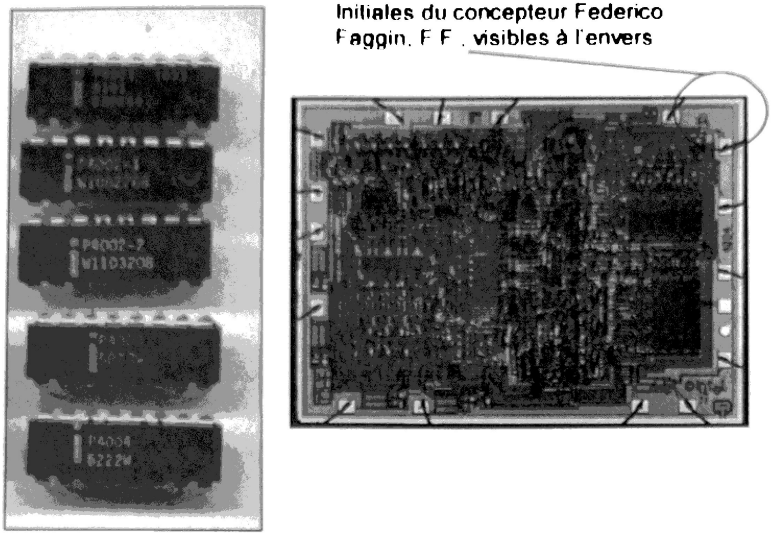
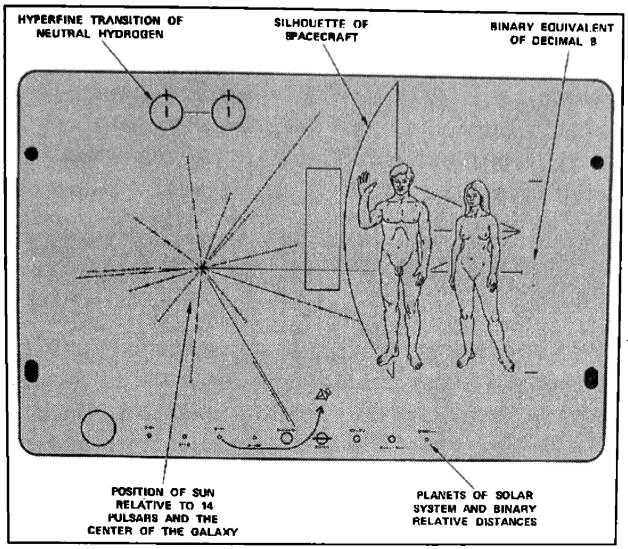


Figure 5.1 Ensemble des cinq circuits originaux du MCS-4 (*Microcomputer system* 4000 de Intel) et photo de la matrice du 4004

A cause de contraintes commerciales liées entre autres à la nécessité d'employer des boîtiers à 16 broches (communs chez Intel à cette époque), le design complet a dû être réparti dans plusieurs circuits :

- 4001 : mémoire ROM (256 x 8 bits) et port d'entrées-sorties de quatre bits.
- 4002-1 et 4002-2 : port d'entrées-sorties de quatre bits et mémoire RAM (320 bits avec quatre bits de sortie). En employant quatre ensembles de deux 4002-1 et de deux 4002-2, un maximum de 5120 bits de RAM était possible pour un MCS-4 (les numéros 4002-1 et 4002-2 font référence à l'adresse des circuits).
- 4003 : registre à décalage de 10 bits (entrée série, sortie parallèle et série) permettant d'accroître le nombre de mémoires ROM, RAM et ports d'entrée-sortie (nous étudierons les registres à décalage au chapitre 6).
- 4004 : microprocesseur à quatre bits pouvant s'interfacer avec un maximum de 4k-8 bits de ROM (pour le programme) et 5120 bits de RAM. Ce CPU contient 16 registres de quatre bits à usage général et possède un jeu de 46 instructions.



http://nssdc.gsfc.nasa.gov/planetary/pioneer10-11.html

Figure 5.2 Plaque incorporée à la sonde spatiale Pioneer 10, lancée le 2 mars 1972 par la NASA

5.1 INTRODUCTION

Aux chapitres précédents, nous avons étudié des circuits dans lesquels l'information circule de gauche à droite. Ces circuits sont dits « en boucle ouverte », c'est-à-dire sans retour de l'information (ou *feed-back*). Dans ce chapitre, nous allons étudier les circuits « avec retour de l'information », donc avec *feed-back*. Nous allons voir qu'une conséquence intéressante du *feed-back*, le circuit peut alors passer dans différents états ; certains stables, d'autres non. Un contrôle judicieux du type de *feed-back* permet d'éviter les états d'instabilités ouvrant la voie à des applications très importantes. Parmi celles-ci, on retrouve toute la question de la mémorisation, laquelle est cruciale pour les circuits numériques avancés tels les séquenceurs qui seront étudiés aux chapitres 6 et 7. Un circuit de mémoire à un bit est un circuit qui retient la valeur du bit qui lui a été confié. À la différence des circuits de type EPROM étudiés au chapitre 4, les circuits de mémorisation dont on parle ici peuvent être écrits (et bien sûr lus) selon les besoins. On peut donc y inscrire un « 1 », puis un « 0 », etc.

Les circuits que nous étudierons dans ce chapitre sont appelés bascules et *flip-flops* (de l'anglais). Ce sont des circuits à un bit dont la valeur peut changer d'état, donc « basculer » entre 0 et 1. Il y a différents types de bascules et de flip-flops :

- R-S
- J-K
- D (Les plus communes ou faciles à comprendre, voir plus loin.)
- T

Les bascules et flip-flops sont fort employés pour la réalisation des circuits synchrones, c'est-à-dire les circuits qui sont synchronisés par rapport à un signal d'horloge commun et qui ne changent d'état que lorsque l'horloge elle-même change d'état. Dans sa version la plus simple, un signal d'horloge est tout simplement, pour les circuits TTL standards, une onde carrée 0-5 V. Les microprocesseurs sont de bons exemples de circuits synchrones : on parlait plus haut du Intel 4004 dont la fréquence d'horloge était de 735 kHz.

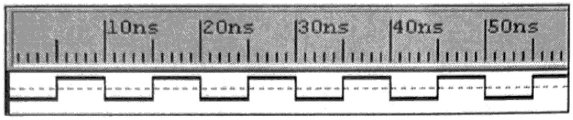


Figure 5.3 Signal d'horloge simple sous la forme d'une onde carrée passant successivement des niveaux logiques 0 à 1, et ainsi de suite. Ce signal particulier est obtenu du simulateur du Xilinx WebPack. Période = 10ns, Fréquence = 100 MHz. La fréquence est l'inverse de la période.

La très grande majorité des circuits numériques (avec mémorisation) sont synchrones ; ils ne le sont pas tous cependant. Il existe ainsi des circuits dits asynchrones qui possèdent aussi une faculté de mémorisation. Contrairement aux circuits synchrones, les circuits asynchrones ne sont pas en phase avec un signal d'horloge. Comme exemple de circuits asynchrones, nous étudierons à la section 5.3 la bascule R-S sans horloge.

Le terme *circuit séquentiel* est un terme générique qui englobe les circuits synchrones et asynchrones, donc qui inclut les circuits qui possèdent une mémorisation de leur état. Ceci se dit par opposition aux circuits sans mémorisation, donc les *circuits combinatoires* étudiés aux chapitres précédents. On se rappelle que les circuits combinatoires réagissent dès que leur(s) entrée(s) change(nt).

Finalement, en revenant aux circuits synchrones, nous distinguerons les *bascules* des *flip-flops*. Nous emploierons le terme « bascule » pour désigner les circuits réagissant au niveau logique du signal d'horloge, donc un signal d'horloge qui est à la valeur « 0 » ou « 1 » logique. Quant au terme « flip-flop », il désignera les bascules à déclenchement sur front d'horloge : front montant (lorsque l'horloge passe du « 0 » au « 1 » logique : ⌐↑) ou front descendant (lorsque l'horloge passe du « 1 » au « 0 » logique : ↓⌐). Ceci sera détaillé plus loin dans le chapitre.

Il est intéressant de noter que l'état de sortie futur (aussi appelé *état suivant*) d'un circuit de mémorisation (bascule, flip-flop) dépend de son *état présent*, donc de son état avant le changement du signal d'horloge ; c'est ce qu'on appelle aussi l'« histoire » du circuit de mémorisation, en quelque sorte. Ces notions *d'état présent* et *d'état suivant* seront expliquées plus loin.

5.2 RETOUR DE L'INFORMATION ET SES CONSÉQUENCES (*FEED-BACK*)

Dans le contexte des circuits électroniques, le *feed-back* est considéré comme le retour de la sortie vers l'entrée, ce qui permet d'influencer le comportement du système. On

distingue aussi le *feed-back* positif ou négatif. Dans le cas du *feed-back* positif, la sortie s'additionne aux entrées, alors que, dans le cas du *feed-back* négatif, elle se soustrait aux entrées. On conçoit que, dans le cas du *feed-back* positif, le système sera poussé à ses limites, et qu'il y aura fort probablement saturation des signaux de sortie. Cela est indiqué à la figure 5.4 dans le cas d'un amplificateur de gain *A* avec *feed-back* positif. Le gain signifie que, si le signal d'entrée « e » possède une amplitude de 0.1 volt et que $A = 10$, alors la sortie *s* aura une amplitude de 0.1 x 10 = 1 Volt. Sur la figure, on voit que l'amplificateur essaie de suivre et d'amplifier le signal d'entrée qui est toujours croissant. Rapidement, il y a saturation puisque la sortie du circuit, est limitée en tension d'alimentation. Par ailleurs, des oscillations peuvent apparaître. Celles-ci sont fonction du délai de propagation du signal dans le circuit.

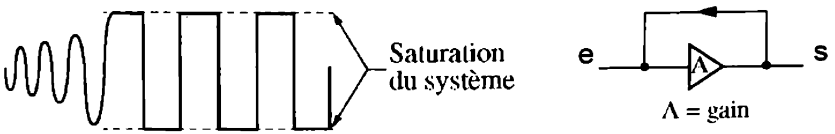


Figure 5.4

Dans le cas des portes logiques, le *feed-back* aussi est possible. Prenons par exemple la cas d'un inverseur ayant un certain délai de propagation Δt. Relions ensuite la sortie à l'entrée comme suit :

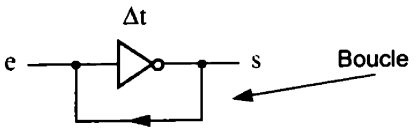


Figure 5.5

La propagation des signaux est alors la suivante. Celle-ci est illustrée à l'aide d'un chronogramme (*timing diagram* en anglais) qui montre la forme du signal en fonction du temps.

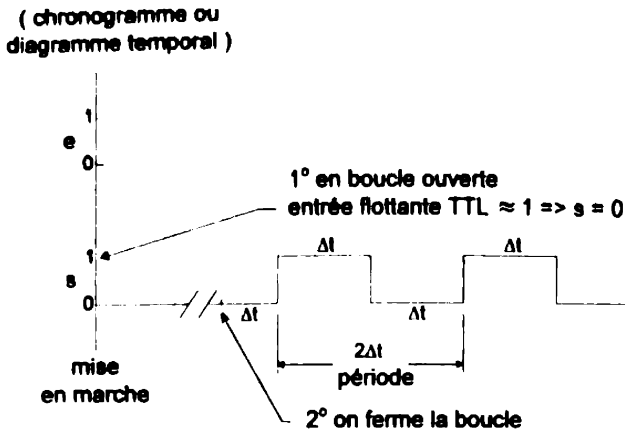


Figure 5.6

Pour ce cas simple, on pourra observer deux possibilités, soit des oscillations 0, 1, 0, 1 de période approximative 2 ΔT tel qu'il a été illustré. Il est aussi possible que le circuit se bloque dans un état mitoyen appelé état « métastable » situé entre les valeurs 0 et 1.

Ce qui est intéressant ici, c'est de constater que la valeur future de la sortie « s » de notre inverseur dépend de la valeur des entrées présentes, donc de son *état présent*. Ainsi, si *s* passe à 0 et que e passe donc aussi à 0, alors la sortie future aussi appelée *état suivant*, sera $s = 1$. Il y a donc un effet de *mémoire* qui apparaît grâce au *feed-back*. Ceci peut d'ailleurs être généralisé : la sortie future d'un circuit séquentiel dépend de la valeur des entrées présentes ET de son état présent (donc des entrées passées), ce qui signifie que ce circuit se souvient des entrées précédentes.

Évidemment, le circuit précédent n'est pas très utile (sauf comme oscillateur). D'un point de vue pratique, nous voudrions avoir un circuit capable de mémoriser un bit selon ses deux valeurs possibles : 0 et 1. Comment faire ? À la section suivante, nous étudierons une première configuration pratique.

5.3 BASCULES R-S

Le principe du *feed-back* exposé à la section précédente est ici repris dans la première bascule que nous allons étudier : la bascule R-S, qui est réalisée non pas à l'aide d'un inverseur, mais plutôt à l'aide de deux portes NON-OU. Le sigle R-S vient de l'anglais, avec R qui signifie *Reset* ou mise à 0, et *S* qui signifie *Set* ou mise à 1. Le schéma est le suivant :

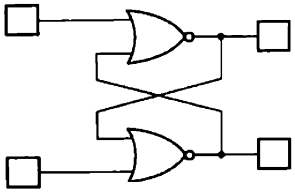


Figure 5.7 Schéma d'une bascule R-S

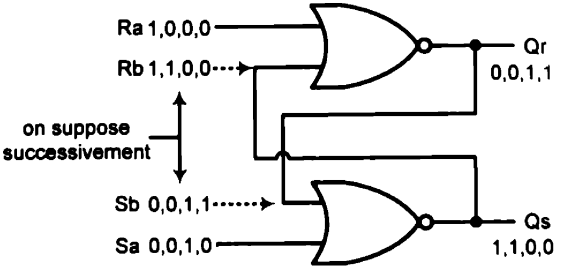
Étudions son comportement. Pour cela, on identifie d'abord les différentes entrées et sorties du circuit telles qu'elles ont été indiquées sur le schéma suivant, où sont aussi indiqués les quatre scénarios consécutifs suivants. *Ra*, *Rb*, *Sa*, *Sb* sont les entrées, et *Qr* et *Qs* sont les sorties.

1. On suppose initialement que les entrées du circuit sont *Ra* = 1 et *Sa* = 0, et que les sorties sont *Qr* = 0 et *Qs* = 1. Ainsi, le « 1 » est donc aussi présent en *Rb*, et le « 0 » en *Sb*. Qr maintient sa valeur à 0 selon l'idiome « $\overline{1+1}=0$ » vu au chapitre 2. Puisque *Sa* = 0, Qs maintient sa valeur de 1 selon l'idiome « $\overline{0+0}=1$ », vu au chapitre 2. Les valeurs de sorties « s'auto-nourissent » avec le *feed-back* et demeurent bien à *Qr* = 0 et à *Qs* = 1. Le circuit est dit dans un état stable.

2. Supposons maintenant que *Ra* passe à « 0 », toujours avec *Sa* = 0. Rien ne change ! Pour la porte d'en bas, l'idiome « $\overline{0+0}=1$ » tient toujours. Pour la porte d'en haut, l'idiome « $\overline{1+0}=0$ » maintient aussi la valeur de Qr à 0.

3. Faisons maintenant passer *Sa* à 1 avec *Ra* à 0. Pour la porte d'en bas, on à cette fois « $\overline{1+X}=0$ » (puisqu'on ne sait pas encore comment réagira la porte d'en haut). Donc *Qs* = 0. Avec *Qs* = 0, on a maintenant pour la porte d'en haut « $\overline{0+0}=1$ » et *Qr* passe donc à 1.

4. Finalement, on remet *Sa* à 0 avec *Ra* à 0. Rien ne change pour la porte d'en bas avec « $\overline{1+0}=0$ » et *Qs* = 0. Rien ne change non plus pour la porte d'en haut avec « $\overline{0+0}=1$ » et *Qr* = 1.



Rappel : la sortie d'une porte NON-OU est à 0 si une de ses entrées est à 1

Figure 5.8

Résumons les scénarios que nous venons de voir :

- Scénario 1 : $Ra = 1$, $Sa = 0 \Rightarrow$ $Qr = 0$ et $Qs = 1$; Mise à 0
- Scénario 2 : $Ra = 0$, $Sa = 0 \Rightarrow$ $Qr = 0$ et $Qs = 1$; Mémorisation (de 0)
- Scénario 3 : $Ra = 0$, $Sa = 1 \Rightarrow$ $Qr = 1$ et $Qs = 0$; Mise à 1
- Scénario 4 : $Ra = 0$, $Sa = 0 \Rightarrow$ $Qr = 1$ et $Qs = 0$; Mémorisation (de 1)

Au tableau précédent, on a aussi indiqué à quoi correspondait chacun des scénarios. Ainsi, quand on est passé du scénario 1 au scénario 2 (ou du scénario 3 au scénario 4), on constate que les sorties n'ont pas changé, ce qui indique bien que le circuit a bien mémorisé le bit. Par contre, pour le scénario 1, on a une mise à 0 ; et pour le scénario 3, on a une mise à 1. Sur ce tableau, on note aussi que Qr est toujours l'inverse de Qs, donc on a en fait $Qs = \overline{Qr}$, ce qui est pratique puisqu'il est parfois utile d'avoir accès à un signal inversé.

La liste précédente indique le comportement du circuit pour trois combinaisons des entrées : (0,0), (0,1), (1,0). Puisqu'il y a deux entrées, il y a forcément une quatrième ($4 = 2^2$) combinaison (1,1). Que se passe-t-il si $Ra = Sa = 1$? Voyons cela. Si $R = S = 1$, alors suivant les raisonnements précédents, on conclut directement que $Qr = Qs = 0$ suivant l'idiome $\overline{1 + X} = 0$:

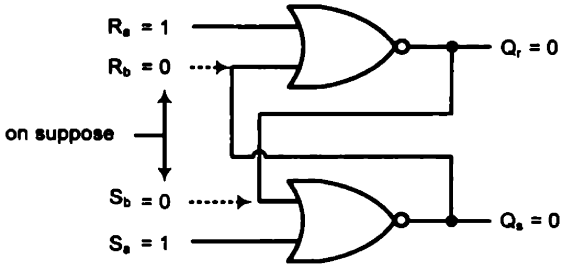


Figure 5.9

Le circuit est stable. Par contre, on a cette fois que $Qr = Qs$ et non l'inverse, comme précédemment. C'est un premier problème. Il y a aussi un second problème.

Considérons en effet la transition (Ra, Sa) = (1, 1) au scénario (Ra, Sa) = (0, 0). En fait, sur une échelle d'une fraction de seconde, on va passer d'abord à (0,1) ou (1,0) avant d'arriver à 00 (car forcément un signal est plus rapide que l'autre). Donc deux situations sont possibles :

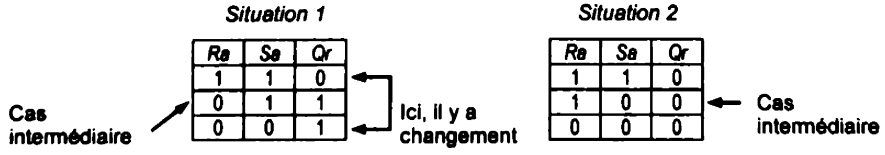


Figure 5.10

Le problème est le suivant : On se rappelle que le scénario (0, 0) correspond à la mémorisation du bit ; il s'agit donc d'un cas pour lequel aucun changement ne doit survenir. Ainsi, la situation 1 est un désastre puisque le bit qui aurait dû être mémorisé passe de 0 à 1. Puisqu'on ne pourra jamais s'assurer de laquelle entrée R ou S changera la première, cette situation est donc critique.

Pour ces deux raisons, on décide simplement d'interdire le scénario (Ra, Sa) = (1, 1).

TABLE DE TRANSITION DE LA BASCULE R-S

La table finale du circuit est la suivante. À noter, pour les circuits séquentiels, on ne parle plus de table de vérité, mais plutôt de « table de transition » :

R	S	Q	$\bar{Q}$	Description
0	0	Q_0	$\bar{Q}_0$	Valeur de l'état précédent, pas de changement
0	1	1	0	Mise à 1
1	0	0	1	Mise à 0
1	1	Interdit		Interdit*

* D'autant plus que les valeurs finales de Q et de $\bar{Q}$ dépendent de la topologie du circuit « R-S », selon qu'on la réalise avec des portes NON-ET ou des portes NON-OU (voir la section *Problèmes résolus*, page 220).

Figure 5.11

Pour le circuit dont les entrées et les sorties sont renommées comme suit :

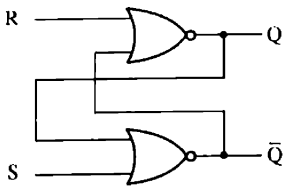


Figure 5.12

Dans la table précédente, Q_0 signifie la valeur au temps $t = 0$, donc la valeur précédente.

Résumons les avantages d'interdire le scénario $RS = 11$:

- Avec (R, S) = (1,1), on viole la règle stipulant que si $S = 1$ on a une mise à 1 donc $Q = 1$ (car on a alors $Q = 0$) ;
- Aussi, en empêchant (R, S) = (1,1), on a pour tous les autres cas que la sortie de la porte NON-OU d'en bas est toujours l'inverse de Q (c'est $\bar{Q}$). Une sortie est ainsi l'inverse de l'autre.

CIRCUIT ANTIREBOND AVEC BASCULE R-S

Le circuit antirebond est un exemple d'utilisation très commun pour les bascules R-S. Le problème est le suivant. Lorsqu'un contact mécanique bascule (par exemple un interrupteur manuel), sur une courte échelle de temps, il y a rebondissement de celui-ci. Ainsi, si on connecte cet interrupteur à un compteur incrémental, on enregistrera

plusieurs transitions avant la stabilisation. La figure 5.13 illustre le basculement de la position *a* à la position *b*, on a dessiné deux rebondissements avant la stabilisation au niveau logique 0. On peut se convaincre de l'existence de ce phénomène en laissant tomber l'extrémité d'un crayon placé à la verticale sur une table : plusieurs rebondissements sont observés avant d'atteindre la position de repos à l'horizontale.

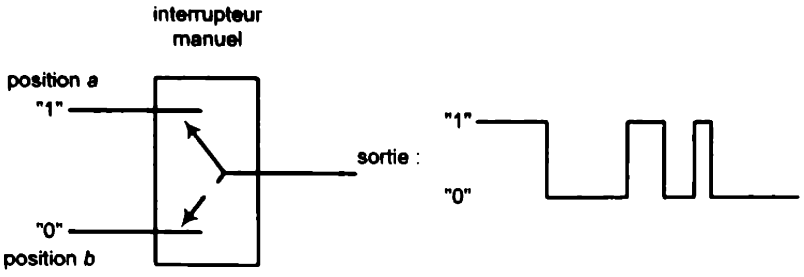


Figure 5.13

Le circuit antirebond est montré à la figure 5.14. Afin de comprendre ce circuit, il est d'abord nécessaire de rappeler le fonctionnement d'une résistance *pull-down* (dont le fonctionnement s'apparente à celui de la résistance *pull-up* discutée au chapitre 4). Une résistance *pull-down* sert à forcer la tension du point de contact à la masse, ce qui amène ce point à la valeur logique 0. Puisque la résistance est grande (et donc le courant faible), il est facile de faire basculer la valeur du point de contact au niveau 1 logique en lui appliquant la tension d'alimentation (et ce, sans risque de court-circuit).

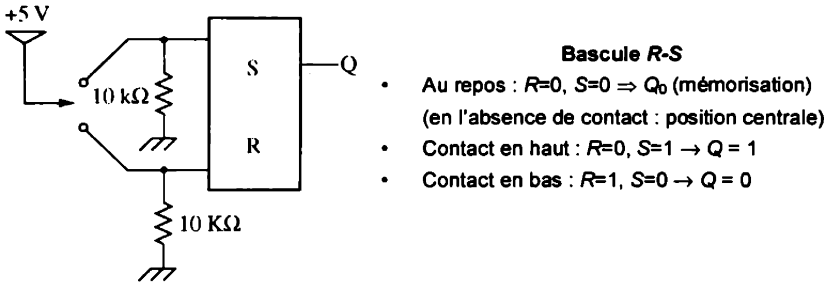
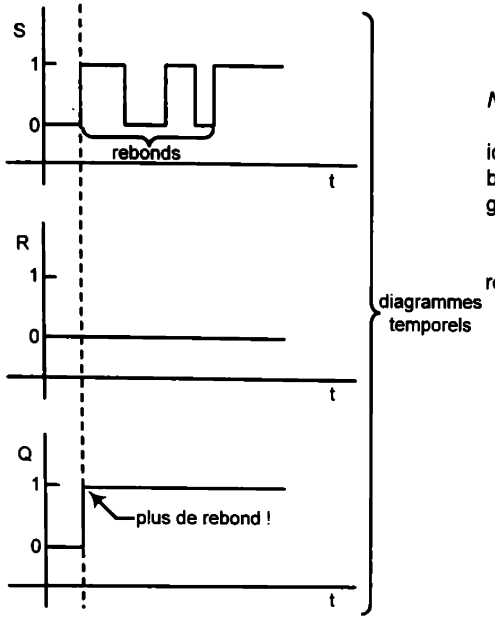


Figure 5.14

Ainsi, lorsque le contact bascule de la position centrale au contact supérieur, on observera une succession de contacts tel qu'il a été illustré dans le diagramme temporel suivant. Par contre, la bascule R-S réagira dès le premier rebond. Les autres rebonds « confirmeront » la valeur finale sans modification de celle-ci. L'effet des rebonds a été supprimé. On note que la condition interdite est *de facto* impossible ici puisqu'on ne pourra jamais avoir le scénario $R = S = 1$.

La valeur finale est atteinte dès le premier rebond.



Notes :

- Par souci de clarté, on a négligé ici le délai de propagation dans la bascule RS, il ne change pas l'effet global de toute façon.
- On assume que Q = 0 au départ.
- Il y a réaction dès le premier rebond.

Figure 5.15

Ce circuit antirebond est fort employé pour « amortir » les interrupteurs mécaniques. On recommande d'ailleurs son usage pour la plupart des applications où un interrupteur mécanique est relié à un circuit logique.

5.4 ÉTAT D'UNE BASCULE, ÉTAT D'UN CIRCUIT SÉQUENTIEL

Aux chapitres précédents, nous avons étudié les *circuits combinatoires*. Les sorties de ceux-ci dépendaient directement des entrées présentes, et, au moindre changement de celles-ci, les sorties changeaient. Dans ce chapitre, nous avons commencé à étudier les *circuits séquentiels* qui possèdent une capacité de mémorisation. Ainsi, la sortie d'un circuit séquentiel dépend des entrées présentes et de « l'historique » de celles-ci. Prenons le cas de la bascule R-S. Même en mettant les entrées R et S à 0, la bascule mémorise sa valeur courante. Cette valeur mémorisée correspond donc à l'état présent de la bascule. Il peut être égal à « 1 » ou à « 0 » selon ce que la bascule a mémorisé. Cette notion d'état est très importante pour l'étude des circuits séquentiels.

Définition : L'état d'une bascule correspond à une de ses conditions stables : sortie $Q = 1$ (état à 1) ou sortie $Q = 0$ (état à 0). Cette notion peut être étendue à un circuit séquentiel quelconque. L'état d'un tel circuit correspond alors aux valeurs mémorisées par ce circuit à un moment donné.

Exemple : Si un circuit contient trois bascules, l'état d'un tel circuit correspond aux valeurs mémorisées par ces bascules au moment où l'on observe le circuit, par exemple : « 1, 0, 1 » etc.

5.5 BASCULE R-S AMÉLIORÉE : LE *LATCH* D

TENTATIVE D'AMÉLIORATION POUR ÉLIMINER L'ÉTAT INTERDIT

La bascule R-S est très sensible aux changements d'entrées ; cela est utile dans certaines applications, comme pour le circuit antirebond vu précédemment. Par contre, pour d'autres applications, on aimerait qu'elle soit dotée d'une certaine immunité aux fluctuations des entrées. En fait, on aimerait avoir un circuit qui ne réagit aux entrées qu'en des instants précis et qui est insensible à celles-ci aux autres moments. Reprenons le schéma de la bascule R-S, et essayons d'en améliorer le design.

La première amélioration à apporter consiste d'abord à inclure un mécanisme pour s'affranchir automatiquement de la condition interdite (R, S) = (1, 1). À cette fin, on peut penser n'avoir qu'une seule entrée et synthétiser l'entrée R à partir de l'entrée S à l'aide d'un inverseur :

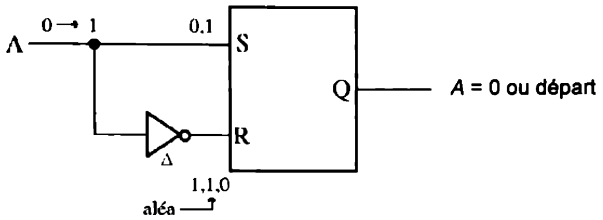


Figure 5.16

En principe, l'inverseur devrait nous empêcher d'avoir la condition interdite $R = S = 1$. Cependant, à cause du délai de propagation Δ de l'inverseur, on l'obtient malgré tout pour un bref instant lorsque la transition $A : 0 \rightarrow 1$ survient. Il se crée alors un aléa indésirable. Cette topologie n'est donc pas très intéressante, d'autant plus qu'il n'y a jamais de mémorisation puisque le scénario $R = S = 0$ n'est jamais présent.

AMÉLIORATION DE LA BASCULE R-S EN LUI AJOUTANT UNE HORLOGE : BASCULE R-S AVEC HORLOGE À NIVEAUX OU LATCH R-S

Une façon de rendre la bascule insensible aux aléas consiste à lui ajouter une horloge. Lorsque l'horloge (*clock* ou Clk ou CK en anglais) est active, donc à la valeur « 1 », la bascule peut changer d'état. Si l'horloge demeure à 0, la bascule reste insensible à tous les changements présents à ses entrées. Cette notion de signal d'horloge est essentielle dans les circuits synchrones. Un signal d'horloge est un signal périodique qui permet de synchroniser les différentes parties d'un circuit dès lors appelé circuit synchrone. Dans sa forme la plus simple, un signal d'horloge est simplement une onde carrée passant successivement de la valeur logique « 0 » à la valeur logique « 1 » à une certaine cadence appelée fréquence de l'horloge (voir la figure 5.3, page 185).

Une façon de rajouter un signal d'horloge à une bascule R-S consiste simplement à lui ajouter deux portes ET à ses entrées tel qu'elles ont été illustrées à la figure 5.17. Ce type de signal d'horloge est appelé signal d'*horloge à niveaux* (*level sensitive clock* en anglais) puisque le circuit ne peut être modifié que si le niveau logique de l'horloge est à « 1 ». Nous verrons plus bas qu'il existe aussi un autre type de signal d'horloge actif lors des

transitions (*horloge à déclenchement sur front* ou *edge-triggered clock* en anglais). Sur la figure, on note aussi le symbole de la « barre » dénotant l'horloge à niveaux et aussi le symbole équivalent pour cette *bascule R-S avec horloge à niveaux* aussi appelée *latch R-S*. À noter que la barre dénotant l'horloge peut être pleine (▬) ou creuse (▭) comme il a été indiqué et comme on le voit généralement dans la plupart des manuels. À droite sur la figure, on montre l'effet de l'horloge (signal *H*) sur le fonctionnement de la bascule. Assumant que $R = 0$, une brève transition apparaît, par hasard, sur *S*. Celle-ci n'a aucun effet sur la sortie *Q*, qui demeure à 0 puisque le signal d'horloge est à 0.

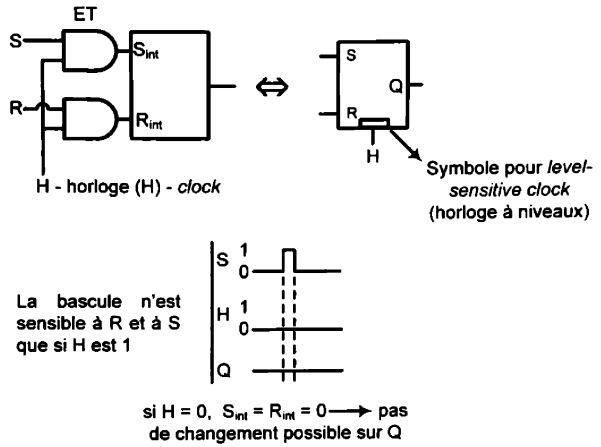


Figure 5.17 Figure illustrant une bascule R-S avec horloge à niveaux.
« int » dénote les signaux internes.

Évidemment, le problème du cas interdit $R = S = 1$ demeure. L'introduction de l'horloge à niveaux n'ayant réglé que la sensibilité « extrême » de la bascule.

La table de transition du latch R-S est la suivante :

Horloge (*H* ou *CK*)	Entrée *R*	Entrée S	Sortie *Q*	Description
0	*X*	*X*	Q_0	Mémorisation
1	0	0	Q_0	Mémorisation
1	1	0	0	Mise à 0
1	0	1	1	Mise à 1
1	1	1	-	Cas interdit

Figure 5.18

AMÉLIORATION DE LA BASCULE R-S : LATCH D

En ajoutant simplement un inverseur et un tampon permettant de combiner les entrées R et S, on obtient ce qu'on appelle un « latch D ». L'ajout du tampon permet d'avoir des parcours de données identiques du côté *R* et du côté *S*, ce qui nous garantit l'absence de l'état interdit $R = S = 1$ (ce qui n'était pas le cas lors de notre première tentative d'amélioration de la bascule R-S.

Un *latch* est un type de bascule possédant une horloge à niveaux. Le circuit de la figure 5.19 est appelé latch *D*, car c'est le nom qui est donné à son entrée unique (*D* pour données, ou *data* en anglais). Le fonctionnement est aussi indiqué sur la figure.

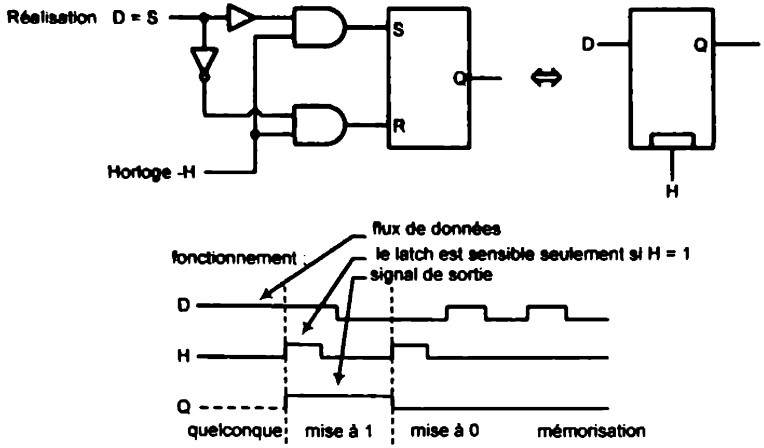


Figure 5.19

De cette figure, on voit bien :

1. que si le signal d'horloge est à 1, les portes ET ont un effet transparent, et alors la sortie suit l'entrée (sortie = entrée) ;
2. que si le signal d'horloge est à 0, la bascule mémorise son dernier état.

La table de transition du latch *D* est d'ailleurs la suivante :

Horloge (*H* ou *CK*)	Entrée *D*	Sortie *Q*	Description
0	*X*	Q_0	Mémorisation
1	0	0	Mise à 0
1	1	1	Mise à 1

Figure 5.20

Les *latches* sont très employés comme unité de mémorisation, particulièrement pour la mémorisation de signaux qui varient peu rapidement (à cause de l'horloge à niveaux). Ils sont souvent employés par groupe de quatre ou de huit bits. Notons aussi que l'IEEE[4] a standardisé les symboles des bascules. La figure 5.21 montre d'ailleurs les schémas explicatifs pour un latch D double à quatre bits (2 × 2). Ce dessin correspond à un circuit 74HC75 de la famille TTL discutée précédemment. Le 74HC75 contient deux latches indépendants de deux bits ayant chacun leur horloge aussi appelée LE pour *Latch Enable* (ou activation du latch). Si LEnn est à 1, les sorties *nQ* suivent les entrées *nD* ; c'est le mode « transparent » ou suiveur du latch (n fait référence au latch d'intérêt 12 ou 34). Si LEnn est à 0, les sorties *nQ* sont fixes et mémorisent la dernière valeur entrée

[4] Institute of Electrical and Electronics Engineers (www.ieee.org).

(donc celles présentes à la transition 1→ 0 de LEnn) ; c'est le mode de « mémorisation » du latch.

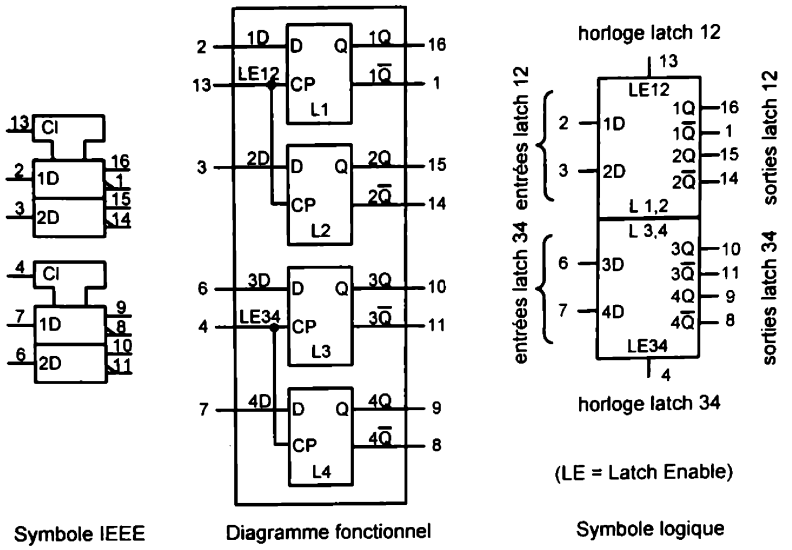


Figure 5.21 Circuit TTL 74HC75. Un 74HC75 possède un boîtier à 16 broches, soit sept broches pour le latch 12, 7 broches pour le latch 34 et deux broches d'alimentation (VCC = broche 5, masse = broche 12)[5].

5.6 BASCULES À DÉCLENCHEMENT SUR FRONT « MAÎTRE-ESCLAVE » : FLIP-FLOP D

Les latches *D* sont des bascules bien améliorées par rapport aux bascules R-S puisque l'état interdit a été supprimé. De plus, ils sont moins sensibles aux fluctuations puisqu'on leur a adjoint une horloge. Néanmoins, pour toute la durée pendant laquelle l'horloge est au niveau logique 1, le latch peut changer de niveau. Ainsi, si plusieurs latches sont connectés en cascade et qu'ils partagent une horloge commune, lorsque celle-ci passe au 1 logique, les données se propagent directement d'un latch à l'autre tant que le signal d'horloge reste actif. Selon le délai de propagation des latches, les données progresseront plus ou moins loin parmi les latches connectés ; c'est ce qu'on appelle une course. Pour éviter ce type de comportement imprévisible (exemple : les délais de propagation dans un circuit varient avec la température), il existe un besoin pour un type de bascule qui réagit non pas aux niveaux logiques d'une horloge, mais plutôt à ses transitions, qu'on appelle aussi les « coups d'horloge » ou encore les fronts montants (transitions : ⌜) ou descendants (transitions : ⌞).

Une façon de réaliser cela consiste à employer une configuration dite « maître-esclave » (*master-slave* en anglais). Dans une telle configuration, il y a toujours deux bascules

[5] Adapté de http://www.semiconductors.philips.com/acrobat_download/datasheets/74HC75_3.pdf

connectées en cascade avec un signal d'horloge commun, mais inversé d'une bascule à l'autre.

Le schéma suivant illustre une configuration « maître-esclave » réalisée au moyen de deux latches D :

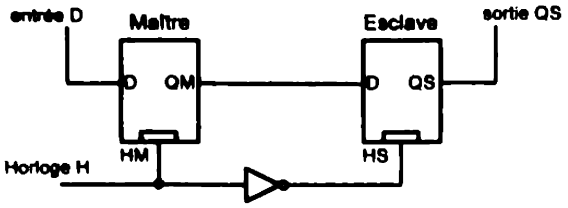


Figure 5.22

Étudions le fonctionnement de ce circuit maître-esclave lorsque soumis aux transitions dessinées à la figure 5.23 :

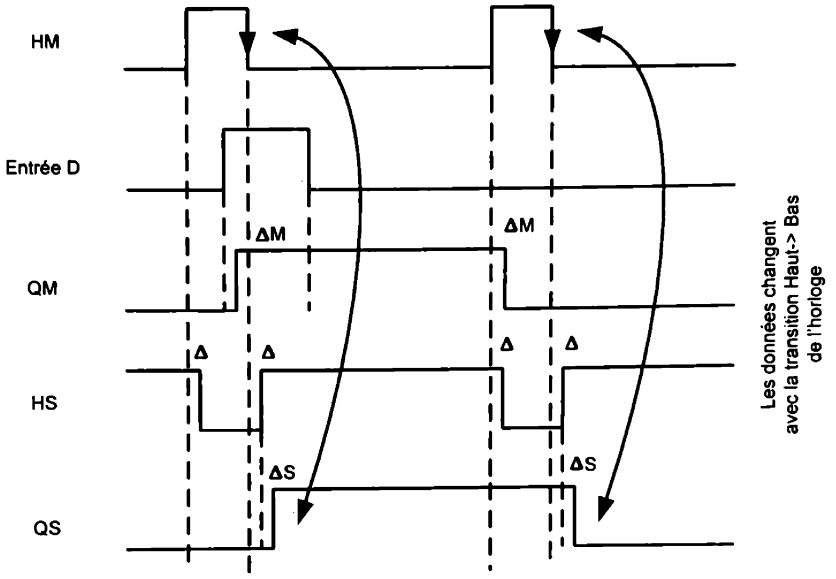


Figure 5.23 Chronogramme (ou *Timing diagram* en anglais)

1. Posons que $D = 1$ et $HM = 1$, donc le latch maître agit de façon transparente, et donc $QM = 1$ (à la figure 5.23, on a inclus le délai de propagation ΔM du latch maître). Quant au latch esclave, $HS = 0$, celui-ci ne réagit donc pas.

2. Posons ensuite que $HM = 0$, donc le latch maître a mémorisé la donnée.

3. À cause du délai de propagation Δ de l'inverseur, les deux horloges restent à 0 pendant une durée Δ. Ainsi, pendant ce bref instant, $HM = HS = 0$. Il n'y a pas de « course ».

4. Passé Δ, $HS = 1$, et alors la sortie QS enregistre la donnée provenant de Qm (après un délai ΔS du latch esclave).

5 Avec la nouvelle montée d'horloge (de HM), le latch maître s'ouvre à nouveau, et alors $QM = 0$ (après un délai ΔM du latch esclave), et plus tard QS passe aussi à 0 lorsque HS repasse à 1, tel qu'il a été indiqué.

Ce qui est essentiel ici, c'est de noter le comportement global du circuit lorsqu'on observe les signaux d'entrée D et de sortie Qs en liaison avec l'horloge HM. On note alors un comportement très intéressant (indiqué avec des flèches sur la figure 5.23) : QS prend la valeur de D sur la transition descendante ( ) de l'horloge. Ce circuit implante ce qu'on appelle un flip-flop *edge-triggered* de type D ou bascule à déclenchement sur front de type D. Ce type de circuit est différent d'un latch qui, comme on l'a vu précédemment, réagit aux niveaux. Pour le distinguer du latch, nous nous référerons à ce type de bascule

à déclenchement sur front avec le vocable « flip-flop ». Son symbole est identique à celui du latch sauf qu'au lieu de représenter l'horloge avec une barre, on la représente avec un triangle tel qu'il a été indiqué sur la figure 5.24 pour le flip-flop *D* :

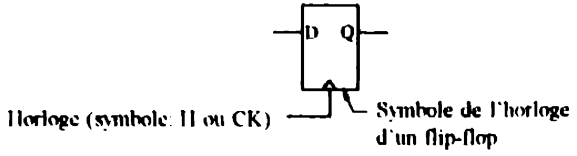


Figure 5.24 Symbole pour un flip-flop D à déclenchement sur front montant (en anglais : « *D* » *type edge-triggered flip-flop*)

Dans les sections suivantes, nous verrons que la configuration maître-esclave peut aussi servir à créer des flip-flops d'autres types comme les flip-flops R-S ou les flip-flops toggle.

Les flip-flops peuvent réagir sur un front descendant (transition Haut-Bas : ⌐, ou *falling edge* en anglais) ou sur un front montant (transition Bas-Haut : ⌐, ou *rising edge* en anglais). On emploiera ainsi les symboles « ↓ ∧ » pour ceux qui réagissent sur les fronts Haut-Bas (⌐) et « ∧ » pour ceux qui réagissent sur les fronts Bas-Haut « ⌐ » plus communs. Voici d'ailleurs le symbole pour un flip-flop D à déclenchement sur front descendant :

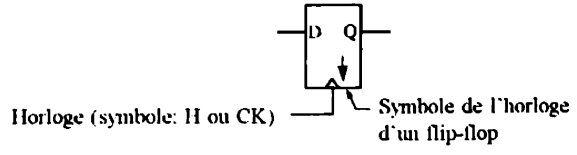


Figure 5.25 Symbole pour un flip-flop *D* à déclenchement sur front descendant

La table de transition du flip-flop *D* est la suivante :

horloge (*H* ou *CK*)	Entrée *D*	Sortie *Q*	Description
0	*X*	*Q0*	Mémorisation
↑	0	0	Mise à 0
↑	1	1	Mise à 1

Figure 5.26

Note : Dans cette table, ↑ dénote un front montant (ou descendant si on considère un flip-flop de ce type). Un tel front est aussi appelé coup d'horloge.

Les flip-flops sont d'une utilité capitale en circuiterie numérique. Ils seront d'ailleurs à la base de tous les circuits étudiés aux chapitres 6 et 7.

Évidemment, pour sa réalisation, chaque unité maître-esclave nécessite deux fois plus de matériel (deux bascules R-S) qu'un simple latch. Par contre, on évite le problème des

« courses » lorsque les latches sont « ouverts ». Il s'agit donc d'une approche très commune. Il est à noter cependant que les circuits maître-esclave sont sujets aux problèmes de la *contamination des 1 et des 0* (*zero and one catching* en anglais) dont nous discuterons plus bas.

Il y a aussi une contrainte sur la durée pendant laquelle le signal d'horloge passe au niveau logique bas, il faut que celle-ci soit supérieure au temps de propagation de l'inverseur pour que l'unité maître-esclave fonctionne correctement. Une illustration extrême de ce phénomène est montrée à la figure 5.27, sur laquelle on note que la donnée *D* ne s'est pas propagée à la sortie puisque le délai Δ dans l'inverseur était plus long que la durée *T* pendant laquelle l'horloge était au niveau logique bas (voir le circuit de la figure 5.22) :

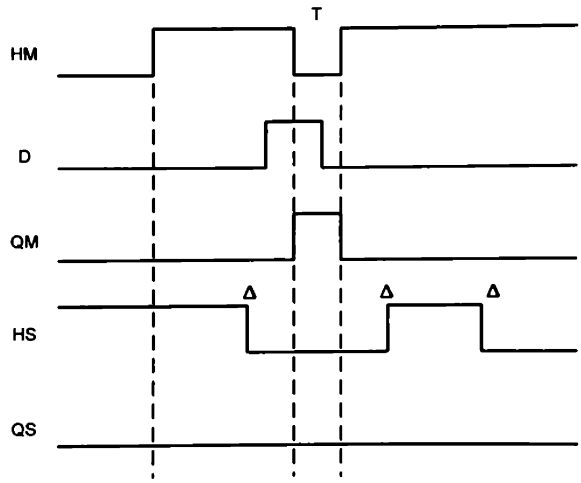


Figure 5.27

Dans cette figure, les délais de propagation pour les bascules maître-esclave sont négligés pour ne pas alourdir le tracé. Nous reviendrons sur les spécifications du signal d'horloge à la section 5.11.

5.7 FORMATION DE REGISTRES À PARTIR DE FLIP-FLOPS D

REGISTRES A DECALAGE (SHIFT REGISTER) : REGISTRES SIPO ET SISO

Comme premier usage des flip-flops, mentionnons les registres à décalage (*shift register* en anglais). Un registre à décalage est un circuit contenant *n* flip-flop et dans lequel, à chaque coup d'horloge, les données progressent d'une position parmi les flip-flops présents connectés dos à dos. Selon les modèles, le décalage peut se faire à gauche (exemple simple sur trois bits : on entre « 0-1-0 » on obtient « 1-0-0 ») ou à droite (exemple simple sur trois bits : on entre « 0-1-0 », on obtient « 0-0-1 »). Le circuit suivant illustre un registre à décalage à quatre bits avec décalage à droite. Puisque l'entrée est série (les données sont introduites une à une par l'entrée unique) et sont disponibles en parallèle,

ce registre est appelé SIPO, de l'anglais *Serial-In Parallel-Out* (entrée série, sortie parallèle). Nous étudierons les registres à décalage en détail au chapitre 6.

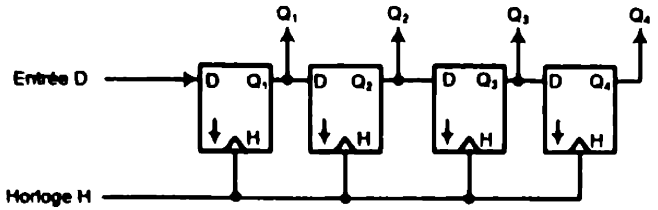


Figure 5.28

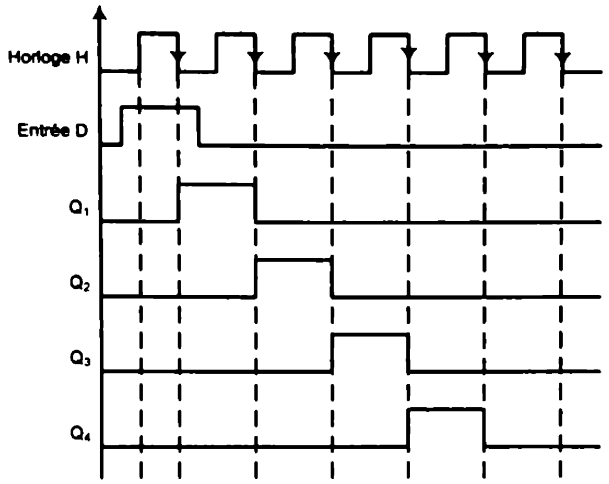


Figure 5.29 Chronogramme pour le circuit du registre à décalage à quatre bits

Du chronogramme précédent, on remarque que la donnée *D* entre de façon asynchrone et en ressort synchronisée. C'est un résultat très important. Nous nous en servirons par exemple pour l'interface entre les données externes (par exemple un interrupteur mécanique) et les circuits numériques synchrones que nous étudierons à partir du chapitre 6.

Si, dans le circuit de la figure 5.29, seule la sortie Q_4 est considérée comme étant la sortie du circuit, nous obtenons un registre à décalage de type SISO, qui signifie, selon son acronyme anglo-saxon : *Serial In Serial Out*. Un registre SISO est utile pour retarder un flux de données. Ainsi, on voit dans le chronogramme précédent que la donnée introduite dans le registre en ressort (par la sortie Q_4) quatre coups d'horloge plus tard (puisque nous avons ici un registre à quatre bits). Ce type de registre est aussi connu sous le nom de FIFO, qui signifie, selon son acronyme anglo-saxon : *First In First Out* puisque la première donnée introduite dans le registre est aussi la première à en sortir.

Finalement, notons que les registres SIPO servent pour l'interface des microcontrôleurs. Si par exemple un microcontrôleur possède un port série, les données obtenues de celui-

ci peuvent être lues en parallèle après transit dans un registre SIPO. La figure 5.30 illustre cet usage qui fournit un port parallèle en sortie à partir de seulement deux lignes d'entrée/sortie du microcontrôleur (une ligne de données et une d'horloge).

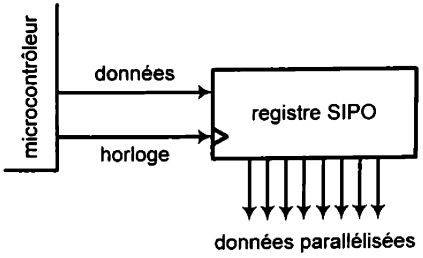


Figure 5.30

REGISTRE PARALLELE (REGISTRE PIPO)

Un autre type de registre très courant est le registre parallèle. Ce registre consiste en une juxtaposition de flip-flops D reliés en parallèle. Dans ce registre, les données entrent en parallèle et sortent du registre de la même façon. Ce type de registre s'appelle registre PIPO, qui signifie, selon son acronyme anglo-saxon : *Parallel In Parallel Out*. Le schéma suivant illustre un registre PIPO de quatre bits. Ce type de registre est utile pour le stockage de données disponibles sur un bus.

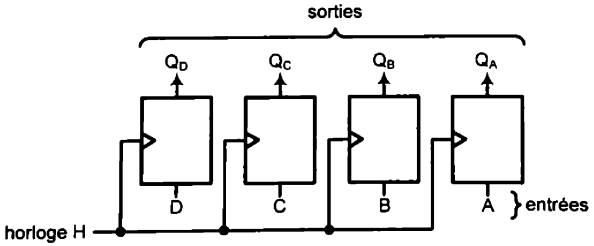


Figure 5.31

Le fonctionnement du registre est le suivant : les valeurs présentes en [*D*, *C*, *B*, *A*] sont mémorisées au coup d'horloge (entrée *H*). Le contenu du registre demeurera inchangé jusqu'au prochain coup d'horloge.

REGISTRE PARALLELE EN SERIE (REGISTRE PISO)

Dans ce registre, les données sont introduites en parallèle, mais sortent en série. Des multiplexeurs (vus au chapitre 4) permettent une sélection externe (introduction des données dans les flip-flops) ou interne (circulation des données à travers les flip-flops). Le circuit est montré plus bas (sur quatre bits) ; son fonctionnement est le suivant :

1. Les données sont présentées aux entrées [*A*, *B*, *C*, *D*], et le bit de sélection *CH* des multiplexeurs est mis à 0 ;
2. Un coup d'horloge survient et mémorise les données [*A*, *B*, *C*, *D*] dans les flip-flops.

3. Le bit de sélection *CH* des multiplexeurs est mis à 1 ;
4. À partir du coup d'horloge suivant, les données [*A*, *B*, *C*, *D*] circulent dans les flip-flops et sortent par le dernier flip-flop (Q_4). *À noter* : le « 0 » qui est introduit dans le premier flip-flop (Q_1). On aurait aussi pu introduire un « 1 » au lieu du « 0 ».

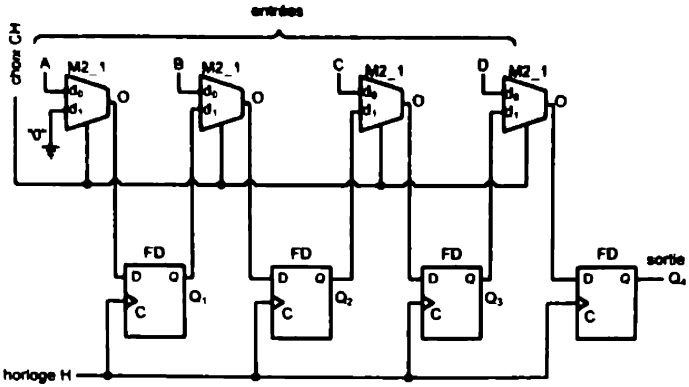


Figure 5.32 Schéma d'un registre PISO

Exemple : Pour le circuit PISO de la figure 5.32, tracer le chronogramme si la donnée [*A*, *B*, *C*, *D*] = (1,0,0,0) est introduite au départ (à temps pour le premier coup d'horloge à $t = 0$). Assumer que *CH* se comporte comme il se doit. Négliger les temps de propagation dans les flip-flops.

Solution : Le schéma du registre PISO fait référence à des flip-flops actifs sur front montant. Un peu avant $t = 0$, les données sont prêtes à être introduites dans les flip-flops (avec $CH = 0$), ce qui survient au coup d'horloge (à $t = 0$). Un peu après, *CH* passe à 1, ce qui permettra la circulation interne des données à partir du prochain coup d'horloge (noter le 0 qui est introduit en Q_1). À partir de schéma et des explications fournies, nous obtenons directement le chronogramme suivant :

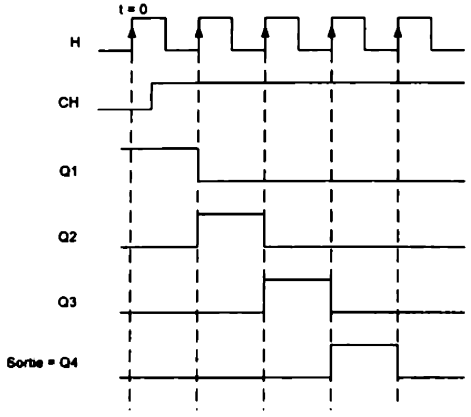


Figure 5.33

5.8 BASCULES À TRANSITION MAÎTRE-ESCLAVE : FLIP-FLOP TOGGLE (FLIP-FLOP T)

FLIP-FLOP TOGGLE SANS ENTREE (FLIP-FLOP T SANS ENTREE)

On peut employer une configuration maître-esclave en reliant deux latches R-S dos à dos comme suit :

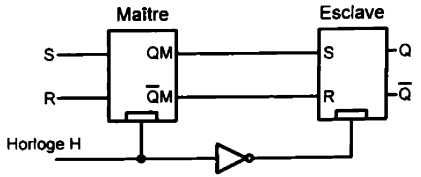


Figure 5.34

Si, par la suite, on relie Q et $\overline{Q}$ de façon croisée avec R et S, on obtient ce qu'on appelle le « flip-flop_toggle » ou simplement flip-flop T, dont l'état change à chaque coup d'horloge (figure 5.35). On se rappelle qu'avec une configuration maître-esclave, d'un point de vue global, on obtient un circuit de bascule à déclenchement sur front ; ce qu'on a appelé un flip-flop (ceci par rapport aux latches qui réagissent aux changements de niveaux du signal d'horloge). Dans cette version minimale du flip-flop toggle, hormis l'horloge, le circuit n'a aucune entrée. En français, « toggle » signifie levier. La figure 5.35 illustre le tout :

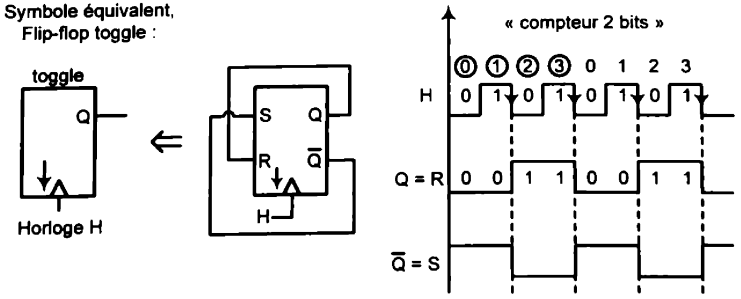


Figure 5.35

Il est à noter que, dans la figure 5.35, les délais de propagation ont été négligés ; nous en reparleront plus loin.

La table de transition du flip-flop toggle sans entrée (flip-flop T sans entrée) est la suivante (Q_0 est la valeur précédente de la sortie Q) :

Horloge (*H* ou *CK*)	Sortie *Q*	Description
0	Q0	Mémorisation
↑	$\overline{Q}_0$	Changement d'état « toggle »

Figure 5.36 Pour un flip-flop T sans entrée réagissant sur un front montant

Réalisation de compteurs à propagation d'horloge ou compteurs ripple avec flip-flop T sans entrée

On voit que ce circuit (figure 5.35) réalise ce qu'on appelle un *compteur logique*, donc un circuit dont la sortie, représentée par un nombre binaire, s'incrémente à chaque coup d'horloge (pour les compteurs décroissants, la sortie se décrémente à chaque coup d'horloge). En effet, si on considère le nombre binaire de deux bits formé de la paire (Q, H ; Q = *MSB*), on obtient successivement les valeurs binaires suivantes : 00, 01, 10, 11, et exprimées en décimal : ⓪ ① ② ③.

Selon le même principe, on peut réaliser un compteur à trois bits en reliant deux flip-flops T en cascade. Chaque flip-flop compte pour un bit, et l'horloge sert de bit moins significatif (*LSB*). Ce principe peut évidemment être étendu à un nombre n de bits. Un compteur pour lequel la sortie d'un flip-flop sert d'horloge au flip-flop suivant, est appelé *compteur à propagation d'horloge* ou en anglais compteur *ripple*. En français, *ripple* signifie ondulation. De façon imagée, c'est donc « l'ondulation » de l'horloge qui se propage, un peu comme la propagation des vaguelettes sur l'eau lorsqu'on y jette un caillou. La figure 5.37 présente le montage d'un compteur à propagation à trois bits (en incluant l'horloge comme bit le moins important, ou deux bits si l'on ne considère pas le signal d'horloge) :

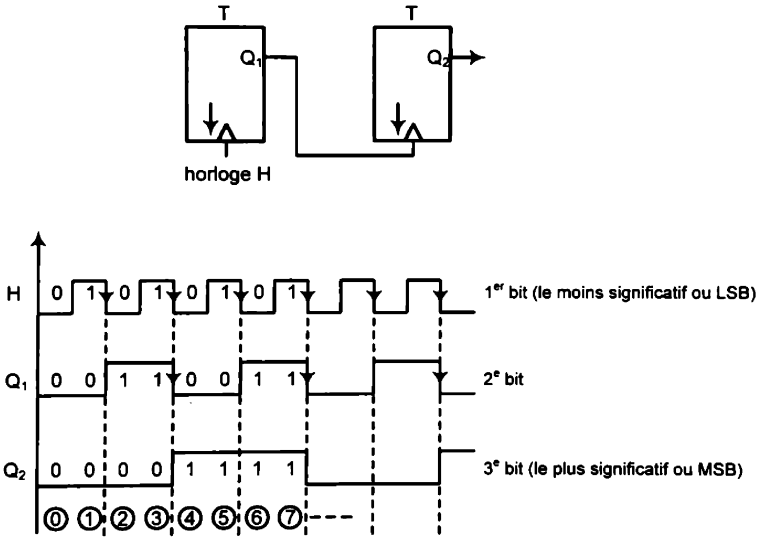


Figure 5.37

Il est à noter que, dans la figure 5.37, les délais de propagation ont été négligés ; nous en reparlerons plus loin.

Un résultat intéressant que l'on observe des chronogrammes des deux derniers circuits, est qu'un flip-flop T divise la fréquence de l'horloge par deux. Ainsi, avec n flip-flops T reliés en cascade, la fréquence de l'horloge est divisée par 2^N.

On l'observe très clairement sur le dernier chronogramme : la fréquence du signal pour Q_1 est la moitié de celle de l'horloge, et la fréquence du signal Q_2 est la moitié de celle du signal Q_1. Ceci est donc à retenir : *on divise la fréquence par deux lorsque le signal est passé dans une bascule toggle.*

Il faut bien sûr réaliser que l'utilisation de circuits dans lesquels le signal d'horloge est « manipulé », comme dans les compteurs ripple, peut avoir des conséquences néfastes, car, à cause des délais de propagation qui s'accumulent, les signaux ne sont plus synchronisés entre eux. À titre d'exemple, on a retracé le chronogramme du compteur ripple à trois bits de la figure 5.37 en incluant le délai de propagation dans chaque flip-flop T :

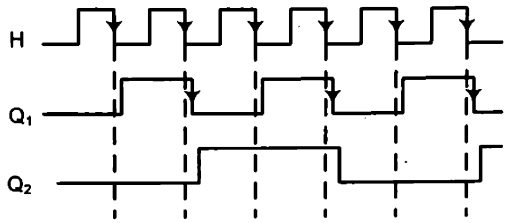


Figure 5.38

On constate deux effets :

1. Les signaux Q_1 et Q_2 ne sont pas synchronisés avec le signal d'horloge H puisqu'on observe un léger décalage dû au temps de propagation dans les flip-flops ;
2. Pire : le délai n'est pas fixe. Ainsi, il s'accroît pour Q_2 par rapport à Q_1 puisque le signal passe dans deux flip-flops pour Q_2, et dans un seul pour Q_1. Plus on rajoutera de flip-flops dans la cascade, plus le délai s'accroîtra.

Si la chronologie des signaux (le *timing* en anglais) est critique pour l'application envisagée, il faudra donc éviter les configurations de type ripple qui ne peuvent pas être considérées comme de vrais circuits synchrones. L'utilisation de tels signaux décalés (on dit alors que les signaux sont déphasés) est dangereuse et peu conduire à des aléas. Dans un vrai circuit synchrone, le signal d'horloge *intact* se propage directement à chacun des flip-flops présents, sans manipulation de celui-ci. Nous reviendrons plus tard sur la façon correcte de propager un signal d'horloge dans un circuit synchrone (au chapitre 6).

FLIP-FLOP T AVEC ENTREE D'ACTIVATION

Il existe une variante très utile du flip-flop toggle : le flip-flop T avec entrée d'activation (en anglais : flip-flop T avec *enable*). Cette entrée sert à activer le fonctionnement du flip-flop. Si l'entrée est au niveau logique « 0 », le flip-flop conserve son état ; il s'agit donc d'une mémorisation. Si l'entrée est au niveau logique « 1 », le flip-flop fonctionne comme un flip-flop toggle normal, et donc, au coup d'horloge, sa sortie s'inversera. Puisque son usage est plus répandu que le flip-flop T sans entrée, de façon courante, le flip-flop T avec entrée d'activation est souvent simplement appelé flip-flop T.

La table de transition du flip-flop toggle avec entrée d'activation (flip-flop T avec entrée) est la suivante :

Horloge (*H* ou CK)	Entrée *T* d'activation	Sortie Q	Description
X	0	Q0	Mémorisation
↑	1	$\bar{Q}_0$	Changement d'état « toggle »

Figure 5.39

Le symbole de ce flip-flop est le suivant :

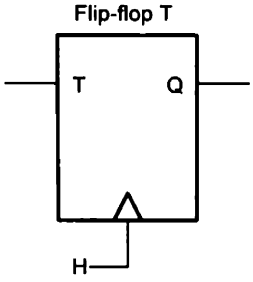


Figure 5.40

Ce flip-flop est fort employé pour la réalisation de compteurs (nous en parlerons au chapitre 6). Finalement, notons que l'entrée *T* est réalisée grâce à deux portes ET placées aux entrées R et S de la configuration R-S maître-esclave utilisée pour bâtir le flip-flop T sans entrée d'activation (figure 5.35).

5.9 FLIP-FLOP R-S MAÎTRE-ESCLAVE ET LES CONTAMINATIONS AUX « 1 » ET AUX « 0 » (« 1 *CATCHING* » OU « 0 *CATCHING* »)

La configuration maître-esclave utilisée jusqu'à présent est sensible aux aléas, d'où bien sûr l'importance de tout mettre en œuvre pour limiter ceux-ci. Cette sensibilité porte deux noms différents selon le type de problème qui survient :

- la contamination aux « 1 » ou « 1 *catching* », qui signifie que l'on « attrape » les 1 ;
- la contamination aux « 0 » ou le « 0 *catching* », qui signifie que l'on « attrape » les 0.

Ces problèmes surviennent lorsque les horloges des latches employés en configuration maître-esclave sont actives. Pendant ces périodes, le moindre aléa sur l'entrée S causera la contamination aux « 1 » et le moindre aléa sur l'entrée *R* causera la contamination aux « 0 ». La figure 5.41 illustre les deux phénomènes (les délais de propagation ont été volontairement négligés dans cette figure).

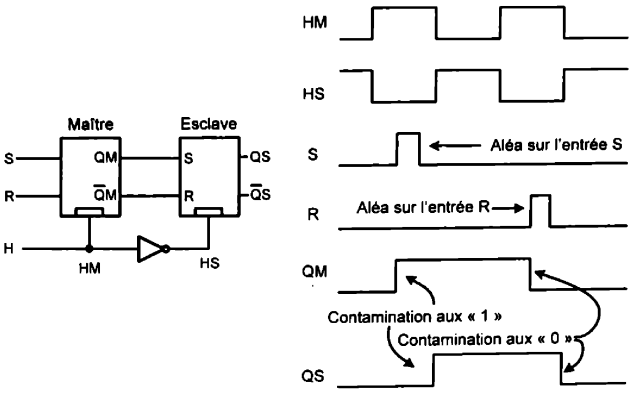


Figure 5.41

On constate donc que, pendant que les signaux d'horloge étaient au 1 logique, à cause de la « transparence » des latches, les effets des aléas se sont propagés. Heureusement, dans la figure, l'aléa sur la ligne S a été par la suite compensé par un aléa sur la ligne R ; une telle combinaison ne survient évidemment pas toujours. Notons aussi que, suite aux aléas, la sortie QS a été contaminée, ce qui peut avoir des conséquences désastreuses selon les applications envisagées (exemple : un distributeur automatique qui compte une pièce de monnaie de plus que ce qui a été introduit, et qui lèse donc le marchand en favorisant le client, etc.)

5.10 AUTRE CONFIGURATION POUR LE FLIP-FLOP D (SECTION FACULTATIVE)

La configuration maître-esclave n'est pas la seule façon de réaliser un flip-flop D à déclenchement sur front (« *edge-triggered* » *D flip-flop*). Dans cette section, nous allons étudier une configuration formée de trois bascules R-S qui implantent un tel flip-flop D. Cette réalisation tient compte des délais de propagation des trois bascules R-S (respectivement Δ1, Δ2, Δ3) et aussi de la rétroaction du circuit. Il est à noter qu'on assume que les bascules R-S sont bâties (à l'interne) avec des portes NON-OU et que le cas interdit (utile ici !) $R = S = 1$ fournit donc des « 0 » aux sorties. Le schéma est le suivant ; il est analysé plus bas avec le chronogramme correspondant :

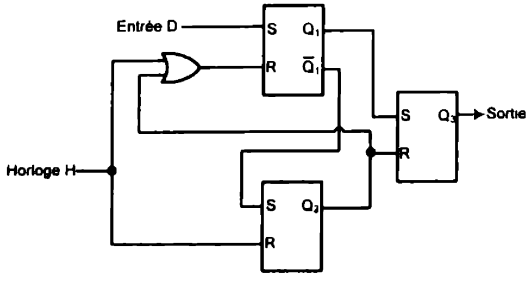


Figure 5.42

Cas 1 : On assume que $Q1 = Q2 = Q3 = D = 0$, et que $H = 1$. On note que le circuit est stable avec :

Pour $Q1$: $R = 1$, $S = 0$, donc $Q1 = 0$ (mise à 0) ;

Pour $Q2$: $R = 1$, $S = 1$, donc $Q2 = 0$ (état interdit qui fournit une sortie nulle) ;

Pour $Q3$: $R = 0$, $S = 0$, donc $Q3 = 0$ (mémorisation).

Cas 2 : D passe à 1, et H reste à 1. On note que rien ne change et que le circuit est stable avec :

Pour $Q1$: $R = 1$, $S = 1$, donc Q1 = 0 (état interdit qui fournit une sortie nulle) ;

Pour $Q2$: $R = 1$, $S = 1$, donc $Q2 = 0$ (état interdit qui fournit une sortie nulle) ;

Pour $Q3$: $R = 0$, $S = 0$, donc $Q3 = 0$ (mémorisation).

Cas 3 : D reste à 1, et H passe à 0. On note que $Q1$ et $Q3$ changent et que le circuit est stable avec :

Pour $Q1$: $R = 0$, $S = 1$, donc $Q1 = 1$ (mise à 1 après un délai $\Delta 1$) ;

Pour $Q2$: $R = 0$, $S = 0$, donc $Q2 = 0$ (mémorisation) ;

Pour $Q3$: $R = 0$, $S = 1$, donc $Q3 = 1$ (mise à 1 après un délai $\Delta 3$ retardé après le changement survenu en $Q1$).

Cas 4 : D passe à 0, et H reste à 0. On note que rien ne change et que le circuit est stable avec :

Pour $Q1$: $R = 0$, $S = 0$, donc $Q1 = 1$ (mémorisation) ;

Pour Q2 : $R = 0$, $S = 0$, donc $Q2 = 0$ (mémorisation) ;

Pour Q3 : $R = 0$, $S = 1$, donc $Q3 = 1$ (mise à 1, aucun changement puisque $Q3$ valait déjà 1).

Cas 5 : D reste à 0, et H passe à 1. On note que rien ne change et que le circuit est stable avec :

Pour $Q1$: $R = 1$, $S = 0$, donc $Q1 = 0$ (mise à 0 après un délai $\Delta 1$) ;

Pour $Q2$: $R = 1$, $S = 1$, donc $Q2 = 0$ (état interdit qui fournit une sortie nulle, aucun changement puisque $Q2$ valait déjà 0) ;

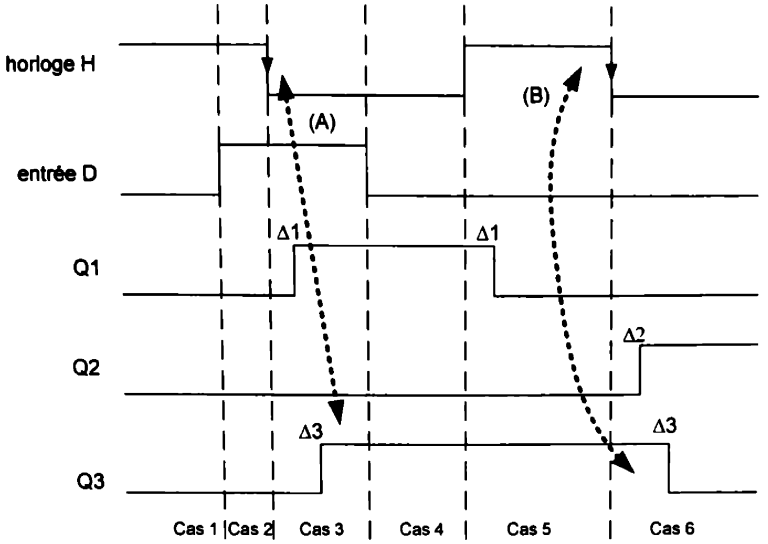
Pour $Q3$: $R = 0$, $S = 0$, donc $Q3 = 1$ (mémorisation ; reste à 1).

Cas 6 : D reste à 0, et H passe à 0. On note que $Q2$ et $Q3$ changent, et que le circuit est stable avec :

Pour $Q1$: $R = 1$, $S = 0$, donc $Q1 = 0$ (mise à 0, aucun changement puisque $Q1$ valait déjà 0) ;

Pour $Q2$: $R = 0$, $S = 1$, donc $Q2 = 1$ (mise à 1 après un délai $\Delta 2$) ;

Pour $Q3$: $R = 1$, $S = 0$, donc $Q3 = 0$ (mise à 0 après un délai $\Delta 3$ retardé après le changement survenu en $Q2$).



Important : À la descente d'horloge, la sortie Q3 du circuit a pris la valeur présente en D.

Figure 5.43

Après avoir bien compris le détail des six cas présentés, on remarque que globalement (cas pointillés gras (*A*) et (*B*) du chronogramme), à la descente d'horloge, la sortie *Q*3 du circuit prend la valeur alors présente à l'entrée *D*. Ce fonctionnement est bien celui d'un flip-flop D à déclenchement sur front descendant, comme on l'avait étudié précédemment.

5.11 QUELQUES PARAMÈTRES IMPORTANTS DES FLIPS-FLOPS SYNCHRONES

Ce livre n'en est pas un d'électronique. Notre attention s'est donc toujours plus attardée aux principes qu'à l'implantation sous-jacente de ceux-ci. Il faut néanmoins être bien conscient des limitations physiques des circuits avec lesquels nous travaillons. On a déjà évoqué la question du temps de propagation (t_P). D'autres considérations sont très importantes pour les flip-flops ; elles seront abordées dans cette section.

Dans le cas des flip-flops à déclenchement sur front, pour qu'elles soient reconnues valides, il faut que les données soient stabilisées un peu avant le coup d'horloge ; c'est ce qu'on appelle le temps de mise en place ou t_{su} (*set up time* en anglais). De même, il faut aussi qu'elles demeurent inchangées pendant un bref instant après le coup d'horloge ; c'est ce qu'on appelle le temps de maintien ou t_h (*hold time* en anglais).

Le coup d'horloge doit aussi survenir « assez rapidement » ; c'est ce qu'on appelle la vitesse de l'horloge (ou en anglais, le *slew rate*) qui est exprimée en volt par seconde ([V/s]). Ne parle-t-on pas de flip-flop à déclenchement de front ? Ainsi, si l'horloge est trop

lente, elle ne sera tout simplement pas reconnue. Cette vitesse d'horloge fait référence à la durée minimale d'une impulsion t_w (*width* en anglais).

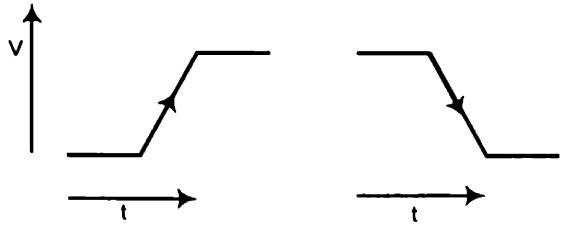


Figure 5.44 Illustration de la vitesse de montée et de descente des fronts d'horloge exprimée en [V/s]

Une autre spécification importante est le temps minimal de rétablissement ou, en anglais, le *minimum recovery time* (t_{rec}). Ce paramètre spécifie le temps qui doit s'écouler après un changement d'entrée asynchrone pour que l'horloge change à nouveau :

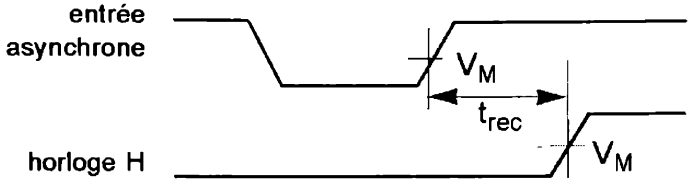


Figure 5.45

On mentionne ici une entrée asynchrone, car, dans le cas des entrées synchrones (donc des entrées dont les changements sont en phase avec l'horloge), la question ne se pose pas puisque ces signaux ne changent pas entre les coups d'horloge.

Évidemment, il y a aussi la fréquence maximale de l'horloge f_{max} qui est un paramètre qui dépend de la famille de circuit employée et qui est bien sûr relié aux autres paramètres (t_{su}, t_h et t_w).

La tension d'alimentation (5 V pour la famille TTL) et les valeurs de tensions associées aux niveaux logiques (par exemple pour la famille TTL standard 0 à 0.8 V pour le « 0 » logique, et 2 à 5 Volt pour le « 1 » logique) sont des paramètres également essentiels pour le bon fonctionnement du circuit.

La figure 5.47 illustre ces paramètres pour un flip-flop D :

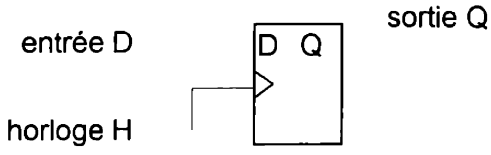
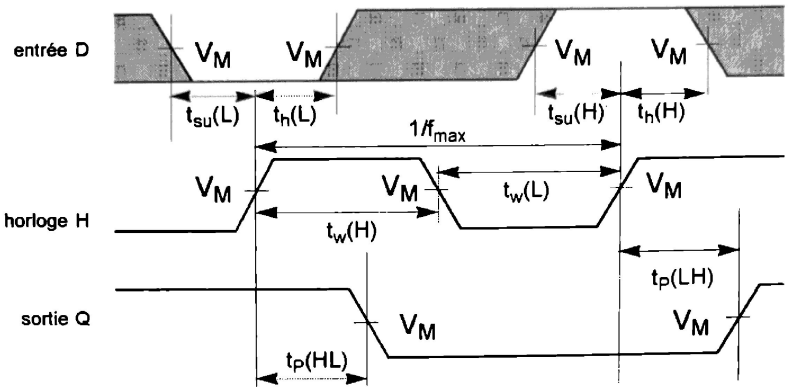


Figure 5.46



Note : Pour l'entrée D, les zones ombragées permettent d'envisager tous les scénarios possibles d'arrivée de la donnée jusqu'aux cas limites.

Figure 5.47 Paramètres de fonctionnement ; cas d'un flip-flop D.

Prenons par exemple le circuit 74F74 de Philips[6] qui contient deux flip-flops D ; les valeurs typiques des paramètres dont nous venons de discuter sont les suivantes :

Paramètre	Valeurs typiques*	Unités	Signification
f_{max}	125	MHz	Fréquence maximale de fonctionnement
t_{PHL} t_{PLH}	6.2 4.6	ns	Temps de propagation du coup d'horloge à la valeur de sortie en Q (en anglais : *clock to Q*)
$t_{su}(H)$ $t_{su}(L)$	2 3	ns	Temps de mise en place (en anglais : *set up*)
$t_w(H)$ $t_w(L)$	4 5	ns	Durée minimale de l'impulsion (en anglais : *width*)
$t_h(H)$ $t_h(H)$	1 1	ns	Temps de maintien (en anglais : *hold*)
t_{rec}	2	ns	Temps minimal de rétablissement (en anglais : *minimum recovery time*)

* Valeurs obtenues avec une tension d'alimentation de 5 V, V_M de 1.5 V (donc une amplitude de signal de 3 V), une charge en sortie composée d'une résistance de 500 Ω en parallèle avec un condensateur de 50 pF, à une température ambiante de 25°C.

Figure 5.48

On note que les valeurs des paramètres ne sont pas toujours symétriques selon les montées (*H* pour *high*, haut) et les descentes (*L* pour *low*, bas) logiques ; cela dépend de la réalisation interne des circuits.

Finalement, il est intéressant de constater que la fréquence maximale f_{max} correspond à peu près au calcul suivant :

$$f_{max} = 125 \text{ MHZ},$$

[6] Voir : http://www.semiconductors.philips.com/acrobat_download/datasheets/74F74_2.pdf

qui donne une période équivalente égale à :

$$\frac{1}{125MHZ} \approx 8ns \approx t_{su} + t_h + t_{tt}$$

$$2.5 + 1 + 4$$

Dans ce livre, lors du tracé des chronogrammes et sauf avis contraire, on ne tiendra pas compte de t_{su}, de t_h, de t_{rec}, de t_w, ni de t_p.

5.12 FLIP-FLOP J-K

Un autre type de flip-flop très employé est le flip-flop J-K. Ce flip-flop est une version évoluée du flip-flop R-S. Tout comme le flip-flop R-S, elle possède deux entrées : l'une appelée *J* et qui sert pour la mise à 1, l'autre, appelée *K*, sert pour la mise à 0. Dans un premier temps, étudions le circuit suivant réalisé au moyen d'une bascule R-S à laquelle on a ajouté des portes ET triple, qui permettent de rajouter un *feed-back* supplémentaire et aussi les entrées *J* et *K* :

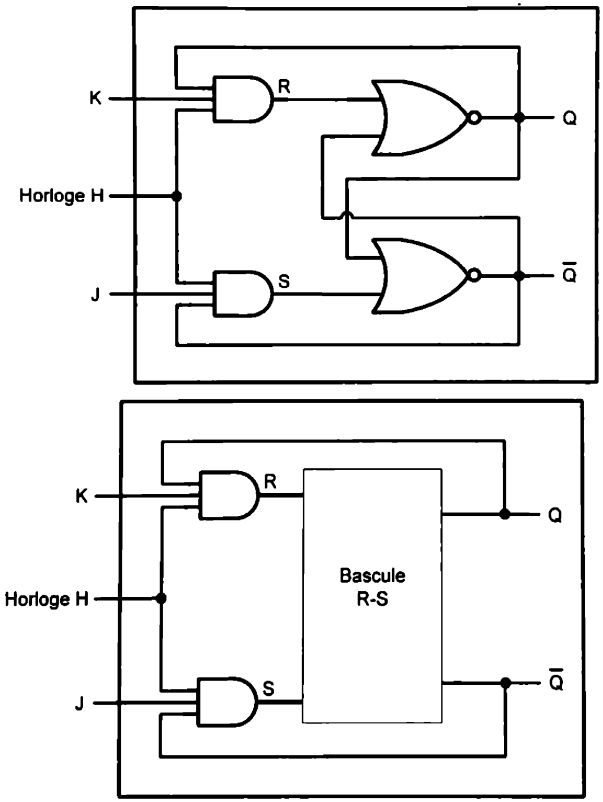


Figure 5.49 Bascule J-K réalisée au moyen d'une bascule R-S.

Il s'agit d'une bascule puisque les niveaux (et non les transitions) de l'horloge affectent le fonctionnement du circuit.

En étudiant un à un les huit cas possibles en prenant comme variables la sortie Q au temps présent, J et K, on peut dresser la table de transition suivante du circuit. Dans celle-ci, $Q_{suivant}$ correspond à la valeur en sortie après la transition de l'horloge. Par exemple, si on a $Q_{présent} = 1$, $J = 0$ et $K = 0$, alors les sorties des deux portes ET sont nulles, et la bascule R-S est en état de mémorisation ; donc, lorsque H deviendra actif (c'est-à-dire qu'il passera par la suite logique 0-1-0 (_┘‾└_)), rien ne se passera. Si maintenant on a $Q_{présent} = 1$, $J = 1$ et $K = 1$, alors, lorsque H deviendra actif, on aura $R = 1$ et $S = 0$ et la bascule R-S passera à $Q_{suivant} = 0$ (mise à 0 de la bascule). Tous les autres cas se dérivent de la même façon.

$Q_{présent}$	J	K	$Q_{suivant}$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Figure 5.50 Table de transition (pour H = 1, si H = 0, alors le circuit est en mode de mémorisation)

De cette table de transition, les fonctions du circuit se résument alors comme ceci (Q correspond à la valeur de sortie au temps présent, et Q_{n+1} à la valeur après activation du signal d'horloge (suite logique 0-1-0, _┘‾└_)) :

Q	J	K	Q_{n+1}	Remarque
Q_0	0	0	Q_0	Mémorisation
x	0	1	0	Mise à 0
x	1	0	1	Mise à 1
Q_0	1	1	$\overline{Q}_0$	Toggle ou changement d'état

Figure 5.51

On remarque que le cas $J = K = 1$, correspond au cas *toggle ou changement d'état*. On se rappelle que le cas $R = S = 1$ était interdit pour les bascules R-S, ce qui en limite l'usage.

On peut relier deux bascules R-S (sans horloge) selon la configuration maître-esclave, étudiée précédemment, et ajouter des portes NON-ET aux entrées pour inclure les entrées J, K et l'horloge H. Un tel circuit implante un flip-flop J-K qui réagit aux transitions descendantes de l'horloge (voir exercice plus bas). Le flip-flop ainsi obtenu est dessiné à la figure 5.52. Il suit la même table de fonctionnement que la bascule J-K.

Note : Pour cette réalisation, des bascules R-S réalisées avec des portes NON-ET ont été employées (voir la section *Problèmes résolus* pour la réalisation de cette bascule).

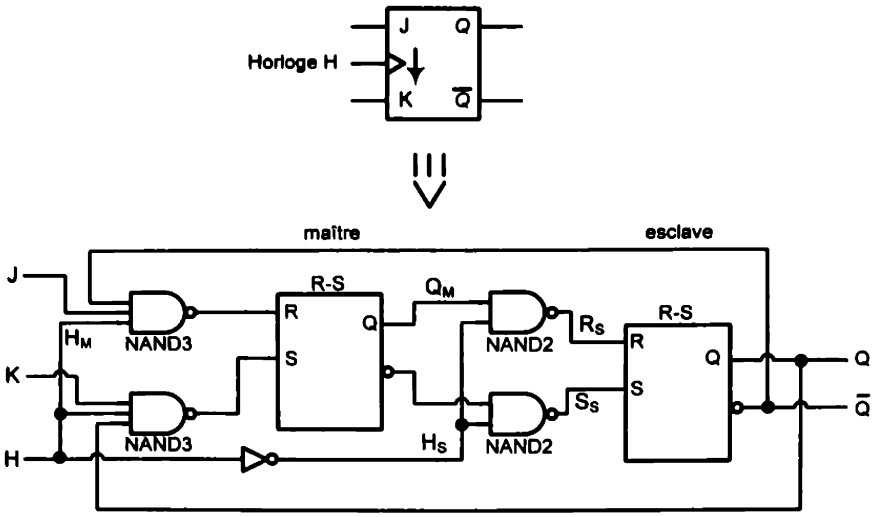


Figure 5.52 Flip-flop J-K avec déclenchement sur front descendant d'horloge (réalisé (à l'interne, à droite) au moyen d'une configuration maître-esclave)

Le flip-flop J-K offre beaucoup de possibilités de design avec ses deux entrées J et K qui lui donnent un niveau de complexité supérieur par rapport aux autres flip-flops étudiés.

Exemple : Vérifier le fonctionnement du flip-flop J-K de la figure 5.52. Tester tous les cas (en négligeant les temps de propagation).

Solution : Il suffit de vérifier tous les cas possibles de $J = K = 0$ à $J = K = 1$. On assume au départ que les deux bascules sont au repos (donc à « 0 »). Cela donne le chronogramme global suivant (en négligeant les temps de propagation) :

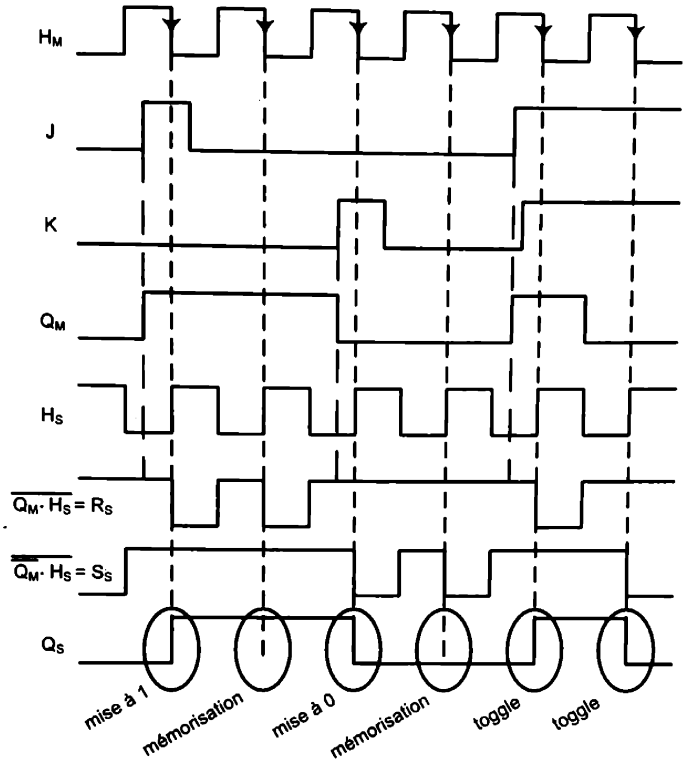


Figure 5.53

Le chronogramme se bâtit petit à petit en considérant les valeurs des signaux présents à chaque transition du signal d'horloge. On vérifie bien que cette configuration maître-esclave se comporte globalement comme un flip-flop J-K à déclenchement sur front descendant.

Exemple : La configuration maître-esclave n'est pas la seule qui permet d'implanter un flip-flop J-K. On peut aussi employer un flip-flop D auquel on ajoute quelques portes simples, comme à la figure 5.54. Pour cette figure, on demande de vérifier le fonctionnement du flip-flop J-K en testant tous les cas.

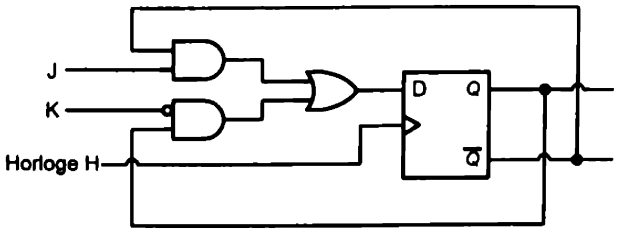


Figure 5.54

Solution : Il suffit de vérifier tous les cas possibles de $J = K = 0$ à $J = K = 1$. On assume au départ que le flip-flop est à « 1 ». Cela donne le chronogramme global suivant (en négligeant les temps de propagation) :

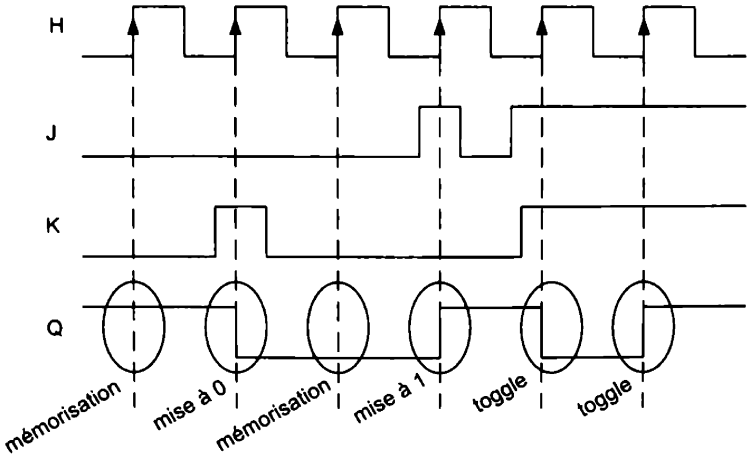


Figure 5.55

Comme pour l'exemple précédent, le chronogramme se bâtit petit à petit en considérant les valeurs des signaux présents à chaque transition du signal d'horloge. On vérifie bien que ce circuit se comporte globalement comme un flip-flop J-K à déclenchement sur front montant. On note que les changements en sortie Q sont synchrones (en phase avec les montées d'horloge), alors que les signaux J et K, tout en respectant les t_{su} et t_h, ne l'étaient pas. Au chapitre suivant nous étudierons la démarche qui permet de concevoir un tel circuit.

5.13 RÉSUMÉ : FLIP-FLOPS D, T, J-K

La table suivante résume le fonctionnement des flip-flops D, J-K et T, que nous avons étudiés dans ce chapitre, en un tableau synoptique qui sera utile au chapitre 6, à la fois pour l'analyse, mais aussi pour la conception (design) des circuits.

Les ingénieures ne s'entendent pas sur le flip-flop le plus employé : le type D ou le type J-K. Les partisanes des flip-flops D affirment que ceux-ci sont les plus employés à cause de leur simplicité : il n'y a qu'une seule entrée à commander. Les partisanes des flip-flops J-K considèrent que ces derniers sont les plus employés à cause des grandes possibilités que leur confèrent leur deux entrées. Ce qui est important de retenir (et nous le verrons au chapitre suivant), c'est que l'on peut réaliser un circuit donné avec n'importe quel type de flip-flops (T, D, J-K). Évidemment, la logique combinatoire complémentaire sera différente et plus ou moins complexe selon le type de flip-flop choisi. Ceci sera discuté au chapitre 6.

État présent Q_N		État suivant (futur) Q_{N+1}	Flip-flop D	Flip-flop T (entrée d'activation)	Flip-flop J-K			
					J	K	J	K
0	→	0	0	0	0 0	1 0	0	X
0	→	1	1	1	1 1	0 1	1	X
1	→	0	0	1	0 1	1 1	X	1
1	→	1	1	0	0 1	0 0	X	0

← ANALYSE
DESIGN →

Figure 5.56

Cette table[7] donne la (les) valeur(s) de l'entrée à fournir au flip-flop pour obtenir la transition indiquée. Par exemple, si l'on désire obtenir une transition 0→1 pour un flip-flop D, on devra mettre l'entrée D du flip-flop à 1. S'il s'agit d'un flip-flop J-K, il suffit que l'entrée J soit mise à 1, peut importe la valeur de K. En effet, si $J = K = 1$, on sera en mode toggle, et le flip-flop passera à 1. Si, par contre, J=1 et K=0, alors on force une mise à 1.

Des aspects intéressants ressortent de cette table. Ainsi, pour un flip-flop D, la valeur de l'état futur correspond toujours à la valeur de D (ce qui est évident par ailleurs). De même, pour un flip-flop T, il n'y aura changement que si l'entrée d'activation T est à un, autrement, il n'y aura pas de changement. Pour le flip-flop J-K, les choses sont plus compliquées, et... on se réfère à la table.

Cette table possède deux utilités selon qu'on la lit de gauche à droite, ce qui sert pour le design, ou de droite à gauche, ce qui sert pour l'analyse. Ces concepts seront étudiés au chapitre suivant.

Le flip-flop R-S n'est pas inclus dans ce tableau, étant donné ses désavantages, dont bien sûr le cas interdit.

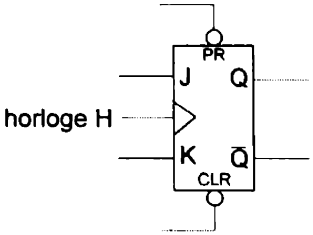
ENTREES DE MISE A 0 ET DE MISE A 1

Outre l'horloge et leurs entrées spécifiques (D pour les types D, J-K pour les types J-K, etc.), certains flip-flops et bascules sont dotés d'entrées supplémentaires qui permettent une mise à 1 (entrée PR ; *preset* en anglais) ou une mise à 0 (entrée CLR, *clear* en anglais). Selon les modèles, ces entrées peuvent être synchrones ou asynchrones. Comme dans la plupart des cas, ces entrées sont asynchrones, donc agissent immédiatement, peu importe l'état de l'horloge ; elles ne servent dans les circuits synchrones qu'au départ pour l'initialisation et n'agissent plus par la suite afin de conserver la nature synchrone du circuit. La figure 5.57 montre un flip-flop J-K doté de telles entrées. On remarque que, pour les entrées PR et CLR, il faut un signal actif bas pour un fonctionnement de celles-ci. Cela est d'ailleurs indiqué sur le chronogramme à droite. Que se passe-t-il si

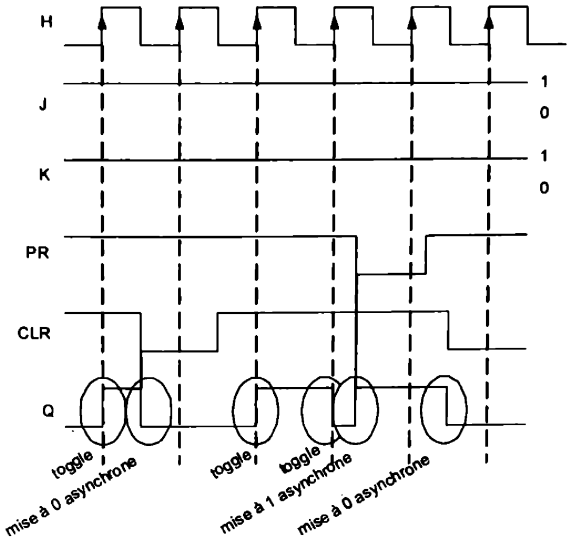
[7] Adaptée de l'ouvrage *Digital Design from Zero to One* de Jerry D. Daniels, p. 312, table 6.1.

PR = CLR = 0 ? En fait, sont-ils vraiment à 0 simultanément ? Cela nous ramène à notre discussion concernant le cas interdit avec $R = S = 1$ pour les bascules R-S. On en avait conclu que, selon toute vraisemblance, un des deux signaux arrivait avant l'autre.

Flip-flop avec entrée PR de mise à 1 et CLR de mise à 0.



Chronogramme des mises à 0 et à 1 asynchrones (la bascule est toujours en mode toggle avec J= K = 1) et les entrées PR et CLR activées comme il a été indiqué.



Lors du tracé, on a négligé le temps de propagation, ce qu'on fera toujours, sauf avis contraire.

Figure 5.57

Dans ce chapitre, nous avons étudié les bascules et les flip-flops, ceux-ci sont à la base des circuits synchrones qui seront étudiés aux chapitres suivants.

PROBLÈMES RÉSOLUS

5.1. Soit le circuit suivant soumis aux entrées S et R fournies. Quel sera le chronogramme correspondant à la sortie Q ?

Note : On assume une bascule R-S réalisée au moyen de portes NON-OU et qu'au début, $Q = 0$. Négliger les temps de propagation.

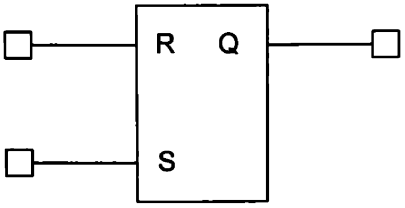


Figure 5.58

Solution : Le chronogramme de Q se bâtit de proche en proche en analysant les entrées à chaque transition et en se rappelant le fonctionnement de la bascule R-S : S à 1 et $R = 0$ pour la mise à 1, R à 1 et $S = 0$ pour la mise à 0, et $R = S = 0$ pour la mémorisation. On se rappelle aussi que, pour une bascule R-S réalisée au moyen de portes NON-OU, le cas interdit $R = S = 1$ donne $Q = 0$.

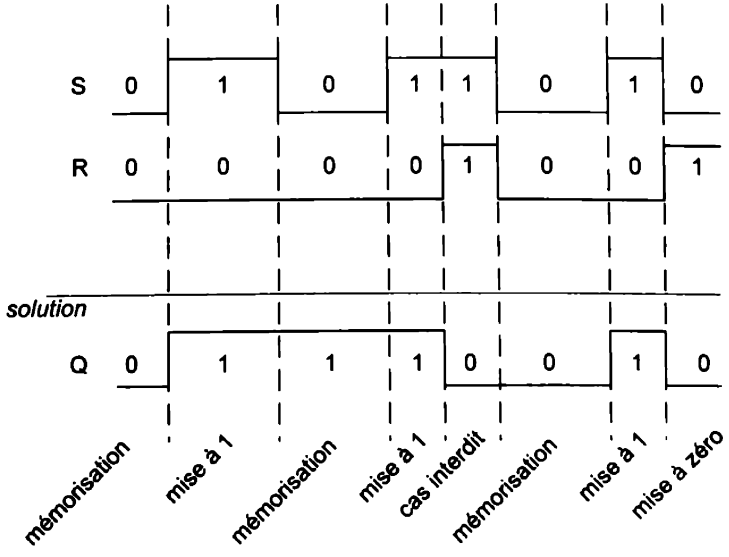


Figure 5.59

5.2. Peut-on réaliser une bascule R-S avec des portes NON-ET ?

Solution : Reprenons la même topologie que pour une bascule R-S réalisée au moyen de portes NON-OU, mais en employant cette fois des portes NON-ET. Cette fois, cela donne le circuit suivant :

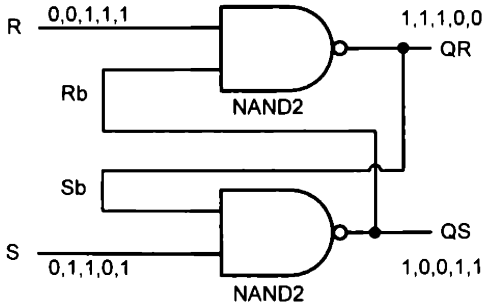


Figure 5.60

D'une façon similaire à notre étude de la bascule R-S réalisée avec des portes NON-OU (section 5.3), on peut trouver les scénarios suivants :

R	S	$QR = Q$	$QS = \overline{Q}$	Commentaires
0	0	1	1	État interdit
0	1	1	0	Mise à 1
1	1	1	0	Mémorisation
1	0	0	1	Mise à 0
1	1	0	1	Mémorisation

Figure 5.61 Table de transition pour une bascule R-S réalisée avec des portes NON-ET

On débute avec (R, S) = (0, 0) en se rappelant l'axiome $\overline{0 \bullet X} = \overline{0} = 1$ pour une porte NON-ET. Les scénarios s'obtiennent les uns après les autres en testant les différentes valeurs tel qu'il a été indiqué. On remarque que cette fois le scénario interdit correspond à $R = S = 0$. L'interdiction de ce scénario nous permet d'avoir toujours une sortie égale à l'inverse de l'autre (Q et $\overline{Q}$). La mémorisation correspond ici au scénario $R = S = 1$. Ce n'est donc pas une « vraie bascule R-S » (si l'on peut dire), mais cela y ressemble. En fait, pour avoir une « vraie bascule R-S », il faudrait ajouter des inverseurs aux entrées et obtenir que R devienne S et que S devienne R. On retrouverait alors la table de transition de la « vraie bascule R-S » (telle que conçue avec deux portes NON-OU). Il n'y a pas grand intérêt à procéder ainsi... Par contre, la bascule R-S, version porte, NON-ET, possède des usages qui lui sont particuliers ; c'est donc néanmoins un circuit intéressant. On a vu d'ailleurs que la bascule R-S, version porte NON-ET pouvait servir pour la réalisation de flip-flops J-K.

5.3. Soit le circuit suivant soumis aux entrées H et D fournies. Quel sera le chronogramme correspondant de la sortie Q ?

Note : On assume qu'on a un latch D et qu'au début, $Q = 0$. Négliger les temps de propagation.

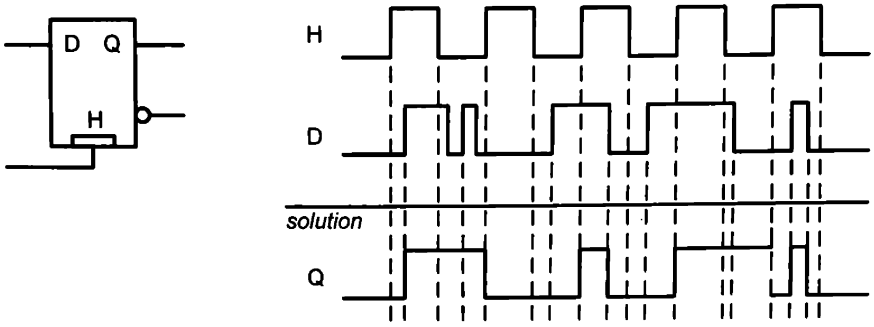


Figure 5.62

Solution : On se rappelle que, pour un latch D, la sortie Q suit l'entrée D lorsque $H = 1$ et que, lorsque $H = 0$, on a mémorisation. Partant de cela, le chronogramme se bâtit de proche en proche tel qu'il a été indiqué.

5.4. Même question que la précédente, mais cette fois, on assume qu'on a un flip-flop D à déclenchement sur front montant.

Note : Au début, $Q = 0$. Négliger les temps de propagation.

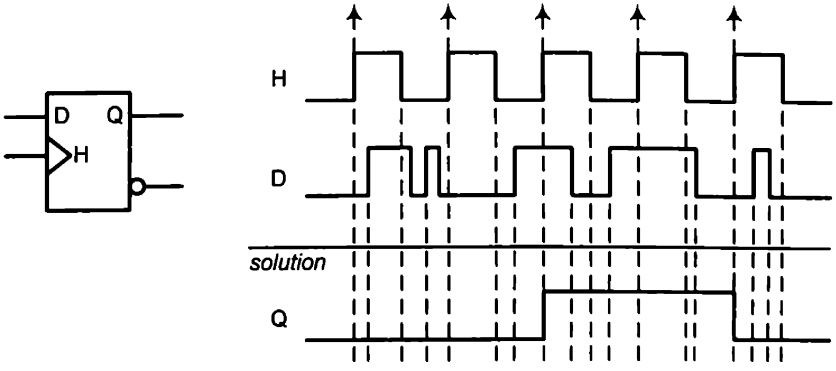


Figure 5.63

Solution : On se rappelle que, pour un flip-flop D à déclenchement sur front montant, la sortie Q prend la valeur de l'entrée D à la transition de l'horloge H (⌐¬, indiquée par des flèches), et qu'aux autres instants, on a mémorisation. Partant de cela, le chronogramme se bâtit de proche en proche tel qu'il a été indiqué. On remarque que la courbe obtenue est très différente de celle tracée pour le latch D. Il ne faut donc pas confondre les deux circuits.

5.5. Soit le circuit suivant soumis aux entrées *H*, *J* et *K* fournies. Quel sera le chronogramme correspondant à la sortie *Q* ?

Note : On assume qu'on a un flip-flop J-K à déclenchement sur front montant et qu'au début, $Q = 1$. Négliger les temps de propagation.

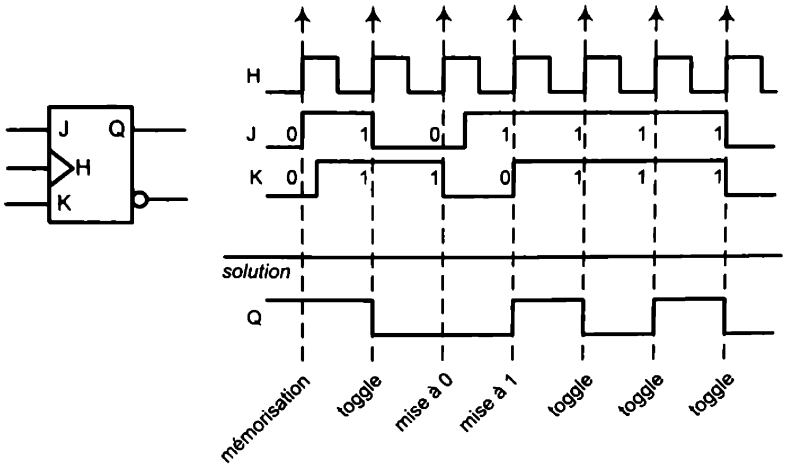


Figure 5.64

Solution : On se rappelle que, pour un flip-flop J-K à déclenchement sur front montant, la sortie *Q* ne changera qu'à la transition de l'horloge *H* (⎍, indiquée par des flèches) selon les valeurs alors présentes aux entrées *J* et *K*, et qu'aux autres instants, on a mémorisation. Partant de cela, le chronogramme se bâtit de proche en proche tel qu'il a été indiqué. À titre indicatif, on a ajouté les valeurs de *J* et de *K*, et aussi le comportement du flip-flop.

EXERCICES

5.1. a) Nommer un problème essentiel des latches R-S.

b) Nommer une raison justifiant la très grande popularité des flip-flops D.

c) Nommer une différence essentielle entre un latch et un flip-flop.

d) Pourquoi, selon vous, n'y a-t-il pas de flip-flops T dans la famille TTL 74xxx ?

5.2. Soit le circuit suivant :

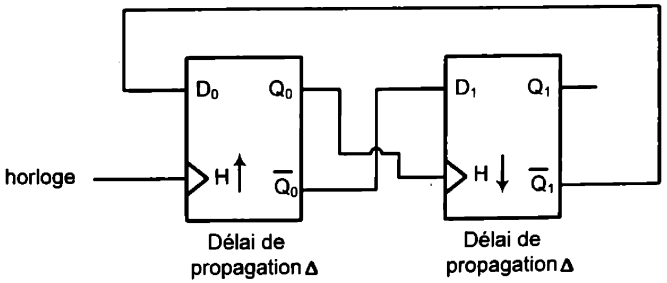


Figure 5.65

a) Nommer un problème de fonctionnement relié à ce circuit.

b) Nommer deux arguments qui font de ce circuit un « mauvais design ».

c) Établir le diagramme temporel complet de tous les états.

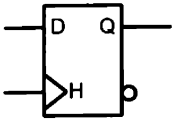
d) Que se passera-t-il si les délais de propagation (identiques pour les deux bascules) sont $\Delta = 4ns$ et que la période d'horloge (signal *ck*) est de $10ns$ avec un $t_{set\text{-}up} = 2ns$ (identique pour les deux bascules) ?

5.3. Fournir le signal de sortie Q pour le circuit suivant en assumant (H: signal d'horloge) :

a) qu'il s'agit d'un latch D ;

b) qu'il s'agit d'un flip-flop D à déclenchement sur front montant ;

c) qu'il s'agit d'un flip-flop D à déclenchement sur front descendant.



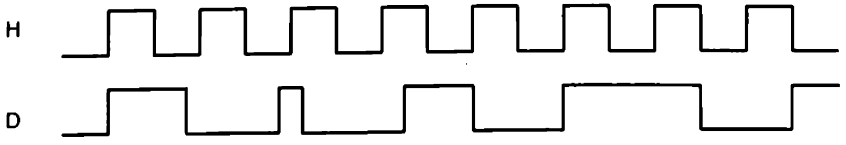


Figure 5.66

5.4. Fournir le signal de sortie Q pour le circuit suivant en assumant (H: signal d'horloge) :

a) qu'il s'agit d'un flip-flop T à déclenchement sur front montant ;

b) qu'il s'agit d'un flip-flop T à déclenchement sur front descendant.

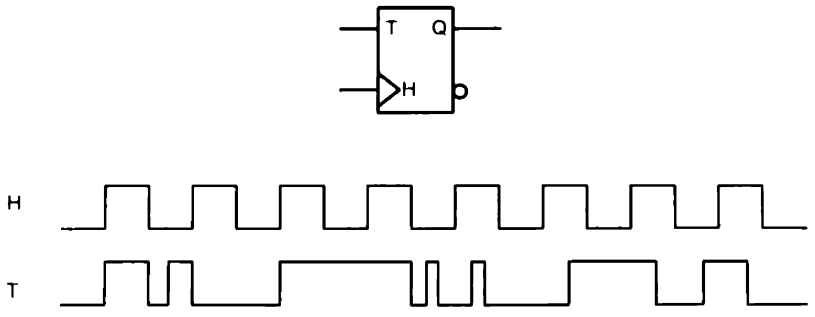


Figure 5.67

5.5. Fournir le signal de sortie Q pour le circuit suivant en assumant (H : signal d'horloge) :

a) qu'il s'agit d'un flip-flop J-K à déclenchement sur front montant ;

b) il s'agit d'un flip-flop J-K à déclenchement sur front descendant.

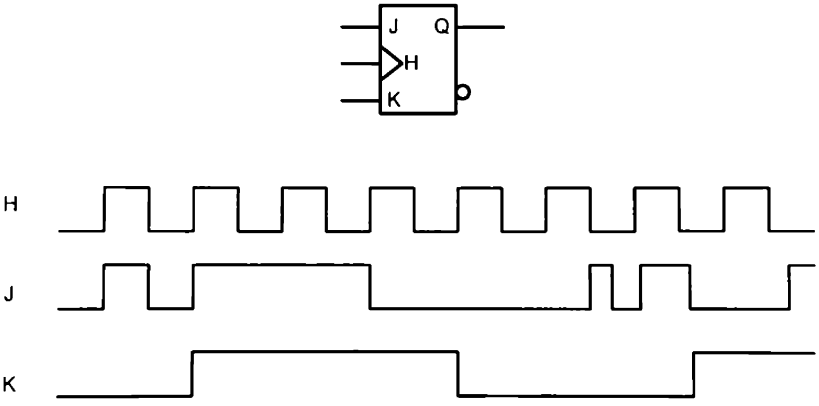


Figure 5.68

6
Séquenceurs synchrones

QUEL DÉFI[1] !

Dans les années soixante, une équipe en génie électrique et en génie informatique de *l'Instrumentation Lab* du MIT (*Massachusetts Institute of Technology*) sous la supervision des Drs Stark Draper (partie matérielle), Peter Adler et Don Eyles (partie logicielle), fut chargée par la NASA de bâtir l'ordinateur pour la commande du module lunaire Apollo de la première mission humaine sur la Lune. En cette période de guerre froide, la NASA avait opté pour une commande locale plutôt que pour une radiocommande, de peur que les Soviets ne brouillent les signaux radio, entraînant la mission à la catastrophe. En fait, deux ordinateurs furent embarqués, un dans le *Command Module* et l'autre dans le *Lunar Module*. Le défi apparaît aujourd'hui énorme puisqu'il s'agissait de commander un vaisseau spatial de 13 tonnes voyageant à plus de 3500 km/h avec pour objectif un alunissage en douceur, un rendez-vous avec le véhicule lunaire orbital et un retour sécuritaire sur Terre ; le tout en fonction de la technologie de l'époque basée sur des circuits intégrés SSI et MSI, des mémoires à tores de ferrite.

Le résultat final est impressionnant : un poids total de 30 kg seulement, 5 000 circuits intégrés, 36 864 mots de mémoire ROM et 2 048 mots de mémoire RAM, le tout sur 15 bits, pas de disque dur. Pour employer l'ordinateur, les astronautes avaient à leur disposition une unité d'affichage et un clavier appelé DSKY (prononcer « disky »). Le clavier n'était pas de type QWERTY, mais contenait des touches spéciales combinées à un clavier numérique. Pour entrer une commande, il fallait presser le bouton « VERB » suivi du numéro correspondant à l'action désirée. Pour entrer une donnée, on pressait une touche nommée « NOUN » suivie d'un numéro. Cela semble simpliste, mais était très efficace. Le système d'opération, bien que primitif, supportait des actions temps réel sous la forme d'une liste de tâches prioritaires fonctionnant par interruption : la tâche la plus prioritaire, appelée l'exécutive (*executive* en anglais), interrompant les moins prioritaires de la « liste d'attente » (*waiting list* en anglais). Toutes les 20 ms, l'affectation de la liste d'attente était revue. Chaque tâche exécutée avait accès à une portion de la mémoire lors de son exécution. Avec si peu de mémoire, les adresses de mémoire pouvaient avoir

---

1 Sources : NASA SP ; 350. *Apollo Expeditions to the Moon*, Washington, DC, Edgar M. Cortright, 1975. Voir aussi: http://www.abc.net.au/science/moon/computer.htm,
http://science.ksc.nasa.gov/history/apollo/apollo-11/apollo-11.html et
http://nssdc.gsfc.nasa.gov/imgcat/html/mission_page/

jusqu'à sept affectations particulières selon le moment de l'exécution. Par exemple, une adresse pouvait représenter l'altitude au-dessus de la Lune pendant l'alunissage, mais représenter le pointage sextant sur une étoile pendant l'approche vers la Lune. Une telle approche rendit les tests de l'ordinateur plutôt fastidieux. Le système avait aussi été prévu pour conserver ses variables critiques intactes en cas de réinitialisation générale (en anglais : *reboot*). Ceci permit d'ailleurs de sauver la mission du 20 juillet 1969.

En effet, cette journée là, Neil Armstrong et Edwin Aldrin était à environ 2 000 mètres au-dessus de la surface lunaire lorsque le témoin lumineux de l'erreur 1202 s'illumina. Il restait 12 minutes à la descente. L'erreur 1202 signifiait une surcharge de la mémoire. Immédiatement, les astronautes appelèrent le Centre de commandement terrestre. La surcharge de la mémoire venait d'un flux inattendu de données provenant du radar de pointage. Comme l'ordinateur avait mis cette tâche en liste d'attente, les données s'accumulaient. La responsabilité de décider si cette erreur était cruciale incomba à Steve Bales (26 ans), l'expert en système de guidage lunaire au Centre de commandement terrestre. Un confrère de travail, Jack Garman, se rappela qu'une erreur similaire avait été notée comme bénigne lors de tests de simulation réalisés une semaine auparavant. Cela poussa Bales à donner son accord, 15 interminables secondes plus tard, pour la poursuite de la mission après une réinitialisation de l'ordinateur. En fait, l'ordinateur dut être réinitialisé plusieurs fois.

Quelques minutes plus tard, le LEM (*Lunar Excursion Module*) se posa sur la Lune, ce qui permit à Neil Armstrong de fouler le sol lunaire et d'écrire une page d'histoire. Bales reçut par la suite la médaille américaine de la Liberté (en anglais : *US Medal of Freedom*) tout comme les trois astronautes de la mission Apollo.

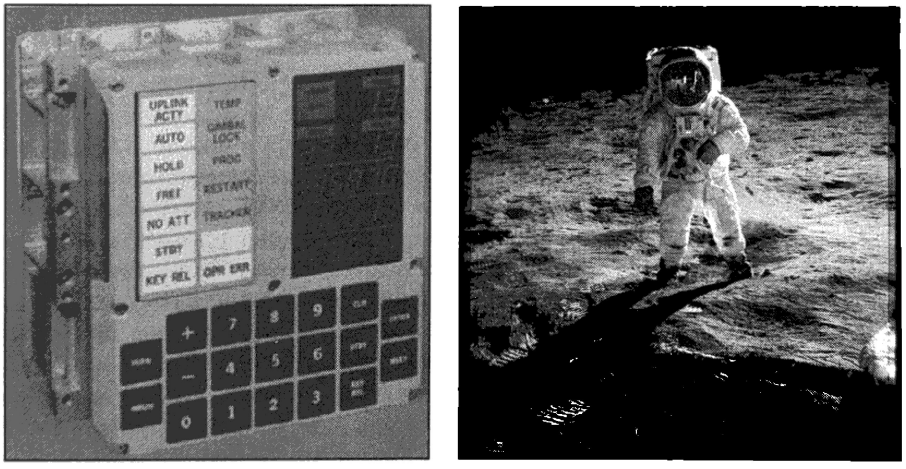


Figure 6.1 À gauche : photo de l'unité d'affichage et du clavier de l'ordinateur équipant les missions Apollo vers la Lune. À droite : le commandant d'Apollo 11, Neil Armstrong, prit cette photo du pilote du module lunaire, Edwin Aldrin, pendant leur promenade lunaire de 2.5 heures. Sur la visière d'Aldrin, on peut apercevoir Armstrong et une partie du module lunaire (courtoisie de la Nasa, document Apollo 11, AS11-40-5903).

6.1 INTRODUCTION

Avec les chapitres précédents, nous sommes maintenant familières avec toutes les notions qui nous permettent d'aborder une classe très importante de circuits numériques : les séquenceurs synchrones simples ou compteurs sans entrée. De tels circuits sont aussi connus sous le nom de machines de Moore. Une *machine de Moore*[2] est un circuit numérique ou un système qui passe à travers une série finie d'états grâce à une horloge. Outre son horloge, un séquenceur simple ne possède aucune autre entrée (sauf éventuellement des entrées d'initialisation : mise à 0 ou à 1 de certains bits).

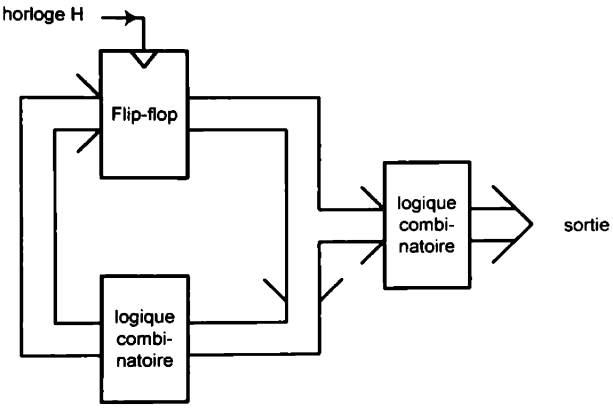


Figure 6.2 Schéma général d'une machine de Moore

Un exemple d'un tel circuit est le séquenceur simple d'un feu de circulation. Dans un tel cas, les différents cycles (vert, jaune, rouge) se succèdent d'une façon immuable, sans relation avec la quantité de trafic présent. De tels séquenceurs simples de feux de circulation ont longtemps été la norme. Aujourd'hui, avec l'accroissement du trafic routier et la disponibilité de circuits électroniques à bas prix, de boucles de détection de véhicules et aussi de caméras numériques, des systèmes plus complexes sont souvent mis en place. On ne parle plus alors de séquenceurs, mais plutôt de *machines à nombre fini d'états* qui, elles, possèdent des entrées externes. Pour ces *machines à nombre fini d'états*, la valeur des entrées externes va affecter l'ordre des états dans lequel passera le circuit. Ces machines seront l'objet du chapitre suivant.

On se rappellera aussi la distinction entre les circuits séquentiels et les circuits synchrones. Dans les deux cas, les éléments de mémoire seront des latches et des flip-flops. Toutefois, dans un circuit synchrone, la même horloge est commune à tous les éléments du circuit, alors que, dans un circuit séquentiel, les horloges peuvent différer dans le circuit, et il peut y avoir aussi des éléments asynchrones. Dans ce chapitre (et le

---

[2] D'après Edward F. Moore , *Gedanken Experiments on Sequential Machines*, paru dans C. E. Shannon et John McCarthy (éd.), *Automata Studies*, Princeton University Press, Princeton, New Jersey, 1956. Prof. Edward F. Moore (23 nov. 1925 - 14 juin 2003). Il fut notamment chercheur au Laboratoires Bell de Chatham (New Jersey) à la fin des années 50 et professeur à l'Université du Wisconsin à Madison (1966-1985), voir: http://www.secfac.wisc.edu/senate/20030929/1727(mem_res).pdf.

suivant), nous étudierons les circuits synchrones ; nous parlerons donc de compteurs synchrones et de séquenceurs synchrones.

Les séquenceurs et compteurs synchrones sont des circuits sans entrée externe (hormis l'horloge) et sont constitués de portes logiques et de flip-flops. À noter cependant qu'un séquenceur ou un compteur synchrone peut néanmoins avoir des entrées statutaires, par exemple les entrées de mise à 1 (*set* en anglais) ou de mise à 0 (*reset* en anglais) qui sont parfois disponibles sur certains flip-flops (section 5.13). Puisque ces entrées ne sont pas synchrones (elles agissent immédiatement, peu importe l'état de l'horloge), elles ne servent généralement que pour l'initialisation du circuit.

La distinction entre un compteur synchrone et un séquenceur synchrone vient de la valeur binaire de la série parcourue. Pour un compteur, la séquence parcourue est régulière (non « brisée »), i.e. que la valeur binaire des états parcourus s'incrémente de +1 ou se décrémente de −1 à chaque coup d'horloge ce qui n'est pas le cas pour un séquenceur synchrone pour lequel la valeur binaire des états parcourus est quelconque. À strictement parler, le terme *séquenceur synchrone* est donc plus général et englobe le terme *compteur synchrone*. Il est à noter que certains auteurs ne font pas la distinction entre les compteurs synchrones et les séquenceurs synchrones.

Au terme de ce chapitre, nous serons en mesure d'analyser, mais aussi de concevoir de tels circuits numériques. Seront aussi couverts dans ce chapitre les registres synchrones comme les registres à décalage cycliques par exemple.

6.2 ÉTATS ET SORTIE

Il convient de se rappeler la notion d'état. À tout moment, l'état d'un circuit séquentiel quelconque correspond aux valeurs mémorisées par les bascules (ou les flip-flops) de ce circuit. Si, par exemple, ce circuit contient trois flip-flops, il s'agira des valeurs mémorisées par ces flip-flops au moment où l'on observe le circuit, par exemple : « 1, 0, 1 ». De façon générale, si on a N flip-flops, il y aura au maximum 2^N états différents. Nous emploierons essentiellement des flip-flops qui, comme nous l'avons vu, sont sensibles au front (ou coup) d'horloge, alors que les bascules (ou latches) sont sensibles aux niveaux de l'horloge. Il est à noter que l'état du circuit n'est pas toujours égal à la sortie des flip-flops, par exemple il peut y avoir de la logique combinatoire en sortie, tel qu'il a été indiqué sur le schéma général de la machine de Moore fournit plus haut. Si des flip-flops ne sont pas directement reliés à la sortie, ils sont dits cachés (en anglais : *hidden*).

Dans le cas des séquenceurs, l'analyse et le design se feront à l'aide d'une *table État Présent → État Suivant* (en abrégé, *table ÉP-ÉS*, ou en anglais, *Present State → Next State PS-NS table*). L'état présent fait référence à l'état du circuit avant le coup d'horloge, alors que l'état suivant fait référence à l'état du circuit après le coup d'horloge. D'ailleurs, pour déterminer l'état suivant d'un circuit séquentiel contenant des flip-flops et ou des bascules, il faut connaître trois choses :

- Le schéma du circuit ;
- Les entrées présentes et aussi ;
- L'état présent du circuit.

Si on veut réaliser une machine de Moore à M états, c'est-à-dire ayant un cycle de M états avec N flip-flops, alors il faudra qu'au minimum la relation suivante soit respectée :

$$2^N \geq M$$

À noter, parfois, il peut être judicieux de mettre plus de flip-flops que le minimum requis par la relation précédente pour faciliter le décodage de sortie. Quant au nombre maximal de flip-flops, il correspond au nombre d'états dans le cycle, donc M. Concevoir le circuit avec plus de M flip-flops compliquerait inutilement le design. Nous reviendrons là-dessus plus tard.

Exemple : Si le cycle désiré pour réaliser une machine de Moore comprend six états, quel est le nombre minimal de flip-flops à prévoir ?

Solution : Le nombre minimal de flip-flops à prévoir pour la réalisation de cette machine sera 3 puisque $2^3 \geq 6$. Deux flip-flops ne permettraient que $2^2 = 4$ états seulement. Ce nombre est un minimum ; on pourrait en fait décider d'avoir un flip-flop par état et concevoir le circuit avec six flip-flops, c'est le nombre maximal.

Lors de la conception d'un circuit, il est judicieux de nommer chacun des états du cycle, ce qui rend le travail de la designer plus aisé. Par exemple, pour un séquenceur simple, on pourrait avoir ceci : ATTENTE, ACCEPTE, DÉMARRE LE MOTEUR, etc. Un choix judicieux des noms des états et des variables facilite à coup sûr le design et sa vérification ultérieure, comme nous le verrons plus tard.

Exemple : Soit le circuit de registre à décalage FIFO suivant. On demande de spécifier s'il y a des flip-flops cachés et aussi de donner l'état du circuit après trois coups d'horloge si l'entrée a été mise à 1 au départ pendant une période d'horloge et que tous les flip-flops étaient initialement au repos.

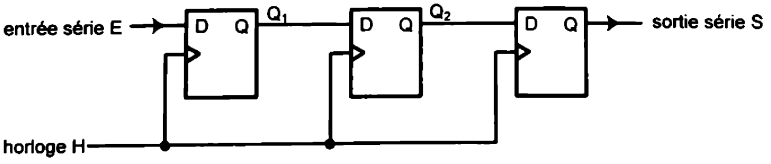


Figure 6.3

Solution : On note que le circuit comprend trois flips-flops dont deux sont cachés puisque seulement le dernier est relié à la sortie. Suivant l'énoncé de la question, on peut tracer le chronogramme suivant :

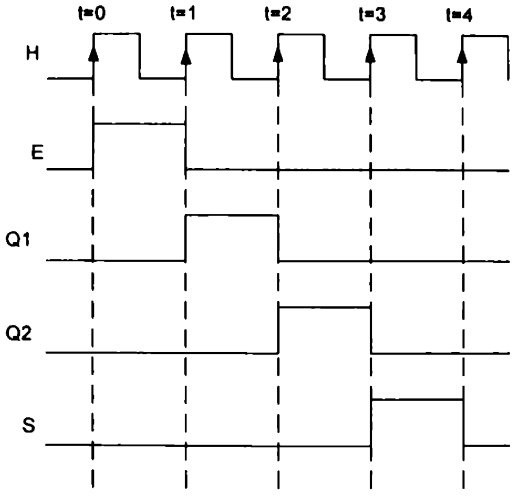


Figure 6.4

Sur ce chronogramme, après trois coups d'horloge, donc entre t = 2 et t = 3, on voit que l'état du circuit défini par [*S*, *Q*2, *Q*1] vaut : [0, 1, 0].

6.3 CIRCUITS SYNCHRONES : CONSIDÉRATIONS À PROPOS DE L'HORLOGE

Dans les circuits synchrones, l'horloge joue un rôle fondamental puisqu'elle en régit tout le fonctionnement. Il y a donc quelques règles simples qu'il faut s'assurer de respecter pour éviter tout problème.

Ainsi, les règles suivantes doivent être respectées dans un circuit synchrone :

- **On n'emploie qu'une seule fréquence d'horloge pour tous les flip-flops et tous les flip-flops sont du même type**, soit à déclenchement sur front montant (↑) ou descendant (↓), mais pas de mélange. Il est à noter qu'il est possible de concevoir des circuits qui fonctionnent avec des horloges à plusieurs phases, ceci déborde cependant du cadre de notre discussion[3]. À cause des avantages de synchronisme, plus de 90 % des circuits sont synchrones et emploient une seule horloge. Un avantage est par exemple l'élimination des courses et aussi de la contamination (*catching*) (discuté au chapitre 5) ;
- **L'horloge ne doit pas être retardée par des portes logiques**. En effet, cela causerait des retards sur l'arrivée des coups d'horloge et la possibilité d'aléas ;
- **L'horloge doit être stable**. Il faut s'assurer que le signal d'horloge ne soit pas dégradé. À cette fin, on pourra employer un circuit basé sur la résonance d'un cristal

[3] Un avantage, c'est que plus de travail peut être accompli par le circuit, par exemple lors des montées et des descentes d'horloge, etc.

de quartz, ou encore un générateur de fonction. Il est à noter que les circuits oscillants basés sur la charge et la décharge d'un condensateur dans une résistance ont une fréquence qui varie beaucoup avec la température (jusqu'à 20 %), ce qui n'est évidemment pas l'idéal ;

- **Les sorties dans le circuit seront alors stables** et ne changeront qu'au coup d'horloge.

FREQUENCE MAXIMALE DE L'HORLOGE

Pour les circuits synchrones, on a la topologie suivante :

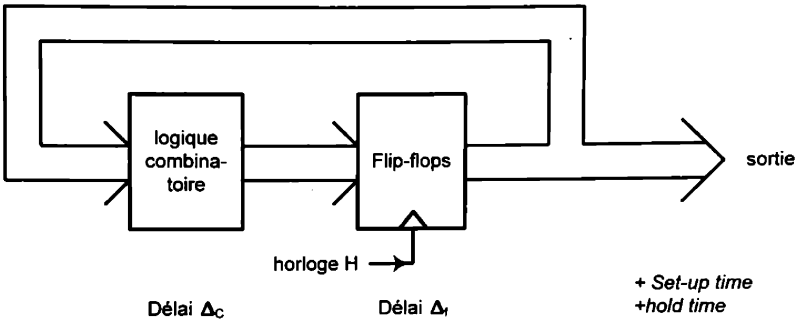


Figure 6.5

Il est intéressant alors de se demander quelle sera la fréquence maximale de l'horloge. Selon ce schéma, on note que la période minimale t_{MIN} est donnée par :

$$t_{MIN} = \Delta_c + \Delta_f + t_{setup} + t_{hold}$$

et la fréquence d'horloge maximale f_{MAX} est alors donnée par :

$$f_{MAX} = \frac{1}{t_{MIN}}$$

Exemple : Quelle est la fréquence maximale approximative pour un circuit contenant des flip-flops 74F74 discutés au chapitre 5. Assumer que la logique combinatoire employée possède un délai de propagation Δc de 5 *ns*.

Solution : Pour le 74F74 de Philips, dont les spécifications sont fournies au chapitre 5, on a les spécifications suivantes (moyenne des cas *HL* ou *LH* et *H* ou *L*) :

$$\Delta_f = t_p = 5.4ns, t_{su} = 2.5ns, t_h = 1ns$$

Selon la relation précédente, on a donc :

$$t_{MIN} = \Delta_c + \Delta_f + t_{setup} + t_{hold} = 5 + 5.4 + 2.5 + 1 = 13.9ns$$

et alors : $f_{MAX} = \frac{1}{t_{MIN}} = \frac{1}{13.9\ \text{ns}} = 72\ \text{MHz}$

Ce chiffre donne une bonne idée des performances maximales du circuit.

DISTRIBUTION MAXIMALE DU SIGNAL D'HORLOGE

Dans un circuit synchrone, puisque l'horloge est distribuée à tous les flip-flops d'un circuit, il s'agit de la capacité du signal d'horloge d'alimenter un nombre donné de flip-flops. Au-delà de ce nombre, le signal d'horloge sera dégradé à cause de la trop grande charge alors imposée. L'expression anglaise *fan-out* (ou *ventilation*) est très imagée, puisqu'on peut s'imaginer l'horloge comme un ventilateur (*fan* en anglais) dont l'air soufflé rafraîchit un nombre donné de circuits (à l'extérieur, *out* en anglais). Le *fan-out* est donc le courant maximal disponible en sortie. L'expression corollaire au *fan-out* est le *fan-in*, c'est-à-dire le courant requis à l'entrée d'un circuit pour que le signal qu'on y injecte soit valide.

Exemple : Dans un circuit synchrone, on demande de calculer le *fan-out* d'un signal d'horloge si le circuit contient 100 flip-flops ayant chacun un *fan-in* de 1 µA.

Solution : On a que : 100 x 1 µ A donne un total de 0.1 mA. En pratique, ce sera un peu plus puisqu'une bonne pratique en ingénierie consiste toujours à rajouter un facteur de sécurité, disons de 15 %, pour, par exemple, prévoir une consommation plus grande suite à une élévation de température ou à une certaine extension du circuit. Dans ce cas, le *fan-out* serait donc plutôt autour de 0.12 mA.

Une question importante se pose alors dans le cas de circuits synchrones de grande ampleur : comment distribuer un signal d'horloge de qualité partout ? Plusieurs solutions sont envisageables :

1. On peut employer un circuit spécial d'horloge qui fournira suffisamment de courant de telle sorte que le *fan-out* sera respecté.

2. On peut augmenter le *fan-out* à l'aide de portes tampons à grande capacité de courant. Les tampons inclus dans le circuit 74LS14 sont spécialement conçus à cette fin. Chacun des six tampons a un *fan-out* de 1 mA /20 mA (valeurs typiques minimales et maximales). À titre d'exemple, le *fan-in* d'un flip-flop 74F74 est de 20 µA /0.6 mA (valeurs typiques minimales et maximales), un 74LS14 employé selon la configuration proposée pourra donc fournir un signal d'horloge de qualité entre 2 et 1000 flip-flops (cas extrêmes selon les valeurs typiques minimales et maximales).

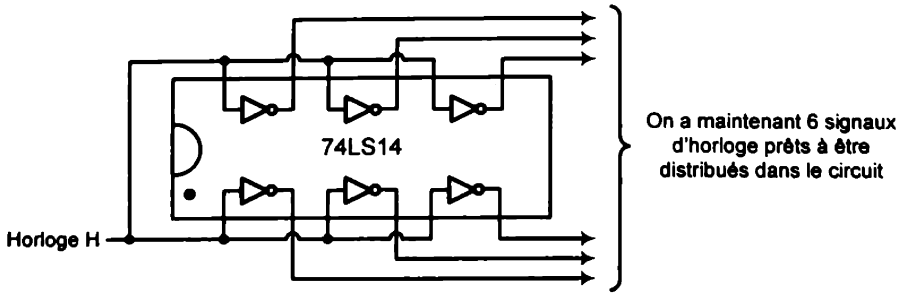


Figure 6.6

Puisque le signal d'entrée *H* n'est employé nulle part ailleurs dans le circuit, l'inversion n'affecte pas le système (on pourrait aussi employer des tampons non inverseurs). Les propriétés des tampons 74LS14 permettent de maintenir une montée rapide du signal d'horloge qui n'est pas déformée. On peut raisonnablement assumer que les six signaux d'horloge qui sont prêts à être distribués sont identiques et en phase[4] puisque les délais de propagation dans les tampons sont similaires et subissent les mêmes variations de température, les tampons étant encapsulés ensemble. L'utilisation du signal *H* (celui qui est injecté au 74LS14) est à proscrire partout ailleurs dans le circuit puisqu'il y a un délai de propagation entre ce signal et les six signaux de sortie ; ce délai pourrait entraîner des aléas aux conséquences nuisibles.

3. Une troisième possibilité consiste à passer le signal d'horloge dans un tampon *Schmitt trigger* avant sa distribution dans le circuit. Il faut savoir qu'un tampon *Schmitt trigger* transforme un signal d'entrée qui change lentement en un signal aux transitions franches (donc avec des montées à 1 et des descentes à 0 rapides grâce à l'hystérésis qui est employée et qui offre ainsi une très bonne immunité au bruit présent dans le signal d'entrée tel qu'il a été indiqué sur la figure 6.8.

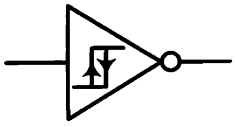


Figure 6.7 Symbole du tampon inverseur « Schmitt trigger » ; à noter l'hystérésis

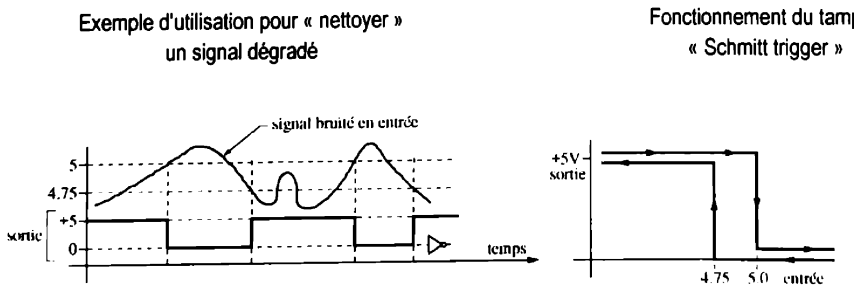


Figure 6.8

Sur la figure 6.8, à droite, les deux courbes en deçà de 4.75 V et au-delà de 5.0 sont normalement confondues ; on les a séparées par souci de clarté de l'explication. Cette figure est adaptée d'une figure du livre *The Art of Electronics*[5] par Paul Horowitz et Winfield Hill, 3e édition, Cambridge University Press, 2015. Cet ouvrage est un véritable

---

[4] C'est-à-dire sans pratiquement de délai l'un par rapport à l'autre (ce sont des signaux *plésiochrones*). Si un déphasage (donc un délai important) existe à différents endroits du circuit pour le signal d'horloge, on dit alors que l'horloge n'est pas synchronisée dans le circuit (en anglais : « *skewed clock* »).

[5] Site Web du livre : http://www.artofelectronics.net/

classique et est recommandé à toutes celles qui s'intéressent à l'électronique (analogique et numérique).

6.4 ÉTAPES D'ANALYSE ET DE DESIGN POUR LES CIRCUITS SYNCHRONES SANS ENTRÉES

Cette section sera brève, car on ne fera que la présentation des étapes d'analyse et de design des circuits que nous allons étudier dans ce chapitre. Ces tableaux seront expliqués dans les sections subséquentes. On les a tous regroupés en début de chapitre aux fins de références pratiques.

6.4A – Étapes d'analyse : circuit synchrone avec flip-flops, sans entrées

1. Séparer les flip-flops et la logique combinatoire de stimulation ;
2. Étudier la logique de stimulation et obtenir une équation booléenne pour chacune des entrées des flip-flops ;
3. Assumer qu'au début, le circuit est dans un état 0, donc mettre « 0 » dans les équations booléennes ;
4. Obtenir les nouvelles valeurs de sortie et les mettre dans les équations pour déterminer l'état suivant ;
5. Reprendre l'étape 4 jusqu'à ce que tous les états suivants aient été déterminés en dressant la table État Présent → État Suivant (ÉP-ÉS), en anglais : table PS-NS (*Present State → Next State*). Indiquer les états dans des tables de Karnaugh ou dans un diagramme en anneaux.

Note importante (pour les registres à décalage – RDC – avec décalage à droite) : Dans l'écriture des tables État Présent → État suivant, *on indiquera le LSB à gauche et le MSB à droite* puisque cet ordre correspond à la progression des bits dans le circuit (c'est-à-dire au décalage des bits).

6.4B – Étapes de design : circuit synchrone avec flip-flops, sans entrées

1. Exprimer chaque nombre de la séquence en binaire. Chaque nombre correspondra à un état ;
2. Déterminer une méthode pour assigner les états aux sorties des flip-flops ;
3. Choisir le type de flip-flop : T, J-K, D (D par défaut, car c'est plus facile, section 5.13) ;
4. Établir la table État Présent → État Suivant (ÉP-ÉS) ;
5. Employer les états présents comme entrées dans les tables de Karnaugh ;

6.1 Pour l'*autocorrection simple* : compléter les tables de Karnaugh en assignant l'*état de retour* aux états inemployés (en anglais : *ground state*) ;

6.2 Pour l'*autocorrection optimisée* : compléter les tables de Karnaugh en assignant des *X-temporaires* (X : valeurs «sans importance») aux états inemployés, optimiser les regroupements possibles dans les tables, vérifier que les simplifications obtenues

ramènent les états inemployés dans la séquence désirée, si ce n'est pas le cas modifier la valeur des *X-temporaires* en conséquence ;

7. Trouver les équations à partir des tables de Karnaugh des points 5 et 6 ;
8. Monter les sorties du circuit comme combinaison des sorties des flip-flops ;
9. Réaliser la logique combinatoire de stimulation à partir des équations obtenues au point 7.

6.4C – Étapes de design : registres à décalage cyclique RDC (*cyclic shift register CSR*)

1. Établir la liste successive des états dans la table État-Présent → État Suivant (ÉP-ÉS) ;
2. Ajouter le ou les cas d'initialisation pour le démarrage automatique (exemple 0000 0000, sur huit bits) et aussi inclure une phase d'autocorrection ;
3. Synthétiser l'entrée $D_{entrée}$ comme la somme des mintermes pour lesquels $D_{entrée} = 1$ dans la liste successive des états de la table ÉP-ÉS dressée aux étapes précédentes.

Note importante (pour les registres à décalage – RDC – avec décalage à droite) : Dans l'écriture des tables État Présent → État suivant pour les RDC, *on indiquera le LSB à gauche et le MSB à droite* puisque cet ordre correspond à la progression des bits dans le circuit (c'est-à-dire au décalage des bits).

On reprend finalement la table[6] (résumé de la section 5.13) de sorte que tout soit disponible simultanément dans cette section pour référence ultérieure :

Q_N		Q_{N+1}	Flip-flop *D*	Flip-flop Toggle (entrée d'activation)	Flip-flop *J-K*			
					J	*K*	*J*	*K*
0	→	0	0	0	0 0	1 0	0	*X*
0	→	1	1	1	1 1	0 1	1	*X*
1	→	0	0	1	0 1	1 1	*X*	1
1	→	1	1	0	0 1	0 0	*X*	0

← ANALYSE
DESIGN →

Figure 6.9

[6] Adaptée de Daniels, Jerry D. Digital Design from Zero to One, 1re édition, figure 6.10, p. 313.

6.5 ANALYSE DE CIRCUITS SYNCHRONES SANS ENTRÉES

Dans cette section, nous allons analyser deux circuits synchrones sans entrées (avec des flip-flops T et J-K). Nous utiliserons la table 6.4A. Nous avons choisi d'employer la méthode « plonger dans la piscine pour apprendre à nager ». En effet, fortes des notions vues jusqu'à présent, nous allons maintenant apprendre par l'exemple.

Exemple 1 : Soit le circuit suivant Q_C étant le MSB. On demande de fournir le diagramme d'état correspondant.

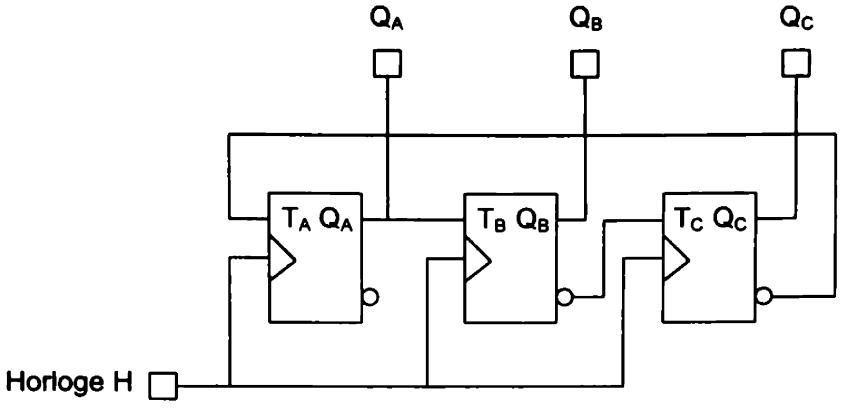


Figure 6.10

Étape 1 On note que la logique de stimulation est bien en évidence. Dans ce problème, trois flip-flops T sont employés, et la logique combinatoire de stimulation est simplement constituée des sorties des flip-flops. Les sorties du circuit sont directement les sorties des flip-flops.

Étape 2 Ce qui permet d'écrire les équations suivantes :

$$T_A = \overline{Q}_C$$

$$T_B = Q_A$$

$$T_C = \overline{Q}_B$$

Étape 3 On assume qu'au départ $Q_C = Q_B = Q_A = 0$, alors, selon les équations précédentes, on a : $T_C = 1$, $T_B = 0$, $T_A = 1$, et donc, au coup d'horloge suivant, se rappelant le fonctionnement d'un flip-flop T (voir la table de la section 5.13 reprise au début du chapitre), on obtiendra $Q_C = 1$, $Q_B = 0$, $Q_A = 1$. À propos du flip-flop T, la sortie (après le coup d'horloge) reste inchangée si $T = 0$ (avant le coup d'horloge), et est inversée par rapport à la valeur présente avant le coup d'horloge si $T = 1$ (principe du toggle).

Étape 4 Ces valeurs peuvent être insérées dans la table État Présent → État Suivant (ÉP-ÉS). Elles constituent la première ligne de rang ⓪, qui donne à l'état suivant (ou

futur) le rang ⑤. La seconde ligne de la table repart de là, donc du rang ⑤, et qui donne à l'état futur ③. Comme la ligne du rang ③ donne à l'état futur ⓪, c'est la fin de la première séquence que parcourt le circuit. Puisque nous avons trois variables, la table aura huit lignes ($2^3 = 8$) ; il faut donc tester les autres valeurs de rangs (①, ②, ④, ⑥, ⑦). L'ordre d'entrée dans la table n'a évidemment pas d'importance, ce qui compte, c'est de tester toutes les combinaisons possibles, ce qui fournira la table complète.

Étape 5 La Table ÉP–ÉS est alors la suivante :

	État Présent (ÉP)			Stimulation			État Suivant (ÉS)		
Rang	Q_C	Q_B	Q_A	T_C	T_B	T_A	Q_C	Q_B	Q_A
⓪	0	0	0	1	0	1	1	0	1
⑤	1	0	1	1	1	0	0	1	1
③	0	1	1	0	1	1	0	0	0
⑦	1	1	1	0	1	0	1	0	1
④	1	0	0	1	0	0	0	0	0
①	0	0	1	1	1	1	1	1	0
⑥	1	1	0	0	0	0	1	1	0
②	0	1	0	0	0	1	0	1	1

Figure 6.11

De cette table, on observe différentes séquences que le circuit peut parcourir :

⓪ → ⑤ → ③→ ⓪ (séquence principale puisqu'elle comprend le plus d'états et qu'elle se répète) ;

⑦ → ⑤ (séquence partielle puisqu'on retrouve la séquence principale à partir de ⑤) ;

④ → ⓪ (séquence partielle puisqu'on retrouve la séquence principale à partir de ⓪) ;

① → ⑥ (séquence cul-de-sac puisqu'on reste sur place à partir de ⑥) ;

② → ③ (séquence partielle puisqu'on retrouve la séquence principale à partir de ③).

Ceci peut aussi se dessiner sous la forme d'un diagramme d'état comme suit :

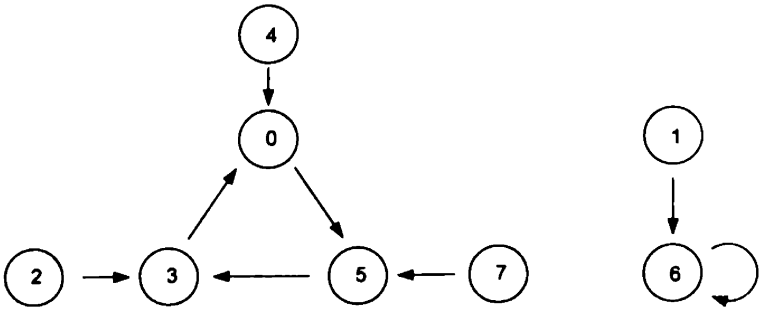


Figure 6.12

Il existe deux types de circuits séquentiels : ceux qui sont « autocorrecteurs » et ceux qui ne le sont pas. Un circuit autocorrecteur est un circuit qui « retombe sur ses pattes », c'est-à-dire qu'on retrouve la séquence principale même si celle-ci est interrompue. Par exemple, si des perturbations électromagnétiques nous conduisent à un état non désiré, on reviendra à la séquence principale après un ou quelques coups d'horloge. Dans le cas de notre exemple, si par accident on se retrouve à ⑦, un coup d'horloge plus tard, on repasse à ⑤ dans la séquence principale. Notre circuit n'est cependant pas autocorrecteur, car s'il aboutit à ① ou à ⑥, on restera coincé à l'état ⑥ qui est un cul-de-sac. De la topologie même du circuit, on constate immédiatement que l'état ④ est un état cul-de-sac puisqu'il se perpétue à chaque coup d'horloge, puisque pour l'état ⑥ on a $T_C = T_B = T_A = 0$ (les flip-flops sont en état de mémorisation).

Plus tard, nous verrons des circuits autocorrecteurs.

Exemple 2 : Soit le circuit suivant Q_B étant le MSB. On demande de fournir le diagramme d'état correspondant.

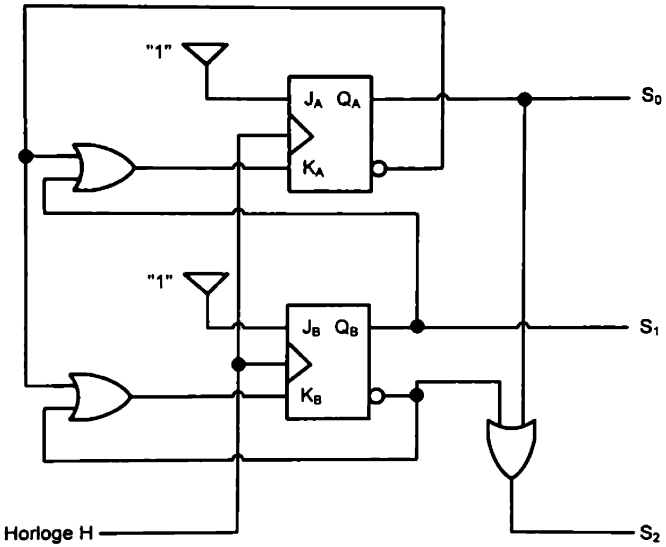


Figure 6.13

Étape 1 On note que la logique de stimulation est bien en évidence. Dans ce problème, deux flip-flops J-K sont employés, et la logique combinatoire de stimulation est constituée de portes OU. Les sorties du circuit sont les sorties des flip-flops et aussi une combinaison de celles-ci pour S_2.

Étape 2 Ce qui permet d'écrire les équations suivantes :

$$J_A = 1 \text{ et } K_A = \overline{Q}_A + Q_B$$

$$J_B = 1 \text{ et } K_B = \overline{Q}_A + \overline{Q}_B$$

Étape 3 On assume qu'au départ $Q_B = Q_A = 0$, alors, selon les équations précédentes, on a : $J_A = 1$, $K_A = 1$, $J_B = 1$, $K_B = 1$, et donc, au coup d'horloge suivant, se rappelant le fonctionnement d'un flip-flop J-K (voir la table de la section 5.13), on obtiendra $Q_B = 1$, $Q_A = 1$. À propos du flip-flop J-K, la sortie reste inchangée après le coup d'horloge si $J = K = 0$, et est inversée par rapport à la valeur présente avant le coup d'horloge si $J = K = 1$ (principe du toggle). La mise à 1 correspond à $J = 1$ et à $K = 0$. La mise à 0 correspond à $J = 0$ et à $K = 1$.

Étape 4 Ces valeurs peuvent être insérées dans la table État Présent → État suivant (ÉP–ÉS). Elles constituent la première ligne de rang ⓪, qui donne à l'état suivant (ou futur) ③. La seconde ligne de la table repart de là, donc au rang ③, et qui donne à l'état futur ②, et ainsi de suite. Puisque nous avons deux variables, la table aura quatre lignes ($2^2 = 4$) ; il faut donc tester les autres valeurs de rangs (①,②). L'ordre d'entrée dans la table n'a évidemment pas d'importance : ce qui compte, c'est de tester toutes les combinaisons possibles, ce qui donne la table complète.

Étape 5 La table ÉP→ÉS est alors la suivante :

	État Présent		Stimulation				État Suivant	
Rang	Q_B	Q_A	J_B	K_B	J_A	K_A	Q_B	Q_B
⓪	0	0	1	1	1	1	1	1
③	1	1	1	0	1	1	1	0
②	1	0	1	1	1	1	0	1
①	0	1	1	1	1	0	1	1

Figure 6.14

De cette table, on observe que le circuit parcourt une seule séquence :

① → ③ → ② (séquence principale et unique qui se répète) ;

⓪ → ③ (séquence partielle puisqu'on retrouve la séquence principale à partir de ③).

Ceci peut aussi se dessiner sous la forme d'un diagramme d'état comme suit :

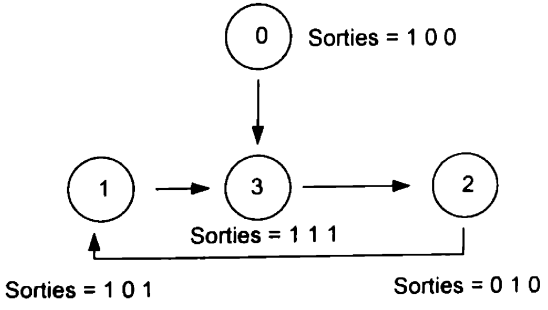


Figure 6.15

Pour ce second exemple, nous constatons que le circuit est autocorrecteur puisque, peu importe ce qu'il adviendra, nous retrouverons toujours la séquence principale. Par exemple, si par accident on se retrouve à ⑥, un coup d'horloge plus tard, on repasse à ③ dans la séquence principale.

Quant aux sorties, on a $S_0 = Q_A$, $S_1 = Q_B$, $S_2 = Q_A + \overline{Q}_B$. À l'aide de ces équations, on trouve la valeur des sorties pour chaque état :

$Q_B Q_A$	$S_2 S_1 S_0$
⓪ = 0 0	1 0 0
① = 0 1	1 0 1
② = 1 0	0 1 0
③ = 1 1	1 1 1

Figure 6.16

Ces valeurs ont été indiquées dans le diagramme d'état précédent.

À la section *Problèmes résolus* en fin de chapitre, un exemple d'analyse de circuits contenant des flip-flops D est fait. Au terme de cette section-ci, nous sommes en mesure d'analyser n'importe quel séquenceur synchrone sans entrées.

6.6 DESIGN DE SÉQUENCEURS SYNCHRONES SANS ENTRÉES

La question suivante est bien évidemment : Comment peut-on concevoir un séquenceur synchrone quelconque à partir de flip-flops T, J-K et D ? Ce sujet sera l'objet de cette section. Premièrement, il est important de réaliser que, alors que l'analyse conduit à une réponse unique (un circuit donné ne peut avoir qu'un seul diagramme d'état) l'inverse n'est pas vrai, à savoir que plusieurs circuits différents peuvent avoir le même diagramme d'état. Par exemple, on peut concevoir un circuit comme étant autocorrecteur ou pas. On peut aussi réaliser celui-ci avec des flip-flops T, D ou J-K. Chaque fois, la « solution » obtenue sera différente.

D'autres critères peuvent aussi conduire à des « solutions » différentes, par exemple la minimisation des états, la présence ou non d'un décodeur de sortie (un exemple sera d'ailleurs fait plus bas à ce sujet). Dans tous les cas, le « flair » de la designer sera un bon guide sachant toutefois que les méthodes rigoureuses que nous étudions dans ce livre fournissent déjà une excellente base.

CONSIDERATIONS PRATIQUES

Les considérations suivantes pourront nous guider dans notre démarche de conception du circuit à réaliser (des exemples seront faits pour illustrer ces cas) :

1. S'il y a M états dans la séquence, alors il y aura au moins N flip-flops avec : $2^N \geq M$.

2. Il est parfois pratique d'avoir plus de flip-flops que le minimum requis, car un nombre plus élevé peut nous permettre de réduire ou même d'éliminer complètement le décodeur de sortie ;

3. Le cas *extrême* consiste évidemment à avoir un flip-flop par état. Bien sûr, un flip-flop « coûte » plus cher en matière de circuiterie, mais les progrès de la microélectronique actuelle permettent de relativiser ce coût[7].

4.1. Autocorrection simple. Il est utile de prévoir un état de retour (*ground state* en anglais), qui est un état où l'on redirige les états inemployés dans un circuit synchrone (avec ou sans entrées), ce qui nous donne alors un circuit autocorrecteur puisque cet état de retour conduit ultimement à la séquence de travail du circuit. Le retour à la séquence principale peut prendre un ou plusieurs coups d'horloge (au minimum un) de l'état inemployé à l'état de retour. Une telle pratique est sage puisqu'en cas de défaillances quelconques (perturbations qui déstabilisent le circuit, entrées non prévues dans le cas des séquenceurs synchrones avec entrées, que nous verrons au chapitre suivant), nous sommes assurés que la séquence normale sera retrouvée après un ou quelques coups d'horloge. Par défaut, cet état de retour sera l'état « 0 » (si cet état existe dans la séquence principale), par contre cela dépend bien sûr du circuit à concevoir. Évidemment, un tel circuit autocorrecteur est plus complexe et requiert un effort supplémentaire de la part de la conceptrice : il est tellement plus commode de mettre des « X » dans les tables lors du design (nous ferons un exemple à ce sujet).

4.2. Autocorrection optimisée. Une autre approche d'autocorrection consiste d'ailleurs à :

   1. mettre des «X-temporaires» dans les tables;
   2. optimiser les regroupements dans celles-ci pour maximiser les simplification;
   3. vérifier où cela mène le circuit.
   4. Si le circuit aboutit à la séquence désirée cela est excellent, autrement on «ajuste« les «X-temporaires» pour un retour vers un des états de la séquence.

   Cette approche est bien sûr plus ardue, mais est cependant la plus optimale (en termes de minimisation de la logique de stimulation et dans la plupart des cas, on aboutira à une autocorrection à coût nul, et même souvent réduit! L'**Exemple 2** plus bas illustrera le concept d'*autocorrection optimisée*.

   Finalement, il est important de bien se rendre compte que, si l'on opte pour un circuit non autocorrecteur, il n'est pas sûr qu'on puisse revenir en cas de défaillance : y porter attention.

Tout comme à la section précédente, nous allons apprendre par l'exemple. Les exemples qui suivent font référence à la table 6.4B.

---

[7] À titre d'information, la revue *IEEE Spectrum* de novembre 2004 présente le dernier-né des microprocesseurs Intel : le Montecito à double processeur contenant 1.7 milliard de transistors.

Exemple 1 : Faire le design d'un séquenceur que répète la séquence 0, 3, 6, 9, 12, 0 et ainsi de suite.

Employer des flip-flops D ; ne pas employer de décodeur de sortie.

Solution : On note que ces spécifications ne donnent pas beaucoup de choix à la designer.

On remarque qu'on a ici cinq états : 0, 3, 6, 9, 12.

Trois flip-flops donnent $2^3 = 8$ états, ce qui serait suffisant. Par contre, ici avec quatre flip flops, le décodage de sortie n'est pas nécessaire, ce qui rend le design plus simple puisque alors les sorties des flip-flops constituent directement la sortie du circuit (ce qui est demandé dans le problème de toute façon). Appelons les quatre flip-flops : Q_3, Q_2, Q_1, Q_0.

Étapes 1 et 2 La sortie du circuit, et donc des flip-flops (puisqu'on n'a pas de décodeur de sortie), est déterminée par les spécifications de la séquence à parcourir :

Rang	Q_3	Q_2	Q_1	Q_0	
0	0	0	0	0	← On boucle en recommençant la séquence
3	0	0	1	1	
6	0	1	1	0	
9	1	0	0	1	
12	1	1	0	0	

Figure 6.17

Étapes 3 et 4 L'étape suivante consiste à choisir le type de flip-flop et à monter la table ÉP-ÉS. On emploie des flip-flops D. Ce choix est imposé ici. On peut s'en réjouir, car, comme nous le verrons, la construction de la table ÉP-ÉS est plus aisée avec ceux-ci.

Il faut ensuite dresser la table État Présent → État Suivant (ÉP-ÉS). La table est constituée de trois parties : la zone *État Présent* où l'on inscrit, pour chacun des flip-flops, la valeur binaire de l'état actuel où l'on se trouve à un moment donné. Une deuxième zone appelée *stimulation*, dont nous reparlerons dans un instant, et enfin une troisième zone : *État Suivant* qui indique, pour un état actuel donné, l'état suivant où l'on veut se trouver *après* le coup d'horloge. Par exemple, on débute la séquence à « 0000 » ; l'état suivant sera donc (selon les spécifications du circuit) : « 0011 », que l'on inscrit dans la table dans la zone *État Suivant*. Au coup d'horloge, cette valeur constituera le nouvel état présent, et le nouvel état suivant sera (selon les spécifications) : « 0110 ». La table se monte ainsi rapidement.

Reste ensuite la zone *stimulation*. Pour remplir cette zone, il faut se référer à la table 5.13 reprise en début de chapitre. Cette fois-ci, la table 5.13 sera lue selon le sens de la flèche « design ». À gauche dans cette table, toutes les transitions possibles d'un flip-flop sont indiquées. On choisit la transition qui nous convient et on prendra la valeur de stimulation du flip-flop que l'on emploie (ici, des flip-flops D selon les spécifications imposées du circuit). Par exemple, (en lisant notre table ÉP-ÉS) pour Q_0 (et Q_1 aussi) à l'état « 0000 » (rang 0), on a besoin d'une transition 0 (ÉP) → 1 (ÉS). La table 5.13 nous indique que, pour un flip-flop D, pour obtenir une transition 0 → 1, il faudra que D soit 1. On indique donc que D_0 (et D_1 aussi) = 1 à l'état « 0000 ».

Prenons maintenant Q_2 (et de même pour Q_3) à l'état « 0000 » (rang 0) ; on a besoin d'une transition 0 (ÉP) → 0 (ÉS). La table 5.13 nous indique que, pour un flip-flop D, pour obtenir une transition 0 → 0, il faudra que D soit 0. On indique donc que D_2 (et D_3 aussi) = 0 à l'état « 0000 ». Le reste de la table se construit de la même manière. On note que, pour un flip-flop D, la valeur à spécifier pour l'entrée D est toujours égale à la valeur que l'on désire à l'état suivant ; la table ÉP-ÉS est donc *très facile* à remplir : un avantage du recours au flip-flop D.

Rang	État Présent (ÉP)					Stimulation					État Suivant (ÉS)			
	Q_3	Q_2	Q_1	Q_0		D_3	D_2	D_1	D_0		Q_3	Q_2	Q_1	Q_0
0	0	0	0	0	→	0	0	1	1	→	0	0	1	1
3	0	0	1	1	→	0	1	1	0	→	0	1	1	0
6	0	1	1	0	→	1	0	0	1	→	1	0	0	1
9	1	0	0	1	→	1	1	0	0	→	1	1	0	0
12	1	1	0	0	→	0	0	0	0	→	0	0	0	0

Figure 6.18

Étapes 5 et 6 Il faut ensuite construire les tables de Karnaugh pour les entrées D des quatre flip-flops. Les variables sont bien sûr les sorties des flip-flops (qui donnent la valeur de l'état présent). Ces tables s'obtiennent directement de la table ÉP-ÉS de l'étape précédente. Par exemple, pour D_3, il nous faut :

0 (rang 0, en position 0 dans la table de Karnaugh de D_3) ;

0 (rang 3, en position 3 dans la table de Karnaugh de D_3) ;

1 (rang 6, en position 6 dans la table de Karnaugh de D_3) ;

1 (rang 9, en position 9 dans la table de Karnaugh de D_3) ;

0 (rang 12, en position 12 dans la table de Karnaugh de D_3).

Il est sage de laisser des 0 dans les tables de Karnaugh pour avoir un *état de retour* connu *ground state* à l'état 0 tel qu'il a été discuté précédemment (autocorrection simple).

Note : L'état de retour aurait aussi pu être un autre état présent dans la séquence (3, 6, 9 ou 12) ; par contre, le plus facile à implanter ici est l'état 0 puisqu'on met des 0 partout dans les tables de Karnaugh.

Les tables sont donc :

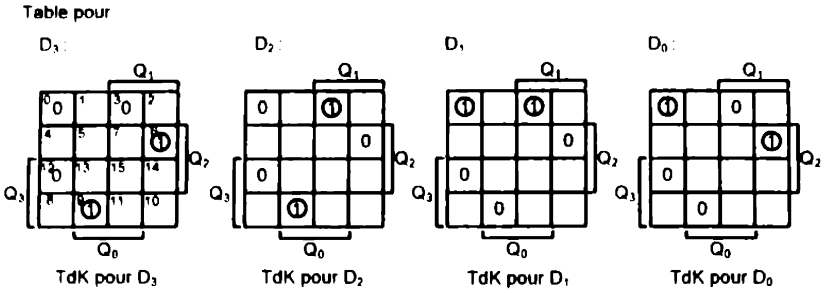


Figure 6.19

Étape 7 On obtient alors les équations suivantes des tables de Karnaugh :

$$D_0 = \bar{Q}_3 Q_2 Q_1 \bar{Q}_0 + \bar{Q}_3 \bar{Q}_2 \bar{Q}_1 \bar{Q}_0$$

$$D_1 = \bar{Q}_3 \bar{Q}_2 Q_1 Q_0 + \bar{Q}_3 \bar{Q}_2 \bar{Q}_1 \bar{Q}_0$$

$$D_2 = \bar{Q}_3 \bar{Q}_2 Q_1 Q_0 + Q_3 \bar{Q}_2 \bar{Q}_1 Q_0$$

$$D_3 = \bar{Q}_3 Q_2 Q_1 \bar{Q}_0 + Q_3 \bar{Q}_2 \bar{Q}_1 Q_0$$

Étapes 8 et 9 Les sorties sont directement obtenues des sorties des flip-flops puisqu'il n'y a pas de décodeur de sortie. On a choisi ici une représentation genre PAL tel qu'il a été vu précédemment au chapitre 4. On a ajouté une entrée asynchrone de « mise à 0 » qui force l'état « 0000 » lorsqu'il y a mise à la masse. Puisque cette entrée est asynchrone, elle ne doit servir que lors de la mise en route du circuit.

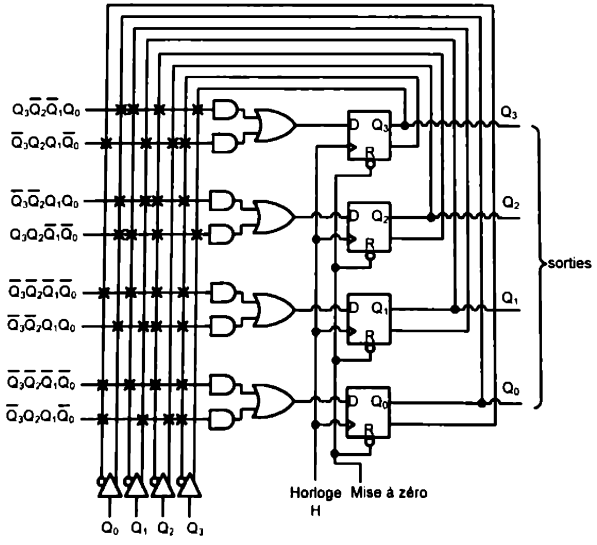


Figure 6.20

Reprenons maintenant l'exemple précédent, mais réalisons-le à l'aide de flip-flops T et en y ajoutant un décodeur de sortie. L'exemple devient donc le suivant :

Exemple 2 : Faire le design d'un séquenceur que répète la séquence 0, 3, 6, 9, 12, 0 et ainsi de suite.

Employer des flip-flops T et employez un décodeur de sortie.

Solution : On remarque qu'on a ici cinq valeurs de sorties et donc cinq états que le circuit devra parcourir sans cesse : 0, 3, 6, 9, 12. Ces valeurs exigent quatre bits de sortie puisque $9_{10} = (1001)_2$ et $12_{10} = (1100)_2$. Appelons les quatre sorties, S_3, S_2, S_1, S_0. Les sorties et leurs valeurs correspondantes sont indiquées dans la table plus bas.

Trois flip-flops donnent $2^3 = 8$ états, ce qui suffit d'autant plus que les spécifications demandent un décodeur de sortie. Appelons les trois flip-flops : Q_2, Q_1, Q_0.

Étapes 1 et 2 Cette fois-ci, il faut recoder les états puisque nous employons seulement trois flip-flops. Les états 0, 3 et 6 se codent sans problème sur trois bits. Pour les états 9 et 12, il faut leur assigner de nouvelles valeurs. On choisit d'une façon un peu arbitraire de les assigner respectivement à 5 et à 4, ce qui limitera le décodeur de sortie puisque avec ces valeurs, $S_1 = Q_1$ et $S_0 = Q_0$. Seuls S_3 et S_2 devront être synthétisés (voir étape 8) ; ceci se voit directement dans la table suivante :

	Sorties du circuit				Nouveaux états			
Sorties	S_3	S_2	S_1	S_0	Q_2	Q_1	Q_0	Rang
0	0	0	0	0	0	0	0	⓪
3	0	0	1	1	0	1	1	③
6	0	1	1	0	1	1	0	⑥
9	1	0	0	1	1	0	1	⑤
12	1	1	0	0	1	0	0	④

Figure 6.21

Étapes 3 et 4 L'étape suivante consiste à choisir le type de flip-flop et à monter la table ÉP-ÉS. On emploie des flip-flops *T*. Ce choix est imposé ici.

Il faut ensuite dresser la table État Présent → État Suivant (ÉP-ÉS). La table est constituée de trois parties tout comme dans l'exemple précédent : la zone *État Présent*, la zone appelée *stimulation* et la zone *État Suivant*. Selon ce qu'on a vu à l'exemple précédent, la table se monte rapidement. Notons que nous employons ici les « nouveaux états » tels que déterminés aux étapes précédentes.

Remplissons maintenant la zone *stimulation*. Pour remplir cette zone, il faut se référer à la table 5.13 reprise en début de chapitre. Cette fois-ci, la table 5.13 sera lue selon le sens de la flèche « design ». À gauche dans cette table, toutes les transitions possibles d'un flip-flop sont indiquées. On choisit la transition qui nous convient, et on prendra la valeur de stimulation du flip-flop que l'on emploie (ici, des flip-flops *T* selon les

spécifications imposées du circuit). Par exemple, (en lisant notre table ÉP-ÉS) pour Q_0 (et Q_1 aussi) à l'état « 000 » (rang 0), on a besoin d'une transition 0 (ÉP) → 1 (ÉS). La table 5.13 nous indique que, pour un flip-flop T, pour obtenir une transition 0 → 1, il faudra que T soit 1. On indique donc que T_0 (et T_1 aussi) = 1 à l'état « 000 ».

Prenons maintenant Q_2 à l'état « 000 » (rang 0) ; on a besoin d'une transition 0 (ÉP) → 0 (ÉS). La table 5.13 nous indique que, pour un flip-flop T, pour obtenir une transition 0 → 0, il faudra que T soit 0. On indique donc que T_2 = 0 à l'état « 000 ». Le reste de la table se construit de la même manière. On note que, pour un flip-flop T, la valeur à spécifier pour l'entrée T est égale à 0 si la valeur à l'état suivant est égale à la valeur à l'état présent (cas : pas de changement). Par contre, T est égale à 1 si la valeur à l'état suivant est différente de la valeur à l'état présent (cas : changement). Notant ceci, la table ÉP-ÉS est donc *assez facile* à remplir.

	État Présent			Stimulation			État Suivant		
Rang	Q_2	Q_1	Q_0	T_2	T_1	T_0	Q_2	Q_1	Q_0
⓪	0	0	0	0	1	1	0	1	1
③	0	1	1	1	0	1	1	1	0
⑥	1	1	0	0	1	1	1	0	1
⑤	1	0	1	0	0	1	1	0	0
④	1	0	0	1	0	0	0	0	0
①	0	0	1	0	0	1	0	0	0
②	0	1	0	0	1	0	0	0	0
⑦	1	1	1	1	1	1	0	0	0

Figure 6.22

Note : Nous parlerons des valeurs aux rangs ①, ②, ⑦ à l'étape 6.

Étapes 5 et 6 Il faut ensuite construire les tables de Karnaugh pour les entrées T des trois flip-flops. Les variables sont bien sûr les sorties des flip-flops (qui donnent la valeur de l'état présent). Ces tables s'obtiennent directement de la table ÉP-ÉS de l'étape précédente. Par exemple, pour T_2, il nous faut :

0 (rang ⓪, en position 0 dans la table de Karnaugh de T_2) ;

1 (rang ③, en position 3 dans la table de Karnaugh de T_2) ;

0 (rang ⑥, en position 6 dans la table de Karnaugh de T_2) ;

0 (rang ⑤, en position 5 dans la table de Karnaugh de T_2) ;

1 (rang ④, en position 4 dans la table de Karnaugh de T_2).

Il est sage d'avoir un *état de retour* connu (*ground state*) à l'état 0 tel qu'il a été discuté précédemment (autocorrection simple).

Note : L'état de retour aurait aussi pu être un autre état présent dans la séquence (3, 6, 5 ou 4). Par contre, cela est un peu plus compliqué que pour l'exemple précédent avec les flip-flops D pour lesquels il suffisait de mettre des 0 dans les tables de Karnaugh. Cette fois, il faut ajouter les trois rangs manquants (①, ②, ⑦) de la séquence dans la table ÉP-ÉS et obtenir les valeurs de stimulation pour une valeur État Suivant égale à « 000 ». Cela est indiqué dans la table ÉP-ÉS. Pour T_2, cela permet de compléter la table de Karnaugh correspondante comme suit :

0 (rang ①, en position 1 dans la table de Karnaugh de T_2) ;

0 (rang ②, en position 2 dans la table de Karnaugh de T_2) ;

1 (rang ⑦, en position 7 dans la table de Karnaugh de T_2).

Les tables sont donc les suivantes :

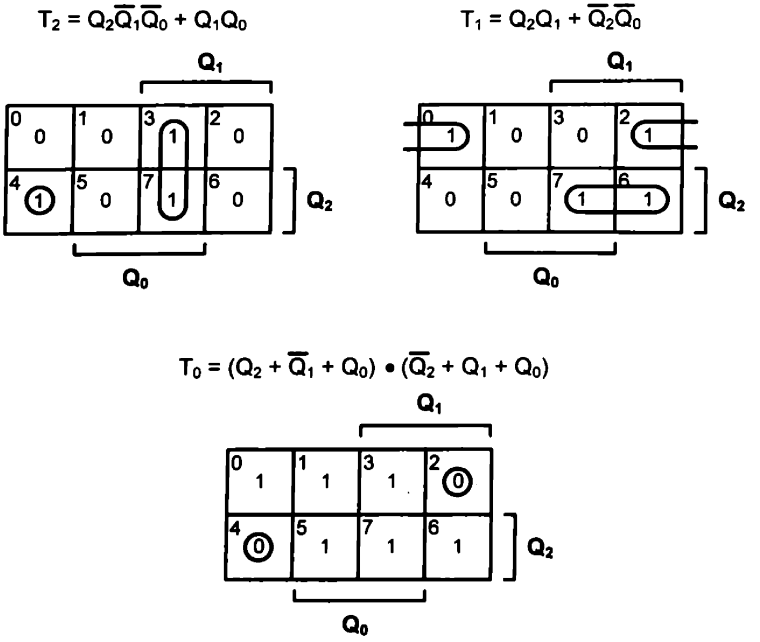


Figure 6.23

Étape 7 Les équations sont indiquées au-dessus des tables de Karnaugh. Pour T_0, on a préféré une solution en « rassemblant les 0 » (PdS) puisqu'il n'y avait que deux 0 dans la table.

Étape 8 Il faut réaliser un décodeur pour l'implantation des sorties puisque nous n'avons que trois flip-flops et qu'il faut quatre sorties. Aux étapes 1 et 2, on avait choisi

d'avoir $S_1 = Q_1$ et $S_0 = Q_0$. Quant à S_3 et S_2, on peut les synthétiser comme une combinaison de Q_2, de Q_1, et de Q_0. Le tableau suivant résume la situation :

Rang du séquenceur	Q_2	Q_1	Q_0	S_3	S_2	S_1	S_0	Sortie désirée
⓪	0	0	0	0	0	0	0	0
③	0	1	1	0	0	1	1	3
⑥	1	1	0	0	1	1	0	6
⑤	1	0	1	1	0	0	1	9
④	1	0	0	1	1	0	0	12
①	0	0	1	1[a]	0[a]	0*	1*	*non prévue*
②	0	1	0	0[b]	1[b]	1*	0*	*non prévue*
⑦	1	1	1	0[c]	0[c]	1*	1*	*non prévue*

* – à cause de notre choix, pour simplifier le décodeur, d'avoir : $S_1 = Q_1$ et $S_0 = Q_0$.
[a] – pour avoir une sortie prévue dans la séquence « 1001 » = 9.
[b] – pour avoir une sortie prévue dans la séquence « 0110 » = 6.
[c] – pour avoir une sortie prévue dans la séquence « 0011 » = 3.

Figure 6.24

On a indiqué dans la table de vérité les valeurs particulières que doivent prendre S_3 et S_2 pour que, même en cas d'état imprévu, les sorties du circuit fassent toujours partie de la séquence désirée. Ceci est un choix prudent (mais non obligatoire) puisque, de toute façon, nous avions déjà prévu un état de retour pour les états imprévus (étapes 5 et 6). Cette table de vérité nous permet de dresser les tables de Karnaugh correspondantes :

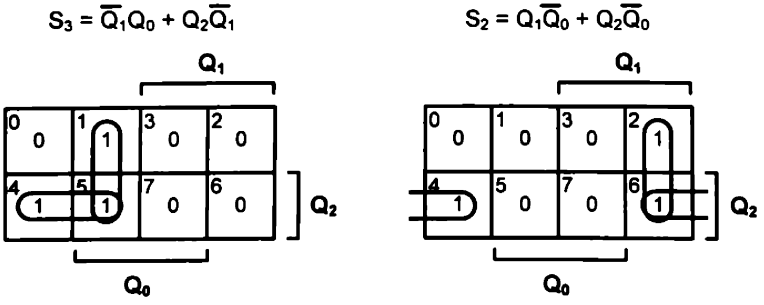


Figure 6.25

Étape 9 La logique combinatoire a été déterminée aux étapes précédentes ; il ne reste qu'à dessiner le schéma du circuit. Ici, on a décidé d'y aller pour une approche *minimaliste* en n'indiquant que les flip-flops et les équations correspondantes, ce qui permet d'alléger le dessin tout en fournissant toute la structure de celui-ci :

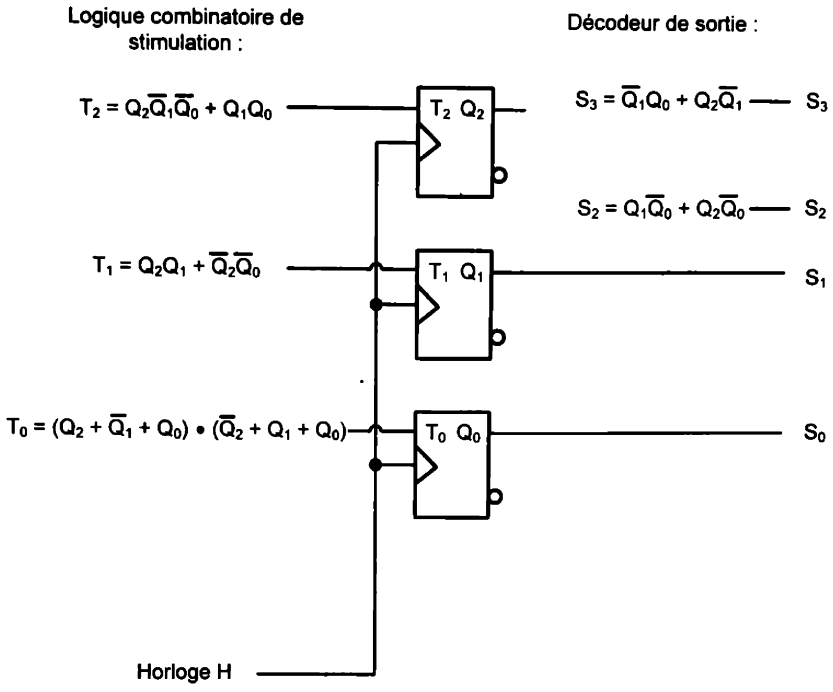


Figure 6.26

Si on le compare au circuit de l'exemple 1 (qui implante la même séquence), on note que le circuit de l'exemple 2 est plus simple ; il a toutefois nécessité un peu plus d'effort de conception pour le recodage des états et le décodeur de sortie.

AUTOCORRECTIONM OPTIMISÉE (CAS DE EXEMPLE 2)

L'approche présentée plus haut pour l'autocorrection est correcte mais certainement non-optimale, on a en effet simplement «forcé» le retour à 0 (le «ground state») aux 3 états hors-séquence (états: ①,②,⑦) cela conduit à une solution valide, facile à implanter, mais non-optimale en termes de ressources consommée. En effet le coût en termes de portes doubles à 2 entrées pour la logique de stimulation des 3 bascules T s'établit comme suit (section 2.4):

Pour T2 : 1 x Triple + 2 x Double = 4 x Double,

Pour T1 : 3 x Double,

Pour T0 : 2 x Triple + 1 x Double = 5 Double.

Au total, la logique de stimulation selon cette approche d'autocorrection simple équivaut à 12 portes doubles (4+3+5).

Voyons maintenant comment on peut réaliser l'*autocorrection optimisée*. Tel que mentionné plus haut, l'idée consiste à placer des «X-temporaires» (dénotés par des ◌') dans les tables de Karnaugh de la logique de stimulation (pour T2, T1, T0), figure suivante. Puis en réfléchissant, on établit la valeur de ces «X-temporaires» de façon à

maximiser la simplification tout en s'assurant que les états hors-séquences nous conduisent finalement à la séquence demandée. Ici, l'utilisation des OU-exclusifs triples s'avère judicieux.

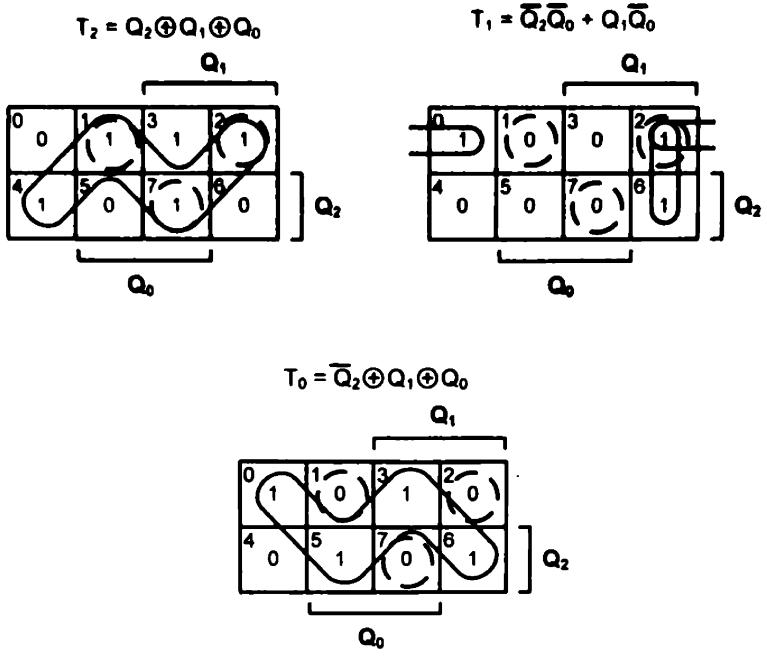


Figure 6.26.1

On vérifie donc que ces choix pour les X-temporaires ramènent les états hors-séquence dans la séquence. On reprend la table précédente (pour les états: ①,②,⑦) et on remarque que nos choix pour les X-temporaires font passer les états hors-séquences (①,②,⑦) aux états déjà présents dans la séquence (⑤,④,③) respectivement. Si cela n'avait pas été le cas, on aurait modifié un/des X-temporaire(s) de façon à nous ramener dans la séquence. L'expérience montre que souvent comme ici, «tout se passe bien» et qu'il faut généralement réaliser seulement quelques «micro-ajustements» des X-temporaires!

Finalement il est intéressant de calculer le coût de la nouvelle logique de stimulation obtenue :

Pour T2 : 2 x Double Ou-Exclusif, donc 6 Doubles au total (1 Ou-Exclusif étant l'équivalent de 3 Doubles).

Pour T1 : 3 x Double,

Pour T0 : 2 x Double Ou-Exclusif, donc 6 Doubles au total (1 Ou-Exclusif étant l'équivalent de 3 Doubles).

Et au total, la logique de stimulation pour l'autocorrection optimisée équivaut à 15 portes doubles, un peu plus que pour l'autocorrection simple et au prix d'un petit effort de réflexion!

Le décodeur reste identique à ce qui a été développé précédemment.

	État Présent			Stimulation			État Suivant			
Rang	Q_2	Q_1	Q_0	T_2	T_1	T_0	Q_2	Q_1	Q_0	*Rang de l'état suivant*
①	0	0	1	1	0	0	1	0	1	⑤
②	0	1	0	1	1	0	1	0	0	④
⑦	1	1	1	1	0	0	0	1	1	③

Figure 6.26.2

À la section *Compteur à décade* de la section 6.7, page 248, nous ferons un exemple de design avec des flip-flops J-K.

6.7 TYPES DE COMPTEURS

Il y a des types particuliers de séquenceurs que nous allons voir dans cette section : les compteurs. Un compteur est un circuit séquentiel synchrone qui assure la fonction de comptage (binaire ou autre) avec 2^N états, sans décodeur de sortie puisque les sorties du circuit sont simplement les sorties des N flip-flops le constituant. Il n'y a évidemment aucune entrée, hormis l'horloge et aussi parfois une entrée d'initialisation (mise à 0). Le compteur peut s'incrémenter (comptage croissant) ou se décrémenter (comptage décroissant).

COMPTEUR BINAIRE A CODAGE POSITIONNEL

Les compteurs binaires à codage positionnel sont des compteurs binaires dont la sortie s'incrémente (+1 à chaque coup d'horloge) ou se décrémente (−1 à chaque coup d'horloge). Autrement dit, la séquence n'est pas brisée comme les deux exemples de la section précédente. On aura ainsi une séquence (en hexadécimal) : 4, 5, 6, 7, 8, 9, *A*, *B*. Le premier problème de la section *Problèmes résolus* est un exemple de compteur binaire à codage positionnel. Pour ces compteurs, les sorties des flip-flops constituent la sortie du circuit exprimée en format binaire.

COMPTEUR A DECADE

Les compteurs à décade sont des compteurs qui possèdent 10 états dans leur séquence qui est soit :

0, 1, 2, 3, 4, 5, 6, 7, 8, 9 (compteurs à décade type incrémental), soit :

9, 8, 7, 6, 5, 4, 3, 2, 1, 0 (compteurs à décade type décrémental).

Pour ces compteurs, les sorties des flip-flops constituent généralement la sortie du circuit.

On peut mettre en cascade plusieurs compteurs à décade pour compter jusqu'à 100, 1000, etc. Nous en reparlerons plus loin.

Exemple : Concevoir un compteur à décade type décrémental qui réalisera la séquence répétée suivante : 9 – 8 – 7 – 6 – 5 – 4 – 3 – 2 – 1 – 0 avec des flip-flops J-K.

Solution : Il faudra quatre flip-flops (2^4 = 16 états) puisque nous avons ici 10 états dans la séquence et que trois flip-flops ne suffisent pas (2^3 = 8 états). Appelons les quatre flip-flops : Q_3, Q_2, Q_1, Q_0.

Étapes 1 et 2 Avec quatre flip-flops, aucun décodeur de sortie ne sera nécessaire : les sorties des flip-flops seront les sorties des états.

Étapes 3 et 4 L'étape suivante consiste à choisir le type de flip-flop et à monter la table ÉP- ÉS. On emploie des flip-flops J-K. Ce choix est imposé ici.

Il faut ensuite dresser la table État Présent → État Suivant (ÉP-ÉS). La table est constituée de trois parties, tout comme dans l'exemple précédent : la zone *État Présent,* la zone appelée *stimulation* et la zone *État Suivant.* Selon ce qu'on a vu aux exemples précédents, la table se monte rapidement.

Remplissons maintenant la zone *stimulation.* Pour remplir cette zone, il faut se référer à la table 5.13 reprise en début de chapitre. Cette fois-ci, la table 5.13 sera lue selon le sens de la flèche « design ». À gauche dans cette table, toutes les transitions possibles d'un flip-flop sont indiquées. On choisit la transition qui nous convient, et on prendra la valeur de stimulation du flip-flop que l'on emploie (ici, des flip-flops J-K selon les spécifications imposées du circuit). Par exemple, (en lisant notre table ÉP-ÉS) pour Q_0 à l'état « 1001 » (rang 9), on a besoin d'une transition 1 (ÉP) → 0 (ÉS). La table 5.13, nous indique que pour un flip-flop J-K, pour obtenir une transition 1 → 0, il faudra que $J = X$ et que $K = 1$. On indique donc ces valeurs pour J_0 et K_0 à l'état « 1001 ».

Prenons maintenant Q_1 à l'état « 1001 » (rang 9), on a besoin d'une transition 0 (ÉP) → 0 (ÉS). La table 5.13 nous indique que, pour un flip-flop J-K, pour obtenir une transition 0 → 0, il faudra que $J = 0$ et que $K = X$. On indique donc ces valeurs pour J_1 et K_1 à l'état « 0000 ». Le reste de la table se construit de la même manière. On note que, pour un flip-flop J-K, la valeur à spécifier pour les entrées J et K ne sont pas évidentes. Il faut donc se référer à la table 5.13.

		État présent				Stimulation								État suivant			
	Rang	Q_3	Q_2	Q_1	Q_0	J_3	K_3	J_2	K_2	J_1	K_1	J_0	K_0	Q_3	Q_2	Q_1	Q_0
Séquence normale	9	1	0	0	1	X	0	0	X	0	X	X	1	1	0	0	0
	8	1	0	0	0	X	1	1	X	1	X	1	X	0	1	1	1
	7	0	1	1	1	0	X	X	0	X	0	X	1	0	1	1	0
	6	0	1	1	0	0	X	X	0	X	1	1	X	0	1	0	1
	5	0	1	0	1	0	X	X	0	0	X	X	1	0	1	0	0
	4	0	1	0	0	0	X	X	1	1	X	1	X	0	0	1	1
	3	0	0	1	1	0	X	0	X	X	0	X	1	0	0	1	0
	2	0	0	1	0	0	X	0	X	X	1	1	X	0	0	0	1
	1	0	0	0	1	0	X	0	X	0	X	X	1	0	0	0	0
	0	0	0	0	0	1	X	0	X	0	X	1	X	1	0	0	1
Autocorrection	15	1	1	1	1	X	0	X	1	X	1	X	0	1	0	0	1
	14	1	1	1	0	X	0	X	1	X	1	1	X	1	0	0	1
	13	1	1	0	1	X	0	X	1	0	X	X	0	1	0	0	1
	12	1	1	0	0	X	0	X	1	0	X	1	X	1	0	0	1
	11	1	0	1	1	X	0	0	X	X	1	X	0	1	0	0	1
	10	1	0	1	0	X	0	0	X	X	1	1	X	1	0	0	1

Figure 6.27

Note : Nous parlerons des valeurs d'autocorrection à l'étape 6.

Étapes 5 et 6 Il faut ensuite construire les tables de Karnaugh pour les entrées J et K des quatre flip-flops. Les variables sont bien sûr les sorties des flip-flops (qui donnent la valeur de l'état présent). Ces tables s'obtiennent directement de la table ÉP-ÉS de l'étape précédente. Par exemple pour J_2, il nous faut :

0 (rang ⑨, en position 9 dans la table de Karnaugh de J_2) ;

1 (rang ⑧, en position 8 dans la table de Karnaugh de J_2) ;

X (rang ⑦, en position 7 dans la table de Karnaugh de J_2) ; etc.

Il est sage d'avoir un *état de retour* (*ground state*) choisi parmi les états disponibles dans la séquence tel que discuté précédemment. Ici, on a décidé *arbitrairement* de prendre 9 comme état de retour puisque c'est « le premier dans la séquence » (mais où commence-t-elle ?).

Note : l'état de retour aurait aussi pu être un autre état présent dans la séquence (8, 7, 6, etc. ou 0).

Il faut aussi déterminer les « K » et les « J » pour tous les autres états (15, 14, 13, 12, 11, 10) dans la table ÉP-ÉS. Cela est indiqué dans la table ÉP-ÉS précédente.

Les tables sont donc les suivantes :

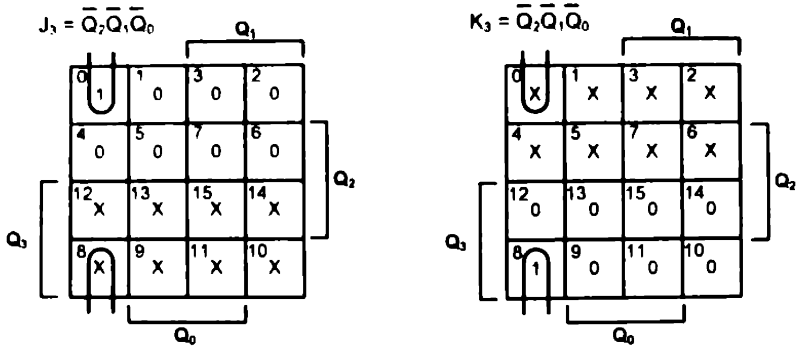


Figure 6.28 a)

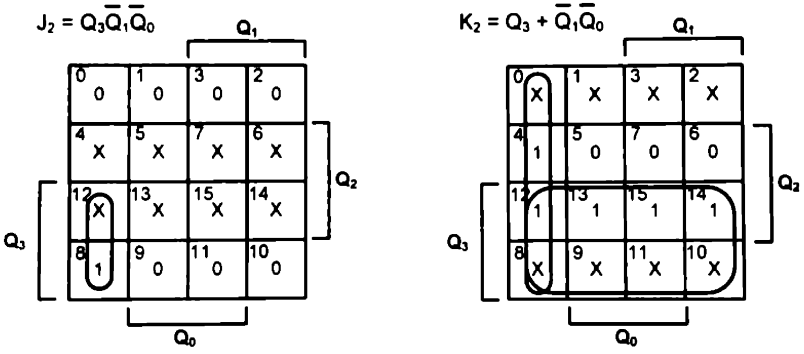


Figure 6.28 b)

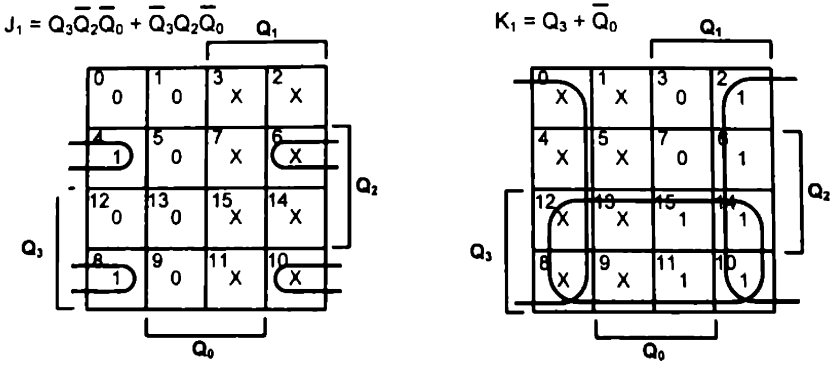


Figure 6.28 c)

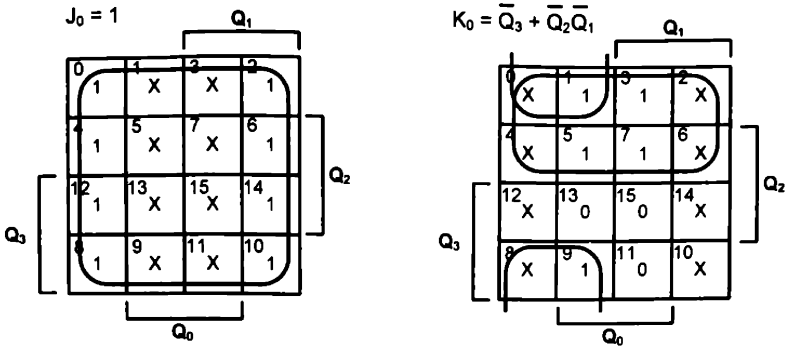


Figure 6.28 d)

Étape 7 Les équations sont indiquées au-dessus des tables de Karnaugh.

Étape 8 Comme les sorties des flip-flops sont les sorties du circuit, aucun décodeur de sortie n'est requis, ce qui simplifie le travail.

Étape 9 La logique combinatoire a été déterminée aux étapes précédentes ; il ne reste qu'à dessiner le schéma du circuit. Ici, on a décidé d'y aller pour une approche *minimaliste* en indiquant que les flip-flops et les équations correspondantes, ce qui permet d'alléger le schéma tout en fournissant toute la structure de celui-ci :

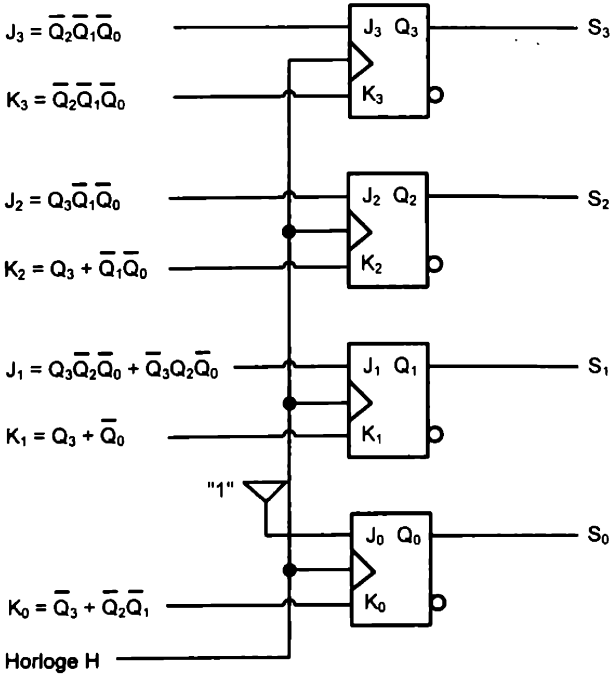


Figure 6.29

Comme on est à même de le constater, la conception avec les flip-flops J-K est plus ardue, car moins « évidente » tout en nécessitant la synthèse de deux entrées (*J* et *K*). Le flip-flop D reste donc le choix à privilégier si aucune autre considération n'est à prendre en compte. Notons finalement que les possibilités plus étendues des flip-flops J-K conduisent généralement à une réduction de la logique combinatoire requise.

CASCADE DE COMPTEURS EN EMPLOYANT LE « RIPPLE CARRY OUTPUT » (RCO)

On peut réaliser des compteurs de format et de séquence quelconques avec la méthode vue à la section précédente ; néanmoins, cette méthode est fastidieuse. Pour certaines applications, il peut être intéressant d'employer des blocs de compteurs tout prêts. Un peu comme l'utilisation des multiplexeurs au chapitre 4 nous a permis de « résoudre instantanément » une table de vérité. Ces blocs se présentent sous la forme de circuits TTL MSI discrets ou encore sous forme de librairies (dans des logiciels de conception de type Xilinx WebPack ou Logisim). Par exemple, la librairie CB4CLED de Xilinx implante un compteur synchrone de quatre bits ; la nomenclature du CB4CLED est la suivante :

C : *Counter* (compteur)
B : *Binary* (binaire)
4 : quatre bits
C : *Clear* (mise à 0)
L : *Load* (chargement d'une valeur quelconque)
E : *Enable* (activation du compteur)
D : *Directional* (directionnel : compte +1 ou -1)

Les librairies Xilinx offrent aussi le CB2CLED et le CB8CLED avec la même nomenclature. Nous étudierons en détail le CB4CLED au prochain chapitre et verrons comment la famille CBiCLED (i = 2,4,8) peut implanter une séquence quelconque. Notons que des circuits TTL aux mêmes capacités, tel le 74HC163 ou le 74HC191, sont aussi disponibles. Le logiciel LOGISIM offre aussi une librairie de compteur toutefois un peu différent du 74HC163. Nous reviendrons plus en détails sur ces compteurs au chapitre 7.

On peut aussi mettre ces blocs en cascade pour avoir des compteurs plus importants de huit bits (2 × 4), de 12 bits (3 × 4), etc.

Certains blocs MSI possèdent aussi une sortie *ripple carry output* ou sortie de la retenue qui peut servir pour la cascade des compteurs. La figure 6.30 illustre le schéma dans le cas d'un bloc implantant un compteur de quatre bits (comprenant quatre flip-flops internes). Pour ce compteur, le RCO sera actif lorsque les quatre bits du compteur seront à 1 puisque, comme on est à même de le constater, $RCO = Q_N \cdot Q_{N-1} \cdot Q_{N-2} \cdot Q_{N-3}$ (dans la figure, par souci de simplicité, on a délibérément omis d'inclure la logique interne de stimulation du compteur).

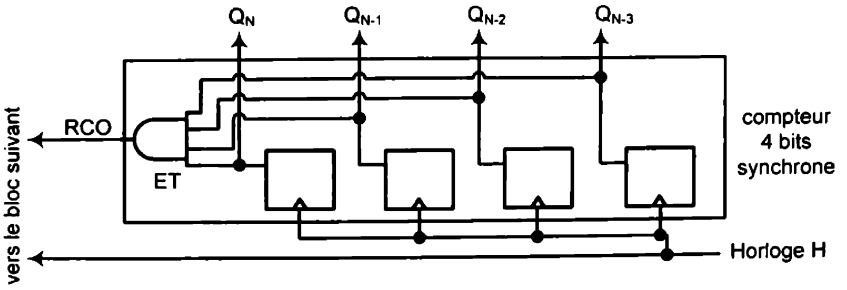


Figure 6.30

La question est alors la suivante : Comment cascader des blocs comme ceux de la figure 6.30 pour implanter des compteurs à plus de bits ? On note sur cette figure que ce type de compteur ne possède pas d'entrée d'activation (*enable*) comme avec le CB4CLED que nous étudierons au chapitre 7. On n'a pas accès non plus à la commande de l'entrée du premier flip-flop. Comme on a seulement accès à l'horloge, alors on peut employer (mais ce n'est pas conseillé, voir pourquoi plus bas) la sortie RCO comme horloge pour le bloc suivant, cela donne ceci :

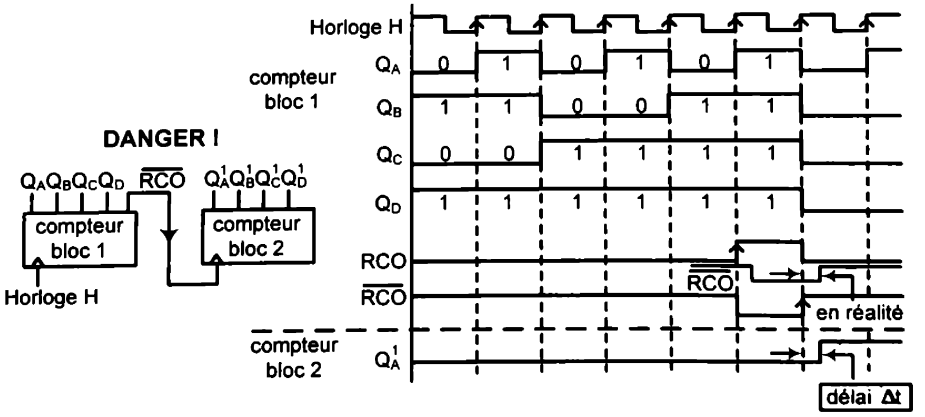


Figure 6.31

Le chronogramme débute à la valeur « 1010 » pour le compteur du 1er bloc. Les coups d'horloge se succèdent, et ce compteur passe successivement par les états : « 1011 », « 1100 », « 1101 », « 1110 » et « 1111 ». Lorsqu'on atteint précisément cette valeur, le RCO passe à 1. Puisque ces blocs-compteurs s'incrémentent sur les fronts montants de l'horloge (↑, *positive edge triggened*), on voit que pour « synthétiser » l'horloge du deuxième compteur (et que celle-ci soit *à peu près en phase* avec l'horloge H), il nous faut employer le RCO inversé (donc : $\overline{RCO}$). À ce moment, le compteur du 2e bloc s'incrémentera de 1 sur Q_A^1. Sur le chronogramme, on a aussi indiqué qu'en fait le RCO, et donc aussi le $\overline{RCO}$, seront légèrement décalés par rapport à l'horloge H à cause du

délai de propagation de la porte ET nécessaire à la synthèse de RCO et aussi de l'inverseur pour obtenir le $\overline{RCO}$. Ainsi, Q'_A sera décalé d'un délai Δt par rapport à l'horloge *H*. *À noter* : Q'_B, Q'_C, Q'_D, seront tous décalés du même délai Δt. Ce signal, et les autres bits du compteur du 2e bloc, ne sont donc plus synchrones par rapport à l'horloge *H*. Il y a donc là un risque d'aléa si l'on utilise les sorties du compteur du 2e bloc (selon notre discussion de la fin du chapitre 3). C'est là le danger indiqué sur la figure 6.31. Néanmoins, cette approche est moins mauvaise qu'une approche *full ripple* employant des flip-flops T en cascade tel que nous l'avons vu au chapitre 5 (section 5.8). En effet, une approche *full ripple* sur huit bits entraînerait un déphasage maximal de huit délais de propagation (pour le 8e bit), alors que l'approche avec deux blocs-compteurs entraîne un délai de propagation supplémentaire Δt par bloc employé. Si trois blocs-compteurs sont employés, il y aura :

- un délai de propagation Δt pour le deuxième bloc employé ;
- deux délais de propagation Δt pour le troisième bloc employé.

6.8 REGISTRES À DÉCALAGE POUR LE DESIGN DE SÉQUENCEURS SYNCHRONES

À la section 5.7, on a introduit la notion de registre à décalage et, notamment, les registres SIPO (*Serial In, Parallel Out* ou entrée série, sortie parallèle). Dans cette section, nous allons revenir sur ces registres et voir les séquences qu'on peut y implanter, ce qui permettra d'obtenir ce qu'on appellera des registres à décalage cyclique (RDC, ou en anglais : *Cyclic Serial Register*, CSR).

On se rappelle qu'un registre à décalage est constitué d'un ensemble de flip-flops D à déclenchement sur front (*edge-triggered*) connectés un à la suite de l'autre et partageant une horloge commune. Il s'agit donc d'un circuit synchrone. La figure 6.32 illustre le schéma général d'un registre à décalage à *n* bits.

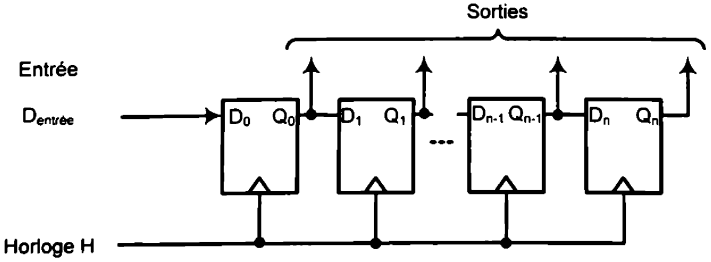


Figure 6.32

Dans cette figure, on a que :

- L'entrée série $D_{entrée}$ vient de l'extérieur et sert à introduire un « 1 » ou un « 0 » dans le registre ;
- Pour D_m, $m \neq 0$, on a $D_m = Q_{m-1}$.

Évidemment, les séquences que l'on pourra réaliser avec ce type de circuit seront restreintes puisqu'on a seulement accès à $D_{entrée}$; elles ne seront pas générales comme dans les cas des circuits que nous avons vus précédemment. Elles sont néanmoins suffisamment intéressantes pour que l'on s'y attarde.

Pour les registres à décalage cyclique (RDC), l'entrée série ($D_{entrée}$) est obtenue comme une combinaison des sorties des flip-flops (Q_0, Q_1, Q_2, etc.). Le terme « cyclique » fait bien sûr référence au fait que la séquence générée par la logique combinatoire se répète. La figure 6.33 illustre le principe d'un RDC sur quatre bits.

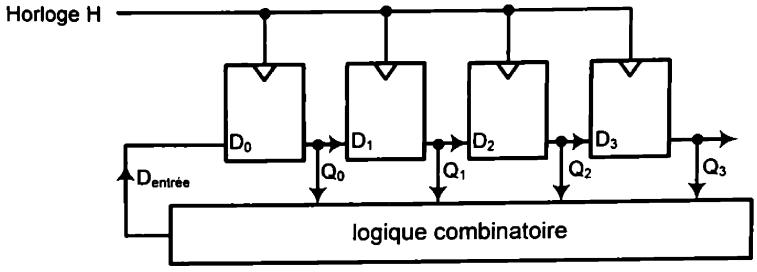


Figure 6.33

LE REGISTRE SIPO 74164

À titre d'illustration des registres à décalage SIPO, nous allons nous attarder au circuit TTL 74164. Il est à noter que ce circuit existe aussi sous forme d'une librairie dans les logiciels de conception comme Xilinx WebPack (librairie x74164). Le 74164 est un registre à décalage de huit bits (en anglais : SRG8, *Shift ReGister* 8 bits) avec mise à 0 asynchrone et entrée double (D_{SA}, D_{SB}), ce qui signifie que le produit des deux entrées est injecté dans le registre ($D_{entrée} = D_{SA} \cdot D_{SB}$).

Le schéma de ce circuit est le suivant :

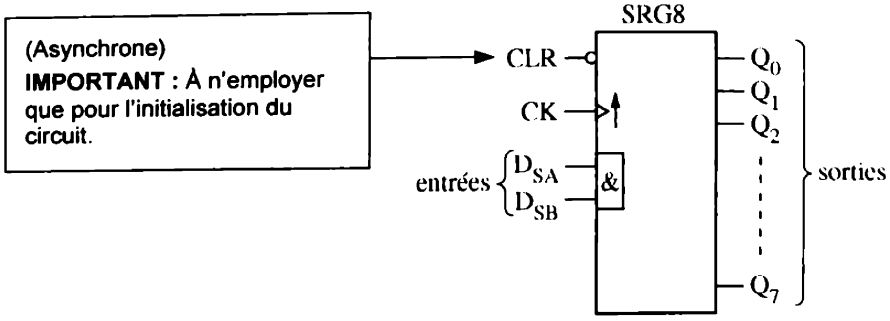


Figure 6.34 Registre à décalage huit bits à entrée série et sorties parallèles
(en anglais : *8-bit serial-in parallel-out shift register)*

Les détails internes du 74164 sont les suivants (tels que reproduits du Philips TTL Data book[8]) :

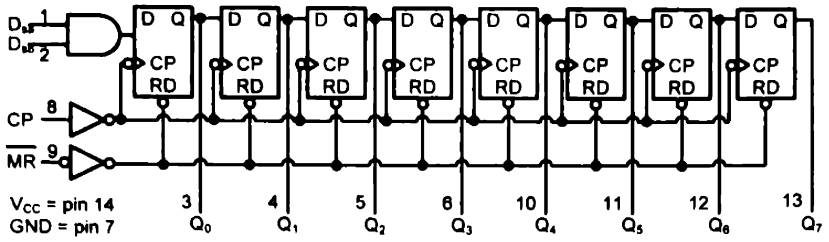


Figure 6.35

La table de vérité est la suivante (reprise du Philips TTL Data book) :

Entrées				Sorties			Mode de fonctionnement
MR	CP (*clock*, horloge)	D_{sa}	D_{sb}	Q_0	$Q_1, \ldots, Q_7$		
L	*X*	*X*	*X*	*L*	*L*	*L*	*Reset (clear)* — mise à 0 asynchrone
H	↑	*l*	*l*	*L*	q_0	q_6	Décalage (*shift*)
H	↑	*l*	*h*	*L*	q_0	q_6	
H	↑	*h*	*l*	*L*	q_0	q_6	
H	↑	*h*	*h*	*H*	q_0	q_6	

Figure 6.36

MR = *Master Reset* en anglais (ou mise à 0 principale en français).

H = *High voltage level* (tension de niveau logique 1).

h = *High voltage level one set-up time prior to the Low-to-High clock transition* (tension de niveau logique 1 présente pendant une durée égale au temps de mise en place – chapitre 5 : t_{su} – avant la transition d'horloge bas-haut).

L = *Low voltage level* (tension de niveau logique 0).

l = *Low voltage level one set-up time prior to the Low-to-High clock transition* (tension de niveau logique 0 présente pendant une durée égale au temps de mise en place – chapitre 5 : t_{su} – avant la transition d'horloge bas-haut).

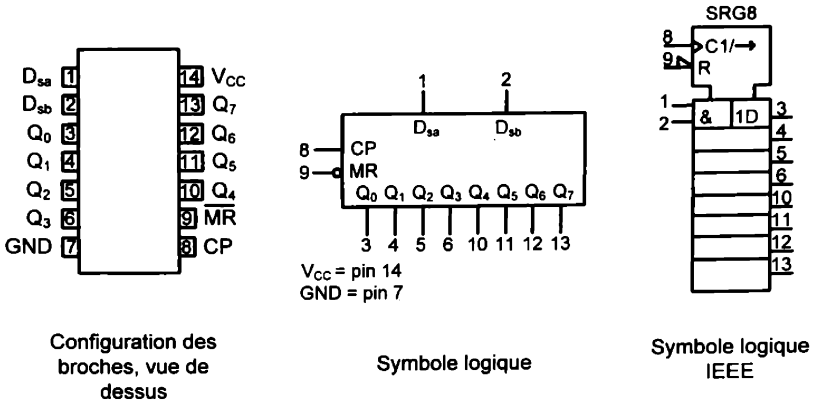
Q_n = *Lower case letters indicate the state of the referenced input (or output) one setup time prior to the Low-to-High clock transition* (les lettres minuscules indiquent l'état de l'entrée en question – ou de la sortie – présente pendant une durée égale au temps de mise en place – chapitre 5 : t_{su} – avant la transition d'horloge bas-haut).

X = *Don't care* (choix sans importance).

[8] http://www.semiconductors.philips.com/acrobat_download/datasheets/74F164_3.pdf

↑ = *Low-to-High clock transition* (transition d'horloge bas-haut)

La figure 6.37 montre les symboles du 74164 avec les numéros de broches (Philips TTL Data book).



Configuration des broches, vue de dessus

Symbole logique

Symbole logique IEEE

Figure 6.37

Le Logiciel LOGISIM contient une librairie de registre à décalage similaire au 74164. Il y a cependant en plus, une entrée d'activation à mettre à 1 pour activer le circuit!

ANALYSE DES REGISTRES A DECALAGE CYCLIQUE (RDC)

Les séquences que l'on peut obtenir avec un registre à décalage cyclique (RDC) doivent respecter les conditions imposées par les équations suivantes (pour un registre de N bits) :

- $Q_0 = D_{entrée} = f(Q_i, i = 0, ..., N)$, $f(..)$ = fonction booléenne des sorties présentes ;
- Décalage : Q_{i+1} prend la valeur de Q_i, après le coup d'horloge ($i = 0, 1, 2, ..., N-1$).

Après un certain nombre de coups d'horloge inférieur ou égal à 2^N, la séquence recommence. En effet, la plus longue séquence possible correspond à 2^N états, qui est la plus longue séquence que l'on puisse implanter dans un RDC puisqu'on a N bits et deux états pour chacun de ceux-ci. Les RDC peuvent implanter toute séquence qui se « déroule comme un patron décalé » (en anglais : *that unfolds with a horizontal shift pattern*). Pour déterminer la séquence, il faut bien sûr connaître la stimulation réalisée par l'équation $f(..)$.

L'analyse avec les RDC procède de la même façon que pour les circuits avec flip-flops étudiés précédemment (selon l'encadré 6.4A, page 234). La différence majeure vient du fait que l'architecture des flip-flops est fixe et que finalement la seule équation qu'il faut déterminer est celle de $D_{entrée}$. L'autre différence majeure est la suivante : Dans le cas de l'analyse (et de la conception) des RDC (avec décalage à droite), on adoptera la convention suivante dans l'écriture des tables État Présent → État suivant, c'est-à-dire qu'on indiquera le LSB à gauche, et le MSB à droite puisque cet ordre correspond à la progression des bits dans le circuit (c'est-à-dire au décalage des bits vers la droite). L'exemple suivant illustre l'analyse d'un RDC implantant une séquence.

Exemple : Déterminer la séquence du RDC suivant :

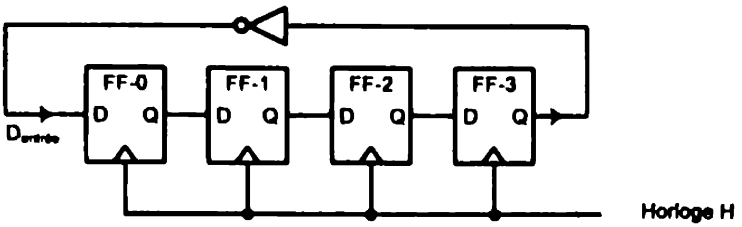


Figure 6.38

Solution : Ici, la logique combinatoire est très simple : un simple inverseur constitue la logique de $D_{entrée}$. Pour déterminer la séquence qui sera parcourue, on assume qu'au départ, le circuit (donc le contenu des flip-flops) est à l'état « 0000 ». Ceci nous permet de « calculer » la valeur de $D_{entrée}$ qui vaut donc 1. Cette valeur sera injectée dans le RDC au prochain coup d'horloge. L'état suivant du circuit sera donc « 1000 ». Cela est indiqué dans la table État Présent → État Suivant telle que dressée plus bas. Pour le présent exemple, on conçoit que le RDC partant de l'état « 0000 » va se remplir de « 1 » puis de « 0 » puisque $D_{entrée} = \overline{Q_3}$, c'est ce qu'on appelle la séquence dite de Möbius[9] qui comprend huit états (ou plus) tel qu'il a été indiqué plus bas. Selon la topologie du circuit, le bit Q_3 ne sert que pour la synthèse de $D_{entrée}$, et est autrement « perdu » après le coup d'horloge.

Étant donné qu'on a quatre flip-flops qui permettent un maximum de 16 états ($2^4 = 16$), il reste donc une possibilité de huit autres états qui constituent la séquence appelée anti-Möbius (au départ de « 0101 »). Ces deux séquences sont indiquées ci-après dans les tables État Présent → État Suivant que l'on a séparées par souci de clarté (on aurait pu aussi les combiner en une seule comme nous le faisions précédemment).

---

9 August Ferdinand Möbius : astronome et mathématicien allemand (1790-1868) qui s'intéressa au calcul vectoriel et inventa la fameuse surface à un seul côté qui porte son nom : le ruban de Möbius. Pour la réaliser, il suffit de prendre une bande de papier et de la tourner sur elle-même puis de rejoindre les deux extrémités une sur l'autre.

	État Présent					État Suivant			
	LSB			MSB	Stimulation	LSB			MSB
Rang	Q_0	Q_1	Q_2	Q_3	$D_{entrée}$	Q_0	Q_1	Q_2	Q_3
0	0	0	0	0	1	1	0	0	0
1	1	0	0	0	1	1	1	0	0
3	1	1	0	0	1	1	1	1	0
7	1	1	1	0	1	1	1	1	1
15	1	1	1	1	0	0	1	1	1
14	0	1	1	1	0	0	0	1	1
12	0	0	1	1	0	0	0	0	1
8	0	0	0	1	0	0	0	0	0

Huit états.
Si on commence à « 0000 », on obtient la séquence dite de Möbius.
Ici, chaque état suivant diffère d'un seul bit de l'état précédent, ce qui correspond au code Gray (vu au chapitre 1).

Figure 6.39

	État Présent					État suivant			
	LSB			MSB	Stimulation	LSB			MSB
Rang	Q_0	Q_1	Q_2	Q_3	$D_{entrée}$	Q_0	Q_1	Q_2	Q_3
10	0	1	0	1	0	0	0	1	0
4	0	0	1	0	1	1	0	0	1
9	1	0	0	1	0	0	1	0	0
2	0	1	0	0	1	1	0	1	0
5	1	0	1	0	1	1	1	0	1
11	1	1	0	1	0	0	1	1	0
6	0	1	1	0	1	1	0	1	1
13	1	0	1	1	0	0	1	0	1

Huit états.
Si on commence à « 0101 » on obtient la séquence anti-Möbius .
Ici, tous les FF changent après le coup d'horloge, sauf un.

Figure 6.40

Ce circuit est ce qu'on appelle en anglais un *Switch-tailed ring sequencer* ou, en français, un séquenceur en anneau à extrémité inversée. Ce circuit porte aussi le nom de séquenceur de Johnson (voir plus bas). La même approche peut être employée pour un RDC à N bits pour lequel deux séquences (Möbius et anti-Möbius) sont obtenues.

DESIGN AVEC LES REGISTRES A DECALAGE CYCLIQUE (RDC)

Cette section fait référence à l'encadré 6.4C de la page 235.

Lorsqu'on envisage une réalisation avec un RDC, il faut tout d'abord s'assurer que la séquence des états désirée peut y être implantée. Tel qu'il a été annoncé en début de section, les RDC peuvent implanter toute séquence qui se « déroule comme un patron décalé ». Si c'est le cas, le design est possible ; sinon, il faut envisager un autre type de séquenceur pour la réalisation. Fidèle à notre habitude, nous allons présenter la méthode

de design de l'encadré 6.4C à l'aide d'un exemple. Il faut cependant se rappeler la convention dans l'écriture des tables État Présent → État Suivant pour les RDC (à décalage à droite) : on indiquera le LSB à gauche, et le MSB à droite puisque cet ordre correspond à la progression des bits dans le circuit (c'est-à-dire au décalage des bits).

Exemple : On demande de concevoir un circuit synchrone qui implantera la séquence répétée suivante :

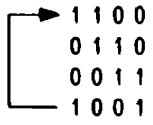


Figure 6.41

Solution : On voit qu'on a ici une séquence de quatre états qui se « déroule comme un patron décalé », le patron étant « 11 » ; elle peut donc être implantée dans un RDC. Notre choix se porte donc tout naturellement sur le registre à décalage 74164. Puisque la séquence est sur quatre bits, seuls les quatre bits les moins significatifs seront employés. Les bits supérieurs (Q_4, Q_5, Q_6, Q_7) ne sont pas employés ici. La séquence se propagera toutefois dans ceux-ci.

Étapes 1 et 2 Il faut tout d'abord dresser la liste continue des états dans la table ÉP-ÉS du circuit, tout en prenant soin d'ajouter une phase d'initialisation du registre puisque nous débutons à l'état « 0000 ». Pour déterminer $D_{\text{entrée}}$, le principe est très simple : à chaque nouvel état présent, Il s'agit de vérifier s'il vaut mieux injecter un « 1 » ou un « 0 » de sorte que :

- dans la phase d'initialisation (ou d'autocorrection), l'état suivant se rapproche de la séquence désirée ;
- l'état suivant soit le prochain état de la séquence désirée.

Pour la progression des bits, il est utile de se rappeler que $Q_0 = D_{\text{entrée}}$ après le coup d'horloge. De même : Q_{i+1} prend la valeur de Q_i, après le coup d'horloge ($i = 0, 1, 2$).

C'est ainsi qu'au départ (à l'état « 0000 »), $D_{\text{entrée}}$ sera égal à « 1 » pour amorcer la séquence. Il en est de même à l'état « 1000 ». Par la suite, on remarque que le patron déroulant est alors présent dans le registre, il faut alors s'assurer de le maintenir sachant que le dernier bit (Q_3) est « perdu ». C'est pourquoi, aux états « 0011 » et « 1001 », on spécifie $D_{\text{entrée}}$ égal à « 1 ».

Tout comme dans les autres circuits que nous avons conçus, il est sage (mais pas obligatoire) d'inclure aussi l'autocorrection. Pour celle-ci, il faut prendre tous les états « qui restent » (ici, les états 2, 4, 5, 7, 8, 10, 11, 13, 14 et 15), les ajouter à la table ÉP-ÉS et déterminer $D_{\text{entrée}}$, de sorte qu'on atteigne au plus vite la séquence désirée. Pour cela, on prend chacun de ces états à tour de rôle. Parfois, il sera nécessaire d'avoir plusieurs états intermédiaires avant de retrouver la séquence désirée. C'est par exemple le cas de l'état 15 qui mène à l'état 14, puis à l'état désiré 12.

La table suivante présente la liste complète des états, incluant l'initialisation, la séquence désirée et les états d'autocorrection.

	État présent				Stimulation		État Suivant				
Rang	Q_0	Q_1	Q_2	Q_3	$D_{entrée}$	H*	Q_0	Q_1	Q_2	Q_3	remarque
0	0	0	0	0	1	→	1	0	0	0	*Initialisation*
1	1	0	0	0	1	→	1	1	0	0	*Initialisation*
3	1	1	0	0	0	→	0	1	1	0	Séquence
6	0	1	1	0	0	→	0	0	1	1	
12	0	0	1	1	1	→	1	0	0	1	
9	1	0	0	1	1	→	1	1	0	0	
2	0	1	0	0	0	→	0	0	1	0	4
4	0	0	1	0	1	→	1	0	0	1	**9**
5	1	0	1	0	1	→	1	1	0	1	11
11	1	1	0	1	0	→	0	1	1	0	**6**
7	1	1	1	0	0	→	0	1	1	1	**14**
13	1	0	1	1	1	→	1	1	0	1	11
14	0	1	1	1	0	→	0	0	1	1	**12**
8	0	0	0	1	1	→	1	0	0	0	1
10	0	1	0	1	0	→	0	0	1	0	4
15	1	1	1	1	0	→	0	1	1	1	14

* Signifie le coup d'horloge (entrée horloge *H*).

Figure 6.42

Étape 3 Il faut maintenant synthétiser $D_{entrée}$. Selon l'encadré 6.4C, $D_{entrée}$ est synthétisé comme la somme des mintermes pour lesquels $D_{entrée} = 1$ dans la liste continue des états de la table ÉP-ÉS dressée aux étapes précédentes. Ici, on voit que :

$$D_{entrée} = \sum m_i(\underbrace{0,1,12,9}_{\text{Initialisation et séquence}},\underbrace{4,5,8,13}_{\text{Autocorrection}})$$

Figure 6.43

Cela est bien logique puisque, pour ces états, il faut injecter un « 1 » dans le registre.

On peut alors établir la table de Karnaugh pour $D_{entrée}$ afin d'obtenir son expression simplifiée :

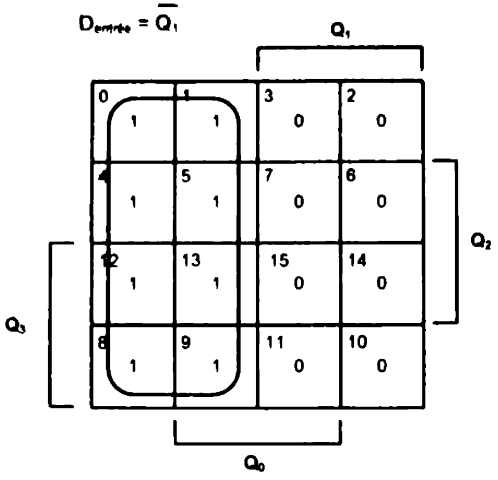


Figure 6.44

Nous sommes maintenant prêts à dessiner le circuit correspondant. Il est somme toute très simple avec l'horloge, la mise à 0 désactivée (MR – *Master Reset*) et la logique combinatoire ($D_{entrée} = \overline{Q_1}$) constituée d'un inverseur.

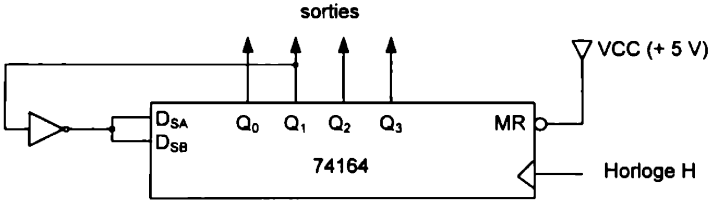


Figure 6.45

Il est également utile de dessiner le diagramme d'état correspondant, ce qui permet de visualiser le fonctionnement global du circuit incluant les phases d'initialisation d'autocorrection et la séquence proprement dite :

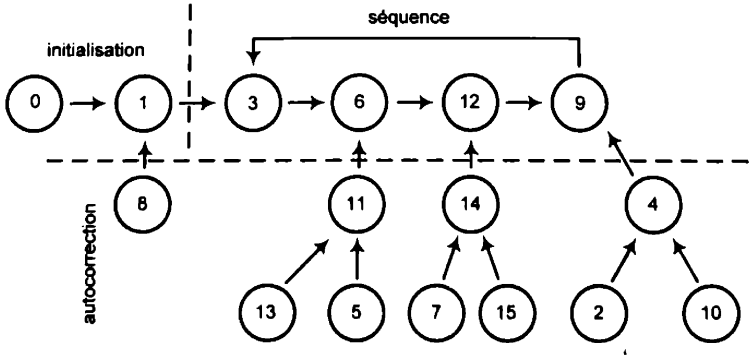


Figure 6.46

À noter : la phase d'autocorrection aurait pu être différente. En effet, pourquoi aller de l'état 8 à l'état 1 ? On aurait pu aussi bien aller de l'état 8 à l'état 0. Par contre, comme on le constate, l'autocorrection proposée ici dans la table ÉP-ÉS conduit à un $D_{entrée}$ très simple ($D_{entrée} = \overline{Q_1}$). En fait, si on ne fait rien, le circuit s'autocorrigera de lui-même! En effet, si pour les états hors séquence on impose S_{in} = 0, alors au bout de n coups d'horloge maximum (où n = nombre de bits du RDC) on reviendra à l'état 0 qui au pire aura été prévu dans la phase d'initialisation (s'il ne fait pas déjà partie de la séquence). Cela n'est pas optimal, mais est rassurant!

Une façon de minimiser le $D_{entrée}$ consiste à bâtir la table ÉP-ÉS et la table de Karnaugh de $D_{entrée}$ simultanément en tâchant, pour chaque état de l'autocorrection, de minimiser l'expression de $D_{entrée}$. Cela n'est toutefois pas toujours évident.

Nous ferons un autre exemple sur les RDC plus bas.

REGISTRES A DECALAGE CYCLIQUE (RDC) : GENERATEUR PSEUDO-ALEATOIRE

Une application intéressante des RDC est la génération de séquences pseudo-aléatoires. En effet, si le registre employé contient un grand nombre de flip-flops, une logique combinatoire appropriée pour la synthèse de $D_{entrée}$ permettra d'avoir en sortie une séquence d'états qui semblera, à première vue, tout à fait due au hasard (au maximum de 2^N états). Rappelons qu'un 74164 contient huit flip-flops, et qu'on peut facilement cascader plusieurs 74164 un à la suite de l'autre en reliant la sortie Q_7 du premier au D_0 du second, etc. Les générateurs pseudo-aléatoires trouvent notamment leur utilité entre autres en cryptage de séquences sérielles et aussi pour la génération de bruits répétitifs.

D'une façon tout à fait remarquable, la logique combinatoire nécessaire pour la synthèse de $D_{entrée}$ est d'une simplicité déconcertante : il suffit d'employer une porte NON-OU-exclusif. L'exemple suivant, qui n'emploie qu'une porte NON-OU-exclusif double avec un registre à décalage à quatre bits, permet de passer dans 15 des 16 états possibles d'une façon pseudo-aléatoire.

Exemple : Quelle est la séquence implantée dans le RDC donné par la figure 6.47 ?

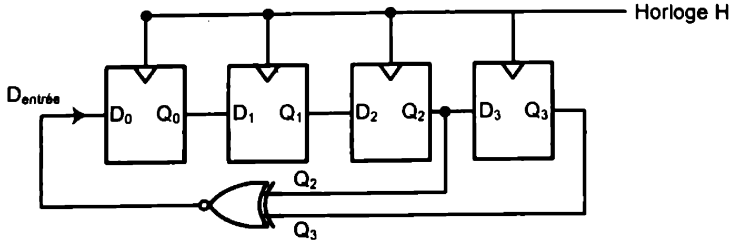


Figure 6.47

Solution : L'analyse du circuit procède selon l'encadré 6.4A, page 234, à la seule différence que l'ordre des bits est inversé (convention expliquée à l'encadré 6.4A). On débute à l'état « 0000 », ce qui nous permet de calculer le $D_{entrée}$ correspondant (=1) qui

devient le Q_0 après le coup d'horloge, les autres bits étant décalés. On aboutit donc à l'état 1, et ainsi de suite.

Q_2	Q_3	$D_{entrée}$
0	0	1
0	1	0
1	0	0
1	1	1

$D_{entrée}$ correspond à un NON-OU-exclusif inversé

Figure 6.48 Rappel : Table du Non-OU-exclusif

De cette manière, on peut dresser la table ÉP-ÉS :

	État Présent				Stimulation	État Suivant				
Rang	LSB			MSB		LSB			MSB	
	Q_0	Q_1	Q_2	Q_3	$D_{entrée}$	Q_0	Q_1	Q_2	Q_3	
0	0	0	0	0	1	1	0	0	0	
1	1	0	0	0	1	1	1	0	0	
3	1	1	0	0	1	1	1	1	0	
7	1	1	1	0	0	0	1	1	1	
14	0	1	1	1	1	1	0	1	1	
13	1	0	1	1	1	1	1	0	1	
11	1	1	0	1	0	0	1	1	0	
6	0	1	1	0	0	0	0	1	1	
12	0	0	1	1	1	1	0	0	1	
9	1	0	0	1	0	0	1	0	0	
2	0	1	0	0	1	1	0	1	0	
5	1	0	1	0	0	0	1	0	1	
10	0	1	0	1	0	0	0	1	0	
4	0	0	1	0	0	0	0	0	1	
8	0	0	0	1	0	0	0	0	0	Retour
15	1	1	1	1	1	1	1	1	1	Cul-de-sac

Figure 6.49

On note que l'état « 1111 » est un cul-de-sac ici puisqu'à cet état, $D_{entrée}$ vaut 1, qui est réinjecté dans le registre, ce qui nous maintient à l'état « 1111 ».

Le circuit passe donc dans 15 des 16 états possibles, ce qui est remarquable, surtout en considérant la simplicité de la logique combinatoire pour $D_{entrée}$ (un simple NON-OU-exclusif double). Finalement, on note que le générateur pseudo-aléatoire peut employer n'importe quelle sortie du registre puisque le patron de sortie n'est pas important ici. Bien évidemment, un tel circuit peut être étendu sur plus de bits.

REGISTRES A DECALAGE CYCLIQUE : SEQUENCEUR EN ANNEAU

Si on fait circuler un ou plusieurs « 1 » ou « 0 » dans un registre à décalage cyclique, on a ce qu'on appelle un séquenceur en anneau dont la logique combinatoire de stimulation est toute simple. Un tel séquenceur possède au maximum autant d'états que de flip-flops : N états maximum dans la séquence, car N flip-flops présents. Plusieurs cycles

sont réalisés, mais généralement un seul est valide (selon les applications). Ces séquenceurs ont besoin d'une initialisation appropriée car l'état « 0 » est un cul-de-sac. Spécifiquement, on les appelle :

- *Séquenceur en anneau à une balle ou à deux balles* (en anglais : *one shot* ou *two-shot sequencer*) selon qu'un ou deux « 1 » circulent, etc.
- *Séquenceur à une glace ou à deux glaces* (en anglais : *one-cold* ou *two-cold*) si un ou deux « 0 » circulent, etc.

La figure 6.50 montre un séquenceur en anneau à une ou à deux balles (selon la séquence retenue) :

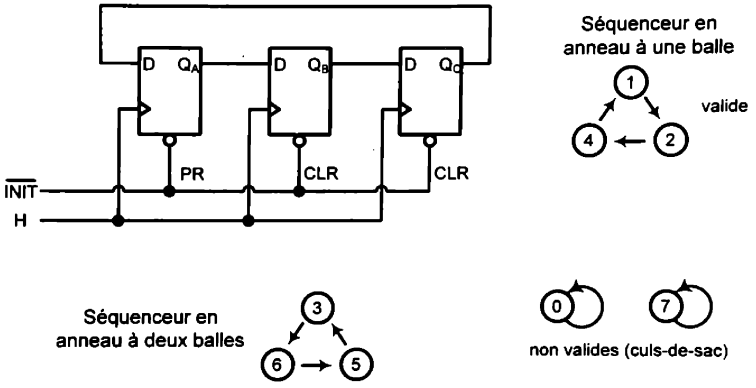


Figure 6.50

On note que l'initialisation force l'état « 001 ». La logique combinatoire consiste simplement à réinjecter le dernier bit dans le premier. Le fonctionnement du circuit s'obtient par simple inspection de celui-ci.

Registres à décalage cyclique : séquenceur de Johnson

Le séquenceur de Johnson est similaire au séquenceur en anneau ; il offre toutefois deux fois plus d'états ($2N$) que le nombre N de flip-flops présents, grâce à un décodage simple. Cette fois, le dernier bit *inversé* est injecté dans le bit le moins significatif. Les états inemployés sont des culs-de-sac. Ces séquenceurs sont aussi appelés, en anglais, *Switch-tailed ring sequencers* ou, en français, séquenceurs en anneau à extrémité inversée. On en avait parlé précédemment dans le cas des séquences de Möbius et d'anti-Möbius. La figure 6.51 présente un séquenceur de Johnson à trois bits.

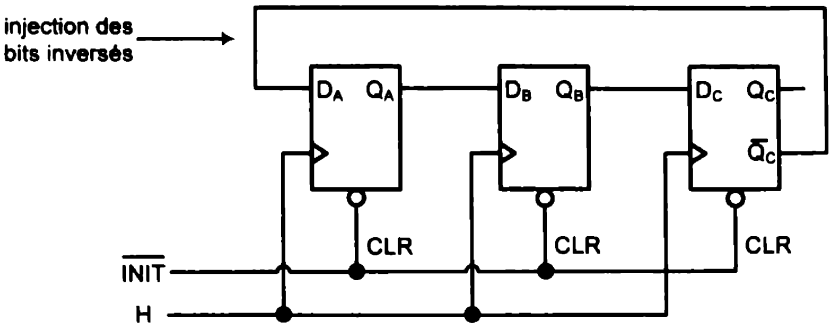


Figure 6.51

La séquence obtenue est la suivante :

Q_A	Q_B	Q_C
0	0	0
1	0	0
1	1	0
1	1	1
0	1	1
0	0	1

Figure 6.52

Avec le diagramme en anneau correspondant:

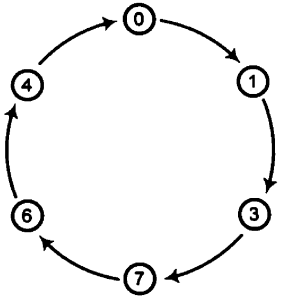


Figure 6.53

Évidemment, un problème des séquenceurs de Johnson concerne les états inemployés qui sont des culs-de-sac dans lesquels il faut éviter de tomber. Pour le circuit illustré, les états « 5 » et « 2 » sont des culs-de-sac :

ÉP				Stimulation				ÉS		
Q_A	Q_B	Q_C		D_A	D_B	D_C		Q_A	Q_B	Q_C
0	1	0	→	1	0	1	→	1	0	1
1	0	1	→	0	1	0	→	0	1	0

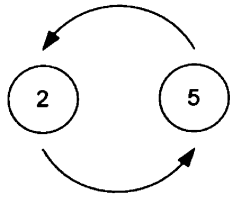


Figure 6.54

6.9 UNITÉS CACHÉES

Il arrive parfois que des états se répètent dans une séquence, par exemple si on doit réaliser le circuit qui implante la séquence répétée suivante :

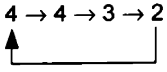


Figure 6.55

On remarque que le « 4 » est répété, c'est-à-dire qu'il sera présent pendant deux périodes complètes d'horloge. Sachant qu'on s'interdit de « toucher à l'horloge » pour la bloquer pendant un cycle pour réaliser la répétition de l'état répété, la question qui se pose alors est : comment implanter de telles séquences ? La réponse à cette question est somme toute assez simple, on a recours à ce qu'on appelle des *unités cachées*. Une unité cachée est un flip-flop que l'on ajoute à un circuit et qui permet d'implanter un état supplémentaire. Comme ce flip-flop n'apparaît pas à la sortie du circuit, de l'extérieur, il semble que le circuit soit arrêté pendant une période supplémentaire d'horloge. Les unités cachées permettent de résoudre des problèmes de répétition de nombres dans des séquences d'états.

Revenons à notre séquence « 4-4-3-2 ». Dans ce cas, on peut régler le problème de répétition en utilisant une autre représentation binaire que 100_2 pour le deuxième « 4 » de la séquence afin qu'il soit possible pour les deux « 4 » d'avoir chacun leur entrée séparée dans la table ÉP-ÉS. Ainsi, on pourrait représenter le premier « 4 » avec quatre bits au lieu de trois (en ajoutant une unité cachée). On obtient alors comme séquence : $0100_2 \rightarrow 1100_2 \rightarrow 0011_2 \rightarrow 0010_2$. En prenant comme sorties seulement les trois derniers bits, on obtient la suite désirée sans aucun décodage.

Une autre possibilité serait de substituer au premier (ou au deuxième) « 4 » une valeur qui n'apparaît pas dans la séquence, et d'ajouter un décodeur pour obtenir un « 4 » lorsque la valeur de remplacement apparaît. Cette solution requiert de la logique combinatoire supplémentaire pour réaliser le décodeur de sortie sans toutefois avoir besoin de recourir aux unités cachées. C'est à la designer de voir ce qui est plus approprié pour son application : unité cachée sans décodeur de sortie ou pas d'unité cachée avec décodeur de sortie.

Exemple : Implanter la séquence répétée suivante dans des flip-flops D :

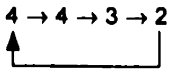


Figure 6.56

Le circuit doit être autocorrecteur et employer la technique des unités cachées sans aucun décodeur de sortie.

Solution : On suit la table 6.4B pour le design des circuits synchrones sans entrée.

Étape 1 Exprimons d'abord les nombres de la séquence en binaire. Puisque nous avons quatre états, trois flip-flops pourraient suffire si aucun décodeur de sortie n'est employé (ce qui est exigé ici selon la donnée du problème) : les sorties des flip-flops constituent les sorties du circuit. Appelons ces états : Q_2, Q_1, Q_0. Puisqu'un état (4) est répété, nous recodons celui-ci selon la technique des unités cachées. Il y aura donc l'état « 0100 » représentant le premier « 4 » et l'état « 1100 » représentant le second « 4 » (donc 4'). Appelons l'unité cachée : Q_3. Ceci est résumé dans le tableau suivant :

	États originaux				États recodés avec une unité cachée			
Rang	Q_2	Q_1	Q_0	Recodé comme :	Unité cachée Q_3	Q_2	Q_1	Q_0
4	1	0	0	→	0	1	0	0
4	1	0	0	→	1	1	0	0
3	0	1	1	→	0	0	1	1
2	0	1	0	→	0	0	1	0

Figure 6.57

Étape 2 Les sorties des flip-flops constituent directement les états de la séquence.

Étapes 3 et 4 On doit maintenant bâtir la table ÉP-ÉS, ce qui est direct dans le cas des flip-flops D exigés ici.

	État Présent				Stimulation				État Suivant			
Rang	Q_3	Q_2	Q_1	Q_0	D_3	D_2	D_1	D_0	Q_3	Q_2	Q_1	Q_0
4=4	0	1	0	0	1	1	0	0	1	1	0	0
4'=12	1	1	0	0	0	0	1	1	0	0	1	1
3=3	0	0	1	1	0	0	1	0	0	0	1	0
2=2	0	0	1	0	0	1	0	0	0	1	0	0
Autres états*	*	*	*	*	0	1	0	0	0	1	0	0

* Tous les autres états non présents dans la séquence, donc les états : 0, 1, 5, 6, 7, 8, 9, 10, 11, 13, 14 et 15.

Figure 6.58

Étapes 5, 6 et 7 On a inclus l'autocorrection vers l'état « 0100 ». Ceci a pu être résumé en une seule ligne tel qu'il a été indiqué. Il faut maintenant monter les tables de Karnaugh pour les D à partir de la table ÉP-ÉS. Comme on le voit dans les tables de Karnaugh, le choix de l'état de retour à « 0100 » est avantageux puisque cet état ne contient qu'un seul « 1 » ; seule la table de D_2 est affectée (l'état « 0010 » aurait pu convenir aussi), ce qui permet une solution « en rassemblant les zéros ». Dans les tables de Karnaugh, on a mis en caractères gras les « 0 » et les « 1 » qui correspondent à la stimulation de la séquence répétée (pas à l'autocorrection).

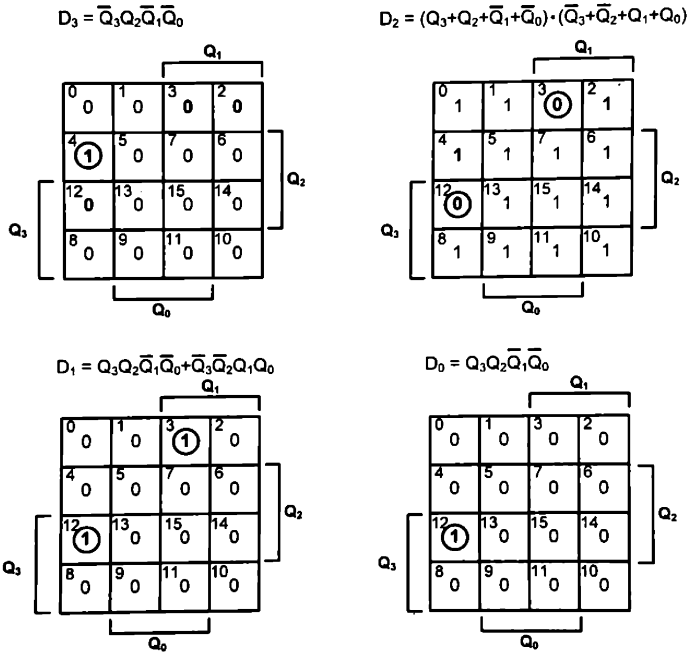


Figure 6.59

Étapes 8 et 9 Il faut maintenant bâtir le circuit, ce qui est aisé puisque les équations sont déjà obtenues et qu'on n'a aucun décodeur de sortie.

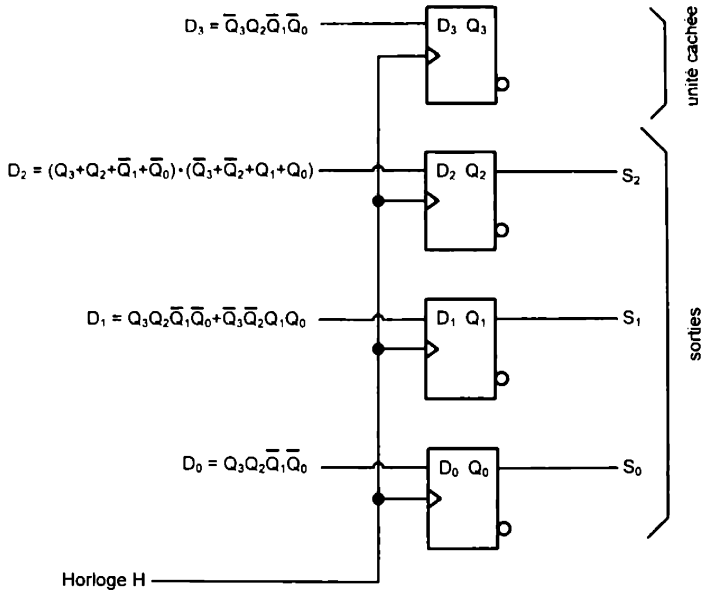


Figure 6.60

L'unité cachée Q_3 ne participe évidemment pas à la sortie du circuit.

Le principe des unités cachées peut parfois aussi être employé pour les RDC tel qu'il a été illustré par un problème de la section *Problèmes résolus*.

6.10 SÉQUENCE ARBITRAIRE OBTENUE AVEC UN COMPTEUR ET UNE MÉMOIRE

Au chapitre 4, on avait étudié les circuits de mémoire. Si on associe un compteur incrémental à une mémoire (ROM pour un contenu permanent), on pourra implanter n'importe quelle séquence. En effet, le compteur sélectionne chaque adresse une après l'autre, et la mémoire contient pour chacune de ces adresses les sorties correspondant aux états désirés. Toutes les répétitions sont possibles puisque le contenu peut être quelconque. C'est une approche qui est *universelle*, car la même structure de matériel peut resservir à d'autres applications en changeant simplement le contenu de la mémoire. C'est une approche coûteuse en logique, mais pratique. En fait, cette approche est à la base de la microprogrammation que nous aborderons en un clin d'œil à la fin du chapitre suivant. Notons que la microprogrammation permet d'implanter des structures de programmation complexes telles que des sauts conditionnels et inconditionnels, des sous-routines, des boucles. En fait à cause de ses avantages (voir l'exemple suivant), la microprogrammation est à la base de plusieurs microcontrôleurs et microprocesseurs (tels les 80X86 de Intel).

Le schéma général est présenté à la figure 6.61. On note qu'une des sorties de la mémoire est réservée à la mise à 0 du compteur, ce qui permet de relancer la séquence si celle-ci

compte moins d'états que le compte maximal du compteur (en remettant le compteur à 0). En fait, si M est le nombre d'états de la séquence désirée, et N le nombre de bits du compteur, on a évidemment : $2^N \geq M$.

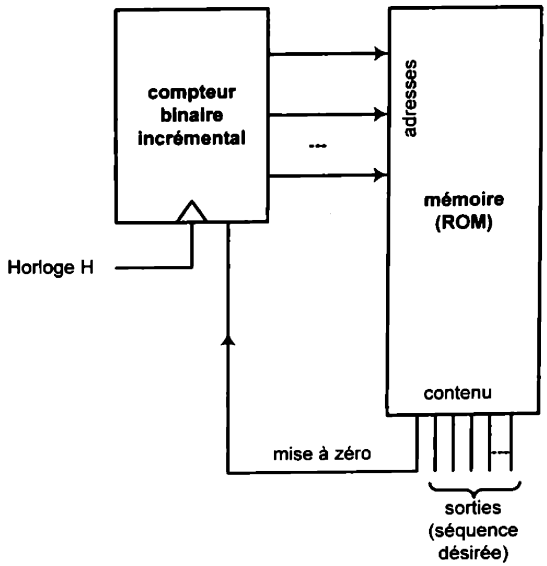


Figure 6.61

Exemple : Implanter la séquence fournie plus bas à l'aide d'un circuit composé d'une mémoire ROM et d'un compteur avec mise à 0 synchrone. Fournir le contenu de la mémoire. Assumer qu'un compteur du nombre de bits requis et qu'une mise à 0 synchrone soient disponibles.

Solution : Il s'agit donc d'implanter la séquence de sept états suivante :

$$F \rightarrow F \rightarrow E \rightarrow C \rightarrow 9 \rightarrow 3 \rightarrow 7$$

Ici, on emploiera un compteur de trois bits, ce qui permet huit états ($2^3 = 8$ états). C'est un état de trop (il nous en faut seulement sept), mais deux bits ne sont pas suffisants ($2^2 = 4$ états), et quatre beaucoup trop ($2^4 = 16$ états). On remettra le compteur à 0 à l'aide d'une ligne de commande venant de la mémoire. La mémoire aura donc cinq bits de large pour satisfaire la séquence (quatre bits : Q_3 à Q_0) et la mise à 0 (un bit = Q_4). Le schéma du circuit est donné plus bas. Le contenu de la mémoire s'obtient directement de la séquence à implanter. On charge l'adresse « 000 » avec F (mais on aurait pu faire commencer la séquence à un autre état : comme la séquence *tourne en rond*, elle n'a en fait ni début, ni fin).

Le contenu est donc le suivant :

Adresse en décimal	Adresse en binaire	Sorties (en binaire) Q_4 (mise à 0)	Q_3	Q_2	Q_1	Q_0	Sorties (hex.)
0	000	0	1	1	1	1	F
1	001	0	1	1	1	1	F
2	010	0	1	1	1	0	E
3	011	0	1	1	0	0	C
4	100	0	1	0	0	1	9
5	101	0	0	0	1	1	3
6	110	1	0	1	1	1	7
7	111	1	0	0	0	0	0

Figure 6.62

Lorsque le compteur atteint l'adresse 110, une mise à 0 (synchrone) fera qu'au prochain coup d'horloge, le compteur aura un contenu de « 000 », ce qui fera recommencer la séquence. Il est important ici d'avoir un compteur avec mise à 0 synchrone, car autrement, la mise à 0 n'attendrait pas le coup d'horloge ; on repasserait directement à 0. Le dernier état de la séquence ne durerait pas une pleine période d'horloge. Pour l'adresse « 7 », qui ne devrait jamais être atteinte, on force une mise à 0 et on met (arbitrairement) toutes les sorties à 0. *À noter* : La mise à 0 se commande ici avec un « 1 » puisqu'il n'y a pas de bulle inverseuse dans le schéma précédent.

Comme on le constate ici, si on doit implanter une autre séquence de sept (ou huit états), il suffit simplement de changer le contenu de la mémoire, le reste du circuit demeurant identique. C'est là un point majeur qui rend la conception très facile. Ce n'est bien sûr pas le cas de la solution proposée pour cette séquence avec le RDC et l'unité cachée (de la section *Problèmes résolus*). Dans ce cas, une autre séquence implique une logique combinatoire de stimulation complètement différente. En fait, tout le travail est à refaire...

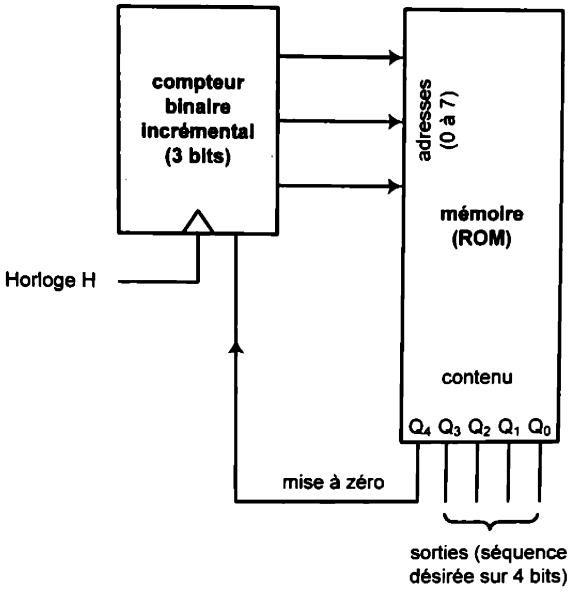


Figure 6.63

Au terme de ce chapitre, nous sommes en mesure d'analyser et de concevoir une panoplie de circuits séquentiels sans entrées : compteurs, séquenceurs, registres à décalage. Au chapitre suivant, nous aborderons les circuits séquentiels avec entrée(s), qui sont véritablement à la base des processeurs modernes.

PROBLÈMES RÉSOLUS

6.1. Soit le circuit suivant Q_C étant le MSB. On demande de fournir le diagramme d'état correspondant.

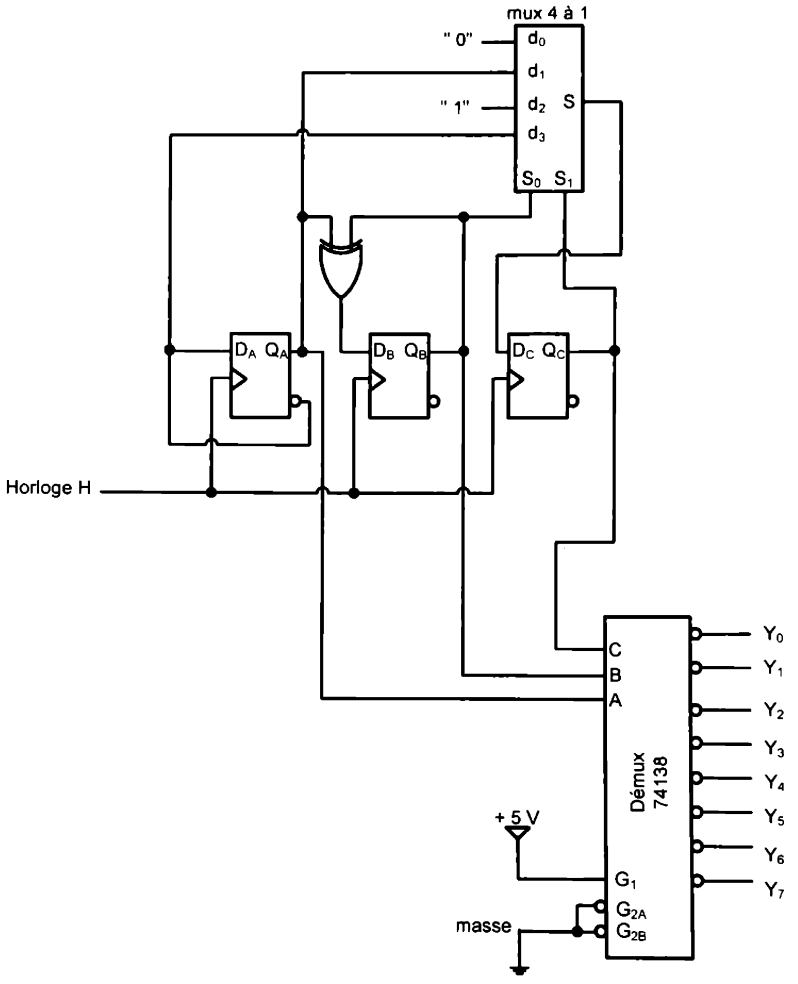


Figure 6.64

Étape 1 On note que la logique de stimulation est bien en évidence. Dans ce problème, trois flip-flops D sont employés, et la logique combinatoire de stimulation est constituée d'une porte OU-Exclusif et d'un multiplexeur 4 à 1 (étudié au chapitre 4). Les sorties du circuit sont les sorties d'un démultiplexeur 74138 (DÉMUX 3 à 8 étudié au chapitre 4).

Étape 2 Ce qui permet d'écrire les équations suivantes :

$$D_A = \overline{Q_A}$$

$$D_B = Q_B \oplus Q_A$$

$$D_C = 0 \cdot d_0 + Q_A \cdot d_1 + 1 \cdot d_2 + \overline{Q_A} \cdot d_3 = \overline{Q_C} Q_B Q_A + Q_C \overline{Q_B} + Q_C Q_B \overline{Q_A}$$

Étape 3 On assume qu'au départ $Q_C = Q_B = Q_A = 0$, alors, selon les équations précédentes, on a : $D_C = 0$, $D_B = 0$, $D_A = 1$, et donc, au coup d'horloge suivant, se rappelant le fonctionnement d'un flip-flop D (voir la figure 5.56, page 217), on obtiendra $Q_C = 0$, $Q_B = 0$, $Q_A = 1$. À propos du flip-flop D, après le coup d'horloge, la sortie est toujours égale à la valeur présente en D avant celui-ci.

Étape 4 Ces valeurs peuvent être insérées dans la *table État Présent → État Suivant* (ÉP–ÉS). Elles constituent la première ligne de rang ⓪, qui donne à l'état suivant (ou futur) ①. La seconde ligne de la table repart de là, donc au rang ①, et qui donne à l'état futur ②, et ainsi de suite. Puisque nous avons trois variables, la table aura huit lignes ($2^3 = 8$) ; il faut donc tester les autres valeurs des rangs (③ à ⑦). L'ordre d'entrée dans la table n'a évidemment pas d'importance : ce qui compte, c'est de tester toutes les combinaisons possibles, ce qui donne la table complète.

Étape 5 La Table ÉP–ÉS est alors la suivante :

	État Présent (ÉP)			Stimulation			État Suivant (ÉS)		
Rang	Q_C	Q_B	Q_A	D_C	D_B	D_A	Q_C	Q_B	Q_A
⓪	0	0	0	0	0	1	0	0	1
①	0	0	1	0	1	0	0	1	0
②	0	1	0	0	1	1	0	1	1
③	0	1	1	1	0	0	1	0	0
④	1	0	0	1	0	1	1	0	1
⑤	1	0	1	1	1	0	1	1	0
⑥	1	1	0	1	1	1	1	1	1
⑦	1	1	1	0	0	0	0	0	0

Figure 6.65

De cette table, on observe que le circuit parcourt une seule séquence comprenant les huit états possibles de ⓪ à ⑦ ($2^3 = 8$) :

⓪ → ① → ② → ③ → ④→ ⑤ → ⑥ → ⑦ (séquence principale et unique qui se répète)

Ceci peut aussi se dessiner sous la forme d'un diagramme d'état comme suit :

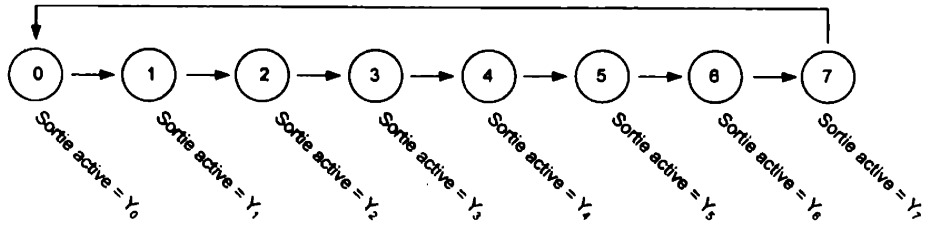


Figure 6.66

Pour ce problème, nous constatons que le circuit est autocorrecteur puisque, peu importe ce qui adviendra, nous retrouverons toujours la séquence principale. De plus, la séquence s'incrémente de +1 à chaque coup d'horloge. Ce circuit est donc un « vrai » compteur.

Quant aux sorties Q_C Q_B Q_A, elles passent directement dans le DÉMUX 74138 qui est toujours actif (puisque les bits d'activation sont $G_{2A} = G_{2B} = 0$ et $G_1 = 1$). Selon ce qu'on a étudié au chapitre 4, il y aura donc une seule sortie active à chaque coup d'horloge. Ceci est indiqué dans la table suivante et au diagramme d'état précédent. On se rappelle que pour le 74138, une sortie active signifie une sortie nulle.

Q_B Q_B Q_A	Sorties
⓪ = 0 0 0	$Y_0 = 0$, $Y_i = 1$ $\forall$ i, $i \neq 0$
① = 0 0 1	$Y_1 = 0$, $Y_i = 1$ $\forall$ i, $i \neq 1$
② = 0 1 0	$Y_2 = 0$, $Y_i = 1$ $\forall$ i, $i \neq 2$
③ = 0 1 1	$Y_3 = 0$, $Y_i = 1$ $\forall$ i, $i \neq 3$
④ = 1 0 0	$Y_4 = 0$, $Y_i = 1$ $\forall$ i, $i \neq 4$
⑤ = 1 0 1	$Y_5 = 0$, $Y_i = 1$ $\forall$ i, $i \neq 5$
⑥ = 1 1 0	$Y_6 = 0$, $Y_i = 1$ $\forall$ i, $i \neq 6$
⑦ = 1 1 1	$Y_7 = 0$, $Y_i = 1$ $\forall$ i, $i \neq 7$

Figure 6.67

6.2. Concevoir le séquenceur numérique simple pour un moteur à explosion monocylindrique de souffleuse à neige (voir la figure 6.68). Le cycle est à quatre temps et il y a deux valves pour le cylindre. Une valve est ouverte, la bougie est allumée avec un « 1 » logique. La valve est fermée, la bougie est éteinte autrement. Le signal d'horloge H fournira le rythme adéquat pour la séquence des quatre temps et les sorties.

La liste suivante fournit les actions à entreprendre pour les quatre temps du moteur :

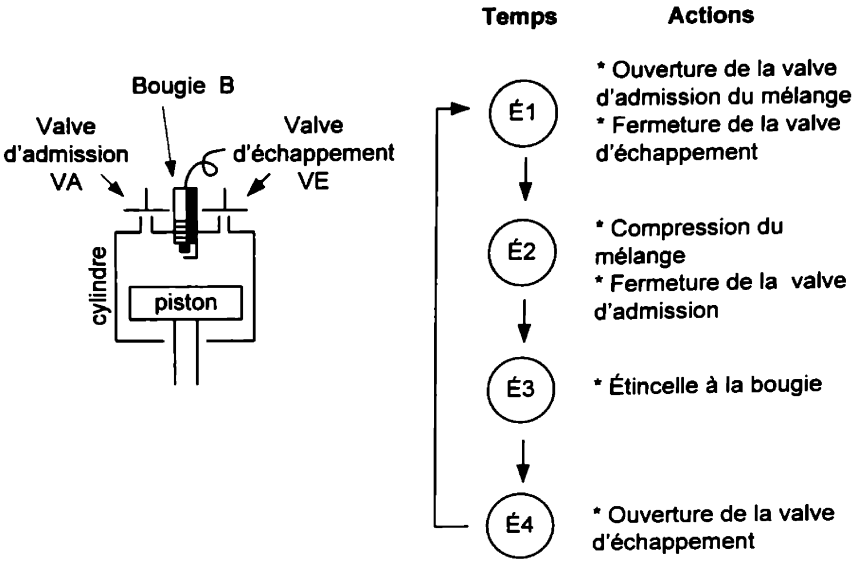


Figure 6.68

Il s'agit donc de commander la valve d'admission (sortie *VA*), celle d'échappement (sortie *VE*), de même que l'étincelle à la bougie (sortie *B*). Il faut réaliser le séquenceur avec des flip-flops J-K.

Solution : Il faudra deux flip-flops ($2^2 = 4$ états) puisque nous avons ici quatre états (*É1*, *É2*, *É3*, *É4*) dans la séquence. Appelons les deux flip-flops Q_B, Q_A. Il y aura aussi trois sorties *VA*, *VE* et *B*.

Étapes 1 et 2 Il nous faut déterminer une méthode pour assigner les sorties des flip-flops aux sorties du séquenceur. On choisit d'employer un codage Gray pour éviter que deux flip-flops ne changent en même temps (cela pourrait avoir des conséquences néfastes allant jusqu'à la destruction du moteur si la bougie est allumée alors que la valve d'admission d'essence est ouverte). La table suivante indique le codage, de même que la valeur des sorties :

Rang	État	Codé avec Q_B Q_A	*VA*	*VE*	*B*
⓪	*É1*	0 0	1	0	0
①	*É2*	0 1	0	0	0
③	*É3*	1 1	0	0	1
②	*É4*	1 0	0	1	0

Figure 6.69

Avec deux flip-flops, un décodeur de sortie sera bien sûr nécessaire, il sera simple puisqu'une seule sortie est activée à la fois.

Étapes 3 et 4 L'étape suivante consiste à choisir le type de flip-flop et à monter la table ÉP-ÉS. On emploie des flip-flops J-K. Ce choix est imposé ici.

Il faut ensuite dresser la table État Présent → État Suivant (ÉP-ÉS). La table est constituée de trois parties : la zone *État Présent*, la zone appelée *Stimulation* et la zone *État Suivant*. Selon ce qu'on a vu précédemment, la table se monte rapidement.

Remplissons maintenant la zone *stimulation*. Pour remplir cette zone, il faut se référer à la table 5.13 reprise en début de chapitre. Cette fois-ci, la table 5.13 sera lue selon le sens de la flèche « design ». À gauche dans cette table, toutes les transitions possibles d'un flip-flop sont indiquées. On choisit la transition qui nous convient, et on prendra la valeur de stimulation du flip-flop que l'on emploie (ici, des flip-flops J-K selon les spécifications imposées du circuit). Par exemple, (en lisant notre table ÉP-ÉS) pour Q_B à l'état « 00 » (rang 0), on a besoin d'une transition 0 (ÉP) → 0 (ÉS). La table 5.13 nous indique que, pour un flip-flop J-K, pour obtenir une transition 0 → 0, il faudra que $J = 0$ et $K = X$. On indique donc ces valeurs pour J_B et K_B à l'état « 00 ». Le reste de la table se construit de la même manière. On note que, pour un flip-flop J-K, la valeur à spécifier pour les entrées J et K ne sont pas évidentes. Il faut donc se référer à la table 5.13.

	État Présent		Stimulation				État Suivant	
Rang	Q_B	Q_A	J_B	K_B	J_A	K_A	Q_B	Q_A
⓪	0	0	0	X	1	X	0	1
①	0	1	1	X	X	0	1	1
③	1	1	X	0	X	1	1	0
②	1	0	X	1	0	X	0	0

Figure 6.70

Puisque les quatre états possibles avec deux flip-flops sont employés ici, il n'y a pas à se préoccuper de l'autocorrection.

Étapes 5 et 6 Il faut ensuite construire les tables de Karnaugh pour les entrées J et K des deux flip-flops. Les variables sont bien sûr les sorties des flip-flops (qui donnent la valeur de l'état présent : Q_B et Q_A). Ces tables s'obtiennent directement de la table ÉP-ÉS de l'étape précédente. Par exemple, pour J_B, il nous faut :

0 (rang ⓪, en position 0 dans la table de Karnaugh de J_B) ;

1 (rang ①, en position 1 dans la table de Karnaugh de J_B) ;

X (rang ③, en position 3 dans la table de Karnaugh de J_B) ; etc.

Les tables sont donc les suivantes :

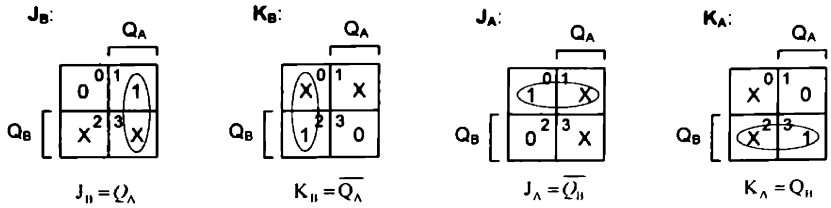


Figure 6.71

Étape 7 Les équations sont indiquées au-dessous des tables de Karnaugh.

Étape 8 Le décodeur de sortie est simplement obtenu de la table précédente (figure 6.69) où on avait spécifié les valeurs des sorties puisque chaque sortie est exprimable avec un seul minterme :

Rang	Codé avec $Q_B\,Q_A$	*VA*	*VE*	*B*	Sortie
⓪	0 0	1	0	0	$VA = \overline{Q_B} \times \overline{Q_A}$
①	0 1	0	0	0	
③	1 1	0	0	1	$B = Q_B \cdot Q_A$
②	1 0	0	1	0	$VE = Q_B \cdot \overline{Q_A}$

Figure 6.72

Étape 9 La logique combinatoire a été déterminée aux étapes précédentes ; il ne reste qu'à dessiner le schéma du circuit, ce qui est donné ci-après :

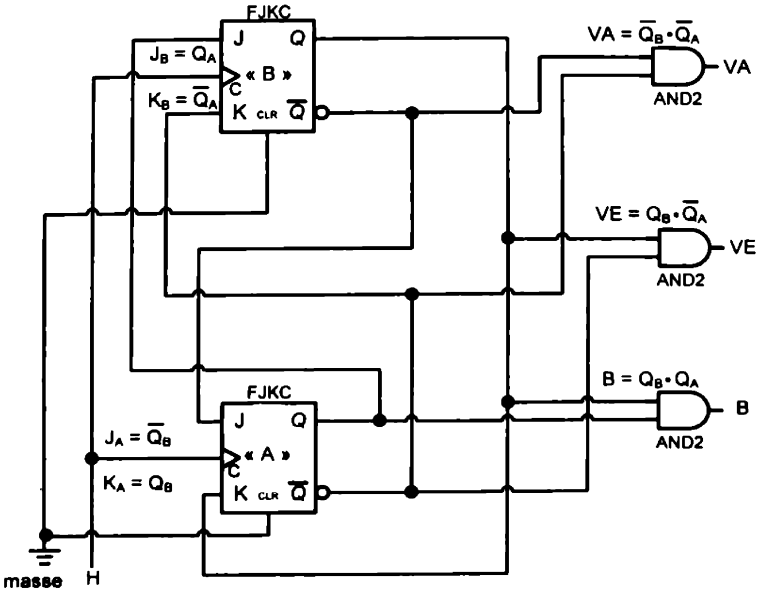


Figure 6.73

6.3. Implanter la séquence répétée suivante dans un 74164 (séquence en hexadécimal) :

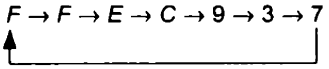


Figure 6.74

Solution : On remarque que la séquence inclut deux fois l'état *F*, ce qui nous amène à recourir à la notion d'unité cachée. Exprimons la séquence en binaire pour voir si elle correspond bien à un patron décalé déroulant. Effectivement, c'est le cas puisque le patron se décale vers la droite d'un bit à chaque état (sauf pour le premier *F*). On emploie la convention des registres à décalage avec le LSB à gauche (encadré 6.4C, page 235). On code le premier *F* comme « 11110 », et le second comme « 11111 » en ajoutant un cinquième bit appelé Q_4. Les autres bits s'appellent simplement Q_0 à Q_3. Les autres états sont recodés aussi en employant Q_4 puisque nous devons tenir compte de l'architecture rigide du RDC qui impose inexorablement le décalage des bits entrés vers la droite.

	États originaux					États recodés avec une unité cachée					
Rang (hex. = déc.)	Q_0	Q_1	Q_2	Q_3	Recodé comme :	Q_0	Q_1	Q_2	Q_3	Q_4	minterme
F = 15	1	1	1	1	→	1	1	1	1	**0**	15
F = 15	1	1	1	1	→	1	1	1	1	**1**	31
E = 14	0	1	1	1	→	0	1	1	1	**1**	30
C = 12	0	0	1	1	→	0	0	1	1	**1**	28
9	1	0	0	1	→	1	0	0	1	**1**	25
3	1	1	0	0	→	1	1	0	0	**1**	19
7	1	1	1	0	→	1	1	1	0	**0**	7

Figure 6.75

Étapes 1 et 2 Il faut tout d'abord dresser la liste continue des états dans la table ÉP-ÉS du circuit tout en prenant soin d'ajouter une phase d'initialisation du registre puisque nous débutons à l'état « 11110 ». Pour déterminer $D_{entrée}$, le principe est très simple : À chaque nouvel état présent, il s'agit de vérifier s'il vaut mieux injecter un « 1 » ou un « 0 », de sorte que :

- dans la phase d'initialisation (ou d'autocorrection) l'état suivant se rapproche de la séquence désirée ;
- l'état suivant soit le prochain état de la séquence désirée.

Pour la progression des bits, il est utile de se rappeler que $Q_0 = D_{entrée}$ après le coup d'horloge. De même : Q_{i+1} prend la valeur de Q_i, après le coup d'horloge (i = 0, 1, 2).

C'est ainsi que, pour l'initialisation (à l'état « 00000 »), $D_{entrée}$ sera égal à « 1 » pour amorcer la séquence. Par la suite, lorsqu'on remarque que le patron déroulant est présent dans le registre, il faut s'assurer de le maintenir sachant que le dernier bit (Q_4) est « perdu ».

On note que l'on n'a pas le choix de recoder l'état F avec Q_4 à 1 et à 0 puisque dans un cas il faut $D_{entrée}$ à 1, et dans l'autre cas, $D_{entrée}$ à 0.

La table suivante présente la liste complète des états incluant l'initialisation et la séquence. On n'a pas inclus l'autocorrection, car elle n'était pas demandée ici. On note néanmoins qu'elle est automatique. En effet, pour tous les états hors séquence et hors initialisation, $D_{entrée}$ se trouve fixé à « 0 ». Le RDC finit donc par se remplir de « 0 » et se retrouver en phase d'initialisation à « 00000 » (ou dans un état déjà prévu dans la phase d'initialisation ou de la séquence). Dans la table, à cause de l'initialisation, la séquence « débute » à 7, mais puisqu'elle « tourne en rond », ceci est sans importance.

		État Présent					Stimulation		État Suivant					
Rang	minterme	Q_0	Q_1	Q_2	Q_3	Q_4	$D_{entrée}$	H^*	Q_0	Q_1	Q_2	Q_3	Q_4	remarque
0	0	0	0	0	0	0	1	→	1	0	0	0	0	*Initialisation*
1	1	1	0	0	0	0	1	→	1	1	0	0	0	*Initialisation*
3	3	1	1	0	0	0	1	→	1	1	1	0	0	*Initialisation*
7	7	1	1	1	0	0	1	→	1	1	1	1	0	
F=15	15	1	1	1	1	0	1	→	1	1	1	1	1	
F=15	31	1	1	1	1	1	0	→	0	1	1	1	1	
E=14	30	0	1	1	1	1	0	→	0	0	1	1	1	Séquence
C=12	28	0	0	1	1	1	1	→	1	0	0	1	1	
9	25	1	0	0	1	1	1	→	1	1	0	0	1	
3	19	1	1	0	0	1	1	→	1	1	1	0	0	
Autres états**	**	*	**	**	**	**	0							Autocorrection

* Signifie le coup d'horloge (entrée horloge H).

** Tous les autres états non présents dans la séquence.

Figure 6.76

Étape 3 Il faut maintenant synthétiser $D_{entrée}$. Selon l'encadré 6.4C, page 235, $D_{entrée}$ est synthétisé comme la somme des mintermes pour lesquels $D_{entrée}$ = 1 dans la liste continue des états de la table ÉP-ÉS dressée aux étapes précédentes. Ici, on voit que :

$$D_{entrée} = \sum m_i(\underbrace{0,1,3}_{\text{Initialisation}},\underbrace{7,15,28,25,19}_{\text{Séquence}})$$

Figure 6.77

Le but du problème étant la synthèse de $D_{entrée}$ et l'étude des RDC, et non l'utilisation d'une table de Karnaugh à cinq variables, nous n'établirons pas l'expression finale. Nous laissons à la lectrice le soin de faire cet exercice (si elle le désire).

Nous sommes maintenant prêts à dessiner le circuit correspondant. Il est somme toute très simple avec l'horloge, la mise à 0 désactivée (MR - *Master Reset*) et la logique combinatoire tel qu'il a été indiqué.

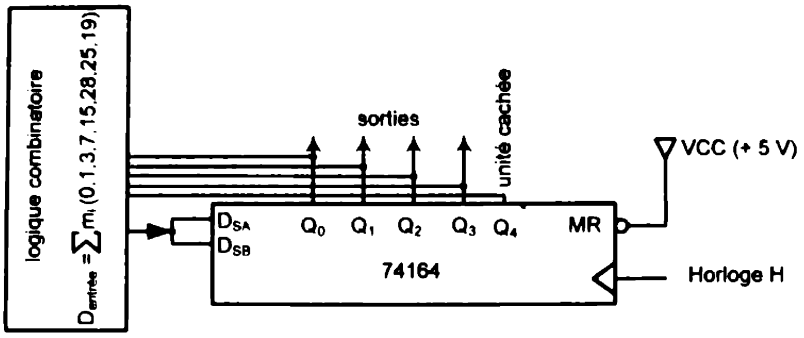


Figure 6.78

Il est également utile de dessiner le diagramme d'état correspondant, ce qui permet de visualiser le fonctionnement global du circuit, incluant les phases d'initialisation d'autocorrection et la séquence proprement dite (sans la phase d'autocorrection puisqu'elle n'a pas été déterminée strictement) :

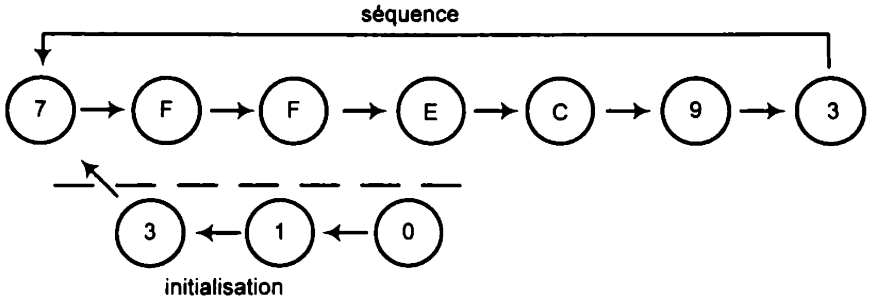


Figure 6.79

On note finalement que ce ne sont pas tous les cas de répétition qui se prêtent à l'implantation dans un RDC avec unité cachée. Remarquons aussi que l'obtention de la solution a exigé plus d'efforts que celle de l'exemple de la section 6.10 qui implante la même séquence dans une mémoire associée à un compteur.

EXERCICES

6.1. Soit le circuit suivant, déterminer le diagramme d'état correspondant (B = *MSB*).

Note : H = signal d'horloge. Il faudra appeler les sorties des flip-flops : Q_B et Q_A. Les sorties du circuit sont directement les sorties des bascules.

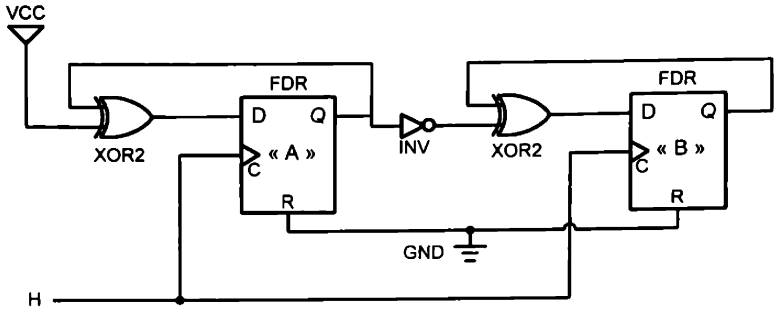


Figure 6.80

6.2. Soit le circuit suivant, déterminer le diagramme d'état correspondant (C = *MSB*).

Note : H = signal d'horloge. Il faudra appeler les sorties des flip-flops : Q_C, Q_B, et Q_A. Les sorties du circuit sont directement les sorties des bascules.

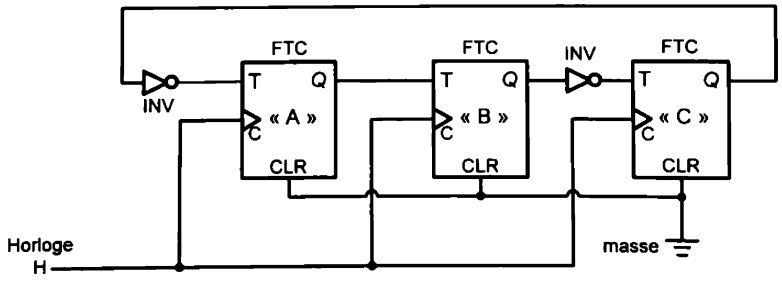


Figure 6.81

6.3. Soit le circuit suivant, déterminez le diagramme d'état correspondant (C = *MSB*).

Note : H = signal d'horloge. Il faudra appeler les sorties des flip-flops : Q_C, Q_B, et Q_A. Les sorties du circuit sont directement les sorties des bascules.

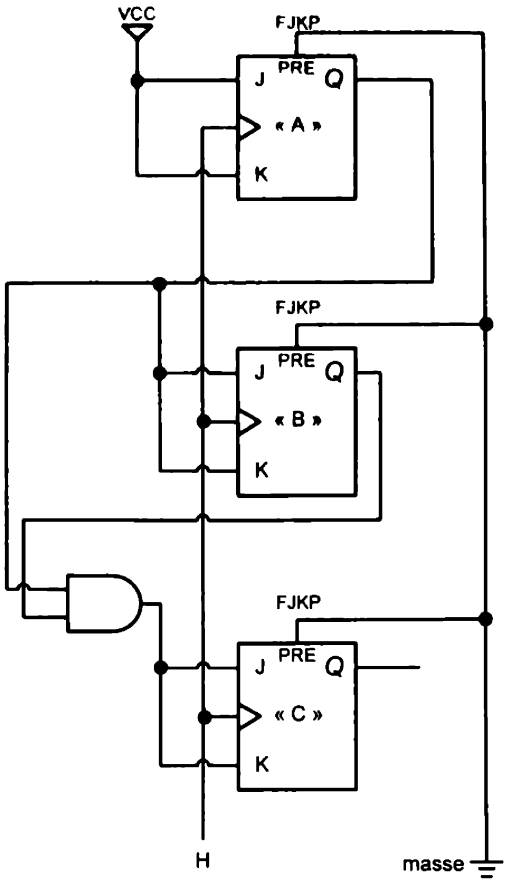


Figure 6.82

6.4. a) Rajouter la logique combinatoire nécessaire et minimale à un flip-flop J-K pour réaliser un flip-flop D. Dessiner le plan du circuit.

b) Rajouter la logique combinatoire nécessaire et minimale à un flip-flop T pour réaliser un flip-flop D. Dessiner le plan du circuit.

6.5. a) Rajouter la logique combinatoire nécessaire et minimale à un flip-flop J-K pour réaliser un flip-flop T. Dessiner le plan du circuit.

b) Rajouter la logique combinatoire nécessaire et minimale à un flip-flop D pour réaliser un flip-flop T. Dessinez le plan du circuit.

6.6. a) Rajouter la logique combinatoire nécessaire et minimale à un flip-flop T pour réaliser un flip-flop J-K. Dessiner le plan du circuit.

b) Rajouter la logique combinatoire nécessaire et minimale à un flip-flop D pour réaliser un flip-flop J-K. Dessiner le plan du circuit.

6.7. a) On vous demande d'implanter la séquence suivante dans des bascules J-K (la séquence se répète indéfiniment). Séquence (en décimal) :

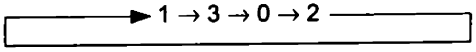


Figure 6.83

b) Répéter la question avec des flip-flops D.

c) Répéter la question avec des flip-flops T.

6.8. Soit le plan suivant. Les états sont formés des bits *QA*, *QB* et *QC* (*MSB*). H est l'horloge. Fournir le diagramme d'état complet correspondant avec :

a) *E* à 0.

b) *E* à 1.

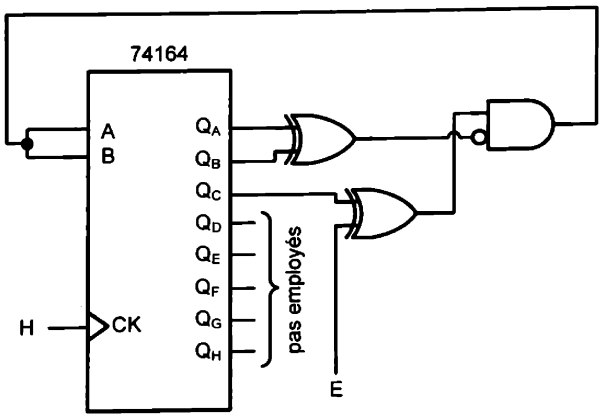


Figure 6.84

6.9. On veut que le patron binaire suivant : 1 1 0 1 0 1 1 circule dans un registre à décalage 74164. Le schéma du circuit est illustré à la figure 6.85. Donner l'équation pour le signal Serial_in en matière de sommes de mintermes. Par exemple : *Serial_in* = *m*0 + *m*1 + *m*2 + ...

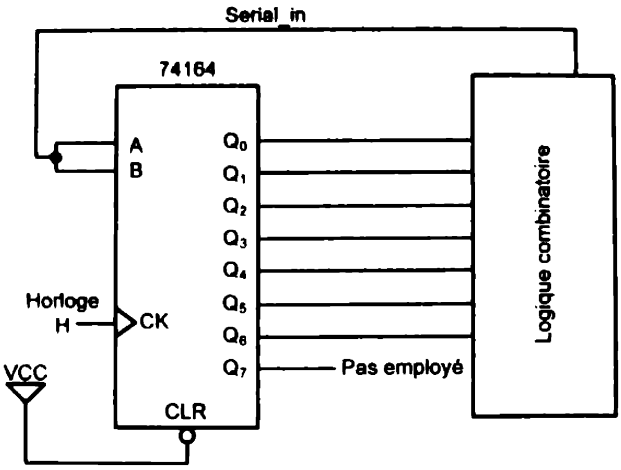


Figure 6.85

6.10. Implanter la séquence suivante dans un registre à décalage 74164 (la séquence se répète indéfiniment). Optimiser le design. On ne demande pas l'autocorrection. Séquence (en hexadécimal) :

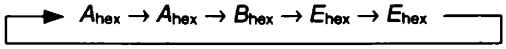


Figure 6.86

6.11. Réaliser la séquence suivante en employant un registre à décalage qu'il faudra construire avec trois flips-flops D individuels. Le séquenceur doit être autocorrecteur.

Faire le diagramme d'état complet du design. Dessiner le plan du circuit. La séquence se répétera indéfiniment : 101 → 010 ...

7
Machine d'états synchrone

L'AVENIR DES CIRCUITS LOGIQUES PASSE PAR LES « NANOTUBES DE CARBONE » !

Le XXIe siècle sera très certainement le siècle des nanotubes de carbones. Telle est l'affirmation de P. Avouris dans son article *Supertubes*[1]. En effet, ce matériau, découvert en 1991 par Sumio Iijima de chez NEC au Japon, peut servir aussi bien pour la réalisation de tissus minces et perméables que pour la réalisation d'écrans de télévisions. En nanoélectronique, ce même matériau peut servir de conducteur métallique ou de semi-conducteur.

Tel qu'il a été relaté au début de ce livre (début du chapitre 1), une course effrénée a suivi le développement du premier transistor planaire, avec pour résultat une réduction fulgurante de la taille des transistors employés dans la réalisation des circuits intégrés (selon la loi de Moore). Les procédés actuels de fabrication atteindront bien vite leurs limites (d'ici une dizaine d'années environ selon les experts du domaine). La poursuite du développement de processeurs toujours plus puissants devra donc passer par un changement de technologie. Les nanotubes de carbone représentent précisément la possibilité d'être la technologie qui prendra la relève, d'autant plus qu'elle peut se combiner aux technologies actuelles de fabrication des circuits CMOS[2] traditionnels.

Mais que sont ces « supertubes » ? Ils sont faits de carbone : le même élément chimique dont sont faites les mines de crayons et... les diamants. Plus spécifiquement, un nanotube de carbone peut être imaginé comme un treillis de carbone ; chaque atome de carbone étant relié à six de ses voisins selon un modèle hexagonal. Les tubes (sans joint) sont obtenus en repliant ce treillis sur lui-même. Les propriétés électriques étonnantes de ces nanotubes dépendent de la géométrie du tube lui-même et, en particulier, de son diamètre et de l'orientation du modèle hexagonal (de perpendiculaire à oblique par

---

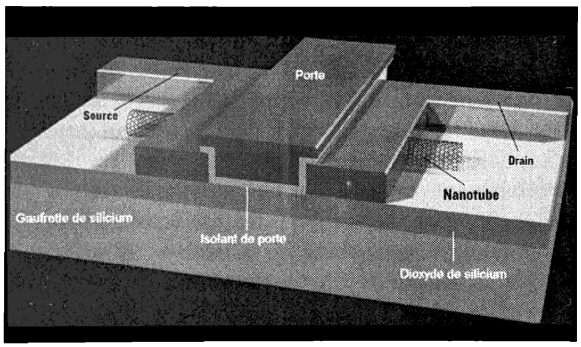
1 Courtland, R., « Carbon Nanotubes, Stuffed and Stacked », IEEE Spectrum, février 2015, p. 13-16.
Voir aussi : Avouris, P. « Supertubes », *IEEE Spectrum*, août 2004, p. 41-45. Voir Avouris P. « Carbon Nanotube Applications in Microelectronics », *IEEE Trans. Comp. Pack. Tech.* Vol. **27**, n° 4, décembre 2004, p. 629-634.

2 CMOS – *Complementary Metal-Oxide Semiconductor* : Semi-conducteur complémentaire à oxyde métallique. Puisque les circuits CMOS sont formés d'une partie positive P-MOS et négative N-MOS ne fonctionnant pas simultanément, ils sont caractérisés par une faible consommation électrique par rapport aux circuits à base de transistors bipolaires.

rapport à l'axe du tube : ||||||||||| à //////). Le diamètre et l'orientation détermineront si le nanotube sera métallique ou semi-conducteur. Par exemple, en réduisant le diamètre du tube, de sorte qu'il ne puisse laisser passer qu'un seul électron de front, on obtient, à la température de la pièce, un conducteur d'une qualité exceptionnelle qui peut faire circuler de 100 à 1000 fois plus de courant que les métaux communs comme l'aluminium ou le cuivre (pour une même section de conducteur). En effet, dans le cuivre, un électron peut se propager sur une distance d'environ 40 nm (10^{-9}) avant de se dissiper. Dans le cas d'un nanotube étroit, cette distance peut atteindre quelques microns (10^{-6} m). On conçoit facilement que la dissipation d'énergie sera très réduite dans un microprocesseur réalisé à partir de transistors à nanotubes. Quand on sait que la dissipation de chaleur devient un problème crucial dans les générations actuelles de circuits intégrés auxquels il faut adjoindre des systèmes de refroidissement de plus en plus sophistiqués, on perçoit tout de suite le potentiel des transistors à nanotubes, et ce, sans compter le gaspillage d'énergie (une autre question très actuelle[3]).

Ces nanotubes peuvent aussi émettre de la lumière. Pour cela, on fait entrer en collision, dans un nanotube de carbone, des électrons d'une part et des trous de l'autre. De plus, en modifiant le diamètre du tube, on peut changer la longueur d'onde de la lumière émise (rouge, vert, etc.). On retrouve donc plein d'avantages pour ces *supertubes* qui équiperont les circuits logiques de l'avenir.

Transistor MOSFET[4]



[3] Dans la foulée du protocole de Kyoto (http://www.ec.gc.ca/pdb/ghg/kyoto_protocol_f.cfm).

[4] *Metal-oxide semiconductor field effect transistor* : transistor à effet de champ employant un semi-conducteur à oxyde métallique.

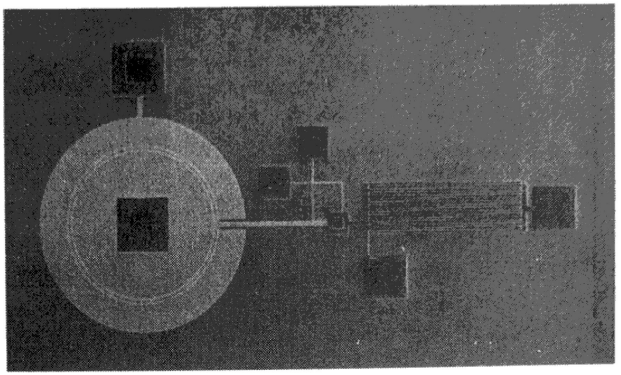


Figure 7.1 Haut : Représentation artistique d'un transistor MOSFET réalisé à partir d'un nanotube de carbone (en réalité, le rapport longueur/diamètre du nanotube peut atteindre plusieurs millions). Bas[1]: Un transistor en nanotube de carbone, des cellules de mémoire, et un transistor en silicium construits sur différentes couches d'un circuit intégré monolithique.

Les transistors MOSFET sont les « blocs LEGO » de base des circuits intégrés (numériques) CMOS. Un transistor MOSFET est un interrupteur à trois connexions : la source, le drain et la porte (*gate* en anglais). Dans un tel transistor, une tension appliquée à la porte commande le courant circulant entre la source et le drain. Par rapport à sa version traditionnelle en silicium, le CNFET (*Carbon nanotube FET* ou transistor à effet de champ à nanotube de carbone) possède des propriétés de transconductance et de courant de sortie supérieures[5].

En 2013, une équipe de l'Université Stanford a fabriqué le premier ordinateur avec des nanotubes de carbone. Avec une vitesse d'horloge de seulement 1 kilohertz et moins de 200 transistors, il n'a aucune chance de rivaliser avec les processeurs actuels à base de silicium (Figure 1.1) mais c'est un premier pas avec des avantages majeurs de consommation électrique et d'encombrement réduits. A suivre!

7.1 INTRODUCTION

Au chapitre 6, nous avons étudié les séquenceurs synchrones sans entrées, aussi appelés « machines de Moore ». Au chapitre 7, nous allons étendre ces notions d'analyse et de conception aux circuits synchrones avec entrées externes asynchrones. Ces nouvelles machines que nous allons étudier s'appellent les « machines de Mealy[6] » ou encore les machines à nombre fini d'états (*Finite State Machine* en anglais). Ces machines sont les circuits logiques avancés qui sont à la base des processeurs modernes, qu'ils soient microprogrammés (voir section 7.9) ou non. Ce chapitre est aussi le plus intéressant de ce livre puisque nous y intégrerons toutes les notions vues précédemment. En fait, pour les lectrices intéressées, il y aura un réel plaisir à concevoir ces machines à nombre finis d'états. Amusez-vous, bien et que l'aventure commence.

---

5 Adapté de : IEEE Spectrum, août 2004, p. 43.

6 D'après Mealy, George H., « A method for synthesizing sequential circuits », *Bell System Technical Journal*, Vol.34, n° 5, 1955, p.1045-1079. Professeur George H. Mealy, Université de Harvard, Boston, Massachusetts.

7.2 MACHINE DE MEALY

Une machine de Mealy est un circuit logique synchrone qui passe à travers une série finie d'états grâce à une horloge ; on l'appelle aussi « séquenceur synchrone avec entrées ». Outre son horloge, une telle machine possède des entrées externes qui peuvent être synchrones ou non et qui lui permettent d'interagir avec le monde extérieur. En plus, elle peut éventuellement posséder des entrées d'initialisation (mise à 0 ou à 1 de certains bits). Ses sorties, après le coup d'horloge de même que la séquence parcourue (c'est-à-dire l'ordonnancement des états parcourus) dépendent évidemment de l'état présent et de la valeur des entrées présentes. La mémorisation de l'état présent s'effectue grâce aux flip-flops inclus dans le circuit. Le schéma général de la machine de Mealy est donné à la figure 7.2. La partie nouvelle par rapport à la machine de Moore est illustrée en ombragé.

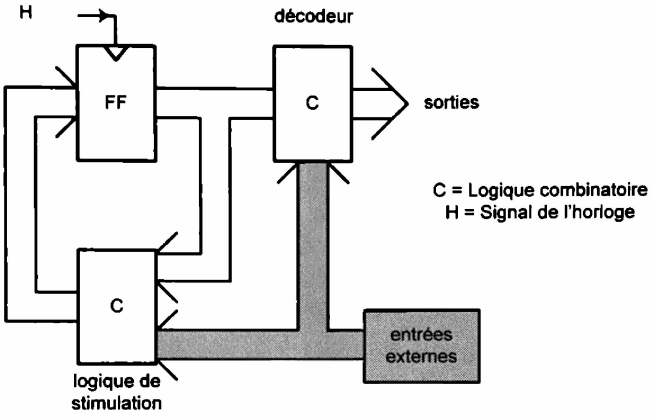


Figure 7.2

Il est à noter que le décodeur de sortie n'est pas toujours présent. De plus, comme les entrées externes peuvent être asynchrones, c'est-à-dire qu'elles peuvent changer à tout moment, et, puisqu'elles affectent la logique combinatoire du décodeur de sortie, on remarque que *les sorties du circuit peuvent changer à tout moment.*

Cela conduit souvent les conceptrices à inclure un flip-flop D entre les entrées externes et le circuit lui-même. Ces flip-flops D permettent de synchroniser les signaux externes et donc d'éviter précisément que les sorties du circuit puissent changer à tout moment. De manière à être la plus sure possible d'attraper des signaux fugitifs, on choisira pour cette application particulière, des flip-flops avec des t_{sup} et t_{hold} faibles, donc des flip-flops très rapides (chapitre 5). Il peut être bon d'employer des flip-flops D discrets plutôt que des flip-flops contenus dans des PAL, des FPGA, des CPLD, etc., car leurs spécifications sont meilleures.

Évidemment, pour s'assurer de capturer les signaux fugitifs transitoires, il faudra que l'horloge soit rapide, ou s'assurer d'avoir un circuit spécial de type « attrapeur d'impulsions » (*pulse catcher* en anglais), comme une bascule R-S dont l'entrée *S* (mise à 1) est reliée à l'entrée externe, et l'entrée *R* (mise à 0) est commandée par le circuit (pour la remise à 0 une fois le bit lu). Tel qu'il a été discuté au chapitre 5, cette approche

est cependant dangereuse puisque l'on peut enregistrer les aléas présents sur la ligne d'entrée. C'est à la conceptrice à agir avec discernement. La figure 7.3 montre un flip-flop D utilisé pour la synchronisation de l'entrée *e* externe.

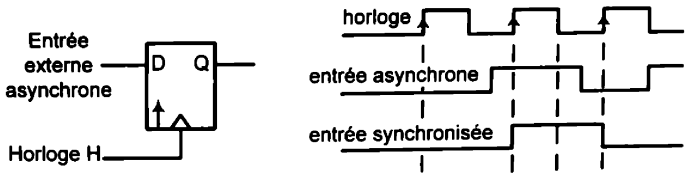


Figure 7.3

7.3 ANALYSE ET DESIGN

Cette section sera brève, car on ne fera que la présentation des étapes d'analyse et de design des circuits que nous allons étudier dans ce chapitre. Ces tableaux seront expliqués dans les sections subséquentes. On les a tous regroupés en début de chapitre aux fins de références pratiques.

7.3A – ÉTAPES D'ANALYSE (circuit synchrone avec entrées externes)

1. Identifier les éléments de la machine de Mealy (logique combinatoire de stimulation, éléments de mémoire – compteurs synchrones, flip-flops J-K, T, D, décodeur de sortie) ;
2. Écrire les équations booléennes pour chaque élément de mémoire (compteurs synchrones, flip-flops J-K, T, D) en incluant les entrées externes ;
3. Établir la table de stimulation État-Présent → État Suivant (ÉP-ÉS) et les sorties. Utiliser les entrées externes comme variables conditionnelles ;
4. Dessiner le diagramme d'état.

7.3B – ÉTAPES DE DESIGN (circuit synchrone avec entrées externes)

1. Obtenir la spécification, par exemple sous la forme d'un diagramme d'état ;
2. Codage des états et réduction de ceux-ci ;

3.1. Avec variables conditionnelles. Établir la table de stimulation État-Présent → État Suivant (ÉP-ÉS) en incluant les entrées externes comme variables conditionnelles ;

3.2. Sans variable conditionnelle. Établir la table de stimulation État-Présent → État Suivant (ÉP-ÉS) en incluant les entrées externes dans la partie État présent de la table, en position moins significative ;

3.3 (comme au chapitre 6, Table 6.4B) Pour l'*autocorrection simple* : compléter les tables de Karnaugh en assignant l'*état de retour* aux états inemployés (en anglais : *ground state*) ;

3.4 (comme au chapitre 6, Table 6.4B) Pour l'*autocorrection optimisée* : compléter les tables de Karnaugh en assignant des *X-temporaires* (X : valeurs «sans importance»)

aux états inemployés, optimiser les regroupements possibles dans les tables, vérifier que les simplifications obtenues ramènent les états inemployés dans la séquence désirée, si ce n'est pas le cas modifier la valeur des *X-temporaires* en conséquence;

4. Établir les tables de Karnaugh pour les flip-flops J-K, T, D ;
5. Synthétiser le décodeur de sortie s'il y a lieu à l'aide de tables de Karnaugh ;
6. Dessiner le schéma complet.

7.3C – RÈGLES DE SYNCHRONISME

1. N'employer comme éléments de mémorisation que des flip-flops J-K, T, D ;
2. L'horloge *H* (ou *CK*, de l'anglais *clock*) doit être commune à tous les éléments de mémorisation ;
3. Les entrées externes doivent être amorties puis synchronisées avec l'horloge ;
4. On ne doit jamais créer de signal qui serait fonction de l'horloge ;
5. On ne doit jamais employer les entrées asynchrones des éléments synchrones (exemple : les entrées de mise à 1 *PRESET* ou PR (de l'anglais) ou de mise à 0 *CLEAR* ou CLR (de l'anglais)) sauf pour initialiser le circuit au temps $t = 0$.

Le « danger » associé à la règle n° 4 précédente est illustré par l'exemple suivant.

Exemple : Créons le signal $H \bullet A$ où A est un signal asynchrone.

Cela entraîne la création d'aléas tel qu'illustré à la figure 7.4 (noter les délais – exagérés ici pour les illustrer - dûs au temps de propagation dans la porte ET):

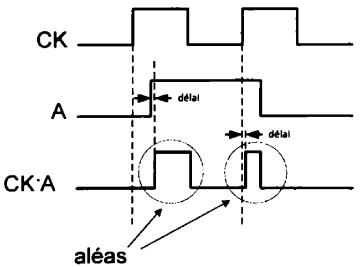


Figure 7.4

7.3D – RÈGLES DE DESIGN – Compteurs MSI (tels le CB4CLED ou le 74191)

0. Obtenir le diagramme d'état s'il n'est pas fourni.
1. Établir la *table des actions* avec : *L* (*Load*, de l'anglais, ou charge une valeur), *CE* (*count enable*, de l'anglais, ou activation du comptage), « + » ou *U* (*count Up*, de l'anglais, ou compte incrémental), « - » ou $\overline{U}$ (*count down*, de l'anglais, ou compte décrémental) ;
2. Pour chaque état, déterminer *L*, *CE*, *U* par étude des variables présentes ;

3. Dresser les tables de Karnaugh pour *L*, *CE*, *U* ;
4. Dresser les tables de Karnaugh des bits de données seulement pour les cas où un *L* est demandé.
5. Si le design est simple, monter les L, CE, U et les bits de données directement comme fonction des entrées avec des *portes simples*. Autrement, employer des *multiplexeurs* :
   - (Avec MUX simplifiés) Dresser la table des Anciens états (= états d'origine) → Nouveaux états (= états recodés) et faire la synthèse des bits de sélection du ou des MUX simplifié(s).
   - (Avec MUX non simplifiés) Faire la synthèse des bits de sélection du ou des MUX ;
6. Faire la synthèse des sorties ;
7. Faire le schéma complet.

CIRCUIT « BLIP » DE COMMANDE IMPULSIONNEL

Par rapport aux horloges des circuits numériques, le temps de réaction humain est très, très lent. Pour réaliser des interfaces homme-machine adéquates, il est donc utile d'ajouter un circuit intermédiaire qui convertit les réactions humaines lentes en impulsions d'une durée égale à une impulsion d'horloge. Un tel circuit s'appelle un circuit « BLIP » (de l'anglais). Un tel circuit est aussi utile pour synchroniser les signaux d'entrées. Le circuit BLIP fournit une impulsion de durée égale à une période d'horloge T peu importe la durée de l'entrée E.

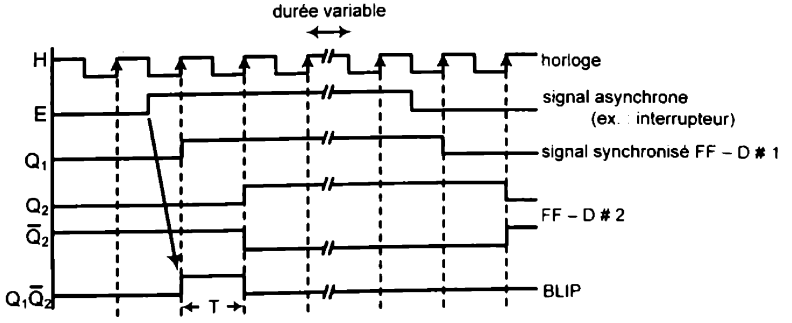


Figure 7.5

Le circuit BLIP est représenté à la figure 7.6.

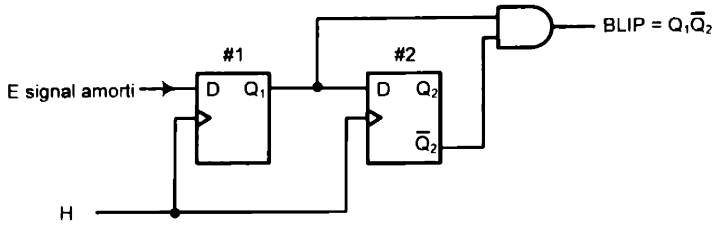


Figure 7.6

Ce circuit fournit une impulsion d'une durée unitaire (= une période d'horloge T), peu importe la durée du signal d'entrée E.

Important : Il est à noter que le signal E aura été au préalable amorti à l'aide d'un circuit antirebond (expliqué au chapitre 5, à la section 5.3, *Circuit antirebond avec bascule R-S*).

7.4 ANALYSE DES CIRCUITS SYNCHRONES AVEC ENTRÉES EXTERNES PAR LA MÉTHODE DES VARIABLES CONDITIONNELLES[7]

La méthode des variables conditionnelles est un outil puissant qui permet l'analyse et la conception des circuits synchrones avec des entrées externes multiples. Sans cette méthode, seuls des circuits très simples pourraient être analysés (voir l'exemple 4).

Exemple 1 : Nous allons déterminer le diagramme d'état du séquenceur suivant en employant la technique des variables conditionnelles (*C* = *MSB*) :

[7] La méthode sans variable conditionnelle est aussi discutée en fin de section.

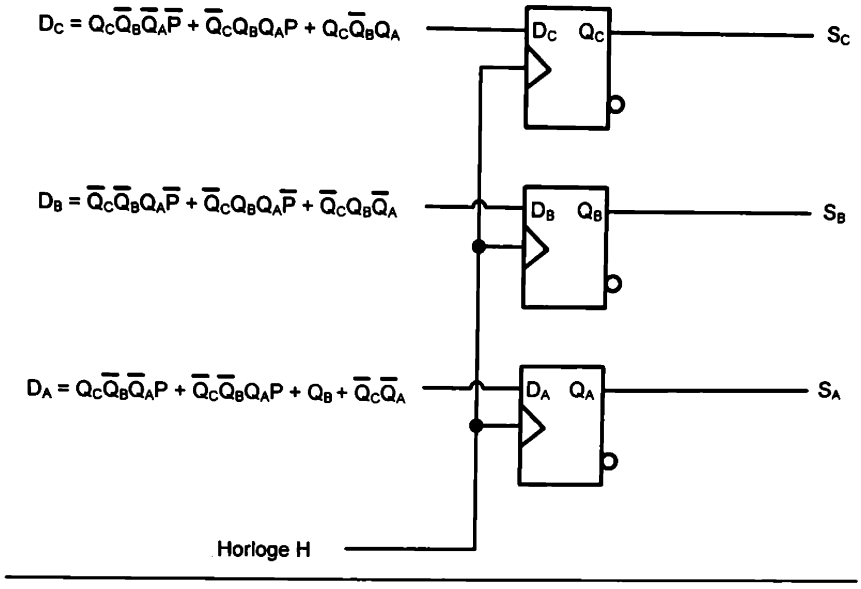


Figure 7.7

Solution : La technique des variables conditionnelles consiste à inclure les entrées externes *directement* dans la table ÉP-ÉS en laissant, pour la définition des sections *stimulation* et *état suivant*, le rang tel qu'il a été défini *seulement* par l'état des flip-flops présents dans le circuit. Cela est montré plus bas. Se référer à l'encadré 7.3A, page 291, pour les étapes suivantes.

Étape 1 Les éléments de la machine de Mealy sont bien séparés : trois flip-flops D, aucun décodeur de sortie et une entrée externe P.

Étape 2 Les équations des trois flip-flops D sont les suivantes, directement fournies sur le plan du circuit, ce qui facilite la tâche :

$$D_C = Q_C\overline{Q_B}\,\overline{Q_A}\,\overline{P} + \overline{Q_C}Q_BQ_AP + Q_C\overline{Q_B}Q_A$$

$$D_B = \overline{Q_C}\,\overline{Q_B}Q_A\overline{P} + \overline{Q_C}Q_BQ_A\overline{P} + \overline{Q_C}Q_B\overline{Q_A}$$

$$D_A = Q_C\overline{Q_B}\,\overline{Q_A}P + \overline{Q_C}\,\overline{Q_B}Q_AP + Q_B + \overline{Q_C}\,\overline{Q_A}$$

Étape 3 Il faut dresser la table ÉP-ÉS en employant l'entrée externe P comme variable conditionnelle. Comme on a trois flip-flops, il y aura 8 ($= 2^3$) lignes dans la table ; une par état. On débute à l'état « 000 ». Ainsi, avec $Q_C = Q_B = Q_A = 0$ dans la section ÉP (= État Présent), on obtient les valeurs suivantes de $D_C = 0$, $D_B = 0$, $D_A = 1$. Passons ensuite à l'état « 001 ». Ainsi, avec $Q_C = 0$, $Q_B = 0$, $Q_A = 1$ dans la zone ÉP, on obtient les valeurs suivantes de $D_C = 0$, $D_B = \overline{P}$, $D_A = P$ et ainsi de suite. Les colonnes Q_C Q_B Q_A, de la section ÉS (= État suivant) sont directement recopiées respectivement des colonnes D_C, D_B, D_A puisque selon la table 5.13 pour un flip-flop D, l'état futur est toujours égal à la stimulation.

	ÉP			Stimulation			ÉS		
Rang	Q_C	Q_B	Q_A	D_C	D_B	D_A	Q_C	Q_B	Q_A
⓪	0	0	0	0	0	1	0	0	1
①	0	0	1	0	$\overline{P}$	P	0	$\overline{P}$	P
②	0	1	0	0	1	1	0	1	1
③	0	1	1	P	$\overline{P}$	1	P	$\overline{P}$	1
④	1	0	0	$\overline{P}$	0	P	$\overline{P}$	0	P
⑤	1	0	1	1	0	0	1	0	0
⑥	1	1	0	0	0	1	0	0	1
⑦	1	1	1	0	0	1	0	0	1

Figure 7.8

Étape 4 La table ÉP-ÉS nous permet de dresser le diagramme d'état. Aux états 0, 6 et 7, on se retrouve inconditionnellement à l'état 1 après le coup d'horloge. À l'état 2, on se retrouve inconditionnellement à l'état 3 après le coup d'horloge. À l'état 5, on se retrouve inconditionnellement à l'état 4 après le coup d'horloge.

À l'état 1, on se retrouve à l'état « 0 $\overline{P}$ P» après le coup d'horloge. Ce qui signifie que, si $P = 1$ (donc P est vrai, ce qu'on écrit simplement P sur le diagramme), on se retrouve à l'état « 001 » = 1 après le coup d'horloge (donc on reste sur place). Par contre, si P = 0 (donc P est faux, ce qu'on écrit simplement $\overline{P}$ sur le diagramme), on se retrouve à l'état « 010 » = 2 après le coup d'horloge.

De même, à l'état 3, on se retrouve à l'état « $P\,\overline{P}\,1$ » après le coup d'horloge. Ce qui signifie que si $P = 1$, on se retrouve à l'état « 101 » = 5 après le coup d'horloge. Par contre, si $P = 0$, on se retrouve à l'état « 011 » = 3 après le coup d'horloge (on reste sur place).

Finalement, à l'état 4, on se retrouve à l'état « $\overline{P}\ 0\ P$» après le coup d'horloge. Ce qui signifie que, si $P = 1$, on se retrouve à l'état « 001 » = 1 après le coup d'horloge. Par contre, si $P = 0$, on se retrouve à l'état « 100 » = 4 après le coup d'horloge (on reste sur place).

De cette discussion, le diagramme d'état suivant est obtenu :

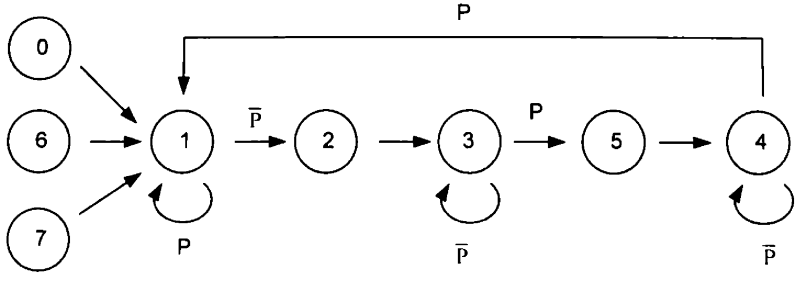


Figure 7.9

Comme on le constate, la technique des variables conditionnelles, la *seule* que nous emploierons, permet d'obtenir une table ÉP-ÉS compacte puisque la variable P est directement incluse dans la section de stimulation (on fera quand même un exemple n'employant pas les variables conditionnelles en fin de chapitre). En n'employant pas cette technique, il faudrait placer P dans la section ÉP (au même titre que Q_C, Q_B, Q_A), ce qui nous ferait quatre colonnes à considérer (Q_C, Q_B, Q_A, P = 4) et donc une table de 16 lignes (= 2^4). Si on avait alors deux variables externes, la table devrait avoir 32 lignes (= 2^5), ce qui serait pour le moins pénible à traiter.

Comme il n'y a pas de décodeur de sortie, les sorties du circuit sont directement les sorties des flip-flops.

Exemple 2 : Nous allons déterminer le diagramme d'état du séquenceur suivant en employant la technique des variables conditionnelles (Q_2 = *MSB*) :

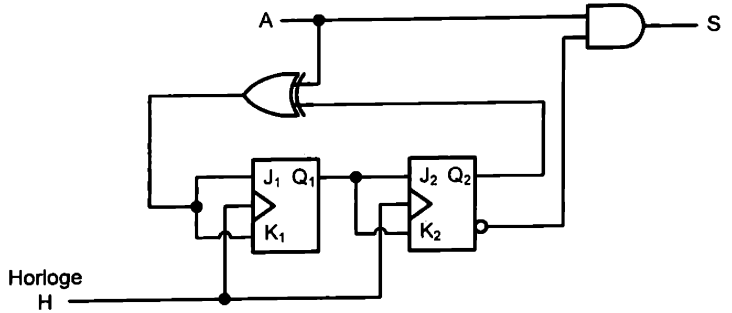


Figure 7.10

Solution : Se référer à la table 7.3A au début du chapitre pour les étapes suivantes.

Étape 1 Les éléments de la machine de Mealy sont bien séparés : deux flip-flops J-K, un décodeur de sortie simple (porte ET) et une entrée externe A.

Étape 2 Les équations des deux flip-flops J-K sont les suivantes, directement obtenues de l'observation du plan du circuit :

$$J_1 = A \oplus Q_2 = A \cdot \overline{Q_2} + \overline{A} \cdot Q_2$$

$$K_1 = J_1$$

$$J_2 = K_2 = Q_1$$

Sortie = $S = A \cdot \overline{Q_2}$

Étape 3 Il faut dresser la table ÉP-ÉS en employant l'entrée externe A comme variable conditionnelle. Comme on a deux flip-flops, il y aura 4 (= 2^2) lignes dans la table ; une par état. On débute à l'état « 00 ». Ainsi, avec $Q_1 = Q_2 = 0$ dans la section ÉP (= État Présent), on obtient les valeurs suivantes de $J_1 = K_1 = A$, $J_2 = K_2 = 0$. Passons ensuite à l'état « 01 ». Ainsi, avec $Q_1 = 1$, $Q_2 = 0$ dans la zone ÉP, on obtient les valeurs suivantes de $J_1 = K_1 = A$,

$J_2 = K_2 = 1$, et ainsi de suite. Les colonnes Q_2 et Q_1 de la section ÉS (= État Suivant) sont obtenues selon la table 5.13 et aussi selon la méthode présentée maintenant.

- Pour le flip-flop Q_2, la table 5.13 avec les valeurs de J_2 et de K_2 suffit. Ainsi, à l'état ⓪, on a $J_2 = K_2 = 0$, et donc, selon la table 5.13 (avec à l'état présent $Q_2 = 0$), on a directement qu'à l'état suivant $Q_2 = 0$ (état de mémorisation du flip-flop).

De même, à l'état ①, on a $J_2 = K_2 = 1$, et donc, selon la figure 5.36, page 213 (avec à l'état présent $Q_2 = 0$), on a directement qu'à l'état suivant $Q_2 = 1$ (état toggle du flip-flop de « 0 » à « 1 »), et ainsi de suite pour l'état ② (mémorisation de « 1 ») et l'état ③ (état toggle de « 1 » à « 0 »).

- Pour le flip-flop Q_1, il faut procéder à l'étude des stimulations et des états suivants possibles en fonction de la valeur de la variable A, comme suit, état par état. Pour chaque état, on bâtit une table en indiquant l'état de départ, puis les valeurs possibles de stimulation selon les valeurs de A ($A = 0$ et $A = 1$), ce qui permet d'établir les états suivants possibles. Par la suite, puisqu'il suffit d'étudier seulement les « 1 » (si ce n'est pas « 1 », c'est donc « 0 »), on procède à une opération d'*Union* des cas qui donnent « 1 » en unissant (opération OU logique) les *valeurs des variables* (A ou $\overline{A}$, selon le cas) qui donnent un « 1 » dans la colonne *États suivants possibles*. Lorsque tous les états ont été étudiés de la sorte, on reporte les valeurs dans la table ÉP-ÉS pour Q_1. La justification de cette procédure vient de ce que ce sont ces variables qui produisent un « 1 » qui spécifieront les états suivants possibles. Une autre façon de voir les choses est de constater qu'on teste toutes les valeurs possibles de la variable ($A = 1$ et $A = 0$) et qu'on les regroupe ensuite en un seul par l'opération d'union.

- à l'état ⓪, les directions possibles sont :

État de départ (Q_1)		Valeur de la variable	J_1	K_1	Mode du flip-flop	États suivants possibles
0	si	A ($A = 1$)	1	1	Toggle	1
0	si	$\overline{A}$ ($A = 0$)	0	0	Mémorisation	0
					Union :	A

Figure 7.11 a)

- à l'état ①, les directions possibles sont :

État de départ (Q_1)		Valeur de la variable	J_1	K_1	Mode du flip-flop	États suivants possibles
1	si	A	1	1	Toggle	0
1	si	$\overline{A}$	0	0	Mémorisation	1
					Union :	$\overline{A}$

Figure 7.11 b)

- à l'état ②, les directions possibles sont :

État de départ (Q_1)		Valeur de la variable	J_1	K_1	Mode du flip-flop	États suivants possibles
0	si	A	0	0	Mémorisation	0
0	si	$\overline{A}$	1	1	Toggle	1
					Union :	$\overline{A}$

Figure 7.11 c)

- à l'état ③, les directions possibles sont :

État de départ (Q_1)		Valeur de la variable	J_1	K_1	Mode du flip-flop	États suivants possibles
1	si	A	0	0	Mémorisation	1
1	si	$\overline{A}$	1	1	Toggle	0
					Union :	A

Figure 7.11 d)

La table ÉP-ÉS finale est alors complète :

	ÉP		Stimulation				ÉS	
Rang	Q_2	Q_1	J_2	K_2	J_1	K_1	Q_2	Q_1
⓪	0	0	0	0	A	A	0	A
①	0	1	1	1	A	A	1	$\overline{A}$
②	1	0	0	0	$\overline{A}$	$\overline{A}$	1	$\overline{A}$
③	1	1	1	1	$\overline{A}$	$\overline{A}$	0	A

Figure 7.12

Étape 4 La table ÉP-ÉS nous permet de dresser le diagramme d'état :

- À l'état ⓪, on se retrouve à l'état ① après le coup d'horloge si $A = 1$ (donc si A), et à l'état ⓪ (donc on reste sur place) après le coup d'horloge si $A = 0$ (donc si $\overline{A}$).
- À l'état ①, on se retrouve à l'état ② après le coup d'horloge si $A = 1$ (donc si A), et à l'état ③ après le coup d'horloge si $A = 0$ (donc si $\overline{A}$).
- À l'état ②, on se retrouve à l'état ② (donc on reste sur place) après le coup d'horloge si $A = 1$ (donc si A) et à l'état ③ après le coup d'horloge si $A = 0$ (donc si $\overline{A}$).
- À l'état ③, on se retrouve à l'état ① après le coup d'horloge si $A = 1$ (donc si A), et à l'état ⓪ après le coup d'horloge si $A = 0$ (donc si $\overline{A}$).

Comme il y a un décodeur de sortie ($S = A\cdot\overline{Q_2}$), il faut calculer la valeur de la sortie. Pour les états ② et ③, on aura toujours 0 puisque $\overline{Q_2} = 0$. Pour les autres états (⓪, ①), $\overline{Q_2} = 1$, et donc $S = A$ tel qu'il a été indiqué.

De cette discussion, le diagramme d'état suivant est obtenu :

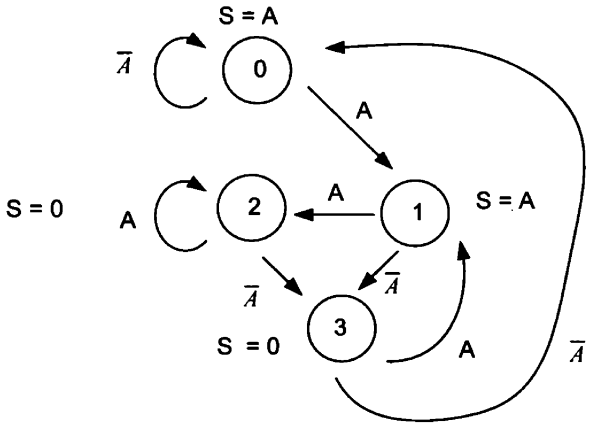


Figure 7.13

Comme on le constate, la technique des variables conditionnelles a permis d'obtenir une table ÉP-ÉS compacte puisque la variable A est directement incluse dans la table. Par ailleurs, le diagramme d'état indique que le circuit implante une fonction relativement complexe.

Exemple 3 : Nous allons déterminer le diagramme d'état du séquenceur suivant en employant la technique des variables conditionnelles (Q_1 = *MSB*) :

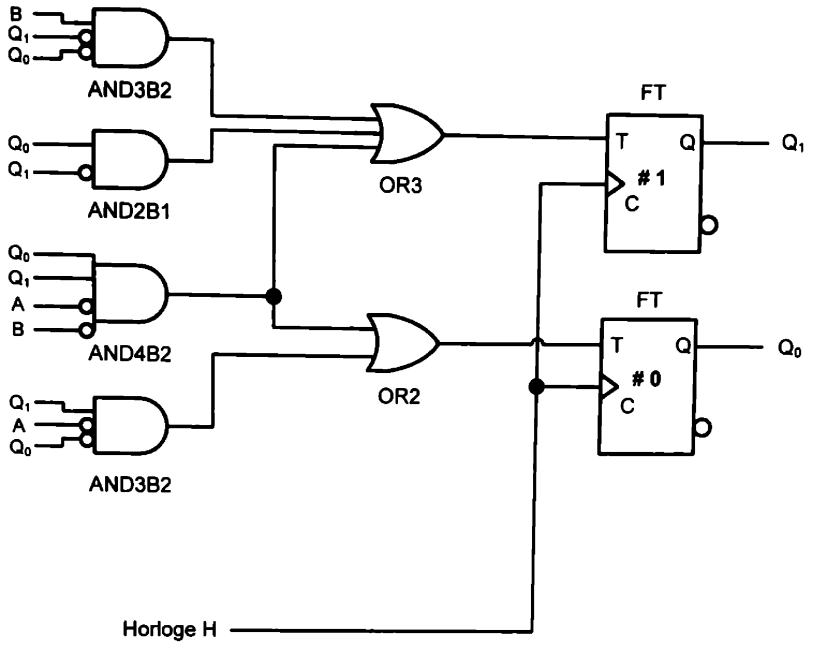


Figure 7.14

Solution : Se référer à l'encadré 7.3A, page 291, pour les étapes suivantes.

Étape 1 Les éléments de la machine de Mealy sont bien séparés : deux flip-flops T, pas de décodeur de sortie et deux entrées externes A et B.

Étape 2 Les équations des deux flip-flops T sont les suivantes, directement obtenues de l'observation du plan du circuit !

$$T_1 = B\overline{Q_1}\,\overline{Q_0} + \overline{Q}_1 Q_0 + \overline{A\,B} Q_1 Q_0$$

$$T_0 = \overline{A} Q_1 \overline{Q_0} + \overline{A}\,\overline{B} Q_1 Q_0$$

Étape 3 Il faut dresser la table ÉP-ÉS en employant les entrées externes A et B comme variables conditionnelles. Comme on a deux flip-flops, il y aura 4 (= 2^2) lignes dans la table ; une par état. On débute à l'état « 00 ». Ainsi, avec $Q_1 = Q_0 = 0$ dans la section ÉP (= État Présent), on obtient les valeurs suivantes de $T_1 = B$, $T_0 = 0$. Passons ensuite à l'état « 01 ». Ainsi, avec $Q_1 = 0$, $Q_0 = 1$ dans la zone ÉP, on obtient les valeurs suivantes de $T_1 = 1$, $T_0 = 0$, et ainsi de suite. Les colonnes Q_1 et Q_0 de la section ÉS (= État Suivant) sont obtenues selon la table 5.13 (lorsque les cas traités n'ont comme paramètres que des « 0 » et des « 1 ») et aussi selon la méthode présentée à l'exemple précédent. Par contre, on peut ramener tous les cas avec variables conditionnelles à seulement deux :

- Si le flip-flop $Q = 0$ à l'état présent et que $T = B$, alors[8] :

État de départ (Q)		Valeur de la variable	T	Mode du flip-flop	États suivants possibles
0	si	B ($B = 1$)	1	Toggle	1
0	si	$\overline{B}$ ($B = 0$)	0	Mémorisation	0
				Union :	B

Figure 7.15 a)

- Si le flip-flop $Q = 1$ à l'état présent et que $T = \overline{A} \cdot \overline{B}$, alors :

État de départ (Q)		Valeur de la variable	T	Mode du flip-flop	États suivants possibles
1	si	$\overline{A} \cdot \overline{B}$ ($\overline{A} \cdot \overline{B} = 1$)	1	Toggle	0
1	si	$\overline{\overline{A} \times \overline{B}}$ ($\overline{A} \times \overline{B} = 0$)	0	Mémorisation	1
				Union :	$\overline{\overline{A} \times \overline{B}} = \overline{\overline{A}} + \overline{\overline{B}} = A + B$ (selon la loi de De Morgan)

Figure 7.15 b)

- Pour les autres cas, il suffit de consulter la figure 5.56, page 227. Par exemple, si Q (état présent) = 1 et que $T = 0$, alors Q (état suivant, après le coup d'horloge) = 1 (mémorisation).

La table ÉP-ÉS finale est alors complète :

	ÉP		Stimulation		ÉS	
Rang	Q_1	Q_0	T_1	T_0	Q_1	Q_0
⓪	0	0	B	0	B	0
①	0	1	1	0	1	1
②	1	0	0	$\overline{A}$	1	$\overline{A}$
③	1	1	$\overline{A} \cdot \overline{B}$	$\overline{A} \cdot \overline{B}$	$A + B$	$A + B$

Figure 7.16

Étape 4 La table ÉP-ÉS nous permet de dresser le diagramme d'état :

- À l'état ⓪, on se retrouve à l'état ② après le coup d'horloge si $B = 1$ (donc si B), et à l'état ⓪ (donc on reste sur place) après le coup d'horloge si $B = 0$ (donc si $\overline{B}$).
- À l'état ①, on se retrouve inconditionnellement à l'état ③ après le coup d'horloge.

[8] Ce cas correspond aussi au cas du rang ② où le flip-flop Q = 0 à l'état présent et que T = $\overline{A}$.

- À l'état ②, on se retrouve à l'état ② (donc on reste sur place) après le coup d'horloge si $A = 1$ (donc si A), et à l'état ③ après le coup d'horloge si $A = 0$ (donc si $\overline{A}$).
- À l'état ③, on se retrouve à l'état ⓪ après le coup d'horloge si $A + B = 0$ (donc si $\overline{A+B} = \overline{A} \cdot \overline{B}$), et à l'état ③ après le coup d'horloge si $A + B = 1$ (donc si $A + B$).

Comme il y n'a pas de décodeur de sortie, les sorties sont directement les sorties des flip-flops (Q_1, Q_0). De cette discussion, le diagramme d'état suivant est obtenu :

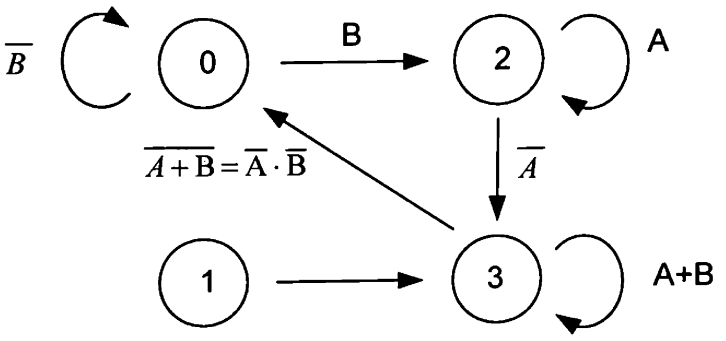


Figure 7.17

Exemple 4 : Qui a dit qu'un circuit à un flip-flop n'est pas possible ? Voici un séquenceur simple contenant un seul flip-flop J-K pour lequel on demande de déterminer le diagramme d'état suivant en n'employant pas la technique des variables conditionnelles (Q_0 = *MSB*) :

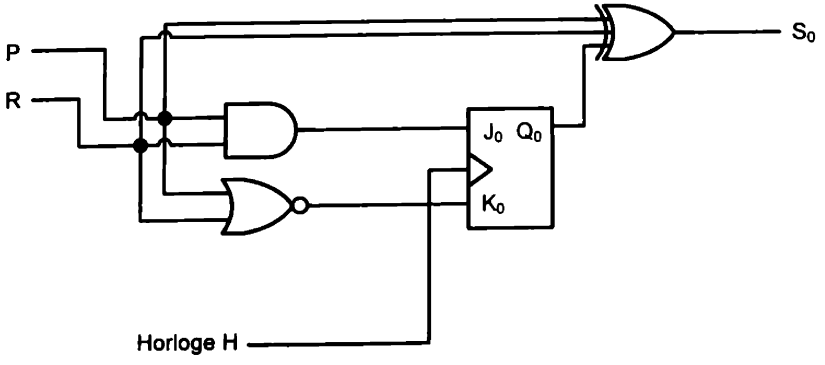


Figure 7.18

Solution : Se référer à la table 7.3A du début de chapitre pour les étapes suivantes (quoique l'on n'emploiera pas ici les variables conditionnelles).

Étape 1 Les éléments de la machine de Mealy sont bien séparés : un flip-flop J-K, un décodeur de sortie et deux entrées externes P et R.

Étape 2 Les équations du flip-flop J-K sont les suivantes, directement établies à partir du plan du circuit :

$$J_0 = P \cdot R$$

$$K_0 = \overline{P + R}$$

$$sortie = P \oplus R \oplus Q_0$$

Étape 3 Il faut dresser la table ÉP-ÉS en employant les entrées externes P et R, non pas comme variables conditionnelles, mais directement dans la table. Comme on a un seul flip-flop et deux variables, il y aura 8 ($= 2^{1+2}$) lignes dans la table, une par combinaison. On débute à l'état « 0 ». Ainsi, avec $Q_0 = 0$ dans la section ÉP (= État Présent), on obtient les valeurs suivantes de Q_0 en fonction de P et de R (figure 7.19). Passons ensuite à l'état « 1 » ; on emploie la même stratégie. La colonne Q_0 de la section ÉS (= État Suivant) est directement obtenue des valeurs de stimulation comme suit et en se rappelant la table 5.13 pour un flip-flop J-K. Cette technique est sans variable conditionnelle.

	ÉP			Stimulation		ÉS	Sortie
Rang	Q_0	P	R	$J_0 = P \cdot R$	$K_0 = \overline{P + R}$	Q_0	Sortie $= P \oplus R \oplus Q_0$
⓪	0	0	0	0	1	0	0
⓪	0	0	1	0	0	0	1
⓪	0	1	0	0	0	0	1
⓪	0	1	1	1	0	1	1
①	1	0	0	0	1	0	0
①	1	0	1	0	0	1	0
①	1	1	0	0	0	1	0
①	1	1	1	1	0	1	1

Figure 7.19

Étape 4 La table ÉP-ÉS nous permet de dresser le diagramme d'état. À l'état ⓪, on voit qu'on reste sur place, sauf si $P \cdot R = 1$ (donc si $\overline{P \cdot R}$). À l'état ①, on reste sur place si $\overline{P + R} = 1$ (donc si $\overline{P + R}$). Quant à la sortie, elle est nulle, à l'état ⓪ si P et R = 0 (et 1 autrement), donc $S_0 = P + R$. À l'état ①, la sortie est 1, si P et R = 1 (et 0 autrement), donc $S_0 = P \cdot R$.

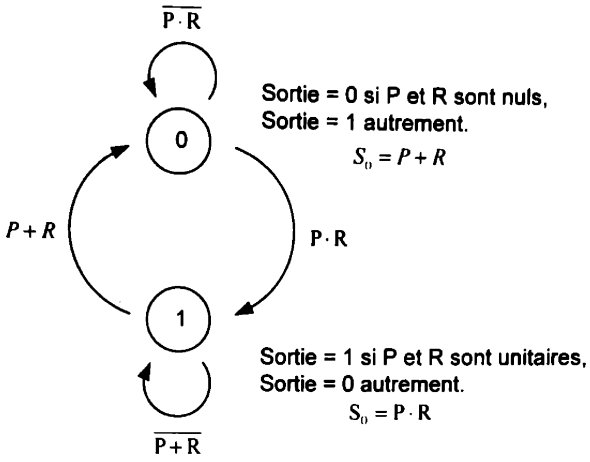


Figure 7.20

Comme on le constate, la technique d'analyse sans variable conditionnelle est plus simple. Par contre, on obtient vite des tables ÉP-ÉS de dimensions telles que cette approche devient impraticable. Selon la situation, il faudra donc choisir la technique la plus appropriée.

7.5 DESIGN DES CIRCUITS SYNCHRONES AVEC ENTRÉES EXTERNES PAR LA MÉTHODE DES VARIABLES CONDITIONNELLES

Tout comme pour l'analyse, cette technique consiste à inclure *directement* les variables dans les sections Stimulation et État Suivant de la table ÉP-ÉS, ce qui permet de réaliser des tables ÉP-ÉS plus compactes. Fidèle à notre habitude, nous allons présenter la méthode à l'aide d'exemples.

Exemple 1 : Réaliser le circuit qui implante le diagramme d'état suivant :

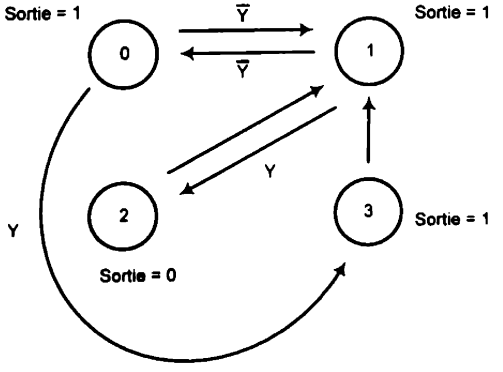


Figure 7.21

Note à propos du diagramme d'état : Parfois, il faut établir le diagramme d'état soi-même. Il est important de noter qu'en établissant un diagramme d'état, tous les cas doivent être couverts. Par exemple, ici à l'état ⓪, on a deux directions possibles ① et ③. La variable prend donc les valeurs Y et $\overline{Y}$, ce qui couvre, à l'état ⓪, toutes les possibilités pour Y, qui est soit vraie (Y), soit fausse ($\overline{Y}$). L'exemple suivant nous présentera un cas plus complexe.

Solution : Il faut suivre les étapes de la table 7.3B au début du chapitre (design) en se rappelant aussi ce dont nous avions parlé au chapitre 6 concernant le design des circuits synchrones sans entrée. Ce que nous avions présenté alors reste valable ; ce n'est qu'un peu plus complet (et complexe) ici avec l'ajout des entrées externes.

Étape 1 La spécification a été obtenue directement de l'énoncé sous la forme d'un diagramme d'état.

Étape 2 *Codage et réduction des états.* Comme on a ici quatre états, deux flip-flops suffisent ($2^2 = 4$), appelons-les Q_1 et Q_0. Par ailleurs, on pourrait employer des flip-flops de type D, T et J-K sans distinction sur la fonctionnalité du circuit. Par contre, comme on l'a déjà vu, les flip-flops D sont les plus faciles à traiter ; nous emploierons donc deux flip-flops D. Les états se codent directement en binaire :

État	Q_1	Q_0
⓪	0	0
①	0	1
②	1	0
③	1	1

Figure 7.22

Étape 3 Il faut maintenant établir la table ÉP-ÉS. La section ÉP s'obtient directement avec la valeur binaire de l'état considéré. La section ÉS s'obtient selon la procédure suivante en examinant le diagramme d'état fourni :

- État ⓪, on établit la liste des directions possibles après le coup d'horloge, donc les valeurs de l'État suivant, et on procède à une opération d'union (OU logique) des cas qui donnent « 1 », ce qui donne :

	Q_1	Q_0		
	0	1	si	$\overline{Y}$
	1	1	si	Y
Union	Y	$Y + \overline{Y} = 1$		

Figure 7.23

Les valeurs obtenues de l'union sont reportées dans la section ÉS pour l'état ⓪.

- État ① : on établit la liste des directions possibles après le coup d'horloge, donc les valeurs de l'État Suivant, et on procède à une opération d'union (OU logique) des cas qui donnent « 1 », ce qui donne :

	Q_1	Q_0		
	0	0	si	$\overline{Y}$
	1	0	si	Y
Union	Y	0		

Figure 7.24

Les valeurs obtenues de l'union sont reportées dans la section ÉS pour l'état ①.

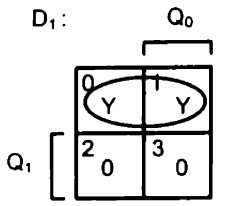
- État ② : on va inconditionnellement à l'état ① ; les valeurs de ÉS pour cet état sont donc $Q_1 = 0$ et $Q_0 = 1$.
- État ③ : on va inconditionnellement à l'état ① ; les valeurs de ÉS pour cet état sont donc $Q_1 = 0$ et $Q_0 = 1$.

Finalement, les valeurs à inscrire pour la section *Stimulation* sont directement recopiées de celles de la section ÉS puisque, pour un flip-flop D, les valeurs d'états suivants sont toujours égales aux valeurs de stimulation (table 5.13) : un petit bonus lié à l'utilisation des flip-flops D. On ajoute aussi les valeurs de la sortie (obtenues du diagramme d'état), ce qui sera utile à l'étape 5.

	ÉP		Stimulation		ÉS		
Rang	Q_1	Q_0	D_1	D_0	Q_1	Q_0	Sortie
⓪	0	0	Y	1	Y	1	1
①	0	1	Y	0	Y	0	1
②	1	0	0	1	0	1	0
③	1	1	0	1	0	1	1

Figure 7.25

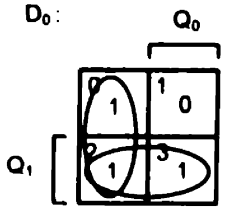
Étape 4 Il faut maintenant établir les tables de Karnaugh pour les D_i selon les valeurs inscrites dans la table ÉP-ÉS. Les tables suivantes sont obtenues :



Ici, il faut voir que Y = 1.Y, ce qui explique la mise en équation (s'il y avait « 1 » plutôt que « Y », on aurait alors eu $D_1 = \overline{Q}_1$).

$D_1 = Y\overline{Q}_1$

Figure 7.26

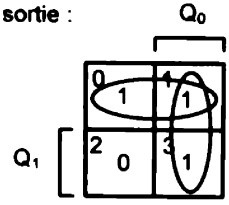


Tout est « régulier » ici :

$D_0 = Q_1 + \overline{Q}_0$

Figure 7.27

Étape 5 Il faut synthétiser la sortie qui vaut (selon la colonne sortie de la table ÉP-ÉS) :



On aurait pu aussi voir directement que

sortie = $\overline{Q}_1 + Q_0$

Figure 7.28

Étape 6 Finalement, le circuit suivant est obtenu :

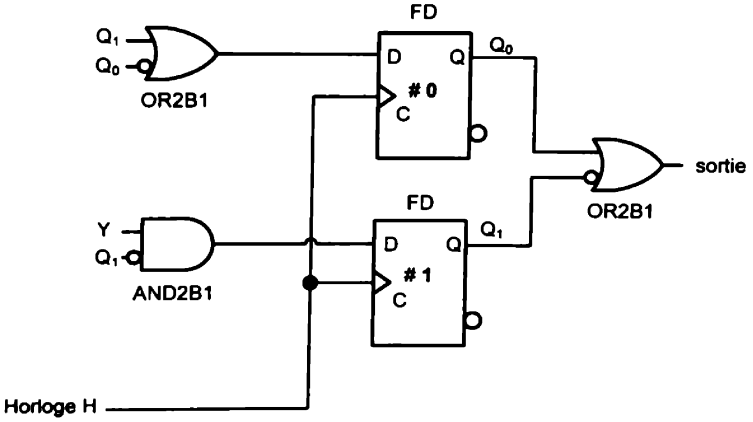


Figure 7.29

Exemple 2 : Réaliser le circuit qui implante le diagramme d'état suivant. Utiliser des flip-flops J-K sans décodeur ni autocorrection. Les sorties sont simplement les valeurs des flip-flops employés.

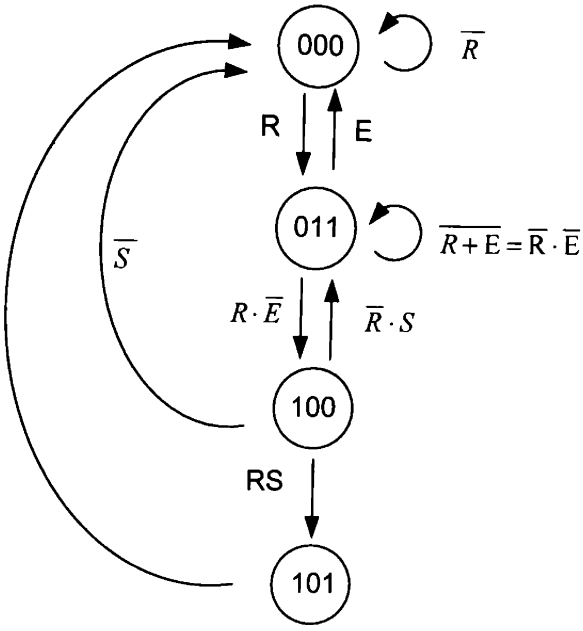


Figure 7.30

Note à propos du diagramme d'état : Ici, plusieurs variables sont impliquées, il faut être bien sûre que, pour un état donné, toutes les possibilités de celles-ci sont couvertes. Autrement, il se pourrait que le circuit « ne sache pas où aller ». Les possibilités correspondent en fait aux mintermes des variables impliquées à cet état. **La règle est simple, il faut que, pour un état donné, *tous* les mintermes des variables impliquées soient utilisés *une seule fois*** (s'ils sont employés plus d'une fois, le circuit « ne saura pas où aller ».). Prenons par exemple l'état « 011 » : on a deux variables (R et E) et trois directions. Avec deux variables et trois directions, on peut dresser la table suivante des mintermes :

R	E	mintermes	Direction : 011 $\overline{R+E} = \overline{R} \cdot \overline{E}$	Direction : 000 $E = \overline{R} \cdot E + R \cdot E$	Direction : 100 $R \cdot \overline{E}$
0	0	$\overline{R} \cdot \overline{E}$	1	0	0
0	1	$\overline{R} \cdot E$	0	1	0
1	0	$R \cdot \overline{E}$	0	0	1
1	1	$R \cdot E$	0	1	0

À noter : **un seul** « 1 » par ligne, pour **chaque** ligne.

Figure 7.31

On remarque que l'auteure de l'exercice a bien fait les choses : les quatre ($2^2 = 4$) mintermes des deux variables impliquées à l'état « 011 » sont tous employés une seule fois.

Vérifions aussi l'état « 100 » avec trois directions et deux variables (R et SP) :

R	S	minterme	Direction : 000 $\overline{S}$	Direction : 011 $\overline{R} \cdot S$	Direction : 101 $R \cdot S$
0	0	$\overline{R} \cdot \overline{S}$	1	0	0
0	1	$\overline{R} \cdot S$	0	1	0
1	0	$R \cdot \overline{S}$	1	0	0
1	1	$R \cdot S$	0	0	1

À noter : **un seul « 1 » par ligne, pour chaque ligne.**

Figure 7.32

Solution : Il faut suivre les étapes de l'encadré 7.3B au début du chapitre (design) en se rappelant aussi ce dont nous avions parlé au chapitre 6 concernant le design des circuits synchrones sans entrée. Ce que nous avions présenté alors reste valable ; ce n'est qu'un peu plus complet (et complexe) ici avec l'ajout des entrées externes.

Étape 1 La spécification a été obtenue directement de l'énoncé sous la forme d'un diagramme d'état.

Étape 2 *Codage et réduction des états.* Comme on a ici quatre états, deux flip-flops suffiraient ($2^2 = 4$). Par contre, puisque aucun décodeur n'est autorisé, nous prendrons trois flip-flops ; appelons-les Q_2 (*MSB*), Q_1 et Q_0. Les flip-flops de type J-K sont imposés ici. Les états se codent directement en binaire :

État	Q_2	Q_1	Q_0
⓪	0	0	0
③	0	1	1
④	1	0	0
⑤	1	0	1

Figure 7.33

Étape 3 Il faut maintenant établir la table ÉP-ÉS. La section ÉP s'obtient directement avec la valeur binaire de l'état considéré. La section ÉS s'obtient selon la procédure suivante en examinant le diagramme d'état fourni :

- État ⓪ : on établit la liste des directions possibles après le coup d'horloge, donc les valeurs de l'État Suivant, et on procède à une opération d'union (OU logique) des cas qui donnent « 1 », ce qui donne :

	Q_2	Q_1	Q_0		
	0	0	0	si	$\overline{R}$
	0	1	1	si	R
Union	0	R	R		

Figure 7.34

Les valeurs obtenues de l'union sont reportées dans la section ÉS pour l'état ⓪.

- État ③ : on établit la liste des directions possibles après le coup d'horloge, donc les valeurs de l'État Suivant, et on procède à une opération d'union (OU logique) des cas qui donnent « 1 », ce qui donne :

	Q_2	Q_1	Q_0		
	0	0	0	si	E
	0	1	1	si	$\overline{R} \cdot \overline{E}$
	1	0	0	si	$R \cdot \overline{E}$
Union	$R \cdot \overline{E}$	$\overline{R} \cdot \overline{E}$	$\overline{R} \cdot \overline{E}$		

Figure 7.35

Les valeurs obtenues de l'union sont reportées dans la section ÉS pour l'état ③.

- État ④ : on établit la liste des directions possibles après le coup d'horloge, donc les valeurs de l'État Suivant, et on procède à une opération d'union (OU logique) des cas qui donnent « 1 », ce qui donne :

	Q_2	Q_1	Q_0		
	0	0	0	si	$\overline{S}$
	0	1	1	si	$\overline{R} \cdot S$
	1	0	1	si	$R \cdot S$
Union	$R \cdot S$	$\overline{R} \cdot S$	$\overline{R} \cdot S + R \cdot S = S$		

Figure 7.36

Les valeurs obtenues de l'union sont reportées dans la section ÉS pour l'état ④.

- État ⑤ : on va inconditionnellement à l'état ⓪ ; les valeurs de ÉS pour cet état sont donc $Q_2 = 0$, $Q_1 = 0$ et $Q_0 = 0$.

Dans le cas d'un flip-flop J-K, un petit travail est nécessaire pour établir, les valeurs à inscrire pour la section *Stimulation*.

Le tout se résume à deux types de transitions à étudier :

1. La transition $0 \rightarrow R$ peut se décomposer comme suit (se référer à la table 5.13 pour le flip-flop J-K) :

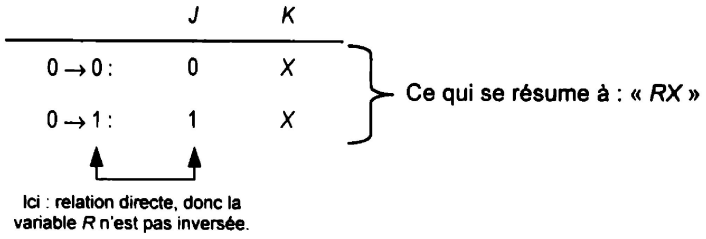


Figure 7.37

On inscrira donc $J = R$ et $K = X$ pour ce type de transition. *Note* : La transition $0 \rightarrow R \cdot \overline{E}$ est évidemment du même type que $0 \rightarrow R$.

2. La transition $1 \rightarrow R$ peut se décomposer comme suit (se référer à la figure 5.56, page 227 pour le flip-flop J-K) :

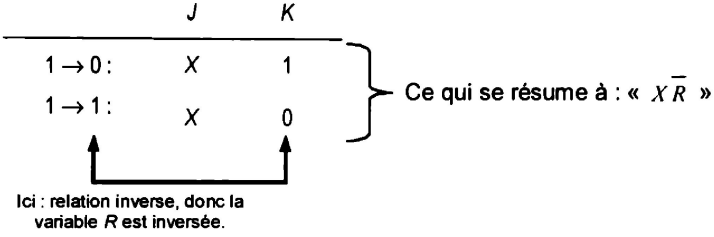


Figure 7.38

On inscrira donc $J = X$ et $K = \overline{R}$ pour ce type de transition.

Les autres types de transitions ($1 \rightarrow 0$, etc.) s'obtiennent directement de la figure 5.56, page 227 (par exemple, pour $1 \rightarrow 0$, il faut $J = X$ et $K = 1$). On est donc prêt à remplir la table ÉP-ÉS, ce qui donne donc :

	ÉP			Stimulation						ÉS		
Rang	Q_2	Q_1	Q_0	J_2	K_2	J_1	K_1	J_0	K_0	Q_2	Q_1	Q_0
⓪	0	0	0	0	X	R	X	R	X	0	R	R
③	0	1	1	$R \cdot \overline{E}$	X	X	$\overline{\overline{R \cdot \overline{E}} = R + E}$ (loi de De Morgan)	X	$\overline{\overline{R \cdot \overline{E}} = R + E}$ (loi de De Morgan)	$R \cdot \overline{E}$	$\overline{R} \cdot \overline{E}$	$\overline{R} \cdot \overline{E}$
④	1	0	0	X	$\overline{R \cdot S} = \overline{R} + \overline{S}$ (loi de De Morgan)	$\overline{R} \cdot S$	X	S	X	$R \cdot S$	$\overline{R} \cdot S$	S
⑤	1	0	1	X	1	0	X	X	1	0	0	0

Figure 7.39

Étape 4 Il faut maintenant établir les tables de Karnaugh pour les J_i et K_i selon les valeurs inscrites dans la table ÉP-ÉS. Les spécifications du problème indiquent que le circuit ne doit pas être autocorrecteur, on peut donc mettre des X aux positions 1, 2, 6 et 7 des tables de Karnaugh (en gras italique ci-dessous), ce qui simplifie bien sûr le problème (toutefois, avec les « risques » dont nous avions parlé précédemment au chapitre 6). Les tables suivantes sont obtenues :

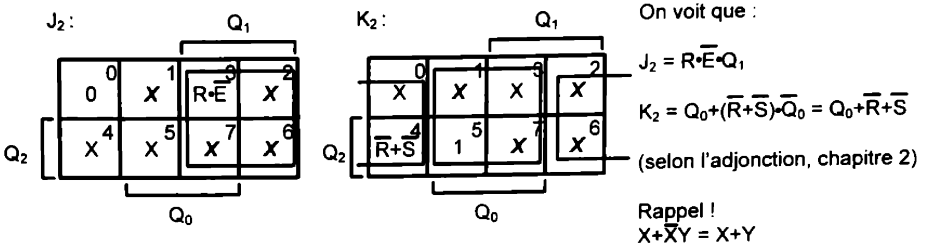


Figure 7.40

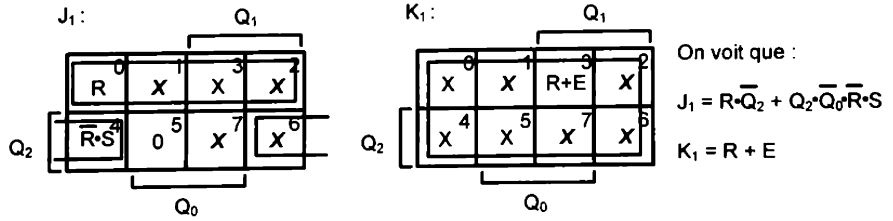


Figure 7.41

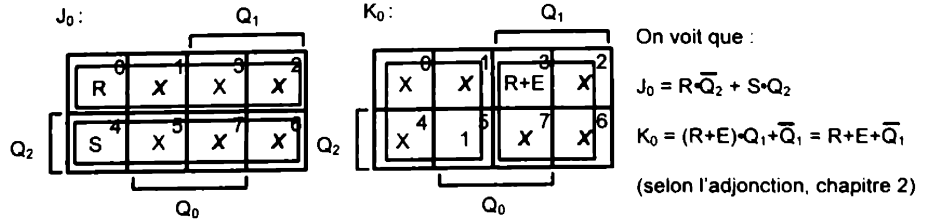


Figure 7.42

Étape 5 Il n'y a pas à synthétiser les sorties puisque celles-ci sont simplement les valeurs des flip-flops (Q_2, Q_1, Q_0) selon l'énoncé du problème.

Étape 6 Finalement, le circuit suivant est obtenu :

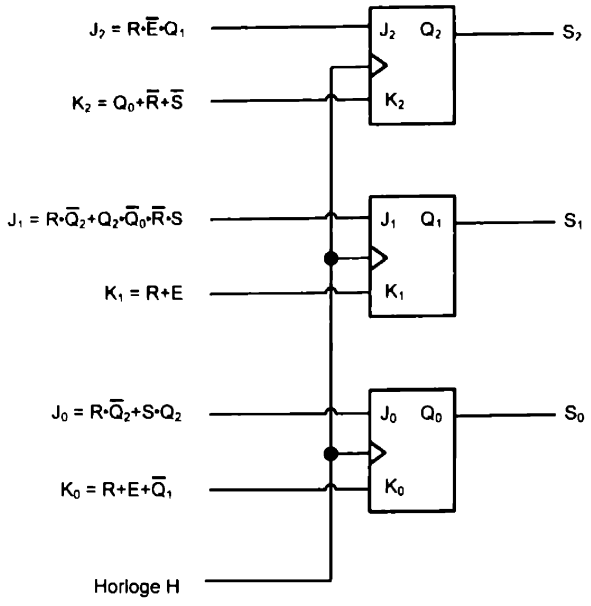


Figure 7.43

Exemple 3 : Réaliser le circuit qui implante un compteur programmable ⓪, ①, ②, ③ avec des flip-flops T. La sortie du circuit doit être décodée avec une sortie par état. Le circuit peut rester à ⓪ ou pas. De là, le compte (+1) se poursuit inconditionnellement jusqu'à ③. À ③, on peut revenir à l'état suivant soit à ②, ①, ⓪ ou rester sur place par simple commande (et de là relancer le comptage).

Solution :

Étape 1 Il faut d'abord traduire cet énoncé en diagramme d'état. On comprend qu'on aura besoin de deux flip-flops T (choix imposé ici) puisqu'on a 4 états ($2^2 = 4$), appelons les $Q_B = MSB$ et Q_A. Puisqu'à ③ nous avons quatre directions possibles (⓪, ①, ②, ③), il nous faudra deux entrées externes pour la commande, appelons les A et B). La table suivante montre qu'il faudra, pour l'état ③, employer les quatre mintermes associés à la combinaison de A et de B :

A	B	mintermes	Direction : 00 $A \cdot B$	Direction : 01 $A \cdot \overline{B}$	Direction : 10 $\overline{A} \cdot \overline{B}$	Direction : 11 $\overline{A} \cdot B$
0	0	$\overline{A} \cdot \overline{B}$	0	0	1	0
0	1	$\overline{A} \cdot B$	0	0	0	1
1	0	$A \cdot \overline{B}$	0	1	0	0
1	1	$A \cdot B$	1	0	0	0

À noter : **un seul** « 1 » par ligne, pour **chaque** ligne.

Figure 7.44

Le choix des mintermes pour les directions est arbitraire ici (c'est le choix de la conceptrice).

Pour l'état ⓪, il nous faut aussi une commande d'arrêt. Prenons la commande A avec arrêt si A (et comptage si $\overline{A}$). Quant à la sortie, nous prendrons simplement un DÉMUX 2 à 4, ce qui nous évitera tout design supplémentaire. De cette discussion, le diagramme d'état peut être établi :

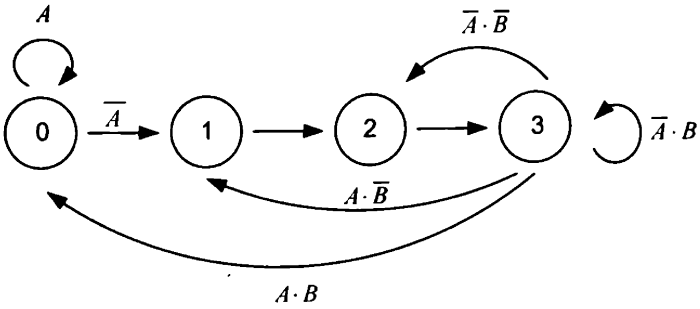


Figure 7.45

Étape 2 *Codage et réduction des états.* Les états se codent directement en binaire ; on le pressentait déjà selon le diagramme d'état.

État	Q_B	Q_A
⓪	0	0
①	0	1
②	1	0
③	1	1

Figure 7.46

Étape 3 Il faut maintenant établir la table ÉP-ÉS. La section ÉP s'obtient directement avec la valeur binaire de l'état considéré. La section ÉS s'obtient selon la procédure suivante en examinant le diagramme d'état fourni :

- État ⓪ : on établit la liste des directions possibles après le coup d'horloge, donc les valeurs de l'État Suivant, et on procède à une opération d'union (OU logique) des cas qui donnent « 1 », ce qui donne :

	Q_1	Q_0		
	0	0	si	A
	0	1	si	$\overline{A}$
Union	0	$\overline{A}$		

Figure 7.47

Les valeurs obtenues de l'union sont reportées dans la section ÉS pour l'état ⓪.

- État ① et ② : il n'y a pas de difficulté puisque les états suivants sont inconditionnels (① → ② et ② → ③).
- État ③ : on établit la liste des directions possibles après le coup d'horloge, donc les valeurs de l'État Suivant, et on procède à une opération d'union (OU logique) des cas qui donnent « 1 », ce qui donne :

	Q_1	Q_0		
	0	0	si	$A \cdot B$
	0	1	si	$A \cdot \overline{B}$
	1	0	si	$\overline{A} \cdot \overline{B}$
	1	1	si	$\overline{A} \cdot B$
Union	$\overline{A} \cdot \overline{B} + \overline{A} \cdot B = \overline{A} \cdot (B + \overline{B}) = \overline{A}$			$A \cdot \overline{B} + \overline{A} \cdot B = A \oplus B$

Figure 7.48

Les valeurs obtenues de l'Union sont reportées dans la section ÉS pour l'état ③.

Dans le cas d'un flip-flop T, un petit travail est nécessaire pour établir les valeurs à inscrire pour la section *Stimulation*.

Le tout se résume à deux types de transitions à étudier :

- La transition $0 \rightarrow \overline{A}$ peut se décomposer comme suit (se référer à la figure 5.56, page 227 pour le flip-flop T) :

	T
$0 \rightarrow 0$:	0
$0 \rightarrow 1$:	1

Ici : relation directe, donc la variable n'est pas inversée (on garde $\overline{A}$)

Figure 7.49

On inscrira donc $T = \overline{A}$ pour ce type de transition.

- La transition $1 \rightarrow \overline{A}$ peut se décomposer comme suit (se référer à la figure 5.56, page 227 pour le flip-flop T) :

	T
$1 \rightarrow 0$:	1
$1 \rightarrow 1$:	0

Ici : relation inverse, donc la variable est inversée (on mettra A)

Figure 7.50

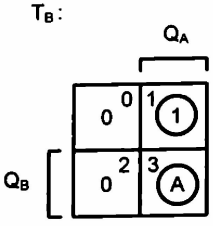
On inscrira donc $T = A$ pour ce type de transition (la transition $1 \rightarrow A \oplus B$ est bien sûr du même type).

Les autres types de transitions ($0 \rightarrow 0$, etc.) s'obtiennent directement de la figure 5.56, page 227 (par exemple, pour $1 \rightarrow$, il faut $T = 0$). On est donc prêt à remplir la table ÉP-ÉS, ce qui donne donc :

	ÉP		Stimulation		ÉS	
Rang	Q_B	Q_A	T_B	T_A	Q_B	Q_A
⓪	0	0	0	$\overline{A}$	0	$\overline{A}$
①	0	1	1	1	1	0
②	1	0	0	1	1	1
③	1	1	A	$\overline{A \oplus B}$	$\overline{A}$	$A \oplus B$

Figure 7.51

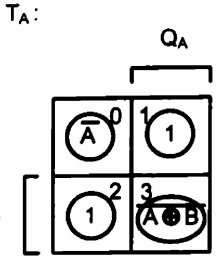
Étape 4 Il faut maintenant établir les tables de Karnaugh pour les T_i selon les valeurs inscrites dans la table ÉP-ÉS. Les tables suivantes sont obtenues :



On voit qu'on a deux termes :

$T_B = A \cdot Q_B \cdot Q_A + \overline{Q_B} \cdot Q_A$

Figure 7.52



On voit qu'on a quatre termes qui peuvent se réduire à trois grâce au OU-exclusif (on pourrait éventuellement réduire tous les termes, ce qui n'est pas le but ici) :

$T_A = \overline{A} \cdot \overline{Q_B} \cdot \overline{Q_A} + Q_B \cdot Q_A \overline{(A \oplus B)} + Q_B \cdot \overline{Q_A} + \overline{Q_B} \cdot Q_A$

$T_A = \overline{A} \cdot \overline{Q_B} \cdot \overline{Q_A} + Q_B \cdot Q_A (A \oplus B) + Q_B \oplus Q_A$

Figure 7.53

Étape 5 Les quatre sorties peuvent s'obtenir facilement avec un DÉMUX, comme on l'a étudié au chapitre 4.

Étape 6 Finalement, le circuit suivant est obtenu.

À noter : les flip-flops T avec mise à 0 (entrée *C – Clear*) que l'on désactive par mise à la masse. Quant au DÉMUX, il est activé avec son entrée d'activation *E* mise au VCC (tension d'alimentation) :

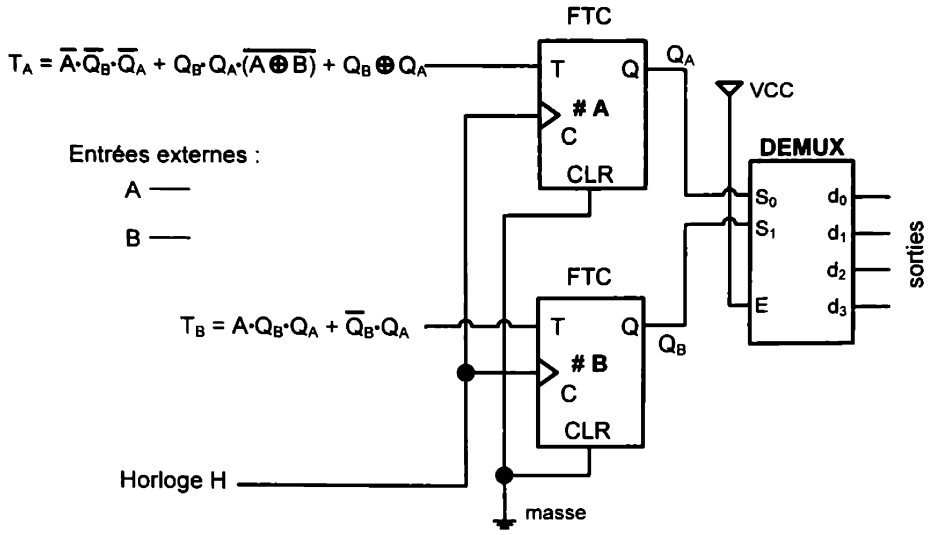


Figure 7.54

Finissons cette section avec un exemple illustrant l'optimisation de l'auto-correction.

Exemple 4 : Réaliser le circuit qui implante la séquence ①, ②, ⓪ illustrée à la figure suivante au moyen de deux flip-flops D. Il y a une variable externe *P*. Les sorties du circuit sont directement les sorties des flip-flops.

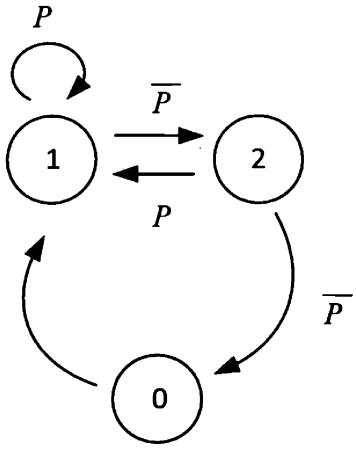


Figure 7.55

Solution :

Étape 1 Le diagramme d'état est déjà fourni

Étape 2 *Codage et réduction des états.* Le choix (et le nombre de flip-flops) étant imposés, on peut facilement coder les 3 états présents directement en binaire avec deux bits ($2^2 = 4$). On remarque cependant que l'état ③ ne faisant pas partie de la liste imposée, il y aura donc besoin d'auto-correction. Appelons les deux bits $Q_1 = MSB$ et Q_0.

État	Q_B	Q_A
⓪	0	0
①	0	1
②	1	0
③	1	1

Figure 7.56

Étape 3 Il faut maintenant établir la table ÉP-ÉS. Considérons d'abord le cas de *l'auto-correction optimisée.* La section ÉP s'obtient directement avec la valeur binaire de l'état considéré. La section ÉS s'obtient selon la procédure suivante en examinant le diagramme d'état fourni :

- État ⓪ : L'état suivant est inconditionnel (①), on écrit donc simplement (pour l'ÉS) : Q_1=0 et Q_0= 1.
- État ① : On établit la liste des directions possibles après le coup d'horloge, donc les valeurs de l'État Suivant, et on procède à une opération d'union (OU logique) des cas qui donnent « 1 », ce qui donne :

	Q_1	Q_0		
	1	0	si	$\overline{P}$
	0	1	si	P
Union	$\overline{P}$	P		

Les valeurs obtenues de l'Union sont reportées dans la section ÉS pour l'état ①.

- État ② : on établit la liste des directions possibles après le coup d'horloge, donc les valeurs de l'État Suivant, et on procède à une opération d'union (OU logique) des cas qui donnent « 1 », ce qui donne :

	Q_1	Q_0		
	0	0	si	$\overline{P}$
	0	1	si	P
Union	0	P		

Les valeurs obtenues de l'Union sont reportées dans la section ÉS pour l'état ②.

• État ③ : Pour l'état 3, on place donc des *X-temporaires*, cas de *l'auto-correction optimisée* (Table 7.3B).

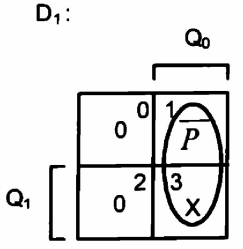
Notre étude des directions pour chaque état nous permet de remplir la table ÉP-ÉS, ce qui donne donc (*auto-correction-optimisée*). On se rappelle aussi que pour un flip-flop D, la stimulation est toujours égale à l'État suivant (ce qui facilite beaucoup le travail!) :

	ÉP		Stimulation		ÉS	
Rang	Q_1	Q_0	D_1	D_0	Q_1	Q_0
⓪	0	0	0	1	0	1
①	0	1	$\overline{P}$	P	$\overline{P}$	P
②	1	0	0	P	0	P
③	1	1	X	X	X	X

Figure 7.57

Étape 4 Il faut maintenant établir les tables de Karnaugh pour les D_i selon les valeurs inscrites dans la table ÉP-ÉS. Les tables suivantes sont obtenues :

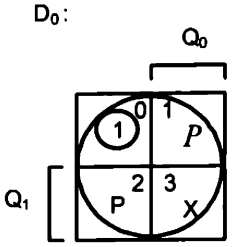
D_1 :



On voit qu'on a un terme en posant que $X = \overline{P}$

$$D_1 = Q_0\overline{P}$$

Figure 7.58



On voit qu'on a deux termes en posant que X = P

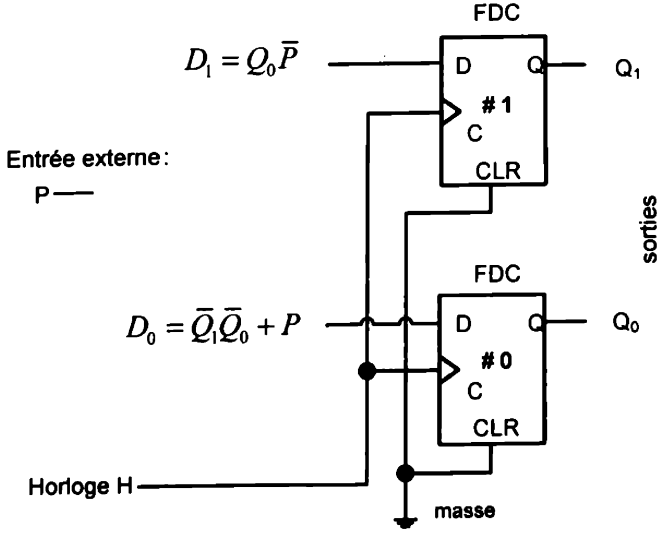
$$D_0 = \bar{Q}_1\bar{Q}_0 + P$$

Figure 7.59

En effet, il faut absolument considérer le «1» seul (il est là et doit donc *toujours* être considéré peu importe la valeur de *P*), on a donc le terme $D_0 = \bar{Q}_1\bar{Q}_0$. Par la suite il y a les trois P (puisqu'on pose que $X = P$), dans ce cas, on peut les associer au «1» car si $P = 1$, il y a des «1» dans toutes les cases (et on peut les grouper ensemble!) et si $P = 0$, alors le regroupement dans l'équation de D_0 disparaît (= 0), simplement. Au final cela donne $D_0 = \bar{Q}_1\bar{Q}_0 + P$.

Étape 5 Les sorties sont directement les sorties des deux flip-flops (selon les spécifications fournies).

Étape 6 Finalement, le circuit suivant est obtenu (*auto-correction optimisée*).



Une question. Que se passe-t-il à l'état ③? A l'état ③, selon les équations des D_i, on a $D_1 = \bar{P}$ et $D_0 = P$. Donc si P = 1 on aboutit à l'état ① et si P = 0 on aboutit à l'état ②. Dans les deux situations on se retrouve dans la séquence (il n'aurait pas fallu qu'on se retrouve

à l'état ③, dans ce cas on se serait retrouvé dans un cul-de-sac!). Si cela était arrivé, on aurait simplement modifié les *X-temporaires* dans la table des D_i de façon à éviter le cul-de-sac (voir le cas d'*auto-correction simple* discuté un peu plus bas).

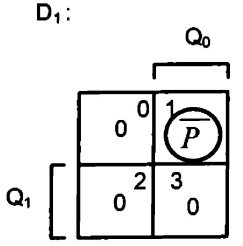
Il est intéressant de calculer le coût de ce circuit en termes de portes double, pour la logique de stimulation. Cela donne 3 portes doubles.

A titre de comparaison, on peut se demander quel serait le coût pour une *auto-correction simple*, en assignant l'état de retour (*ground state*) à l'état inemployé (③ ici). Pour cela on choisit disons l'état ⓪ comme état de retour. La table ÉP-ÉS devient (seule la dernière ligne change):

	ÉP		Stimulation		ÉS	
Rang	Q_1	Q_0	D_1	D_0	Q_1	Q_0
⓪	0	0	0	1	0	1
①	0	1	$\overline{P}$	P	$\overline{P}$	P
②	1	0	0	P	0	P
③	1	1	0	0	0	0

Figure 7.60

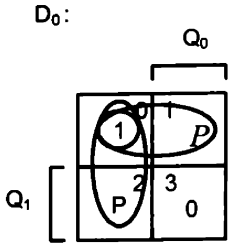
Il faut ensuite établir les tables de Karnaugh pour les D_i selon les valeurs inscrites dans la table ÉP-ÉS modifiée. Les tables suivantes sont obtenues :



On voit qu'on a un terme

$$D_1 = \overline{Q}_1 Q_0 \overline{P}$$

Figure 7.61



On voit qu'on a trois termes

$$D_0 = \bar{Q}_1\bar{Q}_0 + \bar{Q}_1 P + \bar{Q}_0 P$$

Figure 7.62

Les deux équations sont plus complexes. En termes d'équivalent porte double, cela donne une triple donc 2 doubles pour D_1 et 3 doubles plus une triple donc 5 doubles pour D_0. Au total, cela fait 7 portes doubles équivalentes, pour la logique de stimulation. Comme il fallait s'y attendre, l'*auto-correction optimisée* est plus avantageuse (en termes de minimisation) que l'*auto-correction simple*!

7.6 SYNTHÈSE AVEC MSI ET LSI

Comme les sections précédentes l'ont indiqué, la réalisation d'un compteur synchrone en partant de flip-flops élémentaires est une tâche qui peut vite s'avérer complexe. Heureusement, l'emploi de circuits plus élaborés permet de réduire les efforts requis. En effet, en tablant sur la disponibilité de circuits tels des multiplexeurs, compteurs, PLA et PAL, on transfert en partie la complexité de la réalisation dans ceux-ci. La figure 7.63 illustre l'utilisation des multiplexeurs pour la préparation de l'état suivant. On se souvient qu'au chapitre 4, on avait vu que les MUX permettaient d'implanter facilement des tables de vérité ; cela réduit donc le travail de synthèse de la logique combinatoire de stimulation des flip-flops (les éléments de mémoire). D'ailleurs, comme on l'a vu dans les exemples réalisés précédemment, on a parfois simplement écrit la fonction de stimulation sans en détailler l'ensemble correspondant de portes logiques. L'utilisation des MUX va dans le même sens.

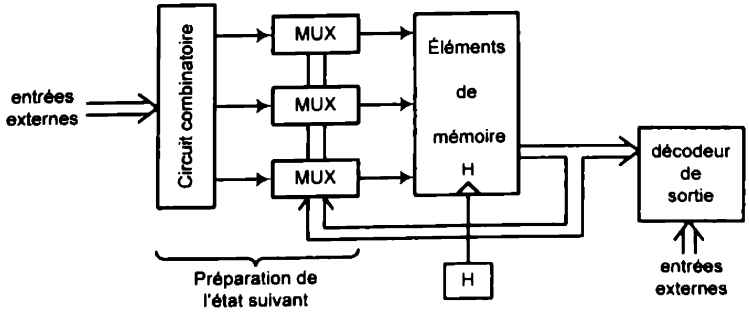


Figure 7.63

Plus loin, nous illustrerons cette utilisation des MUX. Toujours dans la même logique de réduction de l'effort requis, on peut substituer aux simples flip-flops des éléments plus

complexes et tout prêts : des compteurs synchrones. La prochaine section va d'ailleurs en présenter quelques-uns.

EMPLOI DE COMPTEURS

Principaux types de compteurs synchrones

La liste suivante regroupe les principaux compteurs synchrones qui sont d'intérêt pour nous. On se rappelle qu'un compteur synchrone est un compteur dans lequel tous les flip-flops sont reliés à une même horloge (par opposition à une approche de type RCO – *Ripple Carry Output* –, discutée précédemment, où un signal d'horloge se propage de flip-flop en flip-flop). La liste inclut aussi quelques caractéristiques de ceux-ci, comme leur type (binaire, décimal), la possibilité de mise en cascade, la disponibilité d'une commande LOAD ou *PRESET* (charge, en français : on charge le compteur avec une valeur, ce qui permet de « briser » la séquence), le comptage bidirectionnel et la mise à 0 (synchrone – au coup d'horloge suivant – ou asynchrone, donc immédiate). Ces compteurs sont disponibles sous forme de circuits intégrés MSI de la famille TTL, ou encore sous forme de librairie dans Xilinx Foundation, dans le Xilinx WebPack ou dans l'outil LOGISIM. Pour des montages sur *breadboards*, nous recommandons l'utilisation du 74163 puisqu'il s'agit d'un circuit *totalement* synchrone (ou le 74161, ou le 74191[9]). Pour des circuits incorporés dans un FPGA (avec Xilinx Foundation), nous recommandons le CB*x*CLED, où x = 2, 4, 8 ou 16 bits selon le besoin. Dans les sections suivantes, nous allons nous intéresser particulièrement au CB*x*CLED. Par contre, puisque les CB*x*CLED et 74163 ont des fonctionnalités similaires, il sera facile de faire migrer un design de l'un à l'autre. Par ailleurs, puisque les CB*x*CLED ne diffèrent entre eux que par le nombre x de bits, on pourra facilement étendre les designs à plus de bits. La librairie du CB4CLED ainsi que la librairie COUNTER de LOGISIM sont décrites plus bas. Dans le reste du chapitre, nous travaillerons avec le CB4CLED à quatre bits (mais il sera facile de migrer vers une autre plate-forme selon le besoin).

---

[9] L'entrée Charge (Load) étant asynchrone sur ce circuit, il faudra être prudent pour éviter les aléas! Une solution peut être d'adjoindre une bascule D à cette entrée.

Numéro	Xilinx/TTL	Nombre de bits	Compte +1 et −1 (*up/down*)	Charge (*load* ou *preset*)	Activation du comptage (*Enable*)	Mise en cascade	Mise à 0
74160	Xilinx	4 bits, en décimal	Non	Oui	Oui, deux entrées	Oui	Asynchrone
74162	Xilinx	4 bits, en décimal	Non	Oui	Oui, deux entrées	Oui	Synchrone
74161[10]	TTL,Xilinx	4 bits, en binaire	Non	Oui	Oui, deux entrées	Oui	Asynchrone
74163[11]	TTL, Xilinx	4 bits, en binaire	Non	Oui	Oui, deux entrées	Oui	Synchrone
74191[12]	TTL	4 bits, en binaire	Oui	Oui (asynchrone)	Oui, une entrée	Oui	Aucune
CBxCLED, x = 2,4,8,16	Xilinx	2,4,8 ou 16 bits, en binaire	Oui	Oui	Oui, une entrée	Oui	Asynchrone
Counter	LOGISIM	1 à 32 bits (binaire)	Oui	Oui	Oui, une entrée	Oui	Asynchrone

Figure 7.64 a)

Les figures 7.64 b) à 7.64 d) montrent les principaux types de compteurs disponibles dans les logiciels Xilinx Foundation et LOGISIM. Pour les CB8CLED et CB16CLED, figure 7.64 c), on remarquera la notation employée pour la définition d'un bus de données sur 8 ou 16 bits, ce qui permet un schéma plus compact (et aussi pour le compteur de LOGISIM illustré en format 3 bits). Le compteur 74F191[13] est aussi présenté à la figure 7.64 e).

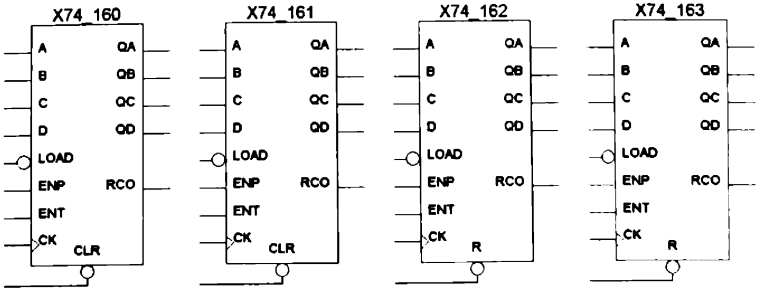


Figure 7.64 b)

[10] http://www.standardproducts.philips.com/products/fast/pdf/74f161a_163a.pdf
[11] http://www.standardproducts.philips.com/products/fast/pdf/74f161a_163a.pdf
[12] http://www.standardproducts.philips.com/products/fast/pdf/74f191.pdf (avec aussi un RCO – *Ripple Carry Output* - et un TC – *Terminal Count*).
[13] http://www.standardproducts.philips.com/products/fast/pdf/74f191.pdf

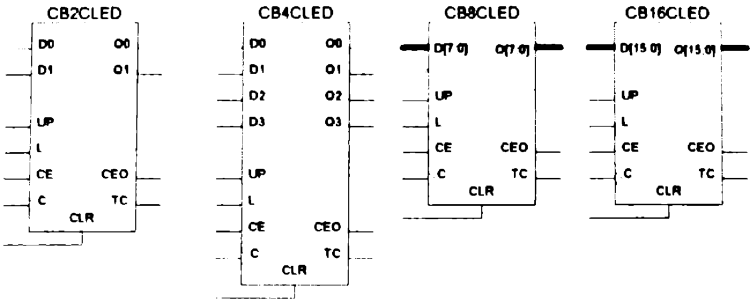


Figure 7.64 c)

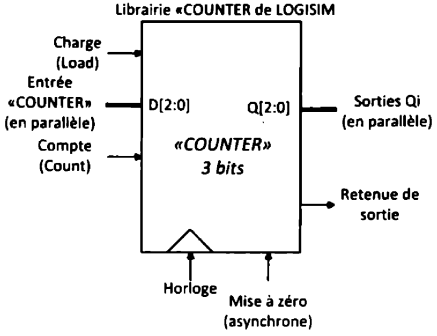


Figure 7.64 d)

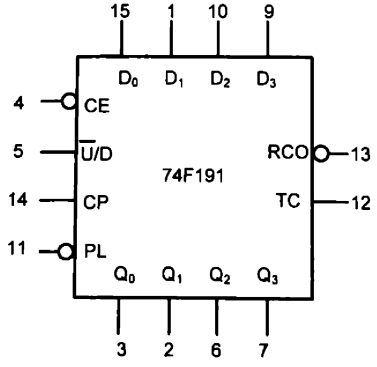


Figure 7.64 e) Compteur 74F191

LE CB4CLED

Le CB4CLED est le compteur que nous emploierons. Nous allons expliquer son fonctionnement dans cette section. Ce compteur est disponible sous forme de librairie dans Xilinx Foundation ou Xilinx WebPack. Il est à noter que son fonctionnement est similaire au 74191, qui peut donc servir si la lectrice désire réaliser un montage sur une plaquette de montage (*breadboard*) avec des composantes discrètes. Ce compteur existe aussi – avec le même fonctionnement – en version 2, 8, 16 bits tel que montré à la figure 7.64c) (CB2CLED, CB8CLED, CB16CLED).Le sigle CB4CLED signifie :

C	*Counter* - compteur
B	*Binary* - binaire
4	4 bits (aussi disponibles en 2, 8, 16 bits : CB2CLED, CB8CLED, CB16CLED)
C	*Clear* – mise à 0 (asynchrone)
L	*Load* (ou *Preset*) – charge une valeur
E	*Enable* – Activation du comptage
D	*Directionnal* – directionnel (compte +1 ou −1)

Le schéma du CB4CLED est repris ici avec les détails sur son fonctionnement :

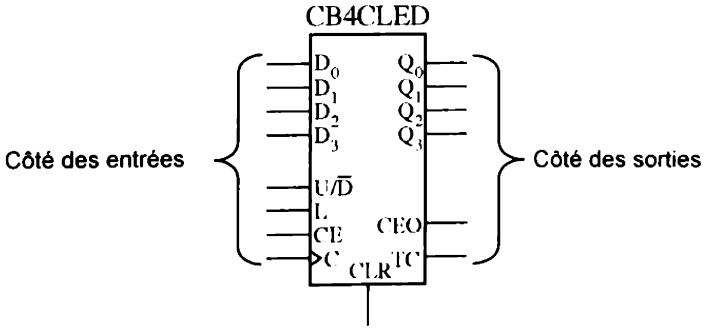


Figure 7.65

Commande	Fonctionnement / Remarque
Mise à 0 / *Clear* : *CLR*	Fonctionnement asynchrone : **ne pas l'employer sauf en cas d'initialisation.**
Choix de direction (*Up/Down*) : $U / \overline{D}$	Si *CE* = 1 : +1 (incrémente) si $U / \overline{D}$ à 1, −1 (décrémente) si $U / \overline{D}$ à 0. Si *CE* = 0 : pas d'effet sur le comptage puisque l'on ne compte pas, mais effet sur *TC* et *CEO* (voir plus bas).
Chargement d'une nouvelle valeur / *Load* : *L*	Charge la valeur présente en D_3 D_2 D_1 D_0. Commande synchrone, **prioritaire sur *CE*.** Au coup d'horloge suivant, ces valeurs apparaîtront aux sorties Q_i.
Activation du comptage / *Count Enable* : *CE*	Comptage activé si à 1, ne compte pas si 0.
Sortie de comptage /*Count Enable Output* : *CEO*	Indique quand les sorties Q_i sont à 0000 ou à 1111 selon la valeur de *U*, à condition que *CE* soit 1. Donc : Si *CE* = 1, on a : *CEO* = 1 si *U* = 1, et sorties Q_i = 1111, *CEO* = 1 si *U* = 0, et sorties Q_i = 0000 ; Si *CE* = 0, *CEO* = 0 (toujours).
Sortie de fin de comptage / *Terminal Count* : *TC*	Indique quand les sorties Q_i sont à 0000 ou à 1111 selon la valeur de U (peu importe la valeur de CE). *TC* = 1 si *U* = 1, et sorties Q_i = 1111, *TC* = 1 si *U* = 0, et sorties Q_i = 0000.
Horloge / *Clock* :*C*	Entrée de l'horloge. Le compteur est synchrone, donc tous les flip-flops réagissent simultanément.

Figure 7.66

La figure 7.67 montre un diagramme temporel reprenant le fonctionnement du CB4CLED selon ses principaux modes de fonctionnement. Une étude approfondie de cette figure permet de bien en saisir le fonctionnement qui est résumé dans la table précédente.

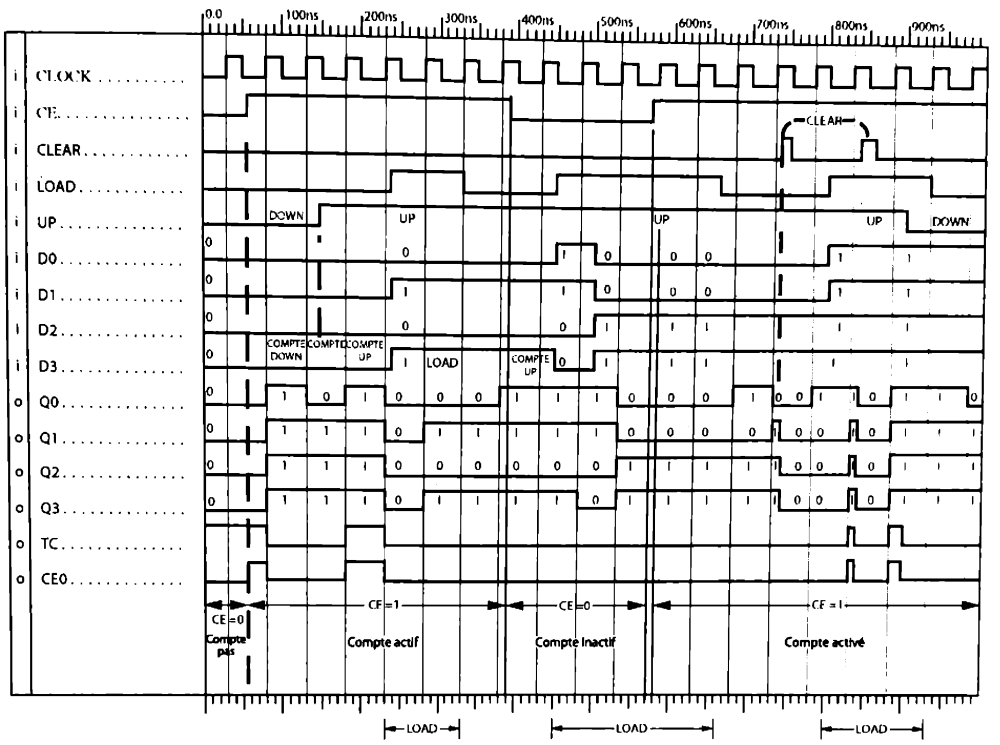


Figure 7.67

Avant de se lancer dans l'analyse ou le design de circuits logiques utilisant des compteurs, il est recommandé de se familiariser avec leur fonctionnement qui sans être complexe offre néanmoins des commandes élaborées par rapport à de simples flip-flops. À titre d'aide, la figure 7.68 montre le plan réalisé dans Xilinx Foundation pour réaliser des tests simples sur un CB2CLED (deux bits sont plus faciles à gérer que quatre bits). On a employé la technique des étiquettes (*label* en anglais) pour le montage du circuit. La mise à 0 a été fixée à zéro, il n'est donc bien sûr pas possible de tester cette fonction (qui met tous les flip-flops à zéro immédiatement, sans attendre le prochain coup d'horloge).

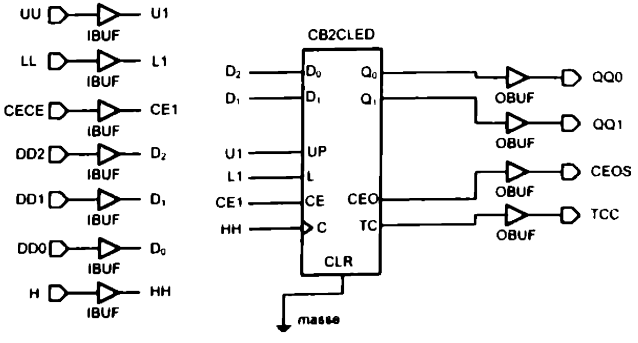


Figure 7.68 Figure illustrant un circuit démonstrateur réalisé dans Xilinx Foundation pour les tests d'un CB2CLED

La figure 7.69 est un chronogramme illustrant les diverses fonctions de ce compteur synchrone. Sur le circuit, les valeurs indiquées des variables correspondent à celles de la toute fin du chronogramme. *Note* : On a employé DD2 et DD1 pour commander respectivement D0 et D1 (les mêmes entrées servent aussi pour un autre circuit).

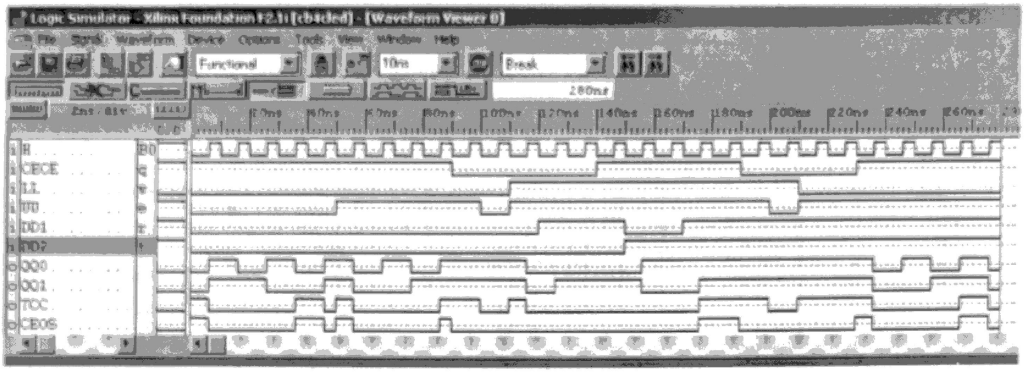


Figure7.69

LE «COUNTER» DE LOGISIM

Le compteur COUNTER de LOGISIM (figure **7.64 d**) est configurable de 1 à 32 bits, son horloge peut être configurée sur front montant ou descendant et lors d'un débordement la valeur de compte peut continuer, s'arrêter, charger la valeur présente aux entrées Di. Le compteur de LOGISIM est donc assez différent du 74163 (ou du CBiCLED). Notons particulièrement :

1) Il y a un mode de compte -1 (DOWN).

2) Le LOAD (Charge) n'est pas prioritaire sur le CE (count).

3) La mise à zéro est asynchrone (donc mieux vaut ne pas s'en servir!).

4) Il y a quatre modes d'opération selon les valeurs de LOAD (Charge) et de COUNT (Compte) :

LOAD (Charge)	COUNT (Compte)	Action
0	0	Pas de changement au compteur
0	1	Le compteur s'incrémente
1	0	Le compteur charge les valeurs présentes aux entrées Di
1	1	Le compteur décrémente

Pour rendre compatible le Compteur de LOGISIM et le 74163, on suggère de ne pas tenir compte de la priorité du LOAD (dans le 74163), mais plutôt de vraiment spécifier ce que l'on veut qui se passe avec LOAD et COUNT selon la table ci-dessus en excluant bien sûr le mode de décrémentation. De plus, puisque le LOAD est inversé (actif bas) sur le 74163, il faudra prévoir un inverseur. Finalement, sur le 74163, il y a deux entrées d'activation du comptage, CET (*Count Enable Trickle*) et CEP (*Count Enable Parallel*). On peut simplement les relier entre elles.

Notons en terminant qu'il est intéressant de savoir qu'une librairie supplémentaire a été développée pour LOGISIM, elle contient plusieurs circuits de la famille 74XXX incluant le 74163. Il est donc bien sûr recommandé d'employer cette librairie si le but final d'une simulation est de vérifier le fonctionnement d'un circuit basé sur le 74163!

7.7 ANALYSE DE MACHINES À NOMBRE FINI D'ÉTATS AVEC COMPTEURS SYNCHRONES

L'analyse de tels circuits se fait essentiellement comme les analyses que nous avons effectuées plus tôt dans ce chapitre et présentées à la section 7.3 (table 7.3A). Évidemment, dans le cas d'un compteur synchrone de type CBxCLED ou 74191, il faudra analyser les diverses commandes telles **L**, **CE**, **U** ainsi que les bits de données D_i. Cette fois encore, nous allons illustrer l'analyse de ces circuits à l'aide d'un exemple résolu, ce qui permettra d'expliquer toutes les étapes.

Exemple : Soit le circuit suivant dessiné dans Xilinx Foundation. On demande de fournir le diagramme d'état correspondant.

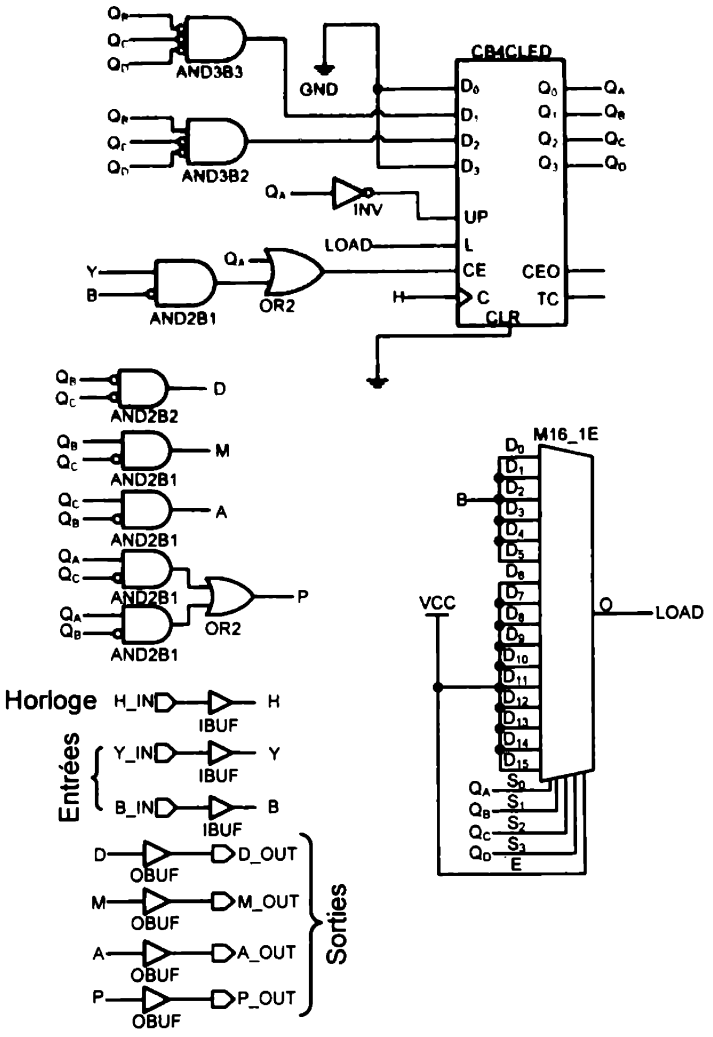


Figure 7.70

Solution : Se référer à la table 7.3A au début du chapitre pour les étapes suivantes.

Étape 1 Il faut identifier les éléments de la machine. Il s'agit d'un circuit avec un CB4CLED dont les sorties sont appelées *D*, *M*, *A* et *P* (avant les tampons de sorties). Les bits Q_0 à Q_3 sont ici appelés Q_A à Q_D. Il y a deux entrées externes, *B* et *Y* (après les tampons d'entrées), qui seront nos variables conditionnelles. Une horloge H est aussi présente. On note aussi que la commande *L* utilise un MUX 16 à 1. De la logique combinatoire simple sert pour les lignes de commande *CE* et *U*. On note que la conceptrice du circuit a employé la technique des étiquettes pour le tracé de celui-ci, ce qui rend le schéma plus facile à lire.

Étape 2 On écrit les équations pour L, CE, U ainsi que pour les bits de données D_i. Du schéma, on tire que :

Pour les lignes de commandes et les bits de données D_i :

- $U = \overline{Q_A}$;
- $CE = Q_A + Y \cdot \overline{B}$.
- Pour L, il faut étudier ce que donne le MUX 16 à 1 :

Valeur de L	Pour les états
B	0, 1, 2, 3, 4, 5
1	6 à 15

Figure 7.71

- Pour les bits de données D_i, on obtient :

$D_0 = 0$

$D_1 = \overline{Q_D} \cdot \overline{Q_C} \cdot \overline{Q_B}$ → donc les mintermes 0 et 1 (Q_A étant quelconque ici)

$D_2 = \overline{Q_D} \cdot \overline{Q_C} \cdot Q_B$ → donc les mintermes 2 et 3 (Q_A étant quelconque ici)

$D_3 = 0$

Pour les sorties :

$D = \overline{Q_B} \cdot \overline{Q_C}$

$M = Q_B \cdot \overline{Q_C}$

$A = \overline{Q_B} \cdot Q_C$

$P = Q_A \cdot \overline{Q_C} + Q_A \cdot \overline{Q_B} = Q_A \cdot (\overline{Q_C} + \overline{Q_B})$

Étape 3 On doit maintenant dresser la table État Présent → État Suivant (ÉP-ÉS) en se servant de la technique des variables conditionnelles. On note que Q_D ne sert jamais sauf pour activer L aux états 6 à 15, ce qui nous ramène alors (valeurs des $D_i = 0$ si $Q_D = 1$) à l'état 0000. Cela correspond donc à l'autocorrection de cette machine. La table ÉP-ÉS aura donc six lignes servant pour l'activité normale du circuit (états 0 à 5 inclusivement). Les autres 10 lignes servant pour l'autocorrection, on les indique pour la valeur des sorties (états 6 à 15). Pour ces lignes, il est inutile de calculer CE et U, car L prioritaire est toujours à 1 : cela nous sauve un peu de travail.

La table se monte par section : d'abord la section *État Présent* (en écrivant les valeurs binaires de tous les états d'intérêt), puis la section *Stimulation* à partir des équations obtenues à l'Étape 2 (en incluant aussi les D_i). Finalement, la section *État Suivant* est remplie, état par état, à partir des valeurs des deux sections précédentes et en effectuant

l'étude notée plus bas. Les valeurs des sorties sont indiquées selon les équations obtenues à l'étape 2.

	État Présent				Stimulation D_i							État Suivant	Sorties
Rang	Q_D	Q_C	Q_B	Q_A	L	CE	U	D_3	D_2	D_1	D_0	Actions	D,L,A,P
0	0	0	0	0	B	$Y \cdot \overline{B}$	1	0	0	1	0	- Charge si B (avec D_i = 0010) - Compte si $Y \cdot \overline{B}$ (+1) - Ne compte pas si $\overline{Y \cdot \overline{B}}$	1, 0, 0, 0
1	0	0	0	1	B	1	0	0	0	1	0	- Charge si B (avec D_i = 0010) - Compte si $\overline{B}$ (−1)	1, 0, 0, 1
2	0	0	1	0	B	$Y \cdot \overline{B}$	1	0	1	0	0	Comme à l'état 0, mais avec D_i = 0100	0, 1, 0, 0
3	0	0	1	1	B	1	0	0	1	0	0	Comme à l'état 1, mais avec D_i = 0100	0, 1, 0, 1
4	0	1	0	0	B	$Y \cdot \overline{B}$	1	0	0	0	0	Comme à l'état 0, mais avec D_i = 0000	0, 0, 1, 0
5	0	1	0	1	B	1	0	0	0	0	0	Comme à l'état 1, mais avec D_i = 0000	0, 0, 1, 1
6	0	1	1	0	1			0	0	0	0	Autocorrection, retour à 0000	0, 0, 0, 0
7	0	1	1	1	1			0	0	0	0	Autocorrection, retour à 0000	0, 0, 0, 0
8	1	0	0	0	1			0	0	0	0	Autocorrection, retour à 0000	1, 0, 0, 0
9	1	0	0	1	1			0	0	0	0	Autocorrection, retour à 0000	1, 0, 0, 1
10	1	0	1	0	1			0	0	0	0	Autocorrection, retour à 0000	0, 1, 0, 0
11	1	0	1	1	1			0	0	0	0	Autocorrection, retour à 0000	0, 1, 0, 1
12	1	1	0	0	1			0	0	0	0	Autocorrection, retour à 0000	0, 0, 1, 0
13	1	1	0	1	1			0	0	0	0	Autocorrection, retour à 0000	0, 0, 1, 1
14	1	1	1	0	1			0	0	0	0	Autocorrection, retour à 0000	0, 0, 0, 0
15	1	1	1	1	1			0	0	0	0	Autocorrection, retour à 0000	0, 0, 0, 0

Figure 7.72

- À l'état 0 : Selon la section stimulation de la table ÉP-ÉS, on voit que les variables conditionnelles sont B et Y ; on peut dresser la table suivante sachant qu'à cet état, on note aussi que $L = B$, $CE = Y \cdot \overline{B}$ et $U = 1$:

B	Y	L	CE	U	Remarque
0	0	0	0	1	Ne compte pas
0	1	0	1	1	Compte +1, car $U = 1$
1	0	1	1 ou X	1	Chargement, car L actif et prioritaire, $D_i = 0010$ (= 2)
1	1	1	0	1	Chargement, car L actif et prioritaire, $D_i = 0010$ (= 2)

Figure 7.73

Cette table, peut être écrite sous la forme d'une table de Karnaugh avec les deux fonctions qui nous intéressent, soit le comptage et le chargement d'une valeur si L est actif. Cela permet de simplifier les expressions, ce qui facilite l'écriture du diagramme d'état (ici, c'est assez évident, mais il survient parfois des cas plus complexes où cette procédure est fort utile) :

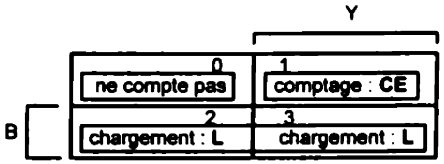


Figure 7.74

Cette table permet de résumer les actions qui seront entreprises :

- Charge si B (avec $D_i = 0010$) ;
- Compte si $Y \cdot \overline{B}$ (+1) ;
- Ne compte pas si $\overline{Y} \cdot \overline{B}$.

Ces valeurs sont indiquées dans la table ÉP-ÉS, section ÉS pour l'état 0.

- À l'état 1 : Selon la section stimulation de la table ÉP-ÉS, on voit que la variable conditionnelle est B ; à cet état, on note aussi que $L = B$, $CE = 1$ et $U = 0$ (donc, si on ne charge pas, on comptera avec −1). Avec seulement une variable impliquée, on peut résumer directement les actions qui seront entreprises :
  - Charge si B (avec $D_i = 0010$) ;
  - Compte si $\overline{B}$ (−1).
- À l'état 2 : Selon la section stimulation de la table ÉP-ÉS, on remarque que nous sommes dans la même situation qu'à l'état 0, sauf que les $D_i = 0100$.
- À l'état 3 : Selon la section stimulation de la table ÉP-ÉS, on remarque que nous sommes dans la même situation qu'à l'état 1, sauf que les $D_i = 0100$.
- À l'état 4 : Selon la section stimulation de la table ÉP-ÉS, on remarque que nous sommes dans la même situation qu'à l'état 0, sauf que les $D_i = 0000$.
- À l'état 5 : Selon la section stimulation de la table ÉP-ÉS, on remarque que nous sommes dans la même situation qu'à l'état 1, sauf que les $D_i = 0000$.
- Aux états 6 à 15 : Selon la section stimulation de la table ÉP-ÉS, on remarque que L est toujours 1. Puisque cette commande est prioritaire sur CE, il y aura toujours chargement avec les $D_i = 0000$. C'est l'autocorrection. Puisque L est prioritaire sur CE et U, il n'est pas nécessaire de calculer leurs valeurs pour ces états.

Étape 4 Diagramme d'état qui se monte directement à partir de la table ÉP-ÉS :

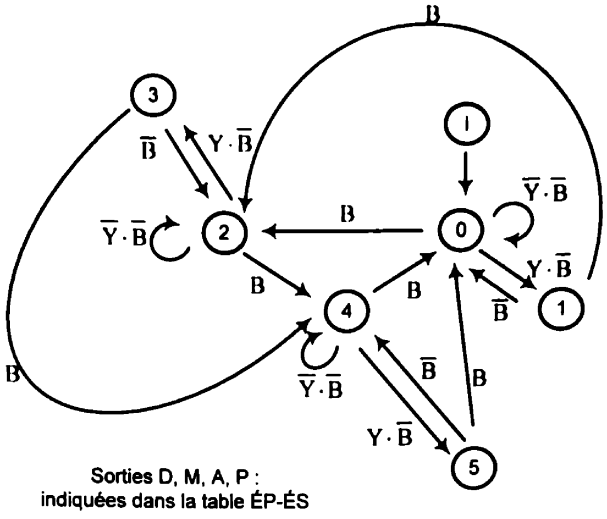


Figure 7.75

Afin de vérifier le travail, le circuit a été simulé dans Xilinx Foundation. La figure 7.76 illustre le chronogramme obtenu qui correspond au diagramme d'état (*i* pour *input*/entrée, *o* pour *output*/sortie).

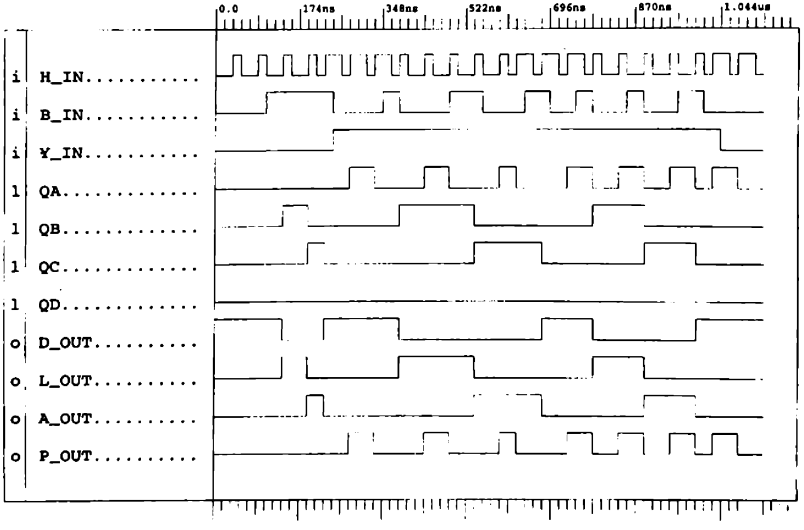


Figure 7.76

7.8 CONCEPTION DE MACHINES À NOMBRE FINI D'ÉTATS AVEC COMPTEURS SYNCHRONES

Cette fois encore, nous allons illustrer la conception à l'aide d'exemples résolus, ce qui permettra d'expliquer toutes les étapes.

Exemple 1 : On demande d'implanter le diagramme d'état suivant dans un CB2CLED (pour rendre les choses plus faciles ici, on débute avec un exemple à seulement deux bits). On demande l'autocorrection, mais avec un minimum de matériel supplémentaire. Les sorties du circuit sont les bits de sortie. Les variables d'entrée sont R et P.

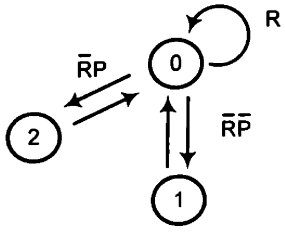


Figure 7.77

Solution : Il faut se reporter à la table 7.3D au début du chapitre pour la liste des étapes à suivre.

Tout d'abord, on remarque qu'il n'y a que trois états impliqués ici, donc deux bits suffisent puisque $2^2 = 4$. Un CB2CLED convient donc parfaitement à la tâche. Appelons ces bits : Q_0 et Q_1. Par ailleurs, l'autocorrection est demandée, mais sans complexification du problème (voir plus bas).

Étape 1 *Établissement de la table des actions.* Contrairement aux circuits avec flip-flops, nous travaillons cette fois avec des circuits plus complexes (ce qui est intéressant). Il faut donc établir la commande (L, CE, U) qu'il convient de réaliser à chacun des états ; c'est ce qu'on appelle la *table des actions*. Rappelons qu'avec un tel compteur, on peut :

Commande	Action
L	Charger une nouvelle valeur dans le compteur *Important* : **L est prioritaire sur CE,** *Important* : **il faut spécifier les D_i ;**
CE	Activer ou non le comptage ;
U	Compter par en haut (+1) ou par en bas (−1).

Figure 7.78

Ces trois commandes que nous avons peuvent être vues comme les mnémoniques d'un circuit programmable élémentaire, donc nos premiers pas vers la programmation de circuits plus complexes tels des microcontrôleurs ou des microprocesseurs.

Revenons à notre *table des actions* ; elle se bâtit en analysant « ce qu'il y a de mieux à faire » à chaque état. Quand on écrit « ce qu'il y a de mieux à faire », c'est bien sûr en

exploitant au mieux les ressources de notre circuit par les commandes *L*, *CE* et *U*. Il est à noter qu'avec la commande *L*, on peut tout faire puisqu'on peut charger une valeur quelconque dans le circuit. Par contre, si on doit passer par les états 00, 01 et 10, il vaut mieux employer la commande *CE* qui effectue le comptage automatiquement à chaque coup d'horloge sans la nécessité de spécifier la valeur des D_i (qui, elle, exige de la circuiterie externe supplémentaire). En résumé, *L* sera employé en dernier recours si on ne peut aller à l'état suivant par simple comptage (+1 ou −1) ; par exemple si on doit aller de 00 à 10 au prochain coup d'horloge.

Donc, voici l'étude, état par état :

- État 00 (case 0) : **CE** sera employé pour aller à 01, et **L** à 10, selon les valeurs de *R* et de *P* (compte +). Si **CE** = **L** = 0, on reste simplement sur place ;
- État 01 (case 1) : **CE** sera employé pour aller à 00 (compte −) ;
- État 10 (case 2) : **L** sera employé à pour aller à 00 ;
- État 11 (case 3 : Pour *l'auto-correction optimisée*, on peut mettre un X à la case 3 de la table des actions. Lors de la simplification, on entoure les cases 2, où l'on a déjà spécifié *L*, et 3, où l'on aurait mis *X*. Ainsi, on note que, sans rendre le circuit plus compliqué, on peut en fait mettre directement un *L* à la case 3. Quant aux D_i, pour cet état, ils importent peu, à condition qu'ils ne nous laissent pas à l'état 11. Il faudra donc vérifier cela plus tard dans l'élaboration de la solution. Ce choix n'aura pas d'influence sur *CE* et *U* puisque *L* est prioritaire. Pour l'*auto-correction simple*, on pourrait choisir un état de retour (par exemple l'état 00) et forcer celui-ci. On aboutira à une solution plus complexe à cause des conditions qu'on doit imposer. Par exemple pour l'état de retour à 00, on doit spécifier L=1 pour l'état 11 et les D_i = 00, ou encore CE+ avec L= 0 puisque sur deux bits 11 + 1 = 00. Dans cet exemple, on a choisi d'y aller avec l'*auto-correction optimisée* avec L = 1 à l'état 11 (cela conduira à une solution plus simple).

Comme on le voit, la table des actions se monte rondement ; elle fixe ce que le compteur doit accomplir à chaque état :

Table des Actions:

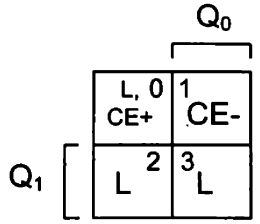


Figure 7.79

Étape 2 Étude des variables pour déterminer *L*, *CE* et *U*

- État 0 : On a deux variables impliquées (*R* et *P*) ; il faut déterminer leur influence, ce qui se fait simplement avec une table de vérité. On remarque (selon nos discussions

précédentes sur l'établissement des diagrammes d'état) que le diagramme d'état a bien été préparé puisque, selon la valeur des variables R et P, on couvre toutes les combinaisons possibles de R et de P, et avec seulement une direction à la fois.

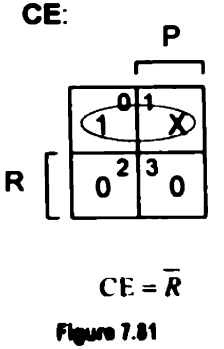
Directions possibles	L	CE	U	Bits de données D_i	Si
État 0	0	0	X	XX	R
État 2	1	X	X	10	$\overline{R} \cdot P$
État 1	0	1	1	XX	$\overline{R} \cdot \overline{P}$
Union :	$\overline{R}P$	$\overline{R}\,\overline{P} + \overline{R}P = \overline{R}$	1	10	

Figure 7.80

Il est à noter que l'on met X à CE à $\overline{R}P = 1$ puisque, L étant prioritaire, la valeur de CE importe peu. De même pour U qui n'a pas d'importance si on ne compte pas ($CE = 0$ ou X). Quant aux valeurs des D_i ($D1$ et $D0$), c'est seulement lorsqu'on effectue un L qu'elles sont importantes (ici, $D1 = 1$ et $D0 = 0$). Ceci simplifie bien sûr le circuit obtenu.

Il faut ensuite calculer L, CE et U selon cette table de vérité par simple simplification algébrique ou selon la technique des tables de Karnaugh si les cas sont plus compliqués.

Donc, pour l'état 0, cela donne, avec l'opération d'union des valeurs mises à 1 (1 ou X mis à 1), $L = \overline{R}P$ et $U = 1$. Pour CE, c'est un peu plus compliqué : $CE = \overline{R}\,\overline{P} + \overline{R}P = \overline{R}$ On peut se convaincre du résultat avec une table de Karnaugh (on doit remarquer que ce sont les variables R et P qui commandent ces tables de Karnaugh, et il faut bien s'assurer que le *MSB* dans la table de vérité est le même que dans la table de Karnaugh) :



$$CE = \overline{R}$$

Figure 7.81

Avec les $D_i = 10$ pour cet état, peu importe R et P (il y a des X dans la table de vérité).

- État 1 : Pas de variable impliquée, et selon la table des actions, CE est actif, donc $CE = 1$ et $U = 0$ (on compte -1). Il faut aussi $L = 0$ puisque cette commande est prioritaire sur CE.

- État 2 : Pas de variable impliquée, et, selon la table des actions, L est actif, donc $L = 1$ et les D_i valent 00 (retour à 0). CE et U sont mis à X puisque L est de toute façon prioritaire sur ceux-ci.
- État 3 : On a mis $L = 1$ pour l'*auto-correction optimisée* avec des D_i sans trop d'importance à condition qu'ils soient différents de 11 (on ne veut pas rester sur place). On verra plus tard ce que cela pourrait donner. L'énoncé du problème spécifie que l'on veut une autocorrection sans matériel supplémentaire, ce qui est le cas ici puisque, si on avait mis $L = X$ au lieu de $L = 1$, on aurait obtenu la même expression de L.

Étape 3 Il faut maintenant déterminer L, CE et U globalement. Cela s'effectue en dressant trois tables de Karnaugh selon les résultats obtenus pour chaque état à l'étape précédente :

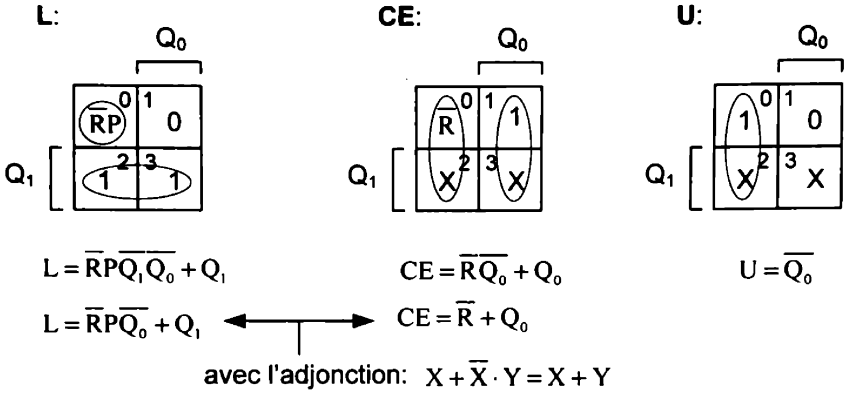


Figure 7.82

Ce qui permet d'obtenir les valeurs de L, de CE et de U.

Étape 4 Il faut maintenant déterminer les bits de données (D_i). On se rappelle que les D_i ne doivent être spécifiés que lorsque L est actif, donc que l'on doit charger de nouvelles valeurs dans le compteur. Ici, les D_i ont été notés précédemment avec $D_i = 10$ à l'état 0, et $D_i = 00$ à l'état 2. À l'état 3, ils peuvent être n'importe quoi sauf 11 (pour ne pas rester sur place). On remarque que, sans forcer leur valeur, on obtient $D_1 = D_0 = 0$, ce qui est excellent pour l'*auto-correction optimisée* (on passera de l'état 3 à l'état 0). Cela donne donc :

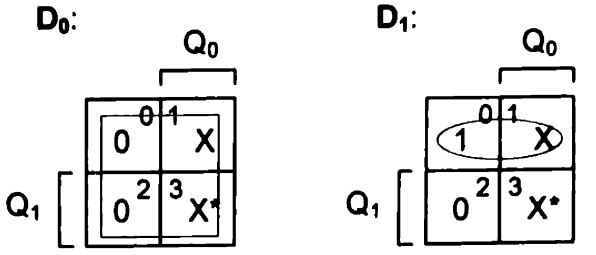
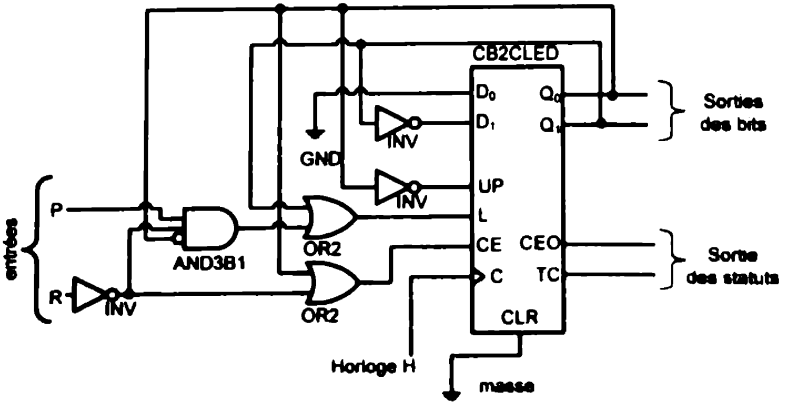


Figure 7.83

Étape 5 Il n'y a pas lieu d'employer de multiplexeur pour ce circuit puisque les fonctions obtenues pour *L*, *CE*, *U* et les D_i sont simples.

Étape 6 Les sorties des bits étant directement les sorties du circuit, il n'y a donc pas de travail à réaliser de ce côté.

Étape 7 On est maintenant prêt à monter le circuit. Il suffit de reprendre les résultats obtenus aux étapes précédentes : étape 3 pour *L*, *CE* et *U* ; étape 4 pour les D_i. Le reste n'est que du montage réalisé ici dans Xilinx Foundation.



L'utilisation d'un compteur rend le design somme toute assez simple. On note que l'autocorrection s'est effectuée ici à coût nul grâce à un peu de bon sens. Finalement, il est toujours recommandé de tester le circuit que l'on vient de concevoir. Xilinx Foundation permet une simulation efficace qui est fournie ici pour quelques situations de fonctionnement.

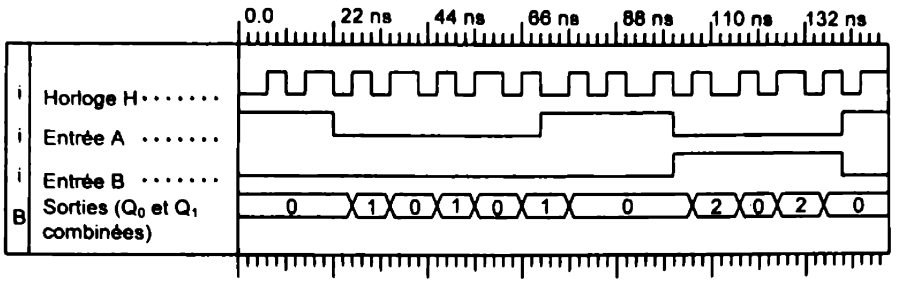


Figure 7.84 Chronogramme illustrant quelques situations de fonctionnement du circuit de l'exemple 1 (*i* pour *input* /entrée ; *B* pour bus de données, car on a combiné Q_0 et Q_1 ensemble)

Exemple 2 : On demande de faire le design d'un distributeur automatique simple dont le fonctionnement est le suivant. Il y a deux choix possibles : bonbons (prix : 25 ¢), croustilles (prix : 50 ¢). La machine n'accepte que des pièces de 25 ¢. Si 50 ¢ ont été introduits, les seuls choix possibles sont la remise de toute la monnaie introduite ou la sélection de croustilles.

Les entrées externes sont :

M - bouton rend toute la monnaie introduite ;

B - bouton de sélection « veut bonbon » ;

C - bouton de sélection « veut croustilles » ;

25 - 25 ¢ introduit dans la machine.

Les sorties sont :

SB - signal qui commande la distribution des bonbons ;

SC - signal qui commande la distribution des croustilles ;

SM - signal qui commande la remise de la monnaie (si la cliente annule sa commande).

Le design se fait avec un CB4CLED. L'horloge est suffisamment lente pour que la partie mécanique de la machine suive la machine d'états réalisée.

Solution : Il faut se reporter à la table 7.3D au début du chapitre pour la liste des étapes à suivre.

Bien évidemment, ce type de problème ne conduit pas à une solution unique ; ce qui est intéressant, puisque la designer peut faire preuve d'imagination. D'une façon toute bête, simplement en changeant les numéros des états, une solution différente sera obtenue.

Étape 0 Il faut d'abord établir le diagramme d'état (figure 7.87). Celui-ci se bâtit d'une façon directe à partir de la description fournie. Les entrées externes *M*, *B*, *C*, 25 seront nos variables conditionnelles. Pour la sélection des variables, une étude état par état est nécessaire.

- État 0 : On ne peut quitter cet état que si une pièce de 25 ¢ a été introduite dans l'appareil, autrement, on reste sur place. Pour cet état, la variable 25 commande la sortie vers un autre état, et donc $\overline{25}$ le surplace. Ceci couvre tous les cas utiles possibles.

- État 1 : L'énoncé du problème ne fait pas mention de la possibilité d'activation simultanée des entrées (par exemple, introduction d'une pièce de monnaie et choix immédiat). Cela nous force à supposer une certaine hiérarchie dans l'ordre des variables. Ainsi, l'introduction d'une pièce de monnaie (variable 25) est prioritaire ; la remise de la monnaie suit raisonnablement (variable *M*). Finalement, puisque ce sont des décisions irréversibles (on ne peut remettre la marchandise dans la machine), la distribution des bonbons (variable *B*) et des croustilles (variables *C*) complète l'ensemble, *C* ayant la plus basse priorité étant donné que la vente d'un sachet de croustilles est en bout de parcours après deux introductions de pièces. Cet ordre a évidemment comme conséquence que, dans certaines situations, la cliente devra réexécuter sa demande. Par exemple, si elle introduit 25 ¢ et, simultanément, veut récupérer l'argent qu'elle a introduit dans la machine, on se déplacera d'abord à l'état 4, et la cliente devra appuyer une seconde fois sur le bouton de remise de monnaie.

  Ceci étant établi, l'usage des variables 25, *M*, *B*, *C* devient évident ; il est repris à la table suivante (figure 7.85). Se rappelant la discussion sur les diagrammes d'état, on constate que toutes les combinaisons des variables sont couvertes, une fois seulement. Les équations des combinaisons de variables sont aussi ajoutées.

Variables				Directions possibles			
				à 4	à 3	à 2	à 1
25	*M*	*B*	*C*	25	$\overline{25} \cdot M$	$\overline{25} \cdot \overline{M} \cdot B$	$\overline{25} \cdot \overline{M} \cdot \overline{B}$
0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

Figure 7.85

- États 2, 3, 5, 6, et 7 : On va inconditionnellement à l'état 0. Aucune variable n'est impliquée ici. Les sorties actives sont indiquées sur le diagramme. L'état 7, inemployé ici, est malgré tout considéré avec un retour inconditionnel à l'état 0 (choix de l'*auto-*

correction simple ici en forcant l'état de retour fixé à 0 ce qui est tout à fait justifiable puisqu'il s'agit de l'état d'attente du circuit).

- État 4 : À cet état, deux 25 ¢ ont été introduits dans la machine, et on est prêt à donner un sachet de croustilles. Deux situations pourraient être embêtantes ici : si la cliente demande des bonbons ou si elle réintroduit 25 ¢. Comme tout est possible, il faut que l'on tienne compte de ces éventualités, même si elles sont peu probables. De façon arbitraire, on décide que, si un 25 ¢ est réintroduit, cela est l'équivalent d'une remise de la monnaie puisque nous n'avons aucun produit disponible à 75 ¢. Il en va de même si des bonbons sont demandés puisque le mécanisme ne permet pas de garder 25 ¢, fournir des bonbons et remettre 25 ¢.

  Ceci étant établi, l'usage des variables 25, M, B, C devient évident ; il est repris à la table suivante (figure 7.86). Se rappelant la discussion sur les diagrammes d'état, on constate que toutes les combinaisons des variables sont couvertes, une fois seulement. Les équations des combinaisons de variables sont aussi ajoutées.

Variables				Directions possibles		
25	M	B	C	à 4 $\overline{25} \cdot \overline{M} \cdot \overline{B} \cdot \overline{C}$	à 5 $\overline{25} \cdot \overline{M} \cdot \overline{B} \cdot C$	à 6 $M + 25 + B$
0	0	0	0	1	0	0
0	0	0	1	0	1	0
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	0	0	1
0	1	0	1	0	0	1
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	0	1
1	0	1	1	0	0	1
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	0	1

Figure 7.86

Tous les états ayant été étudiés, on peut dessiner le diagramme d'état. Dans celui-ci, on a souligné les sorties actives (aux états 2, 3, 5, 6) ; elles sont nulles ailleurs. Les combinaisons de variables sont indiquées aussi. Des commentaires sont spécifiés en italique.

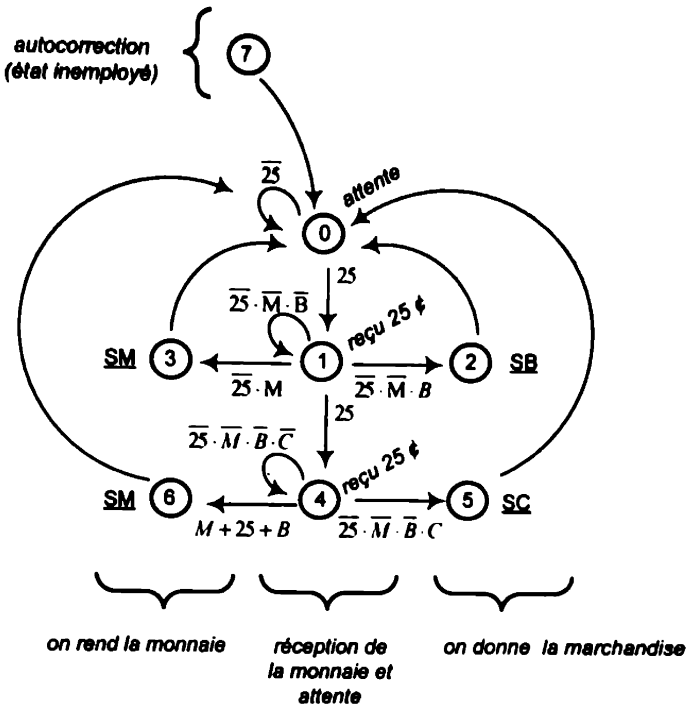


Figure 7.87

On remarque qu'il y a sept états (0 à 6), donc en fait trois bits seront suffisants ($2^3 = 8$) ; deux seraient insuffisants ($2^2 = 4$). Appelons les trois bits : Q_C (*MSB*), Q_B et Q_A correspondant respectivement aux Q_3, Q_2 et Q_0 du CB4CLED. Donc, le quatrième bit du CB4CLED ne sera pas employé (par souci de fiabilité, on mettra l'entrée D_3 à la masse), ce qui nous laisse huit états. Toujours par souci de fiabilité, l'état inemployé (7) nous ramènera à l'état de retour, qui est 0 ici (*auto-correction simple*).

Étape 1 *Établissement de la table des actions.* On se rappelle que la commande *L* (*load*/charge) du CB4CLED est prioritaire sur les commandes *CE* (compte) et *U* (*up-down*/incrémente-décrémente). Du diagramme d'état, on note aussi que le comptage, lorsqu'il aura lieu, sera toujours incrémental (+1), et donc que *U* sera toujours 1. Cela simplifie le problème. En se rappelant la discussion de l'exemple précédent, on étudie chaque état en fonction du diagramme d'état établi à l'étape précédente :

- État 0 (case 0) : *CE* sera employé pour aller à l'état 1 ;
- État 1 (case 1) : *CE* sera employé pour aller à l'état 2. *L* sera employé pour aller aux états 2 et 3 ;
- État 2 (case 2) : *L* sera employé pour aller à l'état 0 ;
- État 3 (case 3) : *L* sera employé pour aller à l'état 0 ;
- État 4 (case 4) : *CE* sera employé pour aller à l'état 5. *L* sera employé pour aller à l'état 6 ;

- État 5 (case 5) : *L* sera employé à pour aller à l'état 0 ;
- État 6 (case 6) : *L* sera employé pour aller à l'état 0. Alternativement, on aurait pu employer *CE* pour aller à l'état 7 et, de là, à l'état 0. Si l'horloge est rapide, l'usagère ne verra pas la différence (ce n'est toutefois pas le cas ici puisque l'horloge doit suivre le système mécanique). Cela aurait limité l'usage de *L*. Cela montre clairement qu'il n'y a pas de solution unique ;
- État 7 (case 7) : *L* sera employé pour aller à l'état 0. On aurait pu aussi y aller avec CE + (sur 3 bits : 111+1 = (1) 000), par contre cela permet de mettre des X pour *CE* et *U* puisque L est prioritaire. Il faudra avoir les Di= 000, mais cela est déjà le cas des états 2, 3, 5, 6. Il y aura simplification comme nous le verrons plus bas.

La table des actions est donc :

Table des Actions

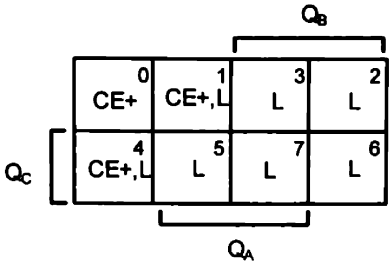


Figure 7.88

Étape 2 Étude des variables pour déterminer *L*, *CE* et *U*. *U* est toujours à 1 (compte +1), car on ne compte jamais avec −1.

- État 0 : Seul *CE* est utilisé si 25. Donc, $L = 0$ et $CE = 25$, et les $D_i = X$ puisqu'ils ne servent pas.
- État 1 : On a quatre variables impliquées (25, *M*, *B*, *C*) ; il faut déterminer leur influence, ce qui se fait simplement avec une table de vérité :

Directions possibles	*L*	*CE*	Bits de données D_i (D_2, D_1, D_0)	si
État 1	0	0	*XXX*	$\overline{25} \cdot \overline{M} \cdot \overline{B}$
État 2	0	1	*XXX*	$\overline{25} \cdot \overline{M} \cdot B$
État 3	1	*X*	011	$\overline{25} \cdot M$
État 4	1	*X*	100	25
Union :	$25 + \overline{25} \cdot M$	**	*	

*, ** = voir figure 7.90.

Figure 7.89

Il est à noter que l'on met *X* à *CE* aux états 3 et 4 puisque, *L* étant prioritaire, la valeur de *CE* importe peu. Quant aux valeurs des D_i (D_2, D_1 et D_0), c'est seulement lorsqu'on

effectue un L qu'elles sont importantes, ce qui permet de mettre des X ailleurs. Ceci simplifie bien sûr le circuit obtenu.

Il faut ensuite calculer L et CE selon cette table de vérité, selon la technique des tables de Karnaugh ou autrement. À ce propos, on note que les associations de variables ($\overline{25}\cdot\overline{M}\cdot\overline{B}$, $\overline{25}\cdot\overline{M}\cdot B$, $\overline{25}\cdot M$ et 25) remplissent totalement la table de Karnaugh, ce qui confirme que toutes les combinaisons de variables (25, M, B et C) sont couvertes pour cet état (c'est une vérification intéressante du travail réalisé à l'étape 0). Donc, pour l'état 1, on voit directement que $L = 25 + \overline{25}\cdot M = 25 + M$ selon l'adjonction.

On voit aussi que $CE = B$, ce résultat est obtenu à l'aide d'une table de Karnaugh (on doit remarquer que ce sont les variables 25, M, B et C qui commandent la table de Karnaugh). Pour les D_i, la simplification est obtenue aussi par table de Karnaugh où l'on a bien pris soin de mettre les X, les « 1 », les « 0 », ce qui permet immédiatement de maximiser la solution. On a ainsi :

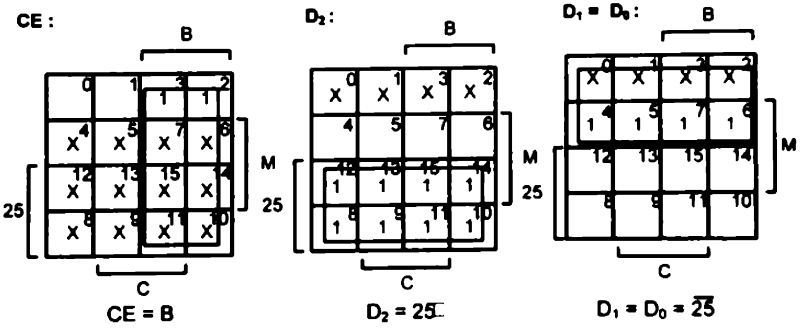


Figure 7.90

- États 2, 3, 5, 6 et 7 : Il n'y a pas de variables impliquées, et, selon la table des actions, L est actif, donc $L = 1$, et les D_i valent 000 (retour à 0). CE et U sont mis à X puisque L est de toute façon prioritaire sur ceux-ci.
- État 4 : On a quatre variables impliquées (25, M, B, C) ; il faut déterminer leur influence, ce qui se fait simplement avec une table de vérité :

Directions possibles	L	CE	Bits de données D_i (D_2, D_1, D_0)	Si
État 4	0	0	XXX	$\overline{25}\ \overline{M}\ \overline{B}\ \overline{C}$
État 5	0	1	XXX	$\overline{25}\ \overline{M}\ \overline{B}\ C$
État 6	1	X	110	$M + 25 + B$
Union :	$M + 25 + B$	*	110	

* = voir figure 7.92.

Figure 7.91

On voit que les D_i sont mis à 110. La simplification pour CE donne :

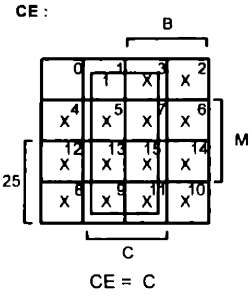


Figure 7.92

Étapes 3 et 4 Il faut maintenant déterminer L, CE et les D_i globalement. Cela s'effectue en dressant les tables de Karnaugh selon les résultats obtenus pour chaque état à l'étape précédente :

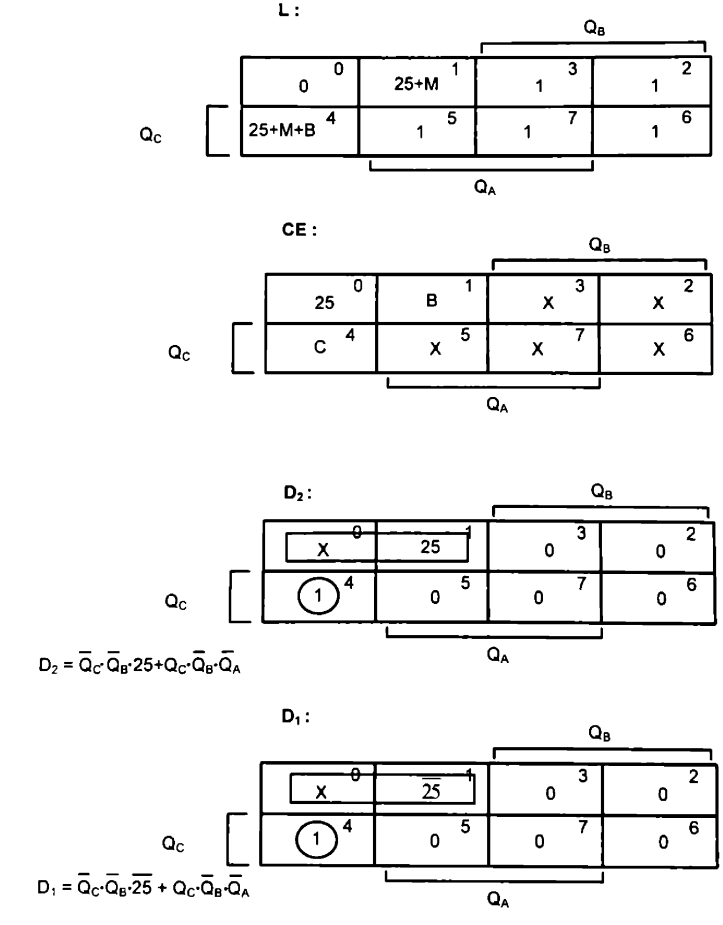


Figure 7.93

Cela permet d'obtenir les valeurs simplifiées. On rappelle que U est toujours égal à 1. Pour ce qui est des D_i, on peut obtenir des expressions relativement simples qui sont indiquées. CE et L, par contre, sont plus complexes, et on opte donc pour l'emploi de multiplexeurs pour implanter les tables correspondantes.

Étape 5.1 (avec MUX simplifié pour L) : Puisque que L ne peut prendre que quatre valeurs différentes : 0, 1, 25 + M et 25 + M + B, au lieu d'un multiplexeur 8 à 1, on peut employer un multiplexeur 4 à 1 avec recodage des états : Anciens états (= états d'origine : 0, 1, 2, 3, 4, 5, 6, 7) → Nouveaux états (= états recodés : 0, 1, 2, 3). Ce MUX 4 à 1 aura deux bits de sélection : S1 et S0 (pour les quatre nouveaux états). Pour la correspondance entre les états anciens et les nouveaux, de façon à simplifier les expressions de S1 et de S0, on essaye de maintenir une certaine similarité entre ceux-ci. Ainsi, avec 0 ↔ 0, 1 ↔ 1, 2 ↔ 2, ceci nous oblige à assigner le nouvel état 2 aux anciens états 2, 3, 5, 6 et 7 à cause de la valeur de L qui vaut 1 pour tous ces cas. Finalement, il reste l'ancien état 4 qui est alors assigné au nouvel état 3 (4 ↔ 3).

Ancien état	valeur de L	Nouvel état	$S1$	$S0$
0	0	0	0	0
1	25 + M	1	0	1
2	1	2	1	0
3	1	2	1	0
4	25 + M + B	3	1	1
5	1	2	1	0
6	1	2	1	0
7	1	2	1	0

Figure 7.94

Il faut ensuite déterminer les expressions pour $S1$ et $S0$ en fonction de la valeur des anciens états (donc de la valeur de Q_C, de Q_B, et de Q_A. Cela s'obtient à l'aide des tables de Karnaugh suivantes :

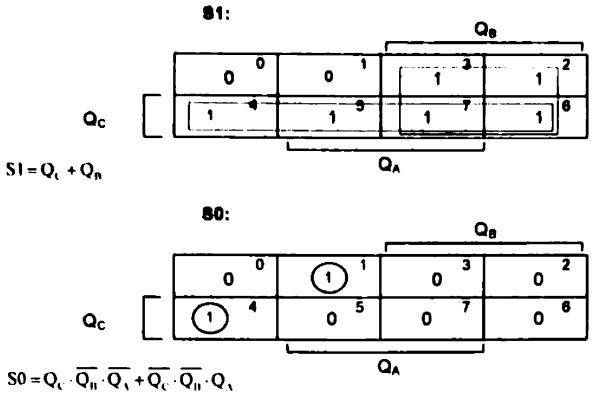


Figure 7.95

Étape 5.2 (avec MUX non simplifié pour *CE*) : *CE* ne prend que trois valeurs (25, *B*, *C*). On pourrait le simplifier selon ce qu'on a vu pour L. Par contre, ici, nous décidons, à titre d'exemple, d'employer un multiplexeur non simplifié, donc un multiplexeur 8 à 1 pour *CE*. Cela évite le recodage des états, et la solution est directe. Les valeurs de X sont mises à 0 par souci de prudence puisqu'il faut bien leur donner une valeur, et les laisser flottantes n'est pas une bonne pratique. On a donc pour les entrées du MUX 8 à 1 :

État	Valeur de CE
0	25
1	*B*
2	*X* = 0
3	*X* = 0
4	*C*
5	*X* = 0
6	*X* = 0
7	*X* = 0

Figure 7.96

Étape 6 Les sorties des bits doivent être synthétisées. Selon le diagramme d'état de l'étape 0, elles sont toujours nulles sauf *SM* (état 3 + 6), *SB* (état 2) et *SC* (état 5). On peut écrire immédiatement les expressions algébriques équivalentes :

$$SM = \overline{Q_C} \cdot Q_B \cdot Q_A + Q_C \cdot Q_B \cdot \overline{Q_A}$$

$$SB = \overline{Q_C} \cdot Q_B \cdot \overline{Q_A}$$

$$SC = Q_C \cdot \overline{Q_B} \cdot Q_A$$

Étape 7 On est maintenant prêt à monter le circuit. Il suffit de reprendre les résultats obtenus aux étapes précédentes : étapes 3, 5.1 et 5.2 pour *L*, *CE* et *U*; étape 4 pour les D_i ; étape 6 pour les sorties. Le reste n'est que du montage, réalisé ici dans Xilinx

Foundation. *Note* : Les signaux sont appelés ici : horloge → *HH*, 25 →*IN_25*, *M* → *IN_M*, *B* → *IN_B*, *C* → *IN_C*. D_3 est mis à la masse, car il est inemployé.

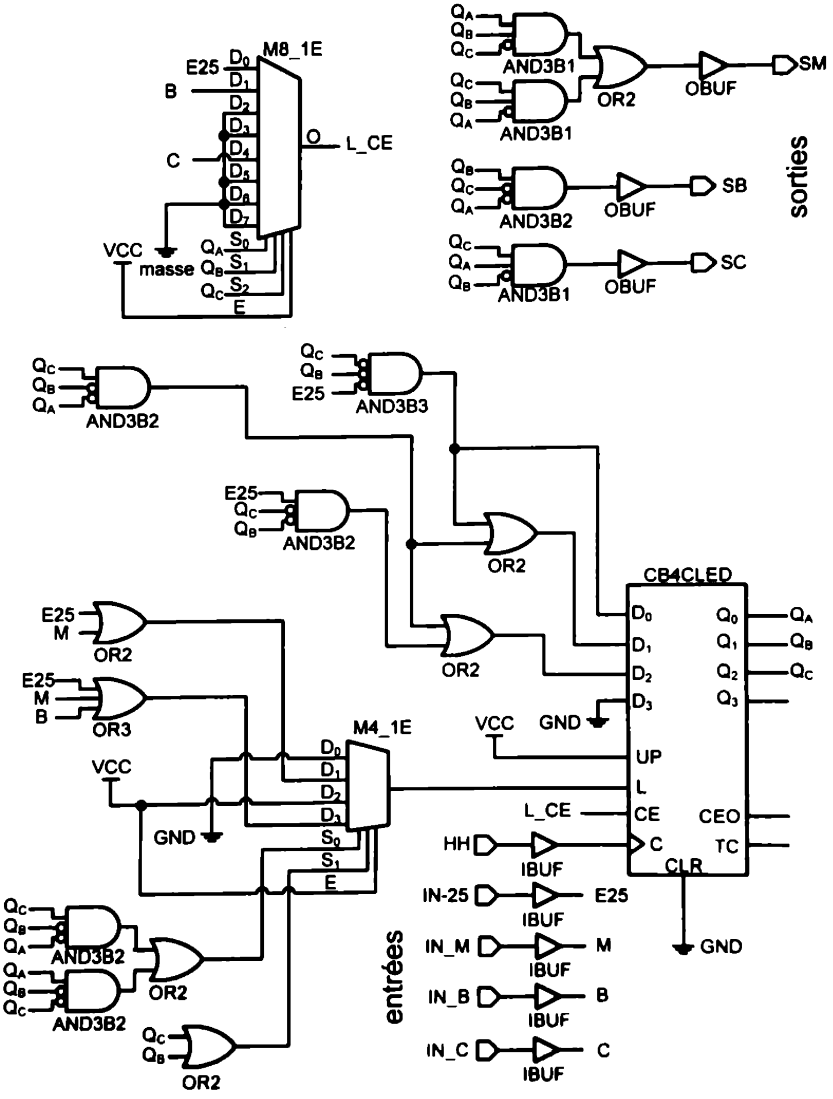


Figure 7.97

L'utilisation d'un compteur rend le design somme toute relativement simple. Finalement, tout comme pour l'exemple précédent, il est recommandé de tester le circuit que l'on vient de concevoir. Xilinx Foundation permet une simulation efficace qui est fournie ici pour quelques situations de fonctionnement (Q_C, Q_B, et Q_A sont ici regroupés en bus) :

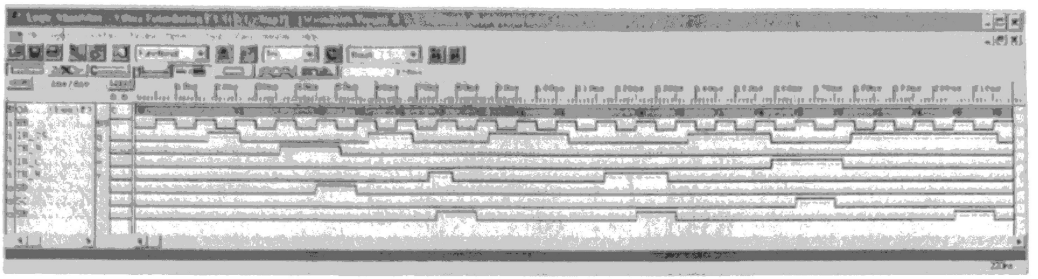


Figure 7.98 Chronogramme illustrant quelques situations de fonctionnement du circuit de l'exemple 2 (*i* pour *input* /entrée, *B* pour bus de données, et *o* pour *output*/sortie).

Tout ce que nous venons d'étudier nous démontre que cette approche de design avec circuits MSI (compteurs synchrones et multiplexeurs) devient vite compliquée. De plus, le moindre changement au design (par exemple le rajout d'une entrée externe supplémentaire) implique presque directement de tout recommencer à partir de 0. Par contre, les designs que nous obtenons sont très efficaces et très compacts.

7.9 MICROPROGRAMMATION

Une solution à la question de complexité des circuits discutée à la fin de la section précédente serait une approche standard configurable d'un design à l'autre. La partie matérielle aurait été bien élaborée une fois et pourrait servir sans ou avec peu de changements pour toutes nos applications. La configuration serait obtenue par un mécanisme de programmation. Essentiellement donc, les changements seraient obtenus par codage plutôt que par câblage.

En fait, tout ce que nous avons vu aux chapitres précédents s'appelle de la *logique câblée*, très efficace, mais coûteuse en travail de conception. Ce dont nous discutons dans cette section est de la *logique programmée* : il s'agit d'une base matérielle « universelle » contenant un certain nombre d'entrées et de sorties qu'il suffit de programmer selon les besoins.

MICROCONTROLEURS

Il existe plusieurs voies en logique programmée. Par exemple, on peut employer un *microcontrôleur*[14], qui est un circuit logique complexe contenant, selon les modèles, une variété d'entrées et de sorties numériques (de type 0, 1), mais aussi parfois analogiques (travaillant sur une plage continue de tension ou de courant). Ces circuits sont parfois aussi appelés *single-chip microcomputer* ou micro-ordinateur à un seul boîtier puisque tout est intégré dans une seule puce. Ces circuits sont d'un usage universel, et on les retrouve partout dans les appareils qui nous côtoient : four à micro-ondes, etc.

Selon notre discussion à la section 1.0, les microcontrôleurs sont des ordinateurs de type Harvard puisque leur code est (d'habitude) immuable, alors que les données traitées, elles, peuvent changer : qui a déjà fait une mise à jour 2.0 du programme de

[14] La revue *Circuit Cellar* est une bonne source de renseignements sur les microcontrôleurs.

fonctionnement de son four à micro-ondes ? Par contre, on se sert de notre four à micro-ondes parfois 30 s pour réchauffer un verre de lait, parfois 1 min pour un bol de soupe, etc. Donc, le programme résidant dans le microcontrôleur est fixe, et les données traitées par celui-ci (puissance, durée de chauffage) sont modifiables.

Pour la sélection du microcontrôleur, il faudra d'abord choisir la taille des mots selon la puissance de calcul requise : 8, 16, 32 ou 64 bits sont les valeurs les plus communes (le quatre bits – comme le COP4 de National[15] – est aujourd'hui désuet). On note que le marché du huit bits occupe encore environ la moitié du marché. Les manufacturiers spécialisent aussi leurs produits par segments de marché. Par exemple, un microcontrôleur donné sera offert avec certaines particularités pour le marché automobile, pour celui des téléphones portables ou pour celui des petits appareils électroménagers. Un même microcontrôleur est ainsi offert avec une variété de mémoires (type, taille), de boîtiers (de quelques pins, par exemple 8 à plus de 40 ou même plus), de plages de température de fonctionnement, de vitesses d'horloge, de tensions d'alimentation, etc. La designer doit donc bien étudier ses besoins avant de choisir un modèle !

Un des critères sera certainement la disponibilité d'outils de développement[16] (programmation, simulation) efficace. Une particularité des microcontrôleurs est en effet leurs langages de programmation. Certains ne sont programmables qu'en langage machine (aussi appelé assembleur), d'autres en langages plus évolués tel le langage C par exemple. Évidemment, la programmation en langages évolués exige généralement plus de mémoire.

Certains manufacturiers offrent aussi leurs microcontrôleurs sous la forme de librairies, on peut alors inclure le microcontrôleur dans notre design dans un circuit FPGA ou CPLD en y ajoutant diverses interfaces. L'avantage est bien sûr que tout est inclus dans une seule puce. Par contre, il est à noter que les FPGA et CPLD sont généralement coûteux (plusieurs centaines de dollars pour les modèles complexes), alors qu'on peut avoir un microcontrôleur pour quelques cent.

Dans la vaste majorité des cas, les microcontrôleurs contiennent un jeu d'instructions fixe. Le jeu d'instructions spécifie ce que le microcontrôleur pourra réaliser. Par exemple, il pourra additionner, avec une instruction de type ADD (ADD est appelé le mnémonique) ; certains modèles pourront aussi diviser (instruction DIV) en arithmétique à point fixe ou flottant. Ici, deux approches sont disponibles :

- Approche *RISC* : *Reduced Instruction Set Computer* ou architecture à jeu d'instructions réduits. L'idée est d'avoir un jeu d'instructions simple et facile à apprendre. Le corollaire étant bien sûr que les opérations complexes exigeront un travail de programmation plus élaboré. Il faudra ainsi écrire ses propres sous-routines pour des opérations complexes, ce qui consommera aussi de la mémoire supplémentaire. Par exemple, certains microcontrôleurs PIC de la compagnie Microchip n'ont que 27 instructions.

---

[15] http://www.microprocessor.sscc.ru/great/e2.html

[16] En Anglais : Integrated Development Environment (IDE)

- Approche *CISC* : *Complex Instruction Set Computer* ou architecture à jeu d'instructions complexe. L'idée est d'avoir un jeu d'instructions très complet. Le corollaire étant bien sûr un apprentissage plus élaboré pour celles qui devront utiliser ce circuit. Par ailleurs, toutes les instructions sont-elles toutes utiles ? Par exemple, le vénérable 8051[17] initialement mis sur le marché par Intel en 1981 (et aujourd'hui disponible chez plus de vingt manufacturiers.), a un jeu d'instructions contenant plus d'une centaine d'instructions dont des instructions peu utilisées de déplacement d'octets codés en BCD. Pour la petite histoire, il est intéressant de savoir que le 8051 a servi (et sert encore) décodage dans les claviers des ordinateurs personnels PC.

Si dans la majorité des cas le jeu d'instructions disponible dans un microcontrôleur (ou un microprocesseur) convient, il existe des applications particulières où ce n'est pas le cas. On doit alors recourir à d'autres techniques telle l'écriture de sous-routines. En effet, il est possible, à partir du jeu d'instructions disponible, d'écrire des fragments de code informatique appelables à la demande et exécutant une tâche particulière ; ceux-ci sont aussi appelés « sous-routines ». Les sous-routines accompliront des tâches plus complexes que celles offertes avec le jeu d'instructions de base. Par exemple, l'instruction qui inverse une matrice n'est généralement pas offerte dans les microcontrôleurs. Toutefois, une programmeuse peut écrire une sous-routine qui réalisera cette opération mathématique. Il y a cependant des limites aux sous-routines, par exemple la place en mémoire et la vitesse d'exécution, pour n'en nommer que deux. Il est important de noter que, contrairement au domaine des ordinateurs personnels PC où la norme est toujours plus vite avec toujours plus de mémoire, dans le domaine des microcontrôleurs ce n'est pas le cas. Ainsi, même si le vénérable 8051 est toujours fort employé, on ne trouvera pas de 8051 fonctionnant à des vitesses d'horloge très supérieures à 40 MHz. Par ailleurs, sa plage standard d'adressage est seulement de 64 kilo-octets. D'autres microcontrôleurs (format huit bits) auront à peu près ces mêmes paramètres. On atteint ainsi vite les limites, ce qui pourra nous mener à opter pour des microcontrôleurs plus « musclés » (architecture 16 bits, etc.). Dans les cas extrêmes, une approche par microprogrammation pourra s'avérer intéressante.

MICROPROGRAMMATION

La microprogrammation est une approche par laquelle on définit le jeu d'instructions auquel un séquenceur programmable répondra. Il fut une époque où cette approche fut florissante : plusieurs manufacturiers offraient des produits génériques dont on pouvait définir le jeu d'instructions à loisir. Un bon exemple est le fameux Am2900 de AMD (Advanced Micro Devices[18]), sorti au début des années quatre-vingt, qui demeure certainement un des exemples les plus fameux (et toujours très vivant sur le Net).

En microprogrammation, on définit d'abord un jeu de macro-instructions ; il s'agit en fait du jeu d'instructions dont nous avons parlé plus haut (à la sous-section précédente). Par la suite, ce jeu de macro-instructions est lui-même défini à l'aide de micro-instructions. En fait, à chaque macro-instruction correspondra une ou plusieurs micro-instructions.

---

[17] http://en.wikipedia.org/wiki/Intel_8051 . Un bon simulateur en ligne se trouve à : http://www.edsim51.com/

[18] http://www10.edacafe.com/book/parse_book.php?article=BITSLICE/bitslcP.html.

Un exemple de macro-instructions est l'instruction *ADD A, B* qui fait l'addition du contenu des deux registres *A* et *B*. Des exemples de micro-instructions sont l'ensemble des opérations nécessaires pour implanter *ADD A, B*. Si notre système contient un ALU (*Arithmetic Logic Unit* ou unité de logique et d'arithmétique, qui est un circuit effectuant les opérations d'arithmétique et de logique), on pourrait penser aux trois micro-instructions suivantes pour implanter le *ADD A, B* :

- 1re micro-instruction : contenu du registre *A* placé dans l'ALU ;
- 2e micro-instruction : contenu du registre *B* placé dans l'ALU ;
- 3e micro-instruction : somme des contenus de *A* et de *B*, et sauvegarde du résultat.

L'ensemble des micro-instructions nécessaires à l'implantation d'une macro-instruction est appelée microséquence. Notre exemple du *ADD A, B* requiert pour son implantation une microséquence de trois micro-instructions.

Évidemment, la première étape consiste à avoir un séquenceur programmable. L'idée de base est celle du compteur associé à une mémoire que nous avons étudiée au chapitre 4. On rajoute des embellissements pour : les interruptions, la pile qui sauve les adresses de retour (*stack* en anglais), les sauts, les boucles, etc. Ce sujet est abordé dans des livres plus avancés. La figure 7.99 reprend cette idée. Notons toutefois que, dans la conception minimaliste illustrée, on a remplacé le compteur par un registre qui emmagasine l'adresse où l'on ira chercher la prochaine ligne de commande dans la mémoire. Cette façon de faire évite d'être astreint à parcourir la mémoire selon une séquence fixe qui débute à 0 et s'incrémente à 1, à 2, etc. Avec un registre, on peut parcourir la mémoire selon une séquence tout à fait arbitraire, ce qui laisse plus de flexibilité. Les signaux de commande du système à la sortie de la mémoire peuvent servir de sorties au circuit pour commander tous les périphériques présents.

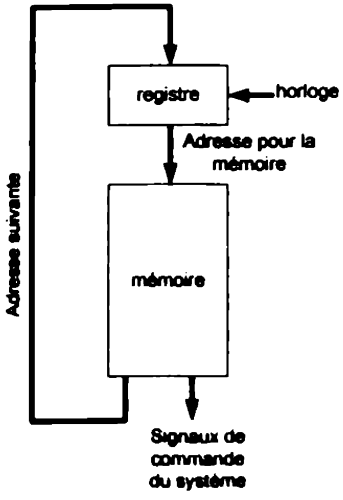


Figure 7.99

Plusieurs microcontrôleurs et microprocesseurs sont microprogrammés ; pensons par exemple au microcontrôleur 8051 discuté plus tôt. L'avantage de l'approche par microprogrammation est évidemment la souplesse : une fois le séquenceur programmable défini, tout est possible (ou presque). En effet, en changeant le microcode, on peut redéfinir le jeu d'instructions auquel ce séquenceur répondra. Il y a cependant un hic. En effet, avec la microprogrammation vient aussi le décodage : chaque macro-instruction correspond à une microséquence contenant plusieurs micro-instructions. Chaque macro-instruction nécessitera donc plusieurs coups d'horloge pour son exécution (c'est ce que l'on appelle le décodage). Comme le temps est précieux (*time is money*), l'aspect souplesse invoqué tantôt n'est pas toujours le critère le plus important. C'est ainsi que les microcontrôleurs et microprocesseurs ne sont pas tous microprogrammés. Certains sont « câblés », c'est-à-dire qu'ils sont construits selon les concepts que nous avons étudiés aux sections précédentes de ce chapitre de sorte qu'alors une instruction s'exécute à chaque coup d'horloge. Il y a là une grande efficacité ; par contre, le design peut vite devenir compliqué. C'est ainsi que beaucoup de machines RISC emploient l'approche câblée, alors que beaucoup de processeurs de type CISC emploient l'approche microprogrammée.

Une question demeure : si on ne travaille pas pour une grande firme de circuits intégrés, (telle Intel, Motorola, IBM, etc.), aura-t-on l'occasion de faire de la microprogrammation ? La réponse est oui. En effet, on retrouve le concept de microprogrammation dans certains processeurs embarqués (aussi appelés *soft processors* puisqu'ils n'existent que sous la forme de bibliothèques et non de « vrais chips ») dans des circuits FPGA et CPLD. Ces *soft processors* étant eux-mêmes microprogrammés. Les PicoBlaze et MicroBlaze de Xilinx sont des exemples d'une telle approche. On peut ajouter ses propres macro-instructions ou modifier le jeu de macro-instructions fourni. Ces macro-instructions sont écrites en langage VHDL.

On peut voir à la figure 7.100 un diagramme-bloc de l'architecture d'un microcontrôleur PicoBlaze[19], celui-ci contenant 49 instructions (mais on peut en ajouter ou en éliminer), huit registres de huit bits, 256 ports adressables directement et indirectement, de même qu'un système d'interruptions masquables (c'est-à-dire qu'on peut ignorer sélectivement). Les programmes incorporés dans le PicoBlaze peuvent normalement contenir 256 instructions (des extensions sont toutefois possibles).

[19] Source : http://direct.xilinx.com/bvdocs/appnotes/xapp387.pdf

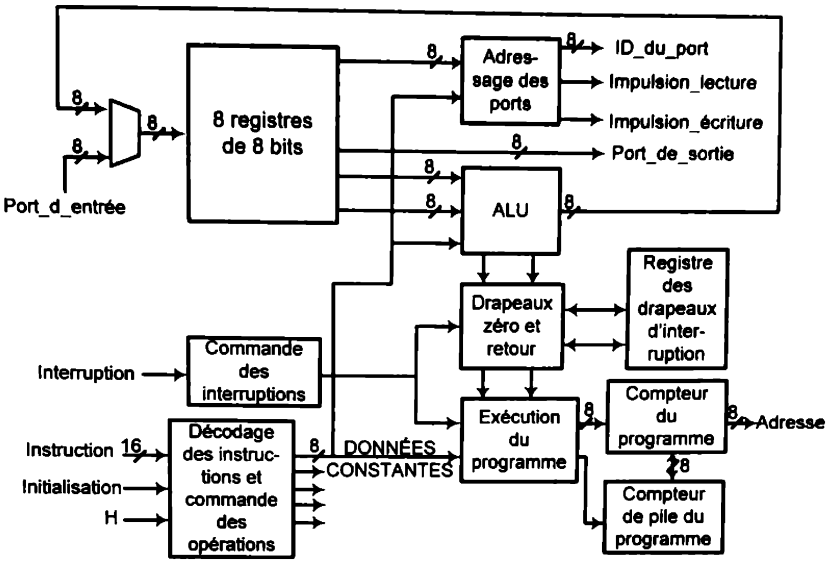


Figure 7.100 Architecture générale du PicoBlaze, avec ses principaux blocs

Un autre exemple est la microprogrammation des processeurs de réseaux (*Network processor* ou XP en anglais). Ainsi, Intel offre des processeurs XP, tel le IXP 2400, très employé par les gros routeurs Internet (par exemple dans la série Tigris d'Ericsson). Un tel chip contient un GPP (*General purpose Processor* en anglais, ou processeur d'application générale) qui est combiné à une série de plusieurs petits processeurs appelés Microengine (petits moteurs) eux-mêmes microprogrammables. L'ingénieure peut ainsi définir son propre jeu de macro-instructions pour faire de la cryptographie, de l'analyse par paquets, etc. Puisque plusieurs Microengines sont utilisés (8, 16 selon les configurations du XP employé), un véritable traitement parallèle est possible, chaque Microengine traitant simultanément un paquet différent ou effectuant un traitement particulier sur chacun de ceux-ci.

Au terme de ce chapitre, et de ce livre, nous sommes maintenant en mesure d'analyser et de concevoir des circuits numériques complexes. Partant des concepts élaborés ici, nous sommes aussi en mesure d'aborder des sujets plus avancés tels les circuits microprogrammés effleurés en fin de chapitre.

PROBLÈMES RÉSOLUS

7.1 Déterminer le diagramme d'état du séquenceur suivant en employant la technique des variables conditionnelles (Q_3 = *MSB*) :

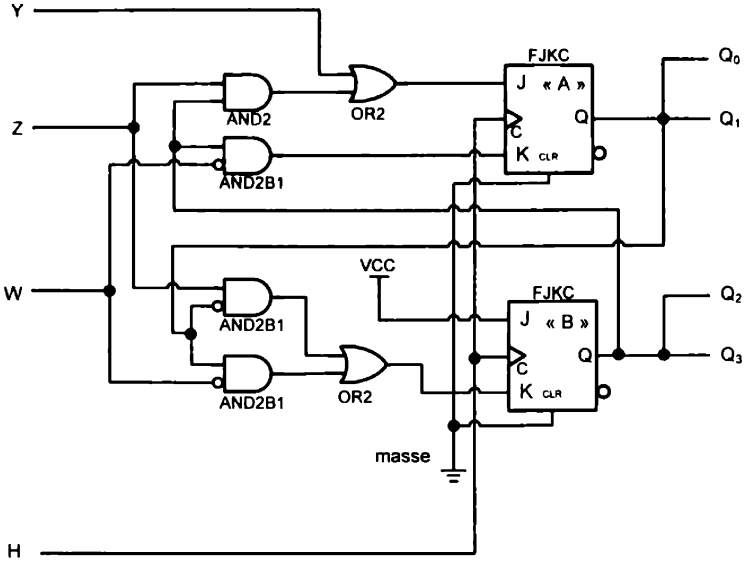


Figure 7.101

Solution : Se référer à la table 7.3A au début du chapitre pour les étapes suivantes.

Étape 1 Les éléments de la machine de Mealy sont bien séparés : deux flip-flops J-K, aucun décodeur de sortie simple (porte ET) et trois entrées externes *Y*, *Z* et *W*.

Étape 2 Les équations des deux flip-flops J-K sont les suivantes, directement obtenues de l'observation du plan du circuit :

$$J_A = Y + Z \cdot Q_B$$

$$K_A = Q_B \cdot \overline{W}$$

$$J_B = 1$$

$$K_B = Z \cdot \overline{Q_A} + Q_A \cdot \overline{W}$$

Étape 3 Il faut dresser la table ÉP-ÉS en employant les entrées externes *Y*, *Z*, *W* comme variables conditionnelles. Comme on a deux flip-flops, il y aura 4 (= 2^2) lignes dans la table, une par état. On débute à l'état « 00 ». Ainsi, avec $Q_A = Q_B = 0$ dans la section ÉP (= État Présent), on obtient les valeurs suivantes $J_A = Y$, $K_A = 0$, $J_B = 1$ et $K_B = Z$. Passons ensuite à l'état « 01 ». Ainsi, avec $Q_A = 1$, $Q_B = 0$ dans la zone ÉP, on obtient les valeurs suivantes $J_A = Y$, $K_A = 0$, $J_B = 1$, $K_B = \overline{W}$, et ainsi de

suite. Les colonnes Q_B et Q_A de la section ÉS (= État Suivant) sont obtenues selon la table 5.13 et aussi selon la méthode présentée maintenant.

- Pour les flip-flops, il faut procéder à l'étude des stimulations et des états suivants possibles en fonction de la valeur des variables, comme suit, état par état. Pour chaque état, on bâtit une table en indiquant l'état de départ, puis les valeurs possibles de stimulation selon les valeurs des variables (0 et 1), ce qui permet d'établir les états suivants possibles. Par la suite, puisqu'il suffit d'étudier seulement les « 1 » (car, si on ne passe pas à « 1 », alors on reste à « 0 »). On procède à une opération d'*union* des cas qui donnent « 1 » en unissant (opération OU logique) les *valeurs des variables* qui donnent un « 1 » dans la colonne *États suivants possibles*. Lorsque tous les états ont été étudiés de la sorte, on reporte les valeurs dans la table ÉP-ÉS pour les états futurs.

  Cas du flip-flop B :

  - À l'état ⓪ et ① (ceci est valable pour les cas $J_B = 1$ et $K_B = Z$ ou $\overline{W}$), les directions possibles sont :

État de départ (Q_B)		Valeur de la variable	J_B	K_B	Mode du flip-flop	États suivants possibles
0	si	Z ($Z = 1$)	1	1	Toggle	1
0	si	$\overline{Z}$ ($Z = 0$)	1	0	Mise à 1	1
					Union :	$Z + \overline{Z} = 1$

Figure 7.102

  - À l'état ② et ③ (ceci est aussi valable pour les cas $J_B = 1$ et $K_B = Z$ ou $\overline{W}$), les directions possibles sont :

État de départ (Q_1)		Valeur de la variable	J_1	K_1	Mode du flip-flop	États suivants possibles
1	si	Z	1	1	Toggle	0
1	si	$\overline{Z}$	1	0	Mise à 1	1
					Union :	$\overline{Z}$

Figure 7.103

Cas du flip-flop « A » :

- À l'état ⓪, les directions possibles sont :

État de départ (Q_A)		Valeur de la variable	J_A	K_A	Mode du flip-flop	États suivants possibles
0	si	Y ($Y = 1$)	1	0	Mise à 1	1
0	si	$\overline{Y}$ ($Y = 0$)	0	0	Mémorisation	0
					Union :	Y

Figure 7.104

- À l'état ①, les directions possibles sont :

État de départ (Q_A)		Valeur de la variable	J_A	K_A	Mode du flip-flop	États suivants possibles
1	si	Y ($Y = 1$)	1	0	Mise à 1	1
1	si	$\overline{Y}$ ($Y = 0$)	0	0	Mémorisation	1
					Union :	$Y + \overline{Y} = 1$

Figure 7.105

- À l'état ②, les directions possibles sont :

État de départ (Q_A)		Valeur de la variable Y	Valeur de la variable Z	Valeur de la variable W	J_A $Y + Z$	K_A $\overline{W}$	Mode du flip-flop	États suivants possibles
0	si	$\overline{Y}$ ($Y = 0$)	$\overline{Z}$ ($Z = 0$)	$\overline{W}$ ($W = 0$)	0	1	Mise à 0	0
0	si	$\overline{Y}$ ($Y = 0$)	$\overline{Z}$ ($Z = 0$)	W ($W = 1$)	0	0	Mémorisation	0
0	si	$\overline{Y}$ ($Y = 0$)	Z ($Z = 1$)	$\overline{W}$ ($W = 0$)	1	1	Toggle	1
0	si	$\overline{Y}$ ($Y = 0$)	Z ($Z = 1$)	W ($W = 1$)	1	0	Mise à 1	1
0	si	Y ($Y = 1$)	$\overline{Z}$ ($Z = 0$)	$\overline{W}$ ($W = 0$)	1	1	Toggle	1
0	si	Y ($Y = 1$)	$\overline{Z}$ ($Z = 0$)	W ($W = 1$)	1	0	Mise à 1	1
0	si	Y ($Y = 1$)	Z ($Z = 1$)	$\overline{W}$ ($W = 0$)	1	1	Toggle	1
0	si	Y ($Y = 1$)	Z ($Z = 1$)	W ($W = 1$)	1	0	Mise à 1	1
							Union :	$Y + Z$

Figure 7.106

L'état suivant possible est alors donné en « unissant » tous les cas qui conduisent à 1 dans la table précédente ; cela donne :

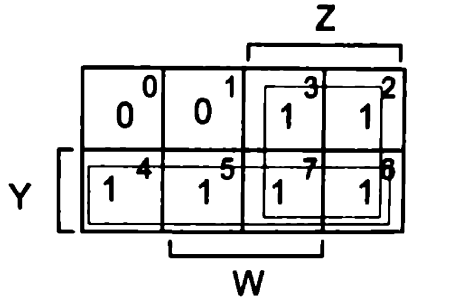


Figure 7.107

- À l'état ③, les directions possibles sont :

État de départ (Q_A)		Valeur de la variable Y	Valeur de la variable Z	Valeur de la variable W	J_A $Y+Z$	K_A $\overline{W}$	Mode du flip-flop	États suivants possibles
1	si	$\overline{Y}$ ($Y=0$)	$\overline{Z}$ ($Z=0$)	$\overline{W}$ ($W=0$)	0	1	Mise à 0	0
1	si	$\overline{Y}$ ($Y=0$)	$\overline{Z}$ ($Z=0$)	W ($W=1$)	0	0	Mémorisation	1
1	si	$\overline{Y}$ ($Y=0$)	Z ($Z=1$)	$\overline{W}$ ($W=0$)	1	1	Toggle	0
1	si	$\overline{Y}$ ($Y=0$)	Z ($Z=1$)	W ($W=1$)	1	0	Mise à 1	1
1	si	Y ($Y=1$)	$\overline{Z}$ ($Z=0$)	$\overline{W}$ ($W=0$)	1	1	Toggle	0
1	si	Y ($Y=1$)	$\overline{Z}$ ($Z=0$)	W ($W=1$)	1	0	Mise à 1	1
1	si	Y ($Y=1$)	Z ($Z=1$)	$\overline{W}$ ($W=0$)	1	1	Toggle	0
1	si	Y ($Y=1$)	Z ($Z=1$)	W ($W=1$)	1	0	Mise à 1	1
							Union :	W

Figure 7.108

L'état suivant possible est alors donné en « unissant » tous les cas qui conduisent à 1 dans la table précédente ; cela donne :

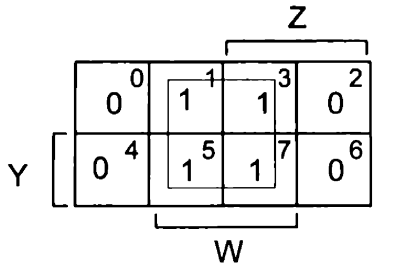


Figure 7.109

La table ÉP-ÉS finale est alors complète :

	ÉP		Stimulation				ÉS	
Rang	Q_B	Q_A	J_B	K_B	J_A	K_A	Q_B	Q_A
⓪	0	0	1	Z	Y	0	1	Y
①	0	1	1	$\overline{W}$	Y	0	1	1
②	1	0	1	Z	$Y+Z$	$\overline{W}$	$\overline{Z}$	$Y+Z$
③	1	1	1	$\overline{W}$	$Y+Z$	$\overline{W}$	W	W

Figure 7.110

Étape 4 La table ÉP-ÉS nous permet de dresser le diagramme d'état :

- À l'état ⓪, on se retrouve à l'état ③ après le coup d'horloge si $Y = 1$ (donc si Y), et à l'état ② après le coup d'horloge si $Y = 0$ (donc si $\overline{Y}$) ;
- À l'état ①, on se retrouve inconditionnellement à l'état ③ après le coup d'horloge ;
- À l'état ②, on se retrouve, après le coup d'horloge, à :

Y	Z	Q_B	Q_A	ÉS	Valeurs des variables
0	0	1	0	②	$\overline{Y} \cdot \overline{Z}$
0	1	0	1	①	$\overline{Y} \cdot Z$
1	0	1	1	③	$Y \cdot \overline{Z}$
1	1	0	1	①	$Y \cdot Z$

Figure 7.111

On ira à l'état 1, avec $\overline{Y} \cdot Z$ ou $Y \cdot Z$, donc $\overline{Y} \cdot Z + Y \cdot Z = Z$;

- À l'état ③, on se retrouve à l'état ③ après le coup d'horloge si $W = 1$ (donc si W), et à l'état ⓪ après le coup d'horloge si $W = 0$ (donc si $\overline{W}$).

Il n'y a pas de décodeur de sortie, par contre il y a une assignation de bits : $Q_B = Q_3 = Q_2$, et $Q_A = Q_1 = Q_0$. Ainsi, les quatre sorties du circuit auront les valeurs hexadécimales suivantes :

Q_B	Q_A	Q_3	Q_2	Q_1	Q_0	Sorties
0	0	0	0	0	0	0_h
0	1	0	0	1	1	3_h
1	0	1	1	0	0	C_h
1	1	1	1	1	1	F_h

Figure 7.112

De cette discussion, le diagramme d'état suivant est obtenu :

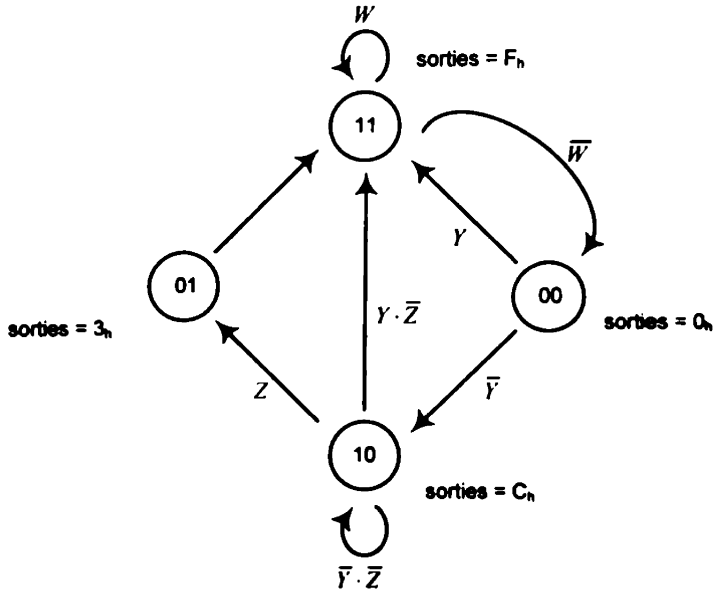


Figure 7.113

Comme on le constate, la technique des variables conditionnelles a permis d'obtenir une table ÉP-ÉS compacte puisque les trois variables sont directement incluses dans la table. Autrement, une table de 32 lignes aurait été nécessaire (3 variables + 2 bits = 5, et $2^5 = 32$). Par ailleurs, le diagramme d'état indique que le circuit implante une fonction relativement complexe.

7.2 Compléter les zones grises du diagramme d'état suivant (il y a trois variables impliquées ici : *C*, *B* et *A* ; *A* = *MSB*) :

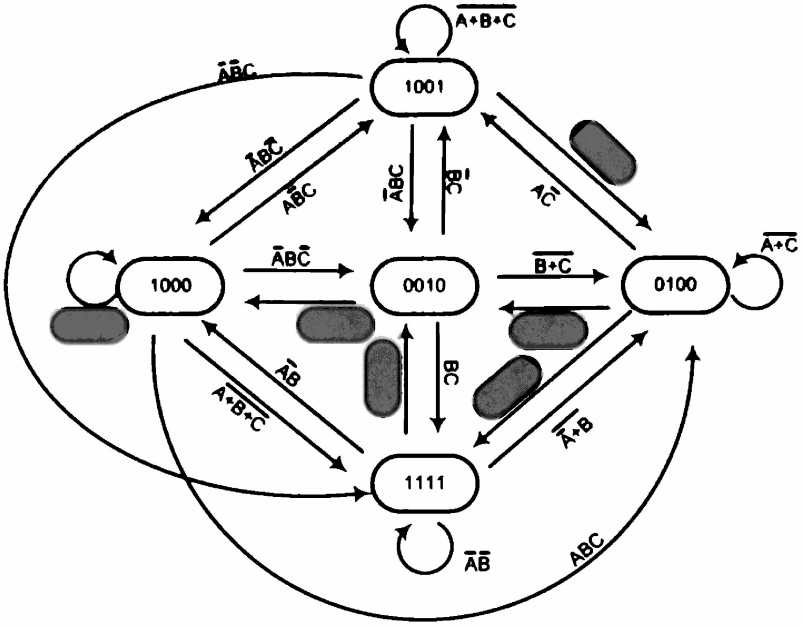


Figure 7.114

Solution : Pour chaque état, il faut :

1. couvrir toutes les possibilités non déjà spécifiées en considérant les variables présentes ;
2. ne pas avoir plus d'une direction possible pour une même combinaison de variables.

S'il y a trois variables impliquées, on aura 8 (= 2^3) combinaisons possibles de variables ; voici d'ailleurs la liste avec les mintermes correspondants :

Rang	Valeur binaire (A, B, C)	minterme
⓪	000	$\overline{A} \cdot \overline{B} \cdot \overline{C}$
①	001	$\overline{A} \cdot \overline{B} \cdot C$
②	010	$\overline{A} \cdot B \cdot \overline{C}$
③	011	$\overline{A} \cdot B \cdot C$
④	100	$A \cdot \overline{B} \cdot \overline{C}$
⑤	101	$A \cdot \overline{B} \cdot C$
⑥	110	$A \cdot B \cdot \overline{C}$
⑦	111	$A \cdot B \cdot C$

Figure 7.115

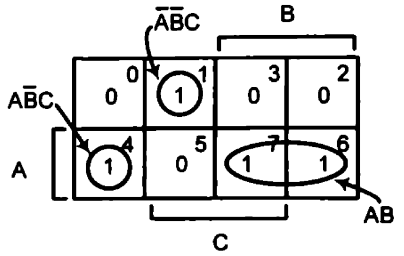
Par la suite, il suffit d'étudier chaque état pour découvrir ce qu'il manque :

- État 1001 : il y a cinq directions indiquées :

Direction	minterme	Rang du minterme
1001	$\overline{A+B+C} = \overline{A}\cdot\overline{B}\cdot\overline{C}$	⓪
0010	$\overline{A}\cdot B\cdot C$	③
0100	?	?
1000	$\overline{A}\cdot B\cdot\overline{C}$	②
1111	$A\cdot\overline{B}\cdot C$	⑤

Figure 7.116

On dresse une table de Karnaugh où l'on mettra des « 1 » pour les rangs des mintermes non présents dans la table précédente (c'est-à-dire : ①, ④, ⑥ et ⑦) ; cela permettra de respecter les deux conditions précédemment énoncées (1°, 2°) :



Combinaison de variables = $AB + \overline{A}\overline{B}C + A\overline{B}C$

Figure 7.117

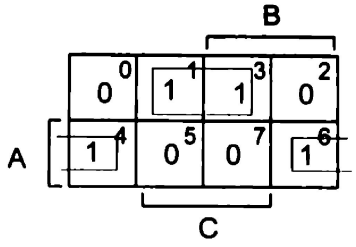
La combinaison manquante est donc : $AB + \overline{A}\overline{B}C + A\overline{B}C$.

- État 1000 : Il y a cinq directions indiquées :

Direction	minterme	Rang du minterme
1000	?	?
1001	$A\cdot\overline{B}\cdot C$	⑤
0010	$\overline{A}\cdot B\cdot\overline{C}$	②
1111	$\overline{A+B+C} = \overline{A}\cdot\overline{B}\cdot\overline{C}$	⓪
0100	$A\cdot B\cdot C$	⑦

Figure 7.118

On dresse une table de Karnaugh où l'on mettra des « 1 » pour les rangs des mintermes non présents dans la table précédente (c'est-à-dire : ①, ③, ④, ⑥), cela permettra de respecter les deux conditions précédemment énoncées (1°, 2°) :



Combinaison de variables = $A \cdot \overline{C} + \overline{A} \cdot C = A \oplus C$

Figure 7.119

La combinaison manquante est donc : $A \oplus C$.

- Pour les trois autres états restants (0010, 1111 et 0100), on remarque : il n'y a que quatre directions indiquées et seulement deux variables impliquées chaque fois ; on aura alors 4 (= 2^2) combinaisons possibles de variables. Voici d'ailleurs la liste avec les mintermes correspondants pour chacun des trois états :

Valeur binaire (B, C)	minterme	Valeur binaire (A, C)	minterme	Valeur binaire (A, B)	minterme
000	$\overline{B+C} = \overline{B} \cdot \overline{C}$	000	$\overline{A+C} = \overline{A} \cdot \overline{C}$	000	$\overline{A} \cdot \overline{B}$
001	$\overline{B} \cdot C$	001	$\overline{A} \cdot C$	001	$\overline{A} \cdot B$
010	$B \cdot \overline{C}$	010	$A \cdot \overline{C}$	010	$\overline{\overline{A}+B} = A \cdot \overline{B}$
011	$B \cdot C$	011	$A \cdot C$	011	$A \cdot B$
État 0010	État 0010	État 0100	État 0100	État 1111	État 1111

Figure 7.120

On a chaque fois indiqué (sur fond blanc) la combinaison manquante qui est celle à indiquer dans le diagramme d'état. On note que, pour l'état 0100, on pourra au choix spécifier l'une ou l'autre combinaison.

Cela nous permet maintenant de compléter le diagramme :

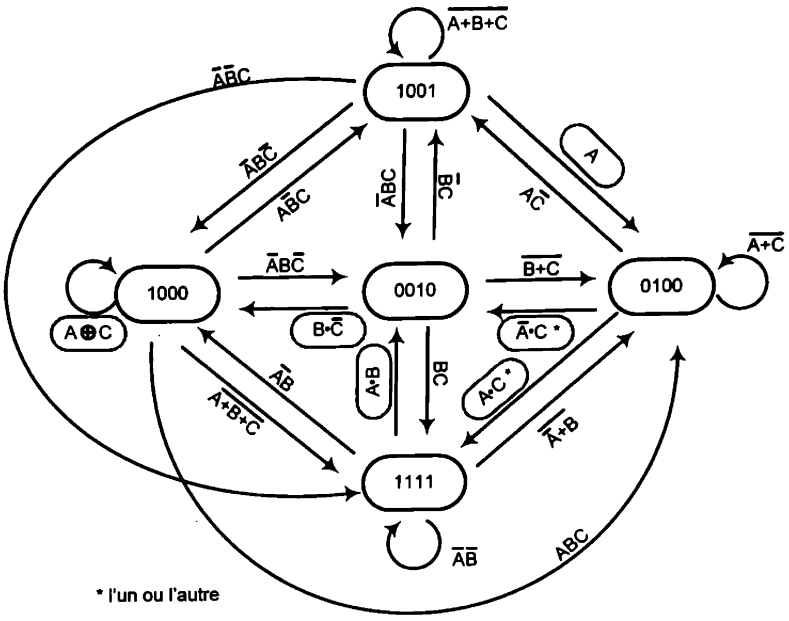


Figure 7.121

7.3 (*Québec, mars 2024*) Vous êtes ingénieure chez CMQ (Corporation des Moteurs du Québec), et votre tâche consiste à réaliser le séquenceur des clignotants et feux de détresse (*hazard lights*) du nouvel utilitaire *Charlevoix 4x4 Électrique 2025* qui sera bientôt vendu dans le monde entier. Les signaux d'entrées auxquels vous avez accès sont les suivants:

- *CG* – si *CG* = 1 → clignote à gauche ;
- *CD* – si *CD* = 1 → clignote à droite ;
- *HA* – si *HA* = 1 → feux de détresse (*hazard lights*) ;
- Une Horloge de 1 Hz (donc une période de 1 s, idéale pour un clignotement *dynamique* selon l'équipe du marketing !).

On a aussi que *HA* est bien sûr prioritaire sur *CD* et *CG*, et qu'on ne peut avoir *CG* et *CD* simultanément (le bras des clignotants étant soit levé, soit abaissé).

Le *Charlevoix 4x4 Électrique 2025* sera équipé de quatre blocs d'éclairage à DEL[20] blanches de haute intensité dédiés aux clignotants et aux feux de détresse :

- *AG* (avant gauche), *AD* (avant droit), *RG* (arrière gauche) et *RD* (arrière droit) (voir la figure 7.122).

Les séquences de clignotement sont les suivantes :

[20] Diodes électroluminescentes.

- clignote à gauche : *AG* et *RG* : allumés (1 s), éteints (1 s), allumés (1 s), et ainsi de suite ;
- clignote à droite : *AD* et *RD* : allumés (1 s), éteints (1 s), allumés (1 s), et ainsi de suite ;
- feux de détresse : AG, *RG*, *AD*, *RD* : allumés (1 s), éteints (1 s), allumés (1 s), et ainsi de suite.

On vous demande de réaliser ce séquenceur (sur deux bits) pour réaliser les fonctions de clignote à gauche, de clignote à droite et de feux de détresse en employant un CB2CLED très populaire au Québec. Votre séquenceur sera inclus dans le CPLD implantant l'ordinateur de bord du *Charlevoix 4x4 Électrique 2025*.

Faire le diagramme d'état complet du design.

Dessiner le plan du circuit en incluant les sorties pour connexions aux blocs d'éclairage (par l'électronique de puissance).

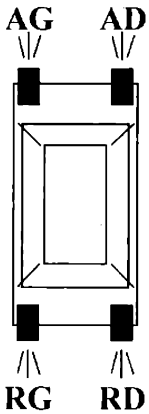


Figure 7.122

Solution : Se référer à la table 7.3D au début du chapitre pour les étapes suivantes.

Étape 0 On peut considérer quatre états permettant de réaliser les fonctions demandées (appelons les deux bits Q_1 et Q_0) :

- Éteint : toutes les sorties à 0 (si rien ne se passe) ; nommons-le : 00 ;
- Allumé : toutes les sorties à 1 (pour la fonction de détresse) ; nommons-le : 11 ;
- TAG : toutes les sorties gauches activées (pour la fonction clignote à gauche) ; nommons-le : 10 ;
- TAD : toutes les sorties droites activées (pour la fonction clignote à droite), nommons-le : 01.

Pour l'auto-correction, elle est obtenue de facto puisque, comme il y a quatre états sur 2 bits ($2^2 = 4$), tous les états possibles sont couverts quoiqu'il advienne.

Pour les combinaisons des variables, on se bâtit une table de vérité en suivant les règles précisées précédemment : *une direction possible par combinaison de variables et en couvrant toutes les combinaisons possibles* :

Variables			Actions				fds
HA	*CG*	*CD*	Allumé	TAG	TAD	Éteint	Combinaison valide
0	0	0	0	0	0	1	$\overline{HA}\cdot\overline{CG}\cdot\overline{CD} = \overline{HA+CG+CD}$
0	0	1	0	0	1	0	$\overline{HA}\ CD$
0	1	0	0	1	0	0	$\overline{HA}\cdot CG$
0	1	1	X	X	X	X	Ne peut se produire
1	0	0	1	0	0	0	*HA*
1	0	1	1	0	0	0	*HA*
1	1	0	1	0	0	0	*HA*
1	1	1	1	0	0	0	*HA*

Figure 7.123

Puisque *CG* et *CD* ne peuvent apparaître simultanément, une certaine simplification est possible. Ainsi, la combinaison $\overline{HA}\cdot CG\cdot CD$ est exclue du design, de même que $\overline{HA}\cdot CD\cdot\overline{CG}$ est ici équivalent à $\overline{HA}\cdot CD$ et $\overline{HA}\cdot\overline{CD}\cdot CG$ est ici équivalent à $\overline{HA}\cdot CG$.

De ces définitions, on peut directement tracer le diagramme d'état suivant (les sorties *AG*, *AD*, *RG*, *RD* sont aussi indiquées) :

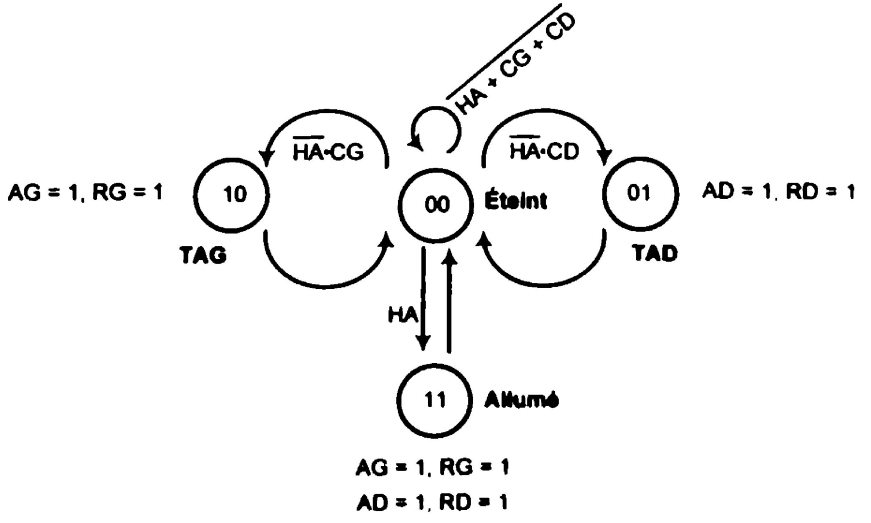


Figure 7.124

On note que les retours sont inconditionnels puisque, après 1 s, l'éclairage doit s'éteindre avant de reprendre selon la valeur des variables.

On peut maintenant entreprendre le design.

Étape 1 *Établissement de la table des actions.* On se rappelle que la commande *L* (*load*/charge) du CB2CLED est prioritaire sur les commandes *CE* (compte) et *U* (*up-down*/incrémente-décrémente). Du diagramme d'état, on note aussi que le comptage, lorsqu'il aura lieu, sera toujours incrémental (+1), et donc que *U* sera toujours 1. Cela simplifie le problème. En se rappelant la discussion de l'exemple précédent, on étudie chaque état en fonction du diagramme d'état établi à l'étape précédente :

- État 0 (case 0) : *CE*+ sera employé pour aller à l'état 1. *CE*– sera employé pour aller à l'état 3, *L* sera employé pour aller à l'état 2 ;
- État 1 (case 1) : *CE*– sera employé pour aller à l'état 0 ;
- État 2 (case 2) : *L* sera employé pour aller à l'état 0 ;
- État 3 (case 3) : *CE*+ sera employé pour aller à l'état 0.

La table des actions est donc :

Table des Actions:

Q_1 \ Q_0		
	0	1
	L, CE-,+	CE-
	2	3
	L	CE+

Figure 7.125

Étape 2 Étude des variables pour déterminer *L*, *CE* et *U*.

- État 0 : On a trois variables impliquées (*HA*, *CG* et *CD*) ; il faut déterminer leur influence, ce qui se fait simplement avec une table de vérité :

Directions possibles	*L*	*CE*	*U*	Bits de données $D_1(D_1, D_0)$	Si
État 0	0	0	*X*	*XX*	$\overline{HA + CG + CD}$
État 1	0	1	1	*XX*	$\overline{HA} \cdot CD$
État 2	1	*X*	*X*	10	$\overline{HA} \cdot CG$
État 3	0	1	0	*XX*	*HA*
Union :	$\overline{HA} \cdot CG$	$HA + CD$	$\overline{HA}$	10	

Figure 7.126

Avec :

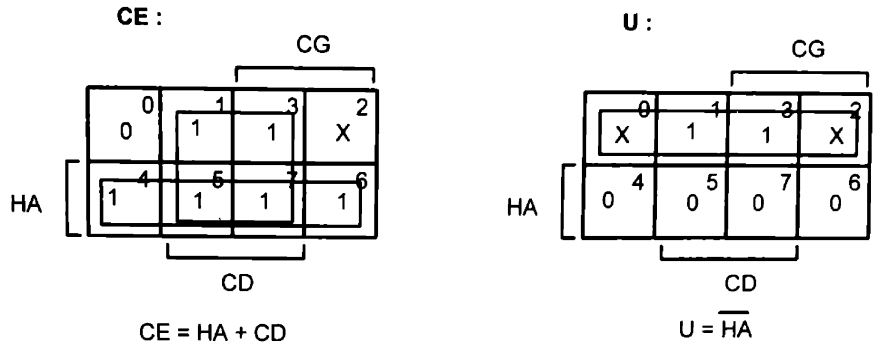


Figure 7.127

Il est à noter que l'on met X à CE à l'état 2 puisque, L étant prioritaire, la valeur de CE importe peu. Quant aux valeurs des D_i (D_1 et D_0), c'est seulement lorsqu'on effectue un L qu'elles sont importantes, ce qui permet de mettre des X ailleurs. Ceci simplifie bien sûr le circuit obtenu.

- État 1 : Seul CE est utilisé inconditionnellement avec $U = 0$. Donc, $L = 0$ et $CE = 1$, et les $D_i = X$ puisqu'ils ne servent pas.
- État 2 : Seul L est utilisé inconditionnellement. Donc, $L = 1$, $U = X$ et $CE = X$, et les $D_i = 00$.
- État 3 : Seul CE est utilisé inconditionnellement avec $U = 1$. Donc, $L = 0$ et $CE = 1$, et les $D_i = X$ puisqu'ils ne servent pas.

Étapes 3 et 4 Il faut maintenant déterminer L, CE et les D_i globalement. Cela s'effectue en dressant les tables de Karnaugh selon les résultats obtenus pour chaque état à l'étape précédente :

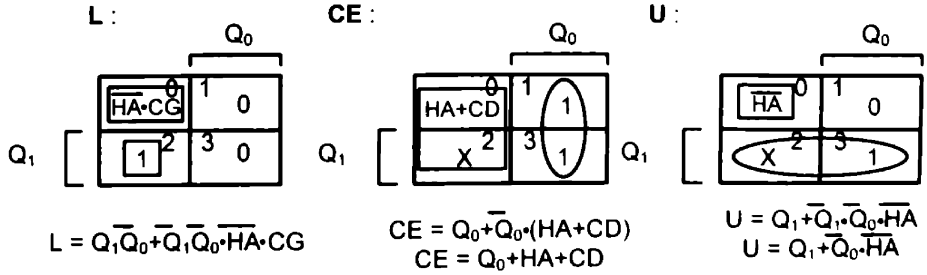


Figure 7.128 a)

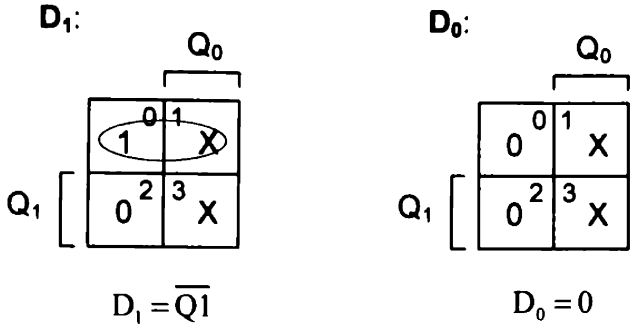


Figure 7.128 b)

Cela permet d'obtenir les valeurs simplifiées. Pour ce qui est de *CE* et des D_i, on peut obtenir des expressions relativement simples qui sont indiquées. *L*, par contre, est plus complexe, et on opte donc pour l'emploi d'un multiplexeur pour implanter la table correspondante.

Étape 5 (avec MUX non simplifié pour *L*) : *L* prend trois valeurs (0, 1 et $\overline{HA} \cdot CG$) ; aucune simplification n'est possible puisqu'il faut au minimum un MUX 4 à 1 (un MUX 2 à 1 n'est pas suffisant). On a donc, pour les entrées du MUX 4 à 1 :

État	Valeur de CE
0	$\overline{HA \cdot CG}$
1	0
2	1
3	0

Figure 7.129

Étape 6 Les sorties des bits doivent être synthétisées selon le diagramme d'état de l'étape 0. On peut écrire immédiatement les expressions algébriques équivalentes : *RG* et *AG* = Q_1, et *RD* et *AD* = Q_0. Cela se voit immédiatement de la table de vérité suivante :

Q_1Q_0	Valeur de *RG* et de *AG*	Valeur de *RD* et de *AD*
0 0	0	0
0 1	0	1
1 0	1	0
1 1	1	1

Figure 7.130

Étape 7 On est maintenant prêt à monter le circuit. Il suffit de reprendre les résultats obtenus aux étapes précédentes : étapes 3 et 5 pour *L* ; étape 3 pour *CE* et *U* ; étape 4 pour les D_i ; étape 6 pour les sorties. Le reste n'est que du montage, réalisé ici dans Xilinx Foundation. *Note* : les signaux sont appelés ici : horloge → *I_HH*3, *HA* → *I_HA*3, *CG* → *I_CG*3, *CD* → *I_CD*3.

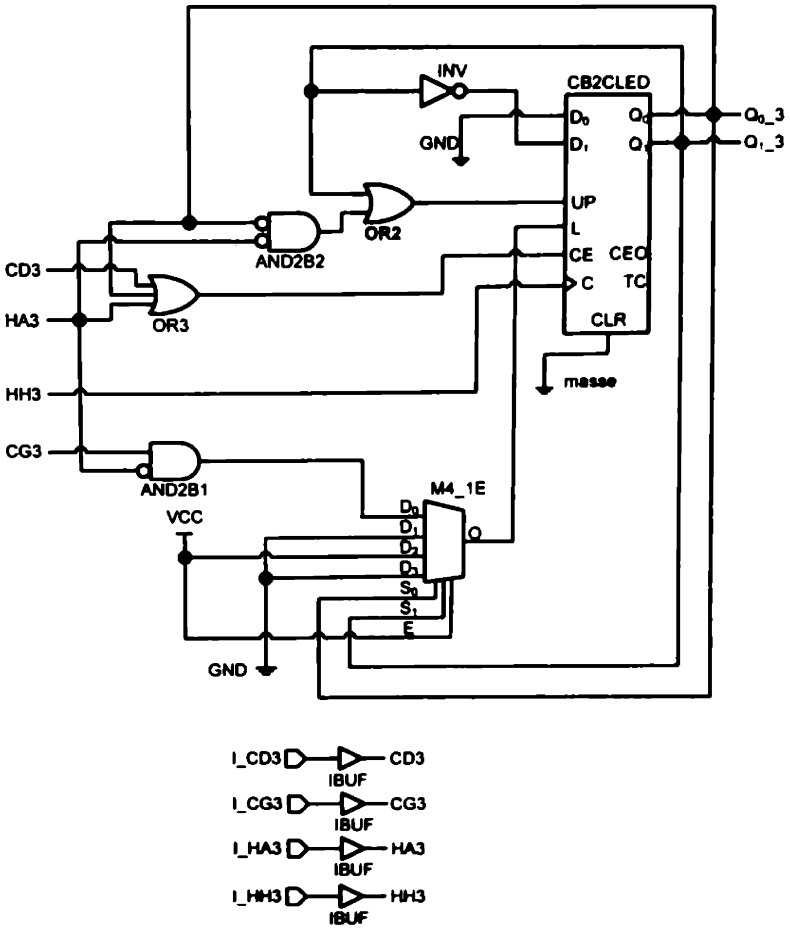


Figure 7.131

L'utilisation d'un compteur rend le design somme toute relativement simple. Finalement, il est recommandé de tester le circuit que l'on vient de concevoir. Xilinx Foundation permet une simulation efficace, qui est fournie ici pour quelques situations de fonctionnement :

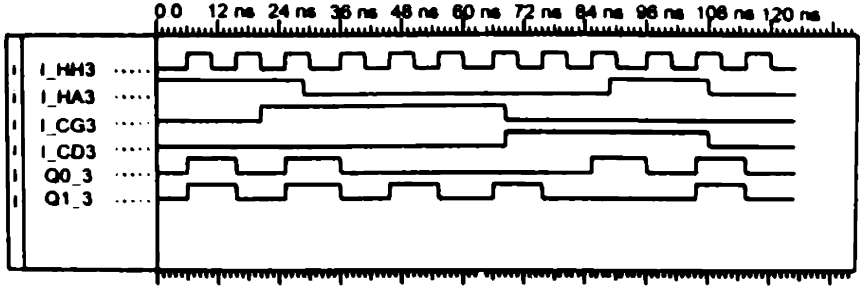
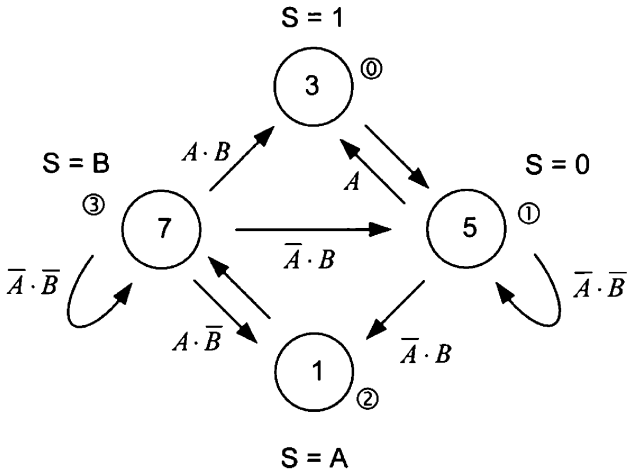


Figure 7.132 Chronogramme illustrant quelques situations de fonctionnement du circuit du problème 3 (*i* pour *input* /entrée, et *I* pour nœud interne)

Votre design est magnifique ; vous avez hâte d'essayer le nouvel utilitaire *Charlevoix 4x4 Électrique 2025*.

7.4. On vous demande de réaliser avec un CB2CLED le circuit qui implantera le diagramme d'état suivant, il s'agit d'un petit contrôleur pour une application d'électrotechnique. Les signaux d'entrées auxquels vous avez accès sont les suivants :

- A, B sont les entrées et S la sortie.
- Une horloge H est fournie également.



Note: les chiffres encerclés désignent les nouveaux états

Figure 7.133

Reprenons maintenant rapidement les diverses étapes de design.

Étape 0 Comme il y a 4 états, 2 bits suffisent, appelons-les Q_y, Q_x. Il faut cependant renommer les anciens états codés sur 3 bits ci-dessous appelés Q_2, Q_1, Q_0. La table suivante illustre le changement :

	Anciens états			Nouveaux états		
Anciens états	Q_2	Q_1	Q_0	Q_y	Q_x	Nouveaux états
①	0	0	1	1	0	②
③	0	1	1	0	0	⓪
⑤	1	0	1	0	1	①
⑦	1	1	1	1	1	③

Figure 7.134

Le choix des nouveaux états permettra une utilisation optimale des fonctions du CB2CLED.

Pour l'auto-correction, elle est obtenue de facto puisque, comme il y a quatre états sur 2 bits ($2^2 = 4$), tous les états possibles sont couverts quoiqu'il advienne.

Étape 1 Établissons maintenant la table des actions correspondante ce qui est assez direct à partir du diagramme d'états avec les nouveaux états :

- État 0 : *CE+* sera employé pour aller à l'état 1.
- État 1 : *CE+* sera employé pour aller à l'état 2. *CE-* sera employé pour aller à l'état 0.
- État 2 : *CE+* sera employé pour aller à l'état 3.
- État 3 : *CE+* sera employé pour aller à l'état 0. *CE-* sera employé pour aller à l'état 2. *L* sera employé pour aller à l'état 1.

La *Table des actions* est donc :

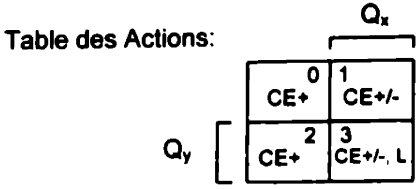


Figure 7.135

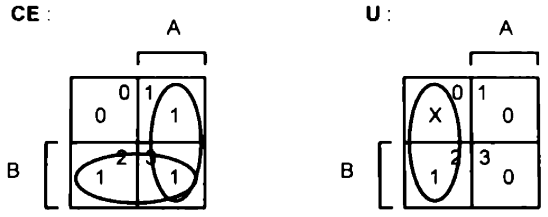
Étape 2 Étude des variables pour déterminer *L*, *CE* et *U* :

- État 0 : seul *CE* est utilisé inconditionnellement avec $U = 1$. Donc $L = 0$ et $CE = 1$ et les $D_i = X$ puisqu'ils ne servent pas.
- État 1 : il nous faut étudier les différents cas possibles obtenus en fonction des valeurs des variables *A* et *B*. Cela se fait aisément à l'aide de la table de vérité suivante :

Variables	B	A	CE	U	Remarques
$\overline{B} \cdot \overline{A}$	0	0	0	X	Reste sur place
$\overline{B} \cdot A$	0	1	1	0	Va à ①, compte -1
$B \cdot \overline{A}$	1	0	1	1	Va à ②, compte +1
$B \cdot A$	1	1	1	0	Va à ③, compte -1

Figure 7.136

De cette table, on déduit immédiatement les valeurs de *CE* et de *U* :



$CE = A + B$ $U = \overline{A}$

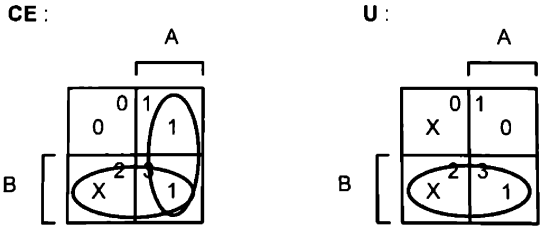
Figure 7.137

- État 2 : seul CE est utilisé inconditionnellement avec $U = 1$. Donc $L = 0$ et $CE = 1$ et les $D_i = X$ puisqu'ils ne servent pas.
- État 3 : il nous faut étudier les différents cas possibles obtenus en fonction des valeurs des variables A et B. Cela se fait aisément à l'aide de la table de vérité suivante :

Variables	B	A	L	CE	U	Dy	Dx	Remarques
$\overline{B} \cdot \overline{A}$	0	0	0	0	X	X	X	Reste sur place
$\overline{B} \cdot A$	0	1	0	1	0	X	X	Va à ②
$B \cdot \overline{A}$	1	0	1	X	X	0	1	Va à ①
$B \cdot A$	1	1	0	1	1	X	X	Va à ⓪

Figure 7.138

De cette table, on déduit immédiatement les valeurs de CE et de U :



$CE = A + B$ $U = B$

Figure 7.139

Quant à L, on voit immédiatement que $L = B\,\overline{A}$. De même pour $D_y = 0$ et $D_x = 1$.

Étapes 3 et 4 Il faut maintenant déterminer L, CE et les D_i globalement. Cela s'effectue, en dressant les tables de Karnaugh selon les résultats obtenus pour chaque état à l'étape précédente :

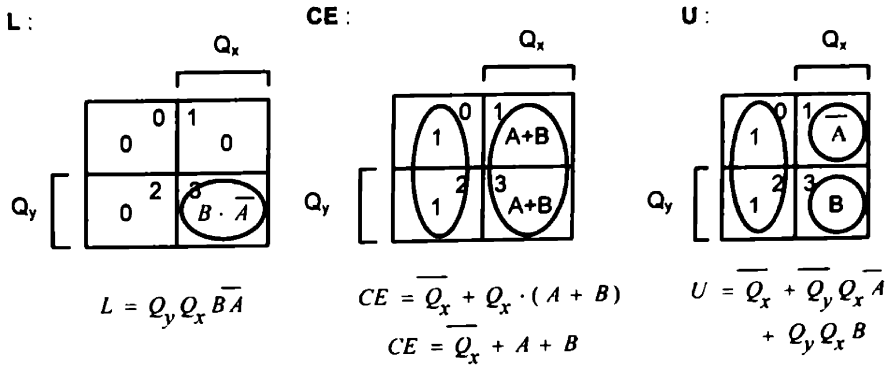


Figure 7.140

Quant aux D_i, on a directement que $D_y = 0$ et $D_x = 1$ puisqu'ils n'interviennent qu'à l'état 3 ($D_i = X$ aux autres états avec $L = 0$).

Comme les expressions sont simples, on n'aura pas recours à un multiplexeur. Il faut cependant « convertir » les anciens états selon les nouveaux états (étape 0), cela se fait selon la table suivante :

Q_yQ_x	Q_2	Q_1	Q_0
00	0	1	1
01	1	0	1
10	0	0	1
11	1	1	1

Figure 7.141

On voit immédiatement que $Q_2 = Q_x$, $Q_1 = \overline{Q_y \oplus Q_x}$ et $Q_0 = 1$.

Étape 6 La sortie S du circuit doit être finalement synthétisée selon le diagramme d'état fourni, cela s'obtient de la table de vérité suivante :

Q_yQ_x	Sortie S
00	1
01	0
10	A
11	B

Figure 7.142

L'idéal est ici d'employer un multiplexeur 4 à 1 pour implanter S (puisqu'on a quatre cas distincts).

Étape 7 On est maintenant prêt à monter le circuit. Il suffit de reprendre les résultats obtenus aux étapes précédentes : Étapes 3 et 4 pour L, CE et les D_i. Étape 6 pour la sortie S. Le reste n'est que du montage, réalisé ici dans Xilinx

Foundation. *Note* : les signaux sont appelés ici : horloge *I_H* avec comme entrées *I_A* (pour *A*) et *I_B* (pour *B*) et comme sortie *S*, le signal SORTIE.

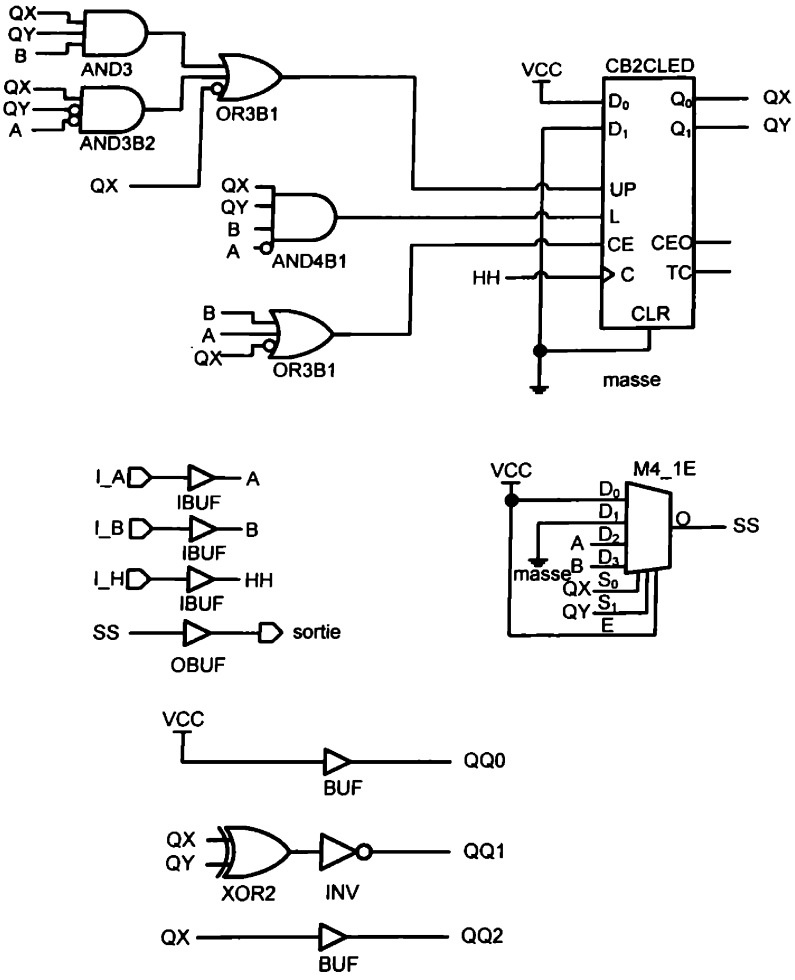


Figure 7.143

L'utilisation d'un compteur rend le design somme toute relativement simple. Finalement, il est recommandé de tester le circuit que l'on vient de concevoir. Xilinx Foundation permet une simulation efficace qui est fournie ici pour quelques situations de fonctionnement :

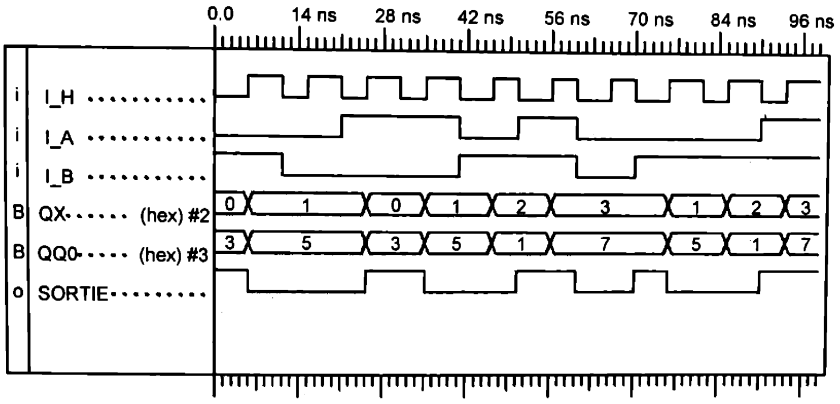


Figure 7.144 Chronogramme illustrant quelques situations de fonctionnement du circuit du problème 4 (*i* pour *input* /entrée et *o* pour *output*/sortie).

À noter : les valeurs de Q_y et de Q_x ont été combinées sous forme d'un bus, de même pour Q_2, Q_1 et Q_0. On remarquera les changements qui surviennent en sortie S (entre deux états).

EXERCICES

7.1. Soit le plan suivant. Fournir le diagramme d'état complet correspondant. S = sortie, A et B sont deux entrées externes, et H est l'horloge. Première bascule = bascule toggle, et seconde bascule = bascule D. Q_1 = *MSB*.

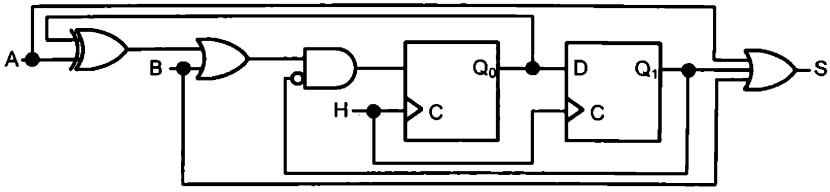


Figure 7.145

7.2. Soit le circuit suivant ($Q2$, $Q1$, $Q0$; $Q2$ = *MSB*) avec une entrée externe « entree » telle qu'indiquée :

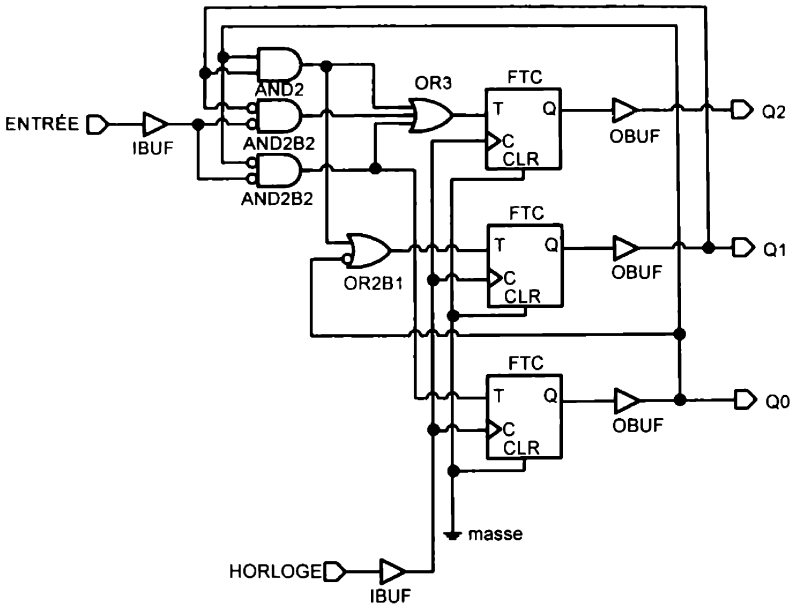


Figure 7.146

a) Donner les équations de stimulation des bascules.

b) Donner la table État Présent → État Suivant complète (tous les états).

c) Dresser le diagramme d'état complet de ce circuit.

7.3. Soit le diagramme d'état suivant : (A, B et C sont des signaux externes) :

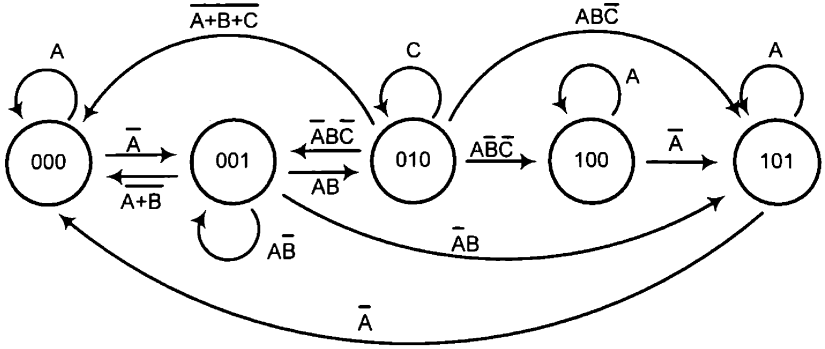


Figure 7.147

Établissez la table État présent → État suivant (sans section de stimulation) en employant la technique des variables conditionnelles. Simplifiez vos expressions. Nommez les 3 bits $Q2$ $Q1$ $Q0$ (avec $Q2$ = *MSB*).

7.4. Implanter le diagramme d'état suivant dans deux bascules $Q1$ (= *MSB*) et $Q0$ (= LSB). On ne demande pas d'autocorrection. La bascule $Q1$ sera une bascule J-K, et la bascule $Q0$ sera une bascule D. L'horloge est commune aux deux bascules. Les sorties S (S = vecteur de trois bits : $S2$, $S1$, $S0$; $S2$ = *MSB*) du circuit sont indiquées en gris. Les variables d'entrée sont A et B.

Employer la méthode systématique avec les variables conditionnelles.

Diagramme d'état :

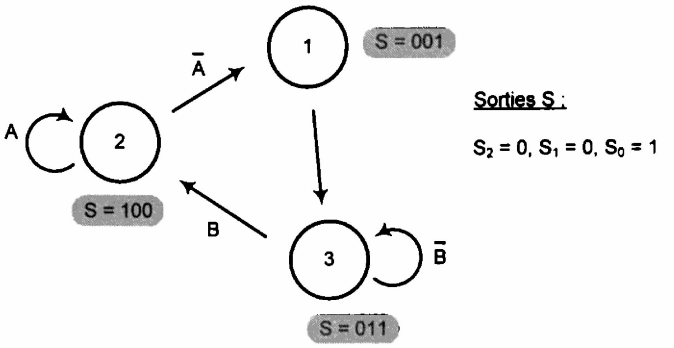


Figure 7.148

7.5.1 Faire le design du circuit qui implante le diagramme d'état suivant. Il faut employer un nombre minimal de flip-flops J-K. Les sorties du circuit sont directement les sorties des bascules.

a) Inclure la table ÉP-ÉS avec la méthode des variables conditionnelles.

b) Fournir les équations de stimulation des bascules.

c) Dessiner le schéma complet du circuit.

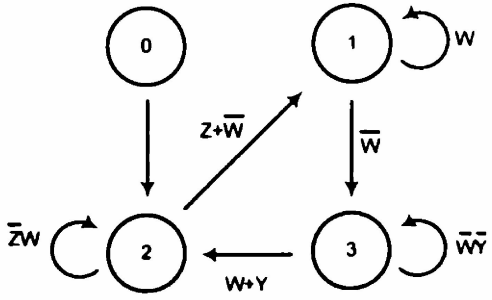


Figure 7.149

7.5.2 Répéter la question 7.5.1 avec des flip-flops D.

7.5.3 Répéter la question 7.5.1 avec des flip-flops T.

7.5.4 Que peut-on dire de l'emploi de la bascule J-K par rapport à la bascule D ou la bascule T (pour les questions précédentes 7.5.1 à 7.5.3)?

7.6. On vous demande d'implanter le diagramme suivant dans un circuit composé de deux bascules J-K *edge-triggered* (*A*, *B*, *C*1 et *C*2 sont les noms des états et aussi les sorties du circuit pour chacun des états définies comme suit : *A* = 1010, *B* = 1011, *C*1 = 1100 et *C*2 = 1101) :

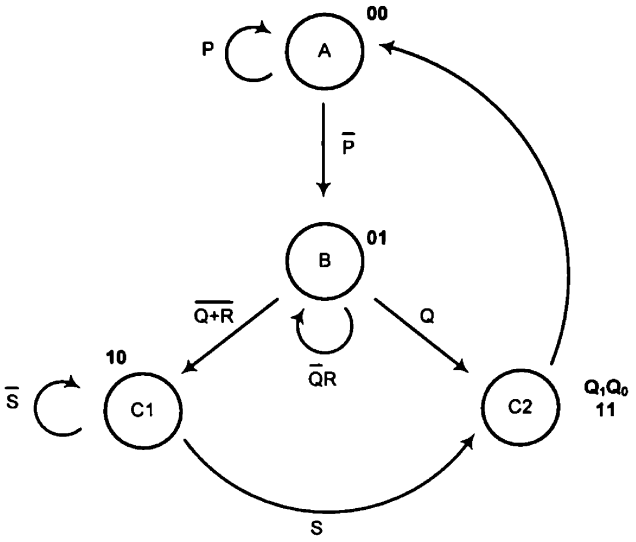


Figure 7.150

Employer le codage des états tel qu'indiqué (*Q*1, *Q*0 ; *Q*1 = *MSB*).

a) De quel type de machine s'agit-il ?

b) Établir la table État Présent → État Suivant en vous servant de la méthode des variables conditionnelles.

c) Donner les tables de Karnaugh pour la logique de stimulation des bascules J-K et les équations correspondantes.

d) Dessiner le schéma du circuit en incluant le décodeur de sortie.

Répéter la question avec des flip-flops D.

Répéter la question avec des flip-flops T.

7.7. Soit le plan suivant. On vous demande de fournir le diagramme d'état complet correspondant.

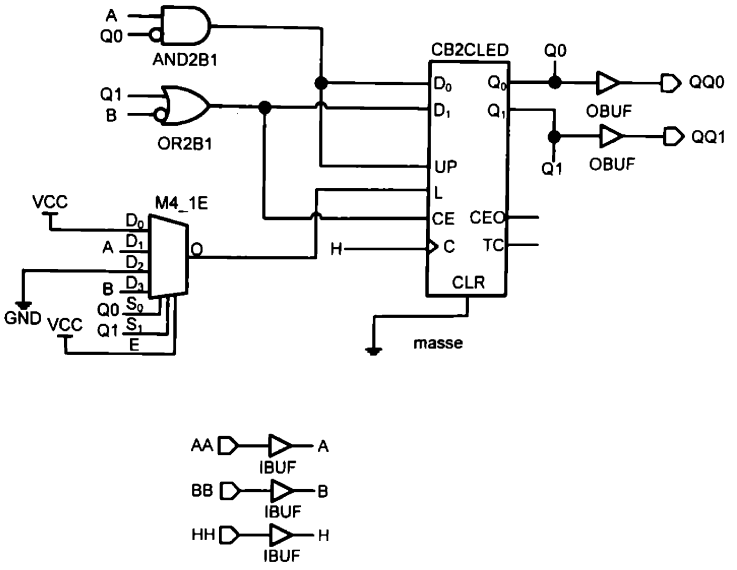


Figure 7.151

7.8. On vous demande de réaliser un module de commande pour une chaudière à mazout (chauffage à air soufflé) en employant un CB4CLED (voir la figure 7.152). Les entrées sont le thermostat de la maison (variable TM) et la température de consigne du brûleur (variable *TB*). Il y a deux sorties : *B* pour la mise en marche du brûleur (si mise à 1), et *V* pour la mise en marche du ventilateur de la soufflerie (si mise à 1). Le système ne devant pas être approuvé par l'ACNOR[21], il n'est pas nécessaire qu'il soit autocorrecteur !

La procédure de fonctionnement de la fournaise est la suivante : lorsque la température de la maison baisse, le thermostat *TM* active la chaudière. Cette activation se passe en deux temps. Le brûleur de mazout est immédiatement allumé puis, après préchauffage de la chaudière jusqu'à la température de consigne (signal *TB*), la soufflerie est mise en marche. Lorsque la température de la maison a atteint la température de consigne, le thermostat *TM* désactive la chaudière. L'arrêt de celle-ci se passe aussi en deux temps : le brûleur est immédiatement arrêté puis, un peu plus tard, selon le signal de *TB*, la soufflerie s'arrête à son tour.

Le fonctionnement des thermostats (entrées *TM* et *TB*) est le suivant :

- si la température est plus petite que la température de consigne, signal = 0 ;
- si la température est plus grande que la température de consigne, signal = 1.

a) Établir le diagramme d'état ;

b) Donner la table des actions, la table ÉP-ÉS ;

c) Dessiner le plan complet du circuit en simplifiant les multiplexeurs au maximum.

[21] Association Canadienne de NORmalisation

Y-a-t'il des contraintes particulières concernant la fréquence de l'horloge ?

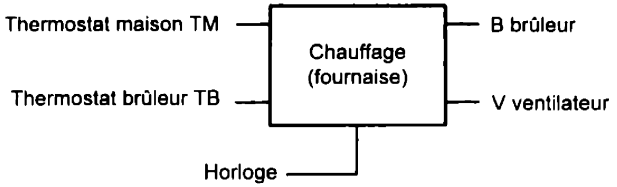


Figure 7.152

7.9. Réaliser un module de commande pour une minuterie à sonnette simple en employant un CB4CLED. Il y a trois entrées (boutons-poussoirs) : *U*, *S*, *A* (voir la figure 7.153). Les sorties sont la sonnette H (activée si H = 1) et aussi l'affichage (on assume qu'un décodeur à sept segments est à notre disposition, nous avons aussi accès à un signal d'horloge de 1 Hz). La minuterie permet de réaliser les séquences suivantes :

$$1 \rightarrow 0,\ 2 \rightarrow 1 \rightarrow 0,\ 3 \rightarrow 2 \rightarrow 1 \rightarrow 0.$$

(Pour simplifier le problème, ce sont les seules séquences possibles.)

La sonnette est activée à la fin de chaque séquence, pour une période d'horloge.

Le fonctionnement de la minuterie est le suivant :

1. On pousse sur *U* pour sélectionner la valeur de départ (0, 1, 2, 3 ; la valeur affichée s'incrémente de 1 chaque fois qu'on pousse sur *U* ; au-delà de 3, on retombe à 0) ;
2. On pousse sur *S* pour faire démarrer la minuterie.

Si l'on appuie sur *A*, il y a retour à 0.

À tout moment, l'affichage à sept segments présente le compte courant (soit 0, 1, 2, 3).

La minuterie fonctionne si l'on relâche les boutons-poussoirs, dès que les valeurs de ceux-ci ont été enregistrées par le coup d'horloge.

a) Établir le diagramme d'état ;

b) Donner la table des actions, des commandes (*L*, *U*, *CE*) et des données (les D_i) ;

c) Dessiner le plan complet du circuit sans simplifier les multiplexeurs.

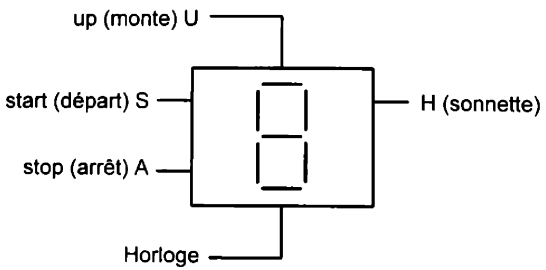


Figure 7.153 Minuterie simple

7.10. Soit le registre à décalage (RàD) directionnel suivant (sur trois bits ; Q_2 = *MSB*) :

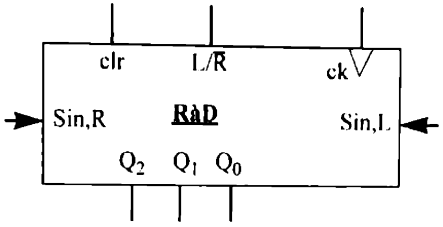


Figure 7.154

Le décalage se produit lors des transitions (coups) d'horloge (sur l'entrée *ck*).

Ce circuit possède les entrées suivantes :

- *clr* : Effacement synchrone (*clear*) de tous les bits ;
- $L / \overline{R}$: Mode décale à gauche (*Left*) si mis à 1, et Mode décale à droite (*Right*) si mis à 0 ;
- *Sin,R* : *Serial in, Righ* : Entrée série, droite ;
- *Sin,L* : *Serial in, Left* : Entrée série, gauche.

Les actions possibles sont donc (mnémoniques) :

- *CLR* – effacement synchrone (clear) de tous les bits (*CLR* prioritaire sur *SL* et *SR*) ;
- *SL* – *Shift Left* – décale à gauche ;
- *SR* – *Shift Right* – décale à droite.

On veut réaliser la séquence :

Q2	Q1	Q0	
0	0	0	
1	0	0	
1	1	0	
1	0	1	
0	1	1	
1	1	1	
0	0	0	C'est à dire que l'on recommence la séquence.

Figure 7.155

Implanter cette séquence avec ce RàD.

a) Donner la table des actions possibles en employant les mnémoniques *CLR*, *SL*, et *SR*.

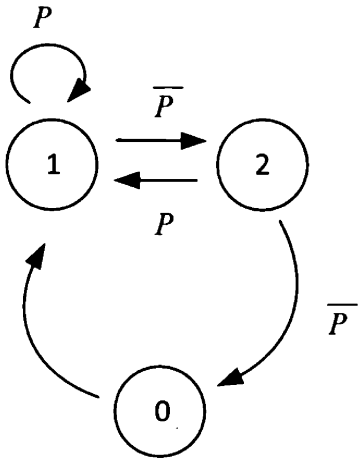
b) Donner la table pour la commande des entrées : *clr*, $L / \overline{R}$, *Sin,R* et *Sin,L*.

c) Donner le schéma du circuit complet en employant des multiplexeurs non simplifiés (donc des MUX 8 à 1).

7.11 Réaliser le circuit qui implante la séquence ①, ②, ⓪ illustrée à la figure suivante au moyen d'un compteur 74163 (sur 2 bits uniquement). Il y a une variable externe *P*. Les sorties du circuit sont directement les sorties du compteur. Optimisez l'auto-correction.

(exemple similaire à l'exemple 4 de la section 7.5 réalisés avec des bascules).

Note : un 74163 peut-être vu comme un CB4CLED mais uniquement avec CE+ (le 74163 fonctionne en incrementation seulement.



Annexe

Méthode rapide pour l'évaluation d'une expression booléenne

Souvent, nous sommes confrontés à l'évaluation d'une expression booléenne, par exemple lorsque sa table de vérité doit être dressée. Généralement, on évalue alors l'expression booléenne pour chaque combinaison des variables. Cela conduit bien sûr aux bonnes valeurs, mais cela peut être long. Par exemple, si l'expression contient quatre variables, il faudra étudier 16 ($=2^4$) combinaisons. La question est donc la suivante : existe-t-il une méthode d'évaluation rapide ? La réponse est oui.

Trois cas peuvent se présenter :

a) *L'expression à évaluer est de type SdP (Somme de Produits).* Dans ce cas, on considère l'expression booléenne comme la somme des mintermes présents. La table de vérité s'obtient immédiatement en entrant des « 1 » dans la table de vérité pour chaque minterme présent. Clarifions ceci à l'aide d'un exemple.

Exemple : Dresser la table de vérité de l'expression booléenne :

$$F(D,C,B,A;D=MSB) = \bar{D}CB + D\bar{C}B\bar{A} + CB\bar{A} + \bar{D}C\bar{B}A + A$$

F est d'abord réécrite en fonction de ses mintermes, ce qui donne :

$$F = (m7 + m6) + m10 + (m14 + m6) + m5 + (m1 + m3 + m5 + m7 + m9 + m11 + m13 + m15)$$

Les termes contenant moins que le nombre de variables de la fonction (ici, quatre variables) doivent être « complétés ». Par exemple, pour A, cela revient à écrire tous les mintermes qui couvrent la zone « A » dans la table de Karnaugh. Pour référence, voici d'ailleurs la table de Karnaugh correspondante à quatre variables :

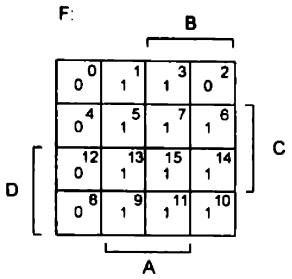


Figure A1

Finalement, on place des « 1 » dans la table de vérité pour tous les mintermes obtenus (et des « 0 » ailleurs). Cela donne :

Rang	*D*	*C*	*B*	*A*	*F*
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	0
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	1
10	1	0	1	0	1
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	1
14	1	1	1	0	1
15	1	1	1	1	1

Figure A2

Comme on est à même de le constater, l'évaluation de l'expression a été rapidement obtenue sans devoir tester les 16 combinaisons une à une.

b) *L'expression à évaluer est de type PdS (Produit de Sommes).* Dans ce cas, on considère l'expression booléenne comme le produit des Maxtermes présents. La table de vérité s'obtient immédiatement en entrant des « 0 » dans la table de vérité pour chaque Maxterme présent. Clarifions ceci à l'aide d'un exemple.

Exemple : Dresser la table de vérité de l'expression booléenne :

$$F\ (D,C,B,A\ ;\ D\ =\ MSB) = (\overline{D}+C+B)\ \cdot\ (\ D+\overline{C}+B+\overline{A})\ \cdot\ (C+\overline{A})\ \cdot\ \overline{B}$$

F est d'abord réécrite en fonction de ses Maxtermes, ce qui donne :

$$F = (M8 \cdot M9) \cdot M5 \cdot (M1 \cdot M3 \cdot M9 \cdot M11) \cdot (M2 \cdot M3 \cdot M6 \cdot M7 \cdot M10 \cdot M11 \cdot M14 \cdot M15)$$

Les termes contenant moins que le nombre de variables de la fonction (ici, quatre variables) doivent être « complétés ». Par exemple, pour $\overline{B}$, cela revient à écrire tous les Maxtermes qui couvrent la zone « $\overline{B}$ » dans la table de Karnaugh. Pour référence, voici d'ailleurs la table de Karnaugh correspondante à quatre variables :

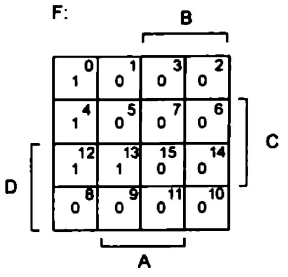


Figure A3

Finalement, on place des « 0 » dans la table de vérité pour tous les Maxtermes obtenus (et des « 1 » ailleurs). Cela donne :

rang	D	C	B	A	F
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	0
7	0	1	1	1	0
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	0
11	1	0	1	1	0
12	1	1	0	0	1
13	1	1	0	1	1
14	1	1	1	0	0
15	1	1	1	1	0

Figure A4

Comme on est à même de le constater, l'évaluation de l'expression a été rapidement obtenue sans devoir tester les 16 combinaisons une à une.

c) *Une combinaison de PdS et de SdP.* Dans ce cas, on peut ramener l'expression en SdP ou de PdS (en effectuant les opérations algébriques (+,·) et traiter le résultat selon les cas précédents (a, b). On peut aussi implanter séparément les parties SdP et PdS dans la table de Karnaugh pour obtenir le résultat global. L'exemple suivant illustre ce qui peut se produire.

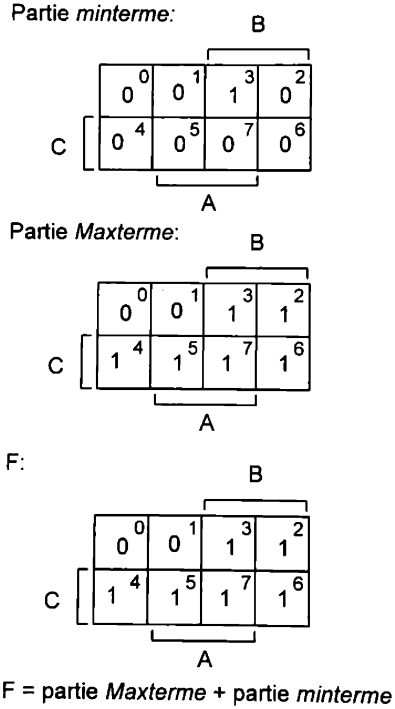
Exemple : Dresser la table de vérité de l'expression booléenne :

$$F(C, B, A\ ; C = MSB) = (C + B)\ +\ \bar{C}BA$$

F est d'abord réécrite en fonction de ses mintermes et de ses Maxtermes, ce qui donne :

$$F = (M0 \cdot M1) + m3$$

Dans l'évaluation de l'équation précédente, on évalue la partie Maxterme d'une part, la partie minterme d'autre part, et puis on fait la somme logique des deux parties (puisqu'un « + » les relie). Les termes contenant moins que le nombre de variables de la fonction (ici, trois variables) ont été « complétés » tel qu'il a été expliqué précédemment. La table de Karnaugh correspondante à trois variables est alors la suivante :



F = partie *Maxterme* + partie *minterme*

Figure A5

Finalement, la table de vérité est obtenue :

Rang	C	B	A	F
0	0	0	0	0
1	0	0	1	0
2	0	1	0	1
3	0	1	1	1
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

Figure A6

Notons que le même résultat est obtenu en considérant F comme ceci avec seulement des mintermes :

$$F = C + B + \overline{C}BA = (m4 + m5 + m6 + m7) + (m2 + m3 + m6 + m7) + m3$$

Lexique[1]

« 0 » *catching* :

Voir **Contamination aux « 0 »**.

« 1 » *catching* :

Voir **Contamination aux « 1 »**.

ABEL :

Acronyme signifiant : *Advanced Boolean Equation Language* (langage pour expression booléenne). ABEL est un HDL (c'est-à-dire un langage pour la description du matériel).

Aléa :

Malfonctionnement des circuits logiques (combinatoires et séquentiels) qui causent des erreurs transitoires qui surviennent lorsqu'une entrée change. On en distingue trois types différents : les aléas statiques, les aléas dynamiques et les aléas essentiels.

AC :

Abréviation anglaise pour *Alternative Current* ou courant alternatif en français. L'alimentation électrique du secteur en est l'exemple le plus commun. Par opposition au DC ou *Direct Current* en anglais (courant continu ou cc en français) fourni par des circuits spéciaux, des piles, des accumulateurs. En CA, la tension fluctue à une fréquence donnée (60 Hz pour le secteur en Amérique du Nord).

Aléas dynamiques :

Transitions multiples, mais momentanées du signal de sortie, transitions obtenues à la suite du changement du signal d'entrée. Sa forme la plus courante est le contact mécanique d'un interrupteur qui rebondit.

Aléas essentiels :

Ce sont des erreurs de fonctionnement du circuit logique, erreurs qui entraînent le circuit dans ses états non valides. Ce type d'aléa se retrouve dans les systèmes séquentiels synchrones.

---

[1] Une bonne source de renseignements est l'encyclopédie http://fr.wikipedia.org/wiki/Accueil disponible dans plusieurs langues.

Aléas statiques :

Transitions simples et momentanées du signal de sortie causées par les délais inégaux entre plusieurs voies de transmission à travers un circuit combinatoire.

Algèbre :

Outil mathématique permettant de résoudre des équations afin de déterminer des inconnues qui sont représentées par des symboles aussi appelés variables.

ALU :

Acronyme anglais signifiant *Arithmetic Logic Unit* ou unité arithmétique logique.

ASCII :

Sigle anglais signifiant *American Standard Code for Information Interchange.*

ASIC :

Acronyme anglais signifiant *Application Specific Integrated Circuit.* Il s'agit d'un circuit intégré réalisé pour une application particulière par opposition à un circuit intégré d'usage général tel un FPGA, un CPLD ou un microcontrôleur. Les premiers ASIC, développés dans les années quatre-vingt étaient constitués d'un réseau de portes dont les masques d'interconnexion métalliques pouvaient être modifiés lors des derniers stades de fabrication.

Autocorrecteur :

Capacité d'un compteur synchrone (avec ou sans entrées) de retrouver sa séquence principale en cas de saut indésirable vers un état non prévu. Le retour à la séquence principale peut prendre un ou plusieurs coups d'horloge. L'autocorrection est dite «simple» si un état de retour est choisi dans la séquence principale et que le design force le retour des états non prévus à cet état. L'autocorrection est dite «optimisée» si un effort est fait afin d'obtenir le retour dans la séquence principale en optimisant les regroupements possibles dans les tables de stimulation.

Bascule à déclenchement sur front :

Type de bascule logique réagissant aux fronts montants ou descendants de l'horloge (*flip-flop edge-triggered* en anglais). Dans ce livre, nous emploierons le terme « flip-flop » pour désigner une bascule à déclenchement sur front d'horloge.

Bascule :

Circuit numérique mémorisant un bit et pouvant *basculer* de l'état logique « 1 » à l'état logique « 0 » grâce à une (des) commande(s) appropriée(s). On distingue les bascules R-S, D, J-K et T. En anglais : *flip-flop.* Dans ce livre, nous emploierons le terme « flip-flop » pour désigner une **bascule à déclenchement sur front** d'horloge (voir ce mot). Le terme « bascule » sera employé pour désigner les circuits réagissant aux niveaux logiques du signal d'horloge (« horloge à niveaux »).

Bit :

Chiffre binaire à une seule position et pouvant donc prendre uniquement deux valeurs : le « 1 » logique ou le « 0 » logique.

Bit d'activation :

Bit de commande qui permet d'activer le fonctionnement normal d'un circuit. Si le bit d'activation est inactif, les sorties du circuit sont désactivées, peu importe la valeur des autres entrées. En anglais : *enable*.

Bit de retenue :

Voir **Retenue (bit de)**.

Bit de signe :

Voir **Signe (bit de)**.

Boîtier :

Appellation commune pour un circuit électronique, synonyme de puce.

Breadboard :

Acronyme anglo-saxon signifiant **plaquette de montage** (voir ce mot).

Bruit :

Variation non-désirée d'un signal quelconque, par exemple une perturbation électromagnétique qui en change la valeur.

Bus :

Lignes multiples où plusieurs entrées et sorties logiques sont connectées ensemble selon le principe du **ET câblé** (voir ce mot). Un bus contient généralement 8, 16, 32 ou 64 lignes individuelles. Chaque ligne individuelle correspond à un bit.

Canonique :

Se dit d'une solution logique non simplifiée. Par exemple, une SdP canonique ou un PdS canonique.

Carry out :

Mot anglais pour **retenue** (voir ce mot).

Chronogramme :

Représentation temporelle de la valeur (0,1) d'un ou de plusieurs signaux numériques (*timing diagram* en anglais).

Circuits combinatoires :

Circuits logiques dont les sorties dépendent seulement des états présents aux entrées ; sortie qui pourra changer dès que survient un changement d'entrée.

Circuits séquentiels :

Circuits logiques dont les sorties dépendent de l'historique des entrées, ce qui veut dire que, pour un état futur donné (après le coup d'horloge), la sortie du circuit dépendra des entrées présentes avant le coup d'horloge, mais aussi de l'état dans lequel le circuit se trouve à cet instant. Ces circuits possèdent une mémoire de leur état présent.

CLP :

Circuit logique programmable (voir **PLD**).

CMOS :

Sigle anglais signifiant *Complementary Metal-Oxide Semiconductor*. Semi-conducteur complémentaire à oxyde métallique. Puisque les circuits CMOS sont formés d'une partie positive (P-MOS) et négative (N-MOS) ne fonctionnant pas simultanément, ils sont caractérisés par une faible consommation électrique par rapport aux circuits à base de transistors bipolaires.

Code :

Représentation d'une valeur sous une autre forme.

Complément 1 :

On obtient la valeur du complément 1 d'un nombre binaire en inversant tous les bits de ce nombre (1 devient 0, et inversement).

Complément 2 :

On obtient la valeur du complément 2 d'un nombre binaire en ajoutant « 1 » à la valeur de son **complément 1** (voir ce mot).

Compteur (logique) :

Circuit séquentiel synchrone qui assure la fonction de comptage (en format binaire ou autre) avec 2^N états sans décodeur de sortie puisque les sorties du circuit sont simplement les sorties des N flip-flops le constituant. Il n'y a évidemment aucune entrée hormis l'horloge, et aussi parfois une entrée d'initialisation (c'est-à-dire de mise à 0). Le compteur peut s'incrémenter (comptage croissant) ou se décrémenter (comptage décroissant).

Compteur à codage positionnel :

Compteur dont la sortie s'incrémente (+1 à chaque coup d'horloge) ou se décrémente (−1 à chaque coup d'horloge). Autrement dit, la séquence n'est pas brisée. On aura ainsi la séquence répétée suivante : 4, 5, 6, 7, 8, 9, *A*, *B*. Pour ce type de compteur, les sorties des flips-flops constituent généralement la sortie du circuit.

Compteur à décade :

Compteur qui possède 10 états dans sa séquence qui est soit : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 (compteur à décade type incrémental), soit : 9, 8, 7, 6, 5, 4, 3, 2, 1, 0 (compteur à décade type décrémental). Pour ce type de compteur, les sorties des flip-flops constituent généralement la sortie du circuit.

Compteur à propagation d'horloge :

Compteur obtenu en cascadant plusieurs flip-flops, la sortie d'un flip-flop servant d'horloge pour le flip-flop suivant. En anglais, compteur *ripple*. Attention : les délais de propagation dans ces compteurs peuvent entraîner des aléas.

Compteur synchrone :

Circuit numérique qui passe à travers une série finie d'états successifs grâce à une horloge. Outre son horloge, un compteur synchrone ne possède aucune autre entrée (sauf éventuellement des entrées d'initialisation : mise à 0 ou à 1 de certains bits). Contrairement à un séquenceur synchrone, la séquence parcourue est régulière (non brisée), c'est-à-dire que la valeur binaire des états parcourus s'incrémente de +1 ou se décrémente de −1 à chaque coup d'horloge.

Condensateur :

Composant électronique qui emmagasine une charge électrique. Unités : le farad (symbole F). Nommé en l'honneur du célèbre scientifique britannique Michael Faraday (1791-1867).

Contamination aux « 0 » :

Dans une configuration R-S maître-esclave, le passage de la sortie à « 0 » suite à la présence d'un aléa sur l'entrée R (« 0 » *catching* en anglais).

Contamination aux « 1 » :

Dans une configuration R-S maître-esclave, le passage de la sortie à « 1 » suite à la présence d'un aléa sur l'entrée S (« 1 » *catching* en anglais).

Coup d'horloge :

Front montant ou transition bas-haut du signal d'horloge. Certains circuits logiques réagissent aussi aux coups d'horloge descendants, donc aux transitions haut-bas.

Course :

Synonyme d'aléa statique.

Coût :

Le coût d'un circuit est ici défini comme le nombre de portes doubles élémentaires (ET, OU, NON-ET, NON-OU) équivalentes qui sont nécessaires pour la réalisation du circuit en question. Les inverseurs ne sont jamais inclus dans le coût d'un circuit puisque considérés déjà disponibles. La porte OU-exclusif n'est pas considérée comme une porte simple, car elle se décompose en trois portes simples.

CPLD :

Complex *Programmable Logic Device* ou réseau logique programmable complexe ayant une structure similaire aux FPGA. Généralement, les CPLD conservent leur configuration logique même si l'alimentation électrique est coupée, grâce à leur mémoire embarquée non volatile. Voir **PLD**.

D :

Bascule ou flip-flop D (*data*).

Décodeur d'adresse :

Autre terme pour **démultiplexeur** (voir ce mot).

Décodeur de minterme :

Autre terme pour **démultiplexeur** (voir ce mot).

Décodeur :

Autre terme pour **démultiplexeur** (voir ce mot).

DEL :

Diode électroluminescente. Composante électronique à deux bornes qui émet de la lumière (exemple : blanche, bleue, verte, jaune, rouge, infrarouge) lorsqu'une tension continue lui est appliquée ; d'habitude 2.5 à 3 volts suffisent. Comme la consommation électrique est faible (typiquement < 20 mA), on peut souvent connecter une DEL directement à une sortie TTL. Abréviation anglaise : LED (*Light Emitting Diode*).

Démultiplexeur :

Circuit qui possède N entrées et 2^N sorties. Une seule sortie est activée à la fois ; elle correspond au poids binaire des entrées N présentes. Ce circuit « décode » donc le poids binaire des entrées.

DÉMUX :

Abréviation de démultiplexeur.

Dépassement :

Bit de statut indiquant une condition de dépassement ; pour une addition, il se calcule comme le OU-exclusif des bits de signe et de retenue (voir **Signe (bit de)** et **Retenue (bit de)**). En anglais : *overflow* ou *OVR*.

Diagramme d'état :

Représentation de l'historique du fonctionnement d'un circuit séquentiel. Cette représentation montre la suite des états possibles de même que la relation entre ceux-ci.

DIP :

Abréviation anglaise pour *Dual Inline Package*. Il s'agit d'un type de boîtier où les broches de contacts forment deux lignes. Parmi les nombres typiques de broches, on retrouve les valeurs suivantes : 4, 8, 14, 16, 20, 28 et 40.

Distribution maximale :

Distribution maximale du signal d'horloge. Dans un circuit synchrone, il s'agit de la capacité du signal d'horloge d'alimenter un nombre donné de flip-flops. Au-delà de ce nombre, le signal d'horloge sera dégradé. En anglais : le *fan-out*. Le *fan-out* est donc le courant maximal disponible.

Download :

Mot anglais signifiant transférer un contenu (des données) d'une source (composant électronique, site web) à l'ordinateur de travail. Opposé de *upload*.

DRAM :

Mémoire **RAM** (voir ce mot) dynamique (*Dynamic Random Access Memory*). Le bit à retenir est mémorisé sous la forme d'une charge électrique dans un condensateur minuscule. Comme cette charge s'efface avec le temps, on doit périodiquement lire toute la mémoire et réécrire son contenu, d'où l'acronyme de RAM dynamique.

EEPROM :

Electrically Erasable Programmable Read-Only Memory, mémoire effacable électriquement et programmable. Contenu non volatil en cas de coupure d'alimentation électrique. Voir **ROM**.

Enable :

Mot anglais signifiant bit d'activation. Voir **Bit d'activation**.

EPROM :

Erasable *Programmable Read Only Memory*, mémoire effacable et programmable. Contenu non volatil en cas de coupure d'alimentation électrique.Voir **ROM**.

État de retour :

État où l'on redirige les états inemployés dans un circuit synchrone (avec ou sans entrées), ce qui nous donne alors un circuit autocorrecteur puisque cet état de retour conduit ultimement à la séquence de travail du circuit. Le retour à la séquence principale peut prendre un ou plusieurs coups d'horloge (au minimum un), de l'état inemployé à l'état de retour. En anglais : *Ground state*.

État :

L'état d'une bascule (ou d'un flip-flop) correspond à une de ses conditions stables : Q = 1 (état à 1) ou Q = 0 (état à 0). Cette notion peut être étendue à un circuit séquentiel quelconque. L'état d'un tel circuit correspond alors aux valeurs mémorisées par les bascules (ou les flip-flops) de ce circuit à un moment donné. Si ce circuit contient trois bascules (ou flip-flops), il s'agira des valeurs mémorisées par ces bascules (ou flip-flops) au moment où l'on observe le circuit, par exemple : « 1, 0, 1 », etc.

ET câblé :

Point où plusieurs sorties (on dit aussi plusieurs lignes de sortie) sont reliées ensemble. On évite les courts-circuits (c'est-à-dire une sortie à 1 et une autre simultanément à 0) en maintenant toutes les lignes connectées ensemble en mode de haute **impédance** (voir ce mot). Seule une ligne passe en mode de basse impédance à la fois.

Étiquette :

Lors du tracé des plans de circuits logiques ou électroniques, il s'agit du nom que l'on donne à un fil. Cela permet de réaliser des connexions multiples en évitant les plans « spaghettis »: tous les fils ayant une étiquette identique sont reliés ensemble sans que cela n'apparaisse directement sur le plan. *Label* en anglais.

Fan-in :

Expression anglaise qui spécifie le courant requis à l'entrée d'un circuit pour que le signal qu'on y injecte soit valide.

Fan-out :

Voir **Distribution maximale**.

FIFO :

Acronyme anglo-saxon qui signifie *First In, First Out*. Dans un tel registre à décalage, la première donnée introduite est aussi la première à en sortir.

Flip-flop (edge-triggered) :

Mot anglais signifiant **bascule à déclenchement sur front** (voir ce mot).

Flip-flop :

Mot anglais signifiant **bascule** (voir ce mot).

Folding :

Voir **Retournement**.

FPGA :

Field Programmable Gate-Array ou réseau de portes programmable par l'usagère. Généralement, les FPGA ont la particularité d'avoir leur configuration logique sauvegardée à l'extérieur ; celle-ci est chargée dans le circuit à l'allumage, ce qui permet les mises à jour faciles. Un **ASIC** (voir ce mot) peut être codé directement dans le silicium avec la configuration d'un FPGA, ce qui permet une consommation moindre et une vitesse accrue. Les FPGA sont reconnus avantageux en matière de coûts de développement et de temps de mise en marché pour les lots de production de moins de 10 000 unités. Voir **PLD**.

Glitch :

Mot anglais pour **aléa** (voir ce mot).

Ground state :

Voir **État de retour**.

Harvard (architecture de) :

Architecture d'ordinateur caractérisée par une mémoire séparée pour les données et les programmes. La plupart des microcontrôleurs utilisent cette architecture. Voir **Von Neumann (architecture de)**.

Haute impédance :

Voir ***Ht-Z*** et **impédance**.

HDL :

Hardware Description Langage, langage de description de systèmes électroniques numériques, langage de description matérielle.

Ht-Z :

Haute (*High*) impédance. Z est le symbole de l'impédance électrique. En état de haute impédance, une ligne « flotte » ; elle n'est alors ni à 1, ni à 0. Par contre, si elle est connectée à d'autres fils, elle pourra suivre la valeur logique de ceux-ci. C'est ce qu'on appelle un **ET câblé** (voir ce mot). Lorsque la ligne repasse en mode de « basse impédance », elle pourra prendre les valeurs de « 0 » ou de « 1 » logiques.

Hold time :

Voir **Temps de maintien**.

Horloge à niveaux :

Signal d'horloge actif lorsque étant au niveau logique « 1 » et inactif lorsque étant au niveau logique « 0 ». Voir **Horloge à transitions**.

Horloge à transitions :

Signal d'horloge actif lors d'une transition du niveau logique « 0 » au niveau logique « 1 » (transition 0 → 1), ou inversement (transition 1 → 0). Voir **Horloge à niveaux**.

Horloge :

Signal périodique qui permet de synchroniser les différentes parties d'un circuit, dès lors appelé circuit synchrone à condition que le signal d'horloge affecte directement et simultanément toutes les parties du circuit. Dans sa forme la plus simple, un signal d'horloge est simplement une onde carrée passant successivement de la valeur logique « 0 » à la valeur logique « 1 » à une certaine cadence appelée fréquence de l'horloge.

IEEE :

Institute of Electrical and Electronics Engineers, Institut des ingénieurs en électricité et en électronique. L'IEEE est la plus grande société professionnelle du monde ; elle regroupe plus de 365 000 ingénieurs dans plus de 150 pays. Pour l'IEEE, le monde est divisé en Régions. Les Régions 1 à 6 sont aux États-Unis ; le Canada constitue la Région 7 ; l'Europe, la Région 8 ; l'Amérique latine, la Région 9 ; et la Région 10 regroupe le reste du monde (Asie-Pacifique). Chaque région est divisée en Sections. Le Québec comprend les Sections IEEE de Montréal, de Québec et de Saint-Maurice (Trois-Rivières). L'IEEE est réputé pour ses nombreux standards en génie électrique et en génie informatique de même que la littérature technique et scientifique qu'il publie.

Impédance :

Le symbole de l'impédance est Z. L'impédance est le rapport de la tension sur le courant. Dans le cas d'un circuit purement résistif, l'impédance est équivalente à la résistance. Pour les circuits avec condensateurs ou inductances, l'impédance est un nombre complexe, dont la partie réelle constitue la partie résistive (symbole R) ; et la pertie imaginaire, la réactance (symbole X) : $Z = R + jX$ (où j est le nombre imaginaire $j = \sqrt{-1}$).

Impliquant principal :

Un minterme qui ne peut être simplifié davantage sauf en altérant la table de vérité correspondante.

Impliquant :

N'importe quel produit d'une expression SdP, donc n'importe quel minterme.

K-Map :

Abréviation anglaise pour table de Karnaugh.

Label :

Mot anglais pour étiquette.

Latch D :

Latch à entrée D unique (*D* pour données, ou *data* en anglais).

Latch :

Type de bascule logique réagissant aux niveaux logiques du signal d'horloge.

LED :

Light Emitting Diode, abréviation anglaise de DEL.

LIFO :

Acronyme anglo-saxon qui signifie *Last In, First Out*. Dans un tel registre, la dernière donnée introduite est aussi la première à en sortir.

Logique câblée :

Approche de design logique où la configuration est obtenue par le câblage même du circuit réalisé. Un circuit donné ne peut être reconfiguré pour exécuter une autre tâche que s'il est modifié. Le circuit contient un certain nombre d'entrées et de sorties. L'exécution est généralement plus rapide par rapport à la logique programmée.Voir **Logique programmée**.

Logique programmée :

Approche de design logique où une base matérielle universelle contenant un certain nombre d'entrées et de sorties est programmée ou microprogrammée selon les besoins. Un circuit donné peut être reconfiguré pour exécuter une autre tâche par simple reprogrammation sans en modifier la structure. L'exécution est généralement plus lente par rapport à la logique câblée. Voir **Logique câblée**.

LSB :

Sigle anglais pour *Least Significant Bit*, ou bit le moins important, en français. Pour un nombre binaire, il s'agit du bit de l'extrême droite, car son poids binaire est le moins élevé (2^0).

LSI :

Sigle anglais signifiant *Large-Scale Integration* (intégration à grande échelle). Fait référence à la fabrication des circuits intégrés. Les circuits LSI implantent jusqu'à environ 200 à 1000 portes logiques (ils contiennent quelques milliers de transistors). Voir **SSI**, **MSI**, **VLSI**.

Machine à nombre fini d'états :

Finite state machine en anglais. Voir **Machine de Mealy**.

Machine de Mealy :

Circuit numérique synchrone qui passe à travers une série finie d'états grâce à une horloge. Outre son horloge, une telle machine possède des entrées externes qui peuvent ou non être synchrones et qui lui permettent d'interagir avec le monde extérieur. En plus, elle peut éventuellement posséder des entrées d'initialisation (mise à 0 ou à 1 de certains bits). Ses sorties après le coup d'horloge, de même que la séquence parcourue (c'est-à-dire l'ordonnancement des états parcourus) dépendent évidemment de l'état présent et de la valeur des entrées présentes. La mémorisation de l'état présent s'effectue grâce aux flip-flops inclus dans le circuit. Les machines de Mealy sont aussi connues sous le nom de Machines à nombre fini d'états *(finite state machine* en anglais*)* ou encore de séquenceur synchrone avec entrées. D'après George H. Mealy, professeur à l'Université de Harvard, Boston, Massachusetts. Voir **Machine de Moore**.

Machine de Moore :

Séquenceur synchrone ou **compteur synchrone** (voir ces mots). D'après Edward F. Moore (1925-2003), chercheur au Laboratoires Bell de Chatham (New Jersey) à la fin des années cinquante et professeur à l'Université du Wisconsin à Madison (1966-1985). Voir aussi **Machine de Mealy**.

Maître-esclave :

Configuration comprenant deux bascules connectées en cascade avec un signal d'horloge commun, mais inversé d'une bascule à l'autre (*master-slave* en anglais).

Master-slave :

Voir **Maître-esclave**.

Maxterme :

C'est une somme qui donne « 0 » dans la table de vérité, pour les valeurs spécifiées des variables présentes en entrée. Ces variables apparaissent non complémentées dans le Maxterme si elles sont à « 0 » dans la table de vérité. Ces variables apparaissent complémentées dans le Maxterme si elles sont à « 1 » dans la table de vérité.

Mémoire morte :

Read *Only Memory*, voir **ROM**. Une mémoire ROM (*Read Only Memory*) est un circuit qui mémorise une donnée (sur un ou plusieurs bits) pour chaque adresse. Le contenu d'une mémoire ROM est inscrit en usine et est inaltérable par la suite (contenu non volatil). La variante PROM (*Programmable Read Only Memory*) est programmable par l'usagère et les variantes RAM (*Random Access Memory*), EPROM (*Erasable PROM*), EEPROM (*Electrically Erasable PROM*) sont effaçables et reprogrammables par l'usagère.

Microcontrôleur :

Composante électronique complexe qui implante toutes les fonctions d'un ordinateur (dans un seul circuit intégré) et qui sert surtout pour des applications de commande.

Microprocesseur :

Composante électronique complexe qui implante les fonctions d'un ordinateur. Nécessite généralement des circuits périphériques externes de support.

Minimum recovery time :

Voir **Temps minimum de rétablissement**.

Minterme :

C'est un produit qui donne « 1 » dans la table de vérité, pour les valeurs spécifiées des variables présentes en entrée. Ces variables apparaissent non complémentées dans le minterme si elles sont à « 1 » dans la table de vérité. Ces variables apparaissent complémentées dans le minterme si elles sont à « 0 » dans la table de vérité.

MOSFET :

Acronyme anglais pour *Metal-Oxide Semiconductor-Field Effect Transistor* ou transistor à effet de champ employant un semi-conducteur à oxyde métallique. Les transistors MOSFET sont les « blocs LEGO » de base des circuits intégrés (numériques) **CMOS** (voir ce mot). Un transistor MOSFET est un interrupteur à trois connexions, la source, le drain et la porte (*gate* en anglais). Dans un tel transistor, une tension appliquée à la porte commande le courant circulant entre la source et le drain.

MSB :

Sigle anglais pour *Most Significant Bit*, ou bit le plus important, en français. Pour un nombre binaire, il s'agit du bit de l'extrême gauche, car son poids binaire est le plus élevé (2^n) pour un nombre de n bits.

MSI

Sigle anglais signifiant *Medium-Scale Integration* (intégration à échelle moyenne). Fait référence à la fabrication des circuits intégrés. Les circuits MSI implantent jusqu'à environ 20 à 100 portes logiques (ils contiennent des centaines de transistors). Voir **SSI**, **LSI**, **VLSI**.

Multiplexeur :

Circuits combinatoires aiguilleurs à 2^N entrées qui permettent de sélectionner (avec N bits de sélection) l'origine de la donnée qui sera transmise à la sortie. Les multiplexeurs peuvent agir sur un bit ou sur plusieurs bits à la fois selon le modèle de multiplexeur employé. Communément appellés MUX.

Nombre signé :

Nombre positif ou négatif selon la valeur de son bit de signe (voir **Signe (bit de)**). Un nombre négatif est obtenu par calcul de son **complément 2** (voir ce mot).

Ohm (loi d') :

Loi qui lie la tension V (en volts) aux bornes d'un composant électrique au courant I (en ampères) traversant celui-ci et à sa résistance interne R (en ohms, Ω) selon : V = RI. Cette loi a été énoncée en 1827 par le physicien allemand Georg Simon Ohm (1789-1854).

OTP :

One Time *Programmable*. Circuit programmable une seule fois. Une tension élevée appliquée au circuit permet de le configurer définitivement en « brûlant » sélectivement des fusibles internes.

Overflow :

Mot anglais pour **dépassement** (voir ce mot).

PAL :

Programmable Array Logic ou réseau logique programmable. La structure d'un PAL est une somme de produits avec réseau ET programmable et réseau OU fixe. Voir **PLD**. PAL™ est une marque de commerce de la compagnie *Monolithic Memories* (achetée par *AMD* en 1987).

Partition :

Fractionnement d'une solution logique dans plusieurs PAL. Voir **Retournement**.

PdS :

Abréviation pour Produits de Sommes.

Pile :

Mécanisme qui sauvegarde l'adresse de retour (des sous-routines). Il s'agit en fait d'un registre de type **LIFO** (voir ce mot) associé à un pointeur de pile. Deux instructions servent normalement pour l'interaction avec la pile : *PUSH* (pousser) une adresse sur la pile et *PULL* (récupérer) une adresse sur la pile.

PIPO :

Registre de type *Parrallel In Parallel Out*, donc entrée parallèle et sortie parallèle.

PISO :

Registre de type *Parrallel In Serial Out*, donc entrée parallèle et sortie série.

Plaquette de montage :

Réceptacle plat à plusieurs plots pour le prototypage rapide des circuits par simple insertion des composantes et des fils de connexion. *Breadboard* en anglais.

PLD :

Programmable *Logic Device*, ou circuit logique programmable, en Français. Terme générique identifiant une famille de circuits configurables par l'usagère. Selon le type, ils peuvent notamment incorporer des configurations de type SdP (Somme de Produits).

Porte double élémentaire (PDÉ) :

Porte à deux entrées qui ne peut être simplifiée davantage. Il s'agit des portes doubles ET, ET-NON, OU et NON-OU. La porte OU-exclusif étant décomposable, elle n'est donc pas considérée comme une PDE.

Prime implicant :

Terme anglais pour impliquant principal.

***Product of Sum* (POS)** :

Abréviation anglaise de PdS.

Produit de Sommes (PdS) :

Fonction booléenne représentant une table de vérité, et obtenue en faisant le produit booléen des Maxtermes présents dans cette table. En anglais : *Product of Sum* (*POS*)

PROM :

Programmable Read Only Memory, mémoire programmable en lecture seulement. Contenu non volatil. Voir **ROM**.

Puce :

Appellation commune pour un circuit électronique, synonyme de boîtier.

***Pull-down* (résistance)** :

Résistance importante, généralement de quelques milliers d'ohms (typiquement 10 k Ω) qui sert à forcer la tension d'un point de contact. Dans le cas d'une résistance *pull-down*, la résistance relie ce point de contact à la masse, ce qui amène ce point à la valeur logique 0. Puisque la résistance est grande (et donc le courant faible), il est facile de faire basculer la valeur du point de contact au 1 logique en lui appliquant la tension d'alimentation (et ce, sans risque de court-circuit). Très utile pour éviter de laisser une entrée (ou une sortie) flottante.

***Pull-up* (résistance)** :

Résistance importante, généralement de quelques milliers d'ohms (typiquement 10 k Ω) qui sert à forcer la tension d'un point de contact. Dans le cas d'une résistance *pull-up*, la résistance relie ce point de contact à la tension d'alimentation, ce qui amène le point à la valeur logique 1. Puisque la résistance est grande (et donc le courant faible), il est facile de faire basculer la valeur du point de contact au 0 logique en lui appliquant une tension faible (même la masse, et ce, sans risque de court-circuit). Très utile pour éviter de laisser une entrée (ou une sortie) flottante.

Race :

Mot anglais signifiant **course** (voir ce mot).

RAM :

Random Access Memory, mémoire à contenu accessible de façon aléatoire et dont le contenu peut être lu et réécrit. Contenu volatil en cas de coupure d'alimentation électrique. Voir **ROM**.

Rang :

Nombre qui correspond à la valeur binaire des entrées correspondantes dans une table de vérité.

RDC :

Voir **Registre à décalage cyclique**.

Registre à décalage cyclique :

Registre à décalage qui implante une séquence qui se « déroule comme un patron décalé » (en anglais : *that unfolds with a horizontal shift pattern*). Pour ces registres, l'entrée série (qu'on appelle $D_{entrée}$) du registre à décalage est obtenue comme une combinaison des sorties des flip-flops (Q_0, Q_1, Q_2, etc.). Le terme « cyclique » fait bien sûr référence au fait que la séquence générée par la logique combinatoire se répète.

Registre à décalage :

Circuit contenant n flip-flops D à déclenchement sur front (*edge-triggered*) connectés un à la suite de l'autre et partageant une horloge commune (il s'agit donc d'un circuit synchrone). À chaque coup d'horloge, les données progressent d'une position parmi les flip-flops D présents (*shift register* en anglais).

Résistance :

Composant électronique servant à limiter le courant dans un circuit électronique suivant la loi d'Ohm.

Retenue (bit de) :

Bit de statut indiquant la valeur de la retenue suite à un calcul. Si le calcul se fait sur n bits, il s'agit donc de la valeur du bit $n+1$. En anglais : *carry out* ou C_{out}.

Retournement :

Quand une(des) sortie(s) d'un PAL sert(servent) d'entrée(s) à celui-ci ou à d'autres PAL pour lesquels la logique a été partitionnée. *Folding* en anglais. Voir **Partition**.

Ripple counter :

Voir **Compteur à propagation d'horloge**.

ROM :

Read Only Memory, ou mémoire morte, en français. Contenu non volatil en cas de coupure d'alimentation électrique. Voir **Mémoire morte**.

Routage (des signaux) :

Dans un circuit électronique, cela consiste à déterminer les parcours pour la circulation interne des signaux (numériques, analogiques et de l'alimentation électrique).

R-S :

Bascule ou flip-flop à deux entrées S (*set*, mise à 1) et R (*reset*, mise à 0).

SdP :

Abréviation pour *Somme de Produits*.

Séquenceur synchrone avec entrées :

Voir **Machine de Mealy**.

Séquenceur synchrone :

Circuit numérique qui passe à travers une série finie d'états grâce à une horloge. Outre son horloge, un séquenceur synchrone ne possède aucune autre entrée (sauf éventuellement des entrées d'initialisation : mise à 0 ou à 1 de certains bits). Contrairement à un compteur synchrone, la séquence parcourue est « brisée », c'est-à-dire que la valeur binaire des états parcourus ne suit pas une progression strictement incrémentale (+1) ou décrémentale (−1).

Set up time :

Voir **Temps de mise en place**.

Shift register :

Voir **Registre à décalage**.

Signal asynchrone :

Dans un circuit asynchrone, qui n'est pas régit par une horloge, il s'agit d'un signal qui a un effet immédiat, peu importe la valeur du signal d'horloge.

Signe (bit de) :

Bit de statut indiquant le signe d'un nombre en représentation binaire signée. Si le nombre est exprimé sur n bits, il s'agit donc de la valeur du bit n. Si $n = 1$, le nombre est négatif ; et si $n = 0$, le nombre est positif. En anglais : *sign bit* ou *SIG*.

SIPO :

Registre à décalage de type *Serial In, Parallel Out* (entrée série, sortie parallèle).

SISO :

Registre à décalage de type *Serial In, Serial Out*, (entrée série et sortie série).

Slew rate :

Voir **Vitesse de l'horloge**.

Somme de produits (SdP) :

Fonction booléenne représentant une table de vérité, et obtenue en faisant la somme booléenne des mintermes présents dans cette table. En anglais : *Sum of Product* (SOP).

Sum of Product (SOP) :

Terme anglais pour Somme de Produits (SdP).

Sous-routine :

Fragment de code informatique appelable à la demande et exécutant une tâche particulière. Une particularité des sous-routines est qu'une fois leur exécution terminée, le cours normal du programme reprend où il avait été laissé. Ceci est possible grâce à un mécanisme de sauvegarde de l'adresse de retour effectué par la pile (*stack* en anglais).

SSI :

Sigle anglais signifiant *Small-Scale Integration* (intégration à petite échelle). Fait référence à la fabrication des circuits intégrés. Les circuits SSI implantent jusqu'à environ 2 à 10 portes logiques (ils contiennent des dizaines de transistors). Voir **MSI**, **LSI**, **VLSI**.

Stack :

Mot anglais pour **pile** (voir ce mot).

State diagram :

Mot anglais signifiant diagramme d'état.

T :

Bascule ou flip-flop toggle.

Table de vérité :

Représentation d'une fonction logique sous forme d'un tableau.

TdV :

Abréviation pour Table de vérité.

Temps de maintien :

Dans le cas des flip-flops à déclenchement sur front, pour qu'elles soient reconnues valides, il faut que les données demeurent inchanchées pendant un bref instant après le coup d'horloge ; c'est ce qu'on appelle le temps de maintien ou t_h (*hold* en anglais).

Temps de mise en place :

Dans le cas des flip-flops à déclenchement sur front, pour qu'elles soient reconnues valides, il faut que les données soient stabilisées un peu avant le coup d'horloge, c'est ce qu'on appelle le temps de mise en place ou t_{su} (*set up time* en anglais).

Temps minimum de rétablissement :

Dans le cas des flip-flops à déclenchement sur front, ce paramètre spécifie le temps qui doit s'écouler après un changement d'entrée asynchrone pour que l'horloge change à nouveau. En anglais : *minimum recovery time* (t_{rec}).

Timing diagram :

Terme anglais signifiant **chronogramme** (voir ce mot).

Transistor :

Composante électronique active à trois broches servant en commutation et en amplification des signaux électriques. Le mot transistor vient de *TRANSfer resISTOR* ou résistance de transfert. Le premier transistor a été inventé aux Laboratoires Bell le 16 décembre 1947. Le transistor a notamment permis la réalisation de circuits intégrés, de microprocesseurs et de toute la révolution numérique qui s'en est suivie.

TTL :

Abréviation pour *Transistor-Transistor Logic*. Désigne toute une famille de circuits numériques, dont la famille 74xxx qui contient les portes et fonctions logiques élémentaires.

Tube à vide :

Composant électronique formé d'une enveloppe de verre de forme cylindrique maintenue sous vide dans laquelle des électrons sont émis et déviés par des plaques métalliques. Ces plaques internes permettent de réaliser diverses fonctions : deux plaques pour une diode (dynode), trois plaques pour un effet d'amplification ou de commutation (triode). Les électrons sont émis grâce à une électrode de chauffage (alimentée en 6.3 V AC) causant la lueur rougeâtre caractéristique des anciens montages électronique construits avec de tels tubes. Quant à elles, les tensions sur les plaques sont élevées et dangereuses (milliers de volts). D'ailleurs à « l'époque », on recommandait toujours de travailler sur ces montages avec une main dans le dos pour éviter tout accident (mortel). On note de nos jours un regain d'intérêt pour ces tubes qui donneraient, dans les amplificateurs audio, selon les puristes « un son chaleureux et enveloppant » non retrouvé dans les amplis à transistors. Les horloges à tubes Nixie (qui sont des tubes spéciaux utilisés pour l'affichage et contenant un mélange de gaz – néon, mercure, argon – à très faible pression) sont des objets très prisés au XXI^{e} siècle.

Unité cachée :

Flip-flop que l'on ajoute à un circuit et qui permet d'implanter un état supplémentaire. Comme ce flip-flop n'apparaît pas à la sortie du circuit, de l'extérieur, il semble que le circuit soit arrêté pendant une période supplémentaire d'horloge. Les unités cachées sont utiles pour implanter des séquences qui incluent des répétitions d'états.

Upload :

Mot anglais signifiant charger un contenu (des données) dans une destination (composant électronique, site web) à partir de l'ordinateur de travail. Opposé de *download*

Variable conditionnelle :

Entrée externe d'un circuit. On l'appelle variable, car on peut l'incorporer directement dans une table État Présent → État Suivant (ÉP-ÉS) dans les sections *stimulation* et dans État *Suivant* (mais pas dans *État Présent*), ce qui rend les tables ÉP- ÉS plus compactes. Elle est conditionnelle, car sa valeur peut prendre deux valeurs (0 ou 1) et ainsi changer le parcours dans le diagramme d'état (selon sa « condition »).

Verilog :

Type de HDL (c'est-à-dire un langage pour la description du matériel).

VHDL :

VHSIC Hardware Description *Language*, langage de description de circuits VHSIC. Le VHDL fut lancé suite à une initiative du gouvernement américain dans les années quatre-vingt. Le VHDL a depuis été standardisé par l'**IEEE** (voir ce mot).

VHSIC :

Very High-Speed Integrated *Circuit*, Circuit intégré à très grande vitesse.

Vitesse de l'horloge :

Dans le cas des flip-flops à déclenchement sur front, le coup d'horloge doit se produire « assez rapidement », c'est ce qu'on appelle la vitesse de l'horloge (ou le taux de changement de la tension du signal d'horloge) qui est exprimée en volt par microseconde ([V/ms]). En anglais : le *slew rate*.

VLSI :

Sigle anglo-saxon signifiant *Very Large Scale Integration* (intégration à très grande échelle). Fait référence à la fabrication des circuits intégrés. Les circuits VLSI contiennent plusieurs milliers de transistors, et donc plus de 1000 portes logiques. Voir **SSI**, **MSI**, **LSI**.

Voisinage à quatre voisins :

Cases voisines selon un arrangement en croix (haut-bas ou gauche-droite), mais pas en oblique.

Von Neumann (architecture de) :

Architecture d'ordinateur caractérisée par une mémoire commune partagée par les données et les programmes. La plupart des microprocesseurs utilisent cette architecture. D'après John Von Neumann (1903-1957), mathématicien américain d'origine hongroise. Voir **Harvard (architecture de)**.

Wire-wrap :

Technique de prototypage rapide des circuits par bobinage des fils de connexion sur les broches des composantes.

Bibliographie

Avouris, P. « Carbon Nanotube Applications in Microelectronics », *IEEE Trans. Comp. Pack. Tech.*, Vol. 27, n°4, décembre 2004, p. 629-634.

Avouris, P. « Supertubes », *IEEE Spectrum*, août 2004, p. 41-45.

Balabanian, Norman ; Carlson, Bradley. *Digital Logic Design Principles*, John Wiley & Sons Inc., 2001, 373 pages.

Brown, Stephen ; Rose, Jonathan. « Architecture of FPGAs and CPLDs : A Tutorial », *IEEE Design and Test of Computers*, Vol. 13, n° 2, 1996, p. 42-57.

Brown, Stephen; Vranesic, Zvonko. *Fundamentals of Digital Logic with Verilog Design*, Mc Graw Hill, 2013 (3e édition), 864 pages.

Courtland, R., «Carbon Nanotubes, Stuffed and Stacked », IEEE Spectrum, février 2015, p. 13-16.

Daniels, Jerry D. *Digital Design from Zero to One*, John Wiley & Sons Inc., 1996, 616 pages.

Dueck, Robert K. *Digital Design with CPLD Applications and VHDL*, Delmar Thomson Learning, 2011 (2e édition), 896 pages.

Floyd, Thomas L. *Digital Fundamentals, Pearson Education*, Inc. (Prentice Hall), 2014 (11e édition), 912 pages.

Gaddis, Tony. *Starting Out with Programming Logic and Design*, Pearson Education, Inc. (Prentice Hall), 2015 (4e édition), 656 pages.

Harris, David; Harris, Sarah. *Digital Design and Computer Architecture*, MK (Elsevier), 2012 (2e édition), 690 pages.

Hayes, John P. *Introduction to Digital Logic & Design*, Prentice Hall, 1993, 815 pages.

Horowitz, Paul ; Hill, Winfield. *The Art of Electronics*, 3e édition, Cambridge University Press, 2015.

Karnaugh, Maurice. « The Map Method for Synthesis of Combinational Logic Circuits », *Trans. AIEE*, Pt I, Vol. 72, n°9, novembre 1953, p. 593-599.

Mack, Chris. «The Multiple Lives of Moore' Law», *IEEE Spectrum*, avril 2015, p. 30-37.

Maldague, X. ; Torresan, H. « A Microprogrammable Processor Teaching Tool and its FPGA Implementation », *IEEE Canadian Journal of Electrical and Computer Engineering*, Vol. 28, n°s 3 et 4, 2003, p. 139-144.

Mano, M. Morris; Ciletti, Michael D. *Digital Design: With an Introduction to the Verilog HDL*, Pearson Education, Inc. (Prentice Hall), 2012 (5e édition), 547 pages.

Marcovitz, Alan. *Introduction to Logic and Computer Design with CD*, McGraw-Hill Science, 2007, 688 pages.

Mealy, George H. « A method for synthesizing sequential circuits », *Bell System Technical Journal*, Vol. 34, no5, 1955, p. 1045-1079. Professeur George H. Mealy, Université de Harvard, Boston, Massachusetts.

NASA SP-350. *Apollo Expeditions to the Moon*, Washington, DC, Edgar M. Cortright, 1975.

Nelson, Victor P. ; Nagle, H. Troy ; Carroll, Bill D. ; Irwin, J. David *Digital Logic Circuit Analysis & Design*, Prentice Hall, 1995, 842 pages.

Roth, Jr. Charles H.; Kinney, Larry L. Fundamentals of Logic Design, Cengage Learning, 2013 (7e édition), 791 pages.

Sandige, Richard S. *Digital Design Essentials*, Prentice Hall, 2002, 670 pages.

Sjoholm, Stefan ; Lindh, Lennart. *VHDL For Designers*, Prentice Hall, 1997, 496 pages.

Uyemura, John P. *A First Course in Digital Systems Design : An Integrated Approach*, Brooks/Cole Publishing Company, 2000, 495 pages.

Wakerly, John F. *Digital Design Principles & Practices*, Prentice Hall, 2001, 946 pages.

Wakerly, John F.. Digital Design: Principles and Practices, Pearson Education, Inc., 2005 (4e édition), 865 pages.

RÉFÉRENCES

Harris, David; Harris , Sarah. Digital Design and Computer Architecture, MK (Elsevier), 2012 (2e édition), 690 pages.

Daniels, Jerry D.. Digital Design from Zero to One, John Wiley & Sons, Inc., 1996, 616 pages.

Floyd, Thomas L.. Digital Fundamentals, Pearson Education, Inc. (Prentice Hall), 2014 (11e édition), 912 pages.

Balabanian , Norman; Carlson, Bradley. Digital Logic Design Principles John Wiley & Sons, Inc., 2001, 373 pages.

Wakerly, John F.. Digital Design: Principles and Practices, Pearson Education, Inc., 2005 (4e édition), 865 pages.

Mano, M. Morris; Ciletti, Michael D.. Digital Design: With an Introduction to the Verilog HDL, Pearson Education, Inc. (Prentice Hall), 2012 (5e édition), 547 pages.

Dueck, Robert K.. Digital Design with CPLD Applications and VHDL, Delmar Thomson Learning, 2011 (2e édition), 896 pages.

Brown, Stephen; Vranesic, Zvonko. Fundamentals of Digital Logic with Verilog Design, Mc Graw Hill, 2013 (3e édition), 864 pages.

Roth, Jr. Charles H.; Kinney, Larry L. Fundamentals of Logic Design, Cengage Learning, 2013 (7e édition), 791 pages.

Marcovitz, Alan. Introduction to Logic and Computer Design with CD, McGraw-Hill Science, 2007, 688 pages.

Gaddis, Tony. Starting Out with Programming Logic and Design, Pearson Education, Inc. (Prentice Hall), 2015 (4e édition), 656 pages.

Index

0 catching, 215
1 catching, 215
1BFA, 36
2764, 160
4004, 3, 191
74HC75, 204
74138, 146
74164, 269
74191, 306, 338
74xx, 56, 128
74xx (compteurs), 338
74xx (DÉMUX), 127,142
74xx (Mux), 126, 129
8051, 368

A

ABEL, 39, 89, 127
Absorption, 66
Ada Byron Augusta Lovelace, 1
Addition de fractions, 27
Addition de nombres binaires, 20
Addition modulaire, 35
Additionneur-complet, 36
Adjacence, 66, 89
Adsorption, 66
afficheur 7 segments, 56
Aléas, 112
Algèbre booléen, 64
Analyse (circuits synchrones avec entrées externes), 291, 294
Analyse (circuits synchrones avec entrées externes, compteurs MSI et LSI), 308
Analyse (circuits synchrones sans entrées), 244, 245
Analyse (registre à décalage cyclique), 271
Analyse des circuits logiques, 38
AND, 33
Antirebond (circuit), 199
Approche universelle, 284, 366
Armstrong (Neil), 236
Arrondissement, 20
ASCII, 28
Axiômes booléens, 65

B

Babbage, Charles, 1
Bardeen, John, 2
Bascules, 191
Bases numériques, 9, 10, 11
BCD, 12
Bell Labs, 2
Binaire, 5, 8
Binaire (représentation), 9, 11
Binaire (roue), 23
Binaire à décimal, 11, 16
Bit de signe, 22, 26
Blip, 307
Bloc d'alimentation, 6
Bobinage de fil, 40
Boole, George, 51
Booléen, 8
Boomerang, 103
Brattain, Walther B., 2
Breadboard, 40
Bruit, 7
Buffer, 32

C

Cascade de compteurs, 266
CB4CLED, 266
CBxCLED, 338
Chronogramme, 115, 195
Cinq variables, 117
Circuit antirebond, 199
Circuit Blip, 307
Circuit imprimé, 40
Circuit séquentiel, 194
Circuits de mémoire, 127, 155
Circuits de mémoire (définition), 155
Circuits logiques programmables, 164
Circuits programmables, 125
Circuits TTL, 42
CISC, 368
Clignoteurs (automobile), 367
CMOS, 45
Code ASCII, 28
Code BCD, 12
Code Gray, 13
Codes, 28
Colonnes, 104
Complément 1, 22
Complément 2, 22
Compteur et mémoire, 284
Compteurs (type de), 261
Compteurs à décade, 261
Compteurs à propagation, 213
Compteurs binaire à codage positionnel, 261
Compteurs ripple, 213
Conception (circuits synchrones avec entrées externes, compteurs MSI et LSI), 337
Contamination aux 0, 215
Contamination aux 1, 215
Conversion analogique-numérique, 7
Conversion de base, 10, 13, 14, 15,17
Conversion de fractions, 19

Correction des aléas, 115
Coût (Mux), 126
Coût d'un circuit, 60
CPLD, 44, 367

D

D, 193, 201
De Morgan (Auguste), 125
De Morgan, 67
Décimal à binaire, 16
Décimal, 11
Décodeur, 142, 149
Demi-additionneur, 35
Démultiplexeurs, 149, 153 Démultiplexeurs (applications), 142
Démultiplexeurs (cascade de), 153
Démultiplexeurs (synthèse avec), 149
DÉMUX, 142
Dépassement, 25
Design, 35, 38, 72, 75,80
Design (circuits synchrones avec entrées externes), 303, 308
Design (circuits synchrones avec entrées externes, compteurs MSI et LSI), 337
Design (circuits synchrones sans entrées), 244, 245
Design (compteurs MSI / CB4CLED / 74191), 266, 306, 338
Design (registre à décalage cyclique), 271
Diagramme d'état (établissement d'un), 320
Diagrammes de Venn, 89
Distributivité booléenne, 65
Distributrice, 53
Domaine continu, 6
Domaine discret, 5
Don't care, 107
Dualité booléenne, 65
Durée minimum d'une impulsion, 218

E

Edge-triggered clock, 202
EDVAC, 2
EEPROM, 127
ENIAC, 2
EPROM, 127, 156, 157
Ère du numérique, 7
ET, 29
État, 9, 238
État d'une bascule, 201
Étiquette, 60
Expression booléenne (évaluation rapide), 401

F

Faggin (Frederico), 191
Feed-back, 193, 194
Flip-flop, 194
Flip flops (nombre minimum dans un design), 239
Flip-flop D, 205, 216
Flip-flop J-K, 221, 225
Flip-flop T, 212, 213
Flip-flop (paramètres), 218
Flip-flop (résumé), 225
Formes équivalentes (ET et OU), 67
Fournaise à mazout, 397
FPGA, 44, 178, 367
Fractions (addition de), 27
Fractions (conversion), 15, 17
Fréquence (horloge), 240
Fréquence maximale de l'horloge, 241
Full-adder, 36
Fusible, 158, 159, 167

G

Générateur pseudo-aléatoire, 277
Grandeur physique, 8
Grands nombres, 14
Groupement de cases (simplification graphique), 99

H

HA, 36
Half-adder, 35
Harvard (architecture de type), 2
HDL, 38
Hexadécimal, 9, 10, 13
Horloge, 202
Horloge (considérations importantes), 240
Horloge (vitesse de), 218
Horloge à déclenchement sur front, 193
Horloge à niveaux, 205

I

Implantation matérielle, 40
Impliquants, 75
Intel, 3
Interrupteur, 9, 31
Inverseur, 32
Inversion d'un bit avec OU-Exclusif), 26

J

J-K, 193, 221
Johnson (séquenceur de), 273

K

Karnaugh (Tables de), 87
Karnaugh, Maurice, 87
K-Map, 88

L

Label, 60
Langage de description matérielle, 38
Latch D, 201, 203
Logiciel de programmation, 180
Logique câblée, 3
Logique programmable, 3
Loi de Moore, 3
Lois de De Morgan, 67, 72
LSI, 3, 42

M

Machine d'état synchrone, 301
Machine de Mealy, 304
Machine de Moore, 237

Maître-esclave, 205
Mark I, 2
Maxterme, 75
Mealy (machine de), 304
Mémoire (définition), 155, 156
Mémoire, 157
Microblaze, 370
Microcontrôleurs, 366
Microprocesseur, 3, 191, 351
Microprogrammation, 366
Minimum recovery time, 219
Minterme, 58
Mise à 0, 196, 221
Mise à 1, 196, 221
Moore (Loi de), 3
Moore (machine de), 237
MOSFET, 302
Mot, 9
Moteur à quatre temps, 290
MSI, 3, 128, 306, 337
Multiplexeurs, 129, 135,136, 221
Multiplexeurs (applications), 133
Multiplexeurs (cascade de), 135
Multiplexeurs (théorème d'expansion de Shannon), 136, 137
Multiplexeurs (utilisation dans les compteurs), 337

N

NAND, 33
Nanotube de carbone, 201
NASA, 191, 235
Neil Armstrong, 236
Nibble, 9
Nobel (prix pour le premier transistor), 2
Nombres (grands), 14
Nombres (représentation des), 5
Nombres binaires (addition de), 20
Nombres binaires (soustraction de), 22
NON-ET, 33
NON-ET, 72
NON-OU, 33
NON-OU, 75
NON-OU-Exclusif, 34
NOR, 33
Noyce, Robert, 3
NXOR, 34

O

Octal, 10, 11, 15
Octet, 9
OR, 34
Ordinateurs (histoire des), 1
OU-Exclusif, 70, 126, 288
Overflow, 24

P

PAL, 164, 170, 171
PAL, 173, 175
Partition (des PAL), 175
Pastilles, 103
PCB, 40
Picoblaze, 349
Pioneer (sonde spatiale), 191
PIPO, 210
PISO, 210
PLA, 178
Plaquette de montage, 40
PLD, 127, 159
Porte à entrées multiples, 34
Porte ET, 33
Porte inverseuse, 32
Portes logiques, 31
Porte NAND, 33
Porte NON-ET, 33
Porte NON-OU, 33
Porte NON-OU-Exclusif, 34
Porte NOR, 33
Porte NXOR, 34
Porte OU-Exclusif, 34
Porte XOR, 34
Postulat de Huntington, 65
Produit de somme (PdS), 53, 72
Programmable Array Logic, 127, 164
Programmable Logic Array, 177
PROM, 127
PROM, 155

Q

Quine-McCluskey, 112

R

RAM, 156
RCO, 266
Registres, 208
Registres (PIPO), 210
Registres (PISO), 210
Registres (SIPO), 208
Registres (SISO), 208
Registres à décalage cyclique (analyse de compteurs), 271
Registres à décalage cyclique (design de compteurs), 245
Règles de synchronisme, 240
Représentation binaire, 10, 27
Résolution de TdV, 51
Retournement (des PAL), 174
Rétro-action, 192, 216
Ripple carry output, 266, 338
Ripple counters, 213
RISC, 367
ROM, 44, 127, 155
Roue binaire, 23
R-S, 193, 194

S

Sans importance, 117
Sans variables conditionnelles, 311
Séquence arbitraire, 284
Séquenceur de Johnson, 273
Séquenceur en anneau, 273
Séquenceurs synchrones, 235, 238
Set up time, 218
Shannon (théorème d'expansion), 136, 139

Shannon (théorème d'expansion, application graphique), 139
Shockley, William, 2
Signal d'horloge, 194
Signe (bit de), 22, 26
Simplification graphique des tables de Karnaugh, 96
SIPO, 208
SIPO, 268
SISO, 208
Slew rate, 218
Smarties, 103
Soft processors, 370
Somme de produits (SdP), 58, 72
Sonde spatiale Pioneer, 192
Soustraction (complément 2), 22
Soustraction de nombres binaires, 22
SSI, 128
SSI, 3
String, 9
Synchronisme (règles de), 301
Synthèse avec démultiplexeurs, 149
Synthèse avec MSI et LSI, 337
Synthèse avec multiplexeurs, 129
Système binaire, 1

T

T, 193, 213
Table de Karnaugh (cas des côtés), 101
Table de Karnaugh (disposition des variables), 93
Table de Karnaugh (plus de quatre entrées), 109
Table de Karnaugh (simplification graphique), 96
Table de Karnaugh (TdK), 88
Table de transition, 198
Table de vérité (TdV), 51, 52
Table ÉP-ÉS, 247
Table État Présent-État Suivant, 247
Tampon, 32
Temps de mise en place, 220
Temps minimum de rétablissement, 220
Théorèmes booléens, 8, 64, 66
Théorème d'expansion de Shannon (multiplexeurs), 136, 139
Timing diagramm, 206
Transistor, 2, 302
Troncature, 20
TTL, 40, 42,
TTL (DÉMUX), 142
TTL (Mux), 129
Tubes à vide, 2

U

Unités cachées, 281
Universelle (approche de design), 284, 366

V

Variables conditionnelles, 308
Variables conditionnelles (sans variables conditionnelles), 308
Veitch (diagrammes de), 94
Venn (diagramme de), 89
Verilog, 39, 127
VHDL, 39, 127, 370
VHSIC, 39
Vitesse de l'horloge, 218
VLSI, 4, 40, 44, 128
Von Neumann (architecture de type), 2

W

Wire-wrap, 40

X

Xilinx, 45, 56, 60, 81
XOR, 34
XOR, 104

Z

Zéros, 15
Zéros (solution en rassemblant les), 105
Zig zag, 103

Xavier Maldague, ing., Ph. D. en génie électrique de l'Université Laval. Il est professeur titulaire à l'Université Laval au Département de génie électrique et de génie informatique. Le livre *Circuits logiques* est issu de son enseignement sur une période de plus de dix ans. Aux cycles supérieurs, il donne un cours sur les applications de la vision numérique à l'inspection industrielle. Depuis 2004, il est titulaire de la chaire de recherche du *Canada MiVim* dont le thème est la vision infrarouge multipolaire. En 1996, il a lancé la technique de thermographie de phase pulsée qui est depuis devenue une technique d'analyse standard de ce domaine. Le Dr Maldague a aussi lancé une série d'ateliers connue sous le nom d'*Ateliers de Québec* sur le thème du traitement du signal pour l'évaluation non destructive des matériaux. Le Dr Maldague est membre de plusieurs comités internationaux œuvrant en thermographie infrarouge, notamment QIRT (*Quantitavive Infrared Thermography*) et Thermosense. Il est aussi impliqué auprès de l'IEEE (*Institute of Electrical and Electronics Engineers*). En 2004, il a reçu le Prix du Conseil de l'est du Canada de l'IEEE-Canada pour ses contributions à l'ingénierie.

Ce livre aborde tous les aspects de l'arithmétique binaire de base aux circuits séquentiels complexes (avec un soupçon de microprogrammation), il est destiné à la clientèle universitaire de premier cycle de génie électrique, de génie informatique, de génie logiciel ou de génie physique qui a dans son curriculum un cours de base sur les circuits logiques : des circuits combinatoires aux circuits séquentiels. Il peut aussi servir à l'enseignement collégial, sans doute en excluant la partie séquentielle et en y ajoutant des notions d'électronique non couvertes ici. D'un format compact, il sera aisément couvert en un trimestre.

Une combinaison d'exposés théoriques magistraux et d'expériences pratiques est à notre avis la combinaison pédagogique idéale. Ce livre a été pensé en conséquence. C'est ainsi que chaque partie y est très détaillée. L'ajout de problèmes et de démonstrations (par exemple de simulations) réalisés en classe complétera judicieusement le contenu de ce volume. On favorise d'ailleurs cette approche d'apprentissage par l'exemple dans ce livre où les notions sont très souvent présentées à même les exemples résolus.

Illustration de la couverture : M101, surnommée la Galaxie du moulin, se situe dans la partie nord de la constellation circumpolaire *Ursa Major* (la Grande Ourse), à une distance de 25 millions d'années-lumière de la Terre. *Gracieuseté de la* NASA *et* ESA.

9 782923 565903 >

Loze-Dion éditeur inc.